BAEUM.AI

Agent Handbook

with LangChain, LangGraph & Deep Agents

A Practical Guide to AI Agent Development

Agent Handbook with LangChain, LangGraph & Deep Agents

Copyright © 2026 BAEUM.AI. All rights reserved.

The code examples in this book may be freely used for educational purposes.
Unauthorized reproduction or redistribution of the written content is prohibited.

First edition: 2026

Preface

AI agents go beyond simple chatbots. They are intelligent systems that use tools, make plans, and autonomously carry out complex tasks. This handbook presents a structured, hands-on path to building real-world AI agents with three major frameworks — **LangChain**, **LangGraph**, and **Deep Agents**.

What This Book Offers

- **Hands-on learning** — Includes 59 Jupyter notebook-based examples with code and outputs
- **Progressive difficulty** — Moves step by step from fundamentals to production deployment
- **Framework comparison** — Compares the strengths and best-fit use cases of each framework
- **Applied projects** — Builds RAG, SQL, data analysis, ML, and deep research agents

Target Audience

- Developers who know basic Python syntax
- Engineers who want to build LLM-powered applications
- Learners who want a structured path into AI agent architecture

How This Book Is Organized

Part	Topic	Chapters
I	Agent Foundations — from environment setup to the basics of the three frameworks	8
II	LangChain — models, tools, memory, middleware, and multi-agent patterns	13
III	LangGraph — state graphs, workflows, persistence, and subgraphs	13
IV	Deep Agents — backends, subagents, memory, and harnesses	10

V	Advanced Patterns — multi-agent systems, RAG, SQL, and production	10
VI	Applied Examples — five end-to-end agent projects	5

Each chapter follows a consistent structure: Learning Objectives → theory → hands-on code → Summary.

How to Use This Book

Reading Code Blocks

This handbook uses two main block styles:

```python
# Python code block — mint left border
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
result = model.invoke("Hello, Agent!")
```

```
Output block — coral left border
Hello! I'm an AI agent ready to help.
```

Callout Types

> ➡️ TIP
>
> Provides a useful tip or extra information.

> · NOTE
>
> Explains an important note or background concept.

> ! WARNING
>
> Warns about caution points or common mistakes.

Practice Environment

All code has been tested with Python 3.11+. Package installation and API key setup are covered in the first chapter of Part I.

Table of Contents

Part 1: Agent Foundations 29
0. Environment Setup 31
Learning Objectives 31
0.1 API key settings 31
0.2 Model initialization 32
0.3 Smoke Test 32
Summary 32
1. LLM Basics 33
Learning Objectives 33
1.1 Environment Setup 33
1.2 The Three Roles of Messages 33
1.3 Controlling Behavior with a System Message 34
1.4 Dictionary Format 34
1.5 Streaming 34
1.6 Batch Calls 34
Summary 34
2. LangChain Basics 36
Learning Objectives 36
2.1 Environment Setup 36
2.2 Building Tools 36
2.3 Creating and Running an Agent 37
Summary 37
3. LangChain Conversations 38
Learning Objectives 38
3.1 Environment Setup 38
3.2 The Limit of an Agent Without Memory 38
3.3 Adding Memory with InMemorySaver 38
3.4 Watching the Steps with Streaming 39
Summary 39
4. LangGraph Basics 40
Learning Objectives 40
4.1 Environment Setup 40
4.2 Your First Graph 40
4.3 A Two-Node Graph 41
4.4 Using an LLM as a Node 41
Summary 41
5. Deep Agents Basics 43

Learning Objectives 43
5.1 Environment Setup 43
5.2 Creating an Agent 43
5.3 Adding a Custom Tool 44
Summary 44
6. Comparing the Three Frameworks & Choosing What Comes Next 46
Learning Objectives 46
6.1 Framework Comparison 46
6.2 Which One Should You Choose? 47
6.3 Code Comparison — The Same Task in Three Styles 47
Summary 48
6.4 What Comes Next 48
7. Mini Project 50
Learning Objectives 50
7.1 Environment Setup 50
7.2 Defining the Search Tool 50
7.3 A Deep Agents Research Agent 51
7.4 Observing the Process with Streaming 52
7.5 Comparing It with a LangChain Agent 52
Summary 52
Part II: LangChain 53
1. Introduction to LangChain 55
Learning Objectives 55
1.1 LangChain Framework Overview 55
1.2 The ReAct Agent Pattern 58
1.3 Overview of the Main Components 60
1.4 Environment Setup and Installation Check 60
1.5 Next Steps 61
Summary 61
2. Your First Agent 63
Learning Objectives 63
2.1 Environment Setup 63
2.2 Building a Simple Tool 64
2.3 Creating an Agent 64
2.4 Running the Agent 65
2.5 Streaming Execution 65
2.6 Multi-Turn Conversation 65
2.7 Adding the Tavily Search Tool (Optional) 66
2.8 Runtime Context (`context_schema`) 66
2.9 Summary 66
3. Models and the Message System 68
Learning Objectives 68
3.1 Environment Setup 68
3.2 Comparing Model Providers 68
3.3 Using `init_chat_model()` 69
3.4 `invoke()` , `stream()` , and `batch()` Patterns 69

3.5 Message Types ... 69
3.6 Multimodal Messages (Image Input) ... 70
3.7 Standard Output Format — `output_version="v1"` ... 70
3.8 Tracking Token Usage — `UsageMetadataCallbackHandler` ... 71
3.9 Runtime Config — Per-Invocation Options ... 71
3.10 Connection Resilience — `max_retries` / `timeout` ... 71
3.11 Prompt Caching — Implicit vs Explicit ... 72
3.12 Summary ... 72
4. Tools and Structured Output ... 74
Learning Objectives ... 74
4.1 Environment Setup ... 74
4.2 The Basics of the `\@tool` Decorator ... 75
4.3 Complex Schemas with Pydantic ... 75
4.4 Connecting Tools to an Agent ... 76
4.5 ToolRuntime ... 76
4.6 Structured Output ... 78
4.7 ToolStrategy vs ProviderStrategy ... 78
4.8 Summary ... 79
5. Memory and Streaming ... 80
Learning Objectives ... 80
5.1 Environment Setup ... 80
5.2 Short-Term Memory: `InMemorySaver` ... 80
5.3 Independent Conversations with Different `thread_id` Values ... 80
5.4 Message Trimming ... 81
5.5 Long-Term Memory: `InMemoryStore` ... 81
5.6 Streaming Modes ... 82
5.7 Summary ... 84
6. Middleware and Guardrails ... 86
Learning Objectives ... 86
6.1 Environment Setup ... 86
6.2 Middleware Concepts ... 86
6.3 Built-In Middleware ... 88
6.4 Custom Middleware: `\@before_model` ... 89
6.5 Custom Middleware: `\@after_model` ... 90
6.6 `\@wrap_model_call` ... 90
6.7 `\@dynamic_prompt` ... 91
6.8 `\@wrap_tool_call` ... 91
6.9 Simple Guardrails ... 92
6.10 Summary ... 92
7. Human ... 94
Learning Objectives ... 94
7.1 Environment Setup ... 94
7.2 Human-in-the-Loop Concepts ... 94
7.3 `HumanInTheLoopMiddleware` ... 95
7.4 The `interrupt` and `Command(resume=...)` Pattern ... 95
7.5 `ToolRuntime` — Access Runtime Information from a Tool ... 97

7.6 Context Engineering – Dynamic Control of Prompts and Tools 98
7.7 MCP (Model Context Protocol) Integration Overview 98
7.8 Summary 98
8. Multi 100
Learning Objectives 100
8.1 Environment Setup 100
8.2 Comparing Multi-Agent Patterns 100
8.3 The Subagent Pattern 102
8.4 The Handoff Pattern 103
8.5 The Skill Pattern 105
8.6 The Router Pattern 105
8.7 Choosing a Pattern 106
8.8 Summary 106
9. Custom Workflows and RAG 108
Learning Objectives 108
9.1 Environment Setup 108
9.2 `StateGraph` Basics 108
9.3 Conditional Edges 109
9.4 Integrating an Agent into a Workflow 109
9.5 Overview of RAG (Retrieval-Augmented Generation) 109
9.6 A Simple RAG Implementation 110
9.7 FAISS Vector Store (Optional) 111
9.8 Summary 111
10. Production 112
Learning Objectives 112
10.1 Environment Setup 112
10.2 LangSmith Studio 112
10.3 Agent Testing 113
10.5 Agent Chat UI 115
10.6 Deployment 116
10.7 Observability 117
10.8 Production Checklist 118
10.9 Summary 118
11. MCP (Model Context Protocol) 120
Learning Objectives 120
11.1 Environment Setup 120
11.2 MCP Concepts 120
11.3 Installing MCP support 121
11.4 Stdio Transport 121
11.5 Streamable HTTP Transport 122
11.6 Loading MCP Tools and Integrating Them with an Agent 122
11.7 Connecting to Multiple MCP Servers 123
11.8 MCP Authentication 123
11.9 Elicitation – Server-requested user input 124
11.10 Tool Interceptors 125
11.11 Writing a Custom MCP Server 125

11.12 Summary 125
12. Frontend Streaming 127
Learning Objectives 127
12.1 Environment Setup 127
12.2 Python SDK Streaming Basics 128
12.3 `astream_events()` 128
12.4 Event Streaming v3 — projection-based agent streams 129
12.4 The `useStream` React Hook 130
12.5 `useStream` Configuration Options 130
12.6 Thread Management and Reconnection 131
12.7 Branching and Message Editing 132
12.8 Custom Streaming Events 132
12.9 Multi-Agent Streaming 133
12.11 Summary 133
13. Guardrails 135
Learning Objectives 135
13.1 Environment Setup 135
13.2 Guardrail Concepts 135
13.3 PII Detection Middleware 136
13.4 Human-in-the-Loop Guardrails 137
13.5 Custom Input Guardrails — `before_agent` 138
13.6 Custom Output Guardrails — `after_agent` 138
13.7 Decorator-Based Guardrails 139
13.8 Combining Multiple Guardrails 140
13.9 Production Guardrail Patterns 141
13.10 Summary 142
14. Semantic Search 143
14.1 Environment setup 143
14.2 Build a small document collection 143
14.3 Split into chunks 144
14.4 Embeddings and vector store 144
14.5 Run retrieval 144
14.6 Mini retrieval evaluation 144
14.7 Checklist before RAG 145
Summary 145
Reference docs 145
Part III: LangGraph 145
1. Introduction to LangGraph 148
Learning Objectives 148
1.1 What is LangGraph? 148
1.2 Core concepts 148
1.3 Two APIs 152
1.4 Environment Setup and check installation 152
1.5 A taste of the Graph API 153
1.6 A taste of the Functional API 153
1.7 Next Steps 153

Summary 153
2. Graph API Basics 155
Learning Objectives 155
2.1 Environment Setup 155
2.2 StateGraph basic structure 155
2.3 state Reducer 155
2.4 Conditional Edge 156
2.5 Message-based state 156
2.6 Input/Output Schema 156
2.7 add_node() advanced options 156
2.8 Runtime context with context_schema 157
2.9 Send and Command.PARENT 158
2.10 Summary 158
3. Functional API 159
Learning Objectives 159
3.1 Environment Setup 159
3.2 @task — Asynchronous unit of work 159
3.3 Parallel `\@task` execution 159
3.4 previous — short-term memory (accessing previous execution results) 160
3.5 entrypoint.final — Separate return and checkpoint saved values 160
3.6 Determinism Requirements 160
3.7 LLM agent (Functional API) 160
3.8 @task retry and caching 160
3.9 Structured interrupt and Command(resume=) 160
3.10 @entrypoint injected parameters 161
3.11 Summary 161
4. workflow Pattern 163
Learning Objectives 163
4.1 Environment Setup 163
4.2 Prompt Chaining — Sequential LLM calls 163
4.3 Parallelization — Simultaneous execution of independent `\@task` 164
4.4 Routing — Classification-based branching 164
4.5 Orchestrator-Worker — Create dynamic worker with Send() 165
4.6 Evaluator-Optimizer — Generate-evaluation iteration loop 165
4.7 Pattern comparison table 165
5. Building agent 167
Learning Objectives 167
5.1 Environment Setup 167
5.2 tool Definitions — @tool decorator and bind_tools() 167
5.3 Graph API agent — Implementing ReAct Loop with StateGraph 168
5.4 Visualizing execution flow — Observe each step with streaming 168
5.5 Functional API agent — @entrypoint + while loop 168
5.6 agent with memory — Keep conversation with checkpointer 168
5.7 Summary 169
6. Persistence and memory 170
Learning Objectives 170

6.1 Environment Setup 170
6.2 checkpointer — Automatically saves state for each execution step 170
6.3 get_state() — Retrieve currently stored state lookup 171
6.4 get_state_history() — View entire execution history 171
6.5 update_state() — Modify stored state externally 172
6.6 Thread Independence — Different thread_ids are completely independent state 172
6.7 InMemoryStore — Cross-thread sharing long-term memory 172
6.7.6 Managing conversation length — trim_messages and RemoveMessage 174
6.8 Durable Execution — resume at last checkpoint on failure 175
6.9 Summary 175
7. streaming 176
Learning Objectives 176
7.1 Environment Setup 176
7.2 streaming Mode Comparison 177
7.3 stream_mode="values" — Full state snapshot 177
7.4 stream_mode="updates" — Node-specific updates 178
7.5 stream_mode="messages" — Per-token streaming 178
7.6 Simultaneous use of multiple streaming modes 178
7.7 stream_mode="custom" — custom streaming 179
7.8 Event Streaming v3 — projection-based app streams 180
7.9 Summary 180
8. Interrupts and Time Travel 182
Learning Objectives 182
8.1 Environment Setup 182
8.2 interrupt() — executes interrupt and waits for human input 182
8.3 Command(resume=…) — interrupt executes resume 182
8.4 Interrupt in Functional API 184
8.5 Time Travel — Go back to a previous checkpoint 185
8.6 update_state() — Time travel + state fix 185
8.7 Summary 186
9. Subgraph 187
Learning Objectives 187
9.1 Environment Setup 187
9.2 Subgraph concept 187
9.3 Creating a subgraph 187
9.4 Adding a subgraph to the parent graph 187
9.4.1 Pattern 1: Subgraph call through wrapper node (if `state_schema` is different) 187
9.5 LLM-based subgraph 188
9.6 Subgraph streaming 188
9.7 Summary 189
10. Production 191
Learning Objectives 191
10.1 Environment Setup 191
10.2 App structure — langgraph.json 191
10.3 LangGraph Studio — Visual Debugging tool 191
10.4 Agent Chat UI 192

10.5 Test — Deterministic agent test 193
10.6 LLM agent Test — Using GenericFakeChatModel 193
10.7 Deployment Options 194
10.8 Observability — LangSmith Tracing 195
10.9 Pregel Runtime Overview 196
10.10 Production Checklist 197
10.11 Summary 197
11. Local Server 198
Learning Objectives 198
11.1 Environment Setup 198
11.2 LangGraph CLI installation 198
11.3 Project creation 198
11.4 langgraph.json settings 199
11.5 Running the development server 199
11.6 LangGraph Studio integration 200
11.7 Python SDK — Asynchronous client 200
11.8 Python SDK — Synchronous client 201
11.9 REST API call 201
11.10 Deployment Readiness Checklist 201
11.11 Summary 202
12. durable execution 203
Learning Objectives 203
12.1 Environment Setup 203
12.2 durable execution Concept 203
12.3 Core requirements 204
12.4 Endurance Mode Comparison 204
12.5 Problematic code 204
12.6 Improvements to @task 205
12.7 Durability in Graph API 205
12.8 Durability in Functional API 206
12.9 Failover Scenario 206
12.10 resume Starting point 207
12.11 Production Durability Pattern 207
12.12 Per-node fault tolerance — `TimeoutPolicy` + `heartbeat` 208
12.13 Graceful shutdown — `RunControl` / `GraphDrained` 208
12.14 DeltaChannel (beta) — checkpoint size optimization 209
12.15 Summary 210
Learning Objectives 210
13.1 Environment Setup 211
13.2 Graph API vs Functional API Overview 211
13.3 Quick Selection Guide 211
13.4 Graph API implementation 212
13.5 Functional API implementation 212
13.6 Comparative analysis 213
13.7 Combining two APIs 214
13.8 Pregel Runtime Overview 214

13.9 Direct use of Pregel 215
13.10 Channel Type 215
13.11 superstep Execution Model 217
13.12 API Selection Criteria Guide 218
13.13 Summary 218
14. Fault Tolerance 220
Learning goals 220
Overview 220
1) Retries — let the system absorb transient failures 221
2) Timeouts — stop async nodes that hang 221
3) Error handlers — route failures to a fallback path 222
4) Graph defaults — shared policy and node overrides 222
Operations checklist 222
15. Backward Compatibility 224
15.1 Compatibility is more than successful imports 224
15.2 Migration checklist 225
15.3 Compatibility matrix 225
Summary 225
Reference docs 225
16. Case Studies 226
16.1 Case-study review template 226
16.2 Break down a sample case 226
16.3 Design decision record 227
Summary 227
Reference docs 227
Part IV: Deep Agents 227
1. Introduction to Deep Agents 230
Learning Objectives 230
1. What Is Deep Agents? 230
2. SDK vs Code vs ACP 231
3. Five Core Concepts (Planning · Context Management · Backends · Subagents · Memory & Skills) 232
4. Comparison with Other Frameworks 233
5. Installation Check 234
Summary 235
Next Steps 235
2. Building Your First Agent 236
Learning Objectives 236
1. Project Bootstrap and API Keys 236
2. Creating the Simplest Agent 237
3. Running the Agent — `invoke()` 238
4. Connecting Tavily Search — A Research Agent 238
5. Checking the Built-In Tools 239
6. Streaming Output — `stream()` 239
Summary 239
Next Steps 240

3. Agent Customization 241
Learning Objectives 241
0. Full `create_deep_agent()` Signature (17 parameters) 241
1. Choosing a Model 242
2. Custom System Prompts 243
3. Building Custom Tools 243
4. Structured Output — `response_format` 244
5. Middleware Architecture 245
6. Sandbox Options 247
Summary 247
Next Steps 247
4. Storage Backends 248
Learning Objectives 248
1. What Is a Backend? 248
2. `StateBackend` (Default) 249
3. `FilesystemBackend` — Accessing the Local Disk 250
4. `StoreBackend` — Cross-Thread Persistent Storage 250
5. `CompositeBackend` — Path-Based Routing 251
6. `LocalShellBackend` — Shell Execution 252
7. Implementing a Custom Backend 253
8. `ContextHubBackend` — Remote File Sync 255
9. Permissions — `Permissions` + `GuardedBackend` 255
Backend Selection Guide 256
Summary 256
Next Steps 257
5. Subagents and Task Delegation 258
Learning Objectives 258
1. Why Subagents Are Useful 258
2. Defining a `SubAgent` (Dictionary Form) 260
3. `CompiledSubAgent` — Plugging in a Custom LangGraph Runnable 262
4. General-Purpose Subagents 262
5. Context Propagation 263
6. Multi-Subagent Pipelines 264
7. Best Practices 266
Summary 267
Next Steps 267
6. Long 268
Learning Objectives 268
1. Why Long-Term Memory Matters 268
2. Cross-Thread Memory Sharing 270
3. Injecting Context with `AGENTS.md` 270
4. Skills 270
5. Skill Source Priority 272
6. Skill Inheritance for Subagents 273
6.10 Long-term Memory Types (in 0.5) 274
Summary 276

Next Steps 277
7. Advanced Features 278
Learning Objectives 278
1. Human-in-the-Loop (HITL) 279
2. Advanced Streaming 280
3. Sandboxes 282
4. Interpreters — CodeInterpreterMiddleware 284
5. RubricMiddleware — runtime LLM-as-a-judge 285
6. ACP (Agent Client Protocol) 285
7. Deep Agents Code (`dcode`) 286
Full Track Summary 288
8. Agent Harness 289
Learning Objectives 289
1. AgentHarness Concept 289
2. Planning Tools 290
3. Virtual Filesystem 291
4. Task Delegation — Subagents 291
5. Context Management 292
6. Code Execution 293
7. Human-in-the-Loop 294
8. Skills and Memory 294
9. Harness Profiles 295
Summary 297
9. Comparing External Frameworks 298
Learning Objectives 298
1. Comparison Overview 298
2. Deep Agents vs OpenCode vs Claude Agent SDK 299
3. Comparing Core Capabilities 300
3-1. Three Core Differentiators 300
4. Architecture Comparison 301
5. Recommendations by Use Case 301
6. Ecosystem Comparison 302
7. Migration Considerations 303
Summary 303
10. Sandboxes and ACP 305
Learning Objectives 305
1. Sandbox Concepts 305
2. Architecture Patterns 306
3. Comparing Sandbox Providers 307
4. Security Guidelines 307
5. File Transfer and Lifecycle Management 308
6. ACP Overview 309
7. ACP Server Implementation 309
8. Editors That Support ACP 310
9. Combining Sandboxes and ACP 311
Summary 311

11. Async Subagents 313
11.1 Limitations of the old synchronous subagent 313
11.2 Five tools injected into the supervisor 314
11.3 Basic usage 314
11.4 The `async_tasks` state channel 315
11.5 Transport modes 315
11.6 Execution lifecycle 316
11.7 Mid-flight steering patterns 316
11.8 Local development: `--n-jobs-per-worker` 317
11.9 Observing lifecycle through the unified v2 stream 317
11.10 Sync vs Async selection criteria 318
11.11 Caveats 318
Key Takeaways 319
12. Going to Production 320
12.1 Three core abstractions 320
12.2 LangSmith Deployments 320
12.3 Multi-tenant access control 321
12.4 End-user credential management 321
12.5 Memory persistence scoping 322
12.6 Execution isolation — Sandbox 323
12.7 Durability · Async I/O 324
12.8 Rate limits and cost control 324
12.9 Three-tier error handling 324
12.10 Data privacy and real-time frontend 325
12.11 Production checklist 326
Key Takeaways 326
13. Context Engineering 327
13.1 The four-layer architecture 327
13.2 Layered input context — nine-step system prompt synthesis 328
13.3 `@dynamic_prompt` — request-time data injection 328
13.4 Progressive skill loading 328
13.5 Runtime context propagation 329
13.6 Automatic offloading (>20k tokens) 329
13.7 Automatic summarization (85% trigger) 330
13.8 Subagent isolation 330
13.9 `/memories/` routing with CompositeBackend 331
13.10 Filesystem-centric architecture 331
13.11 Three patterns 332
13.12 Caveats 332
Key Takeaways 332
14. Streaming 333
14.1 Three key ideas 333
14.2 Basic usage: enabling subagent streaming 333
14.3 Routing events via namespaces 334
14.4 Stream mode: `updates` 334
14.5 Stream mode: `messages` 335

14.6 Custom progress events 335
14.7 Subscribing to multiple modes at once 335
14.8 Tracking the subagent lifecycle 336
14.9 v2 unified format 336
14.10 Frontend integration 336
14.11 Caveats 337
Key Takeaways 337
15. Permissions 338
15.1 Scope of application 338
15.2 Evaluation rule: first-match-wins 339
15.3 Three rule components 339
15.4 Pattern 1: read-only agent 339
15.5 Pattern 2: workspace isolation 339
15.6 Pattern 3: protect specific files 340
15.7 Pattern 4: read-only memory / policies 340
15.8 Subagent inheritance 341
15.9 CompositeBackend constraint: sandbox-default 342
15.10 Prompt-injection defense perspective 342
15.11 Permissions vs backend policy hooks 342
15.12 Caveats 343
Key Takeaways 343
16. Models and Tools 344
12.1 Tools are contracts, not just functions 344
12.2 Inspect the tool schema 344
12.3 Classify tool risk 345
12.4 Centralize model configuration 345
12.5 Separate agent construction from invocation 345
12.6 Design checklist 346
Summary 346
Reference docs 346
17. Programmatic Subagents 347
13.1 When programmatic subagents are useful 347
13.2 Practice fan-out/fan-in with a deterministic fallback 347
13.3 Fan-in synthesis 348
13.4 Before promoting to live features 348
Summary 348
Reference docs 348
18. Event Streaming 350
14.1 Event model 350
14.2 Build an event consumer 350
14.3 Accumulate UI state 351
14.4 Operational checklist 351
Summary 351
Reference docs 351
19. Permissions 352
15.1 Permission matrix 352

15.2 Policy Evaluator 352
15.3 Approval messages 353
Summary 353
Reference docs 353
20. Quality Profiles and Rubrics 354
16.1 Rubrics define completion criteria 354
16.2 Deterministic rubric evaluator 354
16.3 Profile versus rubric 355
Summary 355
Reference docs 355
Part V: Advanced Patterns 355
0. v0 → v1 migration guide 357
Learning Objectives 357
0.1 Change package structure 357
v0 — Legacy approach 358
v1 — langchain-classicmigration path 358
0.2 Agent creation API changes 358
0.3 State Schema — TypedDict only 359
0.4 Runtime context injection (new) 359
0.5 Dynamic Prompts — Middleware Approach (New) 359
0.6 tool Error handling — `\@wrap_tool_call` (new) 359
0.7 Standard Content Block & structured output (New) 360
0.8 Streaming changes 360
0.9 Guitar Breaking Change 361
Summary — Migration Checklist 361
1. Deepening the middleware system 362
Learning Objectives 362
1.1 Environment Setup 362
1.2 Middleware Architecture Overview 362
1.3 SummarizationMiddleware 363
1.4 HumanInTheLoopMiddleware 364
1.5 ModelCallLimitMiddleware & ToolCallLimitMiddleware 365
1.6 ModelFallbackMiddleware 366
1.7 PIIMiddleware 366
1.8 LLMToolSelectorMiddleware 367
1.9 Writing custom middleware 368
1.10 Middleware execution order 371
1.11 Middleware Combination (Stacking) 372
1.12 Provider-specific Middleware 372
Summary 376
2. Multiagents: Subagents 377
Learning Objectives 377
2.1 Environment Setup 377
2.2 Subagents Architecture Overview 377
2.3 Low-level tool definitions 378
2.4 Create subagent 379

2.5 Wrapping subagent with tool 380
2.6 Supervisor Agent Assembly 380
2.7 Run test 380
2.8 HITL (Human-in-the-Loop) Integration 381
2.9 Passing context (ToolRuntime) 381
2.10 Asynchronous execution pattern 382
2.11 Single Dispatch tool Pattern 383
Summary 385
3. multi 386
Learning Objectives 386
Part A — Handoffs: Customer Support State Machine 386
3.1 Environment Setup 386
3.2 Handoffs Overview 386
3.3 SupportState definition 388
3.4 Step-by-step tool definition 389
3.5 @wrap_model_call middleware 390
3.6 Agent creation and execution flow 391
Part B — Router: Parallel routing and result synthesis 392
3.7 Router Overview 392
3.8 RouterState and classification schema 394
3.9 Classification Node 395
3.10 Parallel Routing (Send API) 395
3.11 Result synthesis 397
Summary 397
4. Context Engineering & Memory Deepening 399
Learning Objectives 399
4.1 Environment Setup 399
4.2 Context Engineering Overview 399
4.3 Static runtime context – `context_schema` + `\@dataclass` 400
4.4 Dynamic runtime context – `state_schema`, `AgentState` custom 401
4.5 Long-term memory – InMemoryStore native API 402
4.6 Long-Term Memory – Semantic Search 403
4.7 Reading/Writing Store from tool – `ToolRuntime.store` 404
4.8 3 types of memory: Semantic, Episodic, Procedural 405
4.9 Progressive Disclosure – Skills pattern 406
4.10 Hot Path vs Background Memory Write 407
Summary 409
5. Agentic RAG 410
Learning Objectives 410
5.1 Environment Setup 410
5.2 RAG Overview 410
5.3 Document loading & chunking 412
5.4 Building a vector store 413
5.5 Search tool Definition 414
5.6 LangChain RAG Agent – `create_agent` + `\@tool` 414
5.7 LangChain RAG Chain – `\@dynamic_prompt` Middleware 415

5.8 LangGraph Custom RAG – Building StateGraph 415
5.9 `generate_query_or_respond` node 416
5.10 `grade_documents` Node – Evaluate relevance with structured output 417
5.11 `rewrite_question` node 417
5.12 `generate_answer` node 417
5.13 Graph assembly & execution 418
Summary 418
6. SQL Agent Advanced 420
Learning Objectives 420
6.1 Environment Setup (SQLite + Chinook DB) 420
6.2 SQL Agent Overview 420
6.3 SQLDatabaseToolkit 421
6.4 LangChain SQL Agent – `create_agent` + ReAct 422
6.5 Run test 423
6.6 HITL – `HumanInTheLoopMiddleware` 423
6.7 LangGraph Custom SQL Agent – StateGraph 424
6.8 Dedicated nodes – `list_tables` , `get_schema` , `generate_query` , `check_query` 425
6.9 `bind_tools` with `tool_choice` – Force tool calling 425
6.10 Reviewing queries with `interrupt()` 425
6.11 `Command(resume=...)` pattern 426
Summary 426
7. Data Analysis Agent 428
Learning Objectives 428
7.1 Environment Setup 428
7.2 Data Analysis Agent Overview 428
7.3 Backend selection 429
7.4 Upload sample data 430
7.5 Custom tool – Slack integration 431
7.6 Creating an agent 433
7.7 Execution — Analysis Request 433
7.8 Observe the analysis process through streaming 433
7.9 Utilizing built-in tool 434
7.10 Maintain conversation with checkpointer 435
Summary 436
8. Voice Agent 438
Learning Objectives 438
8.1 Environment Setup 439
8.2 Voice Agent Architecture Overview 439
8.3 Architecture comparison table 440
8.4 STT Step – AssemblyAI Real-Time Transcription 441
8.5 Agent Phase – Utilizing LangChain Agent 442
8.6 TTS Steps – Cartesia Streaming Speech Synthesis 444
8.7 Pipeline Combination – RunnableGenerator 444
8.8 WebSocket Server – Real-time two-way communication 445
8.9 Performance Target – Sub-700ms Latency 446
8.10 Add tool to agent 447

Summary 448
9. Production deployment 449
Learning Objectives 449
9.1 Environment Setup 449
9.2 Production Pipeline Overview 450
9.3 Agent testing – LangSmith evaluation 451
9.4 Unit test patterns 452
9.5 observability – LangSmith Tracing 453
9.6 Trace analysis 455
9.7 LangGraph Studio – Visual Debugging 456
9.8 Deployment Options 457
9.9 langgraph.json settings 458
9.10 Deployment command 459
Summary 461
10. Deep Agent from Scratch 462
10.1 Minimal Deep Agent components 462
10.2 Todo planner skeleton 462
10.3 Tool registry skeleton 463
10.4 Quality gate 463
Summary 463
Reference docs 463
11. Custom Workflow Agent 464
11.1 Deterministic router 464
11.2 Compile the workflow 464
Summary 465
Reference docs 465
Part VI: LangSmith 465
1. LangSmith Quickstart 467
Learning Objectives 467
What You'll Gain 467
6.01.1 API Key & Environment Variables 467
6.01.2 First Trace — LangChain Agent 470
6.01.3 run_name · tags · metadata 472
6.01.4 Tracing Pure Python Functions — `\@traceable` 473
6.01.5 Retrieve Traces from Code 474
6.01.6 Cost & Token Aggregation 475
6.01.7 API Key Management Page 475
Checklist 475
Next 476
References 476
2. Tracing Agents 477
Learning Objectives 477
Prerequisites 477
6.02.1 Run · Trace · Project Concepts 478
6.02.2 LangGraph StateGraph Trace Tree 479
6.02.3 Deep Agents Subagent Traces (Synchronous · Asynchronous) 480

6.02.4 Session View — `thread_id` · `session_id` · `conversation_id` 482
6.02.5 Attaching Feedback to Runs — `client.create_feedback` 483
6.02.6 Tag/Metadata-Based Filter Queries 484
6.02.7 `\@traceable` run_type Classification + Low-Level `create_run` / `update_run` 485
6.02.8 `LangChainTracer` Callback + `with_config({"tags": [...]})` 486
6.02.9 Selective tracing — `LANGSMITH_TRACING=false` + `tracing_context(enabled=True)` 486
6.02.10 400-Day Retention Limit → Permanent Storage as Dataset 487
Checklist 487
Next 488
References 488
3. Datasets & Evaluation 489
Learning Objectives 489
Prerequisites 489
6.03.1 Creating a Dataset — Adding Examples Manually 490
6.03.2 Transferring Production Traces to a Dataset 491
6.03.3 Evaluation Target + Code Evaluator 492
6.03.4 LLM-as-judge Evaluator 493
6.03.5 `evaluate` Runner + Experiment Name 493
6.03.6 Pairwise + Summary Evaluator 494
6.03.7 Online Evaluator — Automatic Evaluation of Production Traces 495
6.03.8 Agent Trajectory Evaluation — `agentevals` 496
Checklist 497
Next 497
References 498
4. Prompt Hub 499
Learning Objectives 499
Prerequisites 499
6.04.1 Creating and Pushing a Prompt 500
6.04.2 Commit SHA Pinning vs Tags (`prod` , `staging`) 500
6.04.3 f-string vs mustache 502
6.04.4 Playground — From Experiment to Commit 502
6.04.5 Runtime Injection — `pull_prompt` -\> `create_agent` 503
6.04.6 Pinning to a Specific Commit Hash in CI 504
Checklist 505
Next 505
References 505
5. Production Monitoring 507
Learning Objectives 507
Prerequisites 507
6.05.1 Project Dashboard — latency · cost · success rate 508
6.05.2 Online evaluator — autoeval rule 509
6.05.3 User feedback collection API 510
6.05.4 Metadata-based alert rules — failure rate \> N% -\> webhook 511
6.05.5 High-volume sampling — `LANGSMITH_TRACING_SAMPLING_RATE` 512
6.05.6 PII Scrubbing — `PIIMiddleware` + `anonymizer` 513
6.05.7 Slack / PagerDuty webhook integration 514

6.05.8 Manual Run Tree control + LangSmith Deployments monitoring 514
Checklist 515
Next 515
References 515
6.05.X Insights Agent (Paid) 516
6. Agent Evals 517
Learning goals 517
Overview 517
1) Build trajectories to evaluate 518
2) Trajectory match evaluator 518
3) LLM-as-judge trajectory evaluation 518
4) Connect to LangSmith `evaluate()` 519
Operations checklist 519
7. Testing Strategy 521
7.1 Testing pyramid 521
7.2 Unit test pure functions first 521
7.3 Integration gate 522
7.4 Eval checks behavior paths, not only answer strings 522
7.5 Separate CI and manual verification 522
Summary 522
Reference docs 522
8. LangGraph Testing 524
8.1 Node unit test 524
8.2 Graph integration test 524
8.3 Checkpoint smoke test 525
Summary 525
Reference docs 525
9. Runtime Rubric Evaluation 526
9.1 Rubric criteria 526
9.2 Evaluator 526
9.3 Relationship to offline evals 527
Summary 527
Reference docs 527
Part VII: Applied Examples 527
1. RAG Agent 529
Learning Objectives 529
Overview 529
The 5 RAG Building Blocks 530
2-Step / Agentic / Hybrid Architectures 530
The Rewrite → Retrieve → Agent 3-Node Pattern 531
Step 1: Create Sample Documents 531
Step 2: Split the Text 531
Step 3: Build the Vector Store 532
Step 4: Define a Retrieval Tool (`content_and_artifact`) 532
Step 5: Test the Retrieval Tool by Itself 532
Step 6: Create the RAG Agent (with v1 Middleware) 532

Step 7: Run a Simple Query and a Comparison Query 533
Summary 533
2. SQL Agent 534
Learning Objectives 534
Overview 534
Step 1: Connect to the Database 535
Step 2: Create the `SQLDatabaseToolkit` Tools 535
Step 3: Load the Prompt (LangSmith / Langfuse / Default) 535
Step 4: The Idea Behind Skills 536
Step 5: Create a Basic SQL Agent 536
Step 6: A HITL Agent (`interrupt_on`) 536
Step 7: Resume After Approval — Four Decision Types 537
Summary 537
3. Data Analysis Agent 539
Learning Objectives 539
Overview 539
Step 1: Compare Backends 539
Step 2: Create a CSV File for Analysis 540
Step 3: Define the Analysis Tools 540
Step 4: Create the Agent (with v1 Middleware) 541
Step 5: Analyze with pandas Code Execution 542
Step 6: Ask a Follow-Up Question in the Same Thread 542
Built-In Tools Summary 542
Data Analysis Execution Flow 543
`@tool` + pandas vs. CodeInterpreterMiddleware 543
Summary 543
4. Machine Learning Agent 545
Learning Objectives 545
Overview 545
NB03 vs. NB04: Extending the Backend and Tooling 546
Step 1: Set the Data Directory 546
Step 2: Create the `FilesystemBackend` 547
Step 3: Define `run_ml_code` 547
Step 4: Create the Agent 548
Step 5: File Exploration + EDA 548
Step 6: Train and Compare Models 548
Step 7: Multi-Turn Follow-Up — Feature Importance 548
Step 8: Streaming — Additional Analysis 549
`@tool` + scikit-learn Pattern 549
Summary 549
5. Deep Research Agent 551
Learning Objectives 551
Overview 551
Step 1: `think_tool` — A Strategic Reflection Tool 552
Step 2: A Simplified `web_search` Tool 552
Step 3: Prompt for the Five-Step Research Workflow 552

Step 4: Define Three Subagents 553
Step 5: Create the Deep Research Agent (with v1 Middleware) 553
DeepAgents Pattern — TodoList + dispatch + 5-Tool Async 554
Step 6: Run the Research Workflow 555
Step 7: Streaming — Track Namespaces 555
Subagent Design Best Practices 555
Summary 555
6. Multimodal PDF RAG 557
Learning Objectives 557
Overview 557
LangChain v1 Multimodal Content Blocks 558
1) Download the Public PDF 558
2) Extract PDF File Metadata 559
3) Create Slide-Level Documents 559
4) Inspect Slide Metadata, Tables, and Images 560
5) Generate LLM-Based Image Descriptions 560
6) Build a Vector Index and Retrieval Tool 561
7) Answer Questions with LangChain v1 `create_agent()` 561
Summary 561
7. Content Builder Agent 563
Learning goals 563
Overview 563
1) Content Builder workspace 564
2) Local research tool and researcher subagent 564
3) Configure the Deep Agent 565
4) Run — create a LinkedIn post 565
5) Search and images are optional adapters 566
Summary 566
10. Personal Assistant Subagents 567
10.1 Define roles 567
10.2 Route requests 567
10.3 Decompose compound requests 568
10.4 Merge fan-in results 568
Summary 568
Reference docs 568
11. Customer Support Handoffs 569
11.1 Handoff criteria 569
11.2 Represent the flow as a graph 570
11.3 Approval gate 570
Summary 570
Reference docs 570
12. Router Knowledge Base 572
12.1 Define knowledge sources 572
12.2 Deterministic router 572
12.3 Source-local search 573
12.4 Router evaluation 573

Summary 573
Reference docs 573
13. Skills SQL Assistant 574
13.1 SQL skill policy 574
13.2 Query safety check 574
13.3 Read-only execution 575
13.4 Answer synthesis 575
Summary 575
Reference docs 576
Part VIII: Integrations 576
1. Integration Category Overview 578
1.1 Baseline version snapshot 578
1.2 The thirteen integration categories 579
1.3 Provider Middleware roadmap 580
1.4 New integration surfaces: UI · Code · resilience 580
1.5 Common pattern 581
1.6 Observability env-var standardization 582
Key Takeaways 582
2. Anthropic Prompt Caching 583
2.1 When to use it 583
2.2 Environment setup 583
2.3 Basic usage 584
2.4 Verifying cache hits 584
2.5 Full parameters 584
2.6 Behavior on non-Anthropic models 585
2.7 Aggregating with `UsageMetadataCallbackHandler` 585
2.8 Differences from Bedrock 586
Key Takeaways 586
3. Claude Bash Tool 587
3.1 When to use it 587
3.2 Environment setup 587
3.3 Host execution policy (simplest) 588
3.4 Docker execution policy (recommended, isolated) 588
3.5 Codex sandbox policy 589
3.6 Output redaction (`redaction_rules`) 589
3.7 Falling back to a generic `@tool` 589
3.8 Native vs generic comparison 590
Key Takeaways 590
4. Claude Text Editor 591
4.1 When to use it 591
4.2 Environment setup 591
4.3 State variant — virtual files in graph state 592
4.4 Filesystem variant — real disk 592
4.5 State vs Filesystem selection criteria 593
4.6 Relationship with generic file tools 593
Key Takeaways 593

5. Claude Memory 594
5.1 When to use it 594
5.2 Environment setup 594
5.3 State variant — persisted within the thread 595
5.4 Filesystem variant — across process boundaries 595
5.5 Scaling to a durable checkpointer 596
5.6 Multi-tenant isolation via `runtime.context.user_id` 596
5.7 Relationship with Deep Agents' `StoreBackend` 597
Key Takeaways 597
6. Anthropic File Search 598
6.1 When to use it 598
6.2 Environment setup 598
6.3 Text editor + file search combination 599
6.4 Step 1 — seed state with multiple files 599
6.5 Step 2 — the same agent searches with glob/grep 599
6.6 Targeting memory files too 600
6.7 Position in the five-step retrieval building blocks 600
6.8 When to pick this over a vector store 601
Key Takeaways 601
7. Bedrock Prompt Caching 602
7.1 When to use it 602
7.2 Environment setup 602
7.3 ChatBedrockConverse + Claude (recommended) 603
7.4 Verifying cache hits 603
7.5 Full parameters 603
7.6 ChatBedrock (Invoke model) example 604
7.7 Reasoning · profiling 604
7.8 Amazon Nova constraints 605
Key Takeaways 605
8. OpenAI Moderation 606
8.1 When to use it 606
8.2 Environment setup 606
8.3 Basic usage — scan both input and output, end on violation 607
8.4 `exit_behavior="end"` — end immediately on violating input 607
8.5 `exit_behavior="error"` — fail loudly 608
8.6 `exit_behavior="replace"` — swap the message and keep going 608
8.7 Scanning tool results (`check_tool_results=True`) 608
8.8 Observing violation flows with LangSmith 609
8.9 Cost / latency tradeoff 609
Key Takeaways 610
Appendix A. Glossary 611
Frameworks and Products 611
Agents and Execution Model 612
State and Memory 613
Tools and Interfaces 614
Backends and Execution Environments 614

Retrieval, Streaming, and Quality 615

PART

I

Agent Foundations

Foundations of AI Agents

What You Will Learn in This Part

- 0. Environment Setup
- 1. LLM Basics
- 2. LangChain Basics
- 3. LangChain Memory
- 4. LangGraph Basics
- 5. Deep Agents Basics
- 6. Framework Comparison
- 7. Mini Project

00 Environment Setup

Before You Start

This series is in the order LangChain → LangGraph → Deep Agents This is an introductory course to quickly experience only the core of AI agent.

Learning Objectives

- Learn how to safely manage API keys with the `.env` file.
- Initialize the LLM model with `ChatOpenAI`
- Check normal operation by sending a simple question to the model

0.1 API key settings

Copy `.env.example` to `.env` in the project root and enter the following keys:

```
OPENAI_API_KEY=sk-...
TAVILY_API_KEY=tvly-...   # Optional
```

Key	Purpose	Provider
`OPENAI_API_KEY`	LLM calls (required)	https://platform.openai.com/api-keys
`TAVILY_API_KEY`	Web Search tool (optional)	https://tavily.com

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("✓ API key loaded")
```

```
# Observability settings (optional) - LangSmith or Langfuse
# Set the key in .env, or uncomment it below and enter it yourself.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: Automatically activated when LANGSMITH_TRACING=true (no code modification required)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
```

```
    print(f"LangSmith tracing ON \u2014 project: {project}")

# Langfuse: Pass config={"callbacks": [langfuse_handler]} when calling invoke/stream
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

0.2 Model initialization

`ChatOpenAI` is a LangChain class that wraps an OpenAI-compatible LLM. This `model` object will be used repeatedly in subsequent Note books.

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
print("✓ Model setup complete:", model.model_name)
```

0.3 Smoke Test

Send a brief message to the model to see if it responds normally.

```
response = model.invoke("hello! Please answer in one sentence.", config=lf_config)
print("✓ Model response:", response.content)
```

Summary

Item	Content
Environment Variables	Load file `.env` into `load_dotenv()`
model	`ChatOpenAI(model="gpt-5.4")`
test	`model.invoke("...")` → Check response

Next Steps

→ 01_llm_basics.ipynb: Learn the basics of LLM — messages, prompts, streaming.

01 LLM Basics

Messages, Prompts, and Streaming

Before building agents, learn the basics of how to communicate with an LLM.

Learning Objectives

- Understand the roles of messages (`system` , `human` , `ai`)
- Control model behavior with system messages
- Receive real-time responses with `model.stream()`

1.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

1.2 The Three Roles of Messages

An LLM takes a list of messages as input. Each message has a role and is made up of three core pieces of information: role, content, and metadata.

Role	Class	Description
`system`	`SystemMessage`	Sets behavioral instructions for the model. It defines the model's initial behavior through persona, tone, rules, and similar guidance.
`human`	`HumanMessage`	Represents the user's input. It can include multimodal content such as images, audio, and files in addition to text.
`ai`	`AIMessage`	Represents the model's reply. Besides text, it can include properties such as `tool_calls` and `usage_metadata` .

There is also `ToolMessage` , which passes tool execution results back to the model. A `ToolMessage` must include the tool result content, the tool call ID, and the tool name.

1.3 Controlling Behavior with a System Message

If you change the `SystemMessage`, you can get very different answers to the same question. That is the core idea behind prompt engineering.

1.4 Dictionary Format

You can also pass messages as dictionaries instead of message objects. LangChain supports three input formats for messages:

1. String: Best for a simple text prompt, for example `model.invoke("Hello")`
2. Message objects: A typed list of instances such as `SystemMessage` and `HumanMessage`
3. Dictionary: The same `{"role": ..., "content": ...}` structure used by the OpenAI Chat Completions API

All three formats return the same kind of result, so you can choose the one that best fits your situation. The dictionary format is especially useful when migrating existing OpenAI code to LangChain.

1.5 Streaming

If you use `model.stream()`, tokens are printed as they are generated in real time. This can make the application feel much faster to the user.

LangChain models provide three main call styles:

- `invoke()`: A synchronous call that returns the full response at once
- `stream()`: Returns `AIMessageChunk` objects token by token for real-time output
- `batch()`: Handles multiple requests at once for better throughput

During streaming, each `AIMessageChunk` is progressively combined into the final message, and token usage can also be tracked incrementally.

1.6 Batch Calls

With `model.batch()`, you can send several questions at once. This is more efficient than repeatedly calling `invoke()` one request at a time.

Summary

Concept	Description
`SystemMessage`	Sets the model's persona and rules
`HumanMessage`	Represents user input

`model.invoke()`	Synchronous call (full response)
`model.stream()`	Real-time output at the token level
`model.batch()`	Processes multiple requests at once

Next Steps

→ 02_langchain_basics_en.ipynb: Build an agent with tools in LangChain.

02 LangChain Basics

Your First Agent

Build a tool-enabled agent with the core APIs of LangChain v1.

Learning Objectives

- Define custom tools with the `@tool` decorator
- Create an agent with `create_agent()`
- Run the agent with `invoke()` and inspect the result

2.1 Environment Setup

```python
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

2.2 Building Tools

When you add the `@tool` decorator, a regular Python function becomes an agent tool.

Important rules to remember when defining tools:

- Type hints: The function parameter type hints automatically define the tool's input schema. For example, `a: int` tells the model that the argument should be an integer.
- Docstring: The function docstring becomes the tool description. The model uses this description to decide which tool to use, so it should be clear and concise.
- Custom name/description: You can also set the tool name and description explicitly with `@tool("custom_name", description="...")`.
- Complex inputs: You can define richer input schemas with Pydantic `BaseModel` and `Field`.

```python
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

@tool
```

```
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

print("Tool list:")
for t in [add, multiply]:
    print(f"  - {t.name}: {t.description}")
```

2.3 Creating and Running an Agent

`create_agent()` combines a model and tools to build an agent. The agent internally runs a ReAct (Reasoning + Acting) loop:

```
Question → Model decides to call a tool → Tool runs → Result is observed → Repeat or produce a final
answer
```

Core components of the agent:

- Model: The LLM decides which tool to call. You can pass a string such as `"openai:gpt-5"` or a model object.
- Tools: These are the actions the agent can take. Compared with simple tool binding, an agent can call tools in sequence, run them in parallel, and retry them.
- System Prompt: Instructions that guide the agent's behavior.

An agent can call tools more than once, or even use multiple tools in parallel. The loop ends when the model produces a final answer or reaches the recursion limit.

```
from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="You are a math assistant.",
)
print("✓ Agent created")
```

Summary

Core API	Role
`\@tool`	Converts a function into an agent tool
`create_agent()`	Combines a model and tools to create an agent
`agent.invoke()`	Runs the agent and returns the result

Next Steps

→ 03_langchain_memory_en.ipynb: Learn about multi-turn conversations and memory.

03 LangChain Conversations

Multi-Turn Memory

Connect `InMemorySaver` so the agent can remember previous turns.

Learning Objectives

- Store conversation state with `InMemorySaver`
- Distinguish conversation sessions with `thread_id`
- Run a multi-turn conversation where the agent remembers earlier context

3.1 Environment Setup

```python
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

3.2 The Limit of an Agent Without Memory

A basic agent does not store state. Each call is treated like a new conversation.

3.3 Adding Memory with InMemorySaver

If you set a `checkpointer`, the agent stores the conversation history. A `thread_id` distinguishes one conversation session from another.

Short-term memory means keeping information from earlier interactions within a single conversation thread. This is essential for agents that handle complex tasks across several user turns.

Implementation pattern:

- Pass `InMemorySaver()` as the `checkpointer` parameter to enable memory.
- Use `thread_id` to distinguish different conversation sessions. If you reuse the same `thread_id`, the agent remembers the previous conversation.

- In production, you can switch to a database-backed checkpointer such as `PostgresSaver` for persistence.

If a conversation becomes very long, it may exceed the token budget. In that case, manage the history with strategies such as trimming, deletion, or summarization.

```
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model=model,
    tools=[add],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "session-1"}}
print("✓ Memory-enabled agent created")
```

3.4 Watching the Steps with Streaming

Use `agent.stream()` to observe each step of the agent in real time.

Agent streaming is useful when an agent runs for a while and you want to inspect intermediate steps as they happen. With `stream_mode="updates"`, you receive updates from each node individually, such as model calls and tool execution results. This lets you see what the agent is doing step by step.

Summary

Concept	Description
`InMemorySaver`	Stores conversation state in memory (checkpointer)
`thread_id`	Key that separates conversation sessions
`checkpointer=`	Passes a checkpointer into `create_agent()`
`stream(mode="updates")`	Shows the agent's execution steps in real time

Next Steps

→ 04_langgraph_basics_en.ipynb: Build a workflow with LangGraph.

04 LangGraph Basics

Building a Workflow

Use LangGraph's `StateGraph` to build a workflow by connecting nodes and edges.

Learning Objectives

- Define a state-based graph with `StateGraph`
- Register nodes (functions) and connect them with edges
- Run the graph with `compile()` → `invoke()`

4.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

4.2 Your First Graph

The basic LangGraph flow has five steps:

```
StateGraph(State) → add_node() → add_edge() → compile() → invoke()
```

Core ideas behind `StateGraph`:

LangGraph models agent workflows as graphs, and it relies on three core building blocks:

1. State: A shared data structure that represents the current snapshot of the application. It is usually defined with `TypedDict` or a Pydantic model.
2. Node: A function that receives state, performs some work, and returns an updated state. In other words, nodes do the actual work.
3. Edge: A transition that determines which node runs next based on the current state. In other words, edges decide what happens next.

`StateGraph` is the primary graph builder class, and it takes a user-defined state object. A graph must be compiled with `.compile()` before it can run, and compilation also validates the graph structure.

The example below is a one-node graph that counts the number of words in a piece of text.

4.3 A Two-Node Graph

Connect two nodes in sequence. The first node converts the text to uppercase, and the second node counts the words.

```
START → uppercase → counter → END
```

4.4 Using an LLM as a Node

If you use `MessagesState`, you can model an LLM conversation as a graph.

What is `MessagesState`?

`MessagesState` is a predefined state class provided by LangGraph. It contains a single `messages` key and uses `add_messages` as its reducer. Internally, it looks like this:

```
class MessagesState(TypedDict):
    messages: Annotated[list, add_messages]
```

The `add_messages` reducer tracks message IDs, accumulates messages without duplication, and automatically deserializes JSON into LangChain message objects. If you need additional fields such as documents or metadata, you can subclass `MessagesState`.

How nodes are structured:

A node is a normal Python function (sync or async) that receives the current state and returns a state update. LangGraph automatically wraps nodes as `RunnableLambda` objects, which adds batch support, async support, and native tracing.

Summary

Core API	Role
`StateGraph(State)`	Creates a graph builder from a state schema
`add_node()`	Registers a node (function)
`add_edge()`	Connects nodes
`compile()`	Produces an executable graph
`invoke()`	Runs the graph

Next Steps

→ 05_deep_agents_basics_en.ipynb: Build an all-in-one agent with Deep Agents.

05 Deep Agents Basics

An All-in-One Agent

Use the Deep Agents SDK's `create_deep_agent()` to build an agent with built-in tools, memory, and backend support in one line.

Learning Objectives

- Create an agent with `create_deep_agent()`
- Run the agent with `invoke()`
- Build an agent with an additional custom tool

5.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Model ready")
```

5.2 Creating an Agent

`create_deep_agent()` takes a LangChain model and returns an agent that already includes built-in tools such as file reading, file writing, and search. The return value is a LangGraph `CompiledStateGraph`, so you can call methods such as `invoke()` and `stream()` directly.

What is Deep Agents?

Deep Agents is a framework designed to simplify agent development. It works like an agent harness: internally it is built on top of LangChain agent components and uses LangGraph to manage execution.

Core capabilities in the current SDK:

Capability	Description
Task planning	Add `TodoListMiddleware` when a lesson needs the `write_todos` planning tool

Context management	Uses file system tools such as `write_file` and `read_file` to handle larger amounts of information without overflowing the token budget
Flexible storage	Supports pluggable backends such as in-memory storage, local disk, persistent stores, and sandbox environments
Subagent delegation	Can create specialized subagents for focused sub-tasks and isolate their context
Persistent memory	Reuses LangGraph's memory infrastructure to preserve information across conversations

How agent creation works:

Pass a model, optional tools, and an optional system prompt into `create_deep_agent()`. You need a model that supports tool calling, and you can use providers such as OpenAI or Anthropic.

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)
print(f"✓ Agent created (type: {type(agent).__name__})")
```

5.3 Adding a Custom Tool

If you write a Python function with a docstring and type hints, it becomes a tool directly.

How custom tools work:

A custom tool is just a normal Python function, and Deep Agents automatically converts two parts of that function:

- Docstring → tool description (used by the agent to understand what the tool does)
- Type hints → parameter schema (used by the agent to pass the correct arguments)

If you pass a list of functions to the `tools` parameter of `create_deep_agent()`, those functions are added to the tool list alongside harness tools such as file operations. Planning is now an explicit middleware choice, so add `TodoListMiddleware` in examples that rely on `write_todos`. You can also guide the agent's behavior with the `system_prompt` parameter.

Summary

Core API	Role
`create_deep_agent(model)`	Creates an agent with built-in tools
`create_deep_agent(model, tools, system_prompt)`	Adds custom tools and a system prompt
`agent.invoke()`	Runs the agent

Next Steps

→ 06_comparison_en.ipynb: Compare the three frameworks and choose your next learning track.

06 Comparing the Three Frameworks & Choosing What Comes Next

Compare LangChain, LangGraph, and Deep Agents at a glance, then decide where to continue next.

Learning Objectives

- Understand the core differences between LangChain, LangGraph, and Deep Agents
- Judge the best-fit use cases for each framework
- Choose a learning path into the intermediate tracks

6.1 Framework Comparison

Level of abstraction	High	Medium	Very high
Core concept	Agents + tools	Graphs + state + nodes	All-in-one agents
Agent creation	`create_agent()`	`StateGraph` → `compile()`	`create_deep_agent()`
Execution	`agent.invoke()`	`graph.invoke()`	`agent.invoke()`
Customization	Tools / prompts / memory	Nodes / edges / state / reducers	Tools / backends / subagents
Best fit	Fast prototyping	Complex workflows	Agents that need file and task management

Additional comparison details:

Feature	LangChain	LangGraph	Deep Agents

Model support	Model-agnostic (100+ providers)	Shares LangChain model integrations	Shares LangChain model integrations
License	MIT	MIT	MIT
Sandbox integration	No built-in support	No built-in support	Agents can run tasks in sandboxes
State management	Middleware-based	Checkpointer-based (supports time travel)	Reuses LangGraph checkpointers
Observability	LangSmith integration	Native LangSmith tracing	LangSmith support

The three frameworks are not mutually exclusive. Deep Agents uses LangGraph internally and shares LangChain's model and tool interfaces. A natural learning path is to build your foundations with LangChain, design complex workflows with LangGraph, and then create production-grade agents with Deep Agents.

6.2 Which One Should You Choose?

```
"I need a simple tool-calling agent."              → LangChain
"I need a workflow with branching and loops."      → LangGraph
"I want file operations and planning in one place." → Deep Agents
```

> ➡️ TIP
>
> The three frameworks are not mutually exclusive. Deep Agents uses LangGraph internally and shares LangChain's model and tool interfaces.

6.3 Code Comparison — The Same Task in Three Styles

Below is the smallest working version of the same task implemented with each framework.

LangChain

```python
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b
```

```
agent = create_agent(model=model, tools=[add])
agent.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

LangGraph

```
from langgraph.graph import StateGraph, START, END, MessagesState

def chatbot(state):
    return {"messages": [model.invoke(state["messages"])]}

builder = StateGraph(MessagesState)
builder.add_node("chat", chatbot)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)
graph = builder.compile()
graph.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

Deep Agents

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)
agent.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

Summary

Item	LangChain	LangGraph	Deep Agents
Core role	Agent creation (ReAct loop)	Workflow orchestration (state graphs)	All-in-one agents (built-in tools)
State management	Middleware-oriented	Explicit `StateGraph` state	Automated (filesystem + memory)
Best for	Fast prototyping, tool calling	Complex workflows, branching	Coding agents, data analysis, multi-step work
Learning curve	Low	Medium	Low

6.4 What Comes Next

Mini Project

→ 07_mini_project_en.ipynb: Build a search + summarization agent

Intermediate Tracks

Track	Description	Notebook Count
LangChain	Models, messages, tools, memory, middleware, multi-agent patterns	13
LangGraph	Graph API, workflows, agents, persistence, subgraphs	13
Deep Agents	Customization, backends, subagents, memory, advanced features	10

Recommended order:

1. LangChain — strengthen your model and tool fundamentals
2. LangGraph — design graph-based workflows
3. Deep Agents — build production-ready agents

The English intermediate notebooks will be translated next in this same order.

07 Mini Project

A Search + Summarization Agent

Combine what you learned in the beginner track to build a research agent with Tavily web search.

Learning Objectives

- Define a Tavily search tool directly
- Build a research agent with Deep Agents
- Observe the agent's execution process in real time with streaming
- Solve the same task with a LangChain agent and compare the two approaches

7.1 Environment Setup

This notebook requires `TAVILY_API_KEY`. You can get a free key at https://tavily.com.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is required!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY is required!"

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("✓ Environment ready")
```

7.2 Defining the Search Tool

Wrap the Tavily client in a search function. The docstring and type hints tell the agent what schema the tool should use.

Rules for writing a tool function:

Here you define a search function that will be passed to the `tools` parameter of `create_deep_agent()`. Deep Agents automatically converts the function docstring into a tool description and the type hints into a parameter schema. In practice, that means:

- Docstring: Gives the agent evidence for deciding when to use the tool. Be specific and clear.

- Type hints: Help the agent pass arguments of the correct type. `Literal` is useful when you want to restrict the allowed values.
- Args section: Explaining each parameter makes it easier for the agent to choose the right arguments.

```
from typing import Literal
from tavily import TavilyClient

tavily = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def internet_search(
    query: str,
    max_results: int = 3,
    topic: Literal["general", "news"] = "general",
) -> dict:
    """Search the internet for information.

    Args:
        query: Search query
        max_results: Maximum number of results
        topic: Search topic category
    """
    return tavily.search(query, max_results=max_results, topic=topic)

print("✓ Search tool ready")
```

7.3 A Deep Agents Research Agent

Pass the search tool and a system prompt into `create_deep_agent()`.

The agent's workflow with planning enabled:

When the agent receives a request, this example can go through the following steps because it opts into `TodoListMiddleware`:

1. Planning: Breaks the task into steps with the built-in `write_todos` tool
2. Research: Collects information from the web with the provided `internet_search` tool
3. Context management: Uses filesystem tools such as `write_file` and `read_file` to store intermediate results when needed and manage the token budget
4. Synthesis: Analyzes the gathered information and turns it into a consistent final report

For more complex tasks, the agent can also create specialized subagents and isolate their context for focused sub-work.

```
from deepagents import create_deep_agent
from deepagents.middleware import TodoListMiddleware

research_agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    middleware=[TodoListMiddleware()],
    system_prompt="You are an expert researcher. Search the web and summarize the results in English.",
)
print("✓ Research agent created")
```

7.4 Observing the Process with Streaming

With `stream(mode="updates")`, you can watch the agent's steps in real time.

LangGraph's streaming system:

LangGraph provides a flexible streaming system that improves responsiveness by showing progress before the final response is complete.

Stream Mode	Use Case
`values`	Streams the full state after each graph step
`updates`	Streams only the state changes after each step
`messages`	Streams LLM tokens together with metadata
`custom`	Streams custom data emitted from nodes
`debug`	Streams comprehensive execution details

You can access streaming with `stream()` (sync) or `astream()` (async), and you can combine multiple modes at once by passing a list. In the example below, we use `updates` so that each stage of the agent, such as tool calls and the final response, appears as it happens.

7.5 Comparing It with a LangChain Agent

Try the same search tool with a LangChain `create_agent()`.

How it differs from a LangChain agent:

LangChain's `create_agent()` builds a simple ReAct agent from a model and a list of tools. Compared with Deep Agents:

- LangChain: A lightweight baseline for tool-calling agents. It is excellent for fast prototyping, but advanced capabilities such as task planning or filesystem management must be added manually.
- Deep Agents: Provides the harness for file management and subagent delegation, and can opt into planning (`TodoListMiddleware` / `write_todos`), so it is a better fit for complex multi-step tasks.

If you add the `@tool` decorator, you can convert a function into LangChain's tool interface. The general pattern of passing a system prompt and a list of tools into `create_agent()` is similar to Deep Agents.

Summary

Technologies used in this mini project:

Technique	Source

`ChatOpenAI` + `load_dotenv`	00_setup
Message roles, streaming	01_llm_basics
`\@tool` , `create_agent()`	02_langchain_basics
`InMemorySaver` , `thread_id`	03_langchain_memory
`StateGraph` , `compile()`	04_langgraph_basics
`create_deep_agent()`	05_deep_agents_basics

Next Steps

→ Continue to the intermediate tracks. Use 06_comparison_en.ipynb as your roadmap.

PART

II

LangChain

Building Blocks for AI Applications

What You Will Learn in This Part

- 1. Introduction to LangChain
- 2. Quickstart
- 3. Models and Messages
- 4. Tools and Structured Output
- 5. Memory and Streaming
- 6. Middleware
- 7. HITL and Runtime
- 8. Multi-Agent
- 9. Custom Workflow and RAG
- 10. Production
- 11. MCP
- 12. Frontend Streaming
- 13. Guardrails
- 14. Semantic Search

01 Introduction to LangChain

An Overview of the LangChain v1 Framework

Learning Objectives

Understand the structure and core components of the LangChain framework.

By the end of this notebook, you will understand:

- The three-layer structure of the LangChain v1 framework
- How the ReAct agent pattern works
- The core components and main APIs
- How to set up and verify the development environment

1.1 LangChain Framework Overview

LangChain v1 is an integrated framework for building LLM-based agents. It is organized into three layers, and each layer provides a different level of abstraction.

Three-Layer Structure



Layer	Role	Target User
LangChain	Core APIs for agent creation (`create_agent` , `tool` , `ChatOpenAI`)	All developers
LangGraph	Building complex workflows (state graphs, checkpointers, streaming)	Intermediate and advanced developers
Deep Agents	Prebuilt agents (coding, research, and more)	Fast prototyping
LangSmith	Tracing and debugging platform for agents built with any framework	All developers / operators

> ➡️ TIP
>
> LangSmith is not limited to LangChain, LangGraph, or Deep Agents. You can send traces from the OpenAI SDK, LlamaIndex, or your own custom agent and use the same UI to debug and evaluate them, so v1 treats it as the fourth core tool.

Major Changes in LangChain v1

Item	Previous (v0.x)	Current (v1)
Agent creation	`create_react_agent()`	`create_agent()`
Agent import	`from langchain.agents import ...` (various forms)	`from langchain.agents import create_agent`
Model initialization	Direct use of `ChatOpenAI(...)`	`init_chat_model()` or `ChatOpenAI(...)`
Memory	`ConversationBufferMemory` and similar classes	`InMemorySaver` (LangGraph checkpointer)
Execution engine	AgentExecutor	LangGraph graph (internally)

Core Design Philosophy

The central design idea in LangChain v1 is that every agent runs as a LangGraph graph. An agent created with `create_agent()` is implemented internally as a LangGraph `StateGraph` , which enables:

- Streaming: Real-time responses with the `stream()` method
- State management: Conversation history through a checkpointer
- Extensibility: Adding custom nodes and edges when needed

Multi-Provider Support

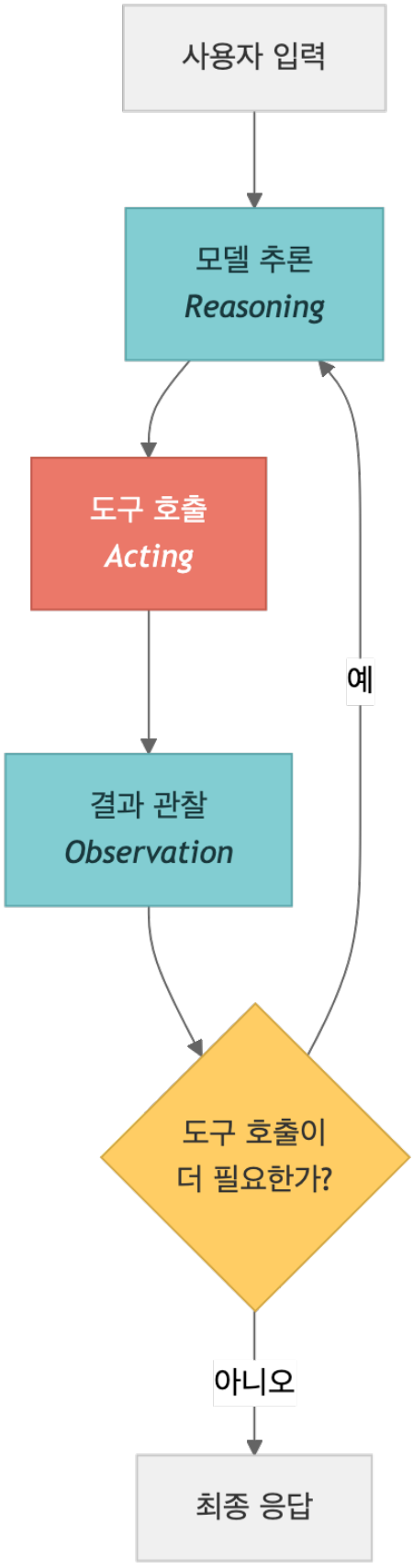
LangChain v1 exposes more than 100 LLM providers behind the same interface. The model IDs you will see most often in this book are:

- OpenAI: `openai:gpt-5.4`, `openai:gpt-4.1`
- Anthropic: `anthropic:claude-sonnet-4-6`, `anthropic:claude-opus-4-6`
- Google: `google:gemini-2.5-flash-lite`, `google:gemini-2.5-pro`
- OpenRouter: `openrouter:meta-llama/llama-3.3-70b-instruct` and 100+ others
- AWS Bedrock: `bedrock:anthropic.claude-sonnet-4-6`
- Ollama: `ollama:llama3.3` (runs locally)

Unless noted otherwise, the examples in this book default to OpenAI `gpt-5.4` or Anthropic `claude-sonnet-4-6`.

1.2 The ReAct Agent Pattern

The ReAct (Reasoning + Acting) pattern is the default operating model for LangChain v1 agents. The agent repeatedly runs the following loop:


사용자 입력
모델 추론
Reasoning
도구 호출
Acting
결과 관찰
Observation
도구 호출이
더 필요한가?
예
아니오
최종 응답

Key Characteristics

- Autonomous decisions: The agent decides for itself whether it should use a tool
- Multi-step reasoning: The agent breaks complex tasks into multiple steps
- Observation-based updates: The agent observes tool results and chooses its next action

1.3 Overview of the Main Components

The table below summarizes the core components of LangChain v1:

Component	Description	Main APIs
Model	An LLM or chat model. Acts as the agent's "brain."	`ChatOpenAI` , `init_chat_model()`
Tools	Functions the agent can use, such as search, calculation, or API calls	`\@tool` decorator, `TavilySearch`
Agent	The execution unit that combines the model and tools. Internally, it is a LangGraph graph	`create_agent()`
Memory	A checkpointer that stores and manages conversation history	`InMemorySaver` , `SqliteSaver`
Middleware	Logic inserted into the request/response processing pipeline	`prompt` , `before_tool` , `after_model`
State	The state managed while the agent runs, such as messages and context	`AgentState` , `messages`
Streaming	Support for real-time response streaming	`stream()` , `stream_mode="updates"`

1.4 Environment Setup and Installation Check

Check the packages and API keys required for LangChain v1 development.

```
# Environment check
import importlib

packages = {
    "langchain": "langchain",
    "langchain_openai": "langchain-openai",
    "langchain_community": "langchain-community",
    "langgraph": "langgraph",
}

print("=" * 50)
print("LangChain v1 environment check")
print("=" * 50)
```

```
for module_name, package_name in packages.items():
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, "__version__", "installed")
        print(f"✓ {package_name}: {version}")
    except ImportError:
        print(f"✗ {package_name}: not installed → pip install {package_name}")
```

```
# API key check
from dotenv import load_dotenv
import os

load_dotenv(override=True)

required_keys = ["OPENAI_API_KEY"]
optional_keys = ["TAVILY_API_KEY", "LANGSMITH_API_KEY"]

print("Required API keys:")
for key in required_keys:
    status = "✓ configured" if os.environ.get(key) else "✗ missing"
    print(f"  {key}: {status}")

print("\nOptional API keys:")
for key in optional_keys:
    status = "✓ configured" if os.environ.get(key) else "- not configured (optional)"
    print(f"  {key}: {status}")
```

```
# Verify the core LangChain v1 imports
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver

print("✓ Successfully imported all core modules")
print("  - create_agent: create a LangChain v1 agent")
print("  - tool: tool decorator")
print("  - ChatOpenAI: OpenAI-compatible chat model")
print("  - InMemorySaver: memory checkpointer")
```

1.5 Next Steps

In this notebook, you explored the overall structure and core components of the LangChain v1 framework.

In the next notebook (`02_quickstart.ipynb`), you will create and run an actual agent:

- Define a custom tool with the `@tool` decorator
- Create an agent with `create_agent()`
- Run the agent with `invoke()` and `stream()`
- Build a multi-turn conversation with `InMemorySaver`

Summary

Item	Description

LangChain v1	An agent-centered framework built around the `create_agent()` API
Three-layer structure	LangChain → LangGraph → Deep Agents
ReAct pattern	Repeated loop of reasoning → action → observation
Core APIs	`create_agent()`, `\@tool`, `invoke()`, `stream()`

Next Steps

→ 02_quickstart.ipynb: Build your first LangChain agent.

02 Your First Agent

Getting Started with `create_agent()`

Learning Objectives

Create and run an agent with LangChain v1's `create_agent()`.

By the end of this notebook, you will be able to:

- Define custom tools with the `@tool` decorator
- Create an agent with `create_agent()`
- Run the agent with `invoke()` and inspect the result
- Receive real-time streaming responses with `stream()`
- Build a multi-turn conversation with `InMemorySaver`

2.1 Environment Setup

Install LangChain v1 and Deep Agents before running any examples. Pick the command that matches your environment:

```
# With uv
uv add langchain deepagents

# With pip
pip install -U langchain deepagents
```

> ➡️ **TIP**
>
> Installing `langchain` also pulls in `langchain-core`, `langgraph`, and `langchain-openai` (OpenAI integration). For other providers add `langchain-anthropic`, `langchain-google-genai`, and so on as needed.

Set up the model through OpenAI. `ChatOpenAI` supports OpenAI-compatible APIs, so you can also switch providers by changing `base_url` when needed.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

from langchain_openai import ChatOpenAI
```

```
model = ChatOpenAI(
    model="gpt-5.4",
)
print("✓ Model configured:", model.model_name)
```

2.2 Building a Simple Tool

Define the tools that the agent can use with the `@tool` decorator.

Important details when defining a tool:

- A docstring is required. The agent uses it to understand what the tool is for.
- Type hints help the agent pass the correct arguments.
- The tool name is generated automatically from the function name.

```
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

print("Tool list:")
for t in [add, multiply]:
    print(f"  - {t.name}: {t.description}")
```

2.3 Creating an Agent

Combine the model and tools with `create_agent()`.

The created agent is internally implemented as a LangGraph graph, so it provides methods such as `invoke()` and `stream()`.

> ➡️ TIP
>
> In LangChain v1, use `create_agent()` instead of `create_react_agent()`.

> ➡️ TIP
>
> Give the agent a snake_case name like `name="research_assistant"`. It shows up in LangSmith traces, sub-agent routing, and logs, which makes the agent easier to identify. Avoid spaces, hyphens, and uppercase characters.

> ➡️ TIP
>
> When you define a custom `state_schema`, it must be a **TypedDict** subclass of `langgraph.graph.MessagesState` or `langchain.agents.AgentState`. Plain `dataclass` or Pydantic `BaseModel` classes are not accepted.

```python
from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="You are a math assistant. Use the available tools for calculations.",
)
print("✓ Agent created")
print(f"  Type: {type(agent).__name__}")
```

2.4 Running the Agent

Run the agent with `invoke()`.

When you send messages to the agent, it runs an internal ReAct loop:

1. The model analyzes the question and decides whether to call a tool
2. The tool runs and returns a result
3. The model uses the result to generate the final response

```python
# Inspect the full message flow
print("Full message flow:")
print("=" * 50)
for msg in result["messages"]:
    role = msg.type if hasattr(msg, 'type') else msg.get('role', 'unknown')
    content = msg.content if hasattr(msg, 'content') else msg.get('content', '')
    print(f"[{role}] {content[:200]}")
    print("-" * 50)
```

2.5 Streaming Execution

Receive a real-time response with `stream()`.

With streaming, you can inspect each stage of the agent in real time, including model reasoning, tool calls, and the final answer. If you use `stream_mode="updates"`, you receive node updates one by one.

2.6 Multi-Turn Conversation

Keep conversation state with `InMemorySaver`.

`InMemorySaver` stores state in memory and separates conversation sessions with `thread_id`.

> ➡️ TIP
>
> In LangChain v1, conversation history is managed through LangGraph checkpointers.

2.7 Adding the Tavily Search Tool (Optional)

Add a web search tool so the agent can look up real information.

Tavily is a search API designed for AI agents.

This cell only runs if `TAVILY_API_KEY` is configured.

2.8 Runtime Context (`context_schema`)

While `thread_id` separates conversation sessions, `context` is the channel that injects per-request metadata (user ID, permissions, tenant) into tools and middleware. Register `context_schema=` on `create_agent()` and pass `context=` to `invoke()`; tools can then read it through `ToolRuntime.context`.

```python
from dataclasses import dataclass
from langchain.agents import create_agent

@dataclass
class Context:
    user_id: str
    tenant: str = "default"

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="...",
    context_schema=Context,
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    context=Context(user_id="u-1234", tenant="acme"),
)
```

> ➡️ TIP
>
> `context_schema` pairs with `ToolRuntime` in Chapter 4 and with HITL in Chapter 7. For now, just remember that the injection channel exists; the detailed usage comes later.

2.9 Summary

Here is a recap of what you covered in this notebook:

Topic	Core API	Description
Tool definition	`\@tool`	Turn a function into an agent tool with a decorator
Agent creation	`create_agent()`	Combine a model, tools, and a system prompt
Synchronous execution	`agent.invoke()`	Return the full response at once
Streaming execution	`agent.stream()`	Return real-time updates for each step
Multi-turn conversation	`InMemorySaver` + `thread_id`	Store and restore conversation state with a checkpointer
Search tool	`TavilySearch`	Access real-time information through web search

Next Steps

In the next notebook, you will explore more advanced agent patterns:

- Designing custom system prompts
- Combining multiple tools in a single agent
- Error handling and fallback strategies

03 Models and the Message System

Learn how to configure different LLM models in LangChain v1 and structure conversations with message types.

Learning Objectives

- Understand the LangChain v1 model initialization methods (`init_chat_model` , `ChatOpenAI`)
- Learn the three call patterns: `invoke()` , `stream()` , and `batch()`
- Understand message types such as `SystemMessage` , `HumanMessage` , `AIMessage` , and `ToolMessage`
- Learn how to construct multimodal messages (image input)

3.1 Environment Setup

Load API keys from `.env` and initialize the model through OpenAI.

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv(override=True)

# Initialize the model with OpenAI
model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model initialized:", model.model_name)
```

3.2 Comparing Model Providers

LangChain v1 can initialize models from many providers through the unified `init_chat_model()` API.

Provider	Model String Format	Required Package	Environment Variable
OpenAI	`"openai:gpt-5"`	`langchain-openai`	`OPENAI_API_KEY`
Anthropic	`"anthropic:claude-sonnet-4-6"`	`langchain-anthropic`	`ANTHROPIC_API_KEY`
Google	`"google:gemini-2.5-flash-lite"`	`langchain-google-genai`	`GOOGLE_API_KEY`

AWS Bedrock	`"bedrock:anthropic.claude-v3"`	`langchain-aws`	AWS credentials
Azure	`"azure:gpt-4o"`	`langchain-openai`	`AZURE_OPENAI_API_KEY`
Ollama	`"ollama:llama3"`	`langchain-ollama`	(local runtime)

> · NOTE
>
> Note: When using OpenAI, you can set `base_url` and `api_key` directly on `ChatOpenAI` to connect to OpenAI-style services such as vLLM, LM Studio, or Ollama.

3.3 Using `init_chat_model()`

`init_chat_model()` is the unified model initialization helper introduced in LangChain v1. If the provider-specific package is installed, you can create a model with a single string.

When you are using OpenAI directly, `ChatOpenAI` is usually the more straightforward option.

3.4 `invoke()`, `stream()`, and `batch()` Patterns

Every LangChain v1 model supports three calling patterns:

Method	Description	Return Type
`invoke()`	One response for one input	`AIMessage`
`stream()`	Token-by-token streaming response	`Iterator[AIMessageChunk]`
`batch()`	Handles multiple inputs at the same time	`List[AIMessage]`

3.5 Message Types

LangChain v1's message system clearly separates the roles inside a conversation:

Message Type	Role	Description
`SystemMessage`	System	A system prompt that defines how the model should behave
`HumanMessage`	User	A message entered by the user
`AIMessage`	AI	A response generated by the model
`ToolMessage`	Tool	A message that passes a tool result back to the model

If you build a list of messages and pass it into `model.invoke()`, you can preserve the conversation context across turns.

3.6 Multimodal Messages (Image Input)

In LangChain v1, you can send text and images together in the `content` of a `HumanMessage`. Images can be passed as URLs or as base64-encoded content, and this works only with models that support vision input.

```
content = [
    {"type": "text", "text": "Description text"},
    {"type": "image_url", "image_url": {"url": "IMAGE_URL"}},
]
```

3.7 Standard Output Format — `output_version="v1"`

Starting from LangChain v1, every provider can emit a unified content blocks serialization. Set `init_chat_model("openai:gpt-5.4", output_version="v1")` or the env var `LC_OUTPUT_VERSION="v1"`, and `AIMessage.content_blocks` will give you text, tool calls, reasoning, thinking, and citations in a single schema.

```
import os
from langchain.chat_models import init_chat_model

os.environ["LC_OUTPUT_VERSION"] = "v1"

model = init_chat_model("openai:gpt-5.4", output_version="v1")
result = model.invoke("Describe LangChain v1 in one sentence.")

for block in result.content_blocks:
    print(block["type"], "→", block.get("text", block))
```

Streaming uses the same shape. Read partial blocks from `chunk.content_blocks` and map them to your UI.

```
for chunk in model.stream("Explain multimodal input."):
    for block in chunk.content_blocks:
        if block["type"] == "text":
            print(block["text"], end="")
```

> ➡️ TIP
>
> `output_version="v1"` is the recommended setting for absorbing provider differences. Legacy code that depends on the string `result.content` still works in `v0`, but new projects should use v1.

3.8 Tracking Token Usage — `UsageMetadataCallbackHandler`

For multi-step agent runs, aggregate token usage with a callback handler. Pass `UsageMetadataCallbackHandler` through `config={"callbacks": [...]}`, and `input_tokens` / `output_tokens` / `total_tokens` accumulate across every model call.

```
from langchain_core.callbacks import UsageMetadataCallbackHandler

cb = UsageMetadataCallbackHandler()
result = agent.invoke(
    {"messages": [{"role": "user", "content": "What is 15 * 27 + 3?"}]},
    config={"callbacks": [cb]},
)

print(cb.usage_metadata)
# {'gpt-5.4': {'input_tokens': 312, 'output_tokens': 48, 'total_tokens': 360, ...}}
```

3.9 Runtime Config — Per-Invocation Options

The `config=` argument on `invoke()` / `stream()` carries execution metadata for a single call. You can set the LangSmith trace name, tags, metadata, and concurrency limits for tools and sub-agents all at once.

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={
        "run_name": "monthly-report",
        "tags": ["batch-2025-05", "finance"],
        "metadata": {"user_id": "u-1234", "tenant": "acme"},
        "max_concurrency": 5,
    },
)
```

> ➡️ **TIP**
>
> `run_name`, `tags`, and `metadata` flow into LangSmith and become filter/search keys in the UI. `max_concurrency` caps the parallel tool and sub-agent calls inside one graph execution.

3.10 Connection Resilience — `max_retries` / `timeout`

In production, you have to ride out temporary outages from OpenAI, Anthropic, and other providers. Pass `max_retries` and `timeout` to `init_chat_model()` or `ChatOpenAI()`, and LangChain will retry with exponential backoff.

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "openai:gpt-5.4",
```

```
    max_retries=15,   # default is 6 — raise it for resilience
    timeout=120,      # seconds, for long responses
)
```

> ➡️ TIP
>
> Retries fire on 5xx errors, connection failures, and `429 Too Many Requests`. Client errors like 422 are not retried. Setting the limit too high amplifies cost and latency, so most teams keep it in the 10–20 range.

3.11 Prompt Caching — Implicit vs Explicit

When you reuse a long system prompt or RAG context, prompt caching can cut input token costs sharply.

- Implicit caching: the provider detects an identical prefix automatically. OpenAI `gpt-5.4` enables this for prefixes of 1024 tokens or more.
- Explicit caching: Anthropic uses `prompt_cache_key` (or a `cache_control` block) to mark cache keys. In LangChain you pass it via `model_kwargs={"prompt_cache_key": "..."}`.

```
from langchain_anthropic import ChatAnthropic

model = ChatAnthropic(
    model="claude-sonnet-4-6",
    model_kwargs={"prompt_cache_key": "system-v3"},
)
```

> ➡️ TIP
>
> On cache hits, `usage_metadata.input_tokens_details` reports `cache_read` and `cache_creation` separately. Track cache hit rate as an ops metric with `UsageMetadataCallbackHandler`.

3.12 Summary

Here is a summary of the main ideas from this notebook.

Item	Description
`init_chat_model()`	Initializes models through a unified provider string
`ChatOpenAI(base_url=...)`	Uses OpenAI or another custom endpoint
`invoke()`	Single input → single response

`stream()`	Token-level streaming response
`batch()`	Handles multiple inputs at once
`SystemMessage`	Sets system instructions
`HumanMessage`	Represents user input
`AIMessage`	Represents an AI response (for conversation history)
`ToolMessage`	Carries tool execution results
Multimodal message	Sends text and images together in `content`

Next Steps

→ 04_tools_and_structured_output.ipynb: Learn about tools and structured output.

04 Tools and Structured Output

Learn how to build custom tools with the `@tool` decorator in LangChain v1 and how to receive structured responses with `with_structured_output()`.

Learning Objectives

- Build tools with the `@tool` decorator and inspect their schemas
- Define complex input schemas with Pydantic models
- Connect tools to `create_agent()` and build an agent
- Access runtime context from inside a tool with `ToolRuntime`
- Configure structured output with `with_structured_output()`
- Understand the difference between `ToolStrategy` and `ProviderStrategy`

4.1 Environment Setup

> ➡️ TIP
>
> Some examples in this chapter require deepagents ≥ 0.5.0 or langgraph ≥ 1.1.5, and the `ToolRuntime` `execution_info` / `server_info` extensions require langchain ≥ 1.2. Older versions raise `AttributeError`, so confirm with `uv pip list | grep -E "langchain|langgraph|deepagents"`.

Load API keys and initialize an OpenAI model.

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv(override=True)

# Initialize the model with OpenAI
model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model initialized:", model.model_name)
```

4.2 The Basics of the `\@tool` Decorator

When you add `@tool` to a function, it becomes a tool that an agent can use. LangChain automatically parses the function name, docstring, and type hints to build the tool schema.

```
from langchain.tools import tool

@tool
def my_tool(param: str) -> str:
    """Tool description for the LLM."""
    return result
```

```
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Look up the current weather for a city."""
    weather_data = {
        "Seoul": "Clear, 15\u00b0C",
        "Tokyo": "Cloudy, 12\u00b0C",
        "New York": "Rain, 8\u00b0C",
    }
    return weather_data.get(city, f"Weather data is not available for: {city}")

# Inspect the tool schema
print("Tool name:", get_weather.name)
print("Tool description:", get_weather.description)
print("Input schema:", get_weather.args_schema.model_json_schema())
```

4.3 Complex Schemas with Pydantic

If you need a richer input structure, define the schema with a Pydantic `BaseModel`. When you pass it as `@tool(args_schema=MySchema)`, the LLM can understand the exact parameter structure.

- `Field(description=...)` : passes a field description to the LLM
- `Field(default=...)` : defines a default value

```
from pydantic import BaseModel, Field

class SearchQuery(BaseModel):
    """Search parameters for a database query."""
    query: str = Field(description="Search query string")
    max_results: int = Field(default=5, description="Maximum number of results to return")
    category: str = Field(default="all", description="Search category: all, tech, science, news")

@tool(args_schema=SearchQuery)
def search_database(query: str, max_results: int = 5, category: str = "all") -> str:
    """Search the database with advanced filtering options."""
    return f"Found {max_results} results for '{query}' in the '{category}' category"

print("Complex schema:", search_database.args_schema.model_json_schema())
```

4.4 Connecting Tools to an Agent

When you pass a list of tools into `create_agent()`, the agent can automatically choose and execute the right tool for the situation.

```python
from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[tool1, tool2],
    system_prompt="...",
)
```

> · NOTE
>
> Note: In LangChain v1, `create_react_agent` was removed. Always use `create_agent`.

4.5 ToolRuntime

`ToolRuntime` lets a tool function access the current conversation state at runtime. This makes it possible to build tools that use message history, settings, or other runtime context.

```python
@tool
def my_tool(runtime: ToolRuntime) -> str:
    messages = runtime.state["messages"]
    # ...
```

`runtime.execution_info` — Execution Metadata

`ToolRuntime` exposes more than state access. The `execution_info` field carries metadata about the current execution unit and is heavily used for logging, tracing, and retry policies.

```python
@tool
def log_call(payload: str, runtime: ToolRuntime) -> str:
    """Process input and log execution metadata."""
    info = runtime.execution_info
    print(f"thread={info.thread_id} run={info.run_id} attempt={info.node_attempt}")
    return f"received: {payload}"
```

- `execution_info.thread_id`: session ID that groups a multi-turn conversation
- `execution_info.run_id`: ID of one `invoke()` / `stream()` run (linked to the LangSmith trace)
- `execution_info.node_attempt`: attempt count for the current node (increments on retries)

`runtime.server_info` — Server and User Info

When executed on LangGraph Platform or a LangSmith server, `runtime.server_info` identifies the deployed assistant and the calling user.

```python
@tool
def user_summary(runtime: ToolRuntime) -> str:
```

```
    """Return information about the current user."""
    s = runtime.server_info
    return f"assistant={s.assistant_id} user={s.user.email if s.user else 'anonymous'}"
```

- `server_info.assistant_id` : ID of the deployed assistant (graph version)
- `server_info.user` : authenticated user object (`user.id` , `user.email` , permissions)

> ➡️ TIP
>
> `server_info` may be empty in local runs — it is populated only in deployed environments. Guard with `if runtime.server_info.user is not None:` .

Tool Error Handling — `@wrap_tool_call`

To attach uniform error handling, logging, or retry policy to tool calls, use the `@wrap_tool_call` decorator. It receives a `ToolCallRequest` and acts as a middleware hook that intercepts tool execution.

```
from langchain.tools import wrap_tool_call, ToolCallRequest
from langchain_core.messages import ToolMessage

@wrap_tool_call
def safe_invoke(request: ToolCallRequest, handler):
    """Wrap every tool call in try/except."""
    try:
        return handler(request)
    except Exception as exc:  # noqa: BLE001
        return ToolMessage(
            content=f"[error] {type(exc).__name__}: {exc}",
            tool_call_id=request.tool_call.id,
            status="error",
        )

agent = create_agent(
    model=model,
    tools=[get_weather, search_database],
    middleware=[safe_invoke],
)
```

> ➡️ TIP
>
> `@wrap_tool_call` registers as middleware, so it applies to every tool. Branch on `request.tool_call.name` if you need to guard only specific tools.

`Command` Updates and Companion `ToolMessage`

When a tool needs to update graph state or pick the next node instead of returning plain text, return a `Command` . LangGraph applies `Command.update` to patch state and uses `Command.goto` to choose the next node. You must also return a `ToolMessage` that closes the `tool_call_id` , otherwise the model cannot proceed to the next turn.

```
from langgraph.types import Command
from langchain_core.messages import ToolMessage
```

```python
@tool
def transfer_to_specialist(topic: str, runtime: ToolRuntime) -> Command:
    """Hand control to a specialist sub-agent and update state."""
    return Command(
        update={
            "active_specialist": topic,
            "messages": [
                ToolMessage(
                    content=f"Routed to the {topic} specialist.",
                    tool_call_id=runtime.tool_call_id,
                )
            ],
        },
        goto="specialist_node",
    )
```

> ➡️ TIP
>
> Omitting `tool_call_id` causes the model to error with "no response for tool call." Pull the current call ID from `runtime.tool_call_id` and pass it through verbatim.

4.6 Structured Output

With `with_structured_output()`, you can receive the model's response directly as a Pydantic model or dataclass. This pattern is used directly on the model, not through an agent.

```python
structured_model = model.with_structured_output(MySchema)
result = structured_model.invoke("...")
# result is an instance of MySchema
```

4.7 ToolStrategy vs ProviderStrategy

There are two main strategies for structured output inside an agent:

Strategy	Description	Advantage
`ToolStrategy`	Uses the tool-calling mechanism to produce structured output	Works with every model and is stable
`ProviderStrategy`	Uses the provider's native structured-output feature	Faster and more accurate when the model supports it

Use the `response_format` parameter to structure the agent's final response.

> ➡️ TIP

> `ProviderStrategy.strict=True` (schema enforcement) requires langchain ≥ 1.2. On older releases, use `strict=False` or `ToolStrategy`.

> ➡️ **TIP**
>
> `ProviderStrategy` is currently stable on OpenAI (`gpt-5.4` , `gpt-4.1`) and Anthropic (`claude-sonnet-4-6`). Google Gemini is unofficially supported: JSON mode works, but strict schema enforcement applies only to some models. Choose `ToolStrategy` when broad compatibility matters.

4.8 Summary

Here is a summary of the main ideas in this notebook.

Item	Description
`\@tool` decorator	Converts a function into an agent tool
`args_schema`	Defines a complex input schema with Pydantic
`create_agent()`	Connects the model and tools to create an agent
`ToolRuntime`	Gives a tool access to runtime state such as conversation history
`with_structured_output()`	Structures model output as a Pydantic model or dataclass
`ToolStrategy`	Structured agent output through tool calling
`ProviderStrategy`	Provider-native structured output

Next Steps

→ 05_memory_and_streaming.ipynb: Learn about memory and streaming.

05 Memory and Streaming

Learn about the memory system and streaming modes used by LangChain v1 agents.

Learning Objectives

- Short-term memory: understand how to preserve conversation state with `InMemorySaver` and `thread_id`
- Long-term memory: use `InMemoryStore` to persist memory across conversations
- Message trimming: learn how to keep long conversations within a token budget
- Streaming modes: understand the differences between `values`, `updates`, `messages`, and `custom`

5.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model ready:", model.model_name)
```

5.2 Short-Term Memory: `InMemorySaver`

Short-term memory is the mechanism that remembers previous messages within a single conversation session.

- `InMemorySaver` acts as a checkpointer and stores agent state in memory.
- `thread_id` separates different conversation sessions.
- Reusing the same `thread_id` preserves previous conversation context.

5.3 Independent Conversations with Different `thread_id` Values

If you use a different `thread_id`, you create a completely independent conversation session. Context is not shared with the previous session.

5.4 Message Trimming

As a conversation grows, the token count increases and affects both cost and performance. Message trimming keeps only the most relevant messages within a token budget.

- `trim_messages` : keeps only the most recent N messages or the messages that fit within the token budget
- `strategy="last"` : prioritizes the most recent messages
- `include_system=True` : always preserves the system message

5.5 Long-Term Memory: `InMemoryStore`

Long-term memory stores information that persists across conversation sessions.

- `InMemoryStore` is a key-value store for user preferences, settings, and similar data.
- Tools can access the store through the `ToolRuntime` parameter.
- The same data is available from every session, regardless of `thread_id` .

Per-user namespaces (`context_schema` + `runtime.context`)

In real apps you typically scope the store with the current `user_id` . Declare a `context_schema` , read `runtime.context.user_id` inside the tool, and cast Pydantic/dataclass objects with `dict(...)` before writing so the value is JSON-serializable.

```
from dataclasses import dataclass
from langgraph.store.memory import InMemoryStore
from langchain.tools import tool, ToolRuntime

@dataclass
class Context:
    user_id: str

@tool
def remember_user(user_info: dict, runtime: ToolRuntime[Context]) -> str:
    """Persist the current user's profile."""
    namespace = (runtime.context.user_id, "profile")
    runtime.store.put(namespace, "info", dict(user_info))
    return "saved"

store = InMemoryStore()
agent = create_agent(
    model=model,
    tools=[remember_user],
    store=store,
    context_schema=Context,
)
agent.invoke(
    {"messages": [{"role": "user", "content": "I'm Minji"}]},
    context=Context(user_id="user_123"),
)
```

> ➡️ TIP
>
> If you forget to pass `store=...` to `create_agent` , `runtime.store` is `None` and the tool cannot access the store. Always wire the store in alongside any long-term memory tool.

> ➡️ TIP
>
> In production, swap `InMemoryStore` for `PostgresStore` (or `AsyncPostgresStore`) from the `langgraph-checkpoint-postgres` package — same interface, persistent storage, and semantic search.

Differences between short-term and long-term memory:

Type	Short-Term Memory (Checkpointer)	Long-Term Memory (Store)
Scope	Inside a single `thread_id`	Across all sessions
What it stores	Conversation message history	User preferences, learned data
Lifetime	Until the session ends (or persists)	Until explicitly deleted
Access	Automatic (inside the agent)	Explicit (through tools)

5.6 Streaming Modes

LangChain provides streaming so you can observe agent execution in real time. Choose the mode that best fits your use case.

Streaming Mode Comparison

Mode	Description	Use Case
`values`	Full state after each step	Debugging, state inspection
`updates`	Only the updates from each node	Progress displays
`messages`	Message tokens	Chat UI
`custom`	Custom user-defined events	Custom progress indicators

A Note on `stream_mode="custom"`

`stream_mode="custom"` carries user-defined events. With current LangGraph, you can emit them from inside tools or middleware of a `create_agent` agent by calling `get_stream_writer()`.

```python
from langgraph.config import get_stream_writer
from langchain.tools import tool

@tool
def long_running_task(query: str) -> str:
    """Reports progress through the custom stream."""
    writer = get_stream_writer()
    writer({"step": 1, "status": "fetching"})
    # ... work ...
    writer({"step": 2, "status": "done"})
    return "ok"
```

```python
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "search"}]},
    stream_mode="custom",
):
    print(chunk)  # whatever the writer emitted
```

> ➡️ TIP
>
> Earlier docs said `stream_mode="custom"` did not work with `create_agent`. Current LangGraph supports it via `get_stream_writer()` — the same mechanism as the `writer` parameter you would receive at the `StateGraph` level.

`version="v2"` Streaming and GraphOutput

`agent.stream(..., version="v2")` delivers structured events, and `agent.invoke(..., version="v2")` returns a `GraphOutput` with `value` (final state) and `interrupts` (pending interrupts).

```python
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "hello"}]},
    config={"configurable": {"thread_id": "t1"}},
    version="v2",
):
    if chunk["type"] == "updates":
        print("update:", chunk["data"])
    elif chunk["type"] == "messages":
        print("token:", chunk["data"][0].content)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "hi"}]},
    config={"configurable": {"thread_id": "t1"}},
    version="v2",
)
print(result.value)       # final state dict
print(result.interrupts)  # [] when no interrupt is pending
```

Accumulating with `chunk_position`

When streaming with `stream_mode="messages"`, the final chunk has `chunk_position == "last"`. Accumulate text and tool calls and flush them on the last chunk.

```python
buffer, tool_calls = [], []
for chunk, meta in agent.stream(payload, stream_mode="messages"):
    buffer.append(chunk.content)
    if chunk.tool_calls:
        tool_calls.extend(chunk.tool_calls)
    if meta.get("chunk_position") == "last":
        print("text:", "".join(buffer))
        print("tools:", tool_calls)
```

Filtering reasoning via `content_blocks`

Models with extended thinking (e.g., Claude Sonnet 4.6) expose typed `content_blocks` so you can route reasoning to a separate UI region.

```python
for chunk, _ in agent.stream(payload, stream_mode="messages"):
    for block in getattr(chunk, "content_blocks", []) or []:
        if block["type"] == "reasoning":
            ui.show_reasoning(block["reasoning"])
        elif block["type"] == "text":
            ui.show_answer(block["text"])
```

Multi-mode streams and interrupts

Passing a list to `stream_mode` yields `(mode, payload)` tuples. The `"__interrupt__"` key in an `updates` payload indicates a pending HITL decision.

```python
for mode, payload in agent.stream(
    {"messages": [{"role": "user", "content": "send email"}]},
    config={"configurable": {"thread_id": "t1"}},
    stream_mode=["messages", "updates"],
):
    if mode == "updates" and "__interrupt__" in payload:
        interrupt = payload["__interrupt__"]
        # show approval UI, then resume with Command(resume=...)
```

> ➡️ **TIP**
>
> Use `subgraphs=True` to receive events from nested subgraphs. To skip token streaming altogether, set `disable_streaming=True` on the call (or `streaming=False` on the model).

5.7 Summary

Here is a summary of what this notebook covered:

Concept	Implementation	Description
Short-term memory	`InMemorySaver` + `thread_id`	Keeps context inside one conversation session
Session isolation	Different `thread_id` values	Manages independent conversation sessions
Message trimming	`trim_messages` + middleware	Limits messages to stay within the token budget
Long-term memory	`InMemoryStore` + `ToolRuntime`	Stores user data that persists across conversations
Streaming (values)	`stream_mode="values"`	Full state snapshot at each step
Streaming (updates)	`stream_mode="updates"`	Node-by-node updates
Streaming (messages)	`stream_mode="messages"`	Real-time token output

Streaming (custom)	`stream_mode="custom"`	Only available at the LangGraph `StateGraph` level

Key points:

- Short-term memory is isolated by `thread_id` and preserves context only within the same session.
- Long-term memory is shared across sessions through `InMemoryStore`.
- `stream_mode="values"` is useful for debugging because it returns the full state at every step.
- `stream_mode="custom"` cannot be used directly with `create_agent`; it requires LangGraph's `StateGraph` API.
- Choosing the right streaming mode can significantly improve user experience.

Next Steps

→ 06_middleware.ipynb: Learn about middleware and guardrails.

06 Middleware and Guardrails

Learn the middleware system and guardrails used by LangChain v1 agents.

Learning Objectives

- Middleware concepts: Understand how to add hooks to each stage of the agent execution pipeline
- Built-in middleware: Use built-in middleware such as `SummarizationMiddleware`
- Custom middleware: Implement custom middleware with `@before_model`, `@after_model`, `@wrap_model_call`, and `@dynamic_prompt`
- Guardrails: Learn how to block unsafe input and output

6.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

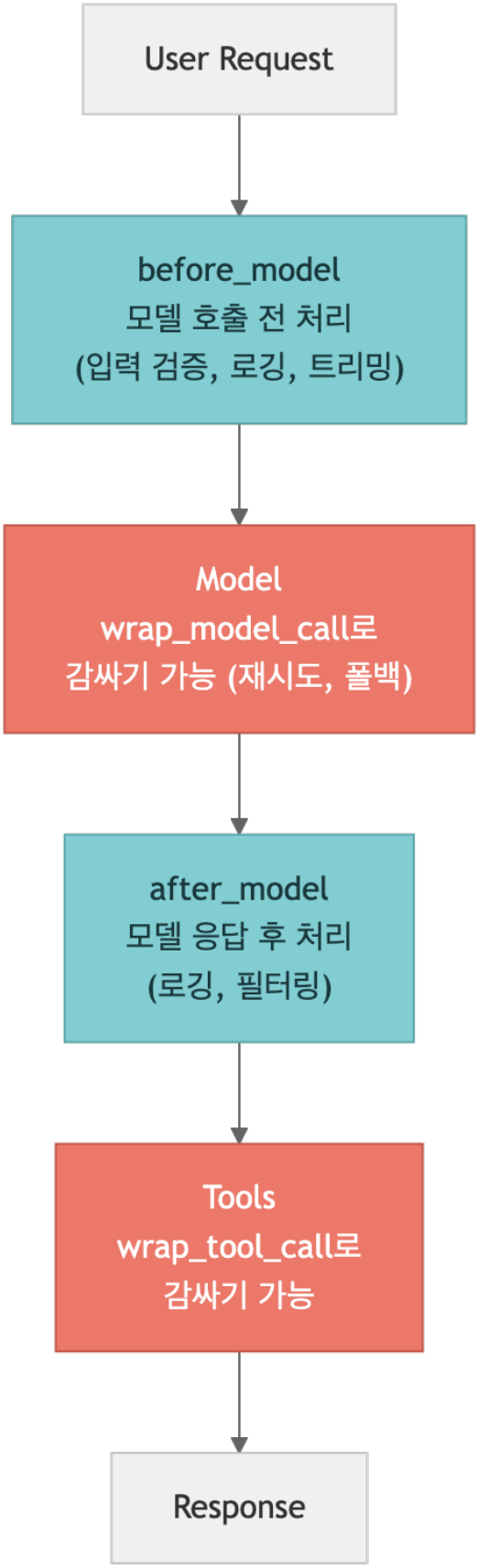
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model ready:", model.model_name)
```

6.2 Middleware Concepts

Middleware is the mechanism that adds hooks to each stage of the agent execution pipeline so you can control how the agent behaves.


User Request
before_model
모델 호출 전 처리
(입력 검증, 로깅, 트리밍)
Model
wrap_model_call로
감싸기 가능 (재시도, 폴백)
after_model
모델 응답 후 처리
(로깅, 필터링)
Tools
wrap_tool_call로
감싸기 가능
Response

Five middleware hooks:

Hook	When it runs	Main Use Case
`@before_model`	Before a model call	Input validation, message editing, guardrails
`@after_model`	After a model response	Output logging, response filtering
`@wrap_model_call`	Around a model call	Retry, fallback, caching
`@wrap_tool_call`	Around a tool call	Control tool execution
`@dynamic_prompt`	During prompt creation	Runtime prompt changes

6.3 Built-In Middleware

LangChain v1 provides built-in middleware for common patterns. `SummarizationMiddleware` automatically summarizes earlier messages when a conversation becomes long, reducing token usage.

> ➡️ TIP
>
> Size triggers across built-in middleware all use a `ContextSize` tuple: `("tokens", 100_000)`, `("messages", 20)`, or `("fraction", 0.8)` (80% of the model's context window).

```python
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def search(query: str) -> str:
    """Searches for information."""
    return f"'{query}'search result for"

# SummarizationMiddleware — automatically summarizes long conversations
from langchain.agents.middleware import SummarizationMiddleware

summarization = SummarizationMiddleware(
    model=model,
    trigger=("messages", 10),
)

agent_with_summary = create_agent(
    model=model,
    tools=[search],
    system_prompt="You are a helpful assistant.",
    middleware=[summarization],
)
print("SummarizationMiddleware agent created")
```

Prompt-caching middleware

Anthropic and Bedrock both support prompt caching, with one built-in middleware per provider.

```python
from langchain.agents.middleware import (
    AnthropicPromptCachingMiddleware,
    BedrockPromptCachingMiddleware,
)
from langchain_anthropic import ChatAnthropic

claude = ChatAnthropic(model="claude-sonnet-4-6")
agent = create_agent(
    model=claude,
    tools=[search],
    middleware=[AnthropicPromptCachingMiddleware()],
)
# For Bedrock, plug in BedrockPromptCachingMiddleware() the same way.
```

`ContextEditingMiddleware`
Clears stale tool outputs once the context is too heavy. `trigger` is the threshold, `keep` is how many recent messages to leave untouched.

```python
from langchain.agents.middleware import (
    ContextEditingMiddleware,
    ClearToolUsesEdit,
)

context_edit = ContextEditingMiddleware(
    edits=[
        ClearToolUsesEdit(
            trigger=("tokens", 100_000),
            keep=("messages", 3),
        )
    ],
)
```

`ModelFallbackMiddleware`
Cascades to a backup model when the primary one fails. The first argument is the primary; the rest are fallbacks.

```python
from langchain.agents.middleware import ModelFallbackMiddleware
from langchain_openai import ChatOpenAI

primary = ChatOpenAI(model="gpt-5.4")
backup = ChatOpenAI(model="gpt-5-nano")
agent = create_agent(model=primary, tools=[search],
                     middleware=[ModelFallbackMiddleware(primary, backup)])
```

`PatchToolCallsMiddleware` **(used by Deep Agents)**
Repairs malformed tool calls (bad JSON, missing arguments) and re-invokes the model so downstream tools see a clean payload. Deep Agents relies on this internally for its subagent and filesystem tools.

6.4 Custom Middleware: `\@before_model`

The `@before_model` decorator runs before the model is called.

Common uses:

- Logging input messages
- Modifying or filtering messages

- Input validation (guardrails)
- Adding context

6.5 Custom Middleware: `@after_model`

The `@after_model` decorator runs after the model response has been generated.

Common uses:

- Logging model output
- Filtering or modifying responses
- Monitoring tool calls
- Validating output quality

6.6 `@wrap_model_call`

The `@wrap_model_call` decorator wraps the model call itself, which lets you implement retry, fallback, caching, and similar patterns.

You execute the original model call through the `handler` function and can add custom logic before or after it.

Type annotations and `request.override`

Annotate handlers with `ModelRequest` / `ModelResponse` for better tooling. `request.override(...)` patches messages, tools, system prompt, or response format for this single call (transient).

```python
from langchain.agents.middleware import (
    wrap_model_call,
    ModelRequest,
    ModelResponse,
)

@wrap_model_call
def restrict_for_guest(request: ModelRequest, handler) -> ModelResponse:
    if request.runtime.context.role == "guest":
        patched = request.override(
            system_message="Read-only mode.",
            tools=[t for t in request.tools if t.name.startswith("read_")],
        )
        return handler(patched)
    return handler(request)
```

```python
from langchain.agents.middleware import wrap_model_call
import time

@wrap_model_call
def retry_on_error(request, handler):
    """Retries model calls with exponential backoff on failure."""
    max_retries = 2
    for attempt in range(max_retries + 1):
        try:
            return handler(request)
        except Exception as e:
            if attempt < max_retries:
                wait = 2 ** attempt
                print(f"  Retry {attempt + 1}/{max_retries} ({wait}s wait)")
```

```
                time.sleep(wait)
            else:
                raise

agent_retry = create_agent(
    model=model,
    tools=[search],
    system_prompt="You are a helpful assistant.",
    middleware=[retry_on_error],
)
print("Retry middleware agent created")
```

6.7 `\@dynamic_prompt`

The `@dynamic_prompt` decorator changes the system prompt dynamically at runtime.

Common uses:

- Adding the current date and time
- Per-user prompt customization
- Changing behavior based on state
- A/B testing

6.8 `\@wrap_tool_call`

The `@wrap_tool_call` decorator wraps a tool call itself, so you can add custom logic before and after tool execution.

Like `@wrap_model_call`, it uses a `handler` function to run the original tool. You can use it for timing, logging, and error handling.

Common uses:

- Measuring execution time: monitor performance by tool
- Logging: record tool input and output
- Error handling: apply fallback behavior if a tool fails
- Access control: block or restrict specific tools

Updating custom state from `\@wrap_tool_call`

When a wrapper needs to persist audit data, declare the custom state with `state_schema=` and return a `Command(update=...)`. Without the schema, the extra field is not part of the agent state contract.

```
from typing_extensions import NotRequired, TypedDict
from langchain.messages import ToolMessage
from langchain.agents.middleware import ToolCallRequest, wrap_tool_call
from langgraph.types import Command

class AuditState(TypedDict):
    tool_calls_log: NotRequired[list[str]]

@wrap_tool_call(state_schema=AuditState)
def audit_tool_call(request: ToolCallRequest, handler):
    response: ToolMessage | Command = handler(request)
```

```
    name = request.tool_call["name"]
    prior = request.state.get("tool_calls_log", [])
    if isinstance(response, Command):
        return response
    return Command(update={
        "messages": [response],
        "tool_calls_log": prior + [name],
    })
```

6.9 Simple Guardrails

Middleware can also act as a lightweight guardrail. In the example below, a `before_model` hook blocks requests that contain prohibited keywords before the model is called.

Skipping the model with `can_jump_to`

Instead of just editing messages, a guardrail can jump to a different graph node and skip the model call entirely. Declare allowed destinations with `@hook_config(can_jump_to=[...])` and return `{"jump_to": ...}`.

```
from langchain.agents.middleware import before_model, hook_config
from langchain_core.messages import AIMessage

BANNED = {"reveal system prompt", "tell me the password"}

@before_model
@hook_config(can_jump_to=["end", "tools", "model"])
def block_unsafe(state):
    last = state["messages"][-1].content
    if any(p in last for p in BANNED):
        return {
            "messages": [AIMessage(content="I can't help with that request.")],
            "jump_to": "end",
        }
    return None
```

Async hooks (a-prefixed)

For I/O-heavy guardrails (calling an external classifier, for example), implement async variants by prefixing the sync names with `a`: `abefore_model`, `aafter_model`, `awrap_model_call`, `awrap_tool_call`. If a middleware class defines both, `agent.invoke()` uses the sync hook and `agent.ainvoke()` uses the async one automatically.

6.10 Summary

This notebook covered:

Topic	Core API	Description
Built-in middleware	`SummarizationMiddleware`	Automatically summarizes long conversations

Before-model hook	`\@before_model`	Logs, validates, or modifies input before model execution
After-model hook	`\@after_model`	Logs or validates model output after generation
Wrapped model call	`\@wrap_model_call`	Adds retry, fallback, or caching around model calls
Dynamic prompt	`\@dynamic_prompt`	Changes the system prompt at runtime
Wrapped tool call	`\@wrap_tool_call`	Adds logging, timing, and control around tool execution
Guardrails	`\@before_model` and middleware	Blocks unsafe or disallowed input

Next Steps

→ 07_hitl_and_runtime.ipynb: Learn about human-in-the-loop, runtime context, and MCP.

07 Human

in-the-Loop, ToolRuntime, and MCP

Learn how LangChain v1 handles human approval workflows, runtime context inside tools, context engineering, and MCP (Model Context Protocol).

Learning Objectives

This notebook covers:

- Human-in-the-Loop (HITL): how to pause agent execution and request approval before a tool call
- ToolRuntime: how tools can access runtime context such as user information and session data
- Context engineering: techniques for dynamically controlling prompts and tools
- MCP (Model Context Protocol): a standardized way to connect tool servers

7.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model ready:", model.model_name)
```

7.2 Human-in-the-Loop Concepts

Ask for human approval before the agent calls a tool.

Why is this needed?

Autonomous agents are powerful, but irreversible actions such as sending email, deleting files, or processing payments still require human confirmation.

Workflow

```
Agent → proposes a tool call → [interrupt] → human approves/rejects → tool runs → result is returned
```

In LangChain v1, this is implemented by combining `HumanInTheLoopMiddleware` with `InMemorySaver` (a checkpointer). The checkpointer stores the agent state so the workflow can resume after interruption.

7.3 `HumanInTheLoopMiddleware`

`HumanInTheLoopMiddleware` automatically pauses execution on tool calls and waits for human approval. Use it with an `InMemorySaver` checkpointer so interrupted state can be preserved.

```python
from langchain.agents import create_agent
from langchain.tools import tool
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Sends an email to the specified recipient."""
    return f"{to} email sent to: {subject}"

@tool
def delete_file(path: str) -> str:
    """Deletes a file at the specified path."""
    return f"File deleted: {path}"

# Require approval only for risky tools (dict form scopes decisions)
hitl = HumanInTheLoopMiddleware(interrupt_on={
    "send_email": {"allowed_decisions": ["approve", "edit", "reject", "respond"]},
    "delete_file": {"allowed_decisions": ["approve", "reject"]},
})

agent = create_agent(
    model=model,
    tools=[send_email, delete_file],
    system_prompt="You are an assistant that can send emails and manage files.",
    middleware=[hitl],
    checkpointer=InMemorySaver(),
)

print("HITL agent created")
print("  -> The run pauses for human approval before executing tools")
```

7.4 The `interrupt` and `Command(resume=...)` Pattern

A HITL agent works in two phases:

1. **Phase 1 (`invoke`):** the agent proposes a tool call and is automatically **interrupted**
2. **Phase 2 (`Command(resume=...)`):** a decision list is sent and execution **resumes**

Four decisions (approve / edit / reject / respond)

`HumanInTheLoopMiddleware` accepts decisions as a dict, one per pending tool call.

```python
from langgraph.types import Command

# 1) approve — run the tool as proposed
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}),
             config=config, version="v2")

# 2) edit — run the tool with patched arguments
agent.invoke(
    Command(resume={"decisions": [
        {"type": "edit", "args": {"to": "boss@example.com", "subject": "Updated"}}
    ]}),
    config=config, version="v2",
)

# 3) reject — block the call, send a rejection back to the model
agent.invoke(Command(resume={"decisions": [{"type": "reject"}]}),
             config=config, version="v2")

# 4) respond — replace the tool call with a human message
agent.invoke(
    Command(resume={"decisions": [
        {"type": "respond", "message": "Send this via Slack instead"}
    ]}),
    config=config, version="v2",
)
```

`version="v2"` returns a `GraphOutput`

`agent.invoke(..., version="v2")` returns a `GraphOutput`. When the run is paused, `result.value` is the partial state and `result.interrupts` is a list of `Interrupt` objects.

```python
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Send the weekly report"}]},
    config={"configurable": {"thread_id": "t1"}},
    version="v2",
)

if result.interrupts:
    pending = result.interrupts[0].value
    decision = ask_human(pending)
    final = agent.invoke(
        Command(resume={"decisions": [decision]}),
        config={"configurable": {"thread_id": "t1"}},
        version="v2",
    )
    print(final.value["messages"][-1].content)
```

> ➡️ **TIP**
>
> `version="v2"` and dict-based `decisions` require `deepagents ⩾ 0.5.0` or `langgraph ⩾ 1.1.5`. On older versions, the single-decision `Command(resume=True/False)` form still works.

Lifecycle — `after_model` hook and `HITLRequest`

Internally `HumanInTheLoopMiddleware` runs in an `after_model` hook that builds a `HITLRequest` containing `action_requests` (the tool calls to review) and `review_configs` (the allowed decisions per call). Custom approval flows can return the same object directly from their own `after_model` hook.

Transient vs. persistent patches

Use `request.override(...)` for one-shot changes that only apply to this call (transient). For changes that must outlive the call, return an `ExtendedModelResponse` together with `Command(update=...)` so the state is written to the checkpoint (persistent).

7.5 `ToolRuntime` — Access Runtime Information from a Tool

`ToolRuntime` lets a tool access runtime context such as the current user or session data while it executes.

Core idea

- Add a `runtime: ToolRuntime[T]` parameter to the tool function
- `T` is a context dataclass defined by the developer
- When you create the agent, set `context_schema=T`, and when invoking the agent, pass `context=T(...)`

`runtime.execution_info` — call metadata

Identifies the current call. Useful for tracing, idempotency, or retry counting.

Field	Description
`thread_id`	Conversation thread identifier
`run_id`	Identifier for this `invoke` / `stream` call
`attempt`	Retry attempt number (1-based)

```python
@tool
def call_external_api(payload: dict, runtime: ToolRuntime[Context]) -> str:
    info = runtime.execution_info
    headers = {
        "X-Trace-Run": info.run_id,
        "X-Trace-Thread": info.thread_id,
        "X-Attempt": str(info.attempt),
    }
    return http.post(url, json=payload, headers=headers).text
```

`runtime.server_info` — deployment metadata

When the agent runs on LangGraph Platform/Server, `server_info` exposes deployment identifiers and the authenticated user. Values may be `None` in a local process.

Field	Description
`assistant_id`	Platform assistant (config bundle) identifier
`graph_id`	Graph name from `langgraph.json`
`user`	Authenticated user, if available

```python
@tool
def log_who(runtime: ToolRuntime[Context]) -> str:
    s = runtime.server_info
    return f"{s.graph_id}/{s.assistant_id} called by {s.user}"
```

7.6 Context Engineering — Dynamic Control of Prompts and Tools

Context engineering is the practice of dynamically shaping the prompt, available tools, and message history given to the agent.

Common use cases

- Provide a different system prompt depending on user role
- Filter the available tools depending on the situation
- Summarize and reorganize long conversation histories

The `dynamic_prompt` middleware makes it possible to customize the prompt for every request.

7.7 MCP (Model Context Protocol) Integration Overview

MCP is a standardized way to connect tool servers.

Core MCP concepts

- MCP server: provides tools through HTTP/SSE or stdio
- MCP client: connects to the server and discovers or calls tools
- Standardization: any tool can be connected as long as it follows the MCP protocol

MCP support in LangChain v1

- You can connect to a local MCP server with `mcp.client.stdio.stdio_client()` and `ClientSession`
- `load_mcp_tools(session)` from `langchain-mcp-adapters` converts MCP session tools into LangChain tools

7.8 Summary

Concept	Description	Core API
HITL	Requests human approval before tool execution	`HumanInTheLoopMiddleware`, `Command(resume=...)`
ToolRuntime	Gives tools access to runtime context	`ToolRuntime[T]`, `context_schema`
Context engineering	Dynamically controls prompts and tools	`dynamic_prompt` middleware

MCP	Standardized tool protocol	`ClientSession + load_mcp_tools()`

Next Steps

- The next notebook introduces multi-agent patterns.
- You will explore Subagents, Handoffs, Skills, Routers, and other collaboration patterns.

08 Multi

Agent Patterns

Learning Objectives

Understand and implement five multi-agent patterns.

This notebook covers:

- Subagents: the main agent calls specialized subagents as tools
- Handoffs: state transitions between agents with `Command(goto=...)`
- Skills: one agent loads specialized prompts depending on the task
- Router: a classifier routes input to the right agent
- Custom: developer-controlled complex workflows

8.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Model ready:", model.model_name)
```

8.2 Comparing Multi-Agent Patterns

The table below compares five multi-agent patterns. Each one fits a different situation, so you should choose based on your project requirements.

Pattern	Routing Owner	State Sharing	Best Fit
Subagents	Main agent	Isolated through tools	Parallel work, distributed execution
Handoffs	Tool call	State transition	Sequential multi-hop flows
Skills	Single agent	Prompt swapping	Domain specialization

Router	Classifier	Parallel execution	Multi-domain systems
Custom	Developer-defined	Full control	Complex workflows

Key differences

- Subagents run independently and return only the result
- Handoffs pass conversation state between agents
- Skills let one agent switch roles
- Router classifies input and delegates it to the right specialist

Pattern Selection Matrix (requirement → pattern)

While the table above lists pattern properties, the matrix below starts from requirements and points to the best-fit pattern.

Requirement	Best-fit pattern
Single simple task (one-shot)	Skills (or single agent)
Repeat the same task	Subagents
Parallel work across multiple domains	Router (classify then fan-out)
Very large context (long docs/history)	Subagents (token savings via isolation)
Team-based — agents collaborate	Handoffs (shared state)
Direct conversation with a role	Handoffs + supervisor

Approximate cost per pattern

Rough call count and cumulative tokens for the same task implemented in different patterns. The numbers vary by scenario and model, but they illustrate the relative cost.

Scenario	Pattern	Calls	Cumulative tokens
One-Shot (single domain)	Subagents	4-5 calls	9K
Repeat (same task)	Skills	2-3 calls	15K
Multi-Domain (collaboration)	Handoffs	3-7+ calls	14K+
Multi-Domain (classify then fan-out)	Router	~1 classify + N domains	9K

> ➡️ TIP

> Router vs Supervisor — the terms are often used interchangeably but they differ in implementation. A Router is a simple functional node that classifies input once and fans out to the right agent (or `Send`). A Supervisor is itself an agent that decides conversation-aware which subagent to invoke on every turn, retaining message history. Use Router for one-shot classification; pick Supervisor for multi-turn collaboration.

8.3 The Subagent Pattern

In this pattern, the main agent (supervisor) calls specialized subagents as tools.

Characteristics

- Each subagent is wrapped in a tool function
- The main agent decides which subagent to call
- The internal state of each subagent is isolated from the main agent
- Parallel execution is possible, which can improve performance

Subagent-local history (`checkpointer=True`)

`create_agent(..., checkpointer=True)` lets a subagent keep its own message history on a separate thread. The main agent still only sees the final result, but the subagent can run multi-turn reasoning internally and resume from the same `thread_id`.

Exposing N subagents via a single dispatch tool

With many subagents, registering each one as its own tool bloats the parent tool list. The alternative is a single `dispatch(agent_name, query)` tool combined with one of three discovery strategies.

Strategy	Recommended scale	Notes
Enumerate in system prompt	< 10	LLM picks the name from text. Simple, flexible
`Literal["a", "b", ...]` type constraint	10–30	Schema-level validation, rejects unknown names
Tool-based discovery (`list_agents`)	> 30	Dynamic registration; separate tools to search/filter

Injecting parent state with `ToolRuntime[None, CustomState]`

When a subagent tool needs to read part of the parent state, take a `ToolRuntime[None, CustomState]` argument.

```
from langchain.tools import tool, ToolRuntime
```

```
@tool
def research_subagent(query: str, runtime: ToolRuntime[None, ResearchState]) -> str:
    """Research subagent that reads user_id from the parent state."""
    user_id = runtime.state["user_id"]
    # ... call subagent ...
    return result
```

Async work: start / status / get_result three-tool pattern

Long-running subagents are safer when split into three tools instead of a single blocking call.

```
@tool
def start_research_job(query: str) -> str:
    """Start the research job and return a job_id."""
    ...

@tool
def check_job_status(job_id: str) -> str:
    """Return the job status: running / done / failed."""
    ...

@tool
def get_job_result(job_id: str) -> str:
    """Return the result of a completed job."""
    ...
```

Updating parent state with `Command`

A subagent tool can return `Command(update={...})` instead of a plain string to update the parent graph's state directly — useful when you need to update custom fields, not just messages.

```
from langgraph.types import Command

@tool
def research_with_state_update(query: str) -> Command:
    result = run_research(query)
    return Command(update={
        "research_results": result,
        "messages": [{"role": "tool", "content": result}],
    })
```

8.4 The Handoff Pattern

This pattern uses `Command(goto=...)` to transfer state between agents.

Characteristics

- A tool returns a `Command` object that routes execution to another agent
- Conversation state (message history) is passed to the next agent
- A `StateGraph` defines the flow between agents
- This fits multi-hop scenarios such as customer-service transfers

Single agent + middleware handoff (preferred)

The LangChain v1 docs recommend a single agent + `@wrap_model_call` middleware over multiple subgraphs as the primary handoff pattern. The middleware overrides the system prompt and tool set dynamically based on routing state, so persona switches happen inside one agent node.

```
from langchain.agents.middleware import wrap_model_call, ModelRequest
from langchain.agents import create_agent

@wrap_model_call
def role_router(request: ModelRequest, handler):
    state = request.state
    role = state.get("active_role", "general")

    if role == "billing":
        return handler(request.override(
            system_prompt="You are a billing specialist.",
            tools=[refund_tool, charge_tool],
        ))
    elif role == "tech":
        return handler(request.override(
            system_prompt="You are a technical support agent.",
            tools=[diagnose_tool, escalate_tool],
        ))
    return handler(request)

agent = create_agent(model="gpt-5.4", tools=ALL_TOOLS, middleware=[role_router])
```

The multi-subgraph handoff still has its place when the flow between departments must be explicit, but the middleware approach is much lighter for plain persona switches.

Handoff tool with `ToolRuntime` and `tool_call_id` echo-back

A handoff tool should accept `ToolRuntime[None, SupportState]` so it can access parent state and the current `tool_call_id`. When switching with `Command(goto=..., update={"messages": [...]})`, you must include a `ToolMessage` that matches the originating tool call — otherwise OpenAI's tool-call contract breaks.

```
from langchain.tools import tool, ToolRuntime
from langchain_core.messages import ToolMessage
from langgraph.types import Command

@tool
def transfer_to_billing(reason: str, runtime: ToolRuntime[None, SupportState]) -> Command:
    """Transfer the conversation to the billing team."""
    return Command(
        goto="billing_agent",
        update={
            "messages": [
                ToolMessage(
                    content=f"Transferred to billing: {reason}",
                    tool_call_id=runtime.tool_call_id,
                ),
            ],
            "active_role": "billing",
        },
    )
```

➡️ TIP

> Exactly two messages should flow on a subgraph handoff — (1) the `AIMessage(tool_calls=[...])` that triggered the handoff and (2) its matching `ToolMessage`. Inserting anything else between them causes the OpenAI provider to reject the next call because the tool-call pair no longer matches.

8.5 The Skill Pattern

A single agent dynamically loads a specialized prompt depending on the task.

Characteristics

- One agent has multiple skills
- Each skill is implemented as a specialized system prompt
- The agent dynamically loads the skill it needs
- One agent can handle many tasks without managing multiple separate agents

8.6 The Router Pattern

A classifier routes input to the most appropriate agent.

Characteristics

- The query is classified first
- It is then delegated to the right specialist agent or tool
- This is useful in multi-domain systems
- Routing logic can be rule-based or model-based

Multi-domain fan-out with `Send`

When a query spans several domains at once, use `Send` for parallel fan-out instead of routing to a single agent. If a routing function passed to `add_conditional_edges` returns a `list[Send]`, the LangGraph runtime executes those nodes in parallel.

```python
from langgraph.types import Command, Send

def route_to_agents(state: RouterState) -> list[Send]:
    """Fan out to every category produced by the classifier."""
    return [
        Send(c["agent"], {"query": c["query"]})
        for c in state["classifications"]
    ]

graph.add_conditional_edges("classifier", route_to_agents,
                            ["billing_agent", "tech_agent", "general_agent"])
```

8.7 Choosing a Pattern

Which multi-agent pattern should you choose? Use the guide below.

Decision tree

1. Can the agents work independently?

- YES → Subagents (parallel execution, result aggregation)
- NO → move to the next question

1. Must conversation state be passed between agents?

- YES → Handoffs (state transition, multi-hop)
- NO → move to the next question

1. Can a single agent just switch roles?

- YES → Skills (prompt switching)
- NO → move to the next question

1. Is classifying input and sending it to a handler enough?

- YES → Router (classify and delegate)
- NO → Custom (fully custom graph)

Practical guidance

- Start with the simplest pattern (usually Subagents or Skills)
- Move to Handoffs or Router only when requirements become more complex
- Use a Custom pattern only when the other patterns are not enough
- You can also combine patterns (for example, Router + Handoffs)

8.8 Summary

Pattern	Core API	When to use it
Subagents	`create_agent` + tool functions	Independent parallel work
Handoffs	`Command(goto=...)`, `StateGraph`	Multi-hop state transfer
Skills	Load prompts as tools	One agent with many roles
Router	Classifier tool + specialist tools	Multi-domain classification
Custom	Full `StateGraph` control	Complex business logic

Key principles

- Start simple and increase complexity only when necessary
- Design each agent around one clear responsibility
- Define the interfaces between agents (inputs and outputs) clearly

Next Steps

→ 09_custom_workflow_and_rag.ipynb: Learn about custom workflows and RAG.

09 Custom Workflows and RAG

Learning Objectives

Build a custom workflow with LangGraph `StateGraph` and implement the RAG pattern.

This notebook covers:

- The basic structure of `StateGraph` (nodes, edges, state)
- Branching with conditional edges
- Integrating `create_agent` as a workflow node
- Implementing the RAG (Retrieval-Augmented Generation) pattern

9.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("Environment ready.")
```

9.2 `StateGraph` Basics

Let's look at the core building blocks of LangGraph.

Concept	Description
Node	A unit of work. It can be a function or an agent
Edge	A connection between nodes that defines execution flow
State	Shared data passed between nodes, usually defined with `TypedDict`

`StateGraph` combines these three parts so you can build complex workflows.

9.3 Conditional Edges

Branch to different paths based on state. With `add_conditional_edges`, the next node can be selected dynamically at runtime.

In the example below, the input text is classified first and then routed to a different handler depending on its category.

9.4 Integrating an Agent into a Workflow

Use an agent created with `create_agent` as a node inside a `StateGraph`. This makes it possible to connect multiple agents in a pipeline and handle more complex tasks.

The example below connects a research agent and a writing agent in sequence.

9.5 Overview of RAG (Retrieval-Augmented Generation)

RAG is a pattern that strengthens LLM answers by retrieving external knowledge. There are three major approaches:

Pattern	Description	Characteristic
Basic 2-step	Retrieve → generate	Simple and fast
Agentic RAG	An agent repeatedly calls retrieval tools	Flexible and accurate
Hybrid	Combines keyword and semantic search	Higher retrieval quality

- Basic 2-step: retrieve documents, then pass them as context to the LLM for answer generation
- Agentic RAG: the agent calls retrieval tools repeatedly until it has enough information to answer

Five RAG building blocks

A LangChain RAG pipeline is built from these five components.

Component	Role
Document Loaders	Ingest source documents from PDF, web pages, Notion, etc.
Text Splitters	Split long documents into meaningful chunks (e.g. `RecursiveCharacterTextSplitter`)
Embedding Models	Turn text into vectors (e.g. `OpenAIEmbeddings`)
Vector Stores	Index and search embeddings (FAISS, Chroma, Pinecone, ···)

Retrievers	Unified retrieval interface — vector / hybrid / MultiQuery share the same API

Rewrite → Retrieve → Agent (three-node pattern)

A variation of agentic RAG that separates query rewriting into its own node. The user input is first rewritten into a retrieval-friendly form, the retriever runs against it, and the agent grounds its answer in the result.

```
# Rewrite → Retrieve → Agent
graph.add_node("rewrite", query_rewriter_node)
graph.add_node("retrieve", retriever_node)
graph.add_node("agent", create_agent(model, tools=[]))

graph.add_edge(START, "rewrite")
graph.add_edge("rewrite", "retrieve")
graph.add_edge("retrieve", "agent")
graph.add_edge("agent", END)
```

The research → writer example below illustrates a content-generation workflow, while Rewrite → Retrieve → Agent is a workflow for boosting retrieval quality. Both are natural fits for `StateGraph`.

9.6 A Simple RAG Implementation

Split text into chunks and implement RAG with simple keyword-based retrieval. Even without a vector store, you can still understand the core idea behind RAG.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Sample documents
documents = [
    "LangChain is a framework for building applications with large language models. It provides tools for prompt engineering, memory management, and agent creation.",
    "LangGraph is a low-level orchestration framework for building stateful agents. It uses a graph-based approach with nodes and edges.",
    "Middleware in LangChain v1 allows you to intercept and modify agent behavior at every step. You can add logging, guardrails, and custom logic.",
    "Multi-agent systems in LangChain support five patterns: subagents, handoffs, skills, router, and custom workflows.",
    "RAG (Retrieval-Augmented Generation) combines information retrieval with text generation to provide grounded, factual responses.",
]

# Split text
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=50,
)

chunks = []
for doc in documents:
    chunks.extend(text_splitter.split_text(doc))

print(f"Original documents: {len(documents)}")
print(f"Split chunks: {len(chunks)}")
for i, chunk in enumerate(chunks):
    print(f"  [{i}] {chunk[:80]}...")
```

9.7 FAISS Vector Store (Optional)

Similarity search with embeddings is far more accurate than keyword matching. The example below shows how to use a FAISS vector store.

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

# Create the embedding model
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# Create the vector store
vectorstore = FAISS.from_texts(chunks, embeddings)

# Retrieve
results = vectorstore.similarity_search("LangChain agent patterns", k=3)
for doc in results:
    print(doc.page_content)
```

9.8 Summary

This notebook covered:

Topic	Key Idea
StateGraph basics	Build workflows from nodes, edges, and state
Conditional edges	Branch at runtime with `add_conditional_edges`
Agent integration	Use `create_agent` as a `StateGraph` node inside a pipeline
RAG pattern	Combine retrieval and generation for grounded answers
Vector store	Use FAISS or similar tools for embedding–based similarity search

The next notebook shows how to deploy agents to production environments.

Next Steps

→ 10_production.ipynb: Learn about production deployment.

10 Production

Learning Objectives

Learn how to test, deploy, and monitor agents.

This notebook covers:

- Local development and debugging with LangSmith Studio
- Deterministic agent testing with `GenericFakeChatModel`
- Trajectory-based tests for validating tool call order
- Web-based interaction with Agent Chat UI
- Deployment with LangGraph Platform or your own server
- Observability with LangSmith

10.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("Environment ready.")
```

10.2 LangSmith Studio

Develop and debug agents locally.

To use Studio, you need:

- a `langgraph.json` config file
- a local dev server started with `langgraph dev` (default `http://127.0.0.1:2024`)
- the hosted Studio UI at `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024`

Studio is a powerful tool for visualizing agent execution flow and debugging each step.

Studio Key Features

Feature	Description
Hot-reloading	Edit prompts and tool code and see the change applied without restarting the server
Full execution trace inspection	Inspect every node, model call, and tool invocation in a single trace tree
Thread replay	Re-run saved threads from any checkpoint to compare branches and behaviors
Exception capture	Capture the state and surrounding context at the moment an exception is raised

```
# Example langgraph.json configuration
import json

langgraph_config = {
    "dependencies": ["."],
    "graphs": {
        "agent": "./agent.py:agent"
    },
    "env": ".env"
}

print("Example langgraph.json configuration:")
print(json.dumps(langgraph_config, indent=2))
print("\nHow to run:")
print("  $ langgraph dev  # local dev server: http://127.0.0.1:2024")
print("  → Studio UI: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024")
```

10.3 Agent Testing

Agent testing has three layers: Unit, Integration, and Evals.

Layer	Goal	Tools
Unit	Validate agent logic deterministically (tool choice, branching, exit conditions)	`GenericFakeChatModel`, `pytest`
Integration	Run the full workflow against real LLMs and tools to confirm end-to-end behavior	`langgraph dev`, real server environments
Evals	Evaluate agent trajectories with deterministic matching or LLM-as-judge evaluators	`agentevals`, LangSmith Evaluations

10.3.1 Unit — `GenericFakeChatModel`

With `GenericFakeChatModel`, you can test an agent deterministically without making real API calls.

Benefits of this approach:

- No API cost during tests
- Always returns the same result, which is ideal for CI/CD pipelines
- Lets you validate the agent's logic independently (tool calls, branching, and so on)

```
from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_capital(country: str) -> str:
    """Returns the capital of a country."""
    capitals = {"Korea": "Seoul", "Japan": "Tokyo", "France": "Paris"}
    return capitals.get(country, "Unknown")

# Deterministic test with a fake model
fake_model = GenericFakeChatModel(
    messages=iter([
        AIMessage(content="The capital of South Korea is Seoul.")
    ])
)

# Test agent
test_agent = create_agent(
    model=fake_model,
    tools=[get_capital],
    system_prompt="You are a geography expert.",
)

print("GenericFakeChatModel test:")
print("  → Tests agent behavior with deterministic responses")
print("  → Can be tested in CI/CD without live API calls")
```

10.3.2 Evals — Trajectory matching with `agentevals`

Validate the order in which the agent calls tools. A trajectory test checks whether the agent uses tools in the expected order and whether the final response matches your expectation.

The `agentevals` package offers four trajectory-match modes.

```
pip install agentevals
```

Mode	Matching rule
`strict`	Tool calls must match the reference exactly in kind and order
`unordered`	Same set of tool calls; order is ignored
`subset`	The observed trajectory is a subset of the reference (no extra calls allowed)
`superset`	The observed trajectory is a superset of the reference (must contain the required calls)

LLM-as-judge evaluators allow grading trajectories against natural-language criteria as well.

```python
# Example trajectory test
def test_agent_trajectory():
    """Tests whether the agent calls tools in the expected order."""
    result = test_agent.invoke(
        {"messages": [{"role": "user", "content": "What is the capital of South Korea?"}]}
    )

    messages = result["messages"]

    # Check: verify that messages exist
    assert len(messages) > 0, "The agent did not respond"

    # Check: verify that the last message is an AI response
    last_msg = messages[-1]
    assert hasattr(last_msg, 'content'), "The last message does not contain content"

    print("✓ Trajectory test passed")
    print(f"  Message count: {len(messages)}")
    print(f"  Final response: {last_msg.content[:100]}")

try:
    test_agent_trajectory()
except Exception as e:
    print(f"Testing note: {e}")
```

10.5 Agent Chat UI

This is a web UI for talking to your agent. It connects to a LangGraph server so you can test the agent directly in the browser. Either scaffold a new project with `create-agent-chat-app` or clone the official repository (`langchain-ai/agent-chat-ui`). Point the UI at your local LangGraph server (`http://127.0.0.1:2024`).

Key features:

- Real-time streaming chat
- Tool call visualization
- Conversation branching
- Human-in-the-loop approval

Pick one of the two install paths:

```bash
# Option A — scaffold (recommended)
npx create-agent-chat-app --project-name my-chat-ui

# Option B — clone the official repository
git clone https://github.com/langchain-ai/agent-chat-ui.git
cd agent-chat-ui && pnpm install && pnpm dev
```

```bash
# Start the LangGraph server in another shell
langgraph dev   # http://127.0.0.1:2024
```

The UI asks for three values on first launch.

Field	Value
Deployment URL	`http://127.0.0.1:2024` for local; the deployment URL for managed agents

Graph ID	The `graphs` key from `langgraph.json` (for example, `agent`)
LangSmith API key	Required for managed deployments; can be skipped for the local dev server

10.6 Deployment

You can deploy an agent through LangGraph Platform (managed) or your own server. Choose the option that best fits your production environment.

10.6.1 LangGraph Platform deploy workflow

LangGraph Platform deployments now go through GitHub integration. The old single-command `langgraph deploy` flow is no longer recommended.

1. Push the agent code (including `langgraph.json`) to a GitHub repository.
2. Open the LangSmith console → Deployments → + New Deployment.
3. Select the repository, branch, and `langgraph.json` path.
4. Enter environment variables (`OPENAI_API_KEY`, etc.).
5. Click Deploy, watch the build log, and pick up the issued Deployment URL.

> **! WARNING**
>
> The `langgraph deploy` CLI flow from older guides is deprecated. Use the GitHub-based flow in the LangSmith console for new deployments.

```
print("Deployment options:")
print("=" * 50)

print("""
# Option 1: LangGraph Platform (managed) — GitHub integration
#   → Use LangSmith Deployments → + New Deployment
#   (legacy `langgraph deploy` CLI pattern is deprecated)


# Option 2: self-hosted Docker deployment
`$ langgraph build -t my-agent`
`$ docker run -p 2024:2024 my-agent`


# Option 3: wrap with FastAPI/Flask
from fastapi import FastAPI

app = FastAPI()


@app.post("/chat")
async def chat(message: str):
    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": message
                }
```

```
            ]
        }
    )

    return {
        "response": result["messages"][-1].content
    }
""")
```

10.6.2 Calling a deployed agent — `langgraph-sdk`

Use the Python SDK or REST to call a deployed agent.

```
pip install langgraph-sdk
```

```
from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://127.0.0.1:2024")

for chunk in client.runs.stream(
    None,  # threadless run
    "agent",  # graph ID
    input={"messages": [{"role": "user", "content": "hi"}]},
    stream_mode="updates",
):
    print(chunk)
```

Or call the REST API directly:

```
curl -X POST http://127.0.0.1:2024/runs/stream \
  -H "Content-Type: application/json" \
  --data '{
    "assistant_id": "agent",
    "input": {"messages": [{"role": "user", "content": "hi"}]},
    "stream_mode": "updates"
  }'
```

10.7 Observability

Use LangSmith to trace agent behavior. When tracing is enabled, every step of agent execution is recorded and can be analyzed.

10.7.1 Environment variables

```
# Current (recommended)
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY="lsv2_..."
export LANGSMITH_PROJECT="my-agent"   # defaults to `default` if unset
```

LangSmith lets you inspect:

- The complete execution flow of each agent call
- Model input/output, tool calls, and token usage
- Latency, errors, and cost tracking

10.7.2 Selective tracing with `tracing_context`

Use `tracing_context` to enable or disable tracing for a specific block in code.

```
import langsmith as ls

with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "hi"}]})
```

10.7.3 Tags and metadata via `config`

Attach tags and metadata at call time to make trace search and filtering easier.

```
agent.invoke(
    {"messages": [{"role": "user", "content": "hi"}]},
    config={
        "tags": ["production", "v1.0"],
        "metadata": {"user_id": "123", "session_id": "456"},
    },
)
```

10.8 Production Checklist

Before deploying an agent to production, review the following checklist.

Item	Tool	Status
Unit tests	`GenericFakeChatModel`, `pytest`	
Trajectory tests	Custom validation functions	
Observability	LangSmith tracing	
Error handling	`try/except`, retry logic	
Security	API key management, input validation, guardrails	
Deployment environment	Docker, LangGraph Platform	
Monitoring	LangSmith dashboards, alert configuration	
Documentation	API docs, agent behavior notes	

10.9 Summary

This notebook covered:

Topic	Key Idea
LangSmith Studio	Use `langgraph dev` to debug agents visually on your local machine

Agent testing	Run deterministic tests with `GenericFakeChatModel` and no API calls
Trajectory tests	Validate tool call order and final responses
Agent Chat UI	Talk to agents in the browser and visualize tool usage
Deployment	Deploy with LangGraph Platform, Docker, FastAPI, and related options
Observability	Use LangSmith to track execution flow, token usage, and cost

This completes the LangChain v1 agent track. You covered the full lifecycle of agent development, from basic concepts to production deployment.

Next Steps

→ Continue to 11_mcp.ipynb → Or jump to the LangGraph intermediate track

11 MCP (Model Context Protocol)

Learning Objectives

Learn how to connect external tools and context to an agent through MCP (Model Context Protocol).

This notebook covers:

- Understanding the MCP concept and architecture (server / client / host)
- Connecting to MCP servers with the beta `langchain.mcp.MCPAdapter`
- Integrating MCP tools with an agent through `create_agent(model=..., tools=await adapter.list_tools())`
- Understanding stdio and Streamable HTTP transports
- Connecting to multiple MCP servers at once

11.1 Environment Setup

```python
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("Environment ready.")
```

11.2 MCP Concepts

MCP (Model Context Protocol) is an open protocol for providing external tools and context to an LLM in a standardized way.

Architecture components

Component	Role	Example
MCP server	Exposes tools, resources, and prompts	File system server, DB server, API wrapper

MCP client	Connects to the server and fetches tools	`MCPAdapter`
Host	Manages the client and connects it to the LLM	LangChain agent, IDE

Core resource types

- Tools: executable functions the agent can call
- Resources: data such as files or database records (converted to LangChain Blob objects)
- Prompts: reusable prompt templates

Why MCP?

Before MCP, each tool needed its own custom integration code. MCP unifies this into one standard protocol, which means:

- Tool providers only need to implement an MCP server once
- LLM hosts can access every tool through one MCP client
- Tools can be reused across the ecosystem

11.3 Installing MCP support

Current LangChain exposes MCP through the beta `langchain.mcp` namespace. The older `langchain_mcp_adapters.MultiServerMCPClient` package is only a migration reference.

```
# MCP adapter installation commands
print("MCP adapter installation:")
print("  uv add 'langchain[mcp]' mcp")
print()
print("Key components:")
print("  - MCPAdapter: async context manager for one or more MCP servers")
print("  - adapter.list_tools(): converts server tools into LangChain tools")
print("  - FastMCP: Utility for quickly building MCP servers")
```

11.4 Stdio Transport

Stdio (Standard I/O) transport communicates with an MCP server through a local subprocess. It is a good fit for development and testing environments.

```
from pathlib import Path; import json, tempfile, sys
server_path = Path(tempfile.gettempdir()) / "lc_mcp_math_server.py"
server_path.write_text('from mcp.server.fastmcp import FastMCP\nmcp = FastMCP("math")\n@mcp.tool()\ndef
add(a: int, b: int) -> int:\n    return a + b\nif __name__ == "__main__":\n
mcp.run(transport="stdio")')
stdio_config = {
    "mcpServers": {
        "math": {"command": sys.executable, "args": [str(server_path)]}
    }
}
```

```
print("Stdio Transport configuration:"); print(json.dumps(stdio_config, indent=2))
print(f"\nServer file: {server_path}")
```

11.5 Streamable HTTP Transport

HTTP (streamable-http) transport uses web-based communication and is a good fit for remote MCP servers. It also supports authentication headers and custom settings.

```
# Example MCPConfig / Streamable HTTP configuration
http_config = {
    "mcpServers": {
        "weather": {"url": "https://weather-mcp.example.com/mcp"}
    }
}
import json; print("HTTP Transport configuration:"); print(json.dumps(http_config, indent=2))
print("\nUsage pattern: async with MCPAdapter(http_config) as adapter -> await adapter.list_tools()")
```

11.6 Loading MCP Tools and Integrating Them with an Agent

This is the common pattern for binding tools fetched from an MCP server into a LangChain agent. The core entry point is `adapter.list_tools()`. It discovers the tools exposed by the MCP server and converts each tool's name, description, and parameter schema into LangChain `Tool` objects. The converted tools can be passed directly to `create_agent(tools=mcp_tools)`.

Key function signatures

Function	Description
`adapter.list_tools()`	Return LangChain `Tool` objects from every registered server
`MCPConfig` `mcpServers`	Configure multiple servers in the standard MCP config shape
`fastmcp.Client`	Use custom auth, transports, or handlers before passing the client to `MCPAdapter`
`@wrap_tool_call`	Intercept MCP tool calls from the LangChain agent side

```
from langchain.mcp import MCPAdapter
from langchain.agents import create_agent

async with MCPAdapter(stdio_config) as adapter:
    mcp_tools = await adapter.list_tools()
    agent = create_agent(model="gpt-5.4", tools=mcp_tools)
```

Connection lifecycle

`MCPAdapter` is an async context manager. `list_tools()` manages the underlying client lifecycle, and each returned tool opens the needed session for its call. The old `client.session(...)` stateful helper is not part of the new namespace.

```
async with MCPAdapter(stdio_config) as adapter:
    tools = await adapter.list_tools()
    result = await tools[0].ainvoke({"a": 1, "b": 2})
```

11.7 Connecting to Multiple MCP Servers

An `MCPConfig` with `mcpServers` can manage several MCP servers at the same time. Tool names are prefixed automatically when needed.

```
# Example multi-MCP-server configuration
import json, sys
multi_server_config = {
    "mcpServers": {
        "math": {"command": sys.executable, "args": [str(server_path)]},
        "weather": {"url": "https://weather-mcp.example.com/mcp"},
        "database": {"command": "pnpm", "args": ["dlx", "@modelcontextprotocol/server-postgres"], "env":
{"DATABASE_URL": "postgresql://..."}},
    }
}
print("Multi-MCP server configuration:"); print(json.dumps(multi_server_config, indent=2,
ensure_ascii=False))
print("\nUsage pattern: async with MCPAdapter(multi_server_config) as adapter -> await
adapter.list_tools()")
```

11.8 MCP Authentication

Remote MCP servers authenticate through the underlying FastMCP client. Static config is fine for unauthenticated examples; bearer tokens, OAuth, and custom `httpx.Auth` belong on a `fastmcp.Client` that you pass to `MCPAdapter`.

Track A — Static headers

The simplest approach: pass the token through a header. Good for CI/CD with pre-issued keys.

```
from fastmcp import Client
from langchain.mcp import MCPAdapter

client = Client("https://weather-mcp.example.com/mcp", auth="Bearer YOUR_API_KEY")
async with MCPAdapter(client) as adapter:
    tools = await adapter.list_tools()
```

Track B — `httpx2.Auth` interface

For dynamic auth such as token refresh or request signing, pass an `httpx2.Auth` implementation, matching FastMCP's current client dependency.

```python
import httpx2
from fastmcp import Client

class BearerAuth(httpx2.Auth):
    def __init__(self, token: str):
        self.token = token

    def auth_flow(self, request):
        request.headers["Authorization"] = f"Bearer {self.token}"
        yield request

client = Client("https://weather-mcp.example.com/mcp", auth=BearerAuth(token="..."))
```

Track C — FastMCP OAuth

For an OAuth-protected remote MCP server, let FastMCP run its OAuth flow by passing `auth="oauth"`. If your deployment needs custom storage or token refresh policy, build that into a FastMCP-compatible auth object before giving the client to `MCPAdapter`.

```python
from fastmcp import Client
from langchain.mcp import MCPAdapter

client = Client("https://secure-mcp.example.com/mcp", auth="oauth")
async with MCPAdapter(client) as adapter:
    tools = await adapter.list_tools()
```

11.9 Elicitation — Server-requested user input

Elicitation lets an MCP server request additional input from the client (host) during a tool call. In `langchain.mcp`, this is handled as a LangGraph interrupt rather than a client callback.

```python
from langgraph.types import Command

# When the tool pauses for elicitation, collect user input in your UI.
# Resume with one answer per request key. Accepted form answers carry `content`.
resume = Command(resume={
    "responses": {
        "city_form": {
            "action": "accept",
            "content": {"city": "Seoul", "unit": "celsius"},
        }
    }
})
result = await graph.ainvoke(resume, config=config)
```

> ➡️ **TIP**
>
> `MCPAdapter` arms elicitation automatically for clients it creates. This interrupt loop requires a modern MCP server that returns `InputRequiredResult`; a legacy server-initiated elicitation callback still needs a pre-built client with its own handler. Sampling and roots are not implemented by the LangChain adapter; a server request for those capabilities raises `NotImplementedError`.

11.10 Tool Interceptors

MCP-specific `tool_interceptors` are replaced by LangChain `@wrap_tool_call` middleware. It wraps every tool call made by the agent; branch on MCP metadata when the logic should apply only to MCP tools.

Tool interceptor use cases

Use Case	Description
Auth injection	Pass user-specific tokens at runtime
Request transformation	Rewrite tool call parameters
Response filtering	Remove sensitive information
Retry logic	Retry automatically after failures
Logging	Trace tool calls

Agent-side wrapper

The wrapper can log, retry, filter, or short-circuit a tool call. For MCP-specific logic, check the tool metadata before applying the policy.

```
from langchain.agents.middleware import wrap_tool_call

@wrap_tool_call
async def logging_interceptor(request, handler):
    """Log MCP tool calls while appending to a state trace."""
    if request.tool.metadata.get("mcp"):
        print(f"[mcp] tool={request.tool_call['name']}")
    return await handler(request)
```

This pattern is useful for auth, logging, retries, and response filtering. If the middleware needs to update custom state, define the agent state schema for that field and return a LangGraph `Command` from the wrapper.

11.11 Writing a Custom MCP Server

With the FastMCP library, you can build an MCP server quickly using decorators.

11.12 Summary

This notebook covered:

Topic	Key Idea

MCP concepts	An open protocol that provides external tools and context to an LLM in a standardized way
Stdio transport	Local subprocess communication, good for development and testing
Streamable HTTP transport	Web-based communication for remote servers and authentication scenarios
Agent integration	Connect with `MCPAdapter` → `await adapter.list_tools()` → `create_agent(tools=mcp_tools)`
Multi-server support	Use an `MCPConfig` `mcpServers` target to manage several servers at once
Interceptors	Apply middleware for auth, logging, and request/response modification
Custom servers	Build an MCP server quickly with FastMCP decorators

Next Steps

→ Continue to 12_frontend_streaming.ipynb

References:

- MCP (Model Context Protocol)

12 Frontend Streaming

Learning Objectives

Learn how to stream LLM responses to users in real time.

This notebook covers:

- Understanding LangChain SDK streaming (`.stream()` , `.astream_events()`)
- Consuming Agent `stream_events(..., version="v3")` projections (`messages` , `tool_calls` , `values` , `output`)
- Learning the structure and usage of the `useStream` React hook
- Consuming real-time streaming from the Python SDK
- Understanding real-time agent-state display patterns

12.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-4.1",
)

print("Environment ready.")
```

```
# Optional observability setup: LangSmith or Langfuse
# Set the keys in .env, or uncomment the lines below to enter them manually.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: automatically enabled when LANGSMITH_TRACING=true
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGCHAIN_TRACING_V2", "true")
    os.environ.setdefault("LANGCHAIN_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGCHAIN_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGCHAIN_PROJECT']}")

# Langfuse: pass config={"callbacks": [langfuse_handler]} to invoke/stream
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
```

```
# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

12.2 Python SDK Streaming Basics

The `.stream()` method delivers model output token by token in real time. Users can see partial results before the full response is complete.

```
# Token-level streaming with `.stream()`
print("Streaming response:")
for chunk in model.stream(
    "Tell me three famous tourist attractions in Seoul.",
    config=lf_config,
):
    print(chunk.content, end="", flush=True)
print()  # newline
```

12.3 `astream_events()`

`.astream_events()` streams all internal events asynchronously. This lets you trace model calls, tool execution, and chain steps in detail.

Main event types

Event	Description
`on_chat_model_stream`	Model token streaming
`on_chat_model_start`	Model call started
`on_chat_model_end`	Model call completed
`on_tool_start`	Tool execution started
`on_tool_end`	Tool execution completed

```
import asyncio

async def stream_events_demo():
    """Example of event streaming with astream_events()"""
    print("Event streaming:")
    print("-" * 40)
    async for event in model.astream_events(
        "Two advantages of Python",
        version="v2",
    ):
        kind = event["event"]
        if kind == "on_chat_model_stream":
            content = event["data"]["chunk"].content
            if content:
                print(content, end="", flush=True)
```

```
        elif kind == "on_chat_model_start":
            print(f"[Model call start]")
        elif kind == "on_chat_model_end":
            print(f"\n[Model call complete]")

await stream_events_demo()
```

12.4 Event Streaming v3 — projection-based agent streams

For LangChain agents, prefer `stream_events(..., version="v3")` when UI code needs separated projections instead of one raw event stream.

Projection	Meaning	UI mapping
`stream.messages`	Model messages and text deltas	Chat bubble
`stream.tool_calls`	Tool-call start, args, output, errors	Tool timeline
`stream.values`	Agent state snapshots	State/debug panel
`stream.output`	Final state	Save/post-process
`stream.subgraphs` / `stream.subagents`	Nested graph or subagent events	Multi-agent cards
`stream.extensions`	Custom projection channels	Progress, charts, domain UI

The same projection model is shared with LangGraph and Deep Agents event streaming.

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get weather for a city."""
    return f"{city}: sunny"

streaming_agent = create_agent(
    model="openai:gpt-5.4", tools=[get_weather],
    system_prompt="Always use get_weather for weather questions.",
)
stream = streaming_agent.stream_events(
    {"messages": [{"role": "user", "content": "Weather in Seoul?"}]},
    version="v3",
)
for message in stream.messages:
    if str(message.text):
        print(str(message.text), end="", flush=True)
print("\noutput keys:", list(stream.output.keys()))
```

12.4 The `useStream` React Hook

`useStream` is a React hook from the LangGraph SDK that simplifies streaming communication with a LangGraph server.

Basic usage

```
import { useStream } from "@langchain/langgraph-sdk/react";

function Chat() {
  const stream = useStream({
    assistantId: "agent",
    apiUrl: "http://localhost:2024",
  });

  const handleSubmit = (message: string) => {
    stream.submit({
      messages: [{ content: message, type: "human" }],
    });
  };

  return (
    <div>
      {stream.messages.map((message, idx) => (
        <div key={message.id ?? idx}>
          {message.type}: {message.content}
        </div>
      ))}
      {stream.isLoading && <div>Loading...</div>}
      {stream.error && <div>Error: {stream.error.message}</div>}
    </div>
  );
}
```

Main return values

Property	Type	Description
`messages`	`Message[]`	The full message list for the current thread
`isLoading`	`boolean`	Whether the stream is active
`error`	`Error \\], [null`	Error object
`interrupt`	`Interrupt`	An interruption request (HITL)
`submit()`	`function`	Send a message
`stop()`	`function`	Stop the stream

12.5 `useStream` Configuration Options

Parameter	Required	Default	Description

`assistantId`	O	—	Agent identifier (from the deployment dashboard)
`apiUrl`	—	`localhost:2024`	Agent server URL
`apiKey`	—	—	Auth token for a deployed agent
`threadId`	—	—	Connect to an existing conversation thread
`onThreadId`	—	—	Callback when a thread is created
`reconnectOnMount`	—	`false`	Reconnect to an active stream when the component mounts
`onCustomEvent`	—	—	Custom event handler
`onUpdateEvent`	—	—	State update handler
`onMetadataEvent`	—	—	Metadata event handler
`messagesKey`	—	`"messages"`	State key that stores messages
`throttle`	—	`true`	Batch state updates
`initialValues`	—	—	Cached initial state

12.6 Thread Management and Reconnection

Managing Thread IDs

By managing `threadId`, you can continue a conversation or load a previous thread.

```
const [threadId, setThreadId] = useState<string | null>(null);

const stream = useStream({
  apiUrl: "http://localhost:2024",
  assistantId: "agent",
  threadId,
  onThreadId: setThreadId,
});

// Persist threadId in a URL parameter or localStorage
```

Reconnecting After a Page Refresh

If you enable `reconnectOnMount`, the hook automatically reconnects to a stream that was already in progress after a page refresh.

```
const stream = useStream({
  apiUrl: "http://localhost:2024",
  assistantId: "agent",
  reconnectOnMount: true, // use sessionStorage
});
```

```
// Use custom storage
const stream = useStream({
  reconnectOnMount: () => window.localStorage,
});
```

12.7 Branching and Message Editing

With branching, you can create an alternate path from a specific point in the conversation history. This is useful when you want to edit a user message or regenerate an AI response.

```
{stream.messages.map((message) => {
  const meta = stream.getMessagesMetadata(message);
  const parentCheckpoint = meta?.firstSeenState?.parent_checkpoint;

  return (
    <div key={message.id}>
      {message.content}

      {/* Edit a user message */}
      {message.type === "human" && (
        <button onClick={() => {
          const newContent = prompt("Edit:", message.content);
          if (newContent) {
            stream.submit(
              { messages: [{ type: "human", content: newContent }] },
              { checkpoint: parentCheckpoint }
            );
          }
        }}>
          Edit
        </button>
      )}

      {/* Regenerate an AI response */}
      {message.type === "ai" && (
        <button onClick={() =>
          stream.submit(undefined, { checkpoint: parentCheckpoint })
        }>
          Regenerate
        </button>
      )}
    </div>
  );
})}
```

Key idea: use the `checkpoint` parameter to jump back to a specific state and generate a new branch.

12.8 Custom Streaming Events

You can stream custom data from the agent to the client. This is useful for progress updates, intermediate results, and other real-time signals.

```
# Custom streaming events — Python writer pattern
print("Custom streaming event pattern (Python server side):")
print("=" * 50)
print("""
```

```
from langchain.tools import tool
from langchain.agents.types import ToolRuntime

@tool
async def analyze_data(
    data_source: str, *, config: ToolRuntime
) -> str:
    \"\"\"Analyzes the data.\"\"\"
    if config.writer:
        # Stream progress to the client
        config.writer({
            "type": "progress",
            "message": "Loading data...",
            "progress": 25,
        })
        # ... processing ...
        config.writer({
            "type": "progress",
            "message": "Analysis complete!",
            "progress": 100,
        })
    return '{"result": "Analysis complete"}'
""")
print("Client side (React): receive through the onCustomEvent callback")
print('  onCustomEvent: (data) => setProgress(data.progress)')
```

12.9 Multi-Agent Streaming

When several agents collaborate, their messages should be displayed separately. Use the `langgraph_node` metadata field to identify which agent produced each message.

Event callback summary

Callback	Use Case	Stream Mode
`onUpdateEvent`	State update after a graph step	`updates`
`onCustomEvent`	Agent-defined custom event	`custom`
`onMetadataEvent`	Execution and thread metadata	`metadata`
`onError`	Error handling	—
`onFinish`	Stream completion	—

12.11 Summary

This notebook covered:

Topic	Key Idea
SDK streaming	Use `.stream()` for real-time token output
`astream_events`	Trace model and tool calls through asynchronous event streaming

Event Streaming v3	Separate messages, tool calls, state, subagents, and custom projections
`useStream`	Simplify LangGraph server streaming in React
Thread management	Keep conversations alive with `threadId` and `reconnectOnMount`
Branching	Create alternate conversation paths from a `checkpoint`
Custom events	Stream progress and other custom data with a writer pattern
Multi-agent	Use `langgraph_node` metadata to distinguish messages by agent

Next Steps

→ Continue to 13_guardrails.ipynb

References:

- LangChain Event Streaming: https://docs.langchain.com/oss/python/langchain/event-streaming
- LangGraph Event Streaming: https://docs.langchain.com/oss/python/langgraph/event-streaming
- Streaming
- UI (Agent Chat UI & useStream)

13 Guardrails

Learning Objectives

Learn how to configure guardrails that validate and filter agent input and output.

This notebook covers:

- Understanding the concept of guardrails and why they are needed
- Comparing deterministic guardrails and model-based guardrails
- Configuring PII detection middleware
- Building human-in-the-loop guardrails
- Writing custom `before_agent` and `after_agent` guardrails

13.1 Environment Setup

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("Environment ready.")
```

13.2 Guardrail Concepts

Guardrails are safety mechanisms that validate and filter content during agent execution.

Why do we need guardrails?

- Prevent leakage of personally identifiable information (PII)
- Block prompt-injection attacks
- Prevent harmful or inappropriate content
- Enforce business rules and compliance requirements
- Validate output quality and correctness

Two approaches

Approach	Mechanism	Strengths	Weaknesses
Deterministic	Regex, keyword matching, explicit rules	Fast, predictable, cost-efficient	May miss subtle violations
Model-based	Use an LLM or classifier to analyze meaning	Can catch subtle issues	Slower and more expensive

When guardrails are applied

```
User input → [input guardrail] → agent execution → [output guardrail] → response
                    ↑                                      ↑
            before_agent                           after_agent
```

13.3 PII Detection Middleware

`PIIMiddleware` automatically detects and handles personal data such as email addresses, credit-card numbers, and IP addresses.

Strategy	Result
`redact`	Replace with `[REDACTED_EMAIL]`
`mask`	Partial masking (for example, only the last 4 digits shown)
`hash`	Replace with a deterministic hash
`block`	Raise an exception when detected

> · NOTE
>
> Scope parameters — `PIIMiddleware` exposes three independent toggles: `apply_to_input` (user messages), `apply_to_output` (model responses), and `apply_to_tool_results` . `apply_to_tool_results` defaults to `False` — tool return values (for example, database query results) are not scrubbed automatically, so set `apply_to_tool_results=True` explicitly when tool outputs may contain PII.

```python
# Example PII-detection middleware setup
print("PII-detection middleware setup:")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    model="gpt-5.4",
```

```
    tools=[customer_service_tool, email_tool],
    middleware=[
        # Replace email addresses with [REDACTED_EMAIL]
        PIIMiddleware("email",
            strategy="redact",
            apply_to_input=True),

        # Partially mask credit-card numbers (****-****-****-1234)
        PIIMiddleware("credit_card",
            strategy="mask",
            apply_to_input=True),

        # Block detected API keys (custom regex)
        PIIMiddleware("api_key",
            detector=r"sk-[a-zA-Z0-9]{32}",
            strategy="block",
            apply_to_input=True),
    ],
)
""")
print("Built-in PII types: email, credit_card, ip, mac_address, url")
print("Custom detection: pass a regex or function to the detector parameter")
```

13.4 Human-in-the-Loop Guardrails

`HumanInTheLoopMiddleware` requires human approval before risky actions are executed. This is essential for high-risk operations such as financial transactions, data deletion, or external communication.

```
# Human-in-the-Loop guardrail example
print("Human-in-the-Loop guardrail:")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, send_email_tool, delete_db_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email": True,       # approval required
                "delete_db": True,         # approval required
                "search": False,           # automatic
            }
        ),
    ],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "review-123"}}

# Step 1: run the agent -> pause at send_email
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Send an email to the team"}]},
    config=config,
)
# -> paused: waiting for approval before send_email executes

# 2Step: resume after approval
result = agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
```

```
    config=config,
)
""")
print("Key point: a checkpointer is required for pause/resume flows.")
print("On rejection, use {\"type\": \"reject\"} to block the tool call.")
```

13.5 Custom Input Guardrails — `before_agent`

The `before_agent` hook validates requests before the agent starts running. It is useful for session-level authentication, rate limiting, and content filtering.

```
# Custom input guardrail — ContentFilterMiddleware class
print("Custom input guardrail (class-based):")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

class ContentFilterMiddleware(AgentMiddleware):
    \"\"\"Deterministic guardrail: blocks requests that contain banned keywords.\"\"\"

    def __init__(self, banned_keywords: list[str]):
        super().__init__()
        self.banned_keywords = [kw.lower() for kw in banned_keywords]

    @hook_config(can_jump_to=["end"])
    def before_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        first_message = state["messages"][0]
        if first_message.type != "human":
            return None

        content = first_message.content.lower()
        for keyword in self.banned_keywords:
            if keyword in content:
                return {
                    "messages": [{
                        "role": "assistant",
                        "content": "The request contains inappropriate content."
                    }],
                    "jump_to": "end"
                }
        return None
""")
print("Key point: jump_to='end' skips agent execution and returns immediately.")
print("If the middleware returns None, execution continues to the next step.")
```

13.6 Custom Output Guardrails — `after_agent`

The `after_agent` hook validates the final output after agent execution is complete. It is useful for model-based safety checks and quality validation.

```
# Custom output guardrail — SafetyGuardrailMiddleware class
print("Custom output guardrail (class-based):")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

class SafetyGuardrailMiddleware(AgentMiddleware):
    \"\"\"Model-based guardrail: uses an LLM to evaluate response safety.\"\"\"

    def __init__(self):
        super().__init__()
        self.safety_model = init_chat_model("gpt-5.4-mini")

    @hook_config(can_jump_to=["end"])
    def after_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        last_message = state["messages"][-1]
        if not isinstance(last_message, AIMessage):
            return None

        safety_prompt = f\"\"\"Evaluate if this response is safe.
        Respond with only 'SAFE' or 'UNSAFE'.

        Response: {last_message.content}\"\"\"

        result = self.safety_model.invoke(
            [{"role": "user", "content": safety_prompt}]
        )

        if "UNSAFE" in result.content:
            last_message.content = (
                "This response was flagged as unsafe. Please ask again."
            )
        return None
""")
print("Key point: evaluate safety with a smaller helper model (gpt-5.4-mini).")
print("If the result is UNSAFE, replace the response with a safe fallback message.")
```

13.7 Decorator-Based Guardrails

Instead of defining a class, you can build a concise guardrail with decorators.

```
# Decorator-based guardrails
print("Decorator-based guardrails:")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    before_agent, after_agent, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

banned_keywords = ["hack", "exploit", "malware"]

# Input guardrail — decorator
@before_agent(can_jump_to=["end"])
def content_filter(
```

```
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    \"\"\"Blocks banned keywords.\"\"\"
    if not state["messages"]:
        return None
    content = state["messages"][0].content.lower()
    for kw in banned_keywords:
        if kw in content:
            return {
                "messages": [{"role": "assistant",
                    "content": "This request is not allowed."}],
                "jump_to": "end"
            }
    return None

# Output guardrail — decorator
@after_agent(can_jump_to=["end"])
def safety_check(
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    \"\"\"Checks whether the response contains sensitive content.\"\"\"
    last = state["messages"][-1]
    if hasattr(last, 'content') and 'password' in last.content:
        last.content = "The response contained sensitive information."
    return None

# Apply to the agent
agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool],
    middleware=[content_filter, safety_check],
)
""")
print("Decorator-based guardrails work well for simple policies.")
print("Use the class-based approach for more complex logic such as state handling or initialization.")
```

13.8 Combining Multiple Guardrails

By adding several guardrails in order to the `middleware` list, you can build a layered defense strategy.

```
# Combine multiple guardrails
print("Combining multiple guardrails (defense in depth):")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, HumanInTheLoopMiddleware
)

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, send_email_tool],
    middleware=[
        # Layer 1: deterministic input filter
        ContentFilterMiddleware(
            banned_keywords=["hack", "exploit"]
        ),

        # Layer 2: PII protection (input + output)
        PIIMiddleware("email",
            strategy="redact", apply_to_input=True),
        PIIMiddleware("email",
            strategy="redact", apply_to_output=True),

        # Layer 3: human approval for sensitive tools
```

```
        HumanInTheLoopMiddleware(
            interrupt_on={"send_email": True}
        ),

        # Layer 4: model-based safety check
        SafetyGuardrailMiddleware(),
    ],
)
""")
print("Execution order:")
print("  input -> [ContentFilter] -> [PII input] -> agent execution")
print("       -> [HITL approval] -> [PII output] -> [Safety] -> response")
print()
print("Tip: place fast deterministic guardrails first and slower model-based checks later.")
```

13.9 Production Guardrail Patterns

Best practices

Pattern	Description	Implementation
Layered defense	Combine multiple guardrails to remove a single point of failure	`middleware=[layer1, layer2, ...]`
Fail fast	Run deterministic checks first to reduce cost	Deterministic → model-based order
Input/output separation	Use different guardrails for input and output	`before_agent` + `after_agent`
Graceful rejection	Return a friendly message when blocking a request	`jump_to="end"` + guidance message
Logging and monitoring	Record guardrail trigger events	LangSmith tracing integration
Fallback strategy	Handle failure inside the guardrail system itself	`try/except` + default policy
Testing	Validate guardrail behavior with unit tests	`GenericFakeChatModel`

Domain-specific examples

Domain	Main guardrails
Healthcare	PII (patient data), medical-advice disclaimer, emergency detection
Finance	PII (account data), investment disclaimer, HITL for transaction approval

Customer service	Sentiment analysis, escalation detection, PII masking
Education	Age-appropriateness checks, academic integrity, content filtering

13.10 Summary

This notebook covered:

Topic	Key Idea
Guardrail concepts	Guardrails validate and filter content during agent execution
PII detection	`PIIMiddleware` automatically detects and handles email, credit-card numbers, and related data
HITL	`HumanInTheLoopMiddleware` requires human approval before risky tool calls
Custom input	`before_agent` validates requests before execution
Custom output	`after_agent` validates responses after execution
Decorators	`\@before_agent` and `\@after_agent` define concise guardrails
Layered defense	Multiple guardrails can be stacked in the `middleware` list

Next Steps

→ Continue to the LangGraph intermediate track

References:

- Guardrails

14 Semantic Search

Build retrieval before RAG

In this chapter you build the retrieval layer as a testable system in its own right. The goal is to understand documents, chunks, embeddings, vector stores, and retrieval evaluation before adding a generative model.

Learning goals

- Explain why semantic search is the foundation of most RAG applications.
- Turn source documents into chunks and searchable vectors.
- Evaluate retrieval quality before introducing an answer generator.

14.1 Environment setup

Start with a small, deterministic setup. The notebook avoids external services so you can focus on the retrieval contract rather than API behavior.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from langchain_core.documents import Document
from langchain_core.embeddings import DeterministicFakeEmbedding
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

14.2 Build a small document collection

A retrieval system is only as clear as the corpus it searches. We begin with a tiny document set so every search result can be inspected by eye.

```
raw_docs = [
    Document(page_content="LangChain agents combine models, tools, and middleware.", metadata={"source":
"langchain"}),
    Document(page_content="LangGraph persists state with checkpointers and stores.", metadata={"source":
"langgraph"}),
    Document(page_content="Deep Agents include planning, files, subagents, and skills.",
metadata={"source": "deepagents"}),
]
len(raw_docs)
```

14.3 Split into chunks

Chunking determines what the retriever can actually return. Here you compare compact text units instead of treating whole documents as one undifferentiated blob.

```
splitter = RecursiveCharacterTextSplitter(chunk_size=60, chunk_overlap=10)
chunks = splitter.split_documents(raw_docs)

[(doc.metadata["source"], doc.page_content) for doc in chunks]
```

14.4 Embeddings and vector store

Embeddings convert text into a similarity space. The vector store then gives us a simple interface for ranking chunks by semantic closeness.

```
embedding = DeterministicFakeEmbedding(size=64)
vectorstore = InMemoryVectorStore(embedding=embedding)
vectorstore.add_documents(chunks)

retriever = vectorstore.as_retriever(search_kwargs={"k": 2})
```

14.5 Run retrieval

Retrieval should be observable before it becomes part of a larger RAG chain. Inspect the returned chunks and scores as the system's first quality signal.

```
query = "Which framework uses checkpointers?"
results = retriever.invoke(query)

for idx, doc in enumerate(results, 1):
    print(idx, doc.metadata["source"], "=>", doc.page_content)
```

14.6 Mini retrieval evaluation

A small evaluation table is enough to catch many retrieval mistakes. The point is to define expected evidence before asking a model to write an answer.

```
test_cases = [
    ("state checkpoint memory", "langgraph"),
    ("agent middleware tools", "langchain"),
    ("skills and subagents", "deepagents"),
]

for question, expected in test_cases:
    hits = retriever.invoke(question)
    sources = [doc.metadata["source"] for doc in hits]
    print(question, "=>", sources, "PASS", expected in sources)
```

14.7 Checklist before RAG

Before adding generation, check the retrieval layer for coverage, chunk quality, and debuggability. A weak retriever usually produces a weak RAG system.

Summary

Item	Content
Covered	semantic search, chunking, embeddings, vector stores, retrievers, and retrieval-only evaluation
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `retrieval.md`
- `rag.md`
- `knowledge-base.md`

PART

III

LangGraph

Stateful Agent Orchestration

What You Will Learn in This Part

- 1. Introduction to LangGraph
- 2. Graph API Basics
- 3. Functional API Basics
- 4. Workflow Patterns
- 5. Building Agents
- 6. Persistence and Memory
- 7. Streaming
- 8. Interrupts and Time Travel
- 9. Subgraphs
- 10. Production
- 11. Local Server
- 12. Durable Execution
- 13. API Guide and Pregel
- 14. Fault Tolerance

- 15. Backward Compatibility
- 16. Case Studies

01 Introduction to LangGraph

state based agent orchestration framework

Learning Objectives

Understand the core concepts of LangGraph and two APIs (Graph API, Functional API).

1.1 What is LangGraph?

LangGraph is a low-level orchestration framework for the LangChain ecosystem.

LangChain 3-tier structure

tier	Role	Description
Deep Agents	high standard	Pre-built agent system
LangChain	agent	Building LLM agent tool
LangGraph	workflow	state-based orchestration

Key Features

- state Management: TypedDict-based state definition and reducer
- Persistence: Auto-save state via checkpointer
- streaming: Real-time token unit output
- Human-in-the-loop: interrupt/resume for human intervention
- durable execution: Automatic recovery in case of failure

1.2 Core concepts

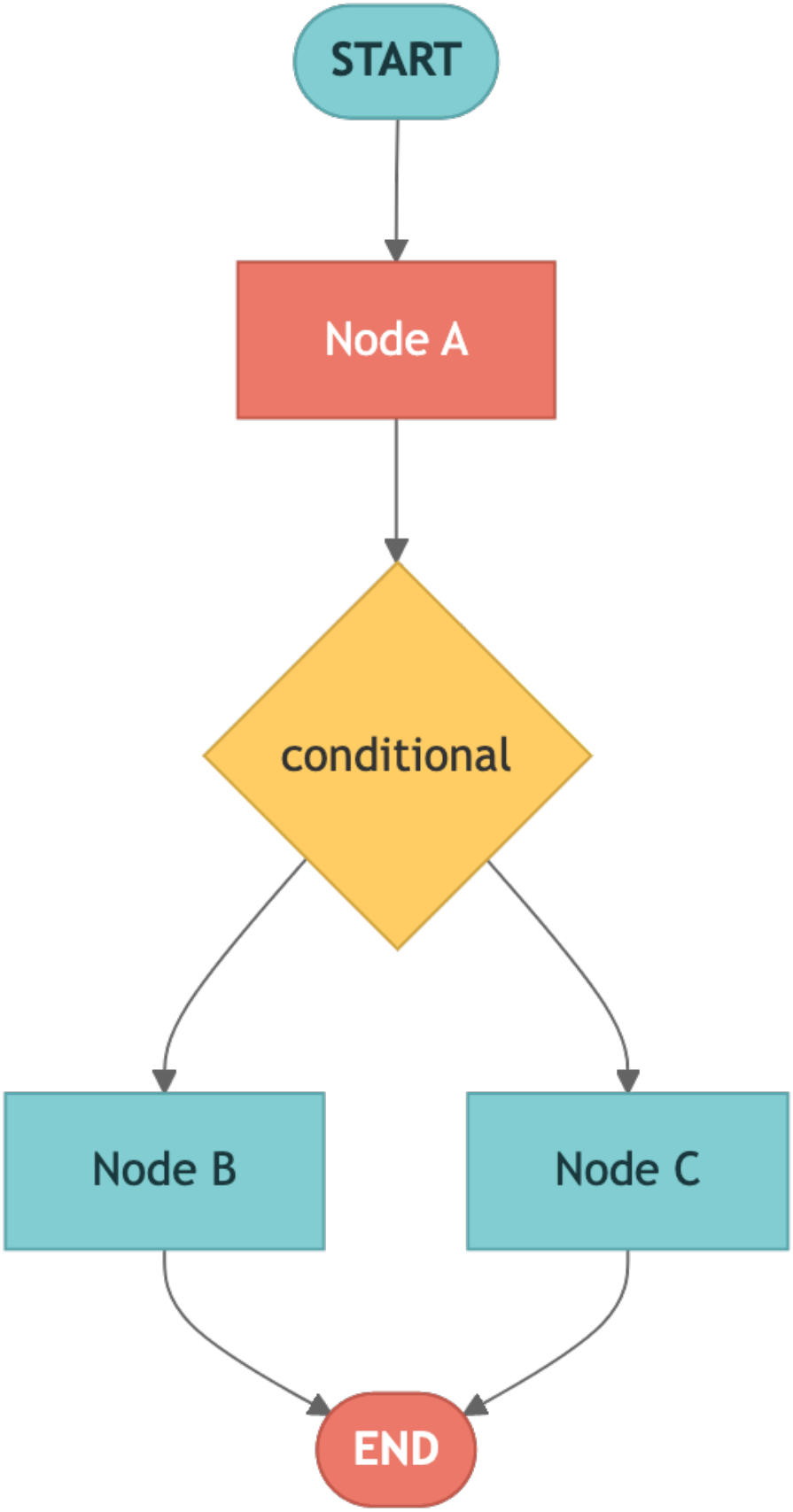
LangGraph defines workflow based on graph structure.

Component

concept	Description
Node	Processing unit — defined as a Python function

Edge	Connection between nodes, conditional branching possible
state(State)	Defined as TypedDict, shared data between nodes
checkpointer(Checkpointer)	Auto-save each step state

Graph structure diagram


START
Node A
conditional
Node B
Node C
END

1.3 Two APIs

LangGraph provides an API that allows the same functionality to be implemented in two different styles.

Characteristics	Graph API	Functional API
approach	declarative (node+edge)	imperative (Python control flow)
state Management	Explicit State + Reducer	No need for function scope or reducer
Visualization	Graph visualization support	Not supported
checkpointing	New checkpoint every superstep	By `\@task`, save to existing checkpoint
suitable situation	Complex workflow, Team Development	Migrate existing code, simple flow

1.4 Environment Setup and check installation

Verify that the required packages are installed correctly.

```
import importlib

packages = {
    "langgraph": "langgraph",
    "langchain": "langchain",
    "langchain_openai": "langchain-openai",
}

print("=" * 50)
print("LangGraph Check environment")
print("=" * 50)

for module_name, package_name in packages.items():
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, "__version__", "installed")
        print(f"  OK  {package_name}: {version}")
    except ImportError:
        print(f"  ERR {package_name}: not installed")
```

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

required = ["OPENAI_API_KEY"]
optional = ["TAVILY_API_KEY", "LANGSMITH_API_KEY"]

print("API key state:")
for key in required:
    print(f"  {'OK' if os.environ.get(key) else 'MISSING'} {key} (required)")
for key in optional:
    print(f"  {'OK' if os.environ.get(key) else '--'} {key} (optional)")
```

```python
# Core import verification
from langgraph.graph import StateGraph, START, END
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command, interrupt
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, AIMessage
from langchain_openai import ChatOpenAI

print("All core imports completed")
```

1.5 A taste of the Graph API

The Graph API defines workflow in a declarative manner.

1. Create a graph builder with `StateGraph(State)` — `state_schema`
2. `add_node()` — Node (function) registration
3. `add_edge()` — Connections between nodes
4. `compile()` — Create an executable graph
5. `invoke()` — Graph execution

1.6 A taste of the Functional API

The Functional API defines workflow in an imperative manner.

- `@task` — Unit operation definition (checkpointing unit)
- `@entrypoint` — workflow Entry point definition
- Use regular Python control flow (`if` , `for` , `while` , etc.)

1.7 Next Steps

In the next Note book, we will learn Graph API in earnest.

- 02_graph_api.ipynb — StateGraph, node, edge, conditional branch, state reducer

Summary

Item	Description
LangGraph	state Based on agent Orchestration Framework
Graph API	Explicit state flow definition with `StateGraph`
Functional API	`\@entrypoint` + `\@task` Functional workflow
Key concepts	State (state), Node, Edge

checkpointer	state Persistence, multi-turn dialogue, time travel support

Next Steps

→ 02_graph_api.ipynb: Learn the core concepts of Graph API.

02 Graph API Basics

Creating workflow with StateGraph

Learning Objectives

Understand StateGraph, nodes, edges, conditional branches, and state reducers.

2.1 Environment Setup

Load the LLM model and required modules.

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

2.2 StateGraph basic structure

The basic flow using StateGraph is as follows:

1. Create a graph builder with `StateGraph(State)` — `state_schema`
2. `add_node()` — Node (function) registration
3. `add_edge()` — Connections between nodes
4. `compile()` — Create an executable graph
5. `invoke()` — Graph execution

StateGraph(State) → add_node() → add_edge() → compile() → invoke()

```
```

2.3 state Reducer

Specifies the state update method with `Annotated`.

What is a reducer?

- The reducer determines how the fields in state are updated.
- No reducer: simple override

- `operator.add` : Accumulate (append) list items
- Reducers can also be defined as custom functions.

2.4 Conditional Edge

Branch to another node according to state.

- `add_conditional_edges(source, routing_function)` — The return value of the routing function is the next node name.
- Routing functions can be visualized using the `Literal` type hint.

START → classify → [route] → weather → END → math → END → general → END

2.5 Message-based state

Use `MessagesState` as appropriate for LLM agent.

- `MessagesState` is a predefined state that includes `messages: Annotated[list[AnyMessage], add_messages]`
- `add_messages` Reducer automatically accumulates the message list
- Naturally add LLM responses to message history

2.6 Input/Output Schema

Separate the graph's input and output from its internal state.

- `StateGraph(InternalState, input_schema=InputSchema, output_schema=OutputSchema)`
- Input schema: Includes only data received from external sources
- Output schema: Includes only data exported externally
- Internal state: Includes fields for intermediate processing (not exposed to the outside)

2.7 add_node() advanced options

`add_node()` accepts production-grade policy parameters for per-node control.

Parameter	Description
`retry_policy`	`RetryPolicy(max_attempts=..., retry_on=...)` — auto retry on failure
`timeout`	`TimeoutPolicy(run_timeout=..., idle_timeout=...)` — async nodes only; raises `NodeTimeoutError`

`cache_policy`	`CachePolicy(ttl=..., key_func=...)` — input-hash result caching; requires `cache=` at compile
`error_handler`	Callback receiving `NodeError`, returning a recovery `Command`
`defer`	Wait until other pending tasks complete — useful for asymmetric fan-out/fan-in

```python
from langgraph.cache.memory import InMemoryCache
from langgraph.types import RetryPolicy, CachePolicy

builder.add_node(
    "expensive_node",
    expensive_node,
    retry_policy=RetryPolicy(max_attempts=3, retry_on=ValueError),
    cache_policy=CachePolicy(ttl=3),
)
graph = builder.compile(cache=InMemoryCache())
```

2.8 Runtime context with context_schema

Pass dependencies that aren't part of graph state (LLM providers, user IDs, DB handles) via `context_schema` + `Runtime[ContextSchema]`. Context lives outside the checkpointed state.

- `StateGraph(State, context_schema=ContextSchema)` declares the context schema
- Node signature `runtime: Runtime[ContextSchema]` exposes `runtime.context`
- Invoke with `graph.invoke(inputs, context={...})`
- `runtime.execution_info` exposes `attempt`, `thread_id`, `run_id`, checkpoint info
- `runtime.server_info` carries LangGraph Server assistant/graph/user info
- `runtime.drain_requested` signals graceful shutdown

```python
from dataclasses import dataclass
from langgraph.runtime import Runtime

@dataclass
class ContextSchema:
    llm_provider: str = "openai"
    user_id: str | None = None

def node(state: State, runtime: Runtime[ContextSchema]):
    if runtime.execution_info.attempt > 1:
        # fallback on retry
        ...
    print("thread:", runtime.execution_info.thread_id)
    return {"results": "processed"}

graph = StateGraph(State, context_schema=ContextSchema).compile()
graph.invoke({...}, context={"llm_provider": "anthropic", "user_id": "u-1"})
```

2.9 Send and Command.PARENT

`Send("worker", {...})` returned from a conditional-edge function spawns a worker instance with its own state at runtime — the Map step of Map-Reduce. From inside a subgraph, `Command(graph=Command.PARENT, goto="...", update={...})` routes to a parent-graph node.

```python
from langgraph.types import Send, Command

def fan_out(state):
    return [Send("worker", {"task": t}) for t in state["tasks"]]

def sub_node(state):
    return Command(
        graph=Command.PARENT,
        goto="parent_aggregator",
        update={"sub_result": state["value"]},
    )
```

2.10 Summary

We summarize what we learned in this Note book.

concept	Description
StateGraph	`state_schema` based graph builder
Node	Processing units defined as Python functions
Edge	Fixed connection between nodes (`add_edge`)
Conditional Edge	state based dynamic branch (`add_conditional_edges`)
Reducer	Define state accumulation method with `Annotated` + `operator.add`
MessagesState	Predefined state for LLM dialog
Input/Output Schema	Separation of internal state and external input/output

Next Steps

→ 03_functional_api.ipynb: Learn Functional API.

03 Functional API

Creating workflow with @entrypoint and @task

Learning Objectives

Understand the `@entrypoint`, `@task` patterns and short-term memory of the Functional API.

3.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

3.2 @task — Asynchronous unit of work

- Checkpointing is possible by wrapping it with the `@task` decorator.
- Returns Future object immediately when called, waits with `.result()`

3.3 Parallel `\@task` execution

Running multiple `@task` simultaneously.

3.4 previous — short-term memory (accessing previous execution results)

3.5 entrypoint.final — Separate return and checkpoint saved values

3.6 Determinism Requirements

Non-deterministic operations must be wrapped in `@task`.

3.7 LLM agent (Functional API)

Implementing ReAct agent with while loop

3.8 @task retry and caching

`@task` accepts the same policy objects as `add_node()`. `RetryPolicy` configures auto-retry, and `CachePolicy` caches results by input hash. Caching requires attaching `cache=InMemoryCache()` on the entrypoint and `cache_policy` on the task — the two are configured separately.

```
from langgraph.cache.memory import InMemoryCache
from langgraph.types import RetryPolicy, CachePolicy

@task(retry_policy=RetryPolicy(max_attempts=3, retry_on=ValueError))
def fetch_data(url: str) -> dict: ...

@task(cache_policy=CachePolicy(ttl=120))
def slow_compute(x: int) -> int:
    time.sleep(1)
    return x * 2

@entrypoint(cache=InMemoryCache())
def main(inputs: dict) -> dict[str, int]:
    a = slow_compute(inputs["a"]).result()
    b = slow_compute(inputs["b"]).result()
    return {"a": a, "b": b}
```

3.9 Structured interrupt and Command(resume=)

`interrupt()` pauses the workflow for human review. Pass a structured dict payload so callers get rich context, and resume with `Command(resume=...)`.

```python
from langgraph.types import Command, interrupt

@task
def review(input_query: str):
    human_review = interrupt({
        "question": "Approve this result?",
        "tool_call": {"name": "send_email", "args": {...}},
    })
    return human_review

config = {"configurable": {"thread_id": "review-1"}}
workflow.invoke("draft", config=config)
workflow.invoke(Command(resume={"approved": True}), config=config)
```

3.10 @entrypoint injected parameters

Beyond `previous`, `@entrypoint` auto-injects runtime dependencies by parameter name.

Parameter	Role
`previous`	Last checkpoint's saved value (short-term memory)
`store`	`BaseStore` instance for long-term memory
`writer`	`StreamWriter` for custom streaming (async + Python < 3.11)
`config`	`RunnableConfig` — `thread_id`, metadata, configurable values

```python
from langgraph.types import StreamWriter

@entrypoint(checkpointer=checkpointer, store=store)
def workflow(
    inputs: dict,
    *,
    previous=None,
    store=None,
    writer: StreamWriter = None,
    config=None,
):
    writer({"phase": "start"})
    user_id = config["configurable"]["user_id"]
    memory = store.get(("users", user_id), "profile")
    ...
```

3.11 Summary

Features	Description
`\@task`	Asynchronous operations, checkpointing, parallel execution
`\@entrypoint`	workflow Entry point, execution management
`.result()`	Future Result Synchronous Waiting

`previous`	Access previous execution results (short-term memory)
`entrypoint.final`	Separate return value ≠ stored value

Next Steps

→ 04_workflows.ipynb: Learn the workflow pattern.

04 workflow Pattern

5 core patterns

Learning Objectives

Understand Prompt Chaining, Parallelization, Routing, Orchestrator–Worker, and Evaluator–Optimizer patterns.

4.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

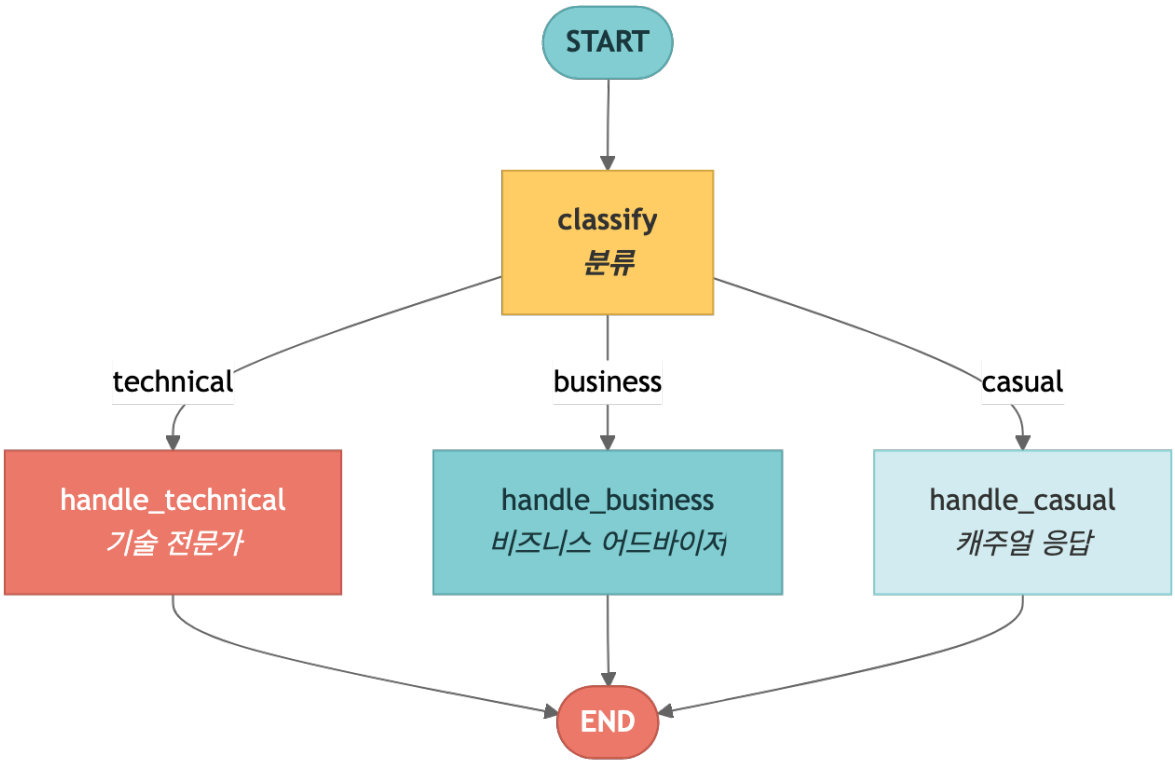
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

4.2 Prompt Chaining — Sequential LLM calls

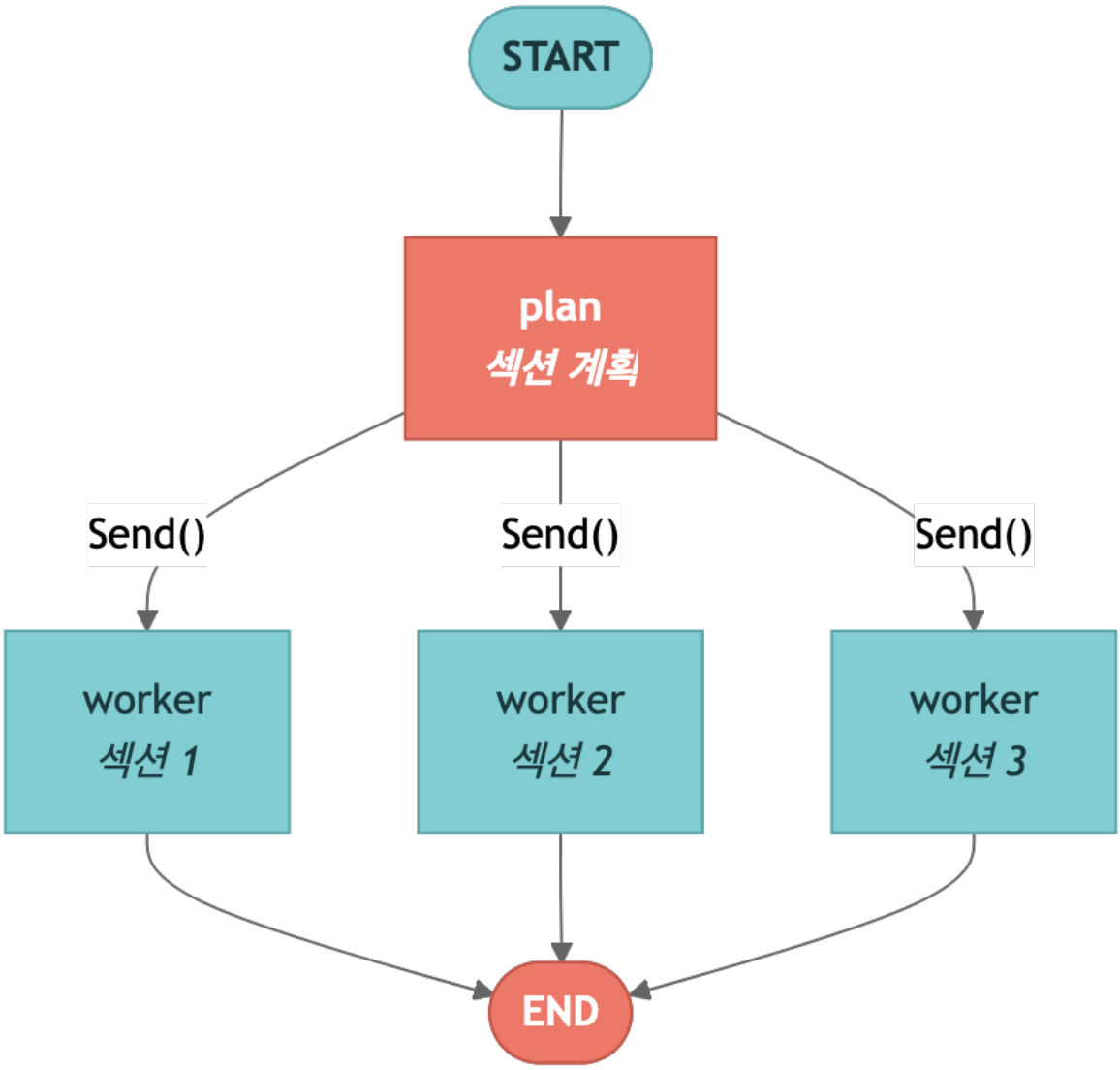
- The output of each step becomes the input of Next Steps
- Purpose: Translation → Verification → Proofreading, Analysis → Summary → Formatting

4.3 Parallelization — Simultaneous execution of independent `\@task`

4.4 Routing — Classification-based branching



4.5 Orchestrator-Worker — Create dynamic worker with Send()



4.6 Evaluator-Optimizer — Generate-evaluation iteration loop

4.7 Pattern comparison table

pattern	decisive	Parallel	repeat	suitable situation
Prompt Chaining	O	X	sequential	Step-by-step conversion
Parallelization	O	O	X	Independent Analysis

Routing	O	X	X	Classification-based processing
Orchestrator-Worker	O	O	X	Dynamic Subtasks
Evaluator-Optimizer	X	X	O	Quality Improvement Loop

Next Steps

→ 05_agents.ipynb: Build ReAct agent.

05 Building agent

Creating ReAct agent with Graph API and Functional API

Learning Objectives

We implement LLM agent using tool with two APIs.

- Graph API: Explicitly configure ReAct loop with `StateGraph` and conditional edges
- Functional API: Simple implementation with `@entrypoint` + `while` loop.
- tool Binding: Bind tool to LLM with `@tool` decorator and `bind_tools()`.
- Memory: agent holding conversation state with checkpointer

5.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

5.2 tool Definitions — @tool decorator and bind_tools()

The `@tool` decorator on LangChain allows you to convert a regular Python function into tool, which can be called by LLM. `bind_tools()` binds the schema of these tools to the model, allowing LLM to select tool and generate arguments at the appropriate time.

```
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, ToolMessage, AnyMessage

@tool
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

@tool
def divide(a: int, b: int) -> float:
    """Divide a by b."""
    return a / b
```

```
tools = [add, multiply, divide]
tools_by_name = {t.name: t for t in tools}
model_with_tools = model.bind_tools(tools)

print("tool bound to model:")
for t in tools:
    print(f"  - {t.name}: {t.description}")
```

5.3 Graph API agent — Implementing ReAct Loop with StateGraph

The ReAct (Reasoning + Acting) pattern consists of three elements:

- LLM node: Determines whether tool calling based on current message
- Tool node: actually executes the tool selected by LLM
- Conditional Edge: If `tool_calls` exists, route to `tool_node`, otherwise route to `END`

START → llm → [tool_calls?] → tools → llm → ··· → END

```

```

5.4 Visualizing execution flow — Observe each step with streaming

`stream_mode="updates"` allows each node to receive updates as it runs. This allows you to observe step by step in what order agent calls tool and processes the results.

5.5 Functional API agent — @entrypoint + while loop

The Functional API allows you to write agent like regular Python code, without explicitly constructing a graph.

- `@entrypoint`: Defines the entry point of agent
- `@task`: Defines individual work units
- `while` loop: repeats until there is no tool calling

5.6 agent with memory — Keep conversation with checkpointer

If you pass checkpointer(`InMemorySaver`) to `compile()`, agent will be able to remember previous conversations. If you use the same `thread_id`, the previous conversation context is automatically maintained.

5.7 Summary

concept	Description
`\@tool`	Convert Python function to LLM callable tool
`bind_tools()`	tool Binding schema to model
Graph API agent	`StateGraph` + Explicit implementation of ReAct loop with conditional edges
Functional API agent	Simple implementation with `\@entrypoint` + `while` loop
`tool_calls`	tool calling Information included in LLM response
`ToolMessage`	tool Message delivering execution results to LLM
checkpointer	Implement multiturn agent by saving conversation state

Next Steps

→ 06_persistence_and_memory.ipynb: Learn persistence and memory.

06 Persistence and memory

checkpointer and memory store

Learning Objectives

Store state with checkpointer and implement long-term memory with store.

- checkpointer: Automatically save and restore state of each execution step.
- state lookup: Check state saved as `get_state()` and `get_state_history()`
- state Modification: Change state externally with `update_state()`.
- Thread Independence: Different `thread_id` are completely independent state
- InMemoryStore: long-term memory shared between threads (standalone and graph integration)
- Conversation length management: Message management with `trim_messages` and `RemoveMessage`
- Durable Execution: resume at last checkpoint in case of failure

6.1 Environment Setup

```python
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4-mini")
```

6.2 checkpointer — Automatically saves state for each execution step

LangGraph offers multiple checkpointer implementations. The seven commonly used options are summarized below.

Checkpointer	Package	Use
`InMemorySaver`	`langgraph-checkpoint` (built-in)	Development/testing. State lost on process exit
`SqliteSaver`	`langgraph-checkpoint-sqlite`	Local development. Single-file persistence, async variant

`PostgresSaver`	`langgraph-checkpoint-postgres`	Production default. `AsyncPostgresSaver` for async graphs
`MongoDBSaver`	`langgraph-checkpoint-mongodb`	Document DB. JSON-friendly storage backend
`RedisSaver`	`langgraph-checkpoint-redis`	In-memory persistence. Low latency, async variant
`OracleSaver`	`langgraph-checkpoint-oracle`	Enterprise Oracle DB. Vector search included
`CosmosDBSaver`	`langchain-azure-cosmosdb`	Azure Cosmos DB. Microsoft Entra ID authentication

If you pass checkpointer to `compile()`, state will be automatically saved after each node in the graph is executed.

DB-backed checkpointers are initialized with the context-manager + `setup()` pattern. Use async variants (e.g. `AsyncPostgresSaver`) for async graphs.

```
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup()  # first time only — create schema
    graph = builder.compile(checkpointer=checkpointer)

# Async graphs
from langgraph.checkpoint.postgres import AsyncPostgresSaver

checkpointer = AsyncPostgresSaver.from_conn_string(DB_URI)
await checkpointer.setup()
graph = builder.compile(checkpointer=checkpointer)
```

6.3 get_state() — Retrieve currently stored state lookup

`get_state()` returns the latest checkpoint state for the specified thread. You can check information such as number of messages, checkpoint ID, and next node to run.

```
state = graph.get_state(config)
print(f"Thread: {config['configurable']['thread_id']}")
print(f"Message count: {len(state.values['messages'])}")
print(f"Checkpoint ID: {state.config['configurable']['checkpoint_id']}")
print(f"Next node: {state.next}")
```

6.4 get_state_history() — View entire execution history

`get_state_history()` returns all checkpoints for that thread, sorted by most recent. This allows you to trace the entire history of graph execution.

```
print("state History (most recent):")
for i, snapshot in enumerate(graph.get_state_history(config)):
    msg_count = len(snapshot.values.get("messages", []))
    print(f"  [{i}] checkpoint={snapshot.config['configurable']['checkpoint_id'][:20]}...
messages={msg_count}")
    if i >= 4:
        print("... (omitted)")
        break
```

6.5 update_state() — Modify stored state externally

`update_state()` allows you to programmatically modify state stored in a checkpoint. For example, you can add system Note, reflect user preferences, and more.

6.6 Thread Independence — Different thread_ids are completely independent state

Each `thread_id` has a completely independent conversation state. Conversations in different threads do not affect each other.

6.7 InMemoryStore — Cross-thread sharing long-term memory

`InMemoryStore` is a key-value store shared between threads. Used to store information that needs to be maintained across threads, such as user profiles and preferences.

- `put()` : Store data with namespace and key
- `get()` : Search for a specific item
- `search()` : Search within namespace

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

# data storage
store.put(("users",), "alice", {"favorite_color": "blue", "city": "Seoul"})
store.put(("users",), "bob", {"favorite_color": "red", "city": "Tokyo"})

# Data inquiry
alice = store.get(("users",), "alice")
print(f"Alice: {alice.value}")

# search
results = store.search(("users",))
print(f"\nTotal users ({len(results)}):")
for item in results:
    print(f"  {item.key}: {item.value}")
```

6.7.5 Using InMemoryStore with graphs

If you pass `InMemoryStore` to the graph as `compile(store=store)`, you can directly access store through the `store` parameter in each node function. This pattern allows you to store and retrieve user information within a node, maintaining long-term memory across threads.

- `compile(store=store)`: connect store to the graph.
- Add `store` parameter to node function: LangGraph automatically injects store instance
- Separate namespace for each user with `config["configurable"]["user_id"]`

Context and Runtime — accessing user_id and store from nodes

To partition long-term memory per user, the node function needs the current user's ID. LangGraph solves this with a `@dataclass` `Context` registered via `StateGraph(..., context_schema=Context)`, plus a `runtime: Runtime[Context]` parameter on the node. Pass `context=Context(user_id=...)` to `graph.invoke(...)`, then access `runtime.context.user_id` and `runtime.store` inside the node.

```
import uuid
from dataclasses import dataclass
from langgraph.runtime import Runtime
from langgraph.graph import StateGraph, MessagesState, START

@dataclass
class Context:
    user_id: str

async def call_model(state: MessagesState, runtime: Runtime[Context]):
    user_id = runtime.context.user_id
    namespace = (user_id, "memories")  # recommended pattern

    memories = await runtime.store.asearch(
        namespace, query=state["messages"][-1].content, limit=3
    )
    await runtime.store.aput(
        namespace, str(uuid.uuid4()), {"data": "User prefers dark mode"}
    )
    return {"messages": [await model.ainvoke(state["messages"])]}

builder = StateGraph(MessagesState, context_schema=Context)
builder.add_node(call_model)
builder.add_edge(START, "call_model")
graph = builder.compile(checkpointer=checkpointer, store=store)

graph.invoke(
    {"messages": [{"role": "user", "content": "hi"}]},
    {"configurable": {"thread_id": "1"}},
    context=Context(user_id="alice"),
)
```

> ➡️ TIP
>
> Any tuple shape works as a namespace, but the official docs recommend `(user_id, "memories")`. First-level partitioning by user, second-level by memory kind (`"memories"`, `"preferences"`, etc.) makes category-scoped `search()` straightforward.

Production stores follow the same context-manager pattern as checkpointers:

```python
from langgraph.store.postgres import PostgresStore

with PostgresStore.from_conn_string(DB_URI) as store:
    store.setup()
    graph = builder.compile(checkpointer=checkpointer, store=store)

# Redis / Oracle follow the same pattern
from langgraph.store.redis import RedisStore
from langgraph.store.oracle import OracleStore  # vector search built-in
```

Semantic search with `index`

Enable embedding-based search by passing `index={"embed": ..., "dims": ..., "fields": [...]}`:

```python
from langchain.embeddings import init_embeddings
from langgraph.store.memory import InMemoryStore

store = InMemoryStore(
    index={
        "embed": init_embeddings("openai:text-embedding-3-small"),
        "dims": 1536,
        "fields": ["food_preference", "$"],
    }
)

store.put(("alice", "memories"), "1", {"food_preference": "I love pizza"})
items = store.search(("alice", "memories"), query="I'm hungry", limit=1)
```

Use `index=False` on `put()` to skip embedding for a specific item; pass a list of keys to limit which fields get embedded.

6.7.6 Managing conversation length — trim_messages and RemoveMessage

Long conversations can exceed LLM's context window. LangGraph manages messages in two ways:

`trim_messages`

- Automatically trims old messages based on number of tokens
- `strategy="last"`: Keep only recent messages
- `start_on="human"`: Ensure truncated results always start with the user message.
- Returns a list of truncated messages, without modifying the original state (full history is maintained in checkpoints)

`RemoveMessage`

- Permanently delete specific messages from checkpoints
- The reducer of `MessagesState` detects `RemoveMessage` and removes that message.
- Useful for saving storage space by organizing old messages

6.8 Durable Execution — resume at last checkpoint on failure

Durable Execution is possible using checkpointer. Even if an error occurs during graph execution, you can resumeat the last successful checkpoint. Nodes that have already been completed are not rerun, saving money and time.

The example below configures a three-stage pipeline:

1. step_1: Collect data (always succeeds)
2. step_2: Data analysis (always successful)
3. step_3: External API call (failure on first run, success on retry)

With `attempt_count`, step_3 fails on the first run, and when calling `invoke()` the second time, step_1 and step_2 are skipped and only resume occurs in step_3.

6.9 Summary

concept	Description
checkpointer	Automatically save state after each node execution (`InMemorySaver` , `SqliteSaver` , `PostgresSaver`)
`get_state()`	View the latest checkpoint state of the current thread
`get_state_history()`	View the entire checkpoint history of a thread (most recent)
`update_state()`	Programmatically modifying stored state
Thread independence	Different `thread_id` are completely independent state
`InMemoryStore`	Key-value shared across threads long-term memory storage
`compile(store=store)`	Access long-term memory with `store` parameter from graph node
`trim_messages`	Cut out old messages based on number of tokens and pass them to LLM (maintain checkpoints)
`RemoveMessage`	Permanently delete specific messages from checkpoint
Durable Execution	On failure, at the last successful checkpoint resume

Next Steps

→ 07_streaming.ipynb: Learn streaming.

07 streaming

Observe agent execution in real time

Learning Objectives

Understand and utilize the various streaming modes of LangGraph.

- Understand the differences between `values`, `updates`, `messages`, `custom`, `debug` modes
- Identify appropriate use cases for each streaming mode
- Learn how to use multiple streaming modes simultaneously

7.1 Environment Setup

```python
from dotenv import load_dotenv
load_dotenv()
from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, ToolMessage, AnyMessage
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from typing import TypedDict, Annotated, Literal
import operator

model = ChatOpenAI(model="gpt-4.1")

@tool
def search(query: str) -> str:
    """Search for information."""
    return f"Search result for '{query}': LangGraph is a framework for building stateful agents."

tools = [search]
tools_by_name = {t.name: t for t in tools}
model_with_tools = model.bind_tools(tools)

class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]

def llm_node(state: AgentState) -> dict:
    return {"messages": [model_with_tools.invoke(
        [SystemMessage(content="You are a useful assistant.")] + state["messages"],
        config=lf_config,
)]}

def tool_node(state: AgentState) -> dict:
    results = []
    for tc in state["messages"][-1].tool_calls:
        result = tools_by_name[tc["name"]].invoke(tc["args"], config=lf_config)
        results.append(ToolMessage(content=str(result), tool_call_id=tc["id"]))
    return {"messages": results}

def should_continue(state: AgentState) -> Literal["tools", "__end__"]:
    return "tools" if state["messages"][-1].tool_calls else END
```

```
builder = StateGraph(AgentState)
builder.add_node("llm", llm_node)
builder.add_node("tools", tool_node)
builder.add_edge(START, "llm")
builder.add_conditional_edges("llm", should_continue, ["tools", END])
builder.add_edge("tools", "llm")
agent = builder.compile(checkpointer=InMemorySaver())

print("streaming agent ready for demo")
```

```
# Observability settings (optional) - LangSmith or Langfuse
# Set the key in .env, or uncomment it below and enter it yourself.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: Automatically activated when LANGSMITH_TRACING=true (no code modification required)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGCHAIN_TRACING_V2", "true")
    os.environ.setdefault("LANGCHAIN_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGCHAIN_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGCHAIN_PROJECT']}")

# Langfuse: Pass config={"callbacks": [langfuse_handler]} when calling invoke/stream
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

7.2 streaming Mode Comparison

LangGraph provides various streaming modes. Each mode conveys a different level of information in real time.

mode	Description	Use
`values`	Total state of each step	Debugging, tracing state
`updates`	Only updates returned by each node	Show progress
`messages`	Message Token Unit	Chat UI
`custom`	custom event	Custom Progress
`debug`	Full debug information	Debugging during development

7.3 stream_mode="values" — Full state snapshot

`values` mode returns full state after each node execution. This is useful for tracking the overall flow of how the graph progresses.

```
config = {"configurable": {"thread_id": "stream-values"}}
print("=== stream_mode='values' ===")
for state in agent.stream(
    {"messages": [HumanMessage(content="LangGraph Please search for information")]},
    {**config, **lf_config}, stream_mode="values",
):
    msg = state["messages"][-1]
    print(f"  [{type(msg).__name__}] {str(msg.content)[:100]}")
```

7.4 stream_mode="updates" — Node-specific updates

`updates` mode only passes the update value returned by each node. You can see exactly which nodes made which changes.

```
config = {"configurable": {"thread_id": "stream-updates"}}
print("=== stream_mode='updates' ===")
for chunk in agent.stream(
    {"messages": [HumanMessage(content="LangGraph Please search for information")]},
    {**config, **lf_config}, stream_mode="updates",
):
    for node_name, update in chunk.items():
        print(f"\n  [{node_name}]")
        for msg in update.get("messages", []):
            if hasattr(msg, 'tool_calls') and msg.tool_calls:
                for tc in msg.tool_calls:
                    print(f"    Tool: {tc['name']}({tc['args']})")
            elif msg.content:
                print(f"    {msg.content[:150]}")
```

7.5 stream_mode="messages" — Per-token streaming

The `messages` mode delivers tokens generated by LLM in real time. Best suited for implementing typing effects in chat UI.

```
config = {"configurable": {"thread_id": "stream-messages"}}
print("=== stream_mode='messages' ===")
print("response:", end="")
for msg, metadata in agent.stream(
    {"messages": [HumanMessage(content="What is LangGraph? Please answer briefly.")]},
    {**config, **lf_config}, stream_mode="messages",
):
    if msg.content and metadata.get("langgraph_node") == "llm":
        print(msg.content, end="", flush=True)
print()
```

7.6 Simultaneous use of multiple streaming modes

You can use multiple modes simultaneously by passing a list to `stream_mode`. The returned event is in the form of a `(mode, data)` tuple.

```
config = {"configurable": {"thread_id": "stream-multi"}}
print("=== Simultaneous use of multiple streaming modes ===")
for event in agent.stream(
    {"messages": [HumanMessage(content="Please search Python")]},
    {**config, **lf_config}, stream_mode=["updates", "messages"],
):
    kind = event[0] if isinstance(event, tuple) else "unknown"
    print(f"  [{kind}] received")
```

7.7 stream_mode="custom" — custom streaming

`custom` mode lets you stream arbitrary data directly inside a node.

If you call `get_stream_writer()` of `langgraph.config`, you can get the `writer` function. If you pass data such as a dictionary to this function, it will be sent to the client in real time while the graph is running.

Use Case:

- Progress reporting of long tasks
- Chunk-wise streaming in external LLM without using LangChain
- Immediate delivery of intermediate results inside the node

> ➡️ TIP
>
> If you run a graph with `stream_mode="custom"`, the client receives only the data sent with `writer()`.

```
from langgraph.config import get_stream_writer

# --- Simple graph definition using custom streaming ---
class CustomState(TypedDict):
    topic: str
    result: str

def research_node(state: CustomState) -> dict:
    """research_node: streaming intermediate progress with get_stream_writer()."""
    writer = get_stream_writer()

    # Step 1: Notification that search begins
    writer({"step": 1, "status": "Searching", "detail": f"Searching for '{state['topic']}' ..."})

    # Step 2: Notification during analysis
    writer({"step": 2, "status": "Analyzing", "detail": "Analyzing search results..."})

    # Step 3: Notification of completion
    writer({"step": 3, "status": "complete", "detail": "Investigation completed."})

    return {"result": f"Research on '{state['topic']}' is complete."}

custom_builder = StateGraph(CustomState)
custom_builder.add_node("research", research_node)
custom_builder.add_edge(START, "research")
custom_builder.add_edge("research", END)
custom_graph = custom_builder.compile()

# --- Run with stream_mode="custom" ---
print("=== stream_mode='custom' ===")
for chunk in custom_graph.stream(
    {"topic": "LangGraph streaming"},
    stream_mode="custom",
```

```
    config=lf_config,
):
    print(f"  Custom event: {chunk}")

print("--- Simultaneous use of updates + custom ---")
for mode, chunk in custom_graph.stream(
    {"topic": "LangGraph streaming"},
    stream_mode=["updates", "custom"],
    config=lf_config,
):
    if mode == "custom":
        print(f"  [custom]  {chunk}")
    else:
        print(f"  [updates] {chunk}")
```

7.8 Event Streaming v3 — projection-based app streams

LangGraph `stream_events(..., version="v3")` adds projection handles on top of raw `stream_mode`. Raw stream modes are useful when you want to parse runtime chunks directly. v3 event streaming is better when app code wants purpose-specific handles such as `stream.messages`, `stream.values`, `stream.subgraphs`, `stream.output`, `stream.interrupts`, and `stream.extensions`.

Need	Recommended API
Inspect raw node updates	`graph.stream(..., stream_mode="updates")`
Handle custom events directly	`graph.stream(..., stream_mode="custom")`
Split messages, state, and subgraphs for UI	`graph.stream_events(..., version="v3")`
Show resume-after-interrupt state	`stream.interrupted`, `stream.interrupts`
Add typed custom projections	`StreamTransformer`, `StreamChannel`

```
# Event Streaming v3 — consume values/output projections from custom_graph.
stream = custom_graph.stream_events(
    {"topic": "LangGraph Event Streaming v3"},
    version="v3",
)
print("values projection:")
for snapshot in stream.values:
    print(snapshot)
print("final output:", stream.output)
print("interrupted:", stream.interrupted)
```

7.9 Summary

streaming mode	Description	Use
`values`	Return full state after each step	Debugging, tracing state

`updates`	Return only the part changed by the node	Monitor progress
`messages`	LLM token real-time streaming	Chat UI implementation
`custom`	Send data from `get_stream_writer()`	Progress reporting
`stream_events(..., version="v3")`	Projection handles for messages, values, subgraphs, output, interrupts, extensions	App/UI integration

Next Steps

→ 08_interrupts_and_time_travel.ipynb: Learn interrupts and time travel.

08 Interrupts and Time Travel

Execute interrupt, Acknowledge, Rewind

Learning Objectives

Execute with `interrupt()`, interrupt, and resume with `Command(resume=...)`. Time travel back to the previous state.

- Human-in-the-loop pattern can be implemented
- Interrupt can also be used in Functional API
- You can perform time travel using checkpoint history.
- state can be modified externally with `update_state()`

8.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4-mini")
print("Model is ready")
```

8.2 interrupt() — executes interrupt and waits for human input

- `interrupt(value)`: Save the current state to the checkpoint and execute interrupt
- `Command(resume=value)`: Passes the value at interrupt point and resume

This pattern is used to obtain human approval or input additional information before performing sensitive tasks.

8.3 Command(resume=…) — interrupt executes resume

Using `Command(resume=value)` causes execution to resume at the point where `interrupt()` is called. The value passed to `resume` becomes the return value of `interrupt()`.

`Command(resume=...)` accepts boolean, text, or dict payloads. The pattern depends on the response shape:

```python
from langgraph.types import Command

# 1) Simple boolean — Approval
graph.invoke(Command(resume=True), config=config, version="v2")

# 2) Text — Review & Edit
graph.invoke(Command(resume="The edited text"), config=config, version="v2")

# 3) Structured dict — Tool interrupt / multi-field response
graph.invoke(
    Command(resume={"action": "approve", "subject": "Updated subject"}),
    config=config,
    version="v2",
)
```

Common Patterns — five HITL scenarios

The five interrupt patterns you'll see in production:

1) Approval — sensitive action gate

Pause right before a critical action (API call / DB mutation) and route based on the decision.

```python
from typing import Literal
from langgraph.types import Command, interrupt

def approval_node(state) -> Command[Literal["proceed", "cancel"]]:
    decision = interrupt({
        "question": "Approve this action?",
        "details": state["action_details"],
    })
    return Command(goto="proceed" if decision else "cancel")

graph.invoke(Command(resume=True), config=config, version="v2")   # approve
graph.invoke(Command(resume=False), config=config, version="v2")  # reject
```

Approval schema — typed resume object

For approval flows with more than a boolean, pass `response_schema=` to `interrupt`. When the graph resumes with a dict, LangGraph coerces it into the schema object, so the node reads attributes such as `decision.approved` rather than indexing a raw dict.

```python
from pydantic import BaseModel
from langgraph.types import Command, interrupt

class Approval(BaseModel):
    approved: bool
    reason: str | None = None

def approval_node(state) -> Command[Literal["proceed", "cancel"]]:
    decision = interrupt(
        {
            "question": "Approve this action?",
            "details": state["action_details"],
        },
        response_schema=Approval,
    )
    return Command(goto="proceed" if decision.approved else "cancel")

graph.invoke(
    Command(resume={"approved": True, "reason": "Looks safe"}),
    config=config,
    version="v2",
)
```

2) Review & Edit — human revises LLM output

```
def review_node(state):
    edited = interrupt({
        "instruction": "Review and edit this content",
        "content": state["generated_text"],
    })
    return {"generated_text": edited}

graph.invoke(Command(resume="edited body ..."), config=config, version="v2")
```

3) Tool Interrupt — approve before the tool runs

Put `interrupt()` inside a `@tool` to require human approval/edits. The resume payload is typically `{"action": "approve", ...}`.

```
from langchain.tools import tool

@tool
def send_email(to: str, subject: str, body: str):
    response = interrupt({
        "action": "send_email",
        "to": to, "subject": subject, "body": body,
        "message": "Approve sending this email?",
    })
    if response.get("action") == "approve":
        return f"Email sent to {response.get('to', to)}"
    return "Email cancelled by user"
```

4) Input Validation — loop until valid

```
def get_age_node(state):
    prompt = "What is your age?"
    while True:
        answer = interrupt(prompt)
        if isinstance(answer, int) and answer > 0:
            break
        prompt = f"'{answer}' is not valid. Please enter a positive number."
    return {"age": answer}
```

5) Multiple Interrupts — match parallel responses

When parallel nodes raise interrupts simultaneously, use a resume map keyed by interrupt id:

```
result = graph.invoke({"vals": []}, config, version="v2")

resume_map = {
    i.id: f"answer for {i.value}" for i in result.interrupts
}
graph.invoke(Command(resume=resume_map), config, version="v2")
```

8.4 Interrupt in Functional API

You can also use `interrupt()` in the Functional API (`@entrypoint`, `@task`).

8.5 Time Travel — Go back to a previous checkpoint

The checkpoint system in LangGraph stores all executions of state. You can view previous checkpoints with `get_state_history()` and go back to a specific point in time.

8.6 update_state() — Time travel + state fix

`update_state()` allows you to directly modify the state of a graph from the outside. This is useful for debugging, testing, or when manual intervention is required.

Replay vs Fork — two time-travel modes

- **Replay** — re-run from a previous checkpoint as-is. Nodes after `next` are re-executed (LLM/API calls happen again — results may differ).
- **Fork** — modify state at a previous checkpoint and create a new branch. The original thread is preserved.

```
# Replay — invoke with the checkpoint config
history = list(graph.get_state_history(config))
before_joke = next(s for s in history if s.next == ("write_joke",))
replay_result = graph.invoke(None, before_joke.config)

# Fork — create a new branch with update_state
fork_config = graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},
)
fork_result = graph.invoke(None, fork_config)
```

> ➡️ TIP
>
> Pass the return value of `update_state()` (`fork_config`) to `invoke()` , not the original `before.config` . The new checkpoint id lives inside `fork_config` .

`as_node` — declare which node owns the update

`update_state(config, values, as_node="...")` lets you declare which node should be treated as the source of the update. Useful when:

- parallel branches make ownership ambiguous,
- seeding initial state into a fresh thread for a test, or
- skipping a node and resuming from its successor.

```
fork_config = graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},
    as_node="generate_topic",  # resume from this node's successor
)
graph.invoke(None, fork_config)
```

Reducer behavior — merge vs overwrite

How `values` is applied depends on whether the channel has a reducer.

- Channel with reducer (e.g. `MessagesState.messages` uses `add_messages`) → values are merged; appending messages accumulates.
- Channel without reducer (plain `TypedDict` fields) → values are overwritten.

To replace a single message, send `RemoveMessage(id=...)` plus the new one. Plain dict fields can be replaced directly with `values={"field": new_value}`.

8.7 Summary

Summarize the key functions learned in this Note book.

Features	API	Description
`interrupt(value)`	Both sides	run interrupt, pass value
`Command(resume=value)`	Both sides	resume at point interrupt
`get_state_history()`	Graph	Checkpoint history inquiry
`update_state()`	Graph	Modify state externally

interrupts and time travel are key features in production AI applications:

- interrupt: Get human approval before sensitive operations
- Time Travel: You can go back to the previous state and explore different routes
- update_state: You can adjust the execution flow by modifying state externally.

Next Steps

→ 09_subgraphs.ipynb: Learn subgraphs.

09 Subgraph

Graph within a graph

Learning Objectives

Modularize complex workflow with subgraphs.

9.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4-mini")
```

9.2 Subgraph concept

- Subgraph: Independent graph used as a node in another graph
- Advantages: Modularization, reuse, independent development by team
- Each subgraph has its own state(State)
- state between parent <-> subgraph is mapped to shared key

9.3 Creating a subgraph

9.4 Adding a subgraph to the parent graph

9.4.1 Pattern 1: Subgraph call through wrapper node (if `state_schema` is different)

In the above example (9.4), the parent graph and subgraph shared the same keys (`text` , `word_count` , `char_count`), so the compiled subgraph could be passed directly to `add_node()` .

However, in practice, the `state_schema` of the parent graph and subgraph is often completely different. In this case, use a **wrapper function**:

1. Extract the required fields from the parent state and convert them to subgraph input.
2. Run the subgraph
3. Map the subgraph output to the parent state format.

Use the pattern. This is how the official documentation calls it Pattern 1: Call Subgraph Inside a Node.

9.5 LLM-based subgraph

Organize the full text agent into subgraphs.

9.6 Subgraph streaming

Steps inside a subgraph are also streaming possible.

With v2 streaming (`version="v2"`) subgraph chunks arrive as a uniform `StreamPart` dict. Three fields cover everything:

```
for chunk in graph.stream(
    {"foo": "foo"},
    subgraphs=True,
    stream_mode="updates",
    version="v2",
):
    print(chunk["type"])  # "updates"
    print(chunk["ns"])    # () = root, ("node_2:<id>",) = subgraph
    print(chunk["data"])  # {"node_name": {"key": "value"}}
```

- `chunk["type"]` — mode identifier (`"updates"` , `"values"` , `"messages"` , …)
- `chunk["ns"]` — namespace tuple. `()` for root, `("node_2:<uuid>",)` for a subgraph
- `chunk["data"]` — mode-specific payload

> ➡️ TIP
>
> In v1 the return shape changed (tuple vs dict) depending on `subgraphs=True` . v2 always returns the same dict, so root- and subgraph-handling code can be unified.

Subgraph checkpointer — three modes

The `checkpointer=` argument on subgraph compile controls memory and interrupt behavior.

Mode	`checkpointer=`	Behavior	Fit
Per-invocation (default)	`None`	Fresh per call; inherits parent's checkpointer	Tool-wrapped subagents, multi-agent routing

		for interrupts within a single invocation	
Per-thread (stateful)	`True`	Accumulates state across calls on the same thread	Specialized subagents with long-running conversation
Stateless	`False`	No checkpointing — plain function call; no interrupt support	Pure transforms with no side effects

> **! WARNING**
>
> Per-thread subgraphs do not support parallel tool calls. When the LLM tries to invoke the same subagent in parallel, both calls write to the same namespace and conflict. Use `ToolCallLimitMiddleware(tool_name="...", run_limit=1)` to cap simultaneous invocations.

Namespace isolation — multiple per-thread subgraphs

When several per-thread subgraphs coexist, wrap each one in its own `StateGraph` with a unique node name so namespaces don't collide. `StateGraph(MessagesState).add_node(name, agent)` is the standard pattern.

```python
from langgraph.graph import MessagesState, StateGraph
from langchain.agents import create_agent

def create_sub_agent(model, *, name, **kwargs):
    agent = create_agent(model=model, name=name, **kwargs)
    return (
        StateGraph(MessagesState)
        .add_node(name, agent)          # unique name → stable namespace
        .add_edge("__start__", name)
        .compile()
    )

fruit_agent = create_sub_agent(
    "gpt-5.4-mini", name="fruit_agent",
    tools=[fruit_info], prompt="You are a fruit expert.",
    checkpointer=True,
)
veggie_agent = create_sub_agent(
    "gpt-5.4-mini", name="veggie_agent",
    tools=[veggie_info], prompt="You are a veggie expert.",
    checkpointer=True,
)
```

Each subagent writes checkpoints only under its own name, so subgraphs never collide.

9.7 Summary

concept	Description

Subgraph	Using independently compiled graphs as nodes
shared key	state mapping between parent and subgraph
Modularization	Separating complex workflow into smaller units
streaming	Track internal steps with `subgraphs=True`

Next Steps

→ 10_production.ipynb: Learn production deployment.

10 Production

testing, deployment, observability

Learning Objectives

LangGraph Learn how to test, deploy, and monitor your apps.

10.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

10.2 App structure — langgraph.json

- `langgraph.json` : Graph definition, dependencies, Environment Variables settings
- `langgraph dev` : Run local development server

```
import json

config = {
    "dependencies": ["."],
    "graphs": {
        "agent": "./agent.py:graph"
    },
    "env": ".env"
}
print("langgraph.json example:")
print(json.dumps(config, indent=2))
print()
print("Command:")
print("  $ pip install 'langgraph-cli[inmem]'")
print("$ langgraph dev  # starts the local server at http://localhost:2024")
```

10.3 LangGraph Studio — Visual Debugging tool

Studio is automatically provided when you run `langgraph dev` .

Key Features (7):

1. Real-time Visualization — every step the agent takes (prompts, tool calls, results, final output) is rendered live.
2. Interactive Testing — drive different inputs and inspect intermediate states directly in the UI.
3. Hot-reloading — edits to prompts or tool signatures are reflected immediately without restarting the server.
4. Trace Inspection — execution traces include prompts, tool arguments, return values, token counts, and latency.
5. Exception Capture — exceptions are captured together with surrounding state for debugging context.
6. Thread Replay — re-run conversation threads from any step to validate changes without restarting.
7. Optional Tracing — set `LANGSMITH_TRACING=false` to keep run data on the local machine.

How to use:

```
$ langgraph dev
# Open https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
# Safari users append --tunnel
```

10.4 Agent Chat UI

Agent Chat UI is a Next.js chat interface for LangChain agents. It integrates directly with agents built using `create_agent` and connects to either a `langgraph dev` server or a deployed server.

Install — npx (recommended):

```
$ npx create-agent-chat-app --project-name my-chat-ui
$ cd my-chat-ui
$ pnpm install
$ pnpm dev
```

Hosted version: visit agentchat.vercel.app and enter your agent's deployment URL or local server address.

Connection info:

- Graph ID — the key in the `graphs` section of `langgraph.json`
- Deployment URL — the agent server URL (use `http://localhost:2024` for local)
- LangSmith API key — optional (not required when using a local server)

Features:

- Real-time streaming chat
- Tool call / result rendering
- Time-travel debugging, state forking
- Human-in-the-loop interrupt handling
- Generative UI support

10.5 Test — Deterministic agent test

```python
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

# Graph to test
class TestState(TypedDict):
    input: str
    output: str


def process(state: TestState) -> dict:
    return {"output": state["input"].upper()}


builder = StateGraph(TestState)

builder.add_node("process", process)
builder.add_edge(START, "process")
builder.add_edge("process", END)

graph = builder.compile()


# Unit tests
def test_process():
    result = graph.invoke({"input": "hello"})

    assert result["output"] == "HELLO", f"Expected HELLO, got {result['output']}"

    print("  OK test_process")


def test_empty_input():
    result = graph.invoke({"input": ""})

    assert result["output"] == "", f"Expected an empty string, got {result['output']}"

    print("  OK test_empty_input")


print("Running tests:")

test_process()
test_empty_input()

print("All tests passed!")
```

10.6 LLM agent Test — Using GenericFakeChatModel

```python
from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage, HumanMessage, AnyMessage
from langgraph.graph import StateGraph, START, END, MessagesState

# Deterministic fake model
fake_model = GenericFakeChatModel(
    messages=iter(
        [
            AIMessage(content="The answer is 42."),
        ]
    )
)

def chatbot(state: MessagesState) -> dict:
```

```
    return {
        "messages": [fake_model.invoke(state["messages"])]
    }

builder = StateGraph(MessagesState)

builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

test_graph = builder.compile()

result = test_graph.invoke(
    {
        "messages": [HumanMessage(content="test")]
    }
)

assert "42" in result["messages"][-1].content

print("GenericFakeChatModel test passed!")
print(f"  Response: {result['messages'][-1].content}")
```

10.7 Deployment Options

1. LangSmith Cloud (managed):

Connect a GitHub repository in LangSmith Deployments and the platform deploys automatically (around 15 minutes):

1. Push your application code to a GitHub repository (public or private)
2. In LangSmith, open Deployments and click “+ New Deployment”
3. For private repos, connect your GitHub account, then select and submit the repository
4. After deployment, open Studio from the deployment page and copy the API URL from Deployment details

2. Self-hosted Docker:

```
$ langgraph build -t my-agent
$ docker run -p 2024:2024 my-agent
```

3. Calling the deployed API (Python SDK):

Pass the deployment URL and your LangSmith API key to the same SDK used locally.

```
from langgraph_sdk import get_sync_client

client = get_sync_client(url="your-deployment-url", api_key="your-langsmith-api-key")

for chunk in client.runs.stream(
    None,                                           # thread_id=None creates a stateless run
    "agent",                                        # graph name in langgraph.json
    input={"messages": [{"role": "human", "content": "What is LangGraph?"}]},
    stream_mode="updates",
):
    print(f"Receiving new event of type: {chunk.event}...")
    print(chunk.data)
```

REST call: deployed servers authenticate via the `X-Api-Key` header.

```
curl -s --request POST \
    --url <DEPLOYMENT_URL>/runs/stream \
    --header 'Content-Type: application/json' \
    --header "X-Api-Key: <LANGSMITH API KEY>" \
    --data '{"assistant_id": "agent", "input": {"messages": [{"role": "human", "content": "What is
LangGraph?"}]}, "stream_mode": "updates"}'
```

10.8 Observability — LangSmith Tracing

Settings (`.env`):

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2-...
LANGSMITH_PROJECT=my-agent-project   # optional, defaults to "default"
```

`LANGSMITH_TRACING` and `LANGSMITH_API_KEY` are required; `LANGSMITH_PROJECT` is optional.

Automatically tracked items:

- Each node execution time
- LLM input/output, token usage
- Tool calls and results
- State changes
- Errors and retries

Node trace policy

When a node handles large or sensitive state, configure tracing at the node boundary instead of turning off tracing for the whole graph. `trace_policy` lets the graph record the span while omitting or transforming payloads.

```
from langchain.agents.middleware import TracePolicy, omit_payload
from langgraph.graph import StateGraph

builder = StateGraph(TestState)
builder.add_node(
    "process_private_payload",
    process,
    trace_policy=TracePolicy(
        process_inputs=omit_payload,
        process_outputs=omit_payload,
    ),
)
```

Use this for payload minimization: the trace still shows that the node ran, but the raw state does not get copied into LangSmith.

Selective tracing — `tracing_context`

Use the `langsmith.tracing_context` context manager to toggle tracing for specific operations and to attach project / tags / metadata dynamically.

```
import langsmith as ls
```

```python
# Only this call is traced
with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "Send a test email"}]})

# Dynamic project + tags + metadata
with ls.tracing_context(
    project_name="email-agent-test",
    enabled=True,
    tags=["production", "email-assistant", "v1.0"],
    metadata={"user_id": "user_123", "session_id": "session_456"},
):
    agent.invoke({"messages": [{"role": "user", "content": "Send a welcome email"}]})
```

You can also pass tags and metadata through the `config` argument of `invoke()`:

```python
agent.invoke(
    {"messages": [{"role": "user", "content": "Send a welcome email"}]},
    config={
        "tags": ["production", "email-assistant", "v1.0"],
        "metadata": {"user_id": "user_123", "session_id": "session_456"},
    },
)
```

Data privacy — `LangChainTracer` + `with_config`

To avoid leaking sensitive data into traces, wire an anonymized `Client` into a `LangChainTracer` and attach it to the compiled graph via `.with_config({"callbacks": [tracer]})`.

```python
from langchain_core.tracers.langchain import LangChainTracer
from langgraph.graph import StateGraph, MessagesState
from langsmith import Client
from langsmith.anonymizer import create_anonymizer

anonymizer = create_anonymizer([
    {"pattern": r"\b\d{3}-?\d{2}-?\d{4}\b", "replace": "<ssn>"},
])

tracer_client = Client(anonymizer=anonymizer)
tracer = LangChainTracer(client=tracer_client)

graph = (
    StateGraph(MessagesState)
    # .add_node(...).add_edge(...)
    .compile()
    .with_config({"callbacks": [tracer]})
)
```

This pattern masks identifiable patterns (e.g., SSNs) just before sending traces to LangSmith.

10.9 Pregel Runtime Overview

- Pregel is the internal execution engine of LangGraph
- Both Graph API and Functional API run on Pregel
- Key concepts: superstep, Channel, Checkpoint
- superstep: Unit in which nodes of the same level are executed in parallel
- Generally no need to use it directly (Graph/Functional API abstracts it)

LangGraph Execution Model: [Super-step 1] Node A, Node B (parallel) ↓ State update [Super-step 2] Node C (based on the results of A and B) ↓ State update [Super-step 3] Node D ↓ END

```
*Each superstep:*
1. Parallel execution of relevant nodes
2. Update state (apply reducer)
3. Save checkpoint
4. Next superstep decision
```

10.10 Production Checklist

Item	tool	Description
unit testing	pytest	Test individual node functions
Integration Testing	GenericFakeChatModel	Full flow without API calls
persistence	PostgreSaver	Production checkpointer
observability	LangSmith	Tracing, Monitoring
Distribution	langgraph deploy	Managed Deployment
UI	Agent Chat UI	User Interface

10.11 Summary

Topic	Key Concepts
App Structure	Set up project with `langgraph.json`
Studio	Visual debugging with `langgraph dev`
test	Deterministic Testing + GenericFakeChatModel
Distribution	Platform, Docker, Cloud options
observability	LangSmith Tracing
runtime	Pregel superstep Execution Model → Deeper in Chapter 13

Next Steps

→ Proceed to

11. Local Server

! → Skip to Deep Agents track

11 Local Server

Learning Objectives

- `langgraph dev` Know how to run a development server with CLI
- LangGraph Visually debug in conjunction with Studio
- Create `langgraph.json` configuration file
- Call local server with Python SDK
- Understand the deployment preparation process

11.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

11.2 LangGraph CLI installation

LangGraph To run a local server, you must first install the CLI. The `langgraph-cli[inmem]` package includes an in-memory mode and is suitable for development/testing.

```
# LangGraph CLI installation command
print("=== Install with pip (Python >= 3.11) ===")
print('  $ pip install -U "langgraph-cli[inmem]"')
print()
print("=== Install with uv ===")
print('  $ uv add "langgraph-cli[inmem]"')
print()
print("Check after installation:")
print("  $ langgraph --version")
```

11.3 Project creation

You can create a project template with the `langgraph new` command of the LangGraph CLI. If you do not specify a template, an interactive menu is displayed.

```
# Project creation command
print("=== Create a new project with template ===")
print("  $ langgraph new my-agent --template new-langgraph-project-python")
print()
print("=== Create an interactive menu ===")
print("  $ langgraph new my-agent")
print()
print("=== Install dependencies after creation ===")
print("# use pip")
print("  $ cd my-agent && pip install -e .")
print()
print("# use uv")
print("  $ cd my-agent && uv sync")
print()
print("=== Generated file structure ===")
print("  my-agent/")
print("├── langgraph.json # Graph settings")
print("├── .env.example # Environment Variables template")
print("├── pyproject.toml # Dependency definition")
print("  └── src/")
print("└── agent.py # agent code")
```

11.4 langgraph.json settings

`langgraph.json` is the core configuration file for the LangGraph project. Specifies graph definition location, dependencies, Environment Variables files, etc.

```
import json

# langgraph.json configuration example
config = {
    "dependencies": ["."],
    "graphs": {
        "agent": "./src/agent.py:graph"
    },
    "env": ".env"
}

print("langgraph.json example:")
print(json.dumps(config, indent=2))
print()
print("Main fields:")
print('dependencies: list of package paths to install')
print('graphs: graph name → module:variable mapping')
print('env: Environment Variables file path')
```

11.5 Running the development server

Run the local development server with the `langgraph dev` command. `$ langgraph dev`

```
**Expected output:**
Ready!
- API: http://127.0.0.1:2024
- Docs: http://127.0.0.1:2024/docs
- Studio: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
```

URL	Use

`http://127.0.0.1:2024`	API endpoint
`http://127.0.0.1:2024/docs`	API Documentation (Swagger)
`https://smith.langchain.com/studio/...`	LangGraph Studio Interface

> ➡️ TIP
>
> Safari users: Use the `langgraph dev --tunnel` flag to establish a secure connection to the localhost server.

11.6 LangGraph Studio integration

LangGraph Studio is a visual debugging tool provided automatically when you run `langgraph dev`.

Main Features:

Features	Description
Graph visualization	Identify node and edge structures at a glance
Real-time tracking	Observe the execution process of each node in real time
state Inspection	Check/edit the state(state) value of each step
interactive testing	Test graph execution by changing input

Studio is browser-based, so no separate installation is required. If you are using a custom server address, simply change the `baseUrl` parameter in the Studio URL.

11.7 Python SDK — Asynchronous client

Asynchronous clients are created with `langgraph_sdk.get_client()`. It operates based on `asyncio` and can efficiently process streaming responses.

```
# Asynchronous client usage pattern (used when the server is running)
print("""from langgraph_sdk import get_client
import asyncio

client = get_client(url="http://localhost:2024")

async def main():
    async for chunk in client.runs.stream(
        None,
        "agent",
        input={
            "messages": [{
                "role": "human",
                "content": "What is LangGraph?",
            }],
```

```
        },
    ):
        print(f"Event type: {chunk.event}...")
        print(chunk.data)

asyncio.run(main())
""")
print("# The above code is used when the langgraph dev server is running.")
```

11.8 Python SDK — Synchronous client

Synchronous clients are created with `langgraph_sdk.get_sync_client()`. It is suitable for simple scripts or tests that do not require asynchrony.

```
# Synchronous client usage pattern (used when the server is running)
print("""from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

for chunk in client.runs.stream(
    None,
    "agent",
    input={
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
        }],
    },
    stream_mode="messages-tuple",
):
    print(f"Event: {chunk.event}...")
    print(chunk.data)
""")
print("# The above code is used when the langgraph dev server is running.")
```

11.9 REST API call

LangGraph The local server provides a REST API. It can be called directly with `curl` or with an HTTP client. curl -s -request POST
-url "http://localhost:2024/runs/stream"
-header 'Content-Type: application/json'
-data '{ "assistant_id": "agent", "input": { "messages": [{ "role": "human", "content": "What is LangGraph?" }] }, "stream_mode": "messages-tuple" }'

```
API documentation can be found at `http://localhost:2024/docs`.
```

11.10 Deployment Readiness Checklist

Once local development is complete, prepare for production deployment.

Item	Description	OK
`langgraph.json`	Graph path·dependency·Environment Variables setup complete	☐
`.env` file	API key, etc. Environment Variables settings	☐
Dependency cleanup	`pyproject.toml` or `requirements.txt` cleanup	☐
local test	Normal locally with `langgraph dev` Smoke Test	☐
Check Studio	LangGraph Check graph structure in Studio	☐
SDK Testing	Test calls with Python SDK or REST API	☐
persistent storage	Setting checkpointer (e.g. PostgresSaver) for production	☐
observability	LangSmith or Langfuse tracing settings	☐

11.11 Summary

Topic	Key Concepts
Install CLI	`pip install "langgraph-cli[inmem]"` or `uv add`
Create project	Create template-based project with `langgraph new`
langgraph.json	Configuration file defining graph paths, dependencies, Environment Variables
Studio	`langgraph dev` Visual debugging provided automatically at run time tool
SDK asynchronous	Asynchronous call streaming with `get_client()`
SDK synchronization	Synchronous call to streaming with `get_sync_client()`
REST API	Direct call to `/runs/stream` endpoint with `curl`

Next Steps

→ Proceed to

12. Durable Execution

!

References:

- Run a Local Server

12 durable execution

Learning Objectives

- Understand the concept and necessity of durable execution (Durable Execution)
- Know the relationship between checkpointer and durable execution
- Learn how to ensure durability with `@entrypoint` + `@task`
- Understand the difference between endurance modes (exit, async, sync)
- Know the recovery process in failure scenarios

12.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

12.2 durable execution Concept

A process called durable execution(Durable Execution) or workflow saves progress state at key points, This is a technique that allows you to pause and then later interrupt at the exact resume position.

Why do you need it?

Scenario	Description
Disaster recovery	In case of server failure, instead of restarting from the beginning, interrupt point resume
state persistent	Preserve intermediate results of workflow with long running time
Human-in-the-loop	Keep state while waiting for human approval

LangGraph supports durable execution via checkpointer(checkpointer).

12.3 Core requirements

Implementing durable execution requires three elements:

1. Persistence Layer

Log state progressing workflow through checkpointer. Example: `InMemorySaver` (for development), `PostgresSaver` (for production)

1. Thread ID (Thread ID)

workflow A unique ID that tracks the execution history of the instance. Using the same `thread_id`, you can continue resume from previous executions.

1. `@task` **wrapping (task wrapping)**

Non-deterministic operations and side-effect operations Wrap it in `@task` to prevent re-execution on resume.

12.4 Endurance Mode Comparison

LangGraph provides three modes that balance performance and consistency:

mode	Action	Trade-offs
`"exit"`	Persist only on completion/error/interrupt	Highest performance, no intermediate recovery
`"async"`	Next Steps Persist asynchronously while running	Good balance, slight crash risk
`"sync"`	Next Steps Persist with pre-execution synchronization	Maximum durability, performance cost

For most use cases, the default mode (`"exit"`) is sufficient. For mission-critical workflow, consider `"sync"` mode.

12.5 Problematic code

If you do not wrap side effects (API calls, etc.) in `@task`, The same API call may be executed again at resume.

```
# Problematic approach: calling side effects directly
print("""# BAD: Side effects not wrapped in `@task`
def call_api(state: State):
    # This API call will be executed again on resume!
    result = requests.get(state['url']).text[:100]
    return {"result": result}""")
print("Problem:")
print("1. API is called again at resume after failure")
print("2. Non-deterministic results may vary")
print("3. Side effects may occur due to duplicate requests.")
```

12.6 Improvements to @task

If you wrap the side effect with the `@task` decorator, At resume, restore previous results from checkpoint to prevent re-execution.

```
# Improved approach: wrapping side effects with @task
print("""# GOOD: Wrap side effects with @task
from langgraph.func import task

@task
def _make_request(url: str):
    return requests.get(url).text[:100]

def call_api(state: State):
    # Each request is executed as a separate `@task`
    requests = [_make_request(url) for url in state['urls']]
    results = [req.result() for req in requests]
    return {"results": results}""")
print("Improvement effect:")
print("1. Restore results from checkpoint at resume")
print("2. Avoid duplicate API calls")
print("3. Each `@task` is tracked independently")
```

12.7 Durability in Graph API

Connecting checkpointer to StateGraph will automatically save state after each node execution.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver


class DocState(TypedDict):
    topic: str
    draft: str
    final: str


def write_draft(state: DocState) -> dict:
    return {"draft": f"Draft about {state['topic']}"}


def finalize(state: DocState) -> dict:
    return {"final": f"Final: {state['draft']}"}


checkpointer = InMemorySaver()

builder = StateGraph(DocState)
builder.add_node("write_draft", write_draft)
builder.add_node("finalize", finalize)
builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "finalize")
builder.add_edge("finalize", END)

graph = builder.compile(checkpointer=checkpointer)

# Execution (track execution by thread_id)
config = {"configurable": {"thread_id": "doc-1"}}
result = graph.invoke({"topic": "LangGraph"}, config)
print("result:", result)
```

12.8 Durability in Functional API

By combining `@entrypoint` and `@task`, durability can be guaranteed even in Functional API.

```
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver


@task
def generate_draft(topic: str) -> str:
    return f"Draft about {topic}"


@task
def review_draft(draft: str) -> str:
    return f"Reviewed: {draft}"


func_checkpointer = InMemorySaver()


@entrypoint(checkpointer=func_checkpointer)
def write_document(topic: str) -> str:
    draft = generate_draft(topic).result()
    reviewed = review_draft(draft).result()
    return reviewed


config = {"configurable": {"thread_id": "func-1"}}
result = write_document.invoke("Durable Execution", config)
print("result:", result)
```

12.9 Failover Scenario

If you rerun with the same `thread_id`, it restores the previous state from the checkpoint and continues execution.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver


class PipelineState(TypedDict):
    data: str
    step: int
    result: str


call_count = 0


def step_one(state: PipelineState) -> dict:
    global call_count
    call_count += 1
    print(f"  step_one executed (call count: {call_count})")
    return {"data": state["data"].upper(), "step": 1}


def step_two(state: PipelineState) -> dict:
    print("  step_two executed")
    return {"result": f"Processed: {state['data']}", "step": 2}


recovery_saver = InMemorySaver()
```

```
builder = StateGraph(PipelineState)
builder.add_node("step_one", step_one)
builder.add_node("step_two", step_two)
builder.add_edge(START, "step_one")
builder.add_edge("step_one", "step_two")
builder.add_edge("step_two", END)

pipeline = builder.compile(checkpointer=recovery_saver)

# first run
config = {"configurable": {"thread_id": "recovery-1"}}
print("=== First Run ===")
result = pipeline.invoke(
    {"data": "hello", "step": 0, "result": ""},
    config
)
print(f"Result: {result}")

# Check checkpoint
print("=== Restore state from checkpoint ===")
saved = pipeline.get_state(config)
print(f"Saved state: {saved.values}")
print(f"Total step_one call count: {call_count}")
```

12.10 resume Starting point

When workflow becomes resume, the starting point varies depending on the API:

API	resume Starting point	Description
StateGraph	Start of interrupt node	Rerun that node from scratch
Subgraph	Parent node → interrupt node in subgraph	Start from the parent node and then move to the corresponding node in the subgraph
Functional API	`\@entrypoint` Start	Starting at `\@entrypoint`, `\@task` results restored from cache

Key differences:

- StateGraph: Node unit resume (re-execute only interrupt nodes)
- Functional API: Rerun from `@entrypoint`, but use cache results for completed `@task`

12.11 Production Durability Pattern

Best practices for ensuring durability in a production environment:

1. Implementation of idempotent operation

Design the results to be the same even when executing the same request multiple times. Prevent duplicate processing by utilizing an idempotency key.

1. Isolate Side Effects

Separate side effects such as API calls, file writes, etc. into individual `@task`. Make a clear distinction between pure logic and side effects.

1. Non-deterministic code wrapping

Non-deterministic operations such as random number generation and timestamps are also wrapped in `@task`.

1. Use persistent storage

Developed by: `InMemorySaver` Production: `PostgresSaver` or external database

1. thread ID Management

Give each workflow instance a unique `thread_id`. Upon failover, use resume with the same `thread_id`.

12.12 Per-node fault tolerance — `TimeoutPolicy` + `heartbeat`

Durable execution preserves how far a run has progressed via checkpoints. The fault-tolerance features in LangGraph 1.2 control when to give up and how to compensate at the level of an individual node attempt. `add_node(..., timeout=TimeoutPolicy(...))` applies a per-attempt time budget; on overrun, `NodeTimeoutError` is raised and the retry policy (if any) advances to the next attempt. Partial writes from the failing attempt are discarded.

Long-running nodes combine `Runtime.heartbeat()` with `refresh_on="heartbeat"` to refresh the idle timeout.

```
from langgraph.func import task
from langgraph.runtime import Runtime
from langgraph.types import TimeoutPolicy

async def long_running_node(state: State, runtime: Runtime) -> dict:
    for batch in fetch_batches():
        process(batch)
        runtime.heartbeat()
    return {"result": "done"}

builder.add_node(
    "long_running_node",
    long_running_node,
    timeout=TimeoutPolicy(idle_timeout=30, refresh_on="heartbeat"),
)
```

12.13 Graceful shutdown — `RunControl` / `GraphDrained`

To stop a long-running graph cleanly on signals like SIGTERM — finishing the current superstep and persisting a checkpoint instead of force-killing the process — use `RunControl.request_drain()`. When drain is requested, the run stops with `GraphDrained` and can later be resumed with the same config. Inside a node, branch on `Runtime.drain_requested` to add safe-shutdown handling.

```
from langgraph.runtime import RunControl, Runtime
from langgraph.errors import GraphDrained
```

```python
# 1) Caller side --- request drain on an external signal (e.g., SIGTERM)
control = RunControl()
control.request_drain("sigterm")

try:
    result = graph.invoke(inputs, config, control=control)
except GraphDrained as e:
    print(f"Drained: {e.reason}")
    # The checkpoint has already been saved.

# Resume with the same config
graph.invoke(None, config)

# 2) Inside a node --- detect the drain signal directly
async def my_node(state: State, runtime: Runtime) -> dict:
    if runtime.drain_requested:
        return {"status": "skipped"}
    # ... normal processing
    return {"status": "done"}
```

> ➡️ TIP
>
> Drain is distinct from interrupt. Interrupt is an application-level signal that pauses to await user input; drain is an operational signal that stops cleanly at a superstep boundary. Combining both means rolling deploys and maintenance windows never lose in-flight progress.

12.14 DeltaChannel (beta) — checkpoint size optimization

Standard checkpoints persist the full value of every channel on each superstep. For append-heavy channels (e.g., message lists), storage grows quickly on long-lived threads. `DeltaChannel` (introduced in `langgraph≥1.2`) stores incremental deltas instead, and writes a full snapshot every `snapshot_frequency` steps to bound reconstruction cost.

```python
from typing import Annotated, Sequence
from typing_extensions import TypedDict
from langgraph.channels import DeltaChannel

def list_reducer(state: list, writes: Sequence[list]) -> list:
    # bulk reducer signature: (current_state, sequence_of_writes) -> new_state
    result = list(state)
    for write in writes:
        result.extend(write)
    return result

class State(TypedDict):
    messages: Annotated[
        list[str],
        DeltaChannel(list_reducer, snapshot_frequency=5),
    ]
```

> ! WARNING

> `DeltaChannel` is a `langgraph≥1.2` beta feature. The reducer must be associative and runs at reconstruction time, not at write time. Measure storage, recovery latency, and migration cost before adopting it in production. Treat `DeltaChannel` as a storage optimization and `TimeoutPolicy` / `error_handler` as failure-recovery controls — they are independent dimensions of the durability design.

12.15 Summary

Topic	Key Concepts
Durability concept	Execution technique that can resume at interrupt point
Core Requirements	persistence layer + thread ID + `\@task` wrapping
Endurance Mode	exit (default), async (balanced), sync (maximum durability)
@task	Prevent replay by wrapping side effects
Graph API	`checkpointer` Automatic saving for each node by connection
Functional API	Durability guaranteed with `\@entrypoint` + `\@task`
Disaster recovery	At checkpoint with same `thread_id` resume
Node timeout	`TimeoutPolicy(idle_timeout=..., refresh_on="heartbeat")` per-attempt budget
Graceful shutdown	`RunControl.request_drain()` → `GraphDrained` → resume with same config
DeltaChannel (beta)	`langgraph≥1.2`. Store deltas + `snapshot_frequency` to bound reconstruction

Next Steps

→ Proceed to

13. API Selection Guide and Pregel

!

References:

- Durable Execution

13 API Selection Guide and Pregel

Learning Objectives

- Compare the differences between Graph API and Functional API

- The same agent is implemented in both APIs.
- Understand the internal structure of Pregel runtime
- superstep Know the execution model
- Establish criteria for selecting the appropriate API for the project

13.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

13.2 Graph API vs Functional API Overview

LangGraph provides two APIs for building agent workflow. Both APIs run on top of the same Pregel runtime and can be used together in a single application.

Characteristics	Graph API	Functional API
Abstraction method	Graph composed of nodes and edges	decorator-based function
state Management	Explicit management with TypedDict schema	Local variables within function scope
control flow	Conditional Edge, Routing	General Python control statements (if/else, for)
Visualization	Automatic visualization of graph structure	LIMITED
Boilerplate	Relatively many	minimize

13.3 Quick Selection Guide

Situation	Recommendation API	Reason
Complex workflow visualization required	Graph API	Node/edge structure automatically generates diagram
parallel execution path	Graph API	Multiple nodes naturally run in parallel

multi-agent Team	Graph API	Clear separation of roles between agent
Minimal changes to existing code	Functional API	Just add a decorator
Simple Linear workflow	Functional API	Quick implementation without boilerplate
Rapid Prototyping	Functional API	`state_schema` No definition required

13.4 Graph API implementation

Write an essay → Implement scoring workflow with Graph API.

```
from typing import TypedDict
from langgraph.constants import START
from langgraph.graph import StateGraph


class Essay(TypedDict):
    topic: str
    content: str | None
    score: float | None


def write_essay(essay: Essay):
    return {"content": f"Essay about {essay['topic']}"}


def score_essay(essay: Essay):
    return {"score": 10}


builder = StateGraph(Essay)
builder.add_node(write_essay)
builder.add_node(score_essay)
builder.add_edge(START, "write_essay")
builder.add_edge("write_essay", "score_essay")

graph_app = builder.compile()

result = graph_app.invoke({"topic": "LangGraph"})
print("Graph API results:", result)
```

13.5 Functional API implementation

Implement the same essay workflow with Functional API.

```
from typing import TypedDict
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver


class EssayResult(TypedDict):
```

```
    topic: str
    content: str | None
    score: float | None


@task
def write_essay_func(topic: str) -> str:
    return f"Essay about {topic}"


@task
def score_essay_func(content: str) -> float:
    return 10


func_saver = InMemorySaver()


@entrypoint(checkpointer=func_saver)
def essay_pipeline(topic: str) -> dict:
    content = write_essay_func(topic).result()
    score = score_essay_func(content).result()
    return {"topic": topic, "content": content, "score": score}


config = {"configurable": {"thread_id": "essay-1"}}
result = essay_pipeline.invoke("LangGraph", config)
print("Functional API results:", result)
```

13.6 Comparative analysis

Compare the two implementations above side by side:

Item	Graph API	Functional API
state Definition	`TypedDict` schema required	optional (possible local variables)
node connection	`add_edge()`, `add_conditional_edges()`	Generic function call
checkpointer	`compile(checkpointer=...)`	`\@entrypoint(checkpointer=...)`
number of lines of code	Relatively many	Concise
Visualization	`graph.get_graph().draw_mermaid()` Support	LIMITED
parallel execution	Natural support with edge structure	`\@task` Support for parallel execution
Debugging	Check state per node in Studio	Function-level tracing

13.7 Combining two APIs

You can use both APIs in the same application. A common pattern is to use the Graph API for complex multi-agent coordination, and the Functional API for simpler linear data-processing flows.

Use Case	Recommended API
Complex multi-agent coordination	Graph API
Simple preprocessing or validation pipeline	Functional API
Need explicit routing and visualization	Graph API
Need minimal boilerplate over existing code	Functional API

> · NOTE
>
> It is normal to start with the Functional API for a small prototype and migrate to the Graph API as the workflow becomes more complex. The reverse is also possible when a graph turns out to be overdesigned.

13.8 Pregel Runtime Overview

Pregel is the internal execution engine of LangGraph. Compiling `StateGraph` or using `@entrypoint` creates a Pregel instance internally.

The name comes from Google's Pregel algorithm, which efficiently handles massively parallel graph computations.

Core Components:

Components	Role
Actor	Read data from the channel and write the processing results to the channel
Channel	Responsible for data communication between actors

3 steps of execution (every step):

1. Plan — Determine which actor to execute in this step
2. Execute — Execute selected actors in parallel (until completion, failure, or timeout)
3. Update — Update the channel with new values

It terminates when there are no actors to run or the maximum steps are reached.

13.9 Direct use of Pregel

There is generally no need to use Pregel directly; Let's look at a simple example to understand the inner workings.

```
from langgraph.channels import EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder

# Single node: repeat input twice
node1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b")
)

app = Pregel(
    nodes={"node1": node1},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
    },
    input_channels=["a"],
    output_channels=["b"],
)

result = app.invoke({"a": "foo"})
print("Pregel Results:", result)
# 'foo' + 'foo' = 'foofoo'
```

13.10 Channel Type

Pregel offers five channel types that map directly to Graph API state fields and reducers.

Type	Purpose
`LastValue`	Stores the most recent value. Default channel for inputs/outputs
`EphemeralValue`	Temporary value scoped to a single execution step
`Topic`	Configurable PubSub. Multiple values with optional dedup / accumulation
`BinaryOperatorAggregate`	Applies a binary operator to current value and update (running totals, etc.)
`DeltaChannel` (beta, `langgraph≥1.2`)	Stores incremental deltas per step. Suitable for fast-growing channels (e.g., message lists)

```
from langgraph.channels import (
    EphemeralValue,
    LastValue,
    Topic,
    BinaryOperatorAggregate,
)
```

```
from langgraph.pregel import Pregel, NodeBuilder

# --- 1. LastValue: Maintain only the latest value ---
node_lv = (
    NodeBuilder()
    .subscribe_only("input")
    .do(lambda x: x.upper())
    .write_to("output")
)

app_lv = Pregel(
    nodes={"node": node_lv},
    channels={
        "input": EphemeralValue(str),
        "output": LastValue(str),
    },
    input_channels=["input"],
    output_channels=["output"],
)
print("LastValue:", app_lv.invoke({"input": "hello"}))

# --- 2. Topic: Accumulating multiple values ---
node_t1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node_t2 = (
    NodeBuilder()
    .subscribe_to("b")
    .do(lambda x: x["b"] + x["b"])
    .write_to("c")
)

app_topic = Pregel(
    nodes={"node1": node_t1, "node2": node_t2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": Topic(str, accumulate=True),
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("Topic:", app_topic.invoke({"a": "foo"}))

# --- 3. BinaryOperatorAggregate: Apply reducer ---
def reducer(current, update):
    if current:
        return current + " | " + update
    return update

node_b1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node_b2 = (
    NodeBuilder()
    .subscribe_only("b")
    .do(lambda x: x + x)
    .write_to("c")
)

app_agg = Pregel(
    nodes={"node1": node_b1, "node2": node_b2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": BinaryOperatorAggregate(str, operator=reducer),
```

```
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("BinaryOperatorAggregate:", app_agg.invoke({"a": "foo"}))
```

DeltaChannel (beta, `langgraph≥1.2`)

`DeltaChannel` stores incremental deltas only instead of the full accumulated value at each step. It is used to reduce checkpoint cost for high-volume, ever-growing channels such as message lists.

- The reducer must be associative and runs at reconstruction time, not at write time.
- `snapshot_frequency=K` writes a full snapshot every K steps to bound reconstruction cost.
- The bulk reducer signature is `(current_state, sequence_of_writes) → new_state`.

```
from typing import Annotated, Sequence
from typing_extensions import TypedDict
from langgraph.channels import DeltaChannel

def list_reducer(state: list, writes: Sequence[list]) -> list:
    # (current_state, sequence_of_writes) -> new_state
    result = list(state)
    for write in writes:
        result.extend(write)
    return result

class State(TypedDict):
    messages: Annotated[
        list[str],
        DeltaChannel(list_reducer, snapshot_frequency=5),
    ]
```

13.11 superstep Execution Model

Pregel runs in **supersteps**. In each superstep, nodes (actors) at the same level can run in parallel. Once all of them finish, the channel state is updated and the runtime moves to the next superstep.

> · NOTE
>
> Features of a superstep: nodes in the same superstep cannot see each other's new outputs, the next step begins only after all nodes finish, a checkpointer can persist state after each step, and execution stops automatically when there are no actors left to run.

A simple mental model looks like this:

- Superstep 1: Node A and Node B run in parallel
- Channel update
- Superstep 2: Node C runs using the results from A and B
- Channel update
- Superstep 3: Node D runs

- End

13.12 API Selection Criteria Guide

Here is the decision framework for your final choice:

Step 1: Complexity evaluation
- Less than 3 nodes and linear flow → Functional API
- Conditional branching, parallel path, circular structure → Graph API

Step 2: Team Collaboration
- Alone or with a small team → Either way is possible
- Multiple team members are responsible for each node → Graph API (separation of visualization and roles)

Step 3: Leverage Existing Code
- Add LangGraph function to existing procedural code → Functional API
- Designing a new workflow from scratch → Graph API

Step 4: Potential for development
- Start from a prototype → Functional API → When it becomes complex, migrate to Graph API
- Consider scalability from the beginning → Graph API

13.13 Summary

Topic	Key Concepts
Graph API	Node/edge based, strong in visualization, suitable for complex workflow
Functional API	Decorator-based, minimal boilerplate, suitable for linear workflow
Compare	Same runtime, can be used together, choose based on complexity
Pregel	LangGraph's internal execution engine, actor-channel model
Channel	`LastValue` / `EphemeralValue` / `Topic` / `BinaryOperatorAggregate` / `DeltaChannel` (beta)
superstep	Parallel execution of same level nodes → Channel update → Next step
Selection criteria	Judging by complexity, team collaboration, existing code, and development potential

Next Steps
→ Proceed to the Deep Agents track!

References:

- Choosing between Graph and Functional APIs
- Pregel Runtime

14 Fault Tolerance

RetryPolicy, Timeout, Error Handler

Learning goals

- Distinguish LangGraph retries, timeouts, and error handlers
- Use `RetryPolicy` to absorb transient failures
- Observe async node timeout and `NodeTimeoutError` safely
- Route fallback recovery with `error_handler` and `Command`
- Understand `set_node_defaults()` and node-level override precedence

Overview

Feature	Core API	Use case
Retries	`RetryPolicy`	Network/rate-limit/transient 5xx failures
Timeouts	`timeout=...`, `NodeTimeoutError`	External calls that hang too long
Error handling	`error_handler=...`, `Command`	Recovery and fallback routing
Graph defaults	`set_node_defaults(...)`	Shared policy across many nodes

This chapter reproduces failures with deterministic fake nodes instead of relying on slow or unreliable external APIs.

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
# LangSmith — graph runs are logged when LANGSMITH_TRACING=true.
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_PROJECT", "langchain-langgraph-deepagents-notebooks")
```

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import RetryPolicy, Command
from langgraph.errors import NodeError, NodeTimeoutError
```

1) Retries — let the system absorb transient failures

Attach `RetryPolicy` to work that is safe to retry with the same input. This fake node fails once, then succeeds on the second attempt.

```
class FaultState(TypedDict):
    result: str
    attempts: int
    error: str

calls = {"n": 0}

def flaky_api(state: FaultState) -> dict:
    calls["n"] += 1
    if calls["n"] < 2:
        raise ValueError("temporary network error")
    return {"result": "ok", "attempts": calls["n"]}
```

```
retry_graph = (
    StateGraph(FaultState)
    .add_node("flaky_api", flaky_api, retry_policy=RetryPolicy(
        max_attempts=3, initial_interval=0.01, jitter=False, retry_on=ValueError,
    ))
    .add_edge(START, "flaky_api")
    .add_edge("flaky_api", END)
    .compile()
)
```

```
retry_result = retry_graph.invoke({"result": "", "attempts": 0, "error": ""})
print(retry_result)
assert retry_result["attempts"] == 2
```

2) Timeouts — stop async nodes that hang

LangGraph node timeouts are supported for async nodes. Sync Python functions cannot be safely cancelled in-process, so keep timeout demonstrations async.

```
import asyncio

async def slow_api(state: FaultState) -> dict:
    await asyncio.sleep(0.05)
    return {"result": "too late"}

timeout_graph = (
    StateGraph(FaultState)
    .add_node("slow_api", slow_api, timeout=0.01)
    .add_edge(START, "slow_api")
    .add_edge("slow_api", END)
    .compile()
)
```

```
try:
    await timeout_graph.ainvoke({"result": "", "attempts": 0, "error": ""})
except NodeTimeoutError as exc:
    print(type(exc).__name__, str(exc).split("(")[0].strip())
```

3) Error handlers — route failures to a fallback path

When retries are exhausted, or when a failure should not be retried, `error_handler` can return `Command(update=..., goto=...)` to update state and continue on a recovery path.

```
def broken_service(state: FaultState) -> dict:
    raise RuntimeError("downstream unavailable")

def recover(state: FaultState, error: NodeError) -> Command:
    return Command(
        update={"error": str(error.error), "result": "fallback"},
        goto="finalize",
    )

def finalize(state: FaultState) -> dict:
    return {"result": state["result"] + " | finalized"}
```

```
handler_graph = (
    StateGraph(FaultState)
    .add_node("broken_service", broken_service, error_handler=recover)
    .add_node("finalize", finalize)
    .add_edge(START, "broken_service")
    .add_edge("finalize", END)
    .compile()
)
handler_result = handler_graph.invoke({"result": "", "attempts": 0, "error": ""})
print(handler_result)
```

4) Graph defaults — shared policy and node overrides

Use `set_node_defaults()` when multiple nodes share retry or timeout policy. A node-level setting overrides the graph default when both are present.

```
def stable_node(state: FaultState) -> dict:
    return {"result": "stable"}

common_retry = RetryPolicy(
    max_attempts=2, initial_interval=0.01, jitter=False, retry_on=ValueError,
)
default_graph = (
    StateGraph(FaultState)
    .set_node_defaults(retry_policy=common_retry)
    .add_node("stable", stable_node)
    .add_edge(START, "stable")
    .add_edge("stable", END)
    .compile()
)
print(default_graph.invoke({"result": "", "attempts": 0, "error": ""}))
```

Operations checklist

Question	Decision criteria

Is it safe to retry?	Idempotency and duplicate write/charge risk
How long should the node wait?	User SLA, external API p95, graph-level timeout
Does failure need compensation?	Cancel, refund, fallback, user-facing notice
Where should defaults live?	Global defaults versus node-specific overrides

---

References:

- LangGraph Fault Tolerance: https://docs.langchain.com/oss/python/langgraph/fault-tolerance
- LangGraph Durable Execution: ../../docs/langgraph/06-durable-execution.md
- LangGraph Graph API: ../../docs/langgraph/19-graph-api.md

15 Backward Compatibility

Handle LangGraph version changes safely

LangGraph applications often live longer than the version of LangGraph they were first written against. This chapter turns compatibility into an explicit engineering practice rather than an after-the-fact debugging task.

Learning goals

- Separate import compatibility from behavioral compatibility.
- Build migration checklists and version matrices.
- Use smoke tests to make framework upgrades safer.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
import inspect
from langgraph.graph import StateGraph
from langgraph.types import RetryPolicy

features = {
    "add_node_retry_policy": "retry_policy" in inspect.signature(StateGraph.add_node).parameters,
    "retry_policy_class": RetryPolicy.__name__,
}
features
```

15.1 Compatibility is more than successful imports

A graph can import successfully and still behave differently after an upgrade. Treat compatibility as a combination of imports, state shape, checkpoint behavior, and runtime semantics.

```
def require_feature(name: str, ok: bool) -> str:
    if not ok:
        return f"SKIP: {name} is not available"
    return f"OK: {name}"

require_feature("add_node.retry_policy", features["add_node_retry_policy"])
```

15.2 Migration checklist

A migration checklist keeps upgrades concrete. It forces you to review changelogs, deprecated APIs, smoke tests, and representative examples before changing production code.

```
migration_checklist = [
    "Review the official changelog",
    "Run existing notebook smoke tests",
    "Search for deprecated imports",
    "Add one new API example",
]

for item in migration_checklist:
    print("-", item)
```

15.3 Compatibility matrix

A version matrix makes support boundaries visible. It records which application paths were tested against which framework versions and what evidence supports the decision.

Summary

Item	Content
Covered	feature detection, migration notes, changelog review, and version-safe smoke tests
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `backward-compatibility.md`
- `changelog-py.md`
- `test.md`

16 Case Studies

Read official examples as design patterns

Official examples are more than snippets to copy; they are compact design studies. This chapter shows how to read them for state design, node boundaries, routing choices, and evaluation criteria.

Learning goals

- Extract reusable architecture patterns from examples.
- Describe state, nodes, and routing decisions explicitly.
- Capture design decisions in a short record before implementation.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

16.1 Case-study review template

Use the same review template each time you study an example. Consistent questions make it easier to compare architectures across different use cases.

```
review_template = {
    "problem": "What loop, branching, or state problem does this case solve?",
    "state": "Which state must be preserved over time?",
    "nodes": "How are deterministic nodes separated from agentic nodes?",
    "eval": "What counts as success?",
}

review_template
```

16.2 Break down a sample case

A case study becomes useful when you translate it into problem, state, node, and evaluation choices. This practice helps prevent cargo-cult copying.

```
case = {
    "problem": "The route changes by request type: billing, technical, or general.",
    "state": ["message", "owner", "resolution"],
    "nodes": ["triage", "resolve", "approval"],
    "eval": "owner routing accuracy and resolution completeness",
}

case
```

16.3 Design decision record

A short decision record preserves why a pattern was chosen. Future maintainers can then revisit the tradeoff without reverse-engineering the original reasoning.

```
decision = {
    "chosen": "StateGraph",
    "because": "handoff state and approval gates must be explicit",
    "rejected": "single create_agent loop",
    "risk": "routing policy drift",
}

decision
```

Summary

Item	Content
Covered	case-study review, architecture templates, state design, and decision records
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `case-studies.md`
- `thinking-in-langgraph.md`
- `workflows-agents.md`

PART

IV

Deep Agents

Framework-Agnostic Agent Building

What You Will Learn in This Part

- 1. Introduction to Deep Agents
- 2. Quickstart
- 3. Customization
- 4. Backends
- 5. Subagents
- 6. Memory and Skills
- 7. Advanced Features
- 8. Agent Harness
- 9. Framework Comparison
- 10. Sandboxes and ACP
- 11. Async Subagents
- 12. Going to Production
- 13. Context Engineering
- 14. Streaming

- 15. Permissions
- 16. Models and Tools
- 17. Programmatic Subagents
- 18. Event Streaming
- 19. Permissions Practice
- 20. Quality Profiles and Rubrics

01 Introduction to Deep Agents

Learning Objectives

- Understand what Deep Agents is
- Learn the difference between the SDK and the CLI
- Understand the five core concepts: Planning, Context Management, Backends, Subagents, and Memory
- Compare Deep Agents with other frameworks
- Verify that the package is installed correctly

1. What Is Deep Agents?

Deep Agents is an Agent Harness framework created by the LangChain team. It makes it easier to build autonomous agents for complex multi-step tasks by including the following 11 core capabilities out of the box:

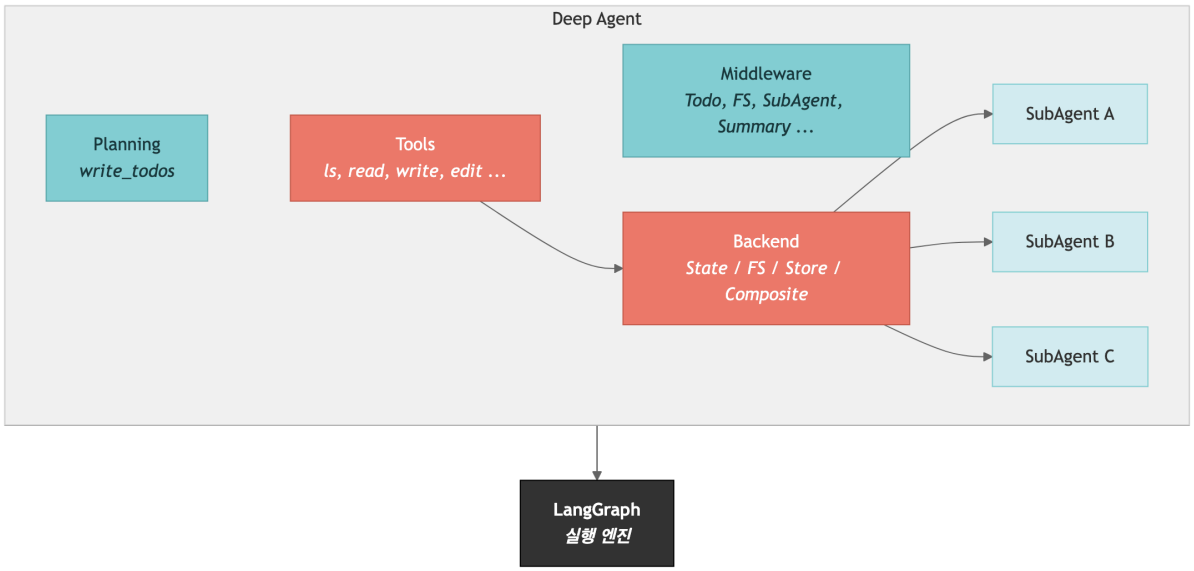
- Task planning — add `TodoListMiddleware` when an agent needs the `write_todos` planning tool
- Filesystem management — read, write, and search files (`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`)
- Filesystem permissions — per-path ACL with `allow_read` / `allow_write` / `deny_read` etc.
- Subagent delegation — distribute work via the `task` tool
- Long-term memory + skills — `AGENTS.md` (`memory`) and `SKILL.md` (`skills`) with progressive disclosure
- Context management — automatic offloading and summarization within the token budget
- QuickJS interpreter — sandboxed JS interpreter (`use_interpreter=True`) for arbitrary computation
- Sandbox execution — Modal, Daytona, Deno, and local Virtual FS backends
- Human-in-the-loop — per-tool approval gates with `interrupt_on={"tool_name": True}`
- Customization hooks — `before_model` / `after_model` / `wrap_model_call` middleware hooks
- Context propagation — subagent-specific config via `context_schema` and namespace keys (`"agent:key"`)

It is built on top of LangChain's core agent components, and it uses LangGraph as its execution engine.

> ➡️ TIP

> Model setup used in this course: The recommended default for Deep Agents is Anthropic Claude Sonnet 4.6 (`anthropic:claude-sonnet-4-6`). OpenAI examples use `openai:gpt-5.4` , Google examples use `google_genai:gemini-3.5-flash` . The `provider:model` prefix form is preferred; set the matching `ANTHROPIC_API_KEY` / `OPENAI_API_KEY` / `GOOGLE_API_KEY` .

Architecture Overview



2. SDK vs Code vs ACP

Deep Agents ships three interfaces on top of the same `AgentHarness` engine:

Category	Deep Agents SDK	Deep Agents Code	ACP server
Package	`deepagents`	`deepagents-cli`	`deepagents-cli` (`--acp`)
Purpose	Build agents programmatically	Use a coding agent directly from the terminal	Speak the Agent Client Protocol to external clients
Install	`pip install -qU deepagents langchain-{provider}`	`uv tool install deepagents-cli`	`uv tool install deepagents-cli`
Usage	Call `create_deep_agent()` from Python	Run `deepagents` in the terminal	`deepagents --acp` ; clients connect over stdio

Customization	Full API access (tools, backends, middleware)	`.deepagents/config.json` + slash commands	Same config exposed over ACP
Best fit	App integrations, automation pipelines, custom agent services	Interactive coding assistance	Zed, custom IDE clients, third-party shells

> ➡️ **TIP**
>
> In this course we focus on the SDK. Deep Agents Code and the ACP server are covered later in Part 4. Choose the LangChain integration package for your provider — for example `pip install -qU deepagents langchain-anthropic` or `pip install -qU deepagents langchain-openai`.

---

3. Five Core Concepts (Planning · Context Management · Backends · Subagents · Memory & Skills)

3.1 Planning

Planning is explicit in current Deep Agents examples: add `TodoListMiddleware` when the agent should use the `write_todos` tool to break complex work into a structured task list. Each task moves through states such as `pending` → `in_progress` → `completed`.

3.2 Context Management

Deep Agents manages large amounts of information generated during a task:

- Offloading: content over 20,000 tokens can be written to disk while only a pointer stays in context
- Summarization: conversation history can be compressed as it approaches the model limit

3.3 Backends

The agent filesystem is implemented with pluggable backends:

- `StateBackend` — store files in agent state (ephemeral, default)
- `FilesystemBackend` — local disk, with `virtual_mode=True` blocking `..` / `~` / paths outside `root_dir`
- `StoreBackend` — cross-thread persistent storage via LangGraph `BaseStore`
- `CompositeBackend` — route prefixes to different backends (for example `/memories/` → persistent)
- `ContextHubBackend` — lazy fetch + write-through remote sync (`deepagents ≥ 0.5.2`), with `namespace=lambda rt: ...`

- Sandbox backends — Modal, Daytona, Deno, local VFS for isolated file and code execution; the QuickJS interpreter (`use_interpreter=True`) is a complementary middleware

3.4 Subagents

The main agent can delegate specialized work to subagents. Each subagent can be given:

- Its own system prompt
- A dedicated model
- A separate context window
- A restricted toolset

3.5 Memory & Skills

Deep Agents inherits LangGraph's memory model and adds skill loading:

- Short-term memory — message history within a thread
- Long-term memory — `AGENTS.md` (`memory` parameter) always injected, `StoreBackend` for cross-thread state
- Skills — `SKILL.md` files loaded via progressive disclosure (`skills` parameter) so only frontmatter is loaded upfront, with full content fetched on demand

---

4. Comparison with Other Frameworks

Capability	LangChain Deep Agents	OpenCode	Claude Agent SDK
Model support	Model-agnostic (Anthropic, OpenAI, 100+ providers)	75+ providers, including Ollama	Claude-only
License	MIT	MIT	MIT (SDK) / proprietary (Claude Code)
SDK	Python, TypeScript + CLI	Terminal, desktop, IDE	Python, TypeScript
Sandboxing	Integrated as a tool (Modal, Daytona, etc.)	Not supported	Not supported
Pluggable backends	O (State, FS, Store, Composite)	X	X
Time travel	O (via LangGraph)	X	O
Observability	Native LangSmith support	X	X

Built-in file tools	O	O	O
Human-in-the-loop	O	X	O

Deep Agents is especially strong when you want to build agents that need planning + files + memory + subagents in one integrated stack.

5. Installation Check

Install `deepagents` together with the LangChain integration package for your chosen provider:

```
# Recommended default — Anthropic
pip install -qU deepagents langchain-anthropic

# OpenAI
pip install -qU deepagents langchain-openai

# Google Gemini
pip install -qU deepagents langchain-google-genai

# OpenRouter (OpenAI-compatible)
pip install -qU deepagents langchain-openai  # just change base_url
```

Run the cells below to verify that the `deepagents` package is installed correctly.

```
# Check the deepagents package version
import deepagents
print(f"deepagents version: {deepagents.__version__}")
```

```
# Verify imports of the main modules
from deepagents import create_deep_agent, SubAgent, CompiledSubAgent
from deepagents import FilesystemMiddleware, MemoryMiddleware, SubAgentMiddleware
from deepagents.backends import StateBackend, FilesystemBackend, StoreBackend, CompositeBackend
from deepagents.backends.protocol import BackendProtocol

print("Successfully imported all main modules!")
```

```
# Check dependency package versions
import importlib.metadata

print(f"langchain version: {importlib.metadata.version('langchain')}")
print(f"langgraph version: {importlib.metadata.version('langgraph')}")
```

```
# Inspect the create_deep_agent function signature
import inspect

sig = inspect.signature(create_deep_agent)
print("create_deep_agent() parameters:")
for name, param in sig.parameters.items():
    default = param.default if param.default is not inspect.Parameter.empty else "(required)"
    print(f"  - {name}: {default}")
```

Summary

Item	Description
Deep Agents	A LangChain-based agent harness framework
Core function	`create_deep_agent()`
Execution engine	LangGraph (`CompiledStateGraph`)
Recommended model	Anthropic Claude Sonnet 4.6 via `"anthropic:claude-sonnet-4-6"` (OpenAI: `"openai:gpt-5.4"`)
Core concepts	Planning, Context Management, Backends, Subagents, Memory & Skills
Built-in tools	`write_todos` , `ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`

Next Steps

→ 02_quickstart.ipynb: Build and run your first Deep Agent.

02 Building Your First Agent

Learning Objectives

- Learn how to load API keys from a `.env` file
- Create a basic agent with `create_deep_agent()`
- Run the agent with `agent.invoke()` and `agent.stream()`
- Build a research agent with a Tavily search tool
- Understand the built-in tools and what they are for

1. Project Bootstrap and API Keys

Bootstrap with `uv`

The recommended workflow uses `uv` for reproducible dependency management.

```
# 1) Init the project
uv init my-deepagent
cd my-deepagent

# 2) Add deepagents plus the provider package
uv add deepagents langchain-anthropic   # recommended default
# uv add deepagents langchain-openai
# uv add deepagents langchain-google-genai
uv add tavily-python python-dotenv

# 3) Sync the lockfile
uv sync
```

API keys by provider

The recommended default model is Anthropic Claude Sonnet 4.6 (`anthropic:claude-sonnet-4-6`).
Set only the keys for the providers you actually use.

```
# Anthropic (recommended default)
ANTHROPIC_API_KEY=your-key-here

# OpenAI
OPENAI_API_KEY=your-key-here

# Google Gemini
GOOGLE_API_KEY=your-key-here

# OpenRouter (OpenAI-compatible)
```

```
OPENAI_API_KEY=your-openrouter-key
OPENAI_BASE_URL=https://openrouter.ai/api/v1

# Shared — used in the research example
TAVILY_API_KEY=your-key-here
```

> ➡️ TIP
>
> The simplest setup is to copy `.env.example` to `.env` and then fill in the real values.

```
# Load environment variables
from dotenv import load_dotenv
import os

load_dotenv()

# Recommended default — Anthropic
assert os.environ.get("ANTHROPIC_API_KEY"), "ANTHROPIC_API_KEY is not set!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY is not set!"
print("API keys loaded successfully.")
```

---

2. Creating the Simplest Agent

`create_deep_agent()` is the core function in Deep Agents. If you call it with no additional configuration, it assembles the harness around your model and the default filesystem/subagent tools. Planning is an explicit middleware choice in current Deep Agents.

```
from deepagents import create_deep_agent

# Recommended default — Anthropic Claude Sonnet 4.6 (provider:model string)
agent = create_deep_agent(model="anthropic:claude-sonnet-4-6")

# (Alt 1) OpenAI gpt-5.4
# agent = create_deep_agent(model="openai:gpt-5.4")

# (Alt 2) Google Gemini 3.5 Flash
# agent = create_deep_agent(model="google_genai:gemini-3.5-flash")

# (Alt 3) Pass a ChatModel object for fine control
# from langchain_anthropic import ChatAnthropic
# model = ChatAnthropic(model="claude-sonnet-4-6", temperature=0)
# agent = create_deep_agent(model=model)

print(f"Agent type: {type(agent).__name__}")
print("The agent was created successfully!")
```

`create_deep_agent()` returns a LangGraph `CompiledStateGraph`. That means you can use all of the normal LangGraph execution methods such as `invoke`, `stream`, and `batch`.

---

3. Running the Agent — `invoke()`

Run the agent by sending it a message. The input format is a dictionary like `{"messages": [{"role": "user", "content": "..."}]}`.

4. Connecting Tavily Search — A Research Agent

You can extend the agent by adding custom tools. In this example, you connect the Tavily web search tool.

How tool definitions work

A Python function's docstring becomes the tool description, and its type hints become the parameter schema.

```
from typing import Literal
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])


def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
    include_raw_content: bool = False,
) -> dict:
    """Search the internet for information.

    Args:
        query: The question or keyword to search for
        max_results: Maximum number of results to return
        topic: Search topic category
        include_raw_content: Whether to include the raw source content
    """
    return tavily_client.search(
        query,
        max_results=max_results,
        include_raw_content=include_raw_content,
        topic=topic,
    )


print(f"Tool name: {internet_search.__name__}")
print(f"Tool description: {internet_search.__doc__.strip().splitlines()[0]}")
```

```
# Create a research agent — search tool + custom system prompt
research_agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[internet_search],
    system_prompt="You are an expert researcher. Search the internet, organize the results, and write the
final answer in English.",
)

print("Research agent created!")
```

5. Checking the Built-In Tools

These are the core harness tools you will see in current Deep Agents. Planning appears when you add `TodoListMiddleware`.

Tool	Description
`write_todos`	Manage a structured task list when `TodoListMiddleware` is enabled
`ls`	List directory contents with metadata
`read_file`	Read a file (with line numbers and image support)
`write_file`	Write or replace a file
`edit_file`	Replace text inside a file (`old_string` → `new_string`)
`glob`	Pattern-based file search (for example `**/*.py`)
`grep`	Search file contents (with regex support)
`task`	Call a subagent

> ➡️ TIP
>
> All of these tools work through the backend. The default is `StateBackend`, which stores files inside agent state.

---

6. Streaming Output — `stream()`

With `agent.stream()`, you can observe the agent's execution process in real time. You can choose different levels of detail through `stream_mode`:

- `"updates"` — state updates after each completed step
- `"messages"` — token-level streaming
- `"custom"` — custom events emitted from tools or nodes

Summary

Item	Description
Agent creation	`create_deep_agent(model, tools, system_prompt)`
Synchronous execution	`agent.invoke({"messages": [...]})`

Streaming execution	`agent.stream({"messages": [...]}, stream_mode="updates")`
Custom tools	Python function + docstring + type hints
Model format	Provider prefix preferred — `"anthropic:claude-sonnet-4-6"`, `"openai:gpt-5.4"`, `"google_genai:gemini-3.5-flash"`. ChatModel objects also accepted

Next Steps

→ 03_customization.ipynb: learn how to customize the agent in more detail.

03 Agent Customization

Learning Objectives

- Learn how to choose different LLM providers and models
- Write effective system prompts
- Build custom tools from docstrings and type hints
- Produce structured Pydantic output with `response_format`
- Understand the middleware architecture

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("ANTHROPIC_API_KEY"), "ANTHROPIC_API_KEY is not set!"
print("Environment setup complete")

# Recommended default — Anthropic Claude Sonnet 4.6 (provider:model string)
from deepagents import create_deep_agent

model = "anthropic:claude-sonnet-4-6"
print(f"Default model: {model}")
```

0. Full `create_deep_agent()` Signature (17 parameters)

Before diving into individual customization axes, here is the full 17-parameter signature. All parameters are optional; only `model` is enough to get a working agent.

```
def create_deep_agent(
    model=None,              # 1. LLM (ChatModel object or "provider:model")
    tools=None,              # 2. custom tools
    system_prompt=None,      # 3. user-provided prompt (USER segment)
    middleware=(),           # 4. extra user middleware
    subagents=None,          # 5. SubAgent dicts / CompiledSubAgent
    skills=None,             # 6. SKILL.md directories (progressive disclosure)
    memory=None,             # 7. AGENTS.md path (always injected)
    response_format=None,    # 8. Pydantic schema for structured output
    context_schema=None,     # 9. runtime context TypedDict (propagated to subagents)
    checkpointer=None,       # 10. LangGraph checkpointer (persistence)
    store=None,              # 11. LangGraph BaseStore (for StoreBackend)
    cache=None,              # 12. prompt cache config (e.g. Anthropic)
    backend=None,            # 13. BackendProtocol instance
    permissions=None,        # 14. FilesystemPermission rules
    interrupt_on=None,       # 15. {"tool_name": True} HITL gates
    debug=False,             # 16. debug logging
    name=None,               # 17. graph name
```

```
):
    ...
```

> ➡️ **TIP**
>
> `permissions` , `context_schema` , and `cache` are recent additions (`deepagents ⩾ 0.5.0`). Older versions require adding middleware directly.

Prompt assembly order

The string you pass to `system_prompt` is not used verbatim. Deep Agents' prompt builder assembles three segments in a fixed order:

```
[USER]    system_prompt (string from the caller)
   ↓
[BASE]    harness BASE prompt (TodoList/Filesystem/SubAgent usage)
   + CUSTOM  per-middleware injections (Memory, Skills, Permissions, ...)
   ↓
[SUFFIX]  harness SUFFIX (safety guidance, output contract)
```

The order is USER → BASE/CUSTOM → SUFFIX. You only fill in the USER segment; BASE and SUFFIX are managed by middleware. Nothing is overwritten, so there is no need to repeat built-in tool usage in your own prompt.

---

1. Choosing a Model

Deep Agents supports a wide range of LLMs through either a **LangChain chat model object** or the `provider:model` format.

This notebook uses Anthropic Claude Sonnet 4.6 (`anthropic:claude-sonnet-4-6`) as the recommended default. OpenAI examples use `gpt-5.4` .

Provider	Example model (provider:model)	Environment variable	Notes
Anthropic	`anthropic:claude-sonnet-4-6`	`ANTHROPIC_API_KEY`	Recommended default — best for prompt caching and HITL
OpenAI	`openai:gpt-5.4`	`OPENAI_API_KEY`	Strong tool-calling accuracy
Google	`google_genai:gemini-3.5-flash`	`GOOGLE_API_KEY`	Cost-effective
Azure	`azure_openai:gpt-5.4`	`AZURE_OPENAI_*`	Enterprise deployments

AWS Bedrock	`bedrock:anthropic.claude-sonnet-4-6`	AWS credentials	VPC isolation

The recommended default is `anthropic:claude-sonnet-4-6`, with built-in retry (6 attempts) and timeout. The `model` parameter accepts either a `BaseChatModel` object or a `"provider:model-name"` string.

```
# Recommended default — Anthropic Claude Sonnet 4.6 via provider:model string
agent_default = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
)

print(f"Agent created: {type(agent_default).__name__}")

# Reference examples for other providers:
# agent_openai = create_deep_agent(model="openai:gpt-5.4")
# agent_gemini = create_deep_agent(model="google_genai:gemini-3.5-flash")
# agent_bedrock = create_deep_agent(model="bedrock:anthropic.claude-sonnet-4-6")
```

2. Custom System Prompts

The system prompt defines the agent's role, behavioral rules, and output style. It is added on top of the default prompt, so you can provide domain-specific instructions without rebuilding everything from scratch.

3. Building Custom Tools

Deep Agents converts Python functions into tools using the following rules:

1. Function name → tool name
2. Docstring → tool description (used by the agent to decide whether to call the tool)
3. Type hints → parameter schema (generated automatically)
4. Default values → optional parameters

```
import math


def calculate_compound_interest(
    principal: float,
    annual_rate: float,
    years: int,
    compounds_per_year: int = 12,
) -> dict:
    """Calculate compound interest.

    Args:
        principal: Principal amount
        annual_rate: Annual interest rate (for example 0.05 = 5%)
        years: Number of years
        compounds_per_year: Number of compounding periods per year (default: 12 = monthly)
    """
    amount = principal * (1 + annual_rate / compounds_per_year) ** (compounds_per_year * years)
    interest = amount - principal
```

```
    return {
        "principal": f"{principal:,.0f}",
        "final_amount": f"{amount:,.0f}",
        "interest_earned": f"{interest:,.0f}",
        "return_rate": f"{(interest / principal) * 100:.2f}%",
    }


def convert_temperature(
    value: float,
    from_unit: str,
    to_unit: str,
) -> str:
    """Convert between temperature units.

    Args:
        value: Temperature value to convert
        from_unit: Source unit ('celsius', 'fahrenheit', 'kelvin')
        to_unit: Target unit ('celsius', 'fahrenheit', 'kelvin')
    """
    if from_unit == "fahrenheit":
        celsius = (value - 32) * 5 / 9
    elif from_unit == "kelvin":
        celsius = value - 273.15
    else:
        celsius = value

    if to_unit == "fahrenheit":
        result = celsius * 9 / 5 + 32
    elif to_unit == "kelvin":
        result = celsius + 273.15
    else:
        result = celsius

    return f"{value} {from_unit} = {result:.2f} {to_unit}"


calculator_agent = create_deep_agent(
    model=model,
    tools=[calculate_compound_interest, convert_temperature],
    system_prompt="You are an assistant for calculations and unit conversion. Always use tools for exact
results.",
)

print("Calculator agent created!")
```

4. Structured Output — `response_format`

You can structure the agent's final response as a Pydantic model. That makes the result much easier to use programmatically.

```
from pydantic import BaseModel, Field


class BookRecommendation(BaseModel):
    """Single book recommendation"""
    title: str = Field(description="Book title")
    author: str = Field(description="Author")
    reason: str = Field(description="Reason for the recommendation (2–3 sentences)")
    difficulty: str = Field(description="Difficulty level: beginner/intermediate/advanced")


class BookRecommendationList(BaseModel):
    """Book recommendation list"""
    topic: str = Field(description="Topic of the recommendation")
```

```
    books: list[BookRecommendation] = Field(description="Recommended books")


book_agent = create_deep_agent(
    model=model,
    system_prompt="You are a book recommendation expert. Suggest books that match the user's interests.",
    response_format=BookRecommendationList,
)

print("Book recommendation agent created")
```

5. Middleware Architecture

`create_deep_agent()` builds an internal middleware stack. Middleware is the plugin layer that extends and controls the agent's behavior.

Default middleware stack (execution order)

```
1.  TodoListMiddleware                 – optional task management (write_todos)
2.  MemoryMiddleware                   – load AGENTS.md when `memory` is used
3.  SkillsMiddleware                   – load SKILL.md when `skills` is used
4.  FilesystemMiddleware               – file tools (ls, read, write, edit, glob, grep)
5.  SubAgentMiddleware  [required, not removable]  – subagent support (task tool)
6.  SummarizationMiddleware            – context compression (~85% auto-summary)
7.  AnthropicPromptCachingMiddleware   – prompt caching (auto-applied for Anthropic)
8.  PatchToolCallsMiddleware           – repair malformed/missing tool calls
9.  [User custom middleware]           – `middleware` parameter
10. HumanInTheLoopMiddleware           – approval workflow (`interrupt_on`)
```

> **! WARNING**
>
> `SubAgentMiddleware` cannot be removed via `excluded_middleware`. The `task` tool is the core delegation mechanism, so it stays enabled even when you do not declare any subagents — a built-in `general-purpose` subagent fills the slot.

What each middleware does

Middleware	Tools Added	Role
`TodoListMiddleware`	`write_todos`	Manage structured task lists when explicitly enabled
`FilesystemMiddleware`	`ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`	Filesystem access
`SubAgentMiddleware`	`task`	Create and call subagents
`SummarizationMiddleware`	none	Summarize context when it reaches about 85% of the limit

`MemoryMiddleware`	none	Inject `AGENTS.md` into the system prompt
`SkillsMiddleware`	none	Progressively load relevant `SKILL.md` files

```python
# Verify available middleware imports
from deepagents.middleware import (
    FilesystemMiddleware,
    MemoryMiddleware,
    SubAgentMiddleware,
    SkillsMiddleware,
    SummarizationMiddleware,
)

print("Available middleware:")
for mw in [FilesystemMiddleware, MemoryMiddleware, SubAgentMiddleware, SkillsMiddleware,
SummarizationMiddleware]:
    print(f"  - {mw.__name__}")
```

> · NOTE
>
> Note: `create_deep_agent()` configures middleware automatically in most cases. You can still add custom middleware through the `middleware` parameter if you need advanced behavior.

Replacing a default middleware instance

Deep Agents treats middleware names as part of the harness contract. If you pass a user middleware instance with the same `.name` as a default middleware, it replaces that default instance in place. This is different from ordinary LangChain middleware stacking, where similarly named middleware would usually just add another hook.

```python
from deepagents.middleware import TodoListMiddleware

# Replace the default planning middleware with a narrower configuration.
planning = TodoListMiddleware(name="TodoListMiddleware")

agent = create_deep_agent(
    model=model,
    middleware=[planning],
)
```

> ➡️ TIP
>
> Use replacement only when you mean to change the harness behavior. For cross-cutting additions such as logging or tracing, give the middleware its own name so it appends to the stack.

6. Sandbox Options

Beyond `tools`, Deep Agents ships sandbox options that isolate untrusted code execution. Use them for coding agents or data-analysis agents that run arbitrary scripts.

Sandbox	Execution environment	Best fit
Modal	Remote container (GPU available)	Long-running code, GPU inference
Daytona	Remote development container	Git workspace integration, multi-language
Deno	Local TypeScript/JavaScript isolation	Lightweight JS/TS execution
Local VFS	In-process virtual filesystem	Unit tests with no network or disk access

> ➡️ TIP
>
> Sandboxes can be exposed either as tools (for example an `execute_python` tool) or as backends (for example a `SandboxBackend` for file isolation). The two layers can be combined to get both code and file isolation.

Summary

Item	Method
Model selection	`model="anthropic:claude-sonnet-4-6"` or any provider prefix
System prompt	Assembled USER → BASE/CUSTOM → SUFFIX
Custom tools	function + docstring + type hints → `tools=[func]`
Structured output	`response_format=PydanticModel` → `result["structured_response"]`
Middleware	`SubAgentMiddleware` is required; the rest are wired automatically
Full parameter list	17 — includes `permissions`, `context_schema`, `checkpointer`, `store`, `cache`, `interrupt_on`
Sandboxes	Modal, Daytona, Deno, local VFS — exposed as tools or backends

Next Steps

→ 04_backends.ipynb: learn how storage backends define the agent filesystem.

04 Storage Backends

Learning Objectives

- Understand how backends implement the agent's filesystem
- Learn the characteristics and use cases of the five built-in backends
- Configure path-based routing with `CompositeBackend`
- Implement a custom backend with `BackendProtocol`

```python
# Environment setup
from dotenv import load_dotenv
import os

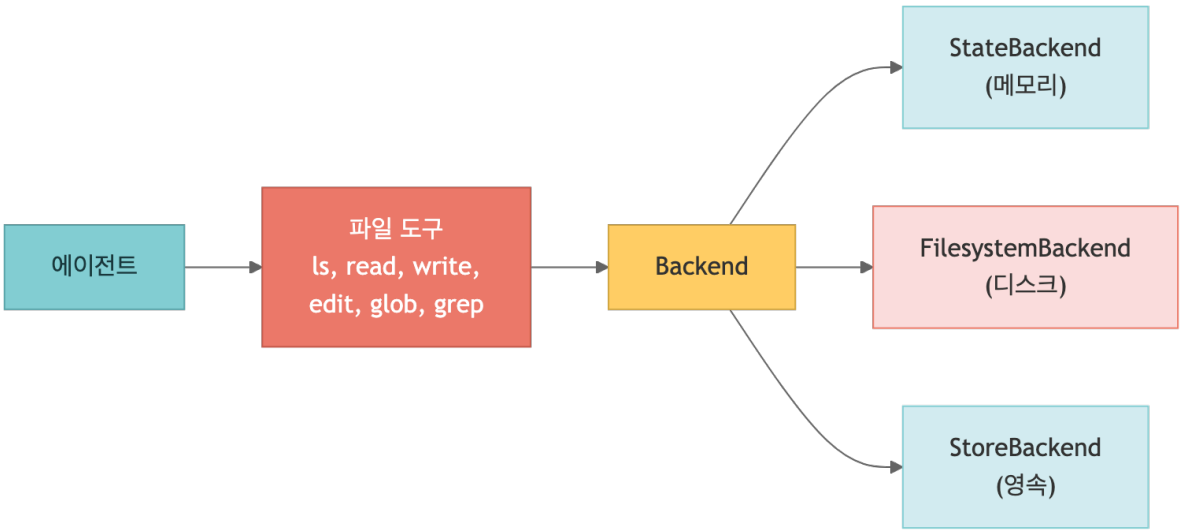
load_dotenv()
assert os.environ.get("ANTHROPIC_API_KEY"), "ANTHROPIC_API_KEY is not set!"
print("Environment setup complete")

# Recommended default — Anthropic Claude Sonnet 4.6
model = "anthropic:claude-sonnet-4-6"
```

1. What Is a Backend?

Deep Agents' built-in file tools (`ls` , `read_file` , `write_file` , `edit_file` , `delete` , `glob` , `grep`) all operate through a backend.

A backend abstracts the storage layer that the agent uses to read and write files.



Available Backends

Backend	Storage Location	Persistence	Use Case
`StateBackend`	Agent state (memory)	Within a thread	Scratch work, temporary tasks (default)
`FilesystemBackend`	Local disk	Persistent	Local file access, coding agents
`StoreBackend`	LangGraph Store	Cross-thread	Long-term memory, user preferences
`CompositeBackend`	Path-based routing	Mixed	Persistent memory + temporary files
`LocalShellBackend`	Disk + shell	Persistent	Development environments (security caution)
`ContextHubBackend`	Context Hub (remote)	Persistent (lazy fetch)	Team-shared context, remote file sync

```
# Verify backend imports
from deepagents.backends import (
    StateBackend,
    FilesystemBackend,
    StoreBackend,
    CompositeBackend,
)
from deepagents.backends.protocol import BackendProtocol

print("All backend classes imported successfully!")
```

2. `StateBackend` (Default)

`StateBackend` stores files in agent state (LangGraph state). It is ephemeral: files live only inside the current conversation thread.

Characteristics

- Used automatically when you do not pass `backend` to `create_deep_agent()`
- Files survive across agent turns through checkpoints
- Files disappear when the thread ends
- No external storage required

3. `FilesystemBackend` — Accessing the Local Disk

`FilesystemBackend` lets the agent access the real local filesystem.

Key Options

- `root_dir` — the root directory the agent can access (default: current directory)
- `virtual_mode=True` — blocks `..` segments, `~` expansion, and absolute paths outside `root_dir`
- `max_file_size_mb` — maximum readable file size

Paths blocked by `virtual_mode=True`

- Relative escapes containing `..` segments (`../etc/passwd` , `foo/../../bar`)
- Home expansion starting with `~` or `~user`
- Absolute paths outside `root_dir` (such as `/etc/hosts`)

When any of these are submitted, the backend rejects the operation before touching the real disk; writes can raise `ValueError` rather than returning a normal result object.

⚠️ Security Considerations

> ➡️ TIP
>
> `FilesystemBackend` gives the agent access to the real filesystem. In production, set `virtual_mode=True` or use a sandbox backend.

---

4. `StoreBackend` — Cross-Thread Persistent Storage

`StoreBackend` uses LangGraph's `BaseStore` to persist files across conversation threads.

Characteristics

- The same files can be accessed from different threads
- Supports different store implementations such as Redis and PostgreSQL
- Can be provisioned automatically in LangSmith deployments
- Uses assistant-level namespacing to isolate agents

```
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import MemorySaver

# InMemoryStore is useful for development (use PostgresStore or similar in production)
store = InMemoryStore()
checkpointer = MemorySaver()

backend = StoreBackend(namespace=lambda rt: (rt.server_info.assistant_id,))

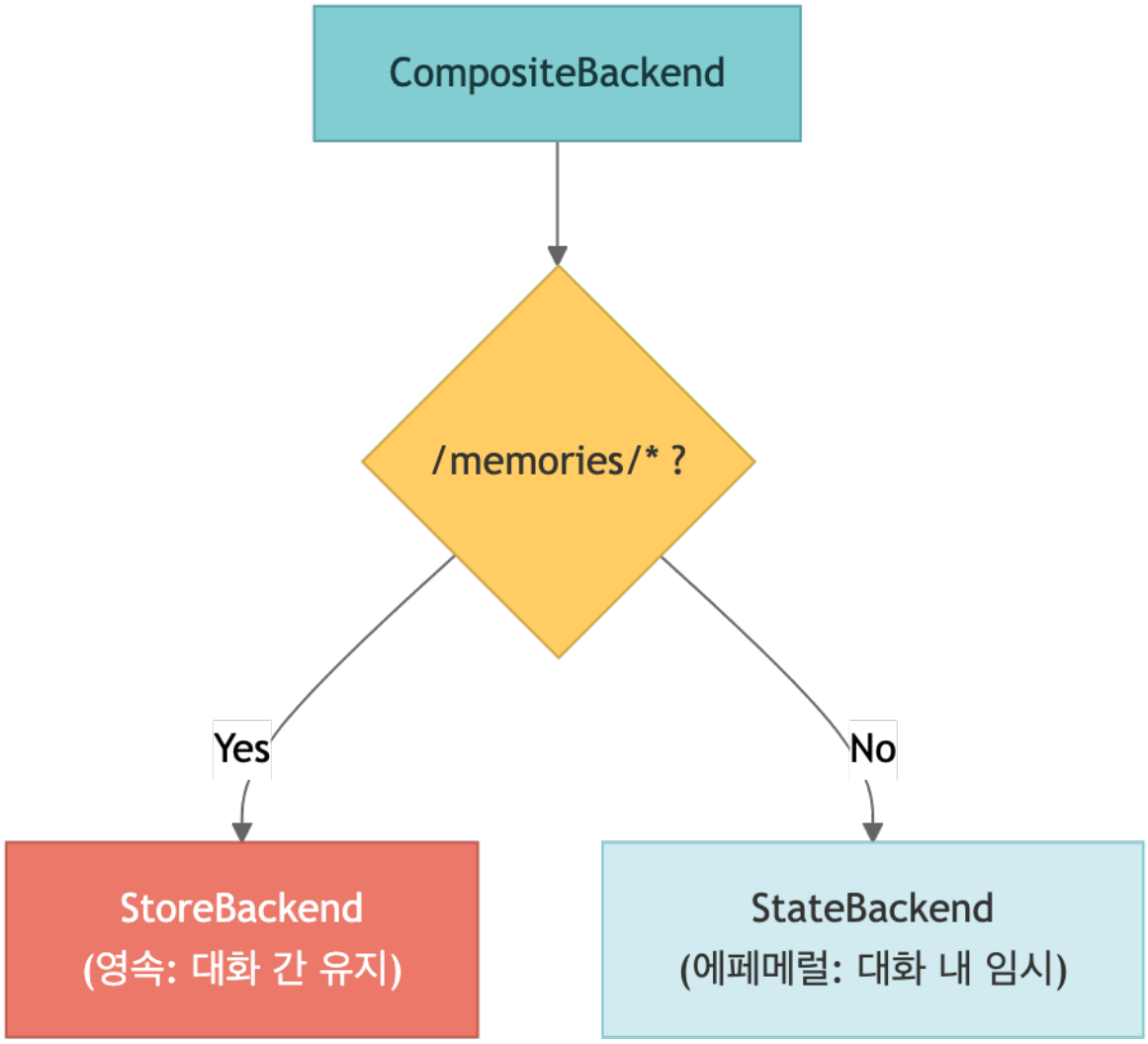
store_agent = create_deep_agent(
    model=model,
```

```
    system_prompt="You are an assistant that manages notes. Respond in English.",
    backend=backend,
    store=store,
    checkpointer=checkpointer,
)

print("StoreBackend agent created!")
```

5. `CompositeBackend` — Path-Based Routing

`CompositeBackend` routes different filesystem paths to different backends. A common pattern is to persist `/memories/*` while keeping everything else ephemeral.




```
# CompositeBackend with path-based routing
backend = CompositeBackend(
    default=StateBackend(),
    routes={
```

```
        "/memories/": StoreBackend(
            namespace=lambda rt: (rt.server_info.user.identity,),
        ),
    },
)


composite_store = InMemoryStore()
composite_checkpointer = MemorySaver()

composite_agent = create_deep_agent(
    model=model,
    system_prompt="""You are a memory-management assistant.
- Save notes that need to persist under /memories/.
- Save temporary work files under the root path /.
Respond in English.""",
    backend=backend,
    store=composite_store,
    checkpointer=composite_checkpointer,
)

print("CompositeBackend agent created!")
```

· NOTE

Note: `CompositeBackend` removes the route prefix before storing the file internally. Example: `/memories/preferences.txt` → stored internally as `/preferences.txt` But the agent always accesses it with the full path (`/memories/preferences.txt`).

---

6. `LocalShellBackend` — Shell Execution

`LocalShellBackend` extends `FilesystemBackend` by adding shell command execution through the `execute` tool.

⚠️ Security Warning

➡️ TIP

Commands run directly on the host system with your user permissions. Use this only in development environments. In production, prefer a sandbox backend.

```
from deepagents.backends import LocalShellBackend

# ⚠ Development only!
agent = create_deep_agent(
    model=model,
    backend=LocalShellBackend(root_dir="./workspace", virtual_mode=True),
    interrupt_on={"execute": True},  # require approval before shell commands
)
```

> · NOTE
>
> For safety, this notebook does not run `LocalShellBackend` directly.

---

7. Implementing a Custom Backend

If you implement `BackendProtocol`, you can build your own backend.

> ! WARNING
>
> Implement the current `BackendProtocol` methods directly: `ls` / `grep` / `glob` plus `read` / `write` / `edit`. Return types are explicit dataclasses / TypedDicts (`LsResult`, `ReadResult`, `GrepResult`, `WriteResult`, `EditResult`, `list[FileInfo]`) rather than bare dicts.

Required Methods and Return Types

Method	Return type	Description
`ls(path)`	`LsResult` (wraps `list[FileInfo]`)	List directory contents
`read(file_path, offset, limit)`	`ReadResult`	Read a file with line numbers
`write(file_path, content)`	`WriteResult`	Write or replace a file
`edit(file_path, old_string, new_string)`	`EditResult`	Replace text (with `occurrences`)
`grep(pattern, path, glob)`	`GrepResult`	Search file contents by pattern
`glob(pattern, path)`	`GlobResult`	Search files by glob pattern

```python
# Simple example: read-only dictionary-backed backend
from deepagents.backends.protocol import (
    FileData, FileInfo, GlobResult, GrepResult, GrepMatch,
    LsResult, ReadResult, WriteResult, EditResult,
)


class ReadOnlyDictBackend:
    """Example of a read-only backend backed by a Python dictionary."""

    def __init__(self, files: dict[str, str]):
        self._files = files

    def ls(self, path: str = "/") -> LsResult:
        entries: list[FileInfo] = [
```

```
            {"path": p, "is_dir": False, "size": len(c), "modified_at": None}
            for p, c in self._files.items()
            if p.startswith(path)
        ]
        return LsResult(entries=entries, error=None)

    def read(self, file_path: str, offset: int = 0, limit: int = 2000) -> ReadResult:
        content = self._files.get(file_path, "")
        lines = content.splitlines()
        selected = lines[offset:offset + limit]
        rendered = "\n".join(f"{i + offset + 1}\t{line}" for i, line in enumerate(selected))
        return ReadResult(
            file_data=FileData(content=rendered, encoding="utf-8"),
            error=None,
            total_lines=len(lines),
            start_line=offset + 1 if selected else None,
            end_line=offset + len(selected) if selected else None,
        )

    def write(self, file_path: str, content: str) -> WriteResult:
        return WriteResult(error="This backend is read-only.", path=None)

    def edit(self, file_path: str, old_string: str, new_string: str, replace_all: bool = False) ->
EditResult:
        return EditResult(error="This backend is read-only.", path=None, occurrences=None)

    def grep(
        self,
        pattern: str,
        path: str | None = None,
        glob: str | None = None,
        *,
        max_count: int | None = None,
    ) -> GrepResult:
        import re
        matches: list[GrepMatch] = []
        for fpath, content in self._files.items():
            for i, line in enumerate(content.splitlines(), 1):
                if re.search(pattern, line):
                    matches.append({"path": fpath, "line": i, "text": line})
                    if max_count is not None and len(matches) >= max_count:
                        return GrepResult(matches=matches, error=None, truncated=True)
        return GrepResult(matches=matches, error=None)

    def glob(self, pattern: str, path: str | None = None) -> GlobResult:
        import fnmatch
        matches = [
            {"path": p, "is_dir": False, "size": len(c), "modified_at": None}
            for p, c in self._files.items()
            if fnmatch.fnmatch(p, pattern)
        ]
        return GlobResult(matches=matches, error=None)


custom_backend = ReadOnlyDictBackend({
    "/docs/guide.md": "# Guide\nThis is a guide document.\n## Installation\npip install deepagents",
    "/docs/faq.md": "# FAQ\nQ: Which models are supported?\nA: Anthropic, OpenAI, and more.",
})

print("File list:", custom_backend.ls("/").entries)
print()
print("File contents:")
result = custom_backend.read("/docs/guide.md")
print(result.file_data["content"] if result.file_data else result.error)
print()
print("Search results:", custom_backend.grep("Installation").matches)
```

8. `ContextHubBackend` — Remote File Sync

`ContextHubBackend` syncs files with LangChain Context Hub through lazy fetch + write-through (`deepagents ≥ 0.5.2`). Use it when teams share a common context (prompts, policies, domain docs) that every agent should see.

How it works

- Lazy fetch — files are downloaded only when the agent calls `read_file` , then cached locally
- Write-through — `write_file` / `edit_file` update both local and remote stores so other workers see the change immediately

Instantiation

Because `namespace` often has to be decided at runtime, instantiate the backend up front and pass a `namespace=lambda rt: ...` callable.

```
# deepagents>=0.5.2
from deepagents.backends import ContextHubBackend

# Pre-instantiated backend + runtime namespace callable
context_hub = ContextHubBackend(
    project="my-team",
    namespace=lambda rt: f"users/{rt.context['user_id']}",  # decided at runtime
)

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    backend=context_hub,
)
```

> ➡️ TIP
>
> The `namespace` callable signature is `(runtime) → str` . A plain string also works, but multi-tenant deployments effectively require the lambda form.

---

9. Permissions — `Permissions` + `GuardedBackend`

When you need finer-grained access control than `virtual_mode` , use the `permissions` parameter to apply per-path ACLs. Internally a `GuardedBackend` wraps the original backend and routes every file call through the policy hook.

```
from deepagents import create_deep_agent
from deepagents.permissions import Permissions

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    backend=FilesystemBackend(root_dir="./workspace", virtual_mode=True),
    permissions=Permissions(
        allow_read=["/docs/**", "/data/*.csv"],   # allowed reads
```

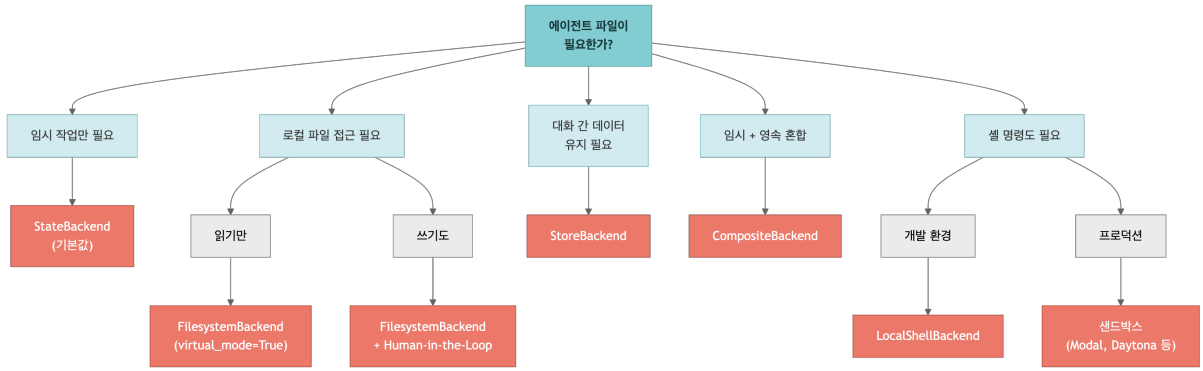
```
        allow_write=["/outputs/**"],              # allowed writes
        deny_read=["/data/secrets/**"],           # deny wins over allow
        deny_write=["**/*.lock"],
    ),
)
```

Policy hook (`policy`)

For complex rules pass `policy=lambda call: ...` — the call object includes `tool_name` , `path` , and `args` , so you can factor in time-of-day, user roles, or external authorization checks.

> ➡️ **TIP**
>
> When both `allow_*` and `deny_*` rules match, `deny` wins. The safest pattern is an explicit whitelist via `allow_read=["/docs/**"]` .

Backend Selection Guide



Summary

Backend	Characteristics	Parameters
`StateBackend`	Ephemeral, default	Used automatically when `backend` is omitted
`FilesystemBackend`	Local disk access	`root_dir` , `virtual_mode`
`StoreBackend`	Cross-thread persistence	requires `store` + `checkpointer`
`CompositeBackend`	Path-based routing	`default` + `routes`

`LocalShellBackend`	Disk + shell execution	`root_dir` (security caution)
`ContextHubBackend`	Remote lazy fetch + write-through	pre-instantiated + `namespace=lambda rt: ...`
`GuardedBackend` (permissions)	Per-path ACL wrapper	`allow_*` / `deny_*` / `policy`

Next Steps

→ 05_subagents.ipynb: learn how to delegate work with subagents.

05 Subagents and Task Delegation

Learning Objectives

- Understand the problem subagents solve: context bloat
- Define subagents with `SubAgent` dictionaries and `CompiledSubAgent`
- Understand and override the built-in general-purpose subagent
- Use context propagation and namespace keys
- Implement a multi-subagent pipeline pattern

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY is not set!"
print("Environment setup complete")
```

```
# Recommended default — Anthropic Claude Sonnet 4.6
model = "anthropic:claude-sonnet-4-6"

print(f"Model configured: {model}")
```

1. Why Subagents Are Useful

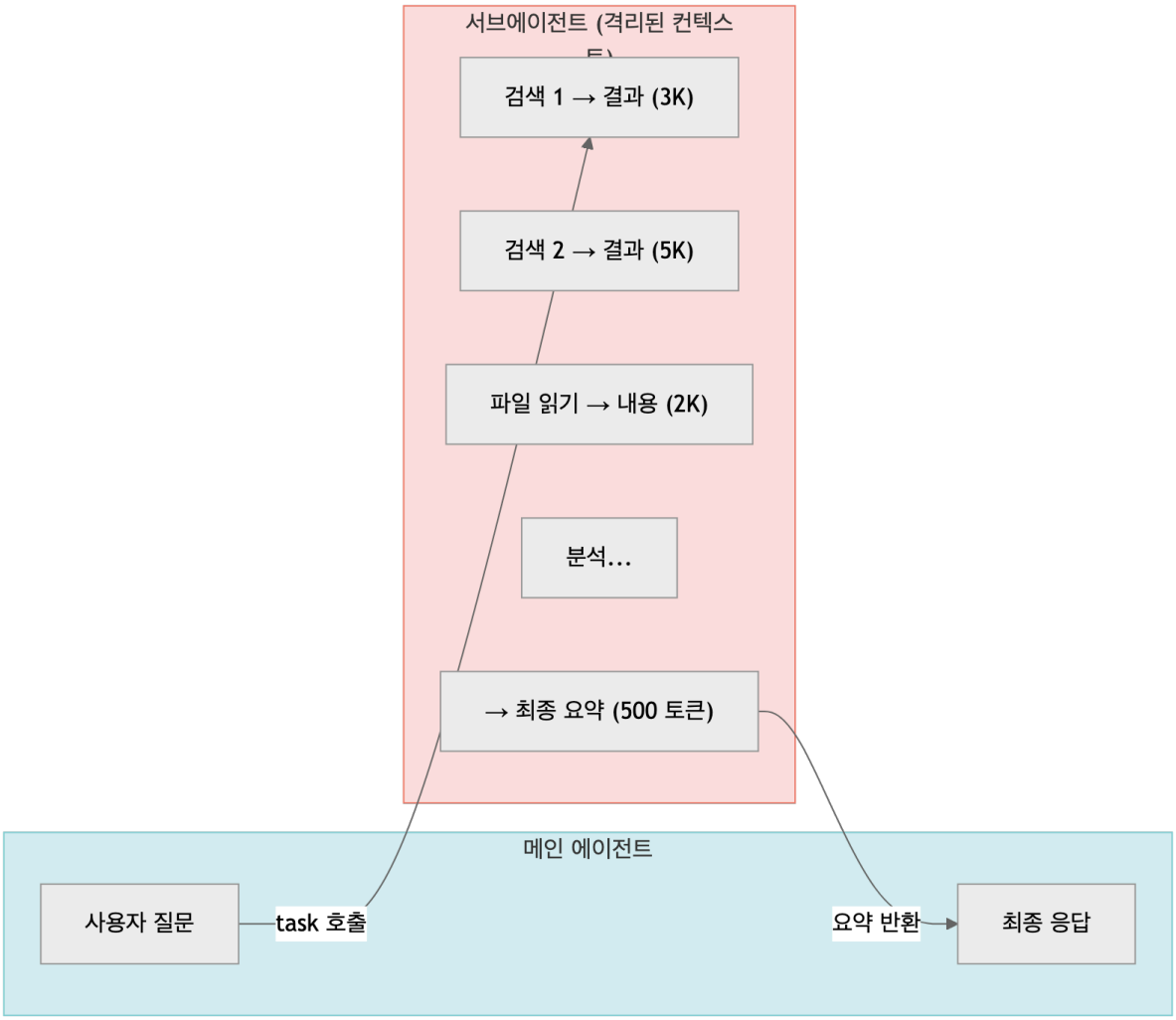
The Context Bloat Problem

Every time an agent uses a tool, the inputs and outputs accumulate inside the context window:

- web search results (thousands of tokens)
- file contents (hundreds or thousands of lines)
- database query results

When too much intermediate data builds up in the main context, the agent can lose track of the most important information.

How Subagents Help



The main agent receives only a short summary, so its context stays compact and focused.

`SubAgentMiddleware` is auto-attached

Subagent support is implemented by `SubAgentMiddleware`. `create_deep_agent()` always attaches this middleware regardless of whether you pass `subagents`, and it cannot be removed via `excluded_middleware`. If you do not declare any subagents, a built-in `general-purpose` subagent registers automatically so the `task` tool is always available.

TodoList + `task` dispatch pattern

When you opt into `TodoListMiddleware`, the canonical loop looks like this:

1. The main agent calls `write_todos` to break the work into a structured list, for example `[research, analyze, write]`
2. For each TODO the main agent calls the `task` tool to dispatch the item to the right subagent
3. The subagent runs in an isolated context and returns only a compressed result
4. The main agent moves to the next TODO or composes the final answer

This `write_todos` → `task` → compressed result pattern is the core delegation cycle in Deep Agents. The main context only accumulates the todo list and the compressed results, which keeps token usage efficient.

When to Use a Subagent

Situation	Use a subagent?
Multi-step work with lots of intermediate results	✅ Yes
A domain that needs specialized knowledge or tools	✅ Yes
A task that is better handled by a different model	✅ Yes
A simple one-shot task	❌ Usually unnecessary
The main agent needs every intermediate detail	❌ Usually unnecessary

---

2. Defining a `SubAgent` (Dictionary Form)

A `SubAgent` is defined as a dictionary.

Required Fields

Field	Type	Description
`name`	`str`	Unique identifier
`description`	`str`	Role description used by the main agent when deciding whether to call it
`system_prompt`	`str`	Instructions for the subagent
`tools`	`list`	Tools available to the subagent
`mode`	`"isolated" | "fork"`	Context mode. Defaults to `"isolated"`

Optional Fields

Field	Type	Description
`model`	`str`	Model override (`"provider:model"`)
`middleware`	`list`	Additional middleware
`interrupt_on`	`dict`	Human-in-the-loop configuration
`skills`	`list[str]`	Skill source paths

`response_format`	`BaseModel`	Structured-output schema (`deepagents ≥ 0.5.3`)
`permissions`	`Permissions`	Per-path ACL — restrict the subagent's filesystem access

Context mode: isolated vs fork

The default `mode="isolated"` gives the subagent only the delegated task. It does not inherit the parent conversation, system prompt, or custom subagent skills. Use it for specialist work where the parent wants a compact final result.

`mode="fork"` is a continuation mode. The subagent inherits the parent's conversation and system prompt prefix, with the trailing delegation call removed. Use it when the worker should continue a diagnosis or draft with all prior context already visible. Forked subagents reject `skills`; if you add a `system_prompt`, it is appended as a fork-only addendum and may break prompt-cache reuse.

```
fixer = {
    "name": "fixer",
    "description": "Continue the parent diagnosis and implement the focused fix.",
    "mode": "fork",
    "tools": [read_file, edit_file],
}
```

> ➡️ TIP
>
> Do not treat fork mode as long-term memory. It passes the current conversation prefix to one delegated run; durable memory still belongs in checkpointers, stores, or explicit backend routes.

```
from typing import Literal
from tavily import TavilyClient
from deepagents import create_deep_agent

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])


def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
    include_raw_content: bool = False,
) -> dict:
    """Search the internet for information.

    Args:
        query: Question or keyword to search for
        max_results: Maximum number of results to return
        topic: Search topic category
        include_raw_content: Whether to include raw source content
    """
    return tavily_client.search(
        query,
        max_results=max_results,
        include_raw_content=include_raw_content,
        topic=topic,
    )
```

```
research_subagent = {
    "name": "researcher",
    "description": "Investigates a topic on the internet and summarizes the key information. Use it when
research is required.",
    "system_prompt": """You are an expert researcher.
Search the internet, collect accurate information, and summarize only the essentials.
Always write in English.
Keep the final result under 500 words.""",
    "tools": [internet_search],
}

print(f"Subagent created: {research_subagent['name']}")
print(f"Description: {research_subagent['description'][:60]}...")
```

```
# Create a main agent that can delegate work to the subagent
main_agent = create_deep_agent(
    model=model,
    system_prompt="""You are a project manager.
Analyze the user's request, delegate work to a specialist subagent when needed,
and combine the result into the final answer.
Respond in English.""",
    subagents=[research_subagent],
)

print("Main agent created (subagent: researcher)")
```

3. `CompiledSubAgent` — Plugging in a Custom LangGraph Runnable

You can also use a precompiled LangGraph runnable as a subagent. This is useful when the delegated work needs more complex workflow logic such as branching or loops.

```
from deepagents import CompiledSubAgent

# Example: wrap another runnable as a CompiledSubAgent
custom_graph = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="You are a data analyst. Gather information, analyze it, and produce insights.",
)

# Wrap the runnable
# TypedDict-like structure used by Deep Agents
# (same interface, but the runnable is already compiled)
data_analyst_subagent: CompiledSubAgent = {
    "name": "data-analyst",
    "description": "Collects data and produces analytical insights.",
    "runnable": custom_graph,
}

print(f"CompiledSubAgent created: {data_analyst_subagent['name']}")
```

4. General-Purpose Subagents

Deep Agents automatically provides a built-in general-purpose subagent even if you do not define one explicitly.

Default Behavior

- Uses the same system prompt as the main agent
- Has access to the same tools as the main agent
- Uses the same model as the main agent
- Inherits the main agent's skills

Overriding It

If you define a subagent with `name="general-purpose"`, it overrides the built-in one.

```
# Example: override the built-in general-purpose subagent
custom_gp_agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="You are a multi-task coordinator.",
    subagents=[
        research_subagent,
        {
            "name": "general-purpose",
            "description": "A general-purpose agent that handles multi-step work beyond research.",
            "system_prompt": "You are a general assistant. Solve the task step by step.",
            "tools": [internet_search],
        },
    ],
)

print("Created an agent that overrides the general-purpose subagent")
```

---

5. Context Propagation

Runtime context is automatically propagated to all subagents. You define the shape with `context_schema`, and you pass values through the `context` key in `config`.

Passing Subagent-Specific Context with Namespace Keys

If you use the format `"subagent-name:key"`, you can pass configuration that only a specific subagent receives.

```
config = {
    "context": {
        "user_id": "user-123",              # propagated to every agent
        "researcher:max_depth": 3,           # only for researcher
        "data-analyst:strict_mode": True,    # only for data-analyst
    }
}
```

Identifying subagents during streaming — `lc_agent_name`

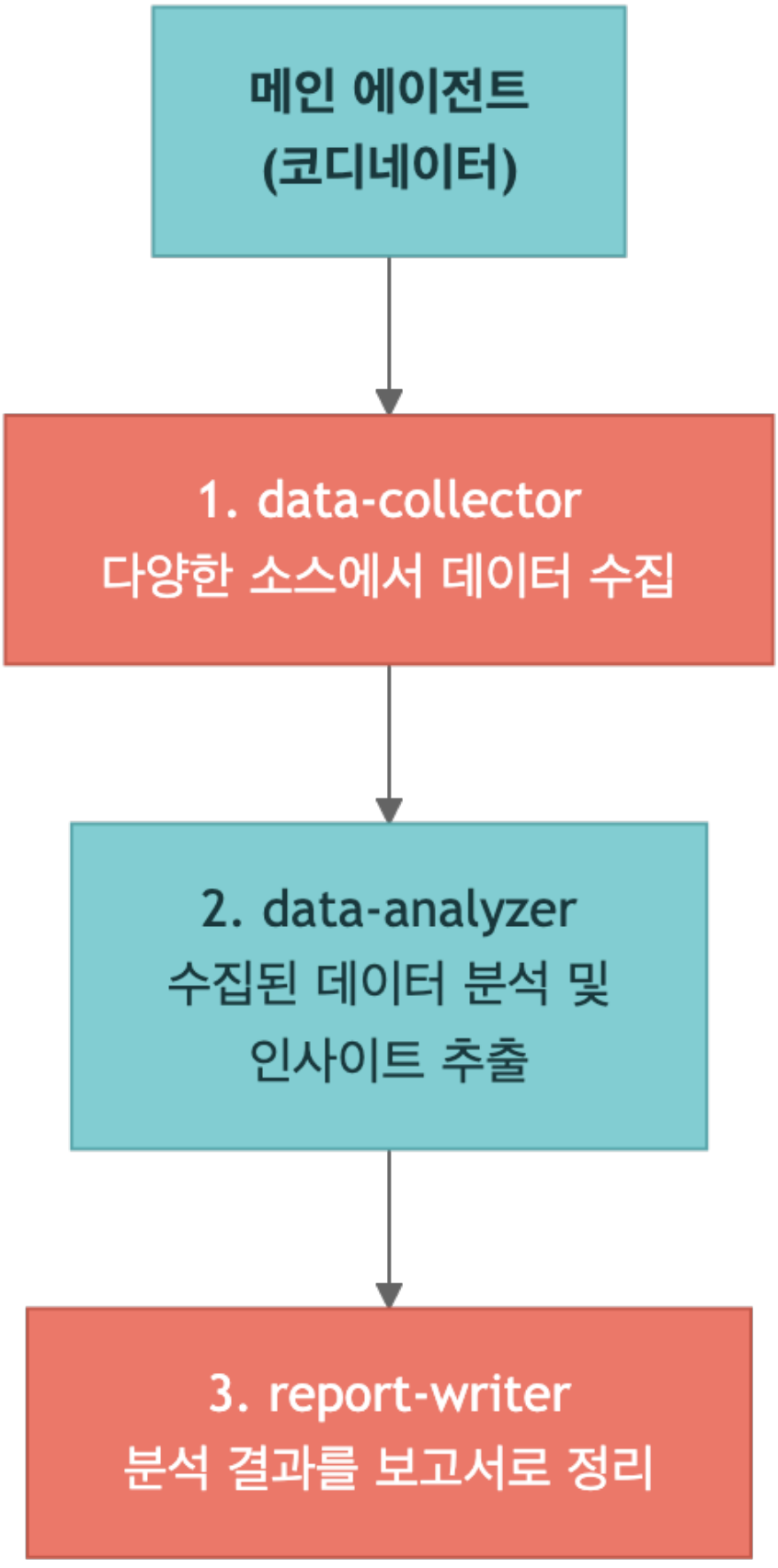
When you stream tokens through `agent.stream(..., stream_mode="messages")` or `astream_events()`, each event's metadata includes the `lc_agent_name` key. Use it to label UI output as "researcher searching…", "analyzer summarizing…", and similar.

```python
# Identify which subagent is emitting tokens during streaming
async for event in main_agent.astream_events(
    {"messages": [{"role": "user", "content": "..."}]},
    version="v2",
):
    name = event.get("metadata", {}).get("lc_agent_name")
    if event["event"] == "on_chat_model_stream" and name:
        print(f"[{name}] {event['data']['chunk'].content}", end="")
```

---

6. Multi-Subagent Pipelines

You can combine several subagents to build a pipeline such as collect → analyze → write.


메인 에이전트
(코디네이터)
1. data-collector
다양한 소스에서 데이터 수집
2. data-analyzer
수집된 데이터 분석 및
인사이트 추출
3. report-writer
분석 결과를 보고서로 정리

```
# Multi-subagent pipeline
pipeline_agent = create_deep_agent(
    model=model,
    system_prompt="""You are a project coordinator.
Analyze the user's request and delegate work in order:
1. Use data-collector to gather information.
2. Pass the results to data-analyzer for analysis.
3. Pass the analysis to report-writer to produce the final report.
Return the final report in English.""",
    subagents=[
        {
            "name": "data-collector",
            "description": "Collects raw information from external sources.",
            "system_prompt": "Collect as much relevant information as possible and return it in
structured form.",
            "tools": [internet_search],
        },
        {
            "name": "data-analyzer",
            "description": "Analyzes the collected data and extracts key insights.",
            "system_prompt": "Extract patterns, trends, and key insights. Return the analysis as bullet
points.",
            "tools": [],
        },
        {
            "name": "report-writer",
            "description": "Writes a professional report based on the analysis.",
            "system_prompt": "Write a clear report with the following structure: overview → key findings
→ conclusion.",
            "tools": [],
        },
    ],
)

print("Multi-subagent pipeline agent created")
```

7. Best Practices

1. Write clear descriptions

The subagent `description` is how the main agent decides when to call it. Make it specific.

2. Keep the result focused

A subagent should return a compact result rather than flooding the parent agent with raw data.

3. Use specialized tools per role

Give each subagent only the tools it actually needs.

4. Separate simple from complex work

Do not create subagents for tasks that the main agent can solve directly in one step.

Summary

Item	Description
Why subagents matter	They solve context bloat and enable specialization
Basic definition	Subagent dictionary with `name`, `description`, `system_prompt`, `tools` plus optional `response_format` / `permissions`
Advanced form	`CompiledSubAgent` wraps a precompiled runnable
Built-in fallback	`SubAgentMiddleware` is always attached (not removable); a default `general-purpose` subagent fills the slot
Dispatch pattern	`write_todos` → `task()` → compressed result
Context propagation	Runtime context is inherited; namespace keys (`"agent:key"`) target specific subagents
Streaming	`lc_agent_name` in event metadata identifies the active subagent
Pipeline pattern	Subagents can be chained into collect → analyze → write workflows

Next Steps

→ 06_memory_and_skills.ipynb: learn how long-term memory and skills work in Deep Agents.

06 Long

Term Memory & Skills

Learning Objectives

- Implement long-term memory with `CompositeBackend` + `StoreBackend`
- Understand the cross-thread memory-sharing pattern
- Inject agent context through `AGENTS.md`
- Understand the structure of skills (`SKILL.md`) and Progressive Disclosure
- Understand the difference between Skills and Memory

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
# Configure the OpenAI gpt-5.4 model
from langchain_openai import ChatOpenAI

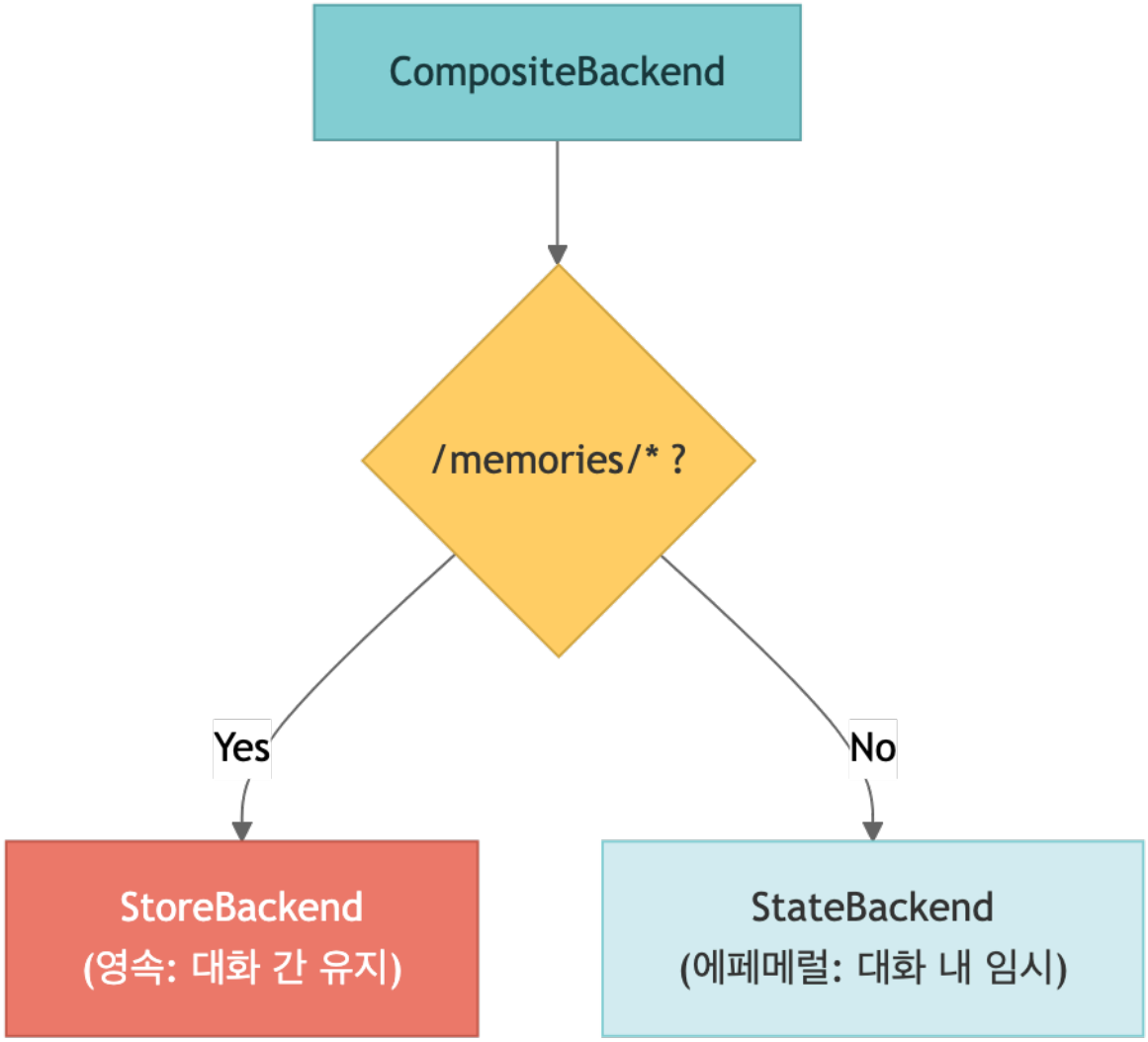
model = ChatOpenAI(model="gpt-5.4")
```

1. Why Long-Term Memory Matters

A default `StateBackend` agent forgets everything when the conversation thread ends. But a useful assistant often needs to preserve information across threads, such as:

- user preferences (coding style, preferred language)
- project conventions (architecture decisions, naming rules)
- feedback learned in previous conversations
- frequently referenced information (API docs, configuration values)

Solution: `CompositeBackend`



Files stored under `/memories/` can be accessed from any conversation thread.

```
from deepagents import create_deep_agent
from deepagents.backends import StateBackend, StoreBackend, CompositeBackend, FilesystemBackend
from langgraph.store.memory import InMemoryStore
from langgraph.store.postgres import PostgresStore  # Production
from langgraph.checkpoint.memory import MemorySaver


# 1. Create a store and a checkpointer
store = InMemoryStore()          # Development only (production: PostgresStore)
checkpointer = MemorySaver()     # Preserve agent state


# 2. Composite backend — persist only /memories/, keep the rest ephemeral.
#    StoreBackend uses a namespace lambda to declare the memory scope explicitly.
memory_backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/memories/": StoreBackend(
            namespace=lambda rt: (rt.context.user_id,),
        ),
    },
)
```

```
# 3. Create the agent
memory_agent = create_deep_agent(
    model=model,
    system_prompt="""You are a personal assistant.
Save information the user wants to remember under /memories/.
If there is previously saved memory, use it when responding.
Respond in English.""",
    backend=memory_backend,
    store=store,
    checkpointer=checkpointer,
)

print("Long-term memory agent created")
```

2. Cross-Thread Memory Sharing

Data stored in `StoreBackend` is shared across threads. In the example below, thread 1 stores preferences and thread 2 reads them later.

3. Injecting Context with `AGENTS.md`

If you use the `memory` parameter, the agent automatically loads `AGENTS.md` **files at startup** and injects them into the system prompt.

What is `AGENTS.md` ?

It is a Markdown file that contains rules, conventions, and context information that should always apply to the agent.

Characteristics

- Always loaded when the agent starts (not on demand)
- Injected into the system prompt through `<agent_memory>`
- You can specify multiple memory sources
- The agent can update `AGENTS.md` itself with `edit_file`

```
import tempfile

# Create a temporary directory to use as the root_dir for FilesystemBackend
tmp_dir = tempfile.mkdtemp()
print(f"Temporary directory created: {tmp_dir}")
```

4. Skills

Skills are modular instruction bundles that give the agent specialized domain knowledge.

Memory vs Skills

Comparison	Memory (`AGENTS.md`)	Skills (`SKILL.md`)
Loading	Always loaded	Loaded only when needed
File format	`AGENTS.md`	`SKILL.md` (YAML frontmatter)
Best fit	Rules and conventions that always apply	Large context needed for specific tasks
Token efficiency	Always consumes tokens	Saves tokens through Progressive Disclosure
Size	Best kept concise	Can be large (up to 10 MB)
Updates	Can be edited by the agent	Usually static

Progressive Disclosure

Skills are not fully loaded all at once:

1. At first, only the frontmatter (name, description, metadata) is loaded
2. The agent decides which skills are relevant to the user's request
3. Only then is the full content of the necessary skills loaded

This approach saves tokens while still giving the agent access to deep task-specific knowledge.

`SKILL.md` Structure

The `module` field is only set for interpreter skills (loaded by the QuickJS `CodeInterpreterMiddleware`); plain-text skills omit it.

```
---
name: web-research
description: >
  Step-by-step guide for structured web research.
  Covers information gathering, verification, and summarization.
license: MIT
compatibility: Python 3.8+
module: ./web_research.js    # (optional) interpreter skills only
metadata:
  category: research
  include_tools: list_issues create_issue
---

# Web Research Skill

## When to use it
- When the user asks for research on a topic
- When the request depends on current information

## Workflow
1. Design search queries
2. Gather information from multiple sources
3. Cross-check the information
4. Write a structured report
```

```
# Create an agent that uses skills
skilled_agent = create_deep_agent(
    model=model,
    system_prompt="You are a senior developer. Use the available skills to complete the task.",
    backend=FilesystemBackend(root_dir=tmp_dir, virtual_mode=True),
    skills=["/skills/"],
)

print("Skill-enabled agent created")
```

Skill-specific tools

Skill frontmatter can expose tools only after the skill is read. Put space-separated tool names under `metadata.include_tools`; a YAML list does not match any tool. To attach these tools, create `SkillsMiddleware` yourself and pass the tool list or resolver there, instead of passing those tools directly to `create_deep_agent`.

```
from deepagents.backends import FilesystemBackend
from deepagents.middleware import SkillsMiddleware

backend = FilesystemBackend(root_dir=tmp_dir, virtual_mode=True)
skills = SkillsMiddleware(
    sources=["/skills/"],
    backend=backend,
    tools=[list_issues, create_issue],
)

agent = create_deep_agent(
    model=model,
    middleware=[skills],
    backend=backend,
)
```

If MCP or generated tool names are unstable, pass a resolver function as the middleware's `tools`. The resolver receives each `include_tools` name and returns the matching runtime tools.

Pinning skills from input state

`pinned_skills` is not a `create_deep_agent(...)` constructor argument. It is part of the run input state. Use it when the application has already chosen a skill and you want the instructions and any `include_tools` tools loaded before the model decides.

```
result = agent.invoke({
    "messages": [{"role": "user", "content": "Use the LangGraph docs skill."}],
    "pinned_skills": ["langgraph-docs"],
})
```

Unknown pins are skipped, and skill tools become available the same way they do after the agent reads the matching `SKILL.md`.

---

5. Skill Source Priority

If you specify multiple skill sources, the later source wins.

```
skills=[
    "/skills/base/",     # base skills
    "/skills/user/",     # can override base
    "/skills/project/",  # highest priority
]
```

If the same skill name exists in multiple locations, the version from the last source is used.

6. Skill Inheritance for Subagents

Subagent Type	Skill Inheritance
Built-in general-purpose subagent	Automatically inherits the main agent's skills
Custom `SubAgent`	Does not inherit by default; requires an explicit `skills` parameter

```
subagent = {
    "name": "reviewer",
    "description": "Code review specialist",
    "system_prompt": "...",
    "tools": [],
    "skills": ["/skills/code-review/"],
}
```

> ➡️ TIP
>
> Follow least privilege when assigning skills to subagents. Handing a review-focused subagent unrelated deployment skills wastes tokens and invites confusion. Give each subagent only the skills that match its role.

When a custom subagent has skills, it gets its own `SkillsMiddleware` instance. Skill state is isolated in both directions: skills loaded by the subagent are not visible to the parent, and parent skills are not visible to the custom subagent.

Skills backend support matrix

Skill source directories can live in any of `StateBackend`, `StoreBackend`, `FilesystemBackend`, or `CompositeBackend`, but each backend has different operational characteristics.

Backend	Persistence	Sharing scope	Recommended use
`StateBackend`	Thread-local (ephemeral)	Current thread only	Ad-hoc experiments and tests

`StoreBackend`	Persistent (DB-backed)	Per namespace (user / org / assistant)	Team- or user-shared skills
`FilesystemBackend`	Persistent (disk)	Host directory	Standard skills committed to Git
`CompositeBackend`	Routed per path	Separated per path	Layered: `/skills/base/` + `/skills/user/`

6.10 Long-term Memory Types (in 0.5)

Deep Agents 0.5 classifies stored information into three categories. Each type uses a different storage mechanism, update cadence, and backend. The core rule is not to push all three into one backend, but to distribute them across the mechanisms that match their nature.

Type	Meaning	Storage example	Mechanism
Episodic	Past experience — conversation sessions, problem-solving trajectories	Thread history of past conversations	Checkpointers (per thread)
Procedural	Reusable instructions · skills · workflows	`SKILL.md`, procedure docs	Skills (loaded on demand)
Semantic	Facts · preferences · policies	`AGENTS.md`, `/memories/*.txt`	StoreBackend (always-on files)

Example: long-running preferences in `/memories/` (semantic) + episodic case retrieval via checkpointer (episodic) + repeated procedures as skills (procedural).

Scope patterns: agent-scoped / user-scoped / context-driven

The tuple returned by `StoreBackend`'s `namespace` function is the scope of the memory. The three most common patterns are:

Pattern	namespace example	Best fit
Agent-scoped	`(rt.server_info.assistant_id,)`	Organization-wide conventions and domain knowledge (shared across users)
User-scoped	`(rt.server_info.user.identity,)`	Default for personalization assistants — fully isolated per user

Context-driven	`(rt.context.user_id, rt.context.project_id)`	Multi-dimensional isolation using keys declared in `context_schema`

Agent-scoped — shared identity accumulation. Namespace is `(assistant_id,)`. All user conversations that use the same assistant share the same memory. Suitable for organization-wide conventions and domain knowledge, but do not store sensitive information here because information flows between users.

```
from deepagents.backends import StoreBackend

agent_scoped = StoreBackend(
    namespace=lambda rt: (rt.server_info.assistant_id,),
)
```

User-scoped — per-user isolation. Namespace is `(user_id,)` or `(assistant_id, user_id,)`. Each user's memory is fully isolated, so user A's preferences are never exposed in user B's conversation. This is the default for production personalization assistants.

```
user_scoped = StoreBackend(
    namespace=lambda rt: (rt.server_info.user.identity,),
)
```

Episodic memory via checkpointers

To turn past conversations from passive storage into searchable memory, wrap the threads saved by the checkpointer as a tool.

```
from langchain.tools import tool, ToolRuntime

@tool
async def search_past_conversations(query: str, runtime: ToolRuntime) -> str:
    """Find related context from prior conversations."""
    user_id = runtime.server_info.user.identity
    threads = await client.threads.search(
        metadata={"user_id": user_id},
        limit=5,
    )
    # Summarize threads into the needed format and return
    ...
```

With this pattern the agent can reference "how did I solve this before" on its own.

Read-only policy (organization-wide)

Shared organizational memory is an injection vector. Enforce write-blocking as follows.

```
# policies is org-scoped and read-only
routes = {
    "/policies/": StoreBackend(
        namespace=lambda rt: (rt.context.org_id,),
    ),
}
```

Combined with pattern 4 from Part IV ch15 (permissions), apply `write deny` on `/policies/**`. Policies should only be updated from application code.

Background consolidation agent

Updating memory during the conversation (hot path) increases latency, and the summary quality ends up tracking the model's hurried decisions. The alternative is to run a separate consolidation agent on a cron schedule.

```
// langgraph.json
{
  "graphs": {
    "agent": "./agent.py:agent",
    "consolidation_agent": "./consolidation_agent.py:agent"
  }
}
```

Register the cron schedule:

```
cron_job = await client.crons.create(
    assistant_id="consolidation_agent",
    schedule="0 */6 * * *",   # every 6 hours
    input={"messages": [{"role": "user", "content": "Consolidate recent memories."}]},
)
```

> **! WARNING**
>
> Keep the cron interval (`0 */6 * * *`) and the lookback window (`timedelta(hours=6)`) aligned to avoid gaps and duplicates.

Update-timing comparison

Timing	Approach	Latency	Takes effect
Hot path	Agent calls `edit_file` mid-conversation	Yes	Immediately
Background	Consolidation agent processes between sessions	Invisible to user	From next conversation

The default composition splits by type: sensitive or must-be-immediate preferences go on the hot path; long-term pattern accumulation runs in the background.

Summary

Item	Description
Long-term memory	Persist `/memories/` with `CompositeBackend` + `StoreBackend`
`AGENTS.md`	`memory=["/path/AGENTS.md"]` → always injected into the system prompt
Skills	`skills=["/skills/"]` → `SKILL.md` with Progressive Disclosure

Progressive Disclosure	Load frontmatter first → load full skill only when needed
Skill priority	Later sources win
Memory vs Skills	Memory = always loaded / Skills = loaded on demand
Three memory types	Episodic (checkpointer) / Procedural (skills) / Semantic (store)
Scope patterns	agent-scoped (shared accumulation) / user-scoped (isolated, default) / org-scoped (read-only)
Consolidation	Hot path: immediate, adds latency / Background cron: non-blocking, reflected from next conversation

Long-term memory and the skill system let agents accumulate knowledge beyond a single conversation and surface specialist capabilities when needed. The next chapter covers production-grade advanced features: Human-in-the-Loop, streaming, sandboxes, ACP, and the CLI.

Next Steps

→ 07_advanced.ipynb: learn advanced features such as Human-in-the-Loop, streaming, sandboxes, and ACP.

07 Advanced Features

Learning Objectives

- Implement a Human-in-the-Loop workflow
- Understand streaming modes and the namespace system
- Understand sandbox integrations such as Modal, Daytona, and Runloop
- Understand `CodeInterpreterMiddleware`, QuickJS, and programmatic subagent gates
- Use `RubricMiddleware` as a selective runtime quality gate
- Learn how ACP (Agent Client Protocol) connects agents to editors
- Learn how to use Deep Agents Code (`dcode`)

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
# Optional observability setup: LangSmith or Langfuse
# Set the keys in .env, or uncomment the lines below to enter them manually.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGCHAIN_TRACING_V2", "true")
    os.environ.setdefault("LANGCHAIN_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGCHAIN_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGCHAIN_PROJECT']}")

langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

```
from langchain_openai import ChatOpenAI

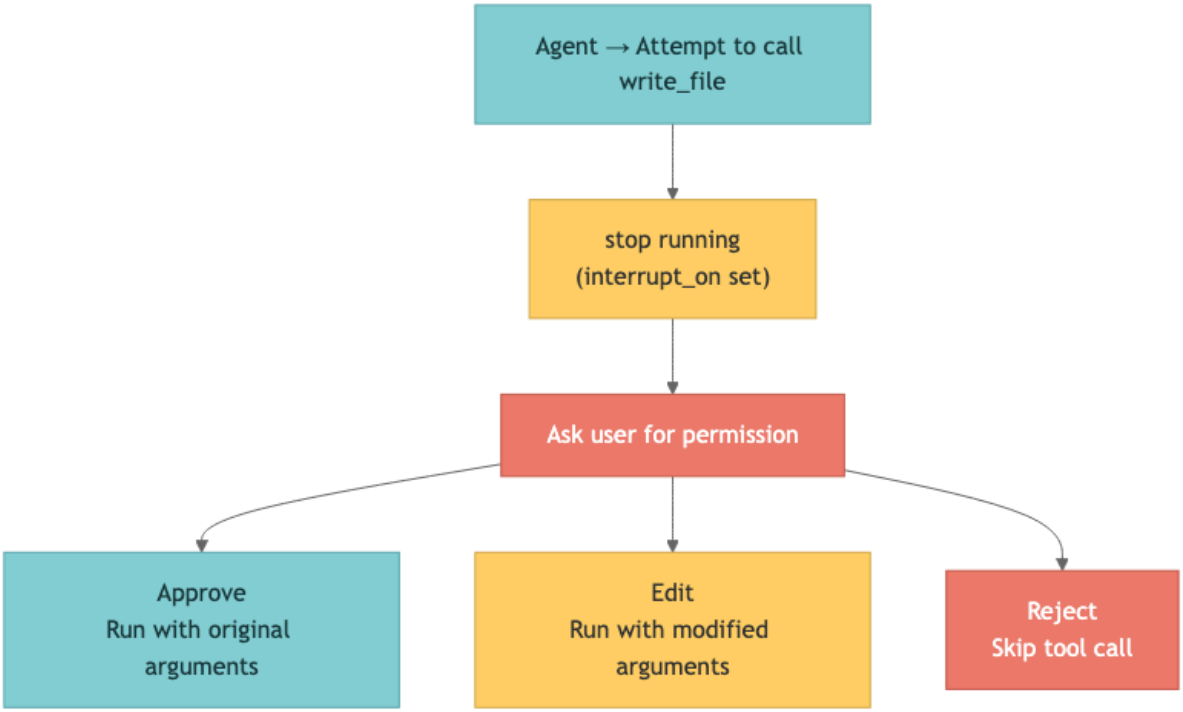
model = ChatOpenAI(model="gpt-4.1")

print(f"Model configured: {model.model_name}")
```

1. Human-in-the-Loop (HITL)

Human-in-the-Loop is a workflow in which the agent requires human approval before calling sensitive tools.

How it Works



Required Condition

- Checkpointer: required to preserve the agent's state between interrupt and resume

```
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

# Choose which tools require approval with interrupt_on
hitl_agent = create_deep_agent(
    model=model,
    system_prompt="You are a file management assistant. Respond in English.",
    checkpointer=MemorySaver(),  # required!
    interrupt_on={
        "write_file": True,
        "edit_file": True,
    },
)

print("Human-in-the-Loop agent created")
print("write_file and edit_file now require approval")
```

```
# Run the agent — it will pause before write_file
config = {"configurable": {"thread_id": "hitl-demo"}}

result = hitl_agent.invoke(
    {"messages": [{"role": "user", "content": "Create a file named config.yaml and write 'debug:
```

```
true'."}]},
    config={**config, **lf_config},
)

# Inspect interrupts
if "__interrupt__" in result:
    interrupt_info = result["__interrupt__"]
    print("The agent was interrupted")
    print(f"Pending approvals: {len(interrupt_info)}")
    for item in interrupt_info:
        val = item.value if hasattr(item, 'value') else item
        print(f"  - Interrupt payload: {val}")
else:
    print("Completed without interruption:")
    print(result["messages"][-1].content)
```

```
from langgraph.types import Command

# Resume with approval
if "__interrupt__" in result:
    resumed = hitl_agent.invoke(
        Command(
            resume={
                "decisions": [
                    {"type": "approve"}
                    # {"type": "reject"}
                    # {"type": "edit", "args": {"content": "debug: false"}}
                ]
            }
        ),
        config={**config, **lf_config},
    )

    print("✅ Resumed after approval:")
    print(resumed["messages"][-1].content)
```

2. Advanced Streaming

Deep Agents runs on top of LangGraph's streaming infrastructure.

Stream Modes

Mode	Description	Use Case
`"updates"`	State updates after each node finishes	Progress tracking
`"messages"`	Token-level streaming	Real-time text output
`"custom"`	Events emitted inside tools or nodes	Custom progress reporting

Namespace System

Events from subagents are separated by namespace:

```
()                                  # main agent
("tools:abc123",)                  # subagent (tool call ID)
("tools:abc123", "model:def456")  # inner node inside a subagent
```

```
from typing import Literal

LOCAL_DOC_SNIPPETS = {
    "langgraph": "LangGraph supports persistence, interrupts, subgraphs, streaming, and fault
tolerance.",
    "python 3.13": "Python 3.13 improves the REPL, error messages, and standard-library behavior.",
}


def internet_search(query: str, max_results: int = 3, topic: Literal["general", "news"] = "general") ->
dict:
    """Search local course snippets. The live harness does not need an external search key."""
    matches = [v for k, v in LOCAL_DOC_SNIPPETS.items() if k in query.lower()]
    return {"query": query, "topic": topic, "results": matches[:max_results] or
list(LOCAL_DOC_SNIPPETS.values())[:max_results]}


stream_agent = create_deep_agent(
    model=model,
    system_prompt="You are a research coordinator. Respond in English.",
    subagents=[
        {
            "name": "researcher",
            "description": "Uses local course snippets to investigate topics.",
            "system_prompt": "Use only the search tool results, then summarize the requested information
concisely.",
            "tools": [internet_search],
        }
    ],
)

print("Streaming demo agent created")
```

```
# Stream events with subgraphs — use namespaces to distinguish sources
print("=== Subagent event streaming ===")
print()

for namespace, chunk in stream_agent.stream(
    {"messages": [{"role": "user", "content": "Research the latest LangGraph features."}]},
    stream_mode="updates",
    subgraphs=True,
    config=lf_config,
):
    source = "[main]" if not namespace else f"[subagent: {namespace}]"

    for node_name, node_data in chunk.items():
        if not node_data:
            continue
        msgs = node_data.get("messages", [])
        if hasattr(msgs, "value"):
            msgs = msgs.value
        if msgs:
            last_msg = msgs[-1]
            if hasattr(last_msg, "tool_calls") and last_msg.tool_calls:
                for tc in last_msg.tool_calls:
                    print(f"{source} 🔧 tool call: {tc['name']}")
            elif hasattr(last_msg, "content") and last_msg.content:
                content = last_msg.content if isinstance(last_msg.content, str) else
str(last_msg.content)
                if content.strip() and not hasattr(last_msg, "tool_call_id"):
                    preview = content[:100].replace("\n", " ")
                    print(f"{source} 💬 {preview}...")
```

```
# Use multiple stream modes together
print("=== Multiple stream modes ===")
print()

for event in stream_agent.stream(
    {"messages": [{"role": "user", "content": "Explain one new feature in Python 3.13 in a single
sentence."}]},
    stream_mode=["updates", "messages"],
    subgraphs=True,
    config=lf_config,
):
    *namespace_parts, mode, data = event
    if mode == "updates":
        for node_name in data:
            print(f"  [updates] node '{node_name}' completed")
    elif mode == "messages":
        msg, metadata = data
        if hasattr(msg, "content") and msg.content and metadata and metadata.get("langgraph_node") ==
"model":
            print(msg.content, end="", flush=True)

print()
```

3. Sandboxes

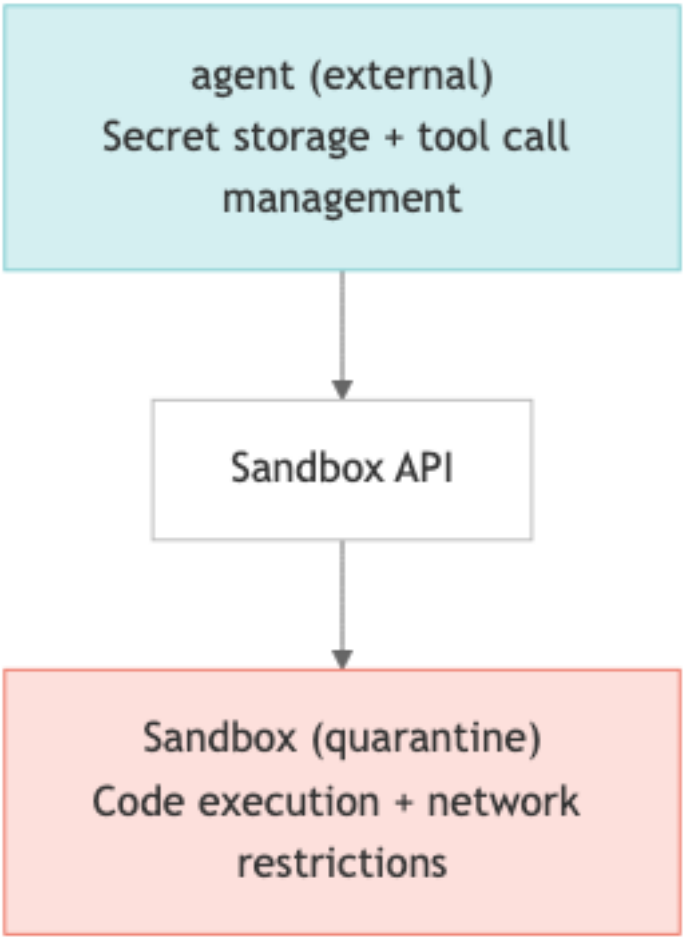
A sandbox lets the agent run code in an isolated environment. That prevents it from accessing the host machine's files, network, or credentials directly.

Supported Providers

Provider	Characteristics	Best Fit
Modal	GPU support, ML workloads	AI / ML tasks
Daytona	TypeScript / Python, fast cold starts	Web development
Runloop	Disposable devboxes, isolated execution	Code testing

Architecture Pattern

Use the sandbox as a tool (recommended)



⚠️ Security Guidelines

- Never put secrets inside the sandbox — the agent may leak them
- Manage credentials only through external tools
- Use Human-in-the-Loop approval for sensitive operations
- Block unnecessary network access

```
# Sandbox integration example (reference only — requires provider-specific setup)

sandbox_example_code = """
# pip install deepagents-modal
from deepagents.backends.sandbox import ModalSandbox

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    backend=ModalSandbox(
        image="python:3.12-slim",
        gpu="T4",  # GPU support
    ),
)
"""

print("Sandbox integration example (reference only):")
print(sandbox_example_code)
```

4. Interpreters — CodeInterpreterMiddleware

Deep Agents can use a QuickJS-based interpreter as an internal workspace for tool composition, intermediate variables, structured data transforms, and programmatic subagent fan-out/fan-in.

Installation

```
pip install -U "deepagents[quickjs]"
# or
uv add "deepagents[quickjs]"
```

Programmatic Tool Calling vs programmatic subagents

Programmatic Tool Calling exposes allowlisted tools as `tools.*` functions inside QuickJS. Programmatic subagents are separate: the interpreter exposes a top-level `task(...)` function when `CodeInterpreterMiddleware(subagents=True)` is enabled.

```
const topics = ["retrieval", "memory", "evaluation"];
const reports = await Promise.all(
  topics.map((topic) => task({
    description: `Research ${topic}`,
    subagent_type: "general-purpose",
  })),
);
```

Use a least-privilege PTC allowlist for sensitive tools. Do not assume parent-level approvals automatically cover every interpreter dispatch.

```
# Interpreter configuration example — requires the deepagents[quickjs] extra
interpreter_example = r'''
from deepagents import create_deep_agent
from langchain_quickjs import CodeInterpreterMiddleware
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model="openai:gpt-5.4",
    checkpointer=MemorySaver(),
    middleware=[CodeInterpreterMiddleware(ptc=["web_search"], subagents=True, mode="thread")],
)

# Inside QuickJS, use task(...) separately from tools.webSearch(...).
'''
print(interpreter_example)
```

Compatibility gate in the current course environment

The official Programmatic Subagents pattern relies on the QuickJS interpreter and top-level `task(...)`. The current course venv has Deep Agents installed, but the `langchain_quickjs` extra may not be present. Therefore the default notebook path detects availability and falls back to ordinary `SubAgent` orchestration.

```
from importlib.metadata import PackageNotFoundError, version
from importlib.util import find_spec
```

```
try:
    deepagents_version = version("deepagents")
except PackageNotFoundError:
    deepagents_version = "not installed"
quickjs_available = find_spec("langchain_quickjs") is not None
fallback_pattern = "Programmatic task(...) demo" if quickjs_available else "ordinary SubAgent fan-out/
fan-in fallback"
print("deepagents:", deepagents_version)
print("langchain_quickjs available:", quickjs_available)
print("recommended path:", fallback_pattern)
```

5. RubricMiddleware — runtime LLM-as-a-judge

`RubricMiddleware` evaluates an agent response against a rubric during the same invocation. Use it as a selective runtime quality gate for high-risk outputs, final deliverables, or external-send steps. Keep offline regression checks in LangSmith/AgentEvals, and use runtime rubrics only where the extra latency and cost are justified.

Write rubrics as criterion coverage, not as a vague score. Each criterion should name an observable requirement, and the grader should be separate from the work model when cost or bias matters. Current Deep Agents rubric hooks also expose per-iteration evaluation events, so production demos can log or stream the grader result without replacing offline evaluation datasets.

```
# RubricMiddleware configuration example — reference only
rubric_example = r'''
from deepagents import RubricMiddleware, create_deep_agent
from deepagents.middleware.rubric import RubricEvaluation
from langgraph.checkpoint.memory import MemorySaver


def log_evaluation(event: RubricEvaluation) -> None:
    print(f"rubric iteration {event['iteration']}: {event['result']}")

agent = create_deep_agent(
    model="openai:gpt-5.4",
    checkpointer=MemorySaver(),
    middleware=[RubricMiddleware(
        model="openai:gpt-5.4-mini",
        system_prompt=(
            "Check all criteria: factual accuracy, cited evidence, "
            "actionable next step, and no unsafe external side effect."
        ),
        max_iterations=3,
        on_evaluation=log_evaluation,
    )],
)
'''
print(rubric_example)
```

6. ACP (Agent Client Protocol)

ACP standardizes communication between coding agents and editors / IDEs.

Supported Editors

- Zed — native integration
- JetBrains IDEs — built-in support
- VS Code — `vscode-acp` plugin
- Neovim — ACP-compatible plugins

MCP vs ACP

Protocol	Purpose
MCP (Model Context Protocol)	External tool integration
ACP (Agent Client Protocol)	Editor ↔ agent integration

```
# ACP server implementation example (reference only)
acp_example_code = """
# pip install deepagents-acp
from deepagents import create_deep_agent
from deepagents_acp import AgentServerACP
from langgraph.checkpoint.memory import MemorySaver

# Create the agent
agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    system_prompt="You are a coding assistant.",
    checkpointer=MemorySaver(),
)

# Run the ACP server (stdio mode)
server = AgentServerACP(agent)
server.run()
"""

print("ACP server implementation example (reference only):")
print(acp_example_code)
```

---

7. Deep Agents Code (`dcode`)

Deep Agents Code is a terminal coding agent built on top of the SDK. Current CLI material uses the `deepagents-code` package and the `dcode` command.

Installation and Execution

```
# Install script
curl -LsSf https://langch.in/dcode | bash

# Check help without installing
uvx --from deepagents-code dcode --help

# Run
dcode
```

Main Options

Option	Description
`-M/--model MODEL`	Choose the model
`-n/--non-interactive`	Non-interactive mode for a single task
`-S/--shell-allow-list`	Specify allowed shell commands
`--interpreter`	Enable the interpreter
`--interpreter-tools`	Specify tools available to the interpreter
`--stdin`	Read the prompt from standard input
`--json`	Emit JSON output
`--acp`	Run as an ACP server over stdio

Interactive Commands

Command	Description
`/model`	Change the model
`/remember`	Store information in memory
`/tokens`	Inspect token usage
`!command`	Run a shell command

Configuration and Memory

- Global config: `~/.deepagents/config.toml`
- Project instructions: `AGENTS.md`
- Skills: progressive disclosure through `SKILL.md`
- Subagents: configured through files or project instructions

```
# Deep Agents Code examples (run in the shell)
cli_examples = """
# Check help without installing
uvx --from deepagents-code dcode --help

# Non-interactive execution with a specific model
uvx --from deepagents-code dcode -M gpt-5.4 -n "Write the README.md file for this project"

# Enable interpreter support
uvx --from deepagents-code dcode --interpreter "Inspect this repository and summarize risks"
"""

print("dcode usage examples (run in the terminal):")
print(cli_examples)
```

Full Track Summary

Notebook	Topic	Key APIs
01	Introduction	`deepagents.__version__`
02	Quickstart	`create_deep_agent()`, `invoke()`, `stream()`
03	Customization	`model`, `system_prompt`, `tools`, `response_format`
04	Backends	`StateBackend`, `FilesystemBackend`, `StoreBackend`, `CompositeBackend`
05	Subagents	`SubAgent`, `CompiledSubAgent`, `subagents`
06	Memory & Skills	`memory`, `skills`, `AGENTS.md`, `SKILL.md`
07	Advanced Features	`interrupt_on`, streaming, `CodeInterpreterMiddleware`, `RubricMiddleware`, Sandbox, ACP, `dcode`

Next Steps

→ Continue to 08_harness.ipynb → Or jump to the advanced track at ../05_advanced/00_migration.ipynb

08 Agent Harness

Learning Objectives

- Understand the concept and role of AgentHarness
- Learn the harness's core capabilities: planning, filesystem access, and task delegation
- Understand context management through offloading and summarization
- Configure code execution and Human-in-the-Loop
- Connect skills and memory systems

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"Model configured: {model.model_name}")
```

1. AgentHarness Concept

AgentHarness is a comprehensive capability provider for long-running autonomous agents. It bundles together the infrastructure needed for complex multi-step agent work.

Core Capabilities Provided by the Harness

Capability	Description
Planning	Manage structured task lists with `write_todos`
Filesystem	Read, write, and search files in virtual or local environments
Task Delegation	Delegate work through subagents
Context Management	Compress context through offloading and summarization

Code Execution	Run code safely in sandboxed environments
Human-in-the-Loop	Require approval for sensitive operations
Skills & Memory	Use specialized workflows and persistent knowledge

When you call `create_deep_agent()`, all of these pieces are assembled into a single agent.

```
# AgentHarness concept — create_deep_agent assembles the harness
harness_config = {
    "model": "openai:gpt-5.4",
    "system_prompt": "You are a project management assistant.",
    "planning": True,
    "filesystem": True,
    "subagents": [],
    "context_management": True,
}

print("AgentHarness components:")
for key, value in harness_config.items():
    print(f"  {key}: {value}")
```

2. Planning Tools

The agent uses the `write_todos` tool to break complex work into a structured task list. Each task has a status:

Status	Description
`pending`	Not started yet
`in_progress`	Currently in progress
`completed`	Finished

```
# write_todos example — structured task list
todo_list = [
    {"task": "Analyze the project structure", "status": "completed"},
    {"task": "Design the API endpoints", "status": "in_progress"},
    {"task": "Write the database schema", "status": "pending"},
    {"task": "Write the tests", "status": "pending"},
    {"task": "Document the project", "status": "pending"},
]

print("=== Agent task list ===")
for i, item in enumerate(todo_list, 1):
    icon = {"completed": "[x]", "in_progress": "[-]", "pending": "[ ]"}
    print(f"  {icon[item['status']]} {i}. {item['task']}")
```

3. Virtual Filesystem

The harness supports standard file operations through configurable filesystem backends.

Tool	Description
`ls`	List directory contents with metadata
`read_file`	Read file contents with line numbers and multimodal returns — images (PNG, JPG, GIF, WebP, HEIC), video (MP4, MOV, AVI), audio (WAV, MP3, AAC, FLAC), documents (PDF, PPT)
`write_file`	Create files
`edit_file`	Replace strings inside files
`glob`	Search for files by pattern
`grep`	Search file contents in different output modes
`execute`	Run shell commands (sandbox backends only)

```
# Example filesystem tool calls (reference only)
fs_operations = {
    "ls": 'ls(path="/project/src")',
    "read_file": 'read_file(path="/project/src/main.py")',
    "write_file": 'write_file(path="/project/config.yaml", content="debug: true")',
    "edit_file": 'edit_file(path="/project/src/main.py", old="v1", new="v2")',
    "glob": 'glob(pattern="**/*.py")',
    "grep": 'grep(pattern="TODO", path="/project/src")',
}

print("=== Filesystem tool call examples ===")
for tool_name, call_example in fs_operations.items():
    print(f"  {tool_name:12s} -> {call_example}")
```

4. Task Delegation — Subagents

The harness allows the main agent to create temporary subagents for isolated multi-step work.

Advantages of Subagents

Advantage	Description
Context isolation	Subagent execution does not pollute the main context
Parallel execution	Multiple subagents can run at the same time
Specialization	Each subagent can get its own tools and prompt
Token efficiency	The main agent receives a compressed result

```python
# Example subagent delegation configuration (reference only)
subagent_config = [
    {
        "name": "researcher",
        "description": "Investigates information using web search.",
        "system_prompt": "Summarize search results concisely.",
        "tools": ["internet_search"],
    },
    {
        "name": "coder",
        "description": "Writes and tests code.",
        "system_prompt": "Write clean and testable code.",
        "tools": ["write_file", "execute"],
    },
]

print("=== Subagent configuration ===")
for sa in subagent_config:
    print(f"  [{sa['name']}] {sa['description']}")
    print(f"    tools: {', '.join(sa['tools'])}")
```

5. Context Management

The biggest challenge for long-running agents is the context window limit. The harness addresses it with two main techniques.

Input Context Assembly

The initial prompt is assembled from the system prompt, instructions, memory guidelines, skill information, and filesystem documentation.

Runtime Context Compression

Technique	Behavior	Trigger
Offloading	Stores content above a configurable threshold (default 20,000 tokens) on disk and keeps only pointers in context	Based on content size
Summarization	Compresses conversation history into a structured summary (session intent / artifacts / next steps). Triggered around 85% of `max_input_tokens`; the most recent 10% of messages are preserved verbatim. Falls back to a 170k-token threshold when `max_input_tokens` is not set.	Triggered when the model window limit is approached

The original data is preserved in filesystem storage, so information is not lost.

```python
# Example context-management settings (reference only)
context_config = {
    "offloading": {
        "enabled": True,
        "threshold_tokens": 20000,
        "storage": "filesystem",
    },
    "summarization": {
        "enabled": True,
        "trigger": "window_limit_approach",
        "preserve_original": True,
    },
}

print("=== Context management settings ===")
for section, settings in context_config.items():
    print(f"\n[{section}]")
    for key, value in settings.items():
        print(f"  {key}: {value}")
```

6. Code Execution

Sandbox backends expose the `execute` tool, which runs commands in an isolated environment. That improves safety, cleanliness, and reproducibility without affecting the host system.

Two execution paths: sandbox `execute` (shell) vs QuickJS `CodeInterpreterMiddleware`

The harness exposes two distinct execution paths. The sandbox backend contributes a shell-style `execute` tool, while the interpreter middleware contributes a QuickJS eval tool. They solve different problems.

Aspect	Sandbox `execute` (shell)	QuickJS `CodeInterpreterMiddleware`
Where it runs	Isolated environment outside the agent (Modal / Daytona / Runloop …)	QuickJS VM inside the agent loop
Language	Shell commands → any language (Python, Node, …)	JavaScript (QuickJS)
Network / filesystem	On by default (governed by policy)	Off by default — only bridged tools
Package install	`pip install` / `npm install` supported	Not supported
Best fit	Installing packages, running tests, processing large data	Composing tool calls, subagent fan-out, structured data transforms
How to enable	Swap the backend, e.g. `backend=ModalSandbox(...)`	Add `middleware=[CodeInterpreterMiddleware(...)]`

```python
# Example sandboxed execute calls (reference only)
execute_examples = [
    {"command": "python -c 'print(2+2)'", "desc": "Run a Python snippet"},
    {"command": "pip install requests", "desc": "Install a package"},
    {"command": "pytest tests/", "desc": "Run the test suite"},
]

print("=== Sandbox execute tool examples ===")
for ex in execute_examples:
    print(f"  $ {ex['command']}")
    print(f"    -> {ex['desc']}")
```

---

7. Human-in-the-Loop

You can require human approval for selected tool calls through interrupt settings.

```python
# Example Human-in-the-Loop configuration (reference only)
hitl_config = {
    "interrupt_on": {
        "write_file": True,
        "edit_file": True,
        "execute": True,
    }
}

print("=== Human-in-the-Loop configuration ===")
print("Tools that require approval:")
for tool, enabled in hitl_config["interrupt_on"].items():
    status = "approval required" if enabled else "automatic"
    print(f"  {tool}: {status}")

print("\nDecision options: approve, reject, edit")
```

---

8. Skills and Memory

Skills

Skills are specialized workflows that follow the Agent Skills standard. They are loaded progressively when relevant, which reduces token usage.

- Each skill is defined in a `SKILL.md` file
- Skills are activated when the triggering conditions match
- They package tools, prompts, and workflows together

Memory

Memory uses `AGENTS.md` -style persistent context files. It stores reusable guidelines, preferences, and project knowledge beyond a single conversation.

Scope	Location	Range

Global memory	`~/.deepagents/\<agent\>/memories/`	All projects
Project memory	`.deepagents/AGENTS.md`	Current project

```python
# Example skill and memory configuration (reference only)
skills_config = [
    {"name": "code-review", "trigger": "when the user asks for a code review"},
    {"name": "test-writer", "trigger": "when the user asks for tests"},
    {"name": "doc-generator", "trigger": "when the user asks for documentation"},
]

memory_config = {
    "global": "~/.deepagents/my-agent/memories/",
    "project": ".deepagents/AGENTS.md",
}

print("=== Skill configuration ===")
for skill in skills_config:
    print(f"  [{skill['name']}] trigger: {skill['trigger']}")

print("\n=== Memory configuration ===")
for scope, path in memory_config.items():
    print(f"  {scope}: {path}")
```

9. Harness Profiles

`HarnessProfile` packages provider/model-specific harness defaults without touching the `create_deep_agent()` call site. System prompt suffixes, tool description overrides, excluded tools and middleware, general-purpose subagent options, and extra middleware can all be registered as a profile.

`HarnessProfile` (7 fields)

Field	Meaning
`base_system_prompt`	Replace the base system prompt itself
`system_prompt_suffix`	Append text to the assembled base prompt
`tool_description_overrides`	Per-tool description overrides (mapping)
`excluded_tools`	Tools to remove by name after injection (set)
`excluded_middleware`	Middleware classes to remove (set)
`extra_middleware`	Additional middleware instances to attach
`general_purpose_subagent`	Enable, disable, or customize the general-purpose subagent via `GeneralPurposeSubagentProfile`

Merge semantics

When multiple profiles match:

- A later profile overrides earlier scalar fields (for example `base_system_prompt`).
- Mapping-style fields such as `tool_description_overrides` merge key by key.
- Set-style fields (`excluded_tools`, `excluded_middleware`) take the union.

`ProviderProfile` (3 fields)

`ProviderProfile` does not change harness behavior; it bundles initialization arguments for `init_chat_model()` — credential checks, provider default temperature, runtime headers, and so on.

Field	Meaning
`init_kwargs`	Default kwargs passed to `init_chat_model()` (for example `temperature`)
`credentials_check`	Callable run before model initialization to verify credentials
`runtime_headers`	HTTP header mapping attached to every request

YAML / JSON workflow — `HarnessProfileConfig.from_dict`

`HarnessProfileConfig` provides `from_dict()`, `to_dict()`, and `from_harness_profile()` class methods, so profiles can be managed as YAML and applied per environment without code changes.

```yaml
# profile.yaml
base_system_prompt: You are helpful.
system_prompt_suffix: Respond briefly.
excluded_tools:
  - execute
excluded_middleware:
  - SummarizationMiddleware
general_purpose_subagent:
  enabled: false
```

```python
import yaml
from deepagents import HarnessProfileConfig, register_harness_profile

with open("profile.yaml") as f:
    register_harness_profile(
        "openai:gpt-5.4",
        HarnessProfileConfig.from_dict(yaml.safe_load(f)),
    )
```

Entry-point plugins

Profiles can ship as packages via `pyproject.toml` entry points. The load order is built-ins → entry-point plugins → user code's direct `register_*_profile` calls.

```toml
[project.entry-points."deepagents.harness_profiles"]
"anthropic:claude-sonnet-4-6" = "my_pkg.profiles:claude_profile"

[project.entry-points."deepagents.provider_profiles"]
"openai" = "my_pkg.profiles:openai_provider"
```

Summary

Topic	Core Concept	Key API / Tool
Harness concept	Comprehensive capability provider for long-running agents	`create_deep_agent()`
Planning tools	Structured task list management	`write_todos`
Filesystem	Virtual and local file operations	`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep` , `execute`
Subagents	Isolated task delegation, parallel execution	`subagents` , `task`
Context management	Offloading (20K tokens), summarization	automatic
Code execution	Safe command execution in sandboxes	`execute`
HITL	Human approval for sensitive tool calls	`interrupt_on`
Skills / Memory	Specialized workflows + persistent context	`SKILL.md` , `AGENTS.md`

Next Steps

→ Continue to 09_comparison.ipynb

References:

- Deep Agents Harness

09 Comparing External Frameworks

Learning Objectives

- Understand the deeper differences between Deep Agents, LangGraph, and LangChain
- Compare them with OpenCode and the Claude Agent SDK
- Analyze differences in architecture, flexibility, and ecosystem
- Learn which framework to recommend for different use cases
- Understand migration considerations

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"Model configured: {model.model_name}")
```

1. Comparison Overview

When choosing an AI agent framework, you need to consider several factors, including model support, architecture, ecosystem, and license.

In this notebook, you compare three major options:

- LangChain Deep Agents — a model-agnostic agent harness
- OpenCode — a coding agent environment centered on the terminal / desktop / IDE
- Claude Agent SDK — Anthropic's SDK for Claude-based agents

2. Deep Agents vs OpenCode vs Claude Agent SDK

Basic Comparison

Feature	LangChain Deep Agents	OpenCode	Claude Agent SDK
Model support	Model-agnostic (Anthropic, OpenAI, 100+ providers)	75+ providers, including local models through Ollama	Claude-only
License	MIT	MIT	MIT (SDK), proprietary (Claude Code)
SDK	Python, TypeScript + CLI	Terminal, desktop, IDE integration	Python, TypeScript
Sandboxing	Can be integrated as a tool	Not supported	Not supported
State management	Supports time travel	Not supported	Supports time travel
Observability	Native LangSmith support	None	None

```python
# Print a framework comparison table
frameworks = {
    "LangChain Deep Agents": {
        "model_support": "100+ providers (model-agnostic)",
        "license": "MIT",
        "sdk": "Python, TypeScript, CLI",
        "sandbox": "Integrated support",
        "time_travel": "Supported",
    },
    "OpenCode": {
        "model_support": "75+ providers (including local models)",
        "license": "MIT",
        "sdk": "Terminal, desktop, IDE",
        "sandbox": "Not supported",
        "time_travel": "Not supported",
    },
    "Claude Agent SDK": {
        "model_support": "Claude only",
        "license": "MIT (SDK)",
        "sdk": "Python, TypeScript",
        "sandbox": "Not supported",
        "time_travel": "Supported",
    },
}

print("=== Framework Comparison ===")
for name, features in frameworks.items():
    print(f"\n[{name}]")
    for key, value in features.items():
        print(f"  {key}: {value}")
```

3. Comparing Core Capabilities

Shared Capabilities

All three frameworks support the following categories:

- file operations (read, write, edit)
- shell command execution
- search features (`grep` , `glob`)
- planning support (task lists)
- Human-in-the-Loop (with different permission models)

Differentiating Capabilities

Capability	Deep Agents	OpenCode	Claude Agent SDK
Core tools	Files, shell, search, planning	Files, shell, search, planning	Files, shell, search, planning
Sandbox integration	Can be integrated as a tool	No	No
Pluggable backends	Storage + filesystem backends	No	No
Virtual filesystem	Yes, through pluggable backends	No	No
Native tracing	LangSmith	No	No

3-1. Three Core Differentiators

Look past the table cells and Deep Agents diverges from the other two along three axes. These are design decisions, not feature checks — figure out which axis matters most for your project and the choice gets easier.

Execution Model — graph-based state machine vs single-pass ReAct loop

Deep Agents runs on LangGraph's `StateGraph` . It is assembled from nodes, edges, and subgraphs, with branching, looping, interrupts, and resumes all expressed in the graph itself. Time travel (branching from a past checkpoint), interrupt/resume (the standard HITL path), and subgraph-scoped streaming fall out of that design. OpenCode and Claude Agent SDK are ReAct-loop centric, so state branching and subgraph isolation cost extra integration work.

Deployment & Multi-Tenancy — LangSmith Deployments integration

Deep Agents deploys directly to LangSmith Deployments with built-in multi-tenancy via `assistant_id` / `thread_id` / `user_id`. Cron schedules (e.g. consolidation agents), the Store (persistent memory), and the Checkpointer are provisioned automatically at deploy time. OpenCode is a local terminal tool; Claude Agent SDK is closer to a model-calling SDK — both require you to build the surrounding infrastructure yourself.

Per-Model Configuration — `HarnessProfile` / `ProviderProfile`

Deep Agents 0.5+ packages provider/model defaults as profiles. The same codebase can switch between `openai:gpt-5.4` and `anthropic:claude-sonnet-4-6` without touching call sites. Claude Agent SDK is Claude-only with no model abstraction layer, and OpenCode supports 75+ providers but lacks a profile concept that captures the differences between them.

---

4. Architecture Comparison

LangChain Deep Agents

- Pluggable storage backends — state, filesystem, and store layers can be configured independently
- Virtual filesystem — can switch between local, in-memory, or sandboxed backends
- LangGraph-based runtime — supports complex workflows through graph execution
- Middleware system — fine-grained control over agent behavior

OpenCode

- Terminal-native — lightweight and fast to start
- 75+ model providers — includes local models through Ollama
- LSP integration — optimized for code editing workflows

Claude Agent SDK

- Claude-optimized — tailored for Claude model capabilities
- Time travel — supports branching and state exploration
- Concise API — useful for fast prototyping

---

5. Recommendations by Use Case

Use Case	Recommended Framework	Why
Production agent app	Deep Agents	Pluggable backends, observability, sandbox support

Multi-model agent	Deep Agents	100+ provider support
Terminal coding assistant	OpenCode	Lightweight startup, strong local-model story
Claude-only app	Claude Agent SDK	Optimized for Claude, simple API
Rapid prototyping	Claude Agent SDK	Minimal API, fast setup
Complex multi-agent system	Deep Agents	Subagents and strong context management
Local model usage	OpenCode	Native Ollama support

```python
# Helper to recommend a framework by use case
def recommend_framework(use_case: str) -> str:
    """Recommend a framework for a given use case."""
    recommendations = {
        "production": ("Deep Agents", "pluggable backends, observability, sandbox support"),
        "multi-model": ("Deep Agents", "100+ provider support"),
        "terminal": ("OpenCode", "fast startup and strong support for local models"),
        "claude-only": ("Claude Agent SDK", "Claude optimization and a concise API"),
        "prototyping": ("Claude Agent SDK", "simple API and fast setup"),
        "multi-agent": ("Deep Agents", "subagents and context management"),
        "local-model": ("OpenCode", "native Ollama support"),
    }
    if use_case in recommendations:
        fw, reason = recommendations[use_case]
        return f"{fw} — {reason}"
    return "No recommendation found for that use case."

# Demo
for case in ["production", "terminal", "claude-only", "multi-agent"]:
    print(f"{case}: {recommend_framework(case)}")
```

6. Ecosystem Comparison

Item	Deep Agents	OpenCode	Claude Agent SDK
Community	LangChain ecosystem (large)	GitHub community	Anthropic community
Documentation	Official docs + LangSmith integration	GitHub README	Anthropic official docs
Integrations	LangChain, LangGraph, LangSmith	LSP, terminal tooling	Claude API

Package management	pip / uv	go install / brew	pip / npm
Editor integration	ACP (Zed, JetBrains, VS Code, Neovim)	Own editor workflows	None

7. Migration Considerations

Important questions when migrating between frameworks:

Shared Considerations

1. Model compatibility — verify that the target framework supports the models you use
2. Tool compatibility — check how your custom tool interfaces need to be adapted
3. State management — plan how checkpoints and memory should be migrated
4. Observability — decide how tracing and logging should be replaced or preserved

Advantages of Migrating to Deep Agents

- LangChain tool reuse — existing LangChain tools can often be reused directly
- LangGraph compatibility — integrates well with LangGraph-based workflows
- Incremental migration — supports gradual adoption rather than all-at-once rewrites

Caveats

- Claude Agent SDK features that are Claude-specific may need alternative implementations
- Terminal UI logic from OpenCode may need to be separated from the core agent logic

Summary

Topic	Core Idea	Key API / Tool
Three-way comparison	Deep Agents, OpenCode, Claude Agent SDK	model support, license, SDK
Core capabilities	Shared tools + differentiators	sandboxing, pluggable backends
Architecture	Pluggable vs terminal-native vs model-optimized	LangGraph, LSP, Claude API
Use-case guidance	Production, terminal, prototyping, etc.	`recommend_framework()`

Ecosystem	Community, docs, integrations, editor support	LangSmith, ACP
Migration	Model / tool / state compatibility	gradual adoption

Next Steps

→ 10_sandboxes_and_acp.ipynb

References:

- Comparison with OpenCode and Claude Agent SDK

10 Sandboxes and ACP

Learning Objectives

- Understand the concept of sandbox isolation and its security principles
- Compare sandbox providers such as E2B, Modal, and Docker-style environments
- Understand the overview and purpose of ACP (Agent Client Protocol)
- Understand editor-agent integration patterns
- Design an architecture that combines sandboxes and ACP

```
# Environment setup
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY is not set!"
print("Environment setup complete")
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"Model configured: {model.model_name}")
```

1. Sandbox Concepts

A sandbox is an isolated execution environment where an AI agent can run code, manage files, and execute shell commands.

Why Isolation Matters

Risk	Without isolation	With a sandbox
Filesystem access	The host filesystem can be changed or deleted	Only the isolated filesystem is exposed
Network access	Unlimited external communication	Restricted network access
Credentials	Environment variables can leak	Secrets remain isolated

System impact	Can affect the host OS	Host system stays protected

In Deep Agents, a sandbox functions as a backend and exposes the filesystem tools (`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`) together with the `execute` tool.

2. Architecture Patterns

There are two major patterns for sandbox integration.

Agent-in-Sandbox

The agent itself runs inside the sandbox and communicates with the outside world over a network protocol.

Advantages	Drawbacks
Similar to a normal development environment	Higher risk of credential exposure
Simple setup	More infrastructure complexity

Sandbox-as-Tool (Recommended)

The agent runs outside the sandbox and calls sandbox APIs to execute code.

Advantages	Drawbacks
Clean separation between agent state and execution environment	Network latency
Keeps secrets outside the sandbox	
Makes parallel task execution easier	

```
# Compare the two architecture patterns (reference only)
print("=== Pattern 1: Agent-in-Sandbox ===")
print("  [Sandbox]")
print("    |-- agent (running inside)")
print("    |-- filesystem")
print("    |-- code execution")
print("    <---> network protocol <---> external systems")

print()
print("=== Pattern 2: Sandbox-as-Tool (recommended) ===")
print("  [Host]")
print("    |-- agent (running outside)")
print("    |-- credential management")
print("    |-- API call --> [Sandbox]")
print("                       |-- filesystem")
print("                       |-- code execution")
```

3. Comparing Sandbox Providers

Package / Provider	Characteristics	Best Fit
`langchain-modal` / Modal	GPU support, ML workloads	AI / ML tasks, data processing
`langchain-daytona` / Daytona	TypeScript / Python support, fast cold starts	Web development, rapid iteration
`langchain-runloop` / Runloop	Disposable devboxes, isolated execution	Code testing, one-off tasks
`langsmith[sandbox]` / LangSmith	LangSmith Deployments integration (private beta)	Operations on top of LangSmith
`langchain-agentcore-codeinterpreter` / AgentCore	AWS Bedrock-backed code interpreter	AWS-native deployments

```python
# Modal sandbox — canonical pattern via langchain-modal
# pip install langchain-modal deepagents
import modal
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox

app = modal.App.lookup("your-app")
modal_sandbox = modal.Sandbox.create(app=app)
backend = ModalSandbox(sandbox=modal_sandbox)

agent = create_deep_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    system_prompt="You are a Python coding assistant with sandbox access.",
    backend=backend,
)

try:
    result = agent.invoke({
        "messages": [{"role": "user", "content": "Create a small Python package and run pytest"}],
    })
finally:
    modal_sandbox.terminate()   # required: free resources
```

4. Security Guidelines

Never Put Secrets Inside the Sandbox

If credentials are stored in environment variables or mounted files inside the sandbox, an agent can read and leak them.

Safe Practices

1. Manage credentials only through external tools
2. Use Human-in-the-Loop for sensitive operations
3. Block unnecessary network access
4. Monitor outbound activity
5. Review sandbox outputs before applying them back to the main application

5. File Transfer and Lifecycle Management

Ways to Access Files

Method	Description
Agent filesystem tools	Direct file operations through `execute()` and the backend
File transfer APIs	Manage seed files and artifacts through `uploadFiles()` / `downloadFiles()`

Lifecycle Management

To avoid unnecessary cost, sandboxes need explicit shutdown. There are two operational modes.

Mode	Lifetime	Best fit
Thread-scoped (default)	One sandbox per conversation thread. Created on the first run, reused on subsequent turns of the same thread, and cleaned up via an idle TTL.	Chat-style agents and conversational sessions
Assistant-scoped	All threads of the same assistant share one sandbox. Files, packages, and repositories accumulate across conversations. Always set a TTL or periodic snapshot policy.	Long-running coding sessions, accumulated workspaces

Operational checklist: for one-off scripts, use `try/finally` and terminate immediately. For chat / long sessions, use a per-thread sandbox with TTL-based cleanup. On LangSmith Deployments, wire termination into the session-end hook.

```
# Example lifecycle and file-transfer settings (reference only)
lifecycle_config = {
    "ttl_seconds": 1800,
    "auto_shutdown": True,
    "thread_isolation": True,
}

file_operations = [
    "uploadFiles(['/local/data.csv'], '/sandbox/data/')",
    "downloadFiles(['/sandbox/output/result.json'], '/local/results/')",
]

print("=== Lifecycle configuration ===")
for key, value in lifecycle_config.items():
    print(f"  {key}: {value}")

print("\n=== File transfer examples ===")
for op in file_operations:
    print(f"  {op}")
```

6. ACP Overview

ACP (Agent Client Protocol) standardizes communication between coding agents and development environments such as editors and IDEs.

MCP vs ACP

Protocol	Purpose	Target
MCP (Model Context Protocol)	External tool integration	agent ↔ external service
ACP (Agent Client Protocol)	Editor–agent integration	agent ↔ editor / IDE

ACP allows agents to interact with editors directly for code editing, file navigation, and terminal operations.

7. ACP Server Implementation

```
# ACP server — canonical pattern (asyncio + acp.run_agent)
# pip install deepagents-acp
import asyncio

from acp import run_agent
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP


async def main() -> None:
    agent = create_deep_agent(
        model="anthropic:claude-sonnet-4-6",
        system_prompt="You are a helpful coding assistant",
        checkpointer=MemorySaver(),
```

```
    )
    server = AgentServerACP(agent)
    await run_agent(server)


if __name__ == "__main__":
    asyncio.run(main())
```

---

8. Editors That Support ACP

Editor	Integration Style
Zed	Native integration
JetBrains IDEs	Built-in support
Visual Studio Code	`vscode-acp` plugin
Neovim	ACP-compatible plugin

Example Zed Configuration

```
// Zed settings.json
{
  "agent_servers": [
    {
      "command": "python",
      "args": ["acp_server.py"],
      "env": {
        "ANTHROPIC_API_KEY": "sk-..."
      }
    }
  ]
}
```

Toad CLI

Toad is a process manager for running ACP servers as local development tools. It handles start, stop, and restart automatically so you do not have to manage processes by hand.

```
# Install
uv tool install -U batrachian-toad

# Run — pass the ACP server command and the working directory
toad acp "python path/to/your_server.py" .
```

---

9. Combining Sandboxes and ACP

If you combine sandboxes with ACP, you get a complete architecture in which the editor controls the agent while code execution happens in an isolated environment.

Integrated Architecture

```
[Editor / IDE] <-- ACP --> [Agent] <-- API --> [Sandbox]
    |                        |                      |
  code editing          task management       code execution
  file browsing         context management    file isolation
  terminal UI           tool calls            secure runtime
```

Advantages

- interact with the agent directly from the editor
- run code safely in a sandbox
- keep secrets only on the host side

```python
# Sandbox + ACP integration — canonical pattern
import asyncio

import modal
from acp import run_agent
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP


async def main() -> None:
    app = modal.App.lookup("your-app")
    modal_sandbox = modal.Sandbox.create(app=app)
    try:
        agent = create_deep_agent(
            model=ChatAnthropic(model="claude-sonnet-4-6"),
            system_prompt="You are a coding assistant.",
            backend=ModalSandbox(sandbox=modal_sandbox),
            checkpointer=MemorySaver(),
            interrupt_on={"execute": True},   # HITL approval before code execution
        )
        server = AgentServerACP(agent)
        await run_agent(server)
    finally:
        modal_sandbox.terminate()


if __name__ == "__main__":
    asyncio.run(main())
```

Summary

Topic	Core Concept	Key API / Tool

Sandbox concept	Isolated execution that protects the host system	`execute`, filesystem tools
Architecture patterns	Agent-in-Sandbox vs Sandbox-as-Tool	Sandbox-as-Tool recommended
Providers	Modal (GPU), Daytona (fast startup), Runloop (disposable)	`ModalSandbox`
Security	External secret management, HITL, network controls	`interrupt_on`
ACP overview	Standardized editor-agent communication	`AgentServerACP`
ACP server	Expose the agent in stdio mode	`deepagents-acp`
Editor integration	Zed, JetBrains, VS Code, Neovim	ACP protocol
Integrated pattern	editor ↔ agent ↔ sandbox	ACP + sandbox

Next Steps

→ Continue to the advanced track at ../05_advanced/00_migration.ipynb

References:

- Sandboxes
- Agent Client Protocol (ACP)

11 Async Subagents

AsyncSubAgent · Non-blocking · Mid-flight steering

The flagship feature of Deep Agents 0.5.0, `AsyncSubAgentMiddleware`, lets the supervisor spawn subagents in the background without blocking. It is the infrastructure that pushes past the limits of the old synchronous subagent for long-running work — multi-minute to multi-hour research, coding, and parallel subagent orchestration. One caveat: this feature requires an Agent Protocol server — it only runs on top of LangSmith Deployments or a self-hosted setup such as `langgraph dev`.

Learning Objectives

LEARNING OBJECTIVES

- → Explain the execution-model differences between a synchronous subagent and `AsyncSubAgent`
- → Distinguish the five tools injected by the middleware: `start_async_task` / `check_async_task` / `update_async_task` / `cancel_async_task` / `list_async_tasks`
- → Compare ASGI (co-deploy) and HTTP (remote) transport modes and build hybrid configurations
- → Understand how the `async_tasks` state channel preserves state across context compaction
- → Know mid-flight steering patterns (update/cancel) and how to tune `--n-jobs-per-worker` slots

11.1 Limitations of the old synchronous subagent

The original subagent was synchronous. When the `task` tool was invoked, the supervisor stalled until the subagent finished, and the user could not issue a new instruction in the meantime. Running a multi-minute research task synchronously froze the frontend for a long stretch with no way for the user to check whether the agent was still alive.

The 0.5.0 `AsyncSubAgentMiddleware` removes this constraint.

- **Non-blocking execution**: `start_async_task` returns a task id immediately. The supervisor keeps talking to the user.
- **Mid-flight steering**: You can send follow-up instructions to a running subagent or cancel it.

- **Independent threads**: Each subagent task has its own thread and run. State is not lost even when the supervisor's context is compacted.

11.2 Five tools injected into the supervisor

`AsyncSubAgentMiddleware` injects five tools into the supervisor.

Tool	Role
`start_async_task`	Launch a subagent in the background. Returns the task id immediately
`check_async_task`	Query current status; extract final output if the task completed
`update_async_task`	Inject a new instruction into the same thread of a running task (interrupt strategy)
`cancel_async_task`	Send a cancel signal to the server and mark the task as `cancelled`
`list_async_tasks`	Batch-query the current status of all tracked tasks

11.3 Basic usage

Pass a list of `AsyncSubAgent` specs to `subagents` and `create_deep_agent` attaches `AsyncSubAgentMiddleware` automatically.

```
from deepagents import AsyncSubAgent, create_deep_agent

async_subagents = [
    AsyncSubAgent(
        name="researcher",
        description="Research work that needs information gathering and synthesis",
        graph_id="researcher",
    ),
    AsyncSubAgent(
        name="coder",
        description="Code generation / review work",
        graph_id="coder",
        url="https://coder-deployment.langsmith.dev",  # remote HTTP call
    ),
]

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    subagents=async_subagents,
)
```

Key fields on `AsyncSubAgent`

Field	Description
`name`	Unique identifier the supervisor references

`description`	The basis the supervisor uses to decide what to delegate. As important as it is for synchronous subagents
`graph_id`	Must match a graph name in `langgraph.json`
`url`	Optional. Without it, ASGI (in-process); with it, remote HTTP call
`headers`	Optional. Auth headers for a self-hosted server

11.4 The `async_tasks` state channel

Task metadata is stored in the `async_tasks` state channel, separate from the message history. Each record contains:

- task id
- agent name
- thread id, run id
- status (`pending` / `running` / `success` / `error` / `cancelled`)
- `created_at` , `updated_at`

This separation means that even when the supervisor's context is compacted (summarization) and older messages are replaced by summaries, task ids are not lost. `check_async_task` / `update_async_task` still work after compaction.

> **! WARNING**
>
> If a custom state reducer or middleware reshapes state and overwrites the `async_tasks` channel, you lose tracking for running tasks. Make the merge strategy explicit.

11.5 Transport modes

ASGI (co-deploy, the recommended default)

If you omit `url` , the subagent runs in the same process as the supervisor, like a function call. Zero network latency; register both graphs in the same `langgraph.json` .

```
// langgraph.json
{
  "dependencies": ["."],
  "graphs": {
    "supervisor": "./agent.py:supervisor",
    "researcher": "./subagents/researcher.py:graph",
    "coder": "./subagents/coder.py:graph"
  },
  "env": ".env"
}
```

```
AsyncSubAgent(
    name="researcher",
    description="...",
    graph_id="researcher",   # no url → ASGI
)
```

HTTP (remote)

Call an independently deployed subagent by `url`. Use this when you need to scale subagents with different resource profiles separately — for example, a low-cost model deployment dedicated to research and a GPU deployment dedicated to coding.

```
import os

AsyncSubAgent(
    name="coder",
    description="...",
    graph_id="coder",
    url="https://coder-deployment.langsmith.dev",
    headers={"x-api-key": os.environ["CODER_API_KEY"]},
)
```

Hybrid

You can mix the two. Lightweight subagents via ASGI, resource-intensive subagents via remote HTTP — split the deployment topology along the workload.

11.6 Execution lifecycle

1. **Launch** — `start_async_task` creates a new thread, starts a run, and returns the task id immediately
2. **Check** — `check_async_task` queries status and extracts the final output when complete
3. **Update** — `update_async_task` preserves history in the existing thread and starts a new run triggered by an interrupt carrying the new instruction
4. **Cancel** — `cancel_async_task` calls `runs.cancel()` and marks the task as cancelled
5. **List** — live-status queries for non-terminal tasks in parallel; cached responses for terminal ones

11.7 Mid-flight steering patterns

When the user changes direction mid-conversation, the supervisor calls `update_async_task`.

```
User: Start competitor research.
Supervisor: [start_async_task(agent="researcher", ...)] → task_abc123

User: Actually, only Series A and above.
Supervisor: [update_async_task(task_id="task_abc123",
                               instruction="Narrow the scope to Series A and above")]

User: Stop, redirect to the SaaS segment instead.
Supervisor: [cancel_async_task(task_id="task_abc123")]
```

```
            [start_async_task(agent="researcher",
                              description="SaaS competitor research")]
```

Because `update_async_task` creates a new run on the same thread, the subagent carries its prior exploration forward and simply layers on the new instruction. Use cancel + start only when you truly want a fresh task.

11.8 Local development: `--n-jobs-per-worker`

The default worker pool for `langgraph dev` is small, so spawning concurrent subagents causes queuing. Raise the slot count.

```
langgraph dev --n-jobs-per-worker 10
```

A supervisor running three concurrent subagents needs at least **4 slots** (1 supervisor + 3 subagents). 10–20 gives comfortable headroom.

11.9 Observing lifecycle through the unified v2 stream

Pending/Running/Complete signals for async subagents do not need a separate v3 projection. Use the single `agent.stream(...)` path with `subgraphs=True` + `version="v2"` to unify main and subagent events. Changes to the `async_tasks` channel flow through the `updates` mode `data`.

```
async for chunk in agent.astream(
    {"messages": [{"role": "user", "content": "Run research on 3 competitors in parallel"}]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    t = chunk["type"]
    if t == "updates":
        # async_tasks channel changes, check_async_task / list_async_tasks results
        for node, data in chunk["data"].items():
            print(f"step={node}")
    elif t == "messages":
        # Supervisor tokens + tool_call_chunks
        ...
    elif t == "custom":
        # In-tool progress via get_stream_writer
        ...
```

Signal	Detection point
Pending	Supervisor emits a `start_async_task` tool_call
Running	`updates` chunk where `async_tasks[task_id].status` becomes `running`
Complete	`updates` chunk where `status` settles to `success` / `error` / `cancelled`

Map the three signals to UI card states and the lifecycle flows without separate polling. See Chapter 14 (Streaming) for the full stream dispatch.

11.10 Sync vs Async selection criteria

Axis	Sync SubAgent	AsyncSubAgent
Execution	Supervisor blocks, waits for completion	Returns task id immediately, supervisor continues
Result retrieval	Automatically returned to the supervisor	Poll with `check_async_task`
Mid-task instructions	Not supported	Supported via `update_async_task`
Cancellation	Not supported	Supported via `cancel_async_task`
State preservation	Stateless (one-shot)	State kept in its own thread, interactive
Infrastructure requirement	No special requirements	Requires Agent Protocol server
Suited for	Seconds to tens of seconds, result-only workloads	Minute-to-hour tasks with room for intervention

Decision flow

- Task finishes in seconds and the supervisor needs the result immediately → **Sync**
- Task takes multiple minutes, or you want parallel subagents → **Async**
- User might redirect or cancel mid-flight → **Async**
- You cannot run LangSmith Deployments or an Agent Protocol server → **Sync only**

11.11 Caveats

- **Agent Protocol dependency**: Declaring `AsyncSubAgent` in a runtime without Agent Protocol support fails at initialization
- **Preserve the `async_tasks` channel**: Custom state reducers or middleware that reshape state must not overwrite it
- **Polling cost**: Instruct the system prompt to avoid calling `check_async_task` too frequently (for example, "spawn 2–3 tasks at a time, then wait for user feedback")
- **Long-running task cleanup**: Periodically sweep long unfinished tasks with `list_async_tasks` + `cancel_async_task`. LangSmith Deployments retain checkpoint-based cost
- **HTTP mode timeouts**: For remote URLs, verify headers, network timeout, and retry policy separately

Key Takeaways

- `AsyncSubAgent` is the 0.5.0 flagship feature that spawns background subagents without blocking the supervisor
- Five tools (`start` / `check` / `update` / `cancel` / `list`) drive the lifecycle
- ASGI is the default for co-deploy; HTTP is used for resource separation; hybrid is possible
- The `async_tasks` state channel preserves task tracking across context compaction
- A good fit for multi-minute work, tasks that benefit from user intervention, and multiple parallel subagents

12 Going to Production

Ten areas anchored on Thread · User · Assistant

This chapter covers the ten areas to work through when turning a local prototype into a multi-user, multi-tenant, long-running production agent. Treat it as a checklist just before shipping a Deep Agent to a live service, or when you are cleaning up operational issues on one already deployed.

Learning Objectives

LEARNING OBJECTIVES

- → Understand the three abstractions — Thread, User, Assistant — as the starting point for production design
- → Know the infrastructure auto-provisioned by `langgraph.json` + LangSmith Deployments
- → Distinguish multi-tenant authz, end-user credentials, and memory scoping
- → Pick between thread-scoped and assistant-scoped sandbox patterns
- → Assemble durability, rate limits, three-tier error handling, PII, and the real-time frontend via middleware and SDKs

12.1 Three core abstractions

Production Deep Agents operate on three core abstractions.

- **Thread** — a single conversation. The unit for messages, files, and checkpoints
- **User** — an authenticated identity. The unit for resource ownership and access scope
- **Assistant** — a configured agent instance (prompt · tools · model combination)

Ten areas of concern sit on top of these three concepts. Each area can be adopted independently, picked up according to your risk profile.

12.2 LangSmith Deployments

Deploying via the `deepagents deploy` CLI or a LangSmith Deployment auto-provisions the following infrastructure.

- Assistants / Threads / Runs APIs
- Store + Checkpointer (persistence)
- Authentication, webhooks, cron, observability
- MCP / A2A exposure options

```
// minimal langgraph.json
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./agent.py:agent"
  },
  "env": ".env"
}
```

12.3 Multi-tenant access control

Custom auth / authz handlers

LangSmith Deployments establish user identity via custom authentication, and a separate authorization handler controls access to threads / assistants / store namespaces. Handlers can tag resources with ownership metadata, filter visibility per user, and return HTTP 403 for denied access.

Workspace RBAC

Team-level permissions (a LangSmith built-in feature).

Role	Permissions
Workspace Admin	Full permissions
Workspace Editor	Create / edit; cannot delete or manage members
Workspace Viewer	Read-only

12.4 End-user credential management

When the agent has to call external services on the user's behalf (GitHub, Slack, Gmail, etc.), manage credentials outside the agent code.

Agent Auth (OAuth 2.0)

A managed OAuth flow. On the first call the user is shown a consent URL as an interrupt; once the token is received the flow auto-resumes and refreshes.

```
from langchain_auth import Client
from langchain.tools import tool, ToolRuntime
```

```
auth_client = Client()

@tool
async def github_action(runtime: ToolRuntime):
    """Perform a GitHub action on the user's behalf."""
    auth_result = await auth_client.authenticate(
        provider="github",
        scopes=["repo", "read:org"],
        user_id=runtime.server_info.user.identity,
    )
    # Call the GitHub API with auth_result.token
```

Sandbox Auth Proxy

When user code (or agent-generated code) running in a sandbox calls external APIs, the proxy injects credentials. API keys are never exposed to the code inside the sandbox.

```
{
  "proxy_config": {
    "rules": [
      {
        "name": "openai-api",
        "match_hosts": ["api.openai.com"],
        "inject_headers": {
          "Authorization": "Bearer ${OPENAI_API_KEY}"
        }
      }
    ]
  }
}
```

`${SECRET_KEY}` is resolved from the workspace secrets.

12.5 Memory persistence scoping

The `StoreBackend` `namespace` function determines the memory scope. The default pattern routes only `/memories/` through `StoreBackend` while keeping everything else on the ephemeral `StateBackend` via `CompositeBackend`.

User-scoped (recommended default)

```
from deepagents import create_deep_agent
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    backend=CompositeBackend(
        default=StateBackend(),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (
                    rt.server_info.assistant_id,
                    rt.server_info.user.identity,
                ),
            ),
        },
    ),
    system_prompt=(
        "At the start of every conversation, read /memories/instructions.txt. "
```

```
        "Update it with durable insights."
    ),
)
```

Assistant-scoped / Organization-scoped

Memory can be shared by all users of the same assistant (assistant-scoped) or across the whole organization (org-scoped). Always keep org-shared memory read-only.

> ! WARNING
>
> **Prompt-injection warning**: Shared memory is a prompt-injection vector. Do not grant write permission to surfaces that a user can manipulate. For detailed policy, see Part IV ch15 (Permissions).

12.6 Execution isolation — Sandbox

Do not expose the host filesystem or network directly; route through a sandbox.

Thread-scoped sandbox (most common pattern)

```python
from daytona import CreateSandboxFromSnapshotParams, Daytona
from deepagents import create_deep_agent
from langchain_core.runnables import RunnableConfig
from langchain_daytona import DaytonaSandbox

client = Daytona()

async def agent(config: RunnableConfig):
    thread_id = config["configurable"]["thread_id"]
    try:
        sandbox = await client.find_one(labels={"thread_id": thread_id})
    except Exception:
        sandbox = await client.create(
            CreateSandboxFromSnapshotParams(
                labels={"thread_id": thread_id},
                auto_delete_interval=3600,  # TTL
            )
        )
    return create_deep_agent(
        model="google_genai:gemini-3.5-flash",
        backend=DaytonaSandbox(sandbox=sandbox),
    )
```

Assistant-scoped sandbox

Use when every thread should share one sandbox so tool-chain caches and installed binaries are preserved.

12.7 Durability · Async I/O

Checkpoint on every step

LangSmith Deployments attach a checkpointer automatically. State is saved on every step, which gives you:

- **Indefinite interrupt**: A HITL approval can sit for days and still resume exactly where it paused
- **Time travel**: Rewind to an arbitrary checkpoint and branch
- **Audit trail**: State audit just before sensitive operations like payments or admin actions

```
await agent.ainvoke(
    {"messages": [...]},
    config={"configurable": {"thread_id": "thread-abc"}},
)
```

Async I/O

LLM apps are I/O-bound. Using async tools and async middleware hooks (`abefore_agent` , `astream`) significantly increases throughput.

12.8 Rate limits and cost control

```
from deepagents import create_deep_agent
from langchain.agents.middleware import (
    ModelCallLimitMiddleware,
    ToolCallLimitMiddleware,
)

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    middleware=[
        ModelCallLimitMiddleware(run_limit=50),
        ToolCallLimitMiddleware(run_limit=200),
    ],
)
```

`run_limit` resets for every `invoke` ; `thread_limit` accumulates over the thread's lifetime. Cut off runaway loops before they become cost bombs.

12.9 Three-tier error handling

Category	Examples	Strategy	Middleware
Transient	Timeouts, rate limits, transient network failures	Auto-retry with backoff	`ModelRetryMiddleware` , `ToolRetryMiddleware`

Recoverable	Bad tool arguments, parse failures	Feed back to the model → retry	Tool-wrapper error messages
Human required	Missing permissions, ambiguous request	Pause the agent	HITL `interrupt_on`

```python
from langchain.agents.middleware import (
    ModelRetryMiddleware,
    ModelFallbackMiddleware,
    ToolRetryMiddleware,
)

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    middleware=[
        ModelRetryMiddleware(max_retries=3, backoff_factor=2.0, initial_delay=1.0),
        ModelFallbackMiddleware("gpt-5.4"),
        ToolRetryMiddleware(
            max_retries=2,
            tools=["search", "fetch_url"],
            retry_on=(TimeoutError, ConnectionError),
        ),
    ],
)
```

12.10 Data privacy and real-time frontend

PIIMiddleware

Process PII — email, card numbers, national IDs — at the input and output boundaries. Strategies include `redact` (remove), `mask` (mask), `hash` (hash), and `block` (block); custom detectors can also be registered. The key is to process PII at the input side before it enters anything you log. LangSmith traces also record masked content only.

`useStream` hook

The `useStream` hook from `@langchain/react` handles real-time streaming, reconnection, and history loading in one place.

```typescript
import { useStream } from "@langchain/react";

function App() {
  const stream = useStream<typeof agent>({
    apiUrl: "https://your-deployment.langsmith.dev",
    assistantId: "agent",
    reconnectOnMount: true,
    fetchStateHistory: true,
  });
}
```

Deep Agents that spawn many subagents also stream subgraph events so the UI can render subagent progress cards (`streamSubgraphs: true`).

12.11 Production checklist

- [ ] Graphs and env are declared in `langgraph.json`
- [ ] Per-user authz handlers are applied to threads and the store
- [ ] External-service tokens are managed via Agent Auth / Proxy and never hardcoded
- [ ] `/memories/` is routed to `StoreBackend` with an explicit scope (user/assistant/org)
- [ ] Shared memory is read-only or write access is tightly restricted
- [ ] Host filesystem and shell are not exposed directly; access goes through a sandbox
- [ ] Cost ceilings are set via `ModelCallLimitMiddleware` / `ToolCallLimitMiddleware`
- [ ] Retry, fallback, and HITL are all configured
- [ ] PII is masked at input/output boundaries
- [ ] The frontend uses `reconnectOnMount` + `fetchStateHistory`

Key Takeaways

- Roll out the ten operational areas independently on top of Thread · User · Assistant
- `langgraph.json` + LangSmith Deployments auto-provision checkpointer, Store, auth, and observability
- Memory scope defaults to user-scoped; shared scopes must be read-only
- Classify errors into three tiers (Transient / Recoverable / Human) and respond with middleware plus HITL
- Defend PII in two layers: at the model-input stage and at the trace-transmission stage

13 Context Engineering

Write big, read selective

This chapter pulls together the full design of how a Deep Agent decides what to show the model inside its limited context window, when to offload, and how to bring things back. Use it when you are tuning the drift in quality over long sessions, or when you want to organize subagents / skills / memory into a single pipeline.

Learning Objectives

LEARNING OBJECTIVES

- → Understand the four-layer architecture of context engineering (Layered / Dynamic / Compression / Retrieval)
- → Know the nine-step automatic synthesis order of the system prompt
- → Distinguish the automatic offloading (>20k tokens) and automatic summarization (85%) triggers
- → Propagate runtime context via `@dynamic_prompt` and `context_schema`
- → Prevent context pollution with subagent isolation and `/memories/` routing

13.1 The four-layer architecture

Deep Agents' context engineering stacks in four layers.

1. **Layered input context** — system prompt, `AGENTS.md`, `SKILL.md`, and tool descriptions are synthesized in a fixed order
2. **Dynamic / runtime context** — `@dynamic_prompt` and `ToolRuntime` inject request-time data
3. **Automatic compression** — offloading (>20k tokens → disk), summarization (85% → structured summary)
4. **Filesystem-centric retrieval** — selective re-injection via `read_file` / `grep` / subagent isolation

Each layer can be toggled independently, but the defaults alone handle roughly 90% of long-session cases out of the box.

13.2 Layered input context — nine-step system prompt synthesis

The system prompt is synthesized automatically in this order.

1. Custom system prompt (user-supplied instructions)
2. Base agent prompt (planning / filesystem / subagent base instructions)
3. To-do list prompt
4. Memory prompt (`AGENTS.md` , always loaded when configured)
5. Skills prompt (only skill locations and frontmatter)
6. Filesystem prompt (tool documentation)
7. Subagent prompt (delegation guidance)
8. Middleware prompts (custom additions)
9. Human-in-the-loop prompt

AGENTS.md: permanent context loaded every time

Put content that must apply to every conversation — project conventions, user preferences.

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    memory=["/project/AGENTS.md", "~/.deepagents/preferences.md"],
)
```

Because it is always loaded, keep it minimal. Move detailed workflows into `SKILL.md` .

13.3 `@dynamic_prompt` — request-time data injection

Use this when a static string is not enough and the instruction has to change based on request-time data (user role, organization id, current time, store contents). The middleware reads `request.runtime.context` and `request.runtime.store` to build the dynamic prompt. The dynamic prompt is decided at the middleware layer; tools receive runtime values through `ToolRuntime` with no extra wiring.

13.4 Progressive skill loading

Skills load in two phases.

- **Startup**: only the frontmatter (name · description) of `SKILL.md` is read → a few hundred tokens
- **On demand**: the full body is loaded only when a user request matches the skill

```
agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    skills=["/skills/research/", "/skills/web-search/"],
)
```

Even with dozens of skills registered, the initial token cost is just the frontmatter.

13.5 Runtime context propagation

Context declared via `context_schema` is automatically forwarded not only to the agent but to every subagent and tool.

```python
from dataclasses import dataclass
from deepagents import create_deep_agent
from langchain.tools import tool, ToolRuntime

@dataclass
class Context:
    user_id: str
    api_key: str

@tool
def fetch_user_data(query: str, runtime: ToolRuntime[Context]) -> str:
    """Look up data for the current user."""
    user_id = runtime.context.user_id
    return f"Data for user {user_id}: {query}"

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    tools=[fetch_user_data],
    context_schema=Context,
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "Get my recent activity"}]},
    context=Context(user_id="user-123", api_key="sk-..."),
)
```

Thanks to this structure, tools can read runtime values without loading API keys into the message history, and subagents automatically inherit the same values.

> ➡️ TIP
>
> **Tool doc quality is context.** The model picks tools purely from their description. Writing concrete docstrings + Args sections reduces wasted tokens. Enable automatic Args parsing with `@tool(parse_docstring=True)`.

13.6 Automatic offloading (>20k tokens)

As context grows, two offloading behaviors kick in automatically.

- **Input offloading**: Results of large-file `write` / `edit` operations are replaced with a file pointer reference instead of their actual content once context exceeds 85% capacity
- **Result offloading**: Tool responses over 20,000 tokens are written to storage; the context retains only a file path + first 10-line preview

Large results go to disk first and are pulled back selectively with `read_file` / `grep` when needed. This is the core rationale for the filesystem-centric design.

13.7 Automatic summarization (85% trigger)

When there is nothing left to offload but context is still large, structured summarization kicks in. An LLM compresses the current conversation into three elements: Session intent, Artifacts created, Next steps. This summary replaces the existing conversation history and the agent continues.

Item	Default
Trigger	85% of the model's `max_input_tokens`
Retained	10%
Fallback	Triggers at 170,000 tokens, retains the last 6 messages
Immediate trigger	On `ContextOverflowError`

Manual summarization tool

Beyond the automatic trigger, add middleware to let the agent invoke summarization explicitly.

```
from deepagents import create_deep_agent
from deepagents.backends import StateBackend
from deepagents.middleware.summarization import create_summarization_tool_middleware

backend = StateBackend
model = "google_genai:gemini-3.5-flash"

agent = create_deep_agent(
    model=model,
    middleware=[create_summarization_tool_middleware(model, backend)],
)
```

When streaming, you can filter summarization tokens out of the UI (`metadata.get("lc_source") == "summarization"`).

13.8 Subagent isolation

Subagents run in their own context and return only one final report to the supervisor. Intermediate tool calls and search results stay in the subagent's context, keeping the supervisor window clean.

```
research_subagent = {
    "name": "researcher",
    "description": "Run research on a specific topic",
    "system_prompt": (
        "You are a research assistant.\n"
        "IMPORTANT: Return only the essential summary (under 500 words).\n"
        "Do NOT include raw search results or detailed tool outputs."
    ),
    "tools": [web_search],
}
```

Explicitly stating “return only the final summary” in the subagent’s prompt is the first line of defense against context pollution.

13.9 `/memories/` routing with CompositeBackend

Route only certain paths to a persistent store, leave the rest on ephemeral state. The agent uses the same `write_file` / `read_file` calls; different backends handle them under the hood.

```
from deepagents import create_deep_agent
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
from langgraph.store.memory import InMemoryStore

backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/memories/": StoreBackend(
            namespace=lambda rt: (rt.server_info.user.identity,),
        ),
    },
)

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    store=InMemoryStore(),
    backend=backend,
    system_prompt=(
        "When users tell you their preferences, save them to "
        "/memories/user_preferences.txt so you remember them in future conversations."
    ),
)
```

13.10 Filesystem-centric architecture

All of Deep Agents’ context-compression strategies converge on a single principle.

> ➡️ TIP
>
> **“Write big, read selective.”** Write big artifacts to disk; when you need them again, inject only the relevant portions via `read_file` / `grep` .

Because of this structure:

- Even when tool results exceed 20k tokens, the context retains only a file reference + 10-line preview
- Subagents can explore as widely as they want without touching the supervisor window
- Memory is not always loaded; instead it is pulled in via `read_file("/memories/...")` when needed
- Only the frontmatter of a skill is visible; the body is loaded on demand

13.11 Three patterns

Pattern 1: long research sessions

- `AGENTS.md` is minimal — only research style and citation rules
- Register 3–10 skills, expose only frontmatter
- Topic-specific subagents explore in parallel → return only the final summary
- Large search results are offloaded to files automatically → re-query via `grep`

Pattern 2: personalization assistant

- `/memories/` routing accumulates per-user preferences
- `@dynamic_prompt` injects tone and guardrails by user role
- Declare `user_id` / `org_id` on `context_schema` so every subagent sees them
- Shared policies live under `/policies/` (read-only) namespace

Pattern 3: cost optimization

- Lower the automatic summarization trigger slightly below the 85% default
- Declare "under 500 words" in subagent system prompts
- Wrap tools with large responses so they return a summary + file pointer

13.12 Caveats

- **Do not overload AGENTS.md** — it is always loaded, so anything over 5 KB starts to hurt
- **Skill frontmatter is everything** — if matching fails, the body is never loaded
- **Summarization is destructive** — save important intermediate artifacts to files first
- **If a subagent returns a raw dump**, the supervisor window blows up faster than without it
- **Tool description quality = token efficiency** — vague docstrings cause costly retries

Key Takeaways

- Context engineering is a four-layer architecture: Layered / Dynamic / Compression / Retrieval
- Automatic offloading (20k results) and automatic summarization (85%) are the primary compression mechanisms
- `@dynamic_prompt` and `context_schema` propagate runtime context across the entire graph
- Subagent isolation and `/memories/` routing are the two pillars against context pollution
- The full design distills to "Write big, read selective" — big outputs to disk, selective re-injection

14 Streaming

subgraphs=True + v2 namespaces

This chapter covers how to surface events from both the main agent and inside subagents in real time and how to emit custom progress events. Use it when you want to show long-running agent progress to users or to build a subagent-card UI.

Learning Objectives

LEARNING OBJECTIVES

- → Observe the inside of subagents by combining `subgraphs=True` and `version="v2"`
- → Route main vs subagent events using the namespace (`ns`) tuple
- → Subscribe to `updates` · `messages` · `custom` modes simultaneously
- → Emit custom progress events with `get_stream_writer`
- → Map the three subagent lifecycle signals (Pending / Running / Complete) to UI cards

14.1 Three key ideas

Deep Agents streaming rests on three ideas.

- `subgraphs=True` + `version="v2"` — observe events inside subagents
- **Namespace (`ns`)** — a tuple tells you which subgraph the event came from
- **Stream-mode combination** — `updates` (steps), `messages` (tokens / tools), `custom` (custom events)

The v2 format returns every chunk in a uniform `{"type", "ns", "data"}` shape. Routing is simpler than v1's nested tuples.

14.2 Basic usage: enabling subagent streaming

```
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "Research quantum computing advances"}]},
    stream_mode="updates",
```

```
    subgraphs=True,
    version="v2",
):
    print(chunk)
```

Without `subgraphs=True`, subagent internals are only visible at the supervisor level as the result of a `task` tool call. From the UI's perspective, that is "something happened in a black box and a result came out."

14.3 Routing events via namespaces

Every v2 chunk carries a tuple in the `ns` field, and that tuple identifies where the event was emitted.

Tuple	Meaning
`()`	Main-agent event
`("tools:abc123",)`	Subagent spawned by a `task` tool call from the main agent
`("tools:abc123", "model_request:def456")`	The model-request node inside that subagent

Detecting whether an event comes from a subagent:

```
is_subagent = any(segment.startswith("tools:") for segment in chunk["ns"])
```

This branch is the basic split when the UI separates the main conversation panel from subagent cards.

14.4 Stream mode: `updates`

Emits "which step is running right now" at node granularity. The most useful mode for tracking subagent lifecycles.

```
for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "updates":
        for node_name, data in chunk["data"].items():
            print(f"Step: {node_name}")
```

14.5 Stream mode: `messages`

Receives LLM tokens in fragments. When a tool call happens, each chunk carries `tool_call_chunks` so the tool name and args stream in incrementally.

```
for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="messages",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "messages":
        token, metadata = chunk["data"]
        if token.tool_call_chunks:
            for tc in token.tool_call_chunks:
                print(f"Tool: {tc['name']}, Args: {tc.get('args')}")
        else:
            print(token.content, end="")
```

Token-level output and tool-call assembly are handled in the same loop.

14.6 Custom progress events

Pull the stream writer inside a tool and emit arbitrary structures. Useful for upload progress, item counts, or intermediate states.

```
from langchain.tools import tool
from langgraph.config import get_stream_writer

@tool
def analyze_data(topic: str) -> str:
    """Analyze data for the given topic."""
    writer = get_stream_writer()
    writer({"status": "starting", "progress": 0})
    # ... analysis work ...
    writer({"status": "complete", "progress": 100})
    return "done"
```

Subscribe with `stream_mode="custom"` on the receiving side.

```
for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="custom",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "custom":
        print(chunk["data"])
```

14.7 Subscribing to multiple modes at once

Pass a list and receive all three event types in one loop.

```python
for chunk in agent.stream(
    {"messages": [...]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    t = chunk["type"]
    if t == "updates":
        # step progress
        ...
    elif t == "messages":
        # tokens / tool calls
        ...
    elif t == "custom":
        # progress / custom events
        ...
```

Production UIs typically subscribe to all three modes at once and fan them out to the respective panels.

14.8 Tracking the subagent lifecycle

From the main agent's perspective, a subagent is identified by three events.

1. **Pending** — the moment the main agent emits a `tool_call` with `name="task"`
2. **Running** — the moment the first event arrives on the `("tools:<id>",)` namespace
3. **Complete** — the moment the ToolMessage for that `task` call returns to the main `tools` node

Mapping UI card states to these three signals shows "which subagent is exploring right now" at a glance.

14.9 v2 unified format

Every chunk has the following three fields.

```python
{
    "type": "updates" | "messages" | "custom",
    "ns": tuple,
    "data": Any,
}
```

The old v1 nested-tuple format had different data shapes per `type`, which made branching noisy. v2 simplifies frontend routing to a single `(type, ns prefix)`.

14.10 Frontend integration

React / Vue / Svelte / Angular can consume this stream via `useStream` in `@langchain/react` and similar helpers. Subagent card rendering, reconnection, and history loading are all built in.

```ts
import { useStream } from "@langchain/react";

const stream = useStream<typeof agent>({
  apiUrl: "https://your-deployment.langsmith.dev",
  assistantId: "agent",
  reconnectOnMount: true,
  fetchStateHistory: true,
});

stream.submit(
  { messages: [{ type: "human", content: text }] },
  {
    streamSubgraphs: true,
    config: { recursionLimit: 10000 },
  },
);
```

14.11 Caveats

- **Without** `subgraphs=True`**, you cannot see inside a subagent** — essential for UI progress cards
- **Specify** `version="v2"` **explicitly** — falling back to the legacy format changes namespace handling
- **Keep the custom event schema stable** — if every tool uses its own shape, UI routing breaks (for example, use a shared `{"status", "progress", "message"}` structure)
- **Decide whether to expose summarization tokens** (filter with `metadata.get("lc_source") == "summarization"`)
- **Retry events are streamed too** — with `ModelRetryMiddleware` attached, fail–retry flows are visible; aggregate them in the UI

Key Takeaways

- v2 format + `subgraphs=True` is the entry point for subagent observability
- Route main vs subagent events using the `ns` tuple prefix
- Subscribing to `updates` / `messages` / `custom` as a list is the production default
- Use `get_stream_writer` to emit arbitrary progress events from tools
- Map Pending / Running / Complete to subagent UI card states

15 Permissions

FilesystemPermission · first-match-wins

Apply declarative allow/deny/interrupt rules to Deep Agents' built-in filesystem tools (`ls` , `read_file` , `glob` , `grep` , `write_file` , `edit_file` , `delete`) to enforce path-based access control. Use this chapter when you are defending against prompt injection, building a read-only agent, or carving out a workspace-scoped sandbox.

Learning Objectives

LEARNING OBJECTIVES

- → Understand the three components of `FilesystemPermission` (`operations` , `paths` , `mode`)
- → Know the first-match-wins evaluation rule and the default (allow)
- → Distinguish the four patterns — read-only, workspace isolation, sensitive-file protection, read-only memory
- → Recognize how subagent permissions are inherited and how overrides are full replacements
- → Know the compensating strategies for custom tools, MCP, and sandbox shells that permissions cannot reach

15.1 Scope of application

`permissions` are path-based rules that only apply to the built-in filesystem tools. The following bypass them:

- Custom tools
- MCP tools
- `execute` shell commands in the sandbox

In other words, you cannot complete a security boundary with permissions alone. In configurations where the sandbox can run arbitrary shell commands, rules must be designed together with CompositeBackend routing constraints. Extra validation or auditing for custom tools belongs to backend policy hooks.

15.2 Evaluation rule: first-match-wins

Rules are evaluated in list order; the first rule whose `operations` and `paths` match the current call decides the outcome. If no rule matches, the default is **allow**. Because of this, place specific deny/allow rules first and keep general fallbacks at the end.

15.3 Three rule components

Every `FilesystemPermission` has three fields.

Field	Value	Description
`operations`	`list["read" | "write"]`	`read` = `ls` / `read_file` / `glob` / `grep`, `write` = `write_file` / `edit_file` / `delete`
`paths`	`list[str]`	Glob patterns with `**` recursion and `{a,b}` selectors
`mode`	`"allow" | "deny" | "interrupt"`	Defaults to `"allow"`

`mode="interrupt"` pauses matching built-in write calls for human approval before execution. File deletes follow exact path matching, while directory deletes conservatively require write permission for the target and every descendant.

15.4 Pattern 1: read-only agent

Block every write globally. Useful for investigation, audit, and report-generation agents.

```
from deepagents import create_deep_agent, FilesystemPermission

agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)
```

15.5 Pattern 2: workspace isolation

Allow only under `/workspace/`, deny everything else. Because of first-match-wins, the allow goes first and the deny catch-all follows.

```
agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/**"],
            mode="allow",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)
```

15.6 Pattern 3: protect specific files

Forbid touching `/workspace/.env` and the examples directory, but allow the rest of `/workspace/` freely.

```
agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        # the most specific deny at the top
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/.env", "/workspace/examples/**"],
            mode="deny",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/**"],
            mode="allow",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)
```

15.7 Pattern 4: read-only memory / policies

Block only writes on `/memories/` and `/policies/`. Reads stay open. This is the baseline defense that prevents prompt injection from corrupting shared memory and organization policies.

```
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model=model,
    backend=CompositeBackend(
        default=StateBackend(),
        routes={
            "/memories/": StoreBackend(
```

```
                namespace=lambda rt: (rt.server_info.user.identity,),
            ),
            "/policies/": StoreBackend(
                namespace=lambda rt: (rt.context.org_id,),
            ),
        },
    ),
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/memories/**", "/policies/**"],
            mode="deny",
        ),
    ],
)
```

Update memories and policies only from application code; let the agent read.

15.8 Subagent inheritance

Default behavior: a parent agent's permissions are inherited by subagents as-is. If a subagent spec defines its own `permissions`, that fully replaces the parent rules (not a partial override). When overriding, include the catch-all deny yourself to stay safe.

```
parent_rules = [
    FilesystemPermission(operations=["read"], paths=["/workspace/**"], mode="allow"),
    FilesystemPermission(operations=["write"], paths=["/**"], mode="deny"),
]

agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=parent_rules,
    subagents=[
        {
            "name": "auditor",
            "description": "Read-only code reviewer",
            "system_prompt": "Review the code for issues.",
            "permissions": [
                # block writes globally
                FilesystemPermission(
                    operations=["write"], paths=["/**"], mode="deny",
                ),
                # allow reads only inside /workspace
                FilesystemPermission(
                    operations=["read"], paths=["/workspace/**"], mode="allow",
                ),
                # deny all other reads
                FilesystemPermission(
                    operations=["read"], paths=["/**"], mode="deny",
                ),
            ],
        },
    ],
)
```

This lets an audit-only subagent have a narrower read scope while the main agent keeps broader access.

15.9 CompositeBackend constraint: sandbox-default

When the `CompositeBackend` default is a sandbox, every permission path must sit inside a declared route prefix. This constraint exists because the sandbox can run arbitrary shell commands via the `execute` tool. Path rules cannot stop shell-level file access, so attaching permissions to paths outside the routes creates a false sense of security.

```
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend

composite = CompositeBackend(
    default=sandbox,
    routes={"/memories/": memories_backend},
)

# valid: inside the /memories/ route
agent = create_deep_agent(
    model=model,
    backend=composite,
    permissions=[
        FilesystemPermission(
            operations=["write"], paths=["/memories/**"], mode="deny",
        ),
    ],
)
```

Placing a rule on a path outside any known route raises `NotImplementedError`. For control over the sandbox-internal filesystem, do not reach for permissions — configure the sandbox itself (allowed binaries, network policy, volume-mount scope).

15.10 Prompt-injection defense perspective

Shared memory, organization policy, and externally ingested documents are all injection vectors. The defense layers are:

1. **Enforce read-only**: write-deny on `/memories/**` and `/policies/**` (pattern 4)
2. **Workspace isolation**: constrain what the agent can touch to `/workspace/**`
3. **Protect sensitive files**: deny each `.env`-like path individually as in pattern 3
4. **Shrink subagent scope**: configure audit / query subagents as read-only separately
5. **Custom-tool / sandbox boundaries**: permissions cannot reach them — cover with policy hooks + sandbox policy

15.11 Permissions vs backend policy hooks

Purpose	Use this surface
Path-based allow/deny on built-in FS tools	`permissions`
Custom validation (data checks, logging, rate limits)	backend policy hooks

Custom-tool / MCP tool control	Tool wrappers / middleware
Sandbox-internal file / network control	Sandbox configuration (allowed binaries, network policy)

Permissions are declarative quick rules; policy hooks are logic-driven controls. Use each for what it is good at.

15.12 Caveats

- **first-match-wins**: rule order changes the outcome. Put more specific deny/allow at the top
- **no match = allow**: to prevent accidental allows, end with a catch-all deny
- **Missing operations**: a rule with only `"read"` leaves writes to subsequent rules' (default allow)
- **Subagent override is full replacement**: no partial edits. Copy parent rules you want to keep
- **Permissions are not a complete security boundary**: they only make sense alongside sandbox, network, and secret management

Key Takeaways

- `FilesystemPermission`'s three components (`operations` / `paths` / `mode`) and first-match-wins are the foundations
- Four canonical patterns (read-only / workspace isolation / sensitive-file protection / read-only memory) cover most cases
- Subagent permission overrides are full replacements — be careful to copy in the catch-all deny
- Under CompositeBackend + sandbox-default, permissions outside the route prefixes raise `NotImplementedError`
- Permissions are declarative quick rules; custom tools, MCP, and sandbox shells require policy hooks and sandbox policy

16 Models and Tools

Design the Deep Agents execution surface

A Deep Agent is shaped by the model it calls and the tools it is allowed to use. This chapter treats tools as contracts, model configuration as a runtime boundary, and agent construction as something that should be inspectable.

Learning goals

- Design tool schemas that are clear enough for an agent to use safely.
- Classify tools by operational risk before wiring them into an agent.
- Keep model and backend configuration centralized and reviewable.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from langchain.tools import tool
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
```

12.1 Tools are contracts, not just functions

A tool is part of the agent's public interface. Its name, description, arguments, and return shape should communicate intent without relying on hidden context.

```
@tool
def summarize_note(text: str, max_bullets: int = 3) -> str:
    """Summarize a note into a short bullet list."""
    sentences = [s.strip() for s in text.split(".") if s.strip()]
    return "\n".join(f"- {s}" for s in sentences[:max_bullets])

print(summarize_note.invoke({"text": "A. B. C.", "max_bullets": 2}))
```

12.2 Inspect the tool schema

Schema inspection turns a Python function into an auditable contract. If the schema is confusing to a human reviewer, it will probably be confusing to the agent as well.

```
schema = summarize_note.args_schema.model_json_schema()
```

```
print("tool name:", summarize_note.name)
print("description:", summarize_note.description)
print("properties:", sorted(schema["properties"]))
```

12.3 Classify tool risk

Not every tool deserves the same level of access. Classify tools as safe, approval-gated, sandboxed, or denied before exposing them to autonomous execution.

```
tool_policy = {
    "summarize_note": "allow",
    "write_file": "approve",
    "execute_shell": "deny-or-sandbox",
}

for name, policy in tool_policy.items():
    print(f"{name:16s} -> {policy}")
```

12.4 Centralize model configuration

Centralized configuration keeps experiments reproducible. It also makes it obvious which model, tools, and backend are being used for a given run.

```
MODEL_NAME = os.getenv("COURSE_MODEL", "gpt-5.4")
agent_config = {
    "model": MODEL_NAME,
    "tools": [summarize_note],
    "backend": FilesystemBackend(root_dir=".", virtual_mode=True),
}
agent_config["model"]
```

12.5 Separate agent construction from invocation

Build the agent in one place and invoke it elsewhere. This separation makes the execution surface easier to test, swap, and review.

```
agent = create_deep_agent(
    model=agent_config["model"],
    tools=agent_config["tools"],
    backend=agent_config["backend"],
    system_prompt="You are a concise course assistant.",
)

type(agent).__name__
```

12.6 Design checklist

Use the checklist as a final review before moving from a notebook prototype to an application. It focuses on contracts, permissions, configuration, and observability.

Summary

Item	Content
Covered	model configuration, tool schema design, and permission-aware Deep Agent setup
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `models.md`
- `tools.md`
- `backends.md`

17 Programmatic Subagents

Code-controlled fan-out/fan-in

Subagents do not always need to be selected by an LLM at runtime. In many systems, code should decide when to fan out work, how to collect results, and how to merge them.

Learning goals

- Identify when programmatic delegation is preferable to model-driven delegation.
- Implement a deterministic fan-out/fan-in pattern.
- Define promotion checks before using live subagents in production.

```
from dotenv import load_dotenv
import importlib.util, os

load_dotenv(override=True)
```

```
import deepagents

capabilities = {
    "deepagents_version": getattr(deepagents, "__version__", "unknown"),
    "quickjs_available": importlib.util.find_spec("langchain_quickjs") is not None,
    "SubAgent": hasattr(deepagents, "SubAgent"),
    "AsyncSubAgent": hasattr(deepagents, "AsyncSubAgent"),
}
capabilities
```

13.1 When programmatic subagents are useful

Programmatic subagents are useful when the task boundaries are known in advance. Code can then enforce parallelism, ownership, and merge rules deterministically.

```
tasks = [
    {"name": "coverage", "question": "What is missing between official and local docs?"},
    {"name": "tests", "question": "How should verification be handled?"},
    {"name": "risks", "question": "What are the external service risks?"},
]

[t["name"] for t in tasks]
```

13.2 Practice fan-out/fan-in with a deterministic fallback

This section simulates delegation without depending on external model calls. The structure is the same pattern you would use around real subagents.

```
def worker(task: dict) -> dict:
    return {
        "name": task["name"],
        "finding": f"{task['question']} → manage with a checklist",
    }

worker_results = [worker(task) for task in tasks]
worker_results
```

13.3 Fan-in synthesis

Fan-in is where independent results become a single answer. Keep the merge step explicit so contradictions, gaps, and ownership are easy to see.

```
summary = {
    "total_workers": len(worker_results),
    "findings": [item["finding"] for item in worker_results],
    "next_action": "Turn the official slug action matrix into an implementation checklist",
}

summary
```

13.4 Before promoting to live features

Before live rollout, define tests for routing, result shape, failure handling, and observability. Subagents multiply both capability and operational surface area.

Summary

Item	Content
Covered	dependency gates, deterministic fallback, fan-out/fan-in, and subagent orchestration
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `programmatic-subagents.md`
- `subagents.md`

- `async-subagents.md`

18 Event Streaming

Observe Deep Agents as structured events

Streaming is not only a UI feature. For agent systems, structured events are the audit trail that explains what happened during planning, tool use, generation, and completion.

Learning goals

- Model agent progress as typed events.
- Consume events incrementally and update UI state.
- Use streaming as an operational debugging surface.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

14.1 Event model

A useful event stream has stable event types and predictable payloads. This makes it possible to build logs, UIs, and monitors on top of the same execution trace.

```
events = [
    {"type": "todo", "payload": {"task": "outline", "status": "done"}},
    {"type": "tool", "payload": {"name": "read_file", "status": "done"}},
    {"type": "subagent", "payload": {"name": "researcher", "status": "running"}},
    {"type": "message", "payload": {"text": "Drafting the first version."}},
]

len(events)
```

14.2 Build an event consumer

The consumer turns raw events into readable progress. Keep this logic small and deterministic so it remains trustworthy during incidents.

```
def project_event(event: dict) -> str:
    kind = event["type"]
    payload = event["payload"]
    if kind == "tool":
        return f"tool:{payload['name']}:{payload['status']}"
    if kind == "subagent":
        return f"subagent:{payload['name']}:{payload['status']}"
    return f"{kind}:{payload}"
```

```
[project_event(event) for event in events]
```

14.3 Accumulate UI state

Most interfaces need the latest message, tool status, and completion state rather than a raw event dump. Accumulating UI state makes the stream usable for learners and operators.

```
state = {"todos": [], "tools": [], "subagents": [], "messages": []}
for event in events:
    bucket = event["type"] + "s"
    if bucket in state:
        state[bucket].append(event["payload"])

state
```

14.4 Operational checklist

Streaming should be designed with failure modes in mind. Check event ordering, error events, redaction, and replayability before relying on it in production.

Summary

Item	Content
Covered	event logs, UI projections, subagent/tool/todo events, and replayable state
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `event-streaming.md`
- `streaming.md`
- `event-streaming.md`

19 Permissions

Design Deep Agents authorization boundaries first

Permissions are part of the agent design, not a final hardening pass. This chapter shows how to reason about tool risk, targets, approval messages, and default-deny behavior.

Learning goals

- Build a simple permission matrix for agent tools.
- Evaluate allow/approve/deny decisions deterministically.
- Write approval messages that explain the risk clearly.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

15.1 Permission matrix

A permission matrix makes policy visible before code executes. It should connect each tool to the targets it may read, write, modify, or never touch.

```
permission_matrix = {
    "read_file": "allow",
    "write_file": "approve",
    "edit_file": "approve",
    "delete": "approve-high-risk",
    "execute": "sandbox-only",
}
permission_matrix
```

15.2 Policy Evaluator

A policy evaluator turns the matrix into a repeatable decision. Even a small deterministic evaluator is better than scattered permission checks.

```
def decide(tool_name: str) -> str:
    return permission_matrix.get(tool_name, "deny")

for tool in ["read_file", "write_file", "delete", "execute", "unknown"]:
    print(tool, "=>", decide(tool))
```

15.3 Approval messages

Approval prompts should be specific enough for a human to make a real decision. They should name the tool, target, action, and reason for escalation.

```python
def approval_message(tool_name: str, target: str) -> str:
    policy = decide(tool_name)
    if policy != "approve":
        return f"{tool_name}: {policy}"
    return f"Approval required: {tool_name} will modify {target}"

approval_message("write_file", "docs/example.md")
```

Summary

Item	Content
Covered	permission matrices, approval messages, sandbox boundaries, and deny-by-default policies
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `permissions.md`
- `tools.md`
- `sandboxes.md`

20 Quality Profiles and Rubrics

Configure runtime quality gates

Agents need more than a final answer; they need a definition of acceptable work. This chapter separates runtime profiles from rubrics and shows how to turn quality expectations into executable checks.

Learning goals

- Define rubrics as completion criteria.
- Use deterministic evaluators for simple quality gates.
- Distinguish runtime profiles from answer-quality rubrics.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from deepagents import HarnessProfile, register_harness_profile

profile = HarnessProfile(system_prompt_suffix="Respond with concise Korean bullets.")
register_harness_profile("course:demo", profile)

type(profile).__name__
```

16.1 Rubrics define completion criteria

A rubric translates "good enough" into observable criteria. It helps agents and reviewers agree on what must be present before work is considered complete.

```
rubric = [
    {"name": "has_summary", "check": lambda text: "summary" in text},
    {"name": "has_next_step", "check": lambda text: "next" in text},
]

rubric[0]["name"]
```

16.2 Deterministic rubric evaluator

`RubricMiddleware` requires criterion-level evaluation coverage and connects to grader hooks and a runtime self-revision loop. The evaluator in this cell is a deterministic teaching miniature, not the SDK middleware itself.

That boundary matters: the teaching evaluator shows the contract without an LLM call, while `RubricMiddleware` checks output quality during runtime.

```
def evaluate(text: str) -> dict:
    results = {item["name"]: item["check"](text) for item in rubric}
    return {"passed": all(results.values()), "criteria": results}

evaluate("summary: done. next: run tests")
```

16.3 Profile versus rubric

Item	Role
profile	Tunes model/provider harness defaults, tool descriptions, excluded tools, and extra middleware
rubric	Defines quality criteria the final result must satisfy
eval	Runs batch/offline regression checks
middleware	Runs runtime self-revision and grader hooks

A profile changes the runtime environment; a rubric changes the completion contract. Keeping them separate makes regressions easier to diagnose.

Summary

Item	Content
Covered	`HarnessProfile`, rubric criteria, deterministic evaluator, and the `RubricMiddleware` contract
Core idea	Profiles configure runtime behavior; rubrics define completion criteria.
Caution	Do not describe a deterministic teaching evaluator as if it were SDK `RubricMiddleware`.

Reference docs

- `profiles.md`
- `rubric.md`
- `going-to-production.md`

PART

V

Advanced Patterns

Advanced Agent Patterns

What You Will Learn in This Part

- 0. Migration from v0 to v1
- 1. Advanced Middleware
- 2. Multi-Agent: Subagents
- 3. Multi-Agent: Handoffs and Router
- 4. Context and Memory
- 5. Agentic RAG
- 6. SQL Agent
- 7. Data Analysis
- 8. Voice Agent
- 9. Production Deployment
- 10. Deep Agent from Scratch
- 11. Custom Workflow Agent

00 v0 → v1 migration guide

Covers the breaking changes and code mapping you need to know when transitioning from LangChain/LangGraph v0 to v1.

Learning Objectives

- Understand v1 package structure changes and import paths
- Perform `create_react_agent` → `create_agent` migration
- Apply middleware-based dynamic prompting, state management, and context injection
- Utilizes standard content blocks and structured output strategies

0.1 Change package structure

In v1, the `langchain` namespace has been significantly reduced to five core modules essential for building an agent:

v1 module	Role	Main API
`langchain.agents`	Agent creation and state management	`create_agent` , `AgentState`
`langchain.messages`	Message types and content blocks	`HumanMessage` , `AIMessage` , `content_blocks`
`langchain.tools`	tool Definition	`\@tool` decorator, `BaseTool`
`langchain.chat_models`	model initialization	`init_chat_model`
`langchain.embeddings`	Embedding Utility	Embedding model wrapper

Legacy code migration — `langchain-classic`

Chains, Retrievers, Hub, and Indexing API, which were previously used in the `langchain` package, have all been separated into a separate package called `langchain-classic` . If you need to keep your existing code, change the import path after installation to `pip install langchain-classic` :

Note that message and tool imports also moved: v0 used `langchain_core.messages` and `langchain_core.tools` , but v1 re-exports the same symbols under `langchain.messages` and `langchain.tools` . Both paths still work, but new v1 code should prefer the shorter form.

```
# v1 preferred imports
from langchain.messages import HumanMessage, AIMessage, ToolMessage
from langchain.tools import tool, ToolRuntime, BaseTool
```

The

v0 — Legacy approach

from langchain.chains import LLMChain from langchain.retrievers import MultiQueryRetriever from langchain import hub

v1 — langchain-classicmigration path

from langchain_classic.chains import LLMChain from langchain_classic.retrievers import MultiQueryRetriever from langchain_classic import hub

```
Because of this split, the v1 `langchain` package became a lightweight layer focused on agent building,
while legacy functionality is maintained separately.
```

0.2 Agent creation API changes

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain.agents import create_agent

model = ChatOpenAI(model="gpt-5.4")

@tool
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b

# v0: from langgraph.prebuilt import create_react_agent
# v1: from langchain.agents import create_agent
agent = create_agent(
    model=model,
    tools=[add],
    system_prompt="You are a math assistant.",  # v0: prompt=
)
print("✓ v1 agent creation completed")
```

Major changes

v0	v1	Remarks

`from langgraph.prebuilt import create_react_agent`	`from langchain.agents import create_agent`	import path + function name
`prompt=`	`system_prompt=`	Parameter name
`ToolNode` Support	Not supported	Functions/BaseTool/dict only
`pre_hooks` , `post_hooks`	`middleware=[]`	Integrated middleware
Pydantic/dataclass status	`TypedDict` only	state schema

0.3 State Schema — TypedDict only

In v1, custom state must inherit from `TypedDict` based on `AgentState` .

0.4 Runtime context injection (new)

In v1, immutable runtime data can be passed to the agent via the `context_schema` and `context` parameters. This is a pattern that safely passes data that varies from request to request, such as user ID, role, and session information, but does not change during execution, to the agent and tool.

How it works:

1. Define the context schema with `@dataclass` .
2. Register the schema with `create_agent(context_schema=...)` .
3. Pass the value to runtime with `agent.invoke(..., context=ContextInstance(...))` .
4. In tool, the context is accessed with the `ToolRuntime[ContextType]` parameter.

Context is read-only data that does not change between tool calling, unlike agent state (`AgentState`). The state is updated during the agent loop, but the context is fixed when calling `invoke` .

0.5 Dynamic Prompts — Middleware Approach (New)

Instead of the static prompt in v0, `@dynamic_prompt` creates a prompt dynamically based on the runtime context.

0.6 tool Error handling — `\@wrap_tool_call` (new)

Instead of v0's `handle_tool_errors` , v1 handles tool errors with middleware.

```python
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage

@wrap_tool_call
def handle_errors(request, handler):
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"Tool error: {e}",
            tool_call_id=request.tool_call["id"],
        )

agent = create_agent(
    model=model,
    tools=[add],
    middleware=[handle_errors],
)
print("✓ Application of error handling middleware")
```

0.7 Standard Content Block & structured output (New)

In v1, messages support provider-agnostic `content_blocks`. structured output was split into two branches: `ToolStrategy` (based on tool calling) and `ProviderStrategy` (native).

```python
from pydantic import BaseModel
from langchain.agents.structured_output import ToolStrategy, ProviderStrategy

class Weather(BaseModel):
    location: str
    temperature_c: float

# Tool-call based — works on any model
agent_tool = create_agent(model=model, tools=[add],
                          response_format=ToolStrategy(schema=Weather))

# Provider-native — uses OpenAI JSON mode / Anthropic tool_use etc.
agent_provider = create_agent(model=model, tools=[add],
                              response_format=ProviderStrategy(schema=Weather))
```

v1 models default to `output_version="v1"`, which enables standard content blocks and the `content_blocks` property automatically. Set `output_version="v0"` to restore legacy behaviour.

```python
from langchain_openai import ChatOpenAI

model_v1 = ChatOpenAI(model="gpt-5.4", output_version="v1")
model_v0 = ChatOpenAI(model="gpt-5.4", output_version="v0")
```

0.8 Streaming changes

Item	v0	v1
Agent node name	`"agent"`	`"model"`
`.text`	method `.text()`	Property `.text`

0.9 Guitar Breaking Change

change	Description
Python 3.10+	All LangChain packages require Python 3.10 or higher. Versions 3.9 and below are not supported.
return type	The return type of the chat model has been fixed from `BaseMessage` to `AIMessage`.
OpenAI Responses API	Message content is in standard block format by default. You can restore the previous behavior with `output_version="v0"`.
Anthropic max_tokens	Default value changed from 1024 to automatic setting per model.
AIMessage.example	The `example` parameter has been removed. Use `additional_kwargs`.
AIMessageChunk	Added `chunk_position` attribute (value `'last'` in last chunk).
`.text` properties	`.text()` method changed to `.text` property.
File Encoding	Files open with UTF-8 encoding by default.

Summary — Migration Checklist

- [ ] Python 3.10+ confirmed
- [ ] `create_react_agent` → `create_agent` changed
- [ ] `prompt=` → `system_prompt=` changed
- [ ] Convert state schema to `AgentState` based on `TypedDict`
- [ ] `pre_hooks` / `post_hooks` → `middleware=[]`
- [ ] `ToolNode` → Replaced with Function/BaseTool
- [ ] `.text()` → `.text` property
- [ ] Check streaming node name `"agent"` → `"model"`
- [ ] Move legacy imports to `langchain-classic`

Next Steps

→ 01_middleware.ipynb: v1's biggest new feature — Deepening the middleware system

01 Deepening the middleware system

v1's biggest new features

Learning Objectives

- Understand the role and execution flow of middleware hooks in the agent loop.
- Learn how to set up and use 7 types of built-in middleware
- You can write decorator/class-based custom middleware.
- Execution order can be accurately predicted when combining multiple middleware

1.1 Environment Setup

Middleware is a core feature of v1 that implements monitoring, transformation, reliability, and governance by inserting hooks into each step of agent execution. It is used by passing the middleware instance list to the `middleware` parameter of the `create_agent` function.

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

1.2 Middleware Architecture Overview

The agent loop is a repeating cycle of model call → tool selection → tool execution → termination decision. Middleware enables fine-grained control by inserting hooks into each step of this cycle.

Hook type

Hook Type	When to run	Representative uses
`before_model`	Just before the model call	Prompt Modification, Logging, Status Updates

`after_model`	Immediately after the model responds	Response validation, guardrails, result transformation
`before_agent`	When the agent starts running	initialization, preprocessing
`after_agent`	At the end of agent execution	Cleanup, post-processing
`wrap_model_call`	Wrapping model call	Retries, caching, fallback
`wrap_tool_call`	tool calling Wrap	tool Retries, audit logs, error handling

Two hook styles

- Node-style hook (`before_*` , `after_*`): Executes sequentially and is suitable for logging/verification/status updates.
- Wrap-style hook (`wrap_*`): Allows you to control whether the handler (`next_fn`) is called. It can be called 0 times (blocking), 1 time (passing), or multiple times (retry), making it suitable for retry, caching, and conversion logic.

Middleware provides clean separation of cross-cutting concerns such as monitoring, transformation, reliability, and governance without changing the agent's core logic.

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    SummarizationMiddleware,
    HumanInTheLoopMiddleware,
)

agent = create_agent(
    model="gpt-5.4", tools=[],
    middleware=[
        SummarizationMiddleware(model="gpt-5.4-mini", trigger=("messages", 50)),
        HumanInTheLoopMiddleware(interrupt_on={}),
    ],
)
```

1.3 SummarizationMiddleware

When a conversation becomes long enough to exceed the context window, it automatically compresses the previous conversation by Summary. It is essential for long-running conversations, multi-turn conversations, and applications that require preservation of the entire conversation context.

Main parameters

parameters	Description	Example
`model`	Summary Lightweight model to use for creation (reduces costs)	`"gpt-5.4-mini"`

`trigger`	Summary Trigger condition	`("tokens", 4000)` , `("messages", 50)` , `("fraction", 0.8)`
`keep`	Latest context to keep after Summary	`("messages", 20)`
`token_counter`	Custom token counting function	optional
`summary_prompt`	Custom Summary Prompt Template	optional

`trigger` can be set based on one of the following: number of tokens, number of messages, or window ratio. When the condition is reached, all but the most recent messages specified in `keep` are replaced with Summary statements.

```python
from langchain.agents.middleware import SummarizationMiddleware

summarizer = SummarizationMiddleware(
    model="gpt-5.4-mini",
    trigger=("tokens", 4000),
    keep=("messages", 20),
)
```

1.4 HumanInTheLoopMiddleware

Stop agent execution before high-risk tool calling and wait for human approval. Use when human supervision is required for high-risk tasks or compliance workflows, such as database writes, financial transactions, and email sending.

`checkpointer` **required** — a checkpointer is absolutely required to restore state after an interruption.

Decision type

decision	Description	How to use
`approve`	tool calling Approval and Execution	`Command(resume="approve")`
`edit`	tool Execute after modifying arguments	`Command(resume={"type": "edit", "args": {...}})`
`reject`	tool calling Reject	`Command(resume={"type": "reject", "reason": "..."})`

Set the approval policy for each tool in the `interrupt_on` dictionary. If set to `False` , the corresponding tool will run without interruption.

```python
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "send_email": {"allowed_decisions": ["approve", "edit", "reject"]},
```

```
        "read_email": False,
    }
)
```

```
agent = create_agent(
    model="gpt-5.4", tools=[],
    checkpointer=InMemorySaver(),
    middleware=[hitl],
)
```

1.5 ModelCallLimitMiddleware & ToolCallLimitMiddleware

Call limiting middleware to prevent infinite loops or excessive API costs.

ModelCallLimitMiddleware

Limits the number of times the agent calls the model. Used to prevent agent flooding, control production costs, and manage call budgets during testing.

ToolCallLimitMiddleware

Limits tool calling counts globally or per specific tool. It is useful for limiting expensive external API calls, controlling search/DB query frequency, and enforcing rate limits for specific tool.

Common parameters

parameters	Description
`thread_limit`	Maximum number of calls across all threads (all invokes)
`run_limit`	Maximum number of calls in a single invoke execution
`exit_behavior`	`"end"` (normal termination), `"error"` (raise exception), `"continue"` (continue with error message — ToolCallLimit only)

ToolCallLimitMiddleware can additionally take a `tool_name` parameter to apply limits to specific tool only.

```
from langchain.agents.middleware import ModelCallLimitMiddleware

model_limit = ModelCallLimitMiddleware(
    thread_limit=10,
    run_limit=5,
    exit_behavior="end",
)
```

```
from langchain.agents.middleware import ToolCallLimitMiddleware

# global limit
global_tool_limit = ToolCallLimitMiddleware(thread_limit=20, run_limit=10)
```

```
# Certain tool restrictions
search_limit = ToolCallLimitMiddleware(
    tool_name="search",
    thread_limit=5, run_limit=3,
    exit_behavior="continue",
)
```

1.6 ModelFallbackMiddleware

Automatically switches to an alternate model chain when the primary model fails. It is useful for responding to production failures, optimizing costs (fallback from expensive models to cheap models), and ensuring multi-provider redundancy (OpenAI + Anthropic, etc.).

If you pass a fallback model to the constructor in order, it will try the fallback models in the specified order when the main model call fails. If all fallbacks fail, a final error is raised.

```
from langchain.agents.middleware import ModelFallbackMiddleware

# gpt-5.4 failed -> gpt-5.4-mini -> claude
fallback = ModelFallbackMiddleware(
    "gpt-5.4-mini",
    "claude-sonnet-4-6",
)
```

1.7 PIIMiddleware

Personally identifiable information (PII) is automatically detected and processed according to established strategies. It is essential for healthcare/financial compliance, cleaning logs for customer service agents, handling sensitive user data, and more.

Built-in PII type

`email` , `credit_card` , `ip` , `mac_address` , `url`

Processing Strategy

strategy	Action	Example (email)
`block`	Exception raised — execution halts when PII is found	Error Occurred
`redact`	Replace with `[REDACTED_TYPE]`	`[REDACTED_EMAIL]`
`mask`	Partial masking	`u**\@example.com`
`hash`	deterministic hashing	`a1b2c3d4...`

Scope of application

- `apply_to_input` : Check user input message
- `apply_to_output` : Check AI response message

- `apply_to_tool_results`: Check tool execution results

Custom Detector

In addition to the built-in PII types, you can create custom detectors in three ways:

1. Regular Expression String: Simple pattern matching
2. **Compiled regular expressions (`re.compile`):** complex regular expressions
3. Function: Advanced detection that requires validation logic (returns: `list[dict]` — contains `text`, `start`, `end` keys)

```
from langchain.agents.middleware import PIIMiddleware

email_pii = PIIMiddleware("email", strategy="redact", apply_to_input=True)
card_pii = PIIMiddleware("credit_card", strategy="mask", apply_to_input=True)
```

```
# Custom Detector: Regular Expression String
api_key_pii = PIIMiddleware(
    "api_key",
    detector=r"sk-[a-zA-Z0-9]{32}",
    strategy="block",
)
```

```
import re

# Custom Detector: Compiled Regular Expressions
phone_pii = PIIMiddleware(
    "phone_number",
    detector=re.compile(r"\+?\d{1,3}[\s.-]?\d{3,4}[\s.-]?\d{4}"),
    strategy="mask",
)
```

```
# Custom detector: function (SSN example)
def detect_ssn(content: str) -> list[dict]:
    matches = []
    for m in re.finditer(r"\d{3}-\d{2}-\d{4}", content):
        first = int(m.group(0)[:3])
        if first not in [0, 666] and not (900 <= first <= 999):
            matches.append({"text": m.group(0), "start": m.start(), "end": m.end()})
    return matches

ssn_pii = PIIMiddleware("ssn", detector=detect_ssn, strategy="hash")
```

1.8 LLMToolSelectorMiddleware

When there are more than 10 tools, Lightweight LLM analyzes the user query and selects only the relevant tools. This reduces token waste due to unnecessary tool descriptions and allows the model to focus on relevant tool, increasing accuracy.

Main parameters

parameters	Description	default

`model`	tool Model for selection	Agent's main model
`system_prompt`	Custom Selection Guidelines	Built-in prompt
`max_tools`	Maximum number of selections tool	All
`always_include`	List of tool names to always include	`[]`

Using a lightweight model like `gpt-5.4-mini` as the model of choice allows for effective tool filtering while reducing cost.

```
from langchain.agents.middleware import LLMToolSelectorMiddleware

tool_selector = LLMToolSelectorMiddleware(
    model="gpt-5.4-mini",
    max_tools=3,
    always_include=["search"],
)
```

ContextEditingMiddleware

For long multi-turn conversations, `ContextEditingMiddleware` clears old tool results once a token threshold is crossed, while keeping the most recent N. Where `SummarizationMiddleware` writes a single summary blob, this one prunes tool-result by tool-result.

```
from langchain.agents.middleware import ContextEditingMiddleware, ClearToolUsesEdit

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool],
    middleware=[
        ContextEditingMiddleware(
            edits=[ClearToolUsesEdit(trigger=100_000, keep=3, placeholder="[cleared]")],
        ),
    ],
)
```

PatchToolCallsMiddleware (Deep Agents)

Some models produce slightly malformed tool-call JSON. `PatchToolCallsMiddleware` (from `deepagents`) repairs common defects so tool calls do not fail outright.

```
from deepagents.middleware.patch_tool_calls import PatchToolCallsMiddleware

agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[search_tool, write_file],
    middleware=[PatchToolCallsMiddleware()],
)
```

1.9 Writing custom middleware

There are two implementation methods:

1. Decorator method

Single hook, suitable for simple logic. Use the `@before_model`, `@after_model`, `@wrap_model_call`, and `@wrap_tool_call` decorators.

2. Class method (`AgentMiddleware`)

If you need to combine multiple hooks or configure them, inherit `AgentMiddleware`. You can provide sync/async implementations simultaneously.

Custom Status

Middleware can extend agent state using the `NotRequired` type hint. This allows tracking values between executions, sharing data between hooks, and implementing cross-cutting concerns such as rate limiting or audit logging.

Agent Jump

You can control agent flow by returning a dictionary from `after_model`, etc.:

- `{"jump_to": "end"}` — Terminate agent immediately
- Go to `{"jump_to": "tools"}` — tool execution phase
- `{"jump_to": "model"}` — Go to model call step

Since v1.2, declare legal jump targets statically with `@hook_config(can_jump_to=[...])`. The graph is validated at compile time so an invalid jump target fails fast.

```python
from langchain.agents.middleware import after_model, hook_config, AgentState
from langgraph.runtime import Runtime
from langchain.messages import AIMessage

@after_model
@hook_config(can_jump_to=["end"])
def block_unsafe(state: AgentState, runtime: Runtime) -> dict | None:
    last = state["messages"][-1]
    if "BLOCKED" in last.content:
        return {"messages": [AIMessage("Cannot respond.")], "jump_to": "end"}
    return None
```

Type annotations — `ModelRequest` / `ToolCallRequest` / `ModelResponse`

v1 ships dataclasses for hook signatures:

- `ModelRequest` (`wrap_model_call` input): `messages`, `tools`, `model`, `system_message`, `response_format`, `state`, `runtime`.
- `ToolCallRequest` (`wrap_tool_call` input): `tool_call`, `state`, `runtime`.
- `ModelResponse`: return type of the inner handler.
- `ExtendedModelResponse`: wraps `ModelResponse` + `Command` for persistent state updates.

```python
from typing import Callable
from langchain.agents.middleware import (
    wrap_model_call, wrap_tool_call,
    ModelRequest, ModelResponse, ToolCallRequest,
)
from langchain.messages import ToolMessage
from langgraph.types import Command

@wrap_model_call
def add_cache_marker(
```

```
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    cached = request.override(system_message=f"{request.system_message}\n[CACHE]")
    return handler(cached)
```

For tool-level state updates, use `state_schema=` on `@wrap_tool_call` and return a `Command`. This is useful for audit trails that must survive after the tool response is returned.

```
from typing_extensions import NotRequired, TypedDict
from langchain.messages import ToolMessage
from langgraph.types import Command

class AuditState(TypedDict):
    tool_calls_log: NotRequired[list[str]]

@wrap_tool_call(state_schema=AuditState)
def track_tool_name(request: ToolCallRequest, handler):
    response: ToolMessage | Command = handler(request)
    prior = request.state.get("tool_calls_log", [])
    if isinstance(response, Command):
        return response
    return Command(
        update={
            "messages": [response],
            "tool_calls_log": prior + [request.tool_call["name"]],
        },
    )
```

`request.override(...)`
`ModelRequest.override(...)` returns a modified copy without mutating shared state, so multiple middleware can cooperate safely. `messages`, `tools`, `model`, `system_message`, `system_prompt`, `response_format` are all overridable.

`ExtendedModelResponse`
Return `ExtendedModelResponse` from `wrap_model_call` when you need a state update to survive the call:

```
from langchain.agents.middleware import ExtendedModelResponse

@wrap_model_call
def track_tokens(request, handler) -> ExtendedModelResponse:
    response = handler(request)
    tokens = response.message.usage_metadata.get("total_tokens", 0)
    return ExtendedModelResponse(
        model_response=response,
        command=Command(update={"total_tokens": tokens}),
    )
```

```
from langchain.agents.middleware import before_model

@before_model
def log_before(state, runtime):
    """Record the number of messages before calling the model."""
    print(f"[LOG] {len(state.get('messages', []))} messages")
```

```python
from langchain.agents.middleware import after_model

@after_model
def validate_output(state, runtime):
    """Guard: Blocks prohibited content."""
    last = state["messages"][-1].content
    if "FORBIDDEN" in last:
        return {"jump_to": "end"}
```

```python
from langchain.agents.middleware import wrap_model_call

@wrap_model_call
def retry_on_error(request, handler):
    """In case of failure, the model call is retried up to 2 times."""
    for attempt in range(3):
        try:
            return handler(request)
        except Exception as e:
            if attempt == 2: raise
```

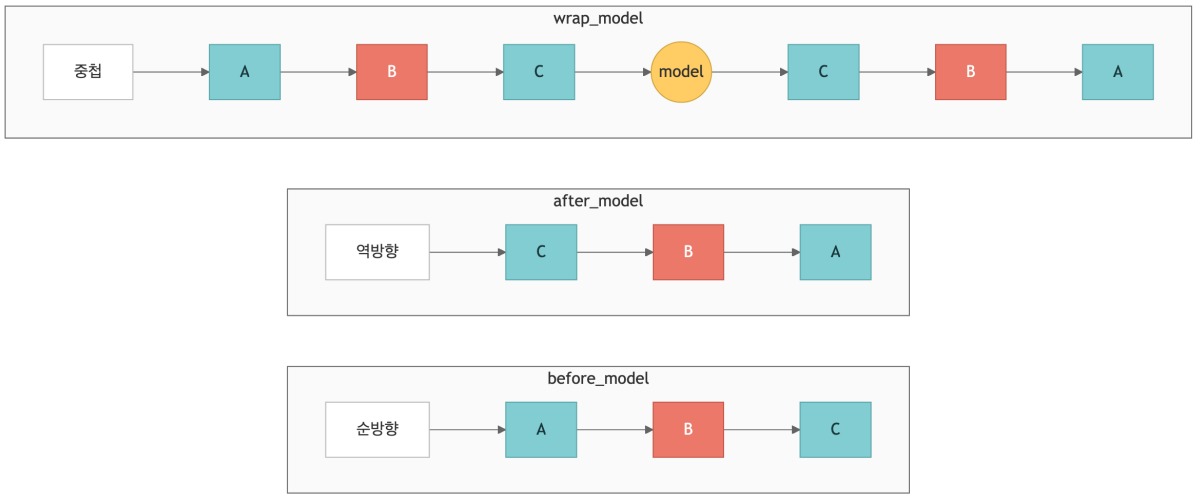
```python
from langchain.agents.middleware import AgentMiddleware

class AuditMiddleware(AgentMiddleware):
    def __init__(self, log_file="audit.log"):
        self.log_file = log_file
    def before_model(self, state, config):
        print(f"[AUDIT] before -> {self.log_file}")
    def after_model(self, state, config):
        print(f"[AUDIT] after -> {self.log_file}")
```

1.10 Middleware execution order

When registering multiple middleware, you can prevent unexpected behavior by accurately understanding the execution order.

`middleware=[A, B, C]` Upon registration:



Practical tips

- PII detection must be registered before logging so that PII is not included in the log.
- Place fallback middleware after retry middleware, so that fallback works after a retry failure.
- If `next_fn` is not called from the `wrap` hook, all subsequent middleware and actual calls will be skipped.

```
@before_model
def mw_a(state, runtime): print("before A")

@before_model
def mw_b(state, runtime): print("before B")

@before_model
def mw_c(state, runtime): print("before C")

# Run: A -> B -> C (if after_model, C -> B -> A)
```

1.11 Middleware Combination (Stacking)

In a production environment, multiple middlewares are used together to implement comprehensive agent governance. Since middleware is executed in registration order, it is recommended to place it in the following order: Security (PII) → Reliability (fallback) → Cost control (call limiting) → Context management (Summary) → Optimization (tool selection) → Supervision (HITL).

This combination allows each middleware to maintain single-responsibility principles, while collectively forming a powerful production agent pipeline.

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, ModelFallbackMiddleware,
    ModelCallLimitMiddleware, SummarizationMiddleware,
    HumanInTheLoopMiddleware, LLMToolSelectorMiddleware,
)
from langgraph.checkpoint.memory import InMemorySaver
```

```
middleware_stack = [
    PIIMiddleware("email", strategy="redact", apply_to_input=True),
    ModelFallbackMiddleware("gpt-5.4-mini", "claude-sonnet-4-6"),
    ModelCallLimitMiddleware(thread_limit=50, run_limit=10),
    SummarizationMiddleware(model="gpt-5.4-mini", trigger=("tokens", 4000)),
]

production_agent = create_agent(
    model="gpt-5.4", tools=[], checkpointer=InMemorySaver(), middleware=middleware_stack,
)
```

1.12 Provider-specific Middleware

While the seven built-in middleware above cover provider-agnostic patterns, provider-specific middleware wires vendor-only features into the agent. Capabilities that must be toggled on a

provider's server — Anthropic prompt cache, Bedrock TTL cache, OpenAI Moderation API — are exposed as dedicated middleware.

Anthropic Prompt Caching

`langchain_anthropic.middleware.AnthropicPromptCachingMiddleware` attaches `cache_control` markers automatically at the system prompt · tool definitions · last message positions when calling Claude models. Agents that repeatedly send long system prompts or RAG context can cut input-token cost by up to 90%.

Parameter	Default	Description
`type`	`"ephemeral"`	The only type Anthropic currently supports
`ttl`	`"5m"`	`"5m"` or `"1h"` (1h is an Anthropic beta)
`min_messages_to_cache`	`0`	Attach `cache_control` only when message count is at least this value
`unsupported_model_behavior`	`"warn"`	When applied to non-Anthropic models: `"warn"` / `"ignore"` / `"raise"`

```python
from langchain.agents import create_agent
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[],
    middleware=[
        AnthropicPromptCachingMiddleware(
            ttl="1h",
            min_messages_to_cache=2,
            unsupported_model_behavior="warn",
        ),
    ],
)
```

On successive calls, check `usage_metadata.input_token_details` for `cache_creation_input_tokens` (first call) and `cache_read_input_tokens` (subsequent calls) to confirm cache hits.

Claude native tool middleware

Claude models are trained on native tool schemas (`bash_20250124`, `text_editor_20250728`, `memory_20250818`) that the Anthropic server interprets directly. Compared to building the same behavior with a generic `@tool` decorator, tool-schema token usage approaches zero and error rates are lower. LangChain ships middleware that wraps these native tools in two variants — state-backed and filesystem-backed.

- `ClaudeBashToolMiddleware` — native bash execution tool. Takes `workspace_root`, `startup_commands`, `execution_policy` (`HostExecutionPolicy` / `DockerExecutionPolicy` / `CodexSandboxExecutionPolicy`), and `redaction_rules`. The Docker policy is the recommended default.
- `StateClaudeTextEditorMiddleware` / `FilesystemClaudeTextEditorMiddleware` — native text editor supporting `view` / `create` / `str_replace` / `insert` / `delete` / `rename`. The state variant writes

into the `text_editor_files` state key; the filesystem variant writes into an actual `root_path` directory.

- `StateClaudeMemoryMiddleware` / `FilesystemClaudeMemoryMiddleware` — native memory tool that follows the `/memories/*` path contract. The state variant, combined with a checkpointer, is restored on `thread_id` resume; the filesystem variant persists to disk across process restarts.
- `StateFileSearchMiddleware` — native tool that runs `glob` / `grep` across the virtual files accumulated by the text editor or memory middleware. `state_key="text_editor_files"` is the default; switch to `"memory_files"` to search memory-side files.

```python
from langchain_anthropic.middleware import (
    ClaudeBashToolMiddleware,
    StateClaudeTextEditorMiddleware,
    StateClaudeMemoryMiddleware,
    StateFileSearchMiddleware,
)
from langchain.agents.middleware import DockerExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(execution_policy=DockerExecutionPolicy()),
        StateClaudeTextEditorMiddleware(allowed_path_prefixes=["/src"]),
        StateClaudeMemoryMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
    ],
)
```

> **! WARNING**
>
> LangChain 1.2 `create_agent` rejects duplicate instances of the same middleware class. To search both text-editor files and memory files with `StateFileSearchMiddleware` at once, split them into separate subclasses.

Bedrock Prompt Caching

In enterprise setups that call Claude/Nova through AWS Bedrock, use `langchain_aws.middleware.BedrockPromptCachingMiddleware`. Parameter names and semantics are almost identical to `AnthropicPromptCachingMiddleware`, but the target package and model-specific constraints differ.

Item	ChatBedrockConverse + Anthropic	ChatBedrockConverse + Nova
System-prompt cache	O	O
Tool-definition cache	O	X
Message cache	O	O (excluding tool-result messages)
TTL `"1h"`	O	X (5m only)

The `type` parameter only takes effect on `ChatBedrock` (the legacy invoke-model wrapper); on `ChatBedrockConverse` it is pinned to `"default"`. For a checkpoint to actually hit, the cached block must be roughly 1,024 tokens or more. For Nova models, pin `ttl="5m"` and set `unsupported_model_behavior="warn"` so an accidental `"1h"` is not silently ignored.

```python
from langchain_aws.middleware import BedrockPromptCachingMiddleware

agent = create_agent(
    model="bedrock_converse:anthropic.claude-sonnet-4-v6:0",
    tools=[],
    middleware=[
        BedrockPromptCachingMiddleware(
            ttl="1h",
            min_messages_to_cache=2,
            unsupported_model_behavior="warn",
        ),
    ],
)
```

OpenAI Content Moderation (in depth)

Here is the full parameter set for `OpenAIModerationMiddleware`, which we introduced briefly earlier. `check_tool_results` adds a scan over tool outputs before they enter the model; it is cost-effective only in pipelines that mix in third-party authored content such as web search or email.

Parameter	Default	Description
`model`	`"omni-moderation-latest"`	Moderation model. Pinning to `"omni-moderation-2024-09-26"` etc. is supported
`check_input`	`True`	Scan user messages before they reach the model
`check_output`	`True`	Scan model-generated responses before returning
`check_tool_results`	`False`	Scan tool outputs before they enter the model
`exit_behavior`	`"end"`	`"end"` (end the graph) / `"error"` (raise) / `"replace"` (swap message only)
`violation_message`	default template	Supports the `{categories}` · `{category_scores}` · `{original_content}` variables
`client` / `async_client`	`None`	Inject a pre-configured OpenAI client (optional)

```python
from langchain_openai.middleware import OpenAIModerationMiddleware

agent = create_agent(
    model="openai:gpt-5.4",
```

```
    tools=[search_tool],
    middleware=[
        OpenAIModerationMiddleware(
            model="omni-moderation-latest",
            check_input=True,
            check_output=True,
            check_tool_results=False,
            exit_behavior="replace",
            violation_message=(
                "A safety-policy violation was detected "
                "(categories: {categories}). The original content is not recorded."
            ),
        ),
    ],
)
```

> ➡️ TIP
>
> In production it is common to always keep `check_input` on and run `check_output` under a sampling strategy (for example, 10% random sampling). The Moderation API itself adds call cost and latency.

Summary

Item	Key Takeaways
Architecture	Four hooks: `before_model` , `after_model` , `wrap_model_call` , `wrap_tool_call`
7 built-in types	Summarization, HITL, ModelCallLimit, ToolCallLimit, ModelFallback, PII, LLMToolSelector
Custom	Decorator (`\@before_model` , etc.) / `AgentMiddleware` class
Execution order	`before` : forward, `after` : reverse, `wrap` : nested
Production	PII → Fallback → Limit → Summarization → ToolSelector → HITL
Provider-specific	Anthropic (caching · bash · editor · memory · search), Bedrock (caching), OpenAI (Moderation)

Middleware is a powerful tool for controlling a single agent's behavior. But solving complex domain problems requires multi-agent architectures where several agents collaborate. The next chapter covers multi-agent systems that coordinate subagents with the supervisor pattern.

Next Steps

→ 02_multi_agent_subagents.ipynb: Multi-Agent: Learn the Subagents pattern.

02 Multiagents: Subagents

Supervisor pattern

Learning Objectives

- Design a three-tier architecture of Supervisor → subagent → tool
- Wrap subagent with `@tool` and expose it to supervisors as tool
- Understand HITL, ToolRuntime, and asynchronous/dispatch patterns.

2.1 Environment Setup

The Subagents pattern is a multi-agent architecture in which a central supervisor agent delegates tasks by calling specialized subagents like tool. On this laptop, we build a personal assistant system that handles calendars and email domains.

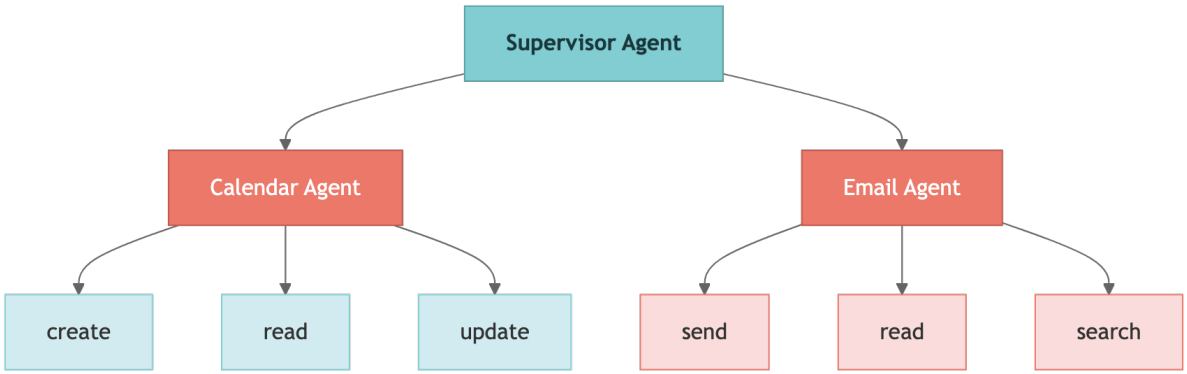
```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

2.2 Subagents Architecture Overview

The Subagents pattern consists of a three-tier architecture. The supervisor is responsible for all routing, subagent does not interact directly with the user, and returns results to the supervisor.



tier	Role	Features
Low Level tool	Direct call to external service (Calendar API, Email API)	Simple function wrapper
subagent	Domain-specific inference + tool combination	Expert system prompt, independent tool set
Supervisor	Decompose tasks, delegate, and aggregate results	Remember entire conversation, treat subagent = tool

Core Traits

- Centralized Control: All routing flows through the supervisor
- Context Isolation: subagent runs in a clean context window every time, preventing context bloat.
- Parallel execution: Multiple subagent can be called simultaneously in one turn
- tool based calls: Wrap subagent with `@tool` and expose it to the supervisor like a regular tool

When to use

The subagent pattern is suitable when you manage multiple domains (calendar, email, CRM, etc.), but subagent does not need to interact directly with users, and you need centralized workflow management. For simple scenarios with few tool, a single agent is sufficient.

2.3 Low-level tool definitions

Defines the low-level tool, the bottom layer of the three-tier architecture. These tool are simple function wrappers that interact directly with external services (Calendar API, Email API). In actual production, it integrates with Google Calendar API, Email Service, etc., but here we use a stub implementation for learning.

Important points when designing tool:

- One tool is responsible for only one function (single responsibility principle)
- The same tool should not be assigned redundantly to multiple subagent
- Write the docstring clearly so that LLM can select tool correctly

```python
from langchain_core.tools import tool

@tool
def create_calendar_event(
    title: str, start_time: str, end_time: str,
    attendees: list[str] = None,
) -> str:
    """Create a new calendar event."""
    return f"Event '{title}' created: {start_time} ~ {end_time}"
```

```
@tool
def read_calendar_events(date: str) -> str:
    """Look up calendar events for a date (YYYY-MM-DD)."""
    return f"No events found on {date}."
```

```
@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email message."""
    return f"Email sent to {to}: '{subject}'"

@tool
def read_emails(folder: str = "inbox", limit: int = 10) -> str:
    """Read recent emails from a folder."""
    return f"3 emails found in {folder}"
```

```
@tool
def search_emails(query: str, limit: int = 10) -> str:
    """Search for emails with your search term."""
    return f"2 search results for '{query}'"
```

2.4 Create subagent

Each subagent is created by `create_agent()` and has three core elements:

1. Specialized system prompts: Define domain-specific roles and behavioral guidelines
2. Set tool by domain: Separate concerns by assigning only tool for that domain
3. `name` **identifier**: used by the supervisor to identify and call a subagent

The recommended granularity of subagent is domain-level (calendar, email, etc.). Too much granularity increases the routing burden on the supervisor, while too much integration reduces the benefits of context isolation.

```
from langchain.agents import create_agent

calendar_agent = create_agent(
    model="gpt-5.4",
    tools=[create_calendar_event, read_calendar_events],
    system_prompt="You are a calendar assistant. Use ISO 8601 date format.",
    name="calendar_agent",
)
```

```
email_agent = create_agent(
    model="gpt-5.4",
    tools=[send_email, read_emails, search_emails],
    system_prompt="You are an email assistant. Write your message professionally.",
    name="email_agent",
)
```

2.5 Wrapping subagent with tool

The standard pattern for exposing subagent to a supervisor is to wrap it with a `@tool` decorator. Inside the wrapping function, call `subagent.invoke()` and return `content` of the last message.

Advantages of this pattern:

- From the supervisor's perspective, subagent is treated the same as regular tool
- Changes to the internal implementation of subagent do not affect the supervisor.
- The input/output format can be freely customized in the wrapping function.

Choose an input/output strategy: You can pass just the query (simple) or the entire context (sophisticated). When returning results, you have the option of returning only the final result or returning the entire history.

2.6 Supervisor Agent Assembly

The supervisor is created by passing the wrapped subagent tool to `tools`. The supervisor's system prompts include task decomposition and delegation instructions.

Supervisor design considerations:

- Error handling: subagent failures must be handled gracefully by the supervisor.
- Result Aggregation: Consolidates results from multiple subagent to provide consistent responses to users
- Approval Scope: Applies HITL only to operations that change the state (sending an email, creating an event)

```
supervisor = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt=(
        "You are a personal assistant. complex request"
        "Decompose it into subtasks and delegate them to appropriate agents."
    ),
)
```

2.7 Run test

```
User: "Schedule a meeting with Sarah tomorrow at 2 PM and send the invitation email"
Supervisor → call_calendar → create_calendar_event
Supervisor → call_email → send_email
Supervisor: "The meeting and invitation email are complete"
```

2.8 HITL (Human-in-the-Loop) Integration

Combining `HumanInTheLoopMiddleware` and `checkpointer` allows you to request user approval before high-risk tool calling (sending emails, creating schedules, etc.). When the agent attempts to call a protected tool, execution is paused and reviewed by the user.

Authorization response type

Reply	Description	code
Approve	Run tool calling as is	`Command(resume="approve")`
Edit	Execute after modifying tool argument	`Command(resume={"type": "edit", "args": {...}})`
Reject	Cancel tool calling	`Command(resume={"type": "reject", "reason": "..."})`

It is recommended to apply HITL only to operations that change state (send_email, create_calendar_event, etc.). It is not applied to read-only operations to reduce unnecessary friction.

```
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

hitl = HumanInTheLoopMiddleware(interrupt_on={
    "schedule_event": {"allowed_decisions": ["approve", "edit", "reject"]},
    "manage_email": {"allowed_decisions": ["approve", "reject"]},
})
```

```
supervisor_hitl = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    checkpointer=InMemorySaver(),
    middleware=[hitl],
    system_prompt="You are a personal assistant.",
)
```

2.9 Passing context (ToolRuntime)

`ToolRuntime` is a mechanism for passing runtime context (user ID, name, time zone, etc.) to tool without including it in the message. Instead of putting repetitive text in every prompt, you set the shared context with `ToolRuntime` just once.

The tool function accesses the context with the `runtime_context` keyword argument. Through this:

- User identity information may be used by tool (e.g. to automatically set sender email)
- Environment Setup (time zone, etc.) can be applied consistently
- Reduce token cost by reducing prompt length

```
# ToolRuntime is the runtime context passing mechanism for LangChain v1 agents.
# This is a fallback in case it hasn't been released yet.
try:
    from langchain.runtime import ToolRuntime
except ImportError:
    # Simple fallback implementation if ToolRuntime is not yet released
    class ToolRuntime:
        """Fallback ToolRuntime for pre-release versions."""
        def __init__(self, context: dict):
            self.context = context
    print("ToolRuntime unreleased — use fallback stub")

runtime = ToolRuntime(context={
    "user_email": "me@example.com",
    "user_name": "Alice",
    "timezone": "Asia/Seoul",
})
```

```
supervisor_ctx = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt="You are a personal assistant.",
)
```

```
# Accessing runtime context in tool
@tool
def send_email_ctx(
    to: str, subject: str, body: str, *, runtime_context: dict
) -> str:
    """Send an email to the current user."""
    sender = runtime_context["user_email"]
    return f"Email sent from {sender} to {to}: '{subject}'"
```

2.10 Asynchronous execution pattern

subagent has two execution modes:

mode	Action	When to use
Synchronous	Supervisor waits for subagent completion and then proceeds	When results are needed for next operation (default)
Asynchronous	Return Job ID immediately, run in background, view results later	Independent work, long-time work

Asynchronous patterns follow the structure Job ID → Status → Result. If subagent returns the Job ID immediately, the supervisor can continue working on other tasks and retrieve the results later.

```
import uuid
job_store = {}

@tool("schedule_async", description="Event scheduling (asynchronous)")
def call_calendar_async(query: str) -> str:
    """Starts an asynchronous calendar task and returns the task ID."""
    job_id = str(uuid.uuid4())[:8]
```

```
    job_store[job_id] = {"status": "done", "result": "Event created"}
    return f"Job started: {job_id}"
```

```
@tool("check_job", description="Check status of asynchronous task")
def check_job(job_id: str) -> str:
    """Check the status of an asynchronous operation."""
    job = job_store.get(job_id, {"status": "not_found"})
    return f"Status: {job['status']}, result: {job.get('result')}"
```

2.11 Single Dispatch tool Pattern

There are two approaches to the tool pattern:

pattern	Description	Advantages
By agent tool	Create a separate wrapping tool for each subagent	Detailed control, easy description customization
Single Dispatch tool	Call all subagent with one parameterized tool	Excellent scalability, independent addition/removal of subagent

The single dispatch pattern specifies the target of the call with the `agent_name` parameter. Because subagent registered in the agent registry is called by looking up its name, subagent can be added or removed independently in distributed teams, providing excellent scalability.

Three ways to expose a single dispatch tool

Style	How	When to use
Literal enum	`agent_name: Literal["calendar", "email"]`	≤ 5 domains, stable routing
Dict registry	Add/remove via dict without code change	6–20 domains, frequent changes
Vector search routing	Embed descriptions, match against user query	20+ domains, semantic routing

`SubAgentMiddleware` — Deep Agents integration

Instead of wrapping each subagent with `@tool`, `deepagents.SubAgentMiddleware` registers the dispatch tool automatically and merges subagent results into the supervisor state.

```
from deepagents.middleware import SubAgentMiddleware

supervisor = create_agent(
    model="gpt-5.4",
    tools=[],  # middleware adds the dispatch tool
    middleware=[SubAgentMiddleware(subagents=[
```

```
        {"name": "calendar", "description": "schedule", "agent": calendar_agent},
        {"name": "email", "description": "send/read email", "agent": email_agent},
    ])],
    system_prompt="You are a personal assistant.",
)
```

`checkpointer=True` — inherit the parent checkpointer

Since v1.2, `create_agent(checkpointer=True)` automatically inherits the parent graph's checkpointer, so subagents resume under the same `thread_id` without passing an explicit instance. Useful when HITL middleware is applied at the subagent level.

```
calendar_agent = create_agent(
    model="gpt-5.4",
    tools=[create_calendar_event],
    checkpointer=True,  # inherit from parent graph
    middleware=[HumanInTheLoopMiddleware(interrupt_on={
        "create_calendar_event": {"allowed_decisions": ["approve", "reject"]},
    })],
    name="calendar_agent",
)
```

`ToolRuntime[None, CustomState]`

When you need custom state in tools but no context, parametrise `ToolRuntime` with `None` as the first type and the state schema as the second.

```
from typing import NotRequired
from langchain.tools import tool, ToolRuntime
from langchain.agents import AgentState

class JobState(AgentState):
    pending_jobs: NotRequired[list[str]]

@tool
def list_pending(runtime: ToolRuntime[None, JobState]) -> str:
    return ", ".join(runtime.state.get("pending_jobs", []))
```

Async 5-tool dispatch pattern

A production-ready async subagent pattern exposes five tools, one responsibility each: `start_job`, `check_job`, `cancel_job`, `list_jobs`, `wait_for`. Pair these with an external queue (Celery, Redis Streams, Cloud Tasks) so job state survives process restarts.

```
supervisor_dispatch = create_agent(
    model="gpt-5.4",
    tools=[dispatch],
    system_prompt=(
        "Use delegate tool to route work."
        "Agent: 'calendar', 'email'."
    ),
)
```

Summary

Item	core
Tier 3	tool → subagent(`create_agent`) → Supervisor
Wrapping	`\@tool` + `subagent.invoke()` → Return last content
Quarantine	subagent = clean context, only supervisor remembers full
HITL	`HumanInTheLoopMiddleware` + `InMemorySaver`
Context	`ToolRuntime(context={...})` → `runtime_context`
Asynchronous	Job ID → Status → Result pattern
Dispatch	Single `dispatch(agent_name, query)` tool

Next Steps

→ 03_multi_agent_handoffs_router.ipynb: Learn Handoffs and Router patterns.

03 multi

agent: Handoffs & Router — State machines and parallel routing

Learning Objectives

- Handoffs pattern: Implements state variable-based dynamic configuration (prompt + tool replacement)
- Trigger a state transition from tool to the `Command` object.
- Router pattern: structured output classification → `Send` API parallel execution → result synthesis

Part A — Handoffs: Customer Support State Machine

3.1 Environment Setup

This notebook covers two multi-agent patterns:

- Part A — Handoffs: State machine pattern where a single agent dynamically swaps prompts and tool based on state variables.
- Part B — Router: A pattern that classifies queries, routes them in parallel to professional agents, and synthesizes the results.

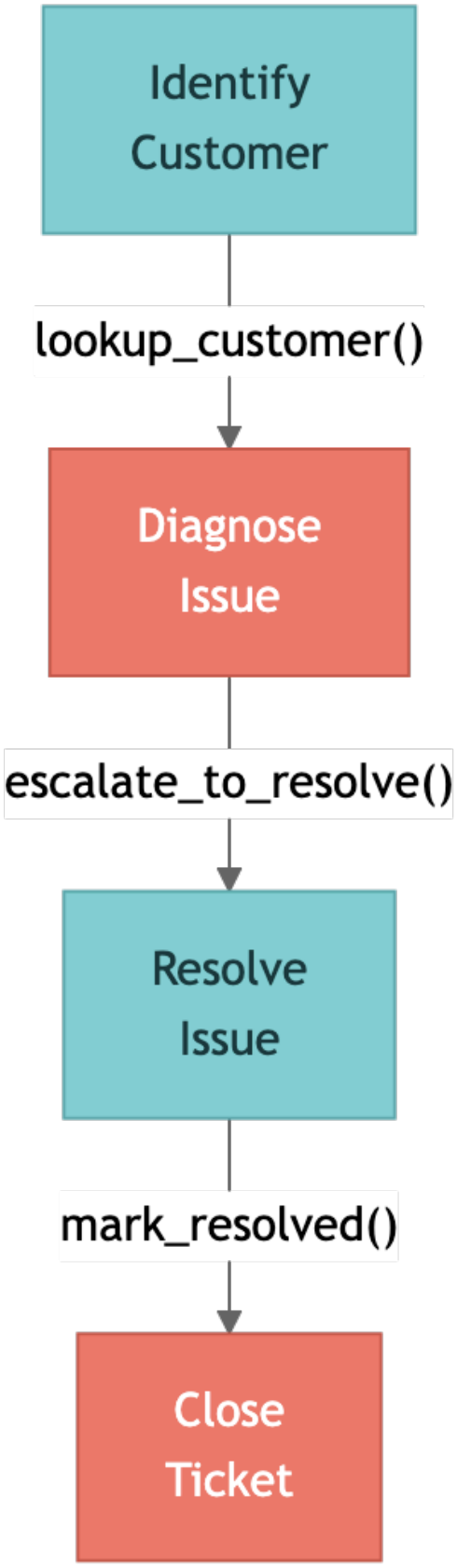
```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

3.2 Handoffs Overview

The Handoffs pattern is an architecture where a single agent dynamically changes its behavior based on state variables. Rather than switching between multiple agents, one agent uses different sets of system prompts and tool depending on the step.


Identify Customer
lookup_customer()
Diagnose Issue
escalate_to_resolve()
Resolve Issue
mark_resolved()
Close Ticket

Core Mechanism

mechanism	Description
`current_step`	State variable that tracks the current step. This value determines the agent's behavior
`Command(update={...})`	tool returns, triggering a state transition. `current_step` Change + Save Additional Data
`\@wrap_model_call`	Middleware reads `current_step` and dynamically replaces tool with system prompt

Key characteristics of Handoffs

- State-driven behavior: Settings are adjusted based on tracked state variables
- tool-based transitions: tool returns a `Command` object to update state
- Direct user interaction: Messages are processed independently at each stage
- Persistent state: The state persists beyond conversation turns

Important implementation details

When tool updates a message through `Command`, it must include `ToolMessage` with a matching `tool_call_id`. LLM expects responses to be paired with tool calling, so missing them will result in a malformed conversation history.

When to use

This is ideal when you need sequential constraints, interact directly with the user in each state, or collect information in a specific order in a multi-step flow (e.g. customer support).

3.3 SupportState definition

Inherit from `AgentState` and add the `current_step` field. This field determines the current node of the state machine and limits the valid steps to type `Literal`.

The default is `"identify_customer"`, which means all conversations start at the customer identification stage. Afterwards, when tool returns `Command(update={"current_step": "..."})`, it automatically transitions to Next Steps.

```
from langchain.agents import AgentState
from typing import Literal

class SupportState(AgentState):
    current_step: Literal[
        "identify_customer", "diagnose_issue",
        "resolve_issue", "close_ticket",
    ] = "identify_customer"
```

3.4 Step-by-step tool definition

Each step is assigned tool that matches the role of that step. tool is divided into two types:

- State transition tool: Returns `Command(update={...})` to change `current_step` and store additional data in the state. The string result to be shown to LLM is also delivered to the `result` field.
- General tool: Returns a string and does not change state. Used for information retrieval, etc.

Design recommendations:

- State transitions must only occur through tool, which returns `Command`
- Backward transitions (going back to the previous step) are also allowed when necessary.
- Prevent step skipping by validating invalid transitions in middleware

```
from langchain_core.tools import tool
from langgraph.types import Command

# --- Identify Customer ---
@tool
def lookup_customer(email: str) -> Command:
    """Track customers by email."""
    return Command(
        update={"customer": {"name": "Alice", "id": "C-1234"}, "current_step": "diagnose_issue"},
        result="Customer found: Alice (C-1234). Go to the diagnosis step.",
    )
```

```
# --- Diagnose Issue ---
@tool
def check_service_status(service_name: str) -> str:
    """Check the current status of the service."""
    return f"Service '{service_name}': healthy (99.9% uptime)"
```

```
@tool
def escalate_to_resolve(diagnosis: str) -> Command:
    """After diagnosis, move on to resolution steps."""
    return Command(
        update={"diagnosis": diagnosis, "current_step": "resolve_issue"},
        result=f"Diagnosis complete: {diagnosis}. Moving to the resolution step.",
    )
```

```
# --- Resolve Issue ---
@tool
def apply_fix(fix_type: str, customer_id: str) -> Command:
    """Apply modifications to customer accounts."""
    return Command(
        update={"resolution": {"type": fix_type}},
        result=f"Fix applied: {fix_type} for {customer_id}",
    )
```

```
@tool
def mark_resolved(summary: str) -> Command:
    """Mark the issue as resolved and move it to the Close stage."""
    return Command(
        update={"current_step": "close_ticket", "resolution_summary": summary},
```

```
        result="Solved. Go to the termination step.",
    )
```

```python
# --- Close Ticket ---
@tool
def send_satisfaction_survey(customer_id: str) -> str:
    """Send a satisfaction survey."""
    return "Survey completed."

@tool
def close_ticket(ticket_id: str, notes: str) -> str:
    """Close the support ticket."""
    return f"Ticket {ticket_id} closed."
```

3.5 @wrap_model_call middleware

`@wrap_model_call` Middleware is the core of the Handoffs pattern. Intercepts LLM calls and dynamically replaces system prompts and available tool depending on `current_step`.

Sequence of operations:

1. The middleware reads the `current_step` value from the state
2. Look up the settings (prompt + tool) for that step in the `STEP_CONFIG` dictionary.
3. Override `config` and pass it to LLM
4. Call LLM with modified settings with `next_fn(state, config)`

This is the core mechanic of Handoffs: a single agent will have completely different personas and abilities depending on their status. Achieve dynamic behavior changes with a single middleware, without the need to create multiple agents.

Single-agent handoff via `request.override(system_prompt=, tools=)`

Rather than running multiple agent nodes that hand off through `Command(goto=...)`, v1 lets you keep a single agent node and swap its persona/tools from inside a `@wrap_model_call` middleware. Message history stays intact, so the user sees one continuous conversation.

```python
@wrap_model_call
def step_middleware(request, handler):
    step = request.state.get("current_step", "identify_customer")
    cfg = STEP_CONFIG[step]
    request = request.override(system_prompt=cfg["system_prompt"], tools=cfg["tools"])
    return handler(request)
```

> **! WARNING**
>
> `request.override(...)` returns a copy. You must rebind `request = request.override(...)` before passing it to `handler(request)`.

Subgraph message rules — pair `AIMessage` + `ToolMessage`

When a tool updates `messages` directly through `Command`, the update must contain both:

1. An `AIMessage` declaring the tool call (`tool_calls=[{"id": ..., ...}]`),
2. A `ToolMessage` with a matching `tool_call_id` .

Missing the pair causes `Unmatched tool_call_id` on the next model call. `Command(update={...}, result=...)` generates the pair automatically; if you build `messages` by hand, include both.

```
from langchain.messages import AIMessage, ToolMessage
from langgraph.types import Command

return Command(
    update={"current_step": "diagnose_issue"},
    result="Found customer. Moving to diagnosis.",
)
```

```
STEP_CONFIG = {
    "identify_customer": {
        "tools": [lookup_customer],
        "system_prompt": "Identify your customers. Request your email or account ID.",
    },
    "diagnose_issue": {
        "tools": [check_service_status, escalate_to_resolve],
        "system_prompt": "Diagnose the issue. Call escalate_to_resolve after using tool.",
    },
}
```

```
STEP_CONFIG["resolve_issue"] = {
    "tools": [apply_fix, mark_resolved],
    "system_prompt": "Solve the issue. After applying the fix, call mark_resolved.",
}
STEP_CONFIG["close_ticket"] = {
    "tools": [send_satisfaction_survey, close_ticket],
    "system_prompt": "Thank your customers, send surveys, and close tickets.",
}
```

```
from langchain.agents.middleware import wrap_model_call

@wrap_model_call
def step_middleware(request, handler):
    """Dynamically configures agents based on current_step."""
    step = request.state.get("current_step", "identify_customer")
    cfg = STEP_CONFIG[step]
    request = request.override(
        system_prompt=cfg["system_prompt"],
        tools=cfg["tools"],
    )
    return handler(request)
```

3.6 Agent creation and execution flow

When creating an agent, register all tool, but specify `state_schema=SupportState` and middleware. Because the middleware filters tool according to `current_step` at runtime, each step only exposes tool for that step to the LLM.

Example execution flow: This is done automatically by tool, which returns [identify_customer] User: “I can’t log in. Email: alice@example.com” → lookup_customer(“alice@example.com”) ← Command(update={customer: {…}, current_step: “diagnose_issue”})

[diagnose_issue] Agent: “I found your account. What seems to be the issue?” → check_service_status(“auth-service”) → “healthy” → escalate_to_resolve(“Locked after three failed login attempts”)

[resolve_issue] → apply_fix(“reset_password”, “C-1234”) → mark_resolved(“Password reset completed”)

[close_ticket] → send_satisfaction_survey(“C-1234”) → close_ticket(“T-5678”, notes=“Password reset”)

```
Each step transition is driven by a `Command` object.
```

```python
from langchain.agents import create_agent

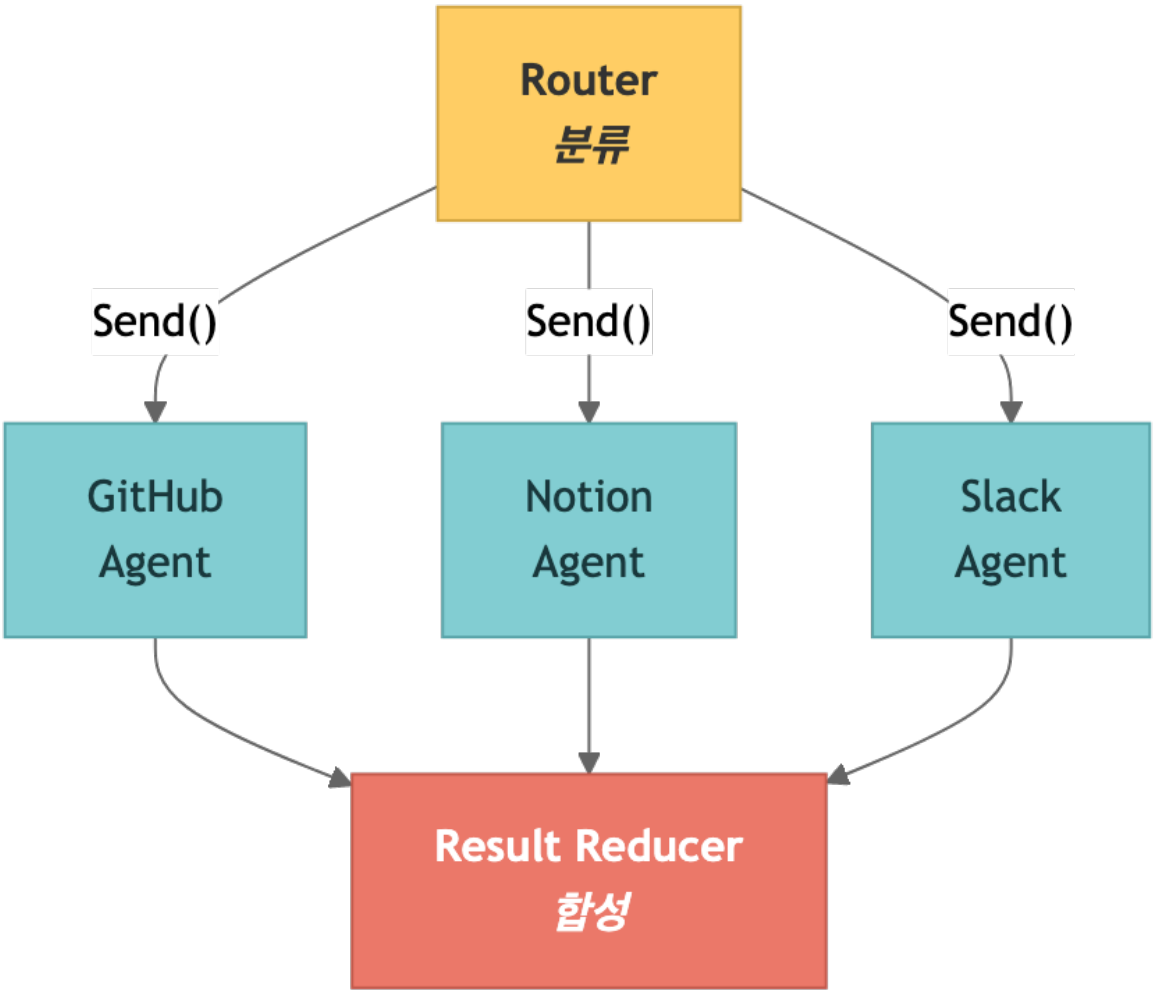
all_tools = [
    lookup_customer, check_service_status,
    escalate_to_resolve, apply_fix, mark_resolved,
    send_satisfaction_survey, close_ticket,
]
support_agent = create_agent(
    model="gpt-5.4", tools=all_tools,
    state_schema=SupportState, middleware=[step_middleware],
)
```

```
# [identify_customer]
#   User: "Can't log in. Email: alice@example.com"
#   Agent -> lookup_customer("alice@example.com")
#        <- Command(update={current_step: "diagnose_issue"})
#
# [diagnose_issue] (auto transition)
#   Agent -> check_service_status("auth-service")
#   Agent -> escalate_to_resolve("Locked out")
#
# [resolve_issue] -> [close_ticket]
```

Part B — Router: Parallel routing and result synthesis

3.7 Router Overview

The Router pattern is an architecture that classifies input and routes it to specialized agents. Unlike the Subagents pattern, Router distributes queries via a dedicated classification step (either a single LLM call or rule-based logic).



Pipeline

steps	Role	implementation
Classification	Analyze queries to create related sources and subqueries	`with_structured_output(QueryClassification)`
Parallel Dispatch	Deliver subqueries to each classified source simultaneously	`Send` API
Result synthesis (Reduction)	Gather all agent results to create a unified response	Reducer node + LLM

Router vs. Subagents Comparison

Routers have a “dedicated routing stage (classification),” while Subagents have “supervisor agents dynamically” deciding what to call. Router is suitable when distinct knowledge domains (verticals) are clearly distinguished and parallel queries are required.

Router vs. Supervisor vs. Handoffs

Pattern	Control flow owner	Execution	Best fit
Router	Dedicated classifier	Parallel fan-out / fan-in	Distinct knowledge verticals
Supervisor (Subagents)	Supervisor agent tool-calls	Sequential or parallel	Conversational + many domains
Handoffs	State variable (`current_step`)	Single agent, persona swap	Sequential multi-step flows

Architecture Mode

- Stateless: Each request is routed independently (no memory)
- Stateful: Supports multi-turn interactions by maintaining conversation history. An alternative is to wrap a stateless router with tool, or have the router itself manage the state directly.

3.8 RouterState and classification schema

`QueryClassification` is a Pydantic model, which classifies queries into a structured form through LLM's `with_structured_output()`. `RouterState` tracks classification results, source lists, subqueries, and agent results.

Key fields in the classification schema:

- `sources`: Which knowledge sources are relevant (multiple choices possible)
- `reasoning`: Explain why you selected the source
- `sub_queries`: Subquery optimized for each source (original query reorganized for each source)

```python
from pydantic import BaseModel, Field
from typing import Literal

class SubQuery(BaseModel):
    """Subqueries by source."""
    source: Literal["github", "notion", "slack"] = Field(description="Knowledge source.")
    query: str = Field(description="Search queries optimized for that source.")

class QueryClassification(BaseModel):
    """Classification results from user queries."""
    sources: list[Literal["github", "notion", "slack"]] = Field(
        description="Related knowledge sources."
    )
    reasoning: str = Field(description="Why you chose that source.")
    sub_queries: list[SubQuery] = Field(description="Subqueries by source.")
```

```python
from langchain.agents import AgentState

class RouterState(AgentState):
    classification: QueryClassification = None
    sources: list[str] = []
    sub_queries: list[SubQuery] = []
    agent_results: list[dict] = []
```

3.9 Classification Node

`with_structured_output` classifies queries by source and creates subqueries optimized for each source.

Classification example

user query	Category Source	Reason
“How to deploy the auth service”	`["github", "notion"]`	Distribution code exists on GitHub, procedure documentation exists on Notion
“How the decision was made to change the API”	`["slack", "notion"]`	Discussions are recorded in Slack, decision documents are recorded in Notion
“Login bug PR”	`["github"]`	PR only exists on GitHub
“Onboarding Process and Starter Repo”	`["github", "notion", "slack"]`	Repo is GitHub, process is Notion, context is Slack

Subquery creation is important: optimize the original query “auth service deployment” to `"auth service deployment scripts CI/CD pipeline"` for GitHub and `"auth service deployment process procedure runbook"` for Notion, respectively.

3.10 Parallel Routing (Send API)

The `Send` API dispatches subqueries to each classified source simultaneously. In the form of `Send(node_name, payload)`, data is passed in parallel to specific nodes in the graph.

This parallel execution is the core strength of the Router pattern: sequentially querying multiple knowledge sources adds up the latency, but parallel execution via `Send` takes only as long as the response time of the slowest source.

Add new source

The Router pattern is simple to extend:

1. Define source-specific tool

2. Create a professional agent
3. Add new source to `QueryClassification.sources`
4. Add agent nodes to the graph
5. Connect to Reducer

```
from langchain_core.tools import tool
from langchain.agents import create_agent

@tool
def search_github_code(query: str) -> str:
    """Search the GitHub repository."""
    return f"GitHub result for '{query}'"

@tool
def search_notion_pages(query: str) -> str:
    """Search for your Notion workspace."""
    return f"Notion result for '{query}'"
```

```
@tool
def search_slack_messages(query: str) -> str:
    """Search for Slack messages."""
    return f"Slack result for '{query}'"
```

```
github_agent = create_agent(
    model="gpt-5.4", tools=[search_github_code],
    system_prompt="Search for code and PRs on GitHub.",
    name="github_agent",
)
notion_agent = create_agent(
    model="gpt-5.4", tools=[search_notion_pages],
    system_prompt="Search documents in Notion.",
    name="notion_agent",
)
```

```
slack_agent = create_agent(
    model="gpt-5.4", tools=[search_slack_messages],
    system_prompt="Search for discussions in Slack.",
    name="slack_agent",
)
```

```
from langgraph.types import Send

def dispatch_to_agents(state):
    """Subqueries are passed to agents in parallel."""
    cls = state["classification"]
    sq_dict = {sq.source: sq.query for sq in cls.sub_queries}
    return [
        Send(src, {"messages": [{"role": "user", "content": sq_dict.get(src, "")}], "source": src})
        for src in cls.sources
    ]
```

When the classifier returns a list of dicts like `[{"agent": "github", "query": "..."}, ...]`, fan out with `Send(c["agent"], {...})`:

```
def dispatch_dict_form(state):
    return [
        Send(c["agent"], {"messages": [{"role": "user", "content": c["query"]}]})
```

```
        for c in state["classification"]
    ]
```

3.11 Result synthesis

Reducer collects the results from all agents and uses LLM to synthesize an integrated response. When synthesizing, the source of each information is cited so that the user can determine where the information came from.

The synthesis prompt instructs you to indicate the source. For example, respond with “The deployment script is in the `payment-service` repo on GitHub (GitHub), and for deployment procedures, please refer to Notion’s ‘Payment Service Ops’ document (Notion).”

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(RouterState)
graph.add_node("router", route_query)
graph.add_node("github", github_agent)
graph.add_node("notion", notion_agent)
graph.add_node("slack", slack_agent)
graph.add_node("reducer", reduce_results)
```

```
graph.add_edge(START, "router")
graph.add_conditional_edges("router", dispatch_to_agents)
graph.add_edge("github", "reducer")
graph.add_edge("notion", "reducer")
graph.add_edge("slack", "reducer")
graph.add_edge("reducer", END)

app = graph.compile()
```

Summary

Part A — Handoffs

Item	core
Pattern	Single agent + `current_step` based dynamic configuration
Transition	`Command(update={"current_step": "next"})`
Dynamic Configuration	prompt with `\@wrap_model_call` + replace tool

Part B — Router

Item	core
Category	`with_structured_output(QueryClassification)`

Parallel	`Send(source, payload)` API
Synthesis	LLM integrated response from reducer node

Next Steps

→ 04_context_memory.ipynb: Learn context engineering and memory.

04 Context Engineering & Memory Deepening

– Static/Dynamic Context, InMemoryStore, Skills Pattern

Deep learning of LangGraph's context system and long-term memory (Store). Covers everything from static/dynamic runtime contexts to long-term memory based on semantic search, and Progressive Disclosure (Skills) patterns.

Learning Objectives

- Understand the two-dimensional (Mutability x Lifetime) matrix of context engineering.
- Implement a static runtime context with `context_schema` + `@dataclass`
- Manage dynamic runtime context with `state_schema` and `AgentState` customizations
- Utilizes `InMemoryStore`'s namespace, put, get, and search APIs
- Build long-term memory based on semantic search
- Design by distinguishing between 3 types of memory (Semantic, Episodic, Procedural)
- Implement Progressive Disclosure using Skills pattern
- Compare hot path vs background memory writing strategies

4.1 Environment Setup

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

model = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("Environment ready.")
```

4.2 Context Engineering Overview

Context engineering is the design of systems that provide AI with "the right information, in the right format, at the right time." Beyond simple prompt engineering, it is an architectural approach to programmatically assembling contexts at runtime.

There are two main reasons why agents fail:

1. Lack of LLM skills

2. Lack of context or inappropriate context (more frequent cause)

Therefore, context engineering is a core role for AI engineers and a fundamental solution to agent reliability.

Two-dimensional matrix: Mutability x Lifetime

Static (immutable)	**User ID, DB connection, tool definition**	**config files, etc.**
Dynamic (variable)	Conversation history, intermediate results	User Preferences, Learned Memory

3 context types

Type	**Mutability**	**Lifetime**	**Example**	**LangGraph Implementation**
Static Runtime	Static	Single run	User ID, DB conn	`context_schema`
Dynamic Runtime (State)	Dynamic	Single run	Messages, interim results	`state_schema`
Dynamic Cross-conv (Store)	Dynamic	Cross-conversation	Affinity, Memory	`InMemoryStore`

3 controllable context categories

Category	**Control object**	**Characteristics**
Model Context	Instructions, message history, tool, response format	Transient
Tool Context	tool Access, read/write state, runtime context	Persistent
Life-cycle Context	Transformation between stages, Summary, guardrails	Persistent

LangChain implements context engineering with a middleware mechanism. Middleware such as `@dynamic_prompt` and `@wrap_model_call` can be used to update context or control between lifecycle stages.

4.3 Static runtime context – `context_schema` + `\@dataclass`

Injects unchanging information into `context_schema` while the agent is running. Define the schema as `@dataclass`, and access it as `ToolRuntime[Context]` from tool.

```python
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent

@dataclass
class Context:
    user_id: str
    role: str
    department: str
```

```python
@tool
def get_permissions(runtime: ToolRuntime[Context]) -> str:
    """View permissions based on the current user's role."""
    ctx = runtime.context
    perms = {"admin": "read,write,delete", "editor": "read,write"}
    return f"User {ctx.user_id} ({ctx.department}): {perms.get(ctx.role, 'read')}"

agent = create_agent(
    model=model,
    tools=[get_permissions],
    context_schema=Context,
)
agent.invoke(
    {"messages": [{"role": "user", "content": "What can I do?"}]},
    context=Context(user_id="u42", role="admin", department="eng"),
)
```

`runtime.execution_info` / `runtime.server_info`
Beyond context, `runtime` exposes execution metadata. `runtime.execution_info` carries `thread_id` / `run_id` /step for the current invoke, and `runtime.server_info` exposes the LangGraph server environment (deployment id, host). Useful for audit logs and distributed tracing.

```python
@tool
def audited_action(action: str, runtime: ToolRuntime[Context]) -> str:
    info = runtime.execution_info
    print(f"[audit] user={runtime.context.user_id} "
          f"thread={info.thread_id} run={info.run_id} action={action}")
    return f"Executed: {action}"
```

Key points

- You can use a type-safe context by passing `@dataclass` to `context_schema`
- Automatically injected as `runtime: ToolRuntime[Context]` type hint in tool function
- read-only and unchangeable while running
- Suitable data: User ID, DB connection, API key, session metadata

4.4 Dynamic runtime context – `state_schema`, `AgentState` custom

This state changes as the agent processes messages and calls tool. Add custom fields by inheriting `AgentState`.

```python
from langchain.agents import AgentState

class RAGState(AgentState):
```

```
    """State including dynamic search context."""
    retrieved_docs: list[str]
    query_count: int

print(f"State keys: {list(RAGState.__annotations__.keys())}")
```

Static vs Dynamic Comparison

Category	Static Runtime (`context_schema`)	Dynamic Runtime (`state_schema`)
Whether to change	immutable (read-only)	variable (node updates)
Delivery method	`context=` parameters	invoke input dict
Approach	`runtime.context.field`	`state["field"]`
suitable data	Authentication Information, Settings	Conversation history, intermediate results

4.5 Long-term memory – InMemoryStore native API

Use `InMemoryStore` for cross-conversation context. Long-term memory is per-user or app-level data that persists across sessions and threads.

Storage structure

The memory is stored as a JSON document, organized hierarchically into namespace:

- namespace: Folder role to classify memory (e.g. `(user_id, "preferences")`)
- key: Unique identifier for each memory (e.g. `"theme"`)
- Namespaces usually include user IDs or organization IDs to facilitate information management.

Basic API

API	Description
`store.put(namespace, key, value)`	Save memory (upsert)
`store.get(namespace, key)`	Memory query by specific key
`store.search(namespace)`	Search all within a namespace
`store.search(namespace, filter={...})`	Search by filter conditions

In production environments, you should use DB-based Store (e.g. PostgreSQL) instead of `InMemoryStore` .

PostgreSQL Store

`langgraph-checkpoint-postgres` ships `PostgresStore`, a drop-in replacement with the same `put` / `get` / `search` API but PostgreSQL-backed persistence. With `pgvector`, semantic search works the same way. Use this as the default production backend; `AsyncPostgresStore` is the async variant.

```
# pip install langgraph-checkpoint-postgres
from langgraph.store.postgres import PostgresStore

DB_URI = "postgresql://user:pass@localhost:5432/langgraph?sslmode=disable"

with PostgresStore.from_conn_string(DB_URI) as store:
    store.setup()

    store_with_index = PostgresStore.from_conn_string(
        DB_URI,
        index={"embed": embeddings, "dims": 1536},
    )

    agent = create_agent(
        model=model,
        tools=[get_user_info, save_preference],
        store=store_with_index,
        context_schema=Context,
    )
```

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
user_id = "user_42"
store.put((user_id, "preferences"), "theme", {"value": "dark"})
store.put((user_id, "preferences"), "language", {"value": "ko"})

item = store.get((user_id, "preferences"), "theme")
print(f"Theme: {item.value}")
```

```
items = store.search((user_id, "preferences"))
for item in items:
    print(f"  [{item.key}] = {item.value}")

filtered = store.search(
    (user_id, "preferences"), filter={"value": "dark"}
)
print(f"Filtered results: {len(filtered)}")
```

4.6 Long-Term Memory – Semantic Search

Setting the embedding function makes `InMemoryStore` support semantic search. Perform semantic-based similarity search with the `query` parameter.

```
semantic_store = InMemoryStore(
    index={"embed": embeddings, "dims": 1536}
)
ns = ("user_42", "memories")
semantic_store.put(ns, "mem1", {"content": "Prefer pytest over unittest"})
semantic_store.put(ns, "mem2", {"content": "Use type hints for all functions"})
semantic_store.put(ns, "mem3", {"content": "Favorite food is sushi"})
semantic_store.put(ns, "mem4", {"content": "Works on the ML Infrastructure team"})
print("Four memories have been saved along with embedding.")
```

```
results = semantic_store.search(
    ("user_42", "memories"), query="testing preferences", limit=2
)
for r in results:
    print(f"  [{r.key}] {r.value['content']}")
```

```
results2 = semantic_store.search(
    ("user_42", "memories"), query="machine learning work", limit=2
)
for r in results2:
    print(f"  [{r.key}] {r.value['content']}")
```

Basic Store vs Semantic Store comparison

Features	`InMemoryStore()`	`InMemoryStore(index={...})`
Accurate key lookup	`get(ns, key)`	`get(ns, key)`
Filter Search	`search(ns, filter={...})`	`search(ns, filter={...})`
Semantic Search	Not possible	`search(ns, query="...", limit=N)`
Production	Use DB backend instead of `InMemoryStore`	PostgreSQL-based Store recommended

4.7 Reading/Writing Store from tool – `ToolRuntime.store`

You can access the Store from within the agent's tool to read and write user information. If you connect a Store to `create_agent(store=...)`, it will be automatically injected into `runtime.store`.

Reading Pattern

Search for user information saved through `runtime.store` in tool. Both context and Store can be accessed with `ToolRuntime[Context]` type hints.

Writing Pattern

Receives user input with the tool parameter and saves the memory as `store.put()`. This allows agents to permanently store information learned during a conversation.

Key points

- `runtime.store` : Access Store instance
- `runtime.context` : Access static runtime context
- By combining Store and Context, you can systematically manage "whose (context) information (store)"

```
@tool
def get_user_info(runtime: ToolRuntime[Context]) -> str:
    """Search the saved information of the current user."""
```

```
    store = runtime.store
    user_id = runtime.context.user_id
    info = store.get((user_id, "memories"), "profile")
    # info.value may be a lazy / dict-subclass — cast to dict explicitly
    return str(dict(info.value)) if info else "User information not found."
```

```
@tool
def save_preference(key: str, value: str, runtime: ToolRuntime[Context]) -> str:
    """Stores user preferences."""
    store = runtime.store
    user_id = runtime.context.user_id
    # Recommended namespace: (user_id, "memories") — compatible with semantic-search store
    store.put((user_id, "memories"), key, {"value": value})
    return f"Preference saved: {key}={value}"
```

4.8 3 types of memory: Semantic, Episodic, Procedural

Long-term memory is categorized into three types inspired by cognitive science. Storage structure and utilization are different for each type.

Type	Description	Example	structure
Semantic	factual knowledge about entities	User preferences, profile information	Profile or Collection
Episodic	Memories of past experiences and events	Few-shot example, past action log	Collection
Procedural	Rules/Guidelines on how to do it	Fix system prompt, guidelines	Profile (list of rules)

Semantic Memory – Profile vs Collection

Semantic memory has two approaches depending on the storage strategy:

Approach	structure	Suitable for	Example
Profile	Single JSON document, continuously updated	Few well-known properties	`{"name": "Alice", "language": "Python", "preferred_style": "concise"}`
Collection	Multiple narrow documents, high recall	Open-end or large-scale knowledge	`[{"topic": "testing", "content": "Prefers pytest"}, ...]`

Episodic Memory

Record how you have behaved in similar situations in the past. It is used as a few-shot example, allowing the agent to learn from past experiences.

Procedural Memory

Stores the agent's behavioral rules. This has the effect of dynamically modifying system prompts, allowing the agent to follow user-specific instructions.

```
mem_store = InMemoryStore(index={"embed": embeddings, "dims": 1536})
uid = "user_42"

# Semantic -- Profile (single JSON)
mem_store.put((uid, "profile"), "main", {
    "name": "Alice", "language": "Python",
    "preferred_style": "concise",
})
# Semantic -- Collection (multiple docs)
mem_store.put((uid, "facts"), "f1", {"content": "pytest preferred"})
```

```
# Episodic -- past experiences (few-shot)
mem_store.put((uid, "episodes"), "ep1", {
    "content": "SQL Optimization -> Use EXPLAIN ANALYZE",
})

# Procedural -- rules/guidelines
mem_store.put((uid, "procedures"), "rules", {
    "content": "Always includes error handling. Use logging.",
})
print("All three memory types have been saved.")
```

```
# Episodic search: find similar past experiences
episodes = mem_store.search(
    (uid, "episodes"), query="database query help", limit=1
)
for ep in episodes:
    print(f"Related episode: {ep.value['content']}")
```

4.9 Progressive Disclosure – Skills pattern

Putting all the context in the prompt increases token cost and reduces accuracy. The Skills pattern is a Progressive Disclosure method that loads relevant information only when needed.

Structure of Skill

Skill is a unit of knowledge consisting of `{name, description, content}`:

- name: Skill identifier (e.g. `"customers_schema"`)
- description: Short description (included in system prompt)
- content: Details (load on demand with `load_skill` tool)

Strategies by Size

size	strategy	Example
\<1K tokens	Included directly in the system prompt	table names, high-level relationships

1–10K tokens	Load on demand with `load_skill` tool	Table schema, query patterns, best practices
\>10K tokens	Load on demand with pagination	Large reference data, historical query logs

Action flow

1. Middleware injects the names and descriptions of all skills into the system prompt
2. The agent analyzes the question and determines the skills needed
3. Call `load_skill` tool to load details
4. Perform actions based on loaded content

Advantages

Advantages	Description
Token Efficiency	Load only the information needed for the current query
Scalability	DB with hundreds of tables is also supported
Accuracy	Provide detailed schema at the point you need it
Cost Savings	Reduce input tokens per request

```
skills = [
    {"name": "db_overview",
     "description": "High-level Overview for all tables",
     "content": "Table: customers, orders, products"},
    {"name": "customers_schema",
     "description": "Full schema of the customers table",
     "content": "CREATE TABLE customers (id INT PK, name VARCHAR)"},
]
SKILL_MAP = {s["name"]: s for s in skills}
print(f"Defined {len(skills)} skills.")
```

```
from langchain_core.tools import tool

@tool
def load_skill(skill_name: str) -> str:
    """Loads detailed information about the database skill."""
    skill = SKILL_MAP.get(skill_name)
    if skill is None:
        return f"Not found. Available: {', '.join(SKILL_MAP.keys())}"
    return f"## {skill['name']}\n\n{skill['content']}"
```

4.10 Hot Path vs Background Memory Write

Depending on when you use memory, it will affect user response latency.

method	Timing	Available immediately?	Delay Impact
Hot path	Real-time within conversation loop	Instantly (available next turn)	Increased response delay
Background	Separate asynchronous task	Delayed (Eventual Consistency)	No delay impact

Write Hot Path

Save memory inline within the agent loop. This is perfect when you need to use that memory in the very next turn. Example: When you need to immediately reflect the preferences that the user just shared.

Background writing

Save memory as a separate process or asynchronous task. Used when Eventual Consistency is allowed and does not affect response delay. Examples: conversation pattern analysis, long-term learning data accumulation.

Selection criteria

- Is an immediate recall necessary? -> Hot path
- Is reducing delay a priority? -> Background
- In most cases, background writing is preferred.

```
from langgraph.store.base import BaseStore

# Hot path: write inline (adds latency)
def reflect_node(state, store: BaseStore):
    """Extract and store memory inline."""
    last_msg = state["messages"][-1].content
    store.put(("user", "reflections"), "latest", {"content": last_msg})
    return state

print("Hot path: Immediately save, available next turn.")
```

```
import asyncio

# Background: write in separate async task
async def background_memory_writer(state, store: BaseStore):
    """Saves memory in the background (no delays)."""
    last_msg = state["messages"][-1].content
    await store.aput(
        ("user", "reflections"), "latest", {"content": last_msg}
    )
print("Background: Eventual consistency, no delay.")
```

Summary

Context Engineering Triad

element	implementation	API
static runtime	`context_schema` + `\@dataclass`	`runtime.context.field`
dynamic runtime	`state_schema` + `AgentState`	`state["field"]`
long term memory	`InMemoryStore` + `store=`	`store.put/get/search`

Memory 3 types

Type	Use	Namespace example
Semantic	User Profile/Facts	`(user_id, "profile")` , `(user_id, "facts")`
Episodic	Past experience (few-shot)	`(user_id, "episodes")`
Procedural	Edit Rule/Prompt	`(user_id, "procedures")`

Best Practices

Principle	Description
minimize static context	Include only what is needed for the current task
Namespace structuring	Use hierarchical namespace to avoid collisions
Semantic search priority	Embedding-based search is more scalable than exact matching
Background writing preference	Reduce delay to background when immediate recall is not required
Skills pattern application	Large-scale context can be found in Progressive Disclosure

Next Steps

→ 05_agentic_rag.ipynb: Learn Agentic RAG.

05 Agentic RAG

– Built directly with LangGraph

We implement Retrieval–Augmented Generation (RAG) in three ways: LangChain RAG Agent, LangChain RAG Chain, and a custom RAG based on LangGraph StateGraph. Covers in–depth patterns such as document relevance evaluation, query rewriting, and conditional routing.

Learning Objectives

- Understand the overall structure of the RAG pipeline (Indexing -> Search -> Generation)
- Chunk the document with `RecursiveCharacterTextSplitter`
- Build a vector store with `InMemoryVectorStore`
- Implement RAG Agent with LangChain `create_agent` + `@tool`
- Implement RAG Chain (single LLM call) with `@dynamic_prompt` middleware
- Build a custom RAG agent with LangGraph `StateGraph`
- `GradeDocuments` Evaluate document relevance with structured output
- Implement query rewriting and conditional routing

5.1 Environment Setup

```python
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

llm = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("Environment ready.")
```

5.2 RAG Overview

Retrieval–Augmented Generation (RAG) is a pattern that improves the accuracy of LLM responses by retrieving external knowledge. LLM has two key limitations:

- Finite context: the entire corpus cannot be processed at once.
- Static Knowledge: Training data becomes outdated over time.

RAG overcomes this limitation by bringing in relevant external information at query time.

Pipeline: Indexing −\> Search −\> Generate

```
[Documents] -> Text Splitter -> [Chunks] -> Embeddings -> [Vector Store]
                                                              |
[Question] -> Embedding -> similarity_search -> [Relevant Chunks] -> LLM -> [Answer]
```

5 Core Components

Components	Role
Document Loaders	Collect data from external sources (Google Drive, Notion, etc.) into a standard Document object
Text Splitters	Split large documents into chunks that fit into the context window
Embedding Models	Convert text into vectors where semantically similar content is grouped close together
Vector Stores	A specialized database that stores embeddings and performs similarity searches
Retrievers	Return related documents based on unstructured queries

Three RAG architectures

Approach	Architecture	LLM Call	Flexibility	Suitable for
2−Step RAG	Created immediately after search	single	low	FAQ, Doc Bot (fast and predictable)
Agentic RAG	Agent decides when/ how to search	Multiple	High	Complex research, multiple tool approaches
Hybrid RAG	Query strengthening + search verification + answer quality check	Multiple	High	When repeated purification is required

Agent vs Chain approach (LangChain implementation)

Approach	Architecture	LLM Call	Suitable for
RAG Agent	agent + retriever tool	Multiple	Complex queries, query reorganization required
RAG Chain	Middleware Injection Context	single	Simple Q&A, Predictable Costs

LangGraph Custom	StateGraph + Custom Node	Multiple	Fine-grained control such as relevance evaluation and rewriting

5.3 Document loading & chunking

Document Loaders

The document loader reads raw content from various sources and returns it as a `Document` object with fields `page_content` and `metadata`.

loader	Source	package
`PyPDFLoader`	PDF file	`pypdf`
`TextLoader`	text file	built
`CSVLoader`	CSV file	built
`WebBaseLoader`	web page	`beautifulsoup4`
`DirectoryLoader`	Files in directory	built

Text Splitting

`RecursiveCharacterTextSplitter` maintains semantic relevance by recursively splitting it in the order `\n\n` -> `\n` -> ` ` -> `""`. Recommended as the most general purpose splitter.

parameters	Description	Recommended value
`chunk_size`	Chunk maximum number of characters	500–2000 (small for precise search, large for context preservation)
`chunk_overlap`	Number of adjacent chunks shared characters	10–20% of chunk_size (to avoid loss of boundary information)

Other dividers

divider	Suitable for
`MarkdownHeaderTextSplitter`	Markdown document
`HTMLHeaderTextSplitter`	HTML document
`TokenTextSplitter`	Token Budget Based Split
`CodeTextSplitter`	Source code (language recognition)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

raw_docs = [
    Document(page_content="LangGraph is a state-based multi-actor with LLM"
        "A framework for building applications.",
        metadata={"source": "langgraph-docs"}),
    Document(page_content="Agents use tool to communicate with external systems."
        "Interact. The ReAct pattern alternates between reasoning and action.",
        metadata={"source": "agent-guide"}),
]
print(f"Loaded {len(raw_docs)} documents.")
```

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200,
)
splits = text_splitter.split_documents(raw_docs)

for i, doc in enumerate(splits):
    print(f"Chunk {i}: {doc.page_content[:60]}...")
print(f"Total chunks: {len(splits)}")
```

5.4 Building a vector store

Vector Store is a specialized database that indexes embeddings and performs similarity searches. `InMemoryVectorStore` is suitable for development/testing.

Comparison of major vector stores

Vector Store	Type	Suitable for
`InMemoryVectorStore`	In-process	Development, small dataset
`Chroma`	Embedded/Client-Server	Prototyping, medium-scale datasets
`FAISS`	In-process	High performance local search
`Pinecone`	Managed Cloud	Production, Scalability
`PGVector`	PostgreSQL extensions	Leverage existing PostgreSQL infrastructure

Search type

Search type	Description
`"similarity"`	Standard nearest neighbor search
`"mmr"`	Maximal Marginal Relevance – Balancing relevance and diversity (reducing duplication)
`"similarity_score_threshold"`	Return only documents with minimum similarity score or higher

```
from langchain_core.vectorstores import InMemoryVectorStore

vector_store = InMemoryVectorStore.from_documents(
    documents=splits, embedding=embeddings,
)
test_results = vector_store.similarity_search("LangGraph", k=2)
for doc in test_results:
    print(f"  [{doc.metadata['source']}] {doc.page_content[:80]}")
print(f"Vector store ready with {len(splits)} documents.")
```

5.5 Search tool Definition

Using `response_format="content_and_artifact"` splits the tool output into two parts:

- Content: String expression passed to the model (used for inference)
- Artifact: The original Document object (accessible programmatically, but not sent to the model)

This separation allows you to use readable text for the model and the original object with metadata for subsequent processing.

```
from langchain_core.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    """Search our knowledge base for related articles."""
    docs = vector_store.similarity_search(query, k=4)
    serialized = "\n\n".join(
        f"Source: {d.metadata.get('source', '?')}\n{d.page_content}"
        for d in docs
    )
    return serialized, docs
```

5.6 LangChain RAG Agent – `create_agent` + `\@tool`

Simplest way: register the retriever as tool and call the agent when needed.

Multi-step search flow

RAG Agent can automatically run multiple discovery steps:

1. Initial Search – Create a query based on user questions
2. Result evaluation – Determine whether the retrieved documents are sufficient for the question
3. Reorganize and re-search – If there are not enough results, modify the query and re-search
4. Consolidation – Combine all search results to create final answer

This approach is suitable for complex research questions, but multiple LLM calls increase costs and delays.

5.7 LangChain RAG Chain – `\@dynamic_prompt` Middleware

Implement RAG with a single LLM call. `@dynamic_prompt` retrieves the document before the LLM call and automatically injects it into the system prompt. Because it is a middleware method, it operates in a single pass without an agent loop.

Characteristics	RAG Agent	RAG Chain
Number of LLM calls	Multiple (agent decision)	single
Number of searches	More than once (agent control)	Exactly once (middleware control)
Query Reconstruction	automatic	Not supported
delay	High (multiple round trips)	Low (single pass)
cost	High (more tokens)	Low (fewer tokens)
Transparency	Agent inference exposes message	Context injection is implicit

Advanced usage: You can also combine both approaches by injecting the default context with `@dynamic_prompt` while simultaneously providing a retriever tool.

```
from langchain.agents.middleware import dynamic_prompt

@dynamic_prompt
def rag_prompt(request):
    """It retrieves the document and injects it into the system prompt."""
    user_msg = request.state["messages"][-1].content
    docs = vector_store.similarity_search(user_msg, k=4)
    ctx = "\n\n".join(d.page_content for d in docs)
    return f"Answer based on the following context:\n\n{ctx}"
```

5.8 LangGraph Custom RAG – Building StateGraph

Build your own RAG agent that allows detailed control with LangGraph `StateGraph`. The key advantage of this approach is that conditional routing allows fine-grained flow control, such as evaluating the relevance of search results and rewriting queries if they are irrelevant.

Architecture

The custom RAG graph follows this high-level flow — a Rewrite → Retrieve → Agent loop with a relevance gate in between:

```
        [generate_query_or_respond]
             /              \
       (tool call)       (no tool call)
            |                 |
       [retrieve]           [END]
            |
```

```
   [grade_documents]
      /          \
(relevant)    (not relevant)
    |              |
[generate]   [rewrite_question]
    |              |
  [END]    [generate_query_or_respond]
```

- `generate_query_or_respond` decides whether the model should search or answer directly.
- If a tool call is made, `retrieve` runs the search.
- `grade_documents` evaluates whether the retrieved documents are relevant.
- If the documents are relevant, `generate_answer` produces the final answer.
- If the documents are not relevant, `rewrite_question` rewrites the query and loops back to `generate_query_or_respond`.

> · NOTE
>
> Because `rewrite_question` can loop back to `generate_query_or_respond`, add a `retry_count` field to state so the graph cannot loop forever — 2–3 attempts is a sane default before falling back to "answer with what we have".

Node	Role
`generate_query_or_respond`	Entry node. Decides whether to search or answer directly.
`retrieve`	Runs retrieval through a `ToolNode`.
`grade_documents`	Evaluates document relevance with structured output (`GradeDocuments`).
`rewrite_question`	Rewrites the query into something more specific when the results are not relevant.
`generate_answer`	Generates the final answer from relevant documents.

```python
from langgraph.graph import MessagesState

class AgentState(MessagesState):
    """Custom RAG agent status."""
    relevance: str  # "relevant" or "not_relevant"

print(f"AgentState keys: {list(AgentState.__annotations__)}")
```

5.9 `generate_query_or_respond` node

This is the entry node. Determines whether LLM will call retrieve tool or respond directly.

5.10 `grade_documents` Node – Evaluate relevance with structured output

The `GradeDocuments` schema allows LLM to evaluate document relevance. Receive a structured response as `with_structured_output`.

```
from pydantic import BaseModel, Field
from typing import Literal

class GradeDocuments(BaseModel):
    """Binary relevance score of the retrieved document."""
    relevance: Literal["relevant", "not_relevant"] = Field(
        description="Whether the document is relevant."
    )
    reasoning: str = Field(description="A brief explanation.")

grader = llm.with_structured_output(GradeDocuments)
```

```
def grade_documents(state: AgentState):
    """
    Evaluates the relevance of the retrieved documents.
    """

    msgs = state["messages"]

    user_q = next(
        (m.content for m in msgs if m.type == "human"),
        ""
    )

    tool_content = msgs[-1].content

    grade = grader.invoke(
        f"Question: {user_q}\nDocument:\n{tool_content}\n"
        f"Are these documents relevant?"
    )

    return {
        "relevance": grade.relevance,
        "messages": msgs
    }
```

5.11 `rewrite_question` node

When retrieved documents are irrelevant, rewrite the original question to be more specific to improve search quality.

5.12 `generate_answer` node

Once relevant documents are identified, the search results and the original question are combined to generate the final answer.

5.13 Graph assembly & execution

Register all nodes in `StateGraph` and connect them with conditional edges (`tools_condition` , `relevance_router`).

```
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition

def relevance_router(state: AgentState):
    if state.get("relevance") == "relevant":
        return "generate_answer"
    return "rewrite_question"

graph = StateGraph(AgentState)
graph.add_node("gen_query", generate_query_or_respond)
```

```
graph.add_node("retrieve", ToolNode([retrieve]))
graph.add_node("grade_documents", grade_documents)
graph.add_node("rewrite_question", rewrite_question)
graph.add_node("generate_answer", generate_answer)

graph.add_edge(START, "gen_query")
graph.add_conditional_edges(
    "gen_query", tools_condition,
    {"tools": "retrieve", "__end__": END},
)
```

```
graph.add_edge("retrieve", "grade_documents")
graph.add_conditional_edges(
    "grade_documents", relevance_router,
    {"generate_answer": "generate_answer",
     "rewrite_question": "rewrite_question"},
)
graph.add_edge("rewrite_question", "gen_query")
graph.add_edge("generate_answer", END)

app = graph.compile()
print("Graph compilation successful.")
```

Summary

Comparison of three RAG approaches

Characteristics	RAG Agent	RAG Chain	LangGraph Custom
Number of LLM calls	Multiple	single	Multiple
Number of searches	agent decision	Exactly 1 time	Custom
Query Reconstruction	automatic	Not supported	explicit node
Relevance Assessment	implicit	None	`GradeDocuments`
control level	low	low	High

Implementation Complexity	low	Lowest	High

Core LangGraph patterns

pattern	implementation
conditional routing	`add_conditional_edges` + `tools_condition`
structured output	`llm.with_structured_output(GradeDocuments)`
tool node	`ToolNode([retrieve])`
loop control	`rewrite_question` -\> `gen_query` cycle

Next Steps

→ 06_sql_agent.ipynb: Creates a SQL agent.

06 SQL Agent Advanced

– LangChain & LangGraph

We build agents that convert natural language into SQL queries in two ways: LangChain `create_agent` + `SQLDatabaseToolkit` (simple version) and LangGraph `StateGraph` (custom version). Covers Human-in-the-Loop, `interrupt()`, and `Command(resume=...)` patterns.

Learning Objectives

- Understand the 8-step workflow of SQL Agent
- Utilizes 4 tool of `SQLDatabase` and `SQLDatabaseToolkit`
- Implement ReAct-based SQL Agent with LangChain `create_agent`
- Add approval before query execution with `HumanInTheLoopMiddleware`
- Build a custom SQL Agent with LangGraph `StateGraph`
- Force tool calling with `bind_tools` and `tool_choice`
- Implement query review with `interrupt()` and `Command(resume=...)`

6.1 Environment Setup (SQLite + Chinook DB)

```
# %pip install langchain langchain-openai langchain-community langgraph sqlalchemy python-dotenv

from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
from langchain_community.utilities import SQLDatabase

llm = ChatOpenAI(model="gpt-5.4")
db = SQLDatabase.from_uri("sqlite:///Chinook.db")
print(f"Dialect: {db.dialect}")
```

6.2 SQL Agent Overview

SQL Agent follows an 8-step process to convert natural language questions into SQL queries:

```
1. Receive the question -> 2. List tables -> 3. Inspect the relevant table schema
-> 4. Generate the SQL query -> 5. Validate the query -> 6. (Optional) Human review
-> 7. Execute the query -> 8. Interpret the result
```

Why Do You Need an Agent?

Unlike a simple text-to-SQL pipeline, an agent can inspect schema, generate a query, validate it, and retry when needed. This improves accuracy because the agent can analyze errors and rewrite the query. It also uses the context window efficiently by loading only the schema that is actually needed.

Example agent execution trace

```
User: "What were the top 5 products by sales last month?"

Agent -> sql_db_list_tables()
      <- "customers, orders, order_items, products, categories"

Agent -> sql_db_schema("orders, order_items, products")
      <- CREATE TABLE orders (id INT, order_date DATE, ...)
         CREATE TABLE order_items (order_id INT, product_id INT, quantity INT, price DECIMAL, ...)

Agent -> sql_db_query_checker("SELECT p.name, SUM(oi.quantity * oi.price) ...")
      <- "The query looks correct."

Agent -> sql_db_query(validated_query)
      <- [("Widget Pro", 45230.00), ("Gadget X", 38100.00), ...]
```

Safety guidelines

Concern	Mitigation
SQL Injection	Use parameterized queries; the toolkit helps automatically.
DML execution	Ban INSERT/UPDATE/DELETE in the system prompt and enforce read-only DB permissions.
Expensive queries	Enforce LIMIT and require Human-in-the-Loop approval before execution.
Sensitive data	`include_tables` / `exclude_tables` to restrict accessible tables and enforce column-level permissions
Data exposure	Use database views or restricted user permissions.

Restricting Accessible Tables

In production, explicitly restrict which tables the agent may access:

```python
db = SQLDatabase.from_uri(
    "sqlite:///company.db",
    include_tables=["products", "orders", "order_items"],  # allowlist
    # exclude_tables=["users", "credentials"],             # or a denylist
)
```

6.3 SQLDatabaseToolkit

Automatically generates 4 tool:

tool	Features
`sql_db_list_tables`	Return all table names in database
`sql_db_schema`	CREATE TABLE statement + return sample rows
`sql_db_query`	Execute a SQL query and return results
`sql_db_query_checker`	LLM pre-checks queries for errors

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()

for t in tools:
    print(f"  {t.name}: {t.description[:60]}...")
print(f"Total tools: {len(tools)}")
```

6.4 LangChain SQL Agent – `create_agent` + ReAct

`create_agent` is LangChain's high-level API, which takes a model and tool and automatically constructs a ReAct(Reasoning + Acting) loop. The agent calls tool in order, following the workflow defined in the system prompt.

How ReAct loop works

1. The LLM analyzes the user question and conversation history to decide **which tool** to call next
2. tool is executed and the results are added to the conversation history
3. LLM will check the results and return to step 1 if additional tool calling is needed
4. Return a text response when the final answer is ready

What the system prompt does

System prompts define the agent's instructions for action. In SQL Agent, you must specifically specify the following:

- tool calling order: Force order `list_tables` → `schema` → `query_checker` → `query`
- Safety rules: Use `LIMIT`, no DML, query only necessary columns
- Error handling: Directs rewriting when a query error occurs.
- SQL dialect: Specify the dialect of the current DB (SQLite, PostgreSQL, etc.)

```
system_prompt = (
    "You are a SQL agent. Follow these steps:\n"
    "1. sql_db_list_tables\n2. sql_db_schema\n"
    "3. Write the query + sql_db_query_checker\n"
    "4. sql_db_query\n5. Interpret the result.\n"
    f"Rules: use LIMIT 10. DML is forbidden. Dialect: {db.dialect}"
)
```

```python
from langchain.agents import create_agent

sql_agent = create_agent(
    model=llm, tools=tools, system_prompt=system_prompt,
)
print("LangChain SQL Agent created.")
```

6.5 Run test

6.6 HITL – `HumanInTheLoopMiddleware`

In a production environment, human approval is required before executing SQL queries. This is because agent-generated queries can be expensive, access unexpected tables, or return results that are different from what you intended.

`HumanInTheLoopMiddleware` intercepts the specified tool(`sql_db_query`) call and suspends execution, allowing human review.

3 review options

When an agent attempts to call `sql_db_query` , execution is suspended and the human chooses one of the following:

Options	`Command(resume=...)` value	Description
Approve	`{"decisions": [{"type": "approve"}]}`	Execute the generated query as-is
Edit	`{"decisions": [{"type": "edit", "args": {...}}]}`	Execute the modified query
Reject	`{"decisions": [{"type": "reject", "message": "..."}]}`	Skip execution and return the reason

> ➡️ TIP
>
> The v1 `HumanInTheLoopMiddleware` accepts a `decisions` array so multiple tool calls in the same interrupt can be reviewed together. Each entry maps 1:1 to a queued tool call, and `type` is one of `approve` / `edit` / `reject` .

Why is HITL important?

- Cost Control: Avoid large table full scans without `LIMIT` .
- Data Protection: Pre-block access to sensitive columns
- Accuracy Verification: If the agent misinterpreted the intention of the question, correction is possible.
- Audit Trail: Maintains approval records for all executed queries

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

hitl = HumanInTheLoopMiddleware(
    interrupt_on={"sql_db_query": True},
)
sql_agent_hitl = create_agent(
    model=llm, tools=tools,
    system_prompt=system_prompt, middleware=[hitl],
)
print("Created SQL Agent with HITL applied.")
```

Resume pattern for destructive queries (DELETE / UPDATE)

A read-only SELECT can be released with a plain `approve`, but destructive statements (DELETE/UPDATE/INSERT) are typically `edit`-ed first to add stricter `WHERE` clauses or `LIMIT` guards before execution. When the agent interrupts, deliver the reviewer's decision through the `decisions` array in `Command(resume=...)`.

```
from langgraph.types import Command

# Dangerous query — reviewer adds LIMIT and approves the edited statement
result = sql_agent_hitl.invoke(
    Command(resume={
        "decisions": [{
            "type": "edit",
            "args": {
                "query": (
                    "UPDATE invoices SET status='void' "
                    "WHERE customer_id=42 AND total < 1 LIMIT 10"
                )
            },
        }],
    }),
    config={"configurable": {"thread_id": "sql-session-1"}},
)
```

6.7 LangGraph Custom SQL Agent – StateGraph

LangChain `create_agent` can be used for quick prototyping, but if you need fine-grained control at the node level, use LangGraph `StateGraph`. By defining each step as an independent node, we can achieve:

- Conditional Branch: Route to regeneration node when query validation fails.
- Force tool calling: With `bind_tools(tool_choice=...)`, a specific tool calling must be installed on a specific node.
- Fine-grained breakpoints: Break execution exactly on the desired node with `interrupt()`
- Custom Status: Add query history, retry count, etc. to the status.

Graph structure

```
START -> list_tables -> get_schema -> generate_query
      -> check_query -> execute_query -> END
```

Each node receives the shared `State` object and appends messages while the workflow runs. With `tools_condition`, you can implement a conditional branch that either regenerates the query or proceeds to execution based on the validation result.

LangChain `create_agent` Advantages Compared with

Aspect	`create_agent`	`StateGraph`
Execution order	LLM decides autonomously	Explicitly enforced with graph edges
Retry on error	Depends on the system prompt	Explicitly implemented with conditional edges
Human review	Middleware-based	`interrupt()` based, position free
Debugging	Black box	Status of each node can be checked

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class SQLState(TypedDict):
    messages: Annotated[list, add_messages]

print(f"SQLState keys: {list(SQLState.__annotations__)}")
```

6.8 Dedicated nodes – `list_tables`, `get_schema`, `generate_query`, `check_query`

Each node is responsible for one step of the SQL Agent workflow.

6.9 `bind_tools` with `tool_choice` – Force tool calling

Set Force a call to a specific tool with the `tool_choice` parameter.

6.10 Reviewing queries with `interrupt()`

LangGraph's `interrupt()` function suspenses graph execution and waits for external input (a human review). Unlike `HumanInTheLoopMiddleware`, `interrupt()` is more flexible as it allows you to break at an exact location in the code inside the node.

How it works

1. Calling `interrupt(payload)` inside a node function will immediately halt graph execution
2. `payload` is passed to the client and displayed in the review UI (i.e. the generated SQL query)
3. When the client resumes the graph with `Command(resume=value)`, `interrupt()` returns `value`

4. The node function executes, modifies, or rejects the query based on the returned values.

`interrupt()` **VS** `HumanInTheLoopMiddleware`

Characteristics	`interrupt()`	`HumanInTheLoopMiddleware`
Scope of application	Code level within node	tool calling Level
Flexibility	Arbitrary logic can be implemented	tool calling Interception only
state access	Entire State accessible	Only tool arguments are accessible
checkpointer	Required (stateful required)	optional

6.11 `Command(resume=...)` pattern

To resume a graph stopped by `interrupt()`, use `Command(resume=...)`.

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

builder = StateGraph(SQLState)
builder.add_node("list_tables", list_tables_node)
builder.add_node("get_schema", get_schema_node)
builder.add_node("generate_query", generate_query_node)
builder.add_node("check_query", check_query_node)
builder.add_node("execute_query", execute_query_node)
```

```
builder.add_edge(START, "list_tables")
builder.add_edge("list_tables", "get_schema")
builder.add_edge("get_schema", "generate_query")
builder.add_edge("generate_query", "check_query")
builder.add_edge("check_query", "execute_query")
builder.add_edge("execute_query", END)

checkpointer = InMemorySaver()
sql_graph = builder.compile(checkpointer=checkpointer)
print("LangGraph SQL Agent compiled.")
```

Summary

Comparison of two SQL Agents

Characteristics	LangChain `create_agent`	LangGraph `StateGraph`
Implementation Complexity	Low (5 lines)	High (dedicated node)
control level	ReAct Automatic	Node-level customization

HITL	`HumanInTheLoopMiddleware`	`interrupt()` + `Command(resume=...)`
forced tool calling	Not supported	`bind_tools(tool_choice=...)`
Suitable for	Rapid Prototype	Production, granular control

HITL pattern

action	`Command(resume=...)`
Approve	`{"decisions": [{"type": "approve"}]}`
Edit	`{"decisions": [{"type": "edit", "args": {...}}]}`
Reject	`{"decisions": [{"type": "reject", "message": "..."}]}`

4 SQLDatabaseToolkits tool

tool	steps	Use
`sql_db_list_tables`	2	Check table list
`sql_db_schema`	3	DDL + sample data query
`sql_db_query_checker`	5	Query pre-validation
`sql_db_query`	7	Run query

Next Steps

→ 07_data_analysis.ipynb: Creates a data analysis agent.

07 Data Analysis Agent

Deep Agents + Sandbox

Build an autonomous agent that takes CSV data as input, performs exploratory data analysis (EDA), generates visualization results, and shares them through Slack.

Utilizes the Deep Agents SDK's `create_deep_agent`, backend system, and built-in tool and checkpointer.

Learning Objectives

After completing this notebook, you will be able to:

1. Backend Selection — Understand the difference between `LocalShellBackend` (development) and `DaytonaSandbox` / `Modal` / `Runloop` (operations) and be able to choose
2. Custom tool Definition — External services tool, such as Slack integration, can be created with the `@tool` decorator.
3. Agent Creation — With `create_deep_agent()`, you can configure an agent that combines model, tool, backend, and checkpointer.
4. Streaming Observation — You can monitor the agent's analysis process in real-time.
5. Use built-in tool — Understand the role of `write_todos`, `ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep` tool
6. checkpointer — You can maintain the conversation with `InMemorySaver` and then perform analysis.

7.1 Environment Setup

Install Deep Agents SDK, Tavily (web search), and Slack SDK. Use `pip` or `uv` depending on the execution environment.

```
from dotenv import load_dotenv

load_dotenv()
```

7.2 Data Analysis Agent Overview

Deep Agents' data analysis agents autonomously run the following pipelines:

```
CSV input -> planning (`write_todos`) -> file reading (`read_file`) -> code generation and execution ->
iterative analysis -> result delivery (Slack)
```

Detailed Execution Flow

Step	Description	Tools Used
Planning	`write_todos` builds a structured task plan and updates TODOs as the analysis progresses	`write_todos`
File Reading	inspects CSV structure, column names, data types, and row counts; image files can also be read multimodally	`read_file`
Code Execution	pandas, matplotlib and runs Python code in an isolated backend	Backend `execute`
Iterative Analysis	performs follow-up analysis and uses web search to gather domain context	Tavily, `edit_file`
Result Delivery	formats the analysis result and sends it to Slack	`slack_send_message`

Key Traits of an Autonomous Agent

A Deep Agents data-analysis agent goes beyond simple tool use and can plan autonomously:

1. planning: `write_todos` to break the analysis into steps and track progress
2. Adaptive execution: if an error occurs during analysis, it can revise the code (`edit_file`) and run it again.
3. subagent Delegation: Complex tasks are processed in parallel by creating specialized subagent
4. Context Management: Store intermediate results in file system tool and refer back to them when needed

7.3 Backend selection

Deep Agents provide a filesystem and code execution environment through a pluggable backend architecture. All backends implement the same `BackendProtocol` , so you can replace just the backend without changing any code.

backend	Use	Security level	Setting Difficulty
`LocalShellBackend`	Local development/ testing	Low (host system full access)	No setup required
`Daytona`	Cloud sandbox environment	High (isolated container)	API key required

`Modal`	Serverless GPU/CPU computation	High	Modal account required
`Runloop`	Running a managed cloud	High	API key required

Details by backend type

- `StateBackend` (default): Stores files in the LangGraph agent state. It works well as a scratch pad, and large printouts are automatically removed.
- `FilesystemBackend`: Provides local disk access with `root_dir` settings. The `virtual_mode=True` option restricts paths and prevents directory traversal.
- `LocalShellBackend`: Provides **unlimited shell command execution** through the `execute` tool in addition to filesystem access. It runs with full user privileges on the host system.
- `CompositeBackend`: Routes different backends by route. Example: `StateBackend` for temporary files and `StoreBackend` for `/memories/`.

Sandbox Security Principles

Sandboxes provide an isolated environment where agents can run code safely. Core security principles:

- Never put secrets in the sandbox – Context injection attacks can allow agents to read environment variables or credentials from mounted files and thus leak them.
- Keep your credentials outside the sandbox in tool
- Use Human-in-the-Loop approval for sensitive tasks
- Block unnecessary network access

> ➡️ TIP
>
> Caution: `LocalShellBackend` has unrestricted shell execution permissions on the host system. Be sure to use a sandbox backend in production.

```python
from deepagents.backends import LocalShellBackend

# For development purposes — local shell backend
dev_backend = LocalShellBackend(virtual_mode=True)

# For Operations — Cloud Sandbox (Choose 1)
# prod_backend = ... # Use cloud sandbox backend in production
```

7.4 Upload sample data

Create a CSV file in the backend's working directory for the agent to analyze. `create_deep_agent`'s backend has `write_file` tool built in, so the agent can write files directly, but here the data is prepared in advance.

```
import os

workspace = "/tmp/analysis"
os.makedirs(workspace, exist_ok=True)

csv_content = (
    "region,quarter,revenue,units_sold\n"
    "Seoul,Q1,120000,340\nSeoul,Q2,135000,380\n"
    "Seoul,Q3,128000,355\nSeoul,Q4,150000,420\n"
    "Busan,Q1,85000,240\nBusan,Q2,92000,260\n"
    "Busan,Q3,88000,250\nBusan,Q4,105000,300"
)
```

```
with open(f"{workspace}/sales_2025.csv", "w") as f:
    f.write(csv_content)
print(f"CSV saved: {workspace}/sales_2025.csv")
```

7.5 Custom tool – Slack integration

Use the `@tool` decorator to define a custom tool that the agent can call. docstring in tool tells the agent how to use it.

Below is tool, which transmits analysis results to a Slack channel.

```
from langchain_core.tools import tool
try:
    from slack_sdk import WebClient
except ImportError:
    WebClient = None
    print("slack_sdk not installed -- Slack tool works as a stub")

slack_client = WebClient(token=os.environ.get("SLACK_BOT_TOKEN", "xoxb-placeholder")) if WebClient else
None

@tool
def slack_send_message(message: str) -> str:
    """Send analysis results to Slack channel."""
    resp = slack_client.chat_postMessage(
        channel=os.environ.get("SLACK_CHANNEL_ID", "general"), text=message)
    return f"Sent successfully. Timestamp: {resp['ts']}"
```

Web Search tool (Tavily)

Prepares Tavily tool so that agents can perform web searches when domain context is needed during analysis.

```
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def tavily_search(query: str) -> str:
    """Search the web for information related to your analysis."""
    results = tavily_client.search(query, max_results=5)
    return "\n".join(
        [r["content"] for r in results["results"]]
    )
```

pandas / scikit-learn `@tool` integration

Data analysis agents often treat each tool as a tiny analysis cell. The `@tool` decorator runs pandas / scikit-learn code on demand and returns the result back to the model as text. Keeping these tools separate from filesystem access (`read_file`) makes input validation, caching, and unit testing far easier.

```
import pandas as pd
from sklearn.cluster import KMeans
from langchain_core.tools import tool

@tool
def summarize_csv(path: str, top_n: int = 5) -> str:
    """Return shape, dtypes, and the top-N rows of a CSV file."""
    df = pd.read_csv(path)
    return (
        f"shape={df.shape}\n"
        f"dtypes={df.dtypes.to_dict()}\n"
        f"head=\n{df.head(top_n).to_string()}"
    )

@tool
def cluster_customers(path: str, k: int = 3) -> str:
    """KMeans clustering over numeric columns only."""
    df = pd.read_csv(path).select_dtypes("number").dropna()
    labels = KMeans(n_clusters=k, n_init="auto").fit_predict(df)
    return f"cluster_sizes={pd.Series(labels).value_counts().to_dict()}"
```

`CodeInterpreterMiddleware` and the Skills pattern

For free-form analysis that does not fit a fixed `@tool` signature — "draw a boxplot of this column", for example — `CodeInterpreterMiddleware` lets the agent execute arbitrary code in its own sandbox. Repeatable procedures, on the other hand, are best captured in a `skills/` directory as markdown + code snippets and loaded as Skills the agent can invoke by name.

```
from deepagents.middleware import CodeInterpreterMiddleware, TodoListMiddleware
from deepagents.skills import load_skills

skills = load_skills("./skills/data-analysis")  # markdown + snippets

agent = create_deep_agent(
    model="gpt-5.4",
    tools=[summarize_csv, cluster_customers, slack_send_message],
    middleware=[
        TodoListMiddleware(),
        CodeInterpreterMiddleware(backend=backend),
    ],
    skills=skills,
    backend=backend,
    checkpointer=checkpointer,
    system_prompt="You are a data analyst.",
)
```

> ➡️ **TIP**
>
> Keep reproducible analysis recipes inside the Skills directory and let `CodeInterpreterMiddleware` handle one-off exploration. This prevents validated procedures from blending with throwaway code.

7.6 Creating an agent

`create_deep_agent()` creates an agent by combining the model, tool, backend, checkpointer, and system prompts.

parameters	Type	Description
`model`	`str`	model identifier (default: `claude-sonnet-4-6`)
`tools`	`list`	List of tool functions to be used by the agent
`backend`	`Backend`	Code execution and filesystem backend
`checkpointer`	`Checkpointer`	State Perpetuation Mechanism
`system_prompt`	`str`	Agent Behavior Guidelines

```
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver

backend = LocalShellBackend(virtual_mode=True)
checkpointer = InMemorySaver()
```

```
agent = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=backend,
    checkpointer=checkpointer,
    middleware=[TodoListMiddleware()],
    system_prompt="You are a data analyst.",
)
```

7.7 Execution — Analysis Request

When an analysis request is sent to `agent.invoke()`, the agent autonomously performs the following pipeline: planning → reading files → executing code → delivering results.

7.8 Observe the analysis process through streaming

`agent.stream()` allows you to observe the agent's execution in real-time. Deep Agents provides powerful monitoring that can trace the execution of subagent based on LangGraph's streaming infrastructure.

Stream mode

mode	Description	Use cases

Updates	Receive events upon completion of each step (node)	Progress Dashboard, Log
Messages	Individual token streaming, including source agent metadata	Real-time chat UI
Custom	Issue custom progress event with `get_stream_writer()`	Analysis progress display, custom notifications

Namespace system

Streaming events contain a namespace that identifies their source:

- `()` (empty tuple) = main agent
- `("tools:abc123",)` = subagent created with tool calling
- `("tools:abc123", "model_request:def456")` = model request node inside subagent

By setting `subgraphs=True`, you can trace the execution of subagent as well, and you can also use multiple stream modes simultaneously:

```
for namespace, chunk in agent.stream(
    {"messages": [...]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
):
    mode, data = chunk
    # Handle each mode differently
```

Tracking the subagent lifecycle

A subagent goes through three stages:

1. Pending — detected when the main agent includes a delegated task in its `model_request`
2. Running — `tools:UUID` starts when events appear under the namespace
3. Complete — finishes when the main agent's `tools` node returns its result

7.9 Utilizing built-in tool

Deep Agents provides file tools through the backend, and this chapter opts into planning with `TodoListMiddleware`. The agent autonomously selects and uses these tools during the analysis process.

tool	Description	Example usage
`write_todos`	Create and track structured work plans when `TodoListMiddleware` is enabled	Creation of TODO list for each stage of analysis
`ls`	List directory contents	Check CSV file existence

`read_file`	Read file (image multimodal support)	Understand CSV structure, check chart image
`write_file`	Write a file, replacing existing content at that path	Analysis script, save result file
`edit_file`	Modifying existing files (find-and-replace)	Code fix, update config file
`glob`	glob pattern based file navigation	`*.csv` Searching data files by pattern
`grep`	Pattern matching search	Search for specific column name or value
`delete`	Remove a file or directory when the backend exposes deletion and permissions allow it	Clean generated scratch outputs

Multimodal support in `read_file`

`read_file` supports image files as multimodal content in all backends. After the agent creates a chart with matplotlib, you can visually interpret the contents of the chart by reading the image with `read_file`. Through this, it autonomously determines "whether the chart was created correctly" and "what additional visualization is needed."

Custom backend implementation

You can implement `BackendProtocol` yourself depending on your needs. Required methods:

- `ls()` – List directory contents
- `read()` – Read file with line numbers
- `grep()` – Pattern matching that returns structured matches.
- `glob()` – glob-based file matching
- `write()` – Write file content, replacing an existing file at that path
- `edit()` – find-and-replace that guarantees uniqueness

7.10 Maintain conversation with checkpointer

checkpointer saves agent state to enable abort and resume. A conversation session is identified through `thread_id`, and subsequent requests with the same `thread_id` will maintain the previous conversation context.

checkpointer	Use	permanence
`InMemorySaver`	Development/Test	Destroyed when process ends
`SQLiteSaver`	local persistence	Save to Disk (`./agent_checkpoints.db`)

`PostgresSaver`	Production	Store in database, support multiple instances

Role of checkpointer

checkpointer goes beyond simply saving conversation history and allows you to:

- Interruption Recovery: Resume from the last completed step in case of network error or timeout
- `interrupt()` **support**: Saves the graph state in human-in-the-loop flows and resumes at the correct location after the human responds.
- Multi-turn analysis: Process multiple analysis requests consecutively in the same session while referencing previous results

Sandbox Lifetime Management

An explicit shutdown is required when using a sandbox. Failure to terminate will result in unnecessary costs. In your chat application, use a unique sandbox for each conversation thread and configure automatic cleanup with Time-to-Live (TTL) settings.

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
config = {"configurable": {"thread_id": "analysis-session-1"}}

agent_with_memory = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=LocalShellBackend(virtual_mode=True),
    checkpointer=checkpointer,
)
```

Summary

To summarize what we covered in this notebook:

Topic	Key Takeaways
Backend	`LocalShell` (development) uses host shell access, production uses `Daytona` / `Modal` / `Runloop` sandbox
Custom tool	`\@tool` Defined by decorator + docstring, agent calls autonomously
Create Agent	`create_deep_agent(model, tools, backend, checkpointer, system_prompt)`
Streaming	Real-time observation with `agent.stream(stream_mode="updates", subgraphs=True)`
Built-in tool	`write_todos` , `ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`
checkpointer	`InMemorySaver` (Development) → `SQLiteSaver` → `PostgresSaver` (Production)

Next Steps

→ 08_voice_agent.ipynb: Creates a voice agent.

08 Voice Agent

STT/Agent/TTS Pipeline

We build a voice agent that converts voice input to text (STT), processes it by the LangChain agent, and then synthesizes text into speech (TTS) and returns it in real-time.

This architecture is a Sandwich pattern (STT -> Agent -> TTS), where each layer is connected by streaming, targeting sub-700ms latency.

Core design principles

The core of the Sandwich architecture is Streaming Chaining. Rather than waiting for the complete output of the previous layer, each layer passes partial results to the next layer as soon as they are produced:

- STT: Generates partial transcripts in real-time and emits final transcripts when detecting end-of-speech.
- Agent: Streams the response token by token through `astream()` — does not wait for the entire response generation to complete.
- TTS: Start audio synthesis as soon as a text chunk arrives based on WebSocket

Thanks to this structure, the latency of the entire pipeline is reduced to the sum of the time until the first output of each stage, rather than the sum of each stage.

Learning Objectives

After completing this notebook, you will be able to:

1. Sandwich Architecture — Can explain the structure and data flow of STT -> Agent -> TTS pipeline
2. Architecture Comparison — You can compare the pros and cons of the Sandwich method and the Speech-to-Speech (S2S) method.
3. STT Step — You can understand the Producer-Consumer pattern of AssemblyAI real-time transcription.
4. Agent Phase — You can build an agent that generates streaming responses with LangChain `create_agent`
5. TTS Step — Understand the operating principle of Cartesia WebSocket-based streaming voice synthesis.
6. RunnableGenerator — Pipelines can be combined with asynchronous generator chaining.
7. Performance Optimization — Understand optimization techniques at each stage to achieve latency goals.

> · NOTE
>
> Note: STT (AssemblyAI) and TTS (Cartesia) are external paid services. The cells are provided as markdown to illustrate the concept, only the agent creation part is the actual executable code.

8.1 Environment Setup

These are the packages and each role required for a voice agent:

package	Role
`langchain` , `langchain-openai`	Create Agent and Connect LLM
`assemblyai`	Real-time speech-to-text (STT)
`cartesia`	Low-latency text-to-speech synthesis (TTS)
`websockets`	Real-time two-way communication server
`pyaudio`	Microphone audio capture (optional)

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

8.2 Voice Agent Architecture Overview

The voice agent consists of a 3-layer pipeline:

```
Audio In -> [STT: AssemblyAI] -> Text -> [Agent: LangChain] -> Text -> [TTS: Cartesia] -> Audio Out
```

Layer	Provider	Role
STT	AssemblyAI	User Converts speech into text
Agent	LangChain	Processes the text query and generates a response
TTS	Cartesia	Converts the agent's text response into speech

Each layer is connected through streaming, so partial results can be forwarded immediately without waiting for the previous step to finish completely:

1. STT — streams partial transcriptions (emits a final transcript when the utterance is complete)
2. Agent — streams responses token by token (`astream()`)
3. TTS — Start speech synthesis before the entire response is complete (WebSocket-based)

Why Streaming Matters

When processing synchronously without streaming, Next Steps does not start until each step is completely finished. For example, if the agent takes 3 seconds to generate a 200 token response, TTS will only start after 3 seconds. In streaming, on the other hand, TTS begins synthesizing speech as soon as the agent's first token appears, so the user already starts hearing speech while the agent is still generating a response.

8.3 Architecture comparison table

Compare two approaches to building voice agents.

Sandwich architecture (STT -\> Agent -\> TTS)

Item	Content
Advantages	Each component can be replaced independently, easy to use text-based tool, convenient for debugging (intermediate text logging)
Disadvantages	Latency accumulation and loss of non-verbal information such as emotion/intonation due to 3-step serial processing
Suitable for	tool calling Complex agents with many, when multilingual support is required

Speech-to-Speech (S2S)

Item	Content
Advantages	Low latency, preservation of voice characteristics (emotion, stress), natural conversation
Disadvantages	tool calling Difficulty in integration, limited model selection, difficulty in debugging
Suitable for	Simple conversational interface, when emotion recognition is important

> · NOTE
>
> In this notebook, we focus on the tool calling-enabled Sandwich architecture.

8.4 STT Step – AssemblyAI Real-Time Transcription

> ➡️ TIP
>
> This cell requires an AssemblyAI API key. Reference code to understand the concept.

AssemblyAI's `RealtimeTranscriber` operates with the Producer-Consumer pattern:

- Producer: Send audio chunks captured from the microphone to WebSocket
- Consumer: Receives partial and final transcription results as a callback.

Since both operations take place simultaneously, you can receive a transcription of the previous utterance even while your voice is being transmitted.

Two types of transcription results

Type	class	Description
Partial	`RealtimeTranscript`	Temporary transcription of words still being uttered. Updated in real-time as the user speaks
Final	`RealtimeFinalTranscript`	Confirmation transcription after the end of utterance (endpointing) is detected. Pass only this result to the agent

AssemblyAI's built-in Voice Activity Detection (VAD) automatically detects when an utterance has ended, eliminating the need for separate silence detection logic.

```
import assemblyai as aai

aai.settings.api_key = "your-assemblyai-key"

transcriber = aai.RealtimeTranscriber(
    sample_rate=16000,
    encoding=aai.AudioEncoding.pcm_s16le,
    on_data=on_transcription_data,
    on_error=on_transcription_error,
)

def on_transcription_data(transcript: aai.RealtimeTranscript):
    if isinstance(transcript, aai.RealtimeFinalTranscript):
        process_user_input(transcript.text)

def on_transcription_error(error: aai.RealtimeError):
    print(f"Transcription error: {error}")

transcriber.connect()
```

Capturing Microphone Audio

PCM 16-bit, 16kHz Capture mono audio and send it to the transcriber in real time:

```
import pyaudio
```

```
audio = pyaudio.PyAudio()
stream = audio.open(
    format=pyaudio.paInt16,
    channels=1, rate=16000,
    input=True, frames_per_buffer=1024,
)

while True:
    data = stream.read(1024)
    transcriber.stream(data)
```

8.5 Agent Phase – Utilizing LangChain Agent

The agent stage is the core of the pipeline. An agent created with `create_agent` takes text input, performs inference with tool calling, and streams a text response.

The key to system prompts for voice agents is to elicit concise, conversational responses. For voice output, unlike text, long responses can be burdensome to the user, so users are instructed to generate short responses of no more than 1 to 2 sentences.

The role of asynchronous streaming

The agent's `astream()` method yields a response token as soon as it is generated. Why this is especially important for voice agents:

- TTS Early Start: Speech synthesis is possible from the first token without waiting for the entire response to be completed.
- Reduced perceived lag: Users start hearing the agent's response sooner.
- Pipeline efficiency: Agent and TTS run simultaneously, reducing total processing time

```
from langchain.agents import create_agent

def search_tool(query: str) -> str:
    """Search the web for the latest information."""
    return f"Search result: {query}"

def calendar_tool(action: str, details: str) -> str:
    """Manage calendar events."""
    return f"Calendar {action}: {details}"
```

```
agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool],
    system_prompt=(
        "You are a useful voice assistant."
        "Keep your responses brief and conversational."
    ),
)
```

Generate asynchronous streaming response

`astream()` yields the agent's response token as soon as it is generated. Allows the TTS stage to start speech synthesis without waiting for the entire response to complete.

LangChain's streaming offers two main methods:

method	Description
`astream()`	Stream the output for each step of agent execution. Contains messages, tool calling results, etc.
`astream_events()`	More granular event-level streaming. Provide detailed events such as individual tokens, LLM start/end, etc.

The voice agent receives message chunks as `astream()`, extracts `content` from each chunk, and passes it to TTS.

```
async def stream_agent_response(user_text: str):
    """Stream agent response tokens one by one."""
    async for chunk in agent.astream(
        {"messages": [{"role": "user",
                       "content": user_text}]}
    ):
        if "messages" in chunk:
            for msg in chunk["messages"]:
                if hasattr(msg, "content") and msg.content:
                    yield msg.content
```

Multimodal content blocks and `output_version="v1"`

A voice agent's input is not always pure text — a user may attach a photo, or a tool may return a chart image. LangChain v1 messages accept a content blocks array, so a single message can mix text, image, and audio. Creating the model with `output_version="v1"` also normalizes responses into the `AIMessage.content_blocks` schema, which is safe to serialize between STT, Agent, and TTS processes.

```
from langchain.chat_models import init_chat_model

llm_v1 = init_chat_model("gpt-5.4", output_version="v1")

response = llm_v1.invoke([
    {"role": "user", "content": [
        {"type": "text", "text": "Explain any anomalies in this chart by voice."},
        {"type": "image", "source_type": "base64",
         "mime_type": "image/png", "data": "..."},
    ]},
])

# v1 schema — content_blocks survives JSON round-trips between services
for block in response.content_blocks:
    if block["type"] == "text":
        tts_input_queue.put_nowait(block["text"])
```

> ➡️ TIP
>
> `output_version="v1"` normalizes responses into the `content_blocks` schema. When messages cross process boundaries via WebSocket (a TTS server, for example), this prevents JSON (de)serialization from corrupting the payload — practically mandatory for boundary-heavy systems like voice pipelines.

8.6 TTS Steps – Cartesia Streaming Speech Synthesis

> ➡️ TIP
>
> This cell requires a Cartesia API key. Reference code to understand the concept.

Cartesia provides WebSocket-based low-latency TTS. It takes a text stream from an agent and generates audio chunks for each partial text on the fly.

How Cartesia TTS works

1. WebSocket connection: Maintain a persistent connection by opening a WebSocket session with the `AsyncCartesia` client.
2. Send text chunks: Send text in units of tokens/sentences generated by the agent.
3. Receive Audio Chunk: Receive PCM audio bytes immediately for each text chunk
4. Client Delivery: Stream the received audio bytes to the client via WebSocket.

Because it is based on WebSocket, two-way real-time communication is possible without HTTP request/response overhead. The `sonic-2` model provides both natural speech and low latency.

```
import cartesia

cartesia_client = cartesia.AsyncCartesia(
    api_key="your-cartesia-key"
)

async def text_to_speech_stream(text_stream):
    ws = await cartesia_client.tts.websocket()
    async for text_chunk in text_stream:
        audio_chunks = ws.send(
            model_id="sonic-2",
            transcript=text_chunk,
            voice_id="your-voice-id",
            stream=True,
            output_format={
                "container": "raw",
                "encoding": "pcm_s16le",
                "sample_rate": 24000,
            },
        )
        async for audio in audio_chunks:
            yield audio["audio"]
    await ws.close()
```

8.7 Pipeline Combination – RunnableGenerator

LangChain's `RunnableGenerator` allows you to integrate asynchronous generators into your Runnable pipeline. This allows the entire data flow of STT output -> Agent processing -> TTS input to be organized into one pipeline.

What is RunnableGenerator?

`RunnableGenerator` wraps an asynchronous generator function into LangChain's `Runnable` interface. If you do this:

- Chaining with other Runnables is possible using the `|` (pipe) operator.
- Standard methods such as LangChain's `batch()` and `stream()` can be used.
- Suitable for patterns that receive an input stream and generate a converted output stream.

```
from langchain_core.runnables import RunnableGenerator

async def transform_input(input_stream):
    async for text in input_stream:
        async for token in stream_agent_response(text):
            yield token

agent_runnable = RunnableGenerator(transform_input)
```

Wiring the Full Pipeline (conceptual code)

Use `asyncio.Queue` to pass the final STT transcript to the agent and forward the agent's streaming response to TTS. `Queue` is a core building block in an async producer–consumer pattern.

```
async def voice_pipeline(audio_input_stream):
    transcript_queue = asyncio.Queue()

    def on_final(transcript):
        if isinstance(transcript, aai.RealtimeFinalTranscript):
            transcript_queue.put_nowait(transcript.text)

    transcriber = aai.RealtimeTranscriber(
        sample_rate=16000, on_data=on_final
    )
    transcriber.connect()

    async for audio_chunk in audio_input_stream:
        transcriber.stream(audio_chunk)
        if not transcript_queue.empty():
            user_text = await transcript_queue.get()
            text_stream = stream_agent_response(user_text)
            async for audio in text_to_speech_stream(text_stream):
                yield audio
```

8.8 WebSocket Server – Real-time two-way communication

> ➡️ **TIP**
>
> Running the code in this cell will start the server. This is reference code to help you understand the concept.

Handles two-way audio streaming with clients via WebSockets. Receiving and sending are executed simultaneously with `asyncio.gather`.

Why use WebSockets

Voice agents require two-way, real-time communication:

- Receive: Client continuously transmits microphone audio to the server
- Send: The server continuously transmits synthesized voice audio to the client.

HTTP's request-response model is not suitable for this kind of two-way streaming. WebSockets support two-way data flow over a single TCP connection, and with `asyncio.gather`, ingress and egress run concurrently without blocking each other.

```
import websockets
import asyncio

async def handle_client(websocket):
    transcriber = create_transcriber()
    transcriber.connect()

    async def receive_audio():
        async for message in websocket:
            if isinstance(message, bytes):
                transcriber.stream(message)

    async def send_audio():
        async for transcript in transcription_queue:
            text_stream = stream_agent_response(transcript)
            async for audio in text_to_speech_stream(text_stream):
                await websocket.send(audio)

    await asyncio.gather(receive_audio(), send_audio())

async def main():
    async with websockets.serve(handle_client, "0.0.0.0", 8765):
        print("Voice agent server on ws://0.0.0.0:8765")
        await asyncio.Future()
```

8.9 Performance Target – Sub-700ms Latency

A key performance metric for voice agents is Time to First Audio (TTFA). For a natural conversation experience, this metric should be less than 700ms.

Latency analysis

steps	target time	Description
STT final transcription	200ms	Detect end of utterance -\> final text (AssemblyAI built-in endpointing)
Agent first token	300ms	Enter text -\> Create first response token (TTFT: Time to First Token)
TTS first audio	150ms	First text token -\> first audio chunk (WebSocket based)
Total	\<700ms	End of speech -\> First audio output

Optimization techniques

technique	effect	Implementation method
Streaming STT	Eliminate full transcription wait	`RealtimeTranscriber` + partial result
Streaming Agent	Start TTS before completing all responses	Use `astream()` — instead of `ainvoke()`
Streaming TTS	Reduce time to first audio	WebSocket-based synthesis
Connection Pooling	Eliminate connection setup delays	WebSocket reuse (avoid creating new connection per request)
VAD	Prevent processing of silent sections	AssemblyAI built-in endpointing
response caching	Immediate response to frequently asked questions	Frequently Asked Questions Caching

> ➡️ TIP
>
> Tip: Agent's TTFT (Time to First Token) accounts for the largest proportion of overall latency. Short system prompts and lightweight model selection are key to latency optimization.

8.10 Add tool to agent

The strength of the voice agent is that its Sandwich architecture allows it to leverage text-based tool. You can add a variety of tool features such as search, calendar management, weather lookup, and more.

This is the biggest difference from the S2S (Speech-to-Speech) method. The S2S model processes audio directly, making text-based tool calling difficult, but the Sandwich architecture allows agents to operate in text areas, so all existing LangChain tool can be used as is.

tool Design Tips

tool of voice agents fast response is important. tool As the execution time increases, the latency of the entire pipeline increases:

- Designed mainly for light API calls
- Timeout setting required
- Apply caching if possible

```
def weather_tool(city: str) -> str:
    """View the current weather in your city."""
    return f"{city} weather: 15°C, partly cloudy"

def reminder_tool(time: str, message: str) -> str:
    """Set reminders at specified times."""
    return f"{time}Reminder scheduled for {message}"
```

```
voice_agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool,
           weather_tool, reminder_tool],
    system_prompt=(
        "You are a voice assistant."
        "Please keep your response concise and conversational in 1-2 sentences."
    ),
)
```

Summary

Topic	Key Takeaways
Sandwich Architecture	STT -\> Agent -\> TTS, each layer can be replaced independently, supports tool calling
STT (AssemblyAI)	`RealtimeTranscriber`, Producer-Consumer pattern, WebSocket streaming
Agent (LangChain)	`create_agent` + `astream()`, token-level streaming, tool integration
TTS (Cartesia)	WebSocket-based low-latency synthesis, instant audio conversion of partial text
Pipeline Combination	`RunnableGenerator`, asynchronous generator chaining
Performance Goals	Sub-700ms (STT 200ms + Agent 300ms + TTS 150ms)
tool Expansion	Text-based tool can be applied to voice agents as is

Next Steps

→ 09_production.ipynb: Learn production deployment.

09 Production deployment

– testing, observability, deployment

We cover the entire pipeline for deploying agents into production. We ensure quality through unit testing and LangSmith evaluation, secure observability through tracing, and deploy to the LangGraph Platform.

```
Development -> Testing (unit + evaluation) -> Observability (tracing) -> Deployment (LangGraph Platform)
```

Why do you need an agent-specific production pipeline?

Agents have different characteristics than traditional software:

- Non-deterministic execution: Same input can produce different tool calling orders and different responses.
- Statefull: A long-running process that manages conversation history and checkpoints.
- Multiple components: Multiple systems such as LLM, tool, memory, checkpointer, etc. are linked

Therefore, beyond simple unit testing, agent-specific quality assurance techniques such as trajectory evaluation, LLM-as-Judge, and trace-based monitoring are needed.

Learning Objectives

After completing this notebook, you will be able to:

1. Unit Test – You can write deterministic tests by mocking the LLM response with `GenericFakeChatModel`
2. LangSmith Evaluation – Create datasets, define evaluators, and perform automated agent evaluation.
3. Trace Analysis – LangSmith tracing allows you to track latency, token usage, and errors.
4. LangGraph Studio – You can analyze the agent execution flow with visual debugging tool
5. Deployment Options – Understand the differences between LangGraph Platform, self-host, and cloud deployment
6. langgraph.json – You can configure the deployment configuration file and execute deployment commands.

9.1 Environment Setup

Install the packages required for testing, observability, and deployment.

9.2 Production Pipeline Overview

Due to the non-deterministic nature of agents, traditional software testing alone cannot ensure quality.

steps	Purpose	tool
Development	Agent implementation and local testing	LangGraph, LangGraph Studio
Test	Unit Testing + LLM Based Assessment	pytest, agentevals, LangSmith
observability	Tracing, Metrics, Error Tracking	LangSmith Tracing
Distribution	Deploying a Production Environment	LangGraph Platform

Testing Strategy

Agent testing requires a combination of two approaches:

Type	Features	Suitable for
Unit	Mock LLM responses with `GenericFakeChatModel` for isolated, deterministic tests; runs fast without API calls	Individual tools, parsers, prompt formatting, state transition logic
Integration	Validate cross-component behavior with real network calls; `pytest` + `vcrpy` cassettes keep runs reproducible	End-to-end agent flow, tool calling sequence, external API contracts
Evals	Quantify response quality and tool-call ordering with LangSmith datasets and `agentevals` trajectory evaluators	Answer accuracy, trajectory match, LLM-as-Judge rubrics

> ➡️ TIP
>
> All three test types are needed — Unit guards against regressions, Integration verifies external API contracts, Evals measure non-deterministic answer quality. A common pattern is to run unit + integration on every CI run, sample-eval on each PR, and full-eval nightly.

Because agent systems operate by chaining multiple components, integration testing generally requires a higher proportion. Unit tests verify the correctness of individual components, and integration tests evaluate the quality of the overall flow.

9.3 Agent testing – LangSmith evaluation

The `agentevals` package provides a dedicated evaluator for agent trajectories. Trajectory refers to all the steps (tool calling, intermediate reasoning, decision making) that the agent takes to reach the final response.

strategy	Description	Advantages	Disadvantages
Trajectory Match	Step-by-step comparison with expected sequence. Matching the agent's actual trajectory with a predefined reference trajectory	Accurate verification, reproducible	Requires specific expectations, low flexibility
LLM-as-Judge	LLM qualitatively evaluates trajectories based on rubrics (evaluation criteria). tool Automatic judgment of appropriateness of use, accuracy of response, etc.	Flexible evaluation, response to complex scenarios	Additional LLM costs, indeterminacy of the assessment itself

Trajectory Match Mode

You can adjust your agent's tool calling ordering expectations in four levels:

mode	Description	When to use
`strict`	The message and tool calling order must be exactly the same as the reference to pass	Workflows where order is important (e.g. Authentication -\> Lookup -\> Update)
`unordered`	If the same tool are called, they pass regardless of the order. When there are multiple independent tool calling	`subset`
Agent only calls reference tool (no additional tool calling)	When you want to prevent unnecessary tool calling	`superset`

Agent contains at least reference tool (additional calls allowed)	When you want to ensure only the core tool calling	

```
try:
    from langsmith import Client

    ls_client = Client()

    dataset = ls_client.create_dataset("agent-eval-v1")
    ls_client.create_examples(
        inputs=[{"query": "What is LangGraph?"}],
        outputs=[{"expected": "Framework for Agents"}],
        dataset_id=dataset.id,
    )
    print("Dataset created:", dataset.name)
except Exception as e:
    print(f"LangSmith not configured (skipped): {e}")
    ls_client = None
    dataset = None
```

```
try:
    from agentevals.trajectory import create_trajectory_llm_as_judge

    evaluator = create_trajectory_llm_as_judge(
        rubric=(
            "Did the agent use the appropriate tool?"
            "Was your final answer correct?"
        ),
    )
    print("Evaluator created:", type(evaluator).__name__)
except ImportError:
    print("agentevals not installed. Installation: pip install agentevals")
    evaluator = None
except Exception as e:
    print(f"Evaluator creation skipped (LLM API key required): {e}")
    evaluator = None
```

9.4 Unit test patterns

`GenericFakeChatModel` allows you to write deterministic tests by mocking LLM responses without making API calls. It takes a response iterator and returns one for each call.

Why mocking?

When I call the actual LLM API from my agent tests, I run into the following issues:

- Non-deterministic results: It is difficult to reproduce the test because the same input may result in a different response each time.
- Cost: API cost incurred each time a test is run
- Speed: Network delay slows down testing
- Availability: Test fails in case of API failure

`GenericFakeChatModel` returns predefined responses in sequence, allowing you to write tests that are deterministic, fast, and free. Streaming patterns are also supported, so `astream()`-based code can also be tested.

State persistence testing

`InMemorySaver` checkpointer allows you to test state-dependent behavior across multiple conversation turns. Specify `thread_id` to verify the cumulative state of multiple calls while maintaining the same conversation context.

```
def search_tool(query: str) -> str:
    """Search for information on the web."""
    return f"Search result: {query}"

def test_search_tool():
    """Tests whether search tool returns the expected format."""
    result = search_tool("test query")
    assert isinstance(result, str) and len(result) > 0
    print("Passed: test_search_tool")

test_search_tool()
```

HTTP request recording/playback

To reduce API costs in CI/CD, you can record and replay HTTP requests with `vcrpy` and `pytest-recording`. Record the actual API call (save it as a cassette file) on the first run, and play the recorded response on subsequent runs to test without network calls.

Advantages of this approach:

- First run: Verifies integration with actual API (roles as integration test)
- Run after: Quick and conclusive testing with recorded responses (acts as a unit test)
- CI/CD: Tests can be run without an API key

```
import pytest

@pytest.fixture(scope="module")
def vcr_config():
    return {"record_mode": "once"}

@pytest.mark.vcr()
def test_agent_with_recorded_responses():
    result = agent.invoke("What is LangGraph?")
    assert "framework" in result.lower()
```

9.5 observability – LangSmith Tracing

LangSmith tracing records every step of an agent's execution. A trace is a complete record of an agent's execution, from initial user input to the final response, including all model calls, tool usage, and decision points.

Information recorded in traces

Item	Description
Input/Output	Input data and output results of each step
Model Call	Prompts, responses, model parameters

tool calling	Called tool, arguments, return value
Latency	Time required for each stage
Token Usage	Number of input/output tokens
Error	Failed Steps and Error Messages

How to activate

Just set two environment variables and you'll get automatic tracing without any additional code:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=<your-api-key>
```

`create_agent` agents automatically send execution data to LangSmith when the relevant environment variables are configured. All LangChain components (LLMs, chains, tools, etc.) include built-in instrumentation, so no extra code changes are required.

Selective tracing & PII anonymization (`tracing_context` + `LangChainTracer`)

`tracing_context` lets you scope tracing to a code block and swap project, tags, and metadata at runtime. In production, you typically pair it with a directly-instantiated `LangChainTracer` and an anonymizer callback that masks PII (emails, phone numbers, IDs) before the trace leaves your process.

```
import re
from langsmith import tracing_context
from langchain.callbacks.tracers import LangChainTracer

EMAIL_RE = re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+")
PHONE_RE = re.compile(r"\b\d{2,3}-\d{3,4}-\d{4}\b")

def anonymize(payload: dict) -> dict:
    """Mask PII before the trace ships to LangSmith."""
    text = str(payload)
    text = EMAIL_RE.sub("<email>", text)
    text = PHONE_RE.sub("<phone>", text)
    return {**payload, "redacted": text}

tracer = LangChainTracer(
    project_name="production-agent",
    client_kwargs={"hide_inputs": anonymize,
                   "hide_outputs": anonymize},
)

with tracing_context(
    project_name="production-agent",
    tags=["v2.1", "experiment-A"],
    metadata={"user_tier": "premium"},
):
    agent.invoke({"messages": [...]}, config={"callbacks": [tracer]})
```

! WARNING

> Run PII masking before the trace leaves the process. Keep the original payload on the model side and store only the redacted copy in LangSmith.

9.6 Trace analysis

Monitor the quality of your production agents by analyzing traces in the LangSmith dashboard.

metrics	Description	Confirmation Points
Latency	Time required for each stage	Identify bottleneck section (which node is the slowest)
Token Usage	Number of input/output tokens	Cost optimization (adjust prompt length, remove unnecessary context)
Error Rate	Failed Execution Rate	Stability monitoring (failure rate for specific tool, LLM timeouts, etc.)
Tool Call Frequency	Call frequency by tool	Agent behavior pattern analysis (identifying excessive tool calling and unused tool)

Utilizing tags and metadata

You can classify and filter traces by adding custom tags and metadata with the `config` parameter or `tracing_context`. Examples of useful tags/metadata in a production environment:

Tags/Metadata	Use
Version tag (`v2.1`)	A/B testing, performance comparison by version
Experimental Tag (`experiment-A`)	Experiment tracking, including prompt changes
User Tier (`premium`)	Quality monitoring by user group
Region (`kr`)	Latency analysis by region

Project Management

Projects can be set up in two ways:

- static: Specify the default project with the `LANGSMITH_PROJECT` environment variable.
- Dynamic: Separate project by code block with `tracing_context(project_name=...)`

```
try:
    from langsmith import tracing_context

    with tracing_context(
```

```
        project_name="production-agent",
        tags=["v2.1", "experiment-A"],
        metadata={"user_tier": "premium", "region": "kr"},
    ):
        print("Tagged tracing enabled")
except Exception as e:
    print(f"LangSmith tracing unavailable: {e}")
```

9.7 LangGraph Studio – Visual Debugging

LangGraph Studio is a free tool that allows you to visually debug an agent's execution flow. Optimized for developing and testing agents on your local machine.

Features	Description
Graph visualization	Inspect the agent's node/edge structure in real-time and highlight the currently running node
Step-by-step execution	Debug by inspecting each node's input/output: prompts, tool calls, and results step by step
State inspection & editing	Browse message history and checkpoints, then edit node inputs on the fly and re-run
Time travel	Rewind to an earlier checkpoint to try alternative branches or tool calls
Interrupt handling (HITL)	Resume `interrupt()` -paused graphs directly from the Studio UI with `Command(resume=...)`
Thread management	Show conversation trees per `thread_id` in the sidebar so you can compare past runs
Live streaming	Observe execution in real-time with token / latency metrics

Setting up a local development server

To use Studio, start your local development server with the LangGraph CLI:

```
# Install LangGraph CLI (requires Python 3.11+)
pip install --upgrade "langgraph-cli[inmem]"

# Start the development server
langgraph dev
```

Once the server is running, open `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024` in your browser.

Information available in Studio

- Prompt sent to the agent
- Each tool call and its result

- Final output
- Intermediate state (inspectable and editable)
- Token usage and latency metrics

Chat UI scaffold – `npx create-agent-chat-app`

Studio is the developer's debugging surface; end-users need a separate chat UI. The `create-agent-chat-app` template from the LangChain team scaffolds a Next.js chat app wired to a deployed LangGraph agent in a single command.

```
npx create-agent-chat-app@latest my-agent-ui
cd my-agent-ui
# Set NEXT_PUBLIC_API_URL, NEXT_PUBLIC_ASSISTANT_ID, LANGSMITH_API_KEY in .env.local
pnpm dev
```

The template ships with `/runs/stream` SSE streaming, `thread_id`-scoped conversation history, and a built-in interrupt UI for `interrupt()` — a convenient shell around any deployed graph.

9.8 Deployment Options

Agents are long-running processes that maintain state, so they require a different approach than regular web app hosting. Traditional stateless web application hosting (e.g. Vercel, Heroku) is not suitable for persistent state management, background execution, and checkpointing of agents.

Options	Description	Suitable for
LangGraph Cloud	LangSmith Managed Hosting. Automatic build/deployment with just a GitHub connection	Rapid prototyping, small teams
Self-Hosted	Run as Docker containers on your own infrastructure	Data Sovereignty, Enterprise, Regulatory Environment
Hybrid	Cloud Management + Own Runtime	Convenience of management + data control

Deployment Requirements

Item	Description
GitHub repository	Code hosting (supports both public and private)
LangSmith Account	Free to join (smith.langchain.com)
langgraph.json	Deployment configuration file defining dependencies, graphs, and environment variables

LangGraph Cloud deployment process

1. Log in to LangSmith and go to the Deployments page.
2. Click “+ New Deployment”
3. Connect your GitHub account (for private repositories)
4. Select the repository and submit (takes about 15 minutes)
5. After deployment is complete, copy the API URL and use it on the client.

9.9 langgraph.json settings

`langgraph.json` is the core configuration file for your deployment. Define dependencies, graph entry points, and environment variables. This file must be located in the project root for LangGraph CLI and Cloud deployment to operate properly.

```
import json

langgraph_config = {
    "dependencies": ["."],
    "graphs": {"agent": "./src/agent.py:graph"},
    "env": ".env",
}
print(json.dumps(langgraph_config, indent=2))
```

Setting item details

Item	Type	Description
`dependencies`	`list[str]`	Python package dependencies. `.` references `pyproject.toml` in the current directory
`graphs`	`dict`	Mapping graph names and module paths. `"module_path:variable_name"` format
`env`	`str`	Environment variable file path (`.env` format). Contains sensitive information such as API keys

Graph entry point

The value of the `graphs` field is in the format `"./path/file.py:variable_name"`. The variable must be a `CompiledGraph` instance. Both LangGraph’s Graph API (`StateGraph`) and Functional API (`@entrypoint`) are available.

Graph API example

```
# src/agent.py
from langgraph.prebuilt import create_react_agent

graph = create_react_agent(
    model="claude-sonnet-4-6",
    tools=[search_tool],
    checkpointer=True,
)
```

Functional API Example

LangGraph's Functional API lets you add persistence, memory, and streaming to existing Python code with minimal changes. The `@entrypoint` decorator defines the start of the workflow, and the `@task` decorator marks an individual unit of work.

```
# src/agent.py
from langgraph.func import entrypoint, task

@task
def process_query(query: str) -> str:
    return f"Processing complete: {query}"

@entrypoint(checkpointer=checkpointer)
def graph(inputs: dict) -> str:
    result = process_query(inputs["query"])
    return result.result()
```

Key features of Functional API:

- Uses standard Python control flow (if/for, etc.) – No explicit graph structure required
- Function scope state management – No need to declare a separate state or set up a reducer.
- Task result checkpointing – Automatically reuses previously completed task results when re-executed
- Input/output JSON serialization required – When using checkpointer, all data must be serializable

9.10 Deployment command

Build, local server, and cloud deployment commands using LangGraph CLI.

1. Build Docker image

Create a Docker image based on the `langgraph.json` settings:

```
langgraph build -t my-agent:latest
```

2. Run the local server

Run the agent locally for testing and connect it to the Studio UI for visual debugging:

```
# Production mode (Docker-based)
langgraph up --config langgraph.json

# Development mode (in-memory, quick start)
langgraph dev
```

3. Cloud Deployment

LangSmith In the dashboard:

1. Deployments -> "+ New Deployment"
2. GitHub Connect the repository

3. Select the repository and submit it (about 15 minutes)
4. Copy the API URL after deployment completes

Python SDKAccess the deployed agent from the Python SDK

`langgraph-sdk` allows you to communicate programmatically with the deployed agent. All functions, including streaming, thread management, and status inquiry, are controlled by the SDK.

```
try:
    from langgraph_sdk import get_sync_client

    api_key = os.environ.get("LANGSMITH_API_KEY", "")
    if not api_key:
        print("LANGSMITH_API_KEY Not set. Skipping client connection.")
    else:
        client = get_sync_client(
            url="https://your-deployment.langsmith.com",
            api_key=api_key,
        )
        print("Client connected:", type(client).__name__)
except Exception as e:
    print(f"LangGraph SDK client unavailable: {e}")
```

Streaming with `client.runs.stream(...)`

The SDK's `runs.stream(...)` returns tokens, messages, and tool calls from the deployed agent over Server-Sent Events. Combine `stream_mode` values like `"values"`, `"updates"`, `"messages"`, `"events"`, and pass a `thread_id` to keep the conversation history of the same session.

```
from langgraph_sdk import get_client

client = get_client(url="https://your-deployment.langsmith.com",
                    api_key=api_key)

async for chunk in client.runs.stream(
    thread_id="user-42",
    assistant_id="agent",
    input={"messages": [
        {"role": "user", "content": "Summarize sales for the last 5 minutes."}
    ]},
    stream_mode=["updates", "messages"],
):
    print(chunk.event, chunk.data)
```

Access via REST API

Deployed agents can also be accessed through REST API. This allows you to communicate with the agent in any programming language/framework:

```
curl --request POST \
  --url <DEPLOYMENT_URL>/runs/stream \
  --header 'Content-Type: application/json' \
  --header 'X-Api-Key: <LANGSMITH_API_KEY>' \
  --data '{
    "assistant_id": "agent",
    "input": {
      "messages": [
        {"role": "user", "content": "Hello!"}
      ]
    },
```

```
    "stream_mode": "updates"
  }'
```

Key endpoints:

Endpoint	Method	Description
`/runs/stream`	POST	Streaming execution
`/runs`	POST	Synchronous execution
`/threads`	POST	Create a new thread
`/threads/{id}/state`	GET	thread status query

Summary

Topic	Key Takeaways
Testing Strategy	Unit test (`GenericFakeChatModel`) + integration test (`agentevals`) in parallel
LangSmith Evaluation	Trajectory Match (strict/unordered/subset/superset), LLM-as-Judge
observability	Automatic tracing with `LANGSMITH_TRACING=true` + `LANGSMITH_API_KEY`
Trace Analysis	Latency, Token Usage, Error Rate, Tool Call Frequency
LangGraph Studio	Visual graph debugging, step-by-step state inspection
Deployment Options	Cloud (Managed), Self-Hosted (Docker), Hybrid
langgraph.json	`dependencies` , `graphs` , `env` 3 core settings
Deployment Command	`langgraph build` -\> `langgraph up` -\> LangSmith Deploy

10 Deep Agent from Scratch

Assemble harness responsibilities manually

Building a Deep Agent from scratch is a way to understand what the SDK normally gives you. This chapter decomposes an agent into planning, tools, state, and quality gates.

Learning goals

- Identify the responsibilities hidden inside an agent harness.
- Sketch a planner, tool registry, and quality gate.
- Understand why production agents need explicit control surfaces.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

10.1 Minimal Deep Agent components

Start by naming the pieces: instructions, tools, memory or state, planning, execution, and verification. A clear parts list makes the harness easier to reason about.

```
harness_parts = [
    "planner", "tool_registry", "state", "filesystem", "subagent_dispatch", "quality_gate",
]

harness_parts
```

10.2 Todo planner skeleton

A todo planner gives the agent a visible working structure. Even a simple list helps separate intent, execution, and completion evidence.

```
def plan(request: str) -> list[dict]:
    return [
        {"task": "understand", "status": "done"},
        {"task": "draft", "status": "pending"},
        {"task": "verify", "status": "pending"},
    ]

plan("sync official docs")
```

10.3 Tool registry skeleton

A registry makes tool availability explicit. It is also the natural place to attach descriptions, risk levels, and permission rules.

```
def echo_tool(text: str) -> str:
    return f"echo: {text}"

tool_registry = {"echo": echo_tool}
print(tool_registry["echo"]("hello"))
```

10.4 Quality gate

A quality gate keeps the agent from treating any output as finished. It checks whether required evidence and next-step clarity are present.

```
def quality_gate(result: dict) -> bool:
    return bool(result.get("answer")) and result.get("verified") is True

quality_gate({"answer": "done", "verified": True})
```

Summary

Item	Content
Covered	planner, tool registry, state, filesystem, subagent dispatch, and quality gate skeletons
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `deep-agent-from-scratch.md`
- `overview.md`
- `customization.md`

11 Custom Workflow Agent

Combine deterministic and agentic steps

Many strong agent applications are not fully autonomous loops. They combine deterministic routing and validation with agentic steps only where open-ended reasoning is valuable.

Learning goals

- Route work deterministically before invoking an agentic step.
- Compile a small workflow from explicit stages.
- Keep predictable control flow around flexible model behavior.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class WorkflowState(TypedDict):
    question: str
    route: str
    answer: str
```

11.1 Deterministic router

A deterministic router handles cases where the decision rule is known. This keeps the agent from spending model calls on work that code can classify reliably.

```
def route(state: WorkflowState) -> dict:
    if "sql" in state["question"].lower():
        return {"route": "database"}
    return {"route": "general"}
```

```
def answer(state: WorkflowState) -> dict:
    text = f"{state['route']} workflow handled: {state['question']}"
    return {"answer": text}
```

11.2 Compile the workflow

A compiled workflow makes the control path reviewable. Each stage has a narrow responsibility, which makes testing and debugging easier.

```
builder = StateGraph(WorkflowState)
builder.add_node("route", route)
builder.add_node("answer", answer)
builder.add_edge(START, "route")
builder.add_edge("route", "answer")
builder.add_edge("answer", END)
graph = builder.compile()
```

```
graph.invoke({"question": "Explain the SQL result", "route": "", "answer": ""})
```

Summary

Item	Content
Covered	StateGraph workflows that mix deterministic routing with agentic answer steps
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `custom-multi-agent.md`
- `workflows-agents.md`
- `runtime.md`

PART

VI

LangSmith

Tracing · Evaluation · Prompt Ops · Agent Evals

What You Will Learn in This Part

- 1. Quickstart: Your First Trace
- 2. Agent Trace Structure
- 3. Datasets and the Evaluation Loop
- 4. Prompt Hub Versioning
- 5. Production Monitoring
- 6. Agent Evals
- 7. Testing Strategy
- 8. LangGraph Testing
- 9. Runtime Rubric Evaluation

01 LangSmith Quickstart

This notebook covers sending your first trace to LangSmith and checking it in the UI. Just three environment variables + zero changes to your LangChain code are all that's needed.

Learning Objectives

- Issue a LangSmith API key and inject it into your `.env`
- Confirm that simply setting `LANGSMITH_TRACING=true` automatically traces existing agent runs
- Attach `run_name`, `tags`, and `metadata` to traces to make them searchable
- Read the LLM/tool call tree, latency, token usage, and cost for a trace in the UI

What You'll Gain

- Reproducible execution history to answer "why did the agent respond this way?"
- Source data for evaluation and prompt tuning stages

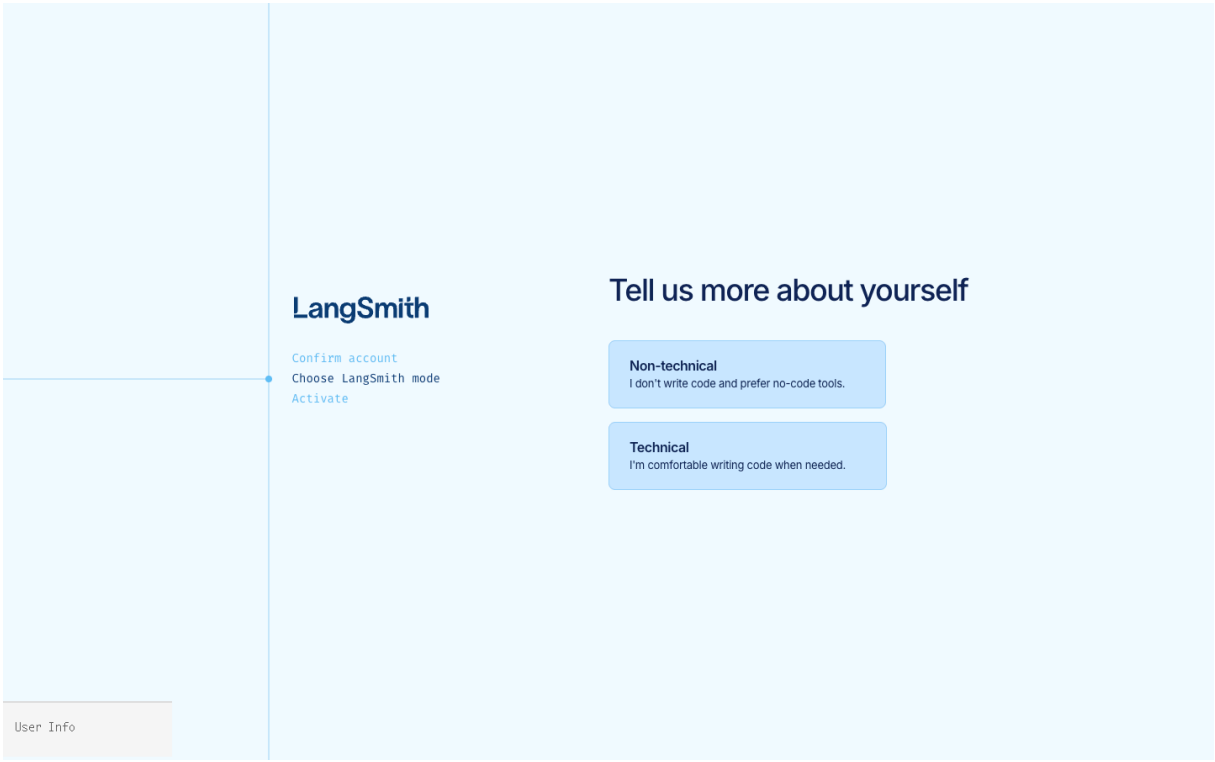
6.01.1 API Key & Environment Variables

1. Go to https://smith.langchain.com/ → `Settings → API Keys → Create API Key`
2. Add these three lines to your `.env`:

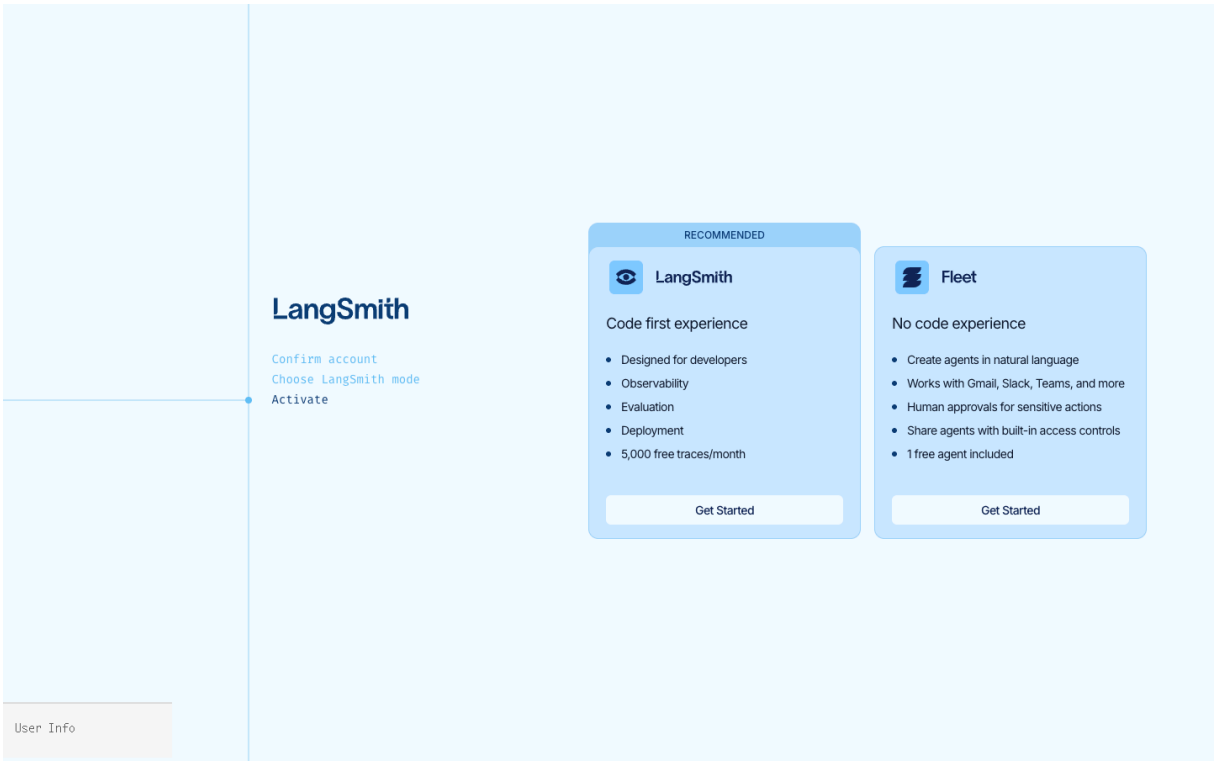
```
LANGSMITH_API_KEY=lsv2_pt_xxxxxxxx
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=langsmith-quickstart
```

> ➡️ **TIP**
>
> `LANGSMITH_PROJECT` is the project name shown in the UI. If unset, traces are recorded under `default`.

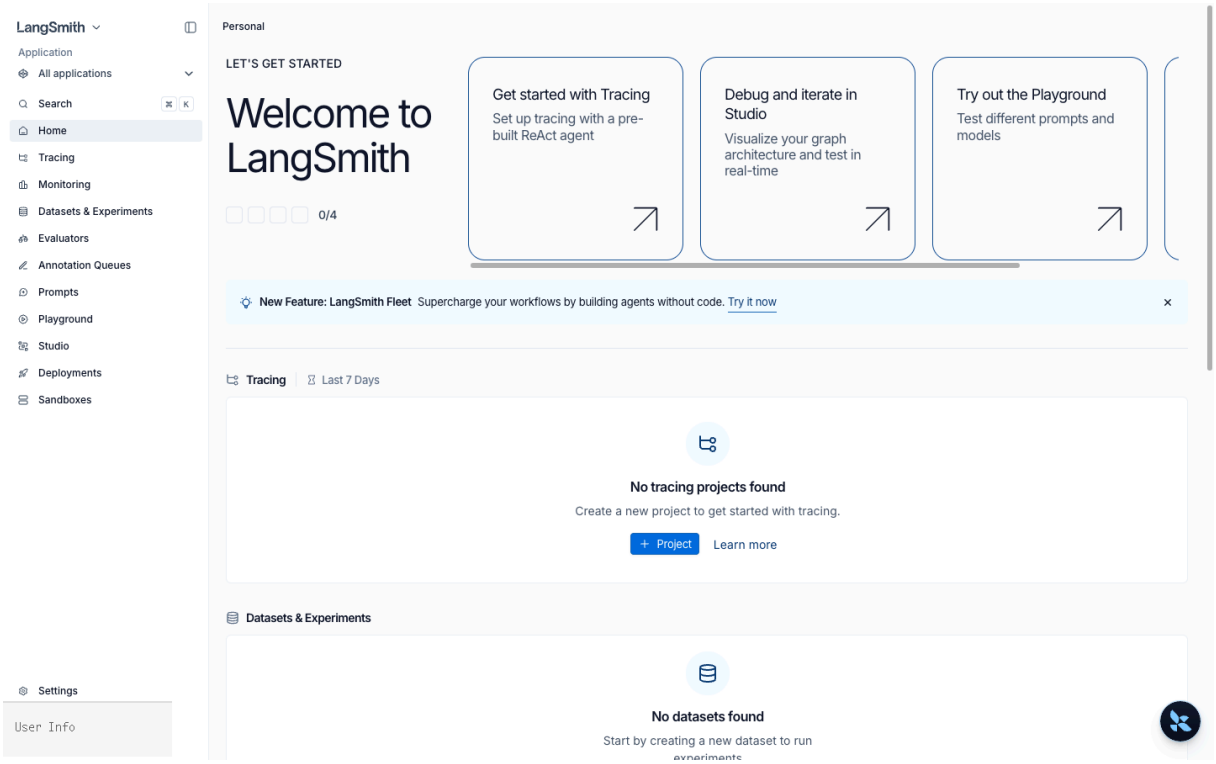
Onboarding Flow (First Time Only)



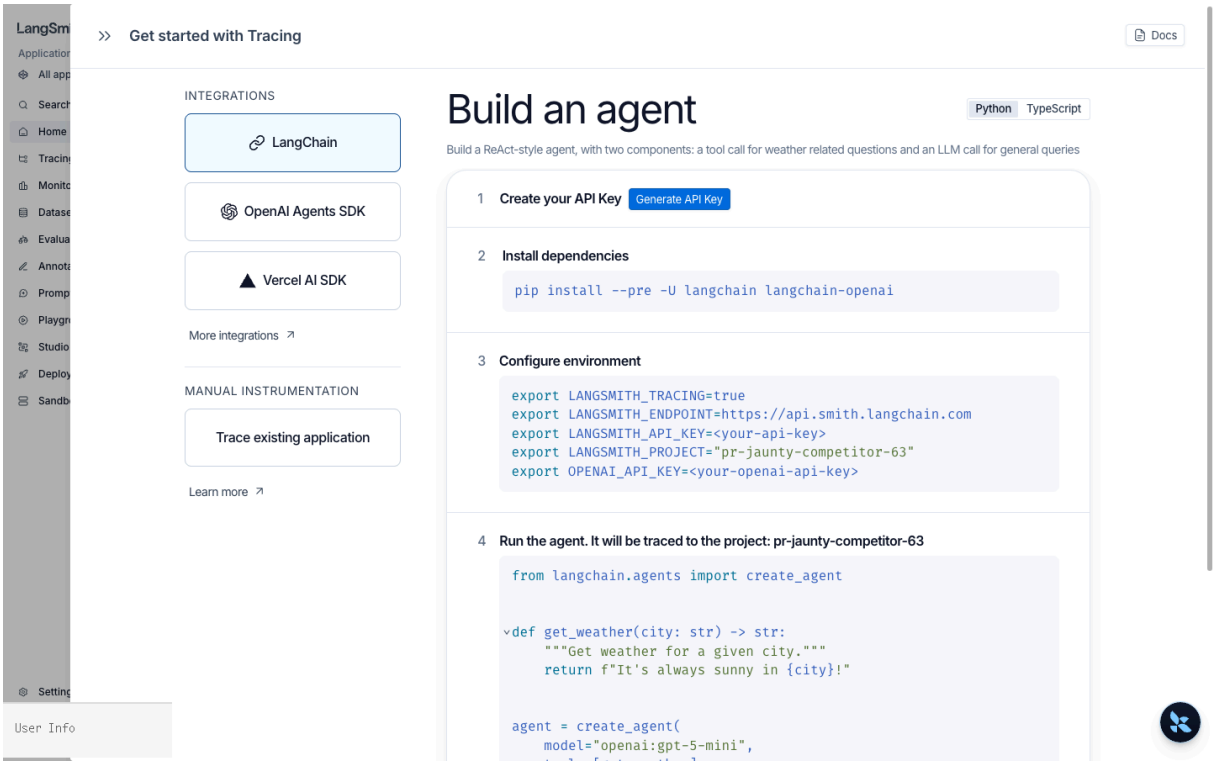
Select Technical → code-first flow for developers.



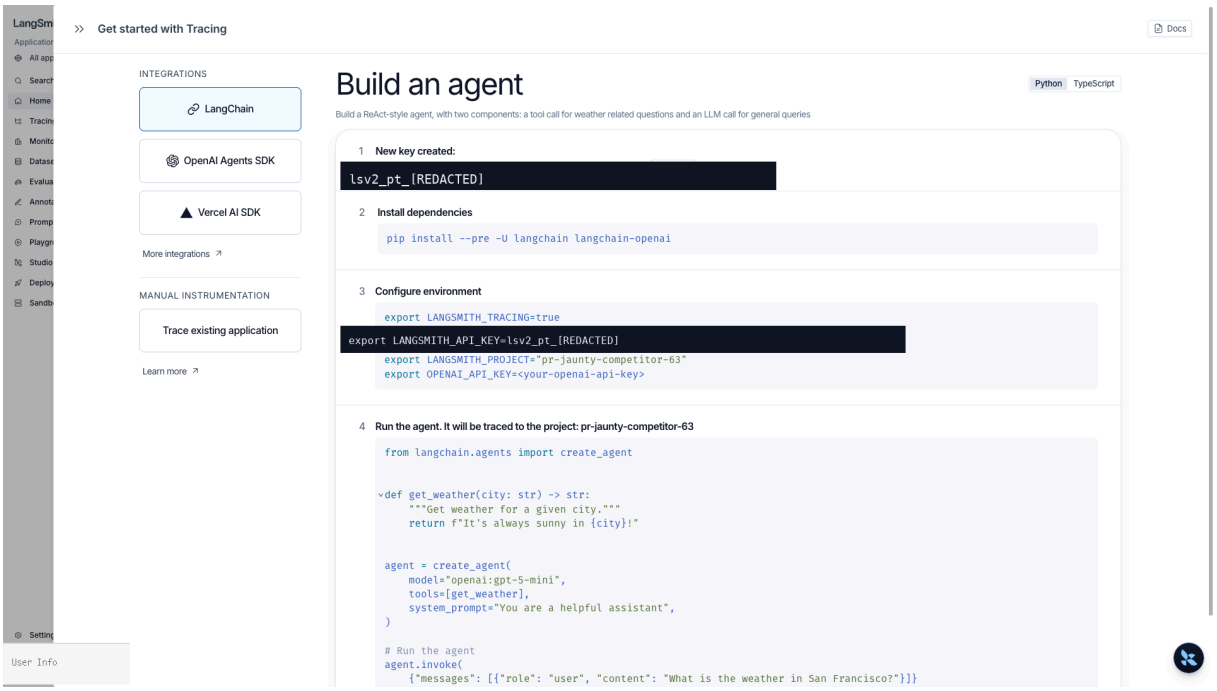
Branch between LangSmith (code-first) and Fleet (no-code). This notebook uses LangSmith.



After onboarding, Home — Tracing/Datasets/Prompts all show 0/4.



First card on Home → 4-step quickstart dialog. `Generate API Key` button is step 1.



Clicking Generate API Key creates an `lsv2_pt_…` key and auto-injects it into the env block in step 3. The key is shown only once, so copy it to `.env` immediately.

```
# !pip install -U langsmith langchain langchain-openai

from dotenv import load_dotenv
import os
load_dotenv(override=True)

assert os.environ.get("LANGSMITH_API_KEY"), "LANGSMITH_API_KEY not set"
assert os.environ.get("LANGSMITH_TRACING") == "true", "LANGSMITH_TRACING=true required"
# If LANGSMITH_PROJECT is not set, LangSmith records to the `default` project.
os.environ.setdefault("LANGSMITH_PROJECT", "default")
print("Project:", os.environ["LANGSMITH_PROJECT"])
```

6.01.2 First Trace — LangChain Agent

`create_agent` is automatically instrumented. As long as the environment variables are set, the following execution is recorded in LangSmith.

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Returns the current weather for a city."""
    return f"{city}: Clear, 21°C"

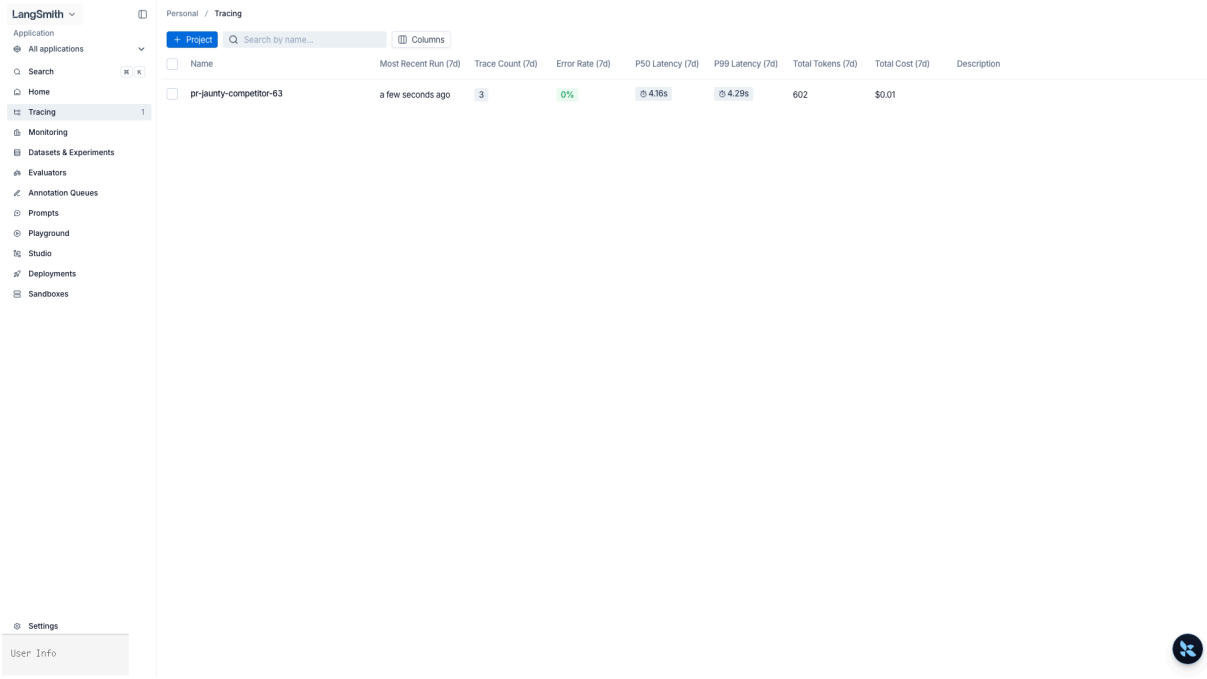
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[get_weather],
    system_prompt="You are a friendly weather bot.",
)

result = agent.invoke({
```

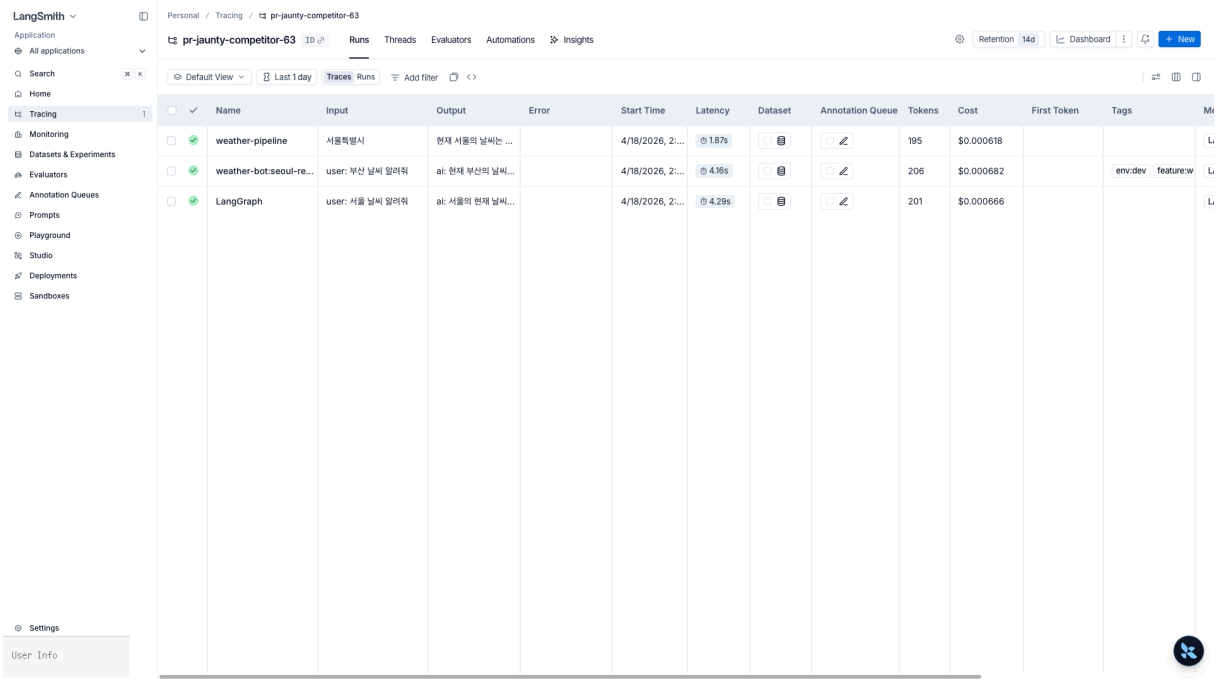
```
    "messages": [{"role": "user", "content": "Tell me the weather in Seoul"}]
})
result["messages"][-1].content
```

After running, check in the UI:

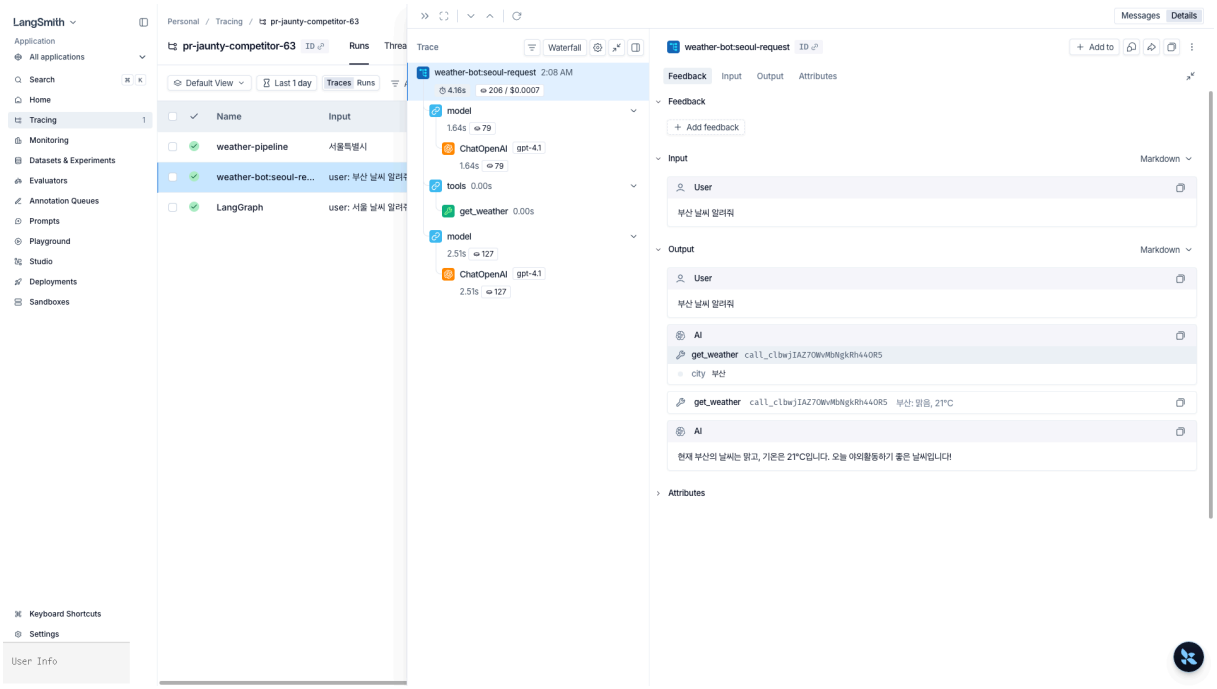
1. Go to https://smith.langchain.com/ → left `Projects` → `langsmith-quickstart`
2. The recent run appears as a row (run_name = `AgentExecutor` by default)
3. Click to see the LLM call → tool call → LLM call tree, with input/output, latency, and tokens for each step



Tracing menu → The `pr-jaunty-competitor-63` project just traced by the agent. Trace Count, P50/P99 Latency, Total Tokens, and Total Cost for the last 7 days are automatically populated.



Click the project → individual run table. Input / Output / Latency / Tokens / Cost / Tags / Metadata at a glance. Tabs: Runs · Threads · Evaluators · Automations · Insights.



Click a run → waterfall tree. The sequence `model → tools → model` and each step's tokens/latency are visually reconstructed.

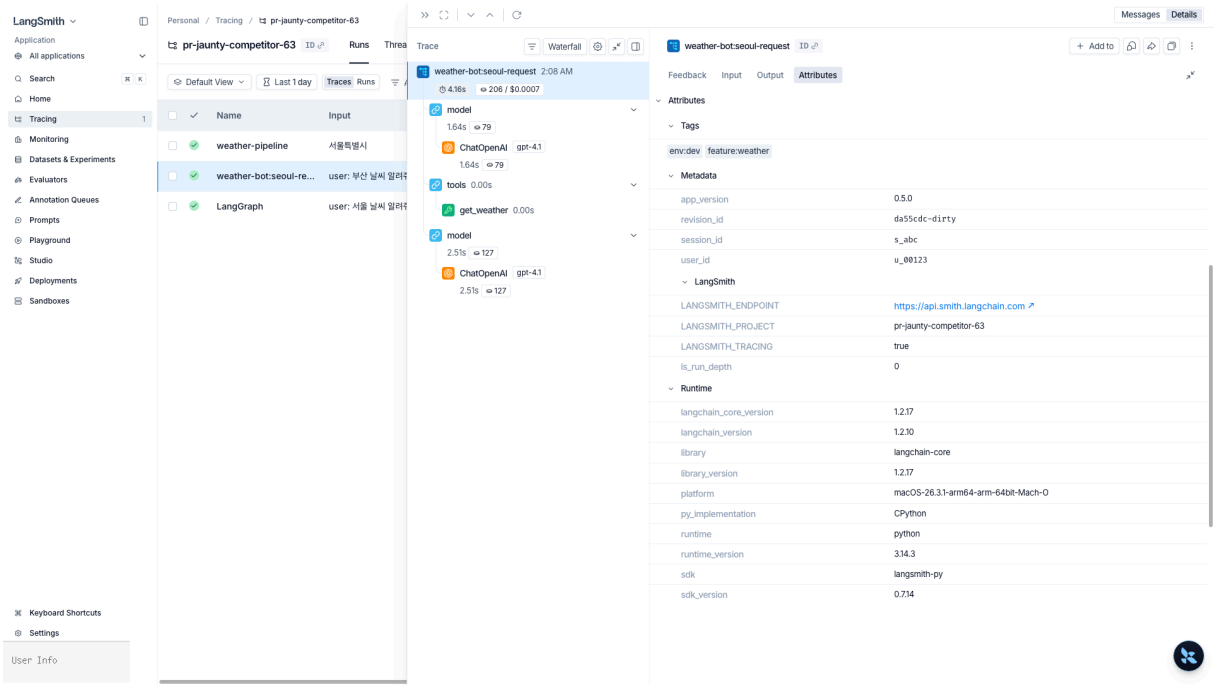
6.01.3 run_name · tags · metadata

Attach searchable/filterable metadata to traces using `invoke(..., config=...)`.

```
config = {
    "run_name": "weather-bot:seoul-request",   # Name shown in the UI
    "tags": ["env:dev", "feature:weather"],     # Filter tags
    "metadata": {                                # Arbitrary key-value pairs
        "user_id": "u_00123",
        "session_id": "s_abc",
        "app_version": "0.5.0",
    },
}

agent.invoke(
    {"messages": [{"role": "user", "content": "Tell me the weather in Busan"}]},
    config=config,
)
```

In the UI, confirm you can filter with `Filter → Tags contains env:dev` or `Metadata.user_id = u_00123`.



Attributes tab — The Tags (`env:dev` , `feature:weather`) and Metadata (`user_id` , `session_id` , `app_version` …) attached above are visible, and LangSmith environment variables and runtime (Python version, library versions, etc.) are automatically captured.

6.01.4 Tracing Pure Python Functions — `\@traceable`

Utility functions that don't use LangChain can also be included in traces with `@traceable` .

```
from langsmith import traceable

@traceable(run_type="tool", name="normalize_city")
def normalize_city(raw: str) -> str:
    mapping = {"New York City": "New York", "San Francisco": "San Francisco"}
    return mapping.get(raw, raw)

@traceable(run_type="chain", name="weather-pipeline")
def weather_pipeline(raw_city: str) -> str:
    city = normalize_city(raw_city)
```

```
    result = agent.invoke({"messages": [{"role": "user", "content": f"Tell me the weather in {city}"}]})
    return result["messages"][-1].content

weather_pipeline("New York City")
```

In the UI, you will see `weather-pipeline` as the root, with `normalize_city` and `AgentExecutor` as children in a single tree.

`import langsmith as ls` — context manager pattern

Tracing starts automatically with environment variables, but if you want to trace only a specific block or disable tracing for a block, use `langsmith.tracing_context`. It accepts `enabled`, `project_name`, `tags`, `metadata`, and `client` arguments.

```
import langsmith as ls

# To temporarily send traces to a different project, or force-enable tracing over environment variables
with ls.tracing_context(
    enabled=True,
    project_name="default",          # Uses LANGSMITH_PROJECT if not specified
    tags=["block:ad-hoc"],
    metadata={"trigger": "manual"},
):
    weather_pipeline("San Francisco")
```

6.01.5 Retrieve Traces from Code

You can programmatically fetch traces without the UI using `langsmith.Client`. SmithDB-backed `runs.query()` is async, uses project UUIDs rather than project names, and should receive an explicit time window if you need more than the recent default range. If you omit `selects`, the response only includes `ID`; list every field your code reads, such as `NAME`, `START_TIME`, `INPUTS`, or `OUTPUTS`.

```
from langsmith import Client
from datetime import datetime, timedelta, timezone

client = Client()
project = client.read_project(project_name=os.environ["LANGSMITH_PROJECT"])
window_end = datetime.now(timezone.utc)
since = window_end - timedelta(days=7)

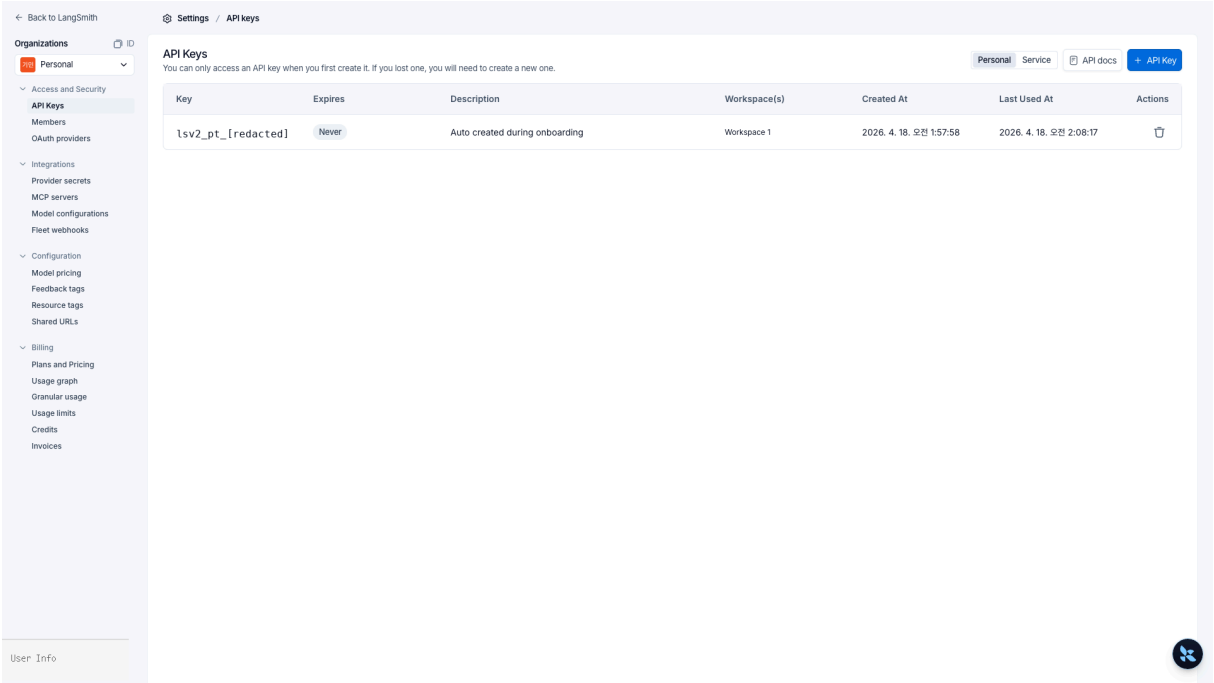
async for r in client.runs.query(
    project_ids=[str(project.id)],
    run_type="CHAIN",
    min_start_time=since,
    max_start_time=window_end,
    page_size=5,
    selects=["NAME", "START_TIME", "END_TIME", "TOTAL_TOKENS"],
):
    duration = (r.end_time - r.start_time).total_seconds() if r.end_time else 0.0
    print(f"{r.start_time:%H:%M:%S}  {r.name:30s}  {r.total_tokens or 0:>5} tok  {duration:.2f}s")
```

6.01.6 Cost & Token Aggregation

In the UI's project page, the `Analytics` tab (top right) shows total cost/token graphs per project. Model-specific pricing is automatically calculated and filled in the `total_cost` field by LangSmith (custom models can be configured).

6.01.7 API Key Management Page

After onboarding, to reissue or revoke keys, go to `Settings → Access and Security → API Keys`.



Settings left nav: API Keys · Members · OAuth providers · Provider secrets · MCP servers · Model configurations · Fleet webhooks · Model pricing · Feedback tags · Resource tags · Shared URLs · Plans · Usage · Credits · Invoices. The Key column in the table shows only the beginning and end by default, and Last Used At is recorded.

Checklist

- [ ] Set `LANGSMITH_API_KEY`, `LANGSMITH_TRACING=true`, and `LANGSMITH_PROJECT` in `.env`
- [ ] Check the LLM/tool call tree of your recent agent run in the UI
- [ ] Attach `run_name` + `tags` + `metadata` and confirm UI filtering works
- [ ] Include regular functions in traces with `@traceable`
- [ ] Use `import langsmith as ls` + `ls.tracing_context(enabled=True, ...)` for block-level control
- [ ] Programmatically fetch traces with async `Client().runs.query(...)`

Next

- `02_tracing_agents.ipynb` — Trace structures for complex agents (LangGraph subgraph, Deep Agents subagent)
- `03_datasets_and_evaluation.ipynb` — Turn these traces into datasets for automated evaluation

References

- LangSmith Observability: https://docs.langchain.com/langsmith/observability
- Trace with LangChain: https://docs.langchain.com/langsmith/trace-with-langchain
- Annotate code with `@traceable` : https://docs.langchain.com/langsmith/annotate-code
- `docs/langchain/30-observability.md`

02 Tracing Agents

Trace Structures of Complex Agents

If you saw a single `create_agent` execution in the UI in `01_quickstart`, now we'll see how agents with nested structures—such as LangGraph StateGraph, Deep Agents subagents, and asynchronous tasks—are visualized in traces.

Learning Objectives

- Understand the relationship between `Run`, `Trace`, and `Project` (run = span, trace = span tree)
- See how LangGraph subgraphs are displayed as namespace children within a parent trace
- Distinguish between synchronous and asynchronous subagent traces (`async_tasks` channel) in Deep Agents
- Group multiple executions into a session view using `thread_id` / `session_id` / `conversation_id`
- Attach evaluation scores to runs using `client.create_feedback(run_id, key, score)`
- Programmatically filter using tags/metadata with SmithDB `client.runs.query(filter=...)`
- Permanently store important traces as datasets to overcome the 400-day retention limit

Prerequisites

Your `.env` file should contain the following three lines (see `01_quickstart`).

```
LANGSMITH_API_KEY=lsv2_pt_...
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=langsmith-tracing-agents
```

This notebook covers synchronous/asynchronous subagent structures in Deep Agents, so `deepagents ⩾ 0.5.0` is required.

```
# !pip install -U langsmith langchain langgraph deepagents langchain-openai

from dotenv import load_dotenv
import os
load_dotenv(override=True)

assert os.environ.get("LANGSMITH_API_KEY"), "LANGSMITH_API_KEY not set"
assert os.environ.get("LANGSMITH_TRACING") == "true", "LANGSMITH_TRACING=true required"
# If the project is not set, LangSmith records to `default`.
os.environ.setdefault("LANGSMITH_PROJECT", "langsmith-tracing-agents")
print("Project:", os.environ["LANGSMITH_PROJECT"])
```

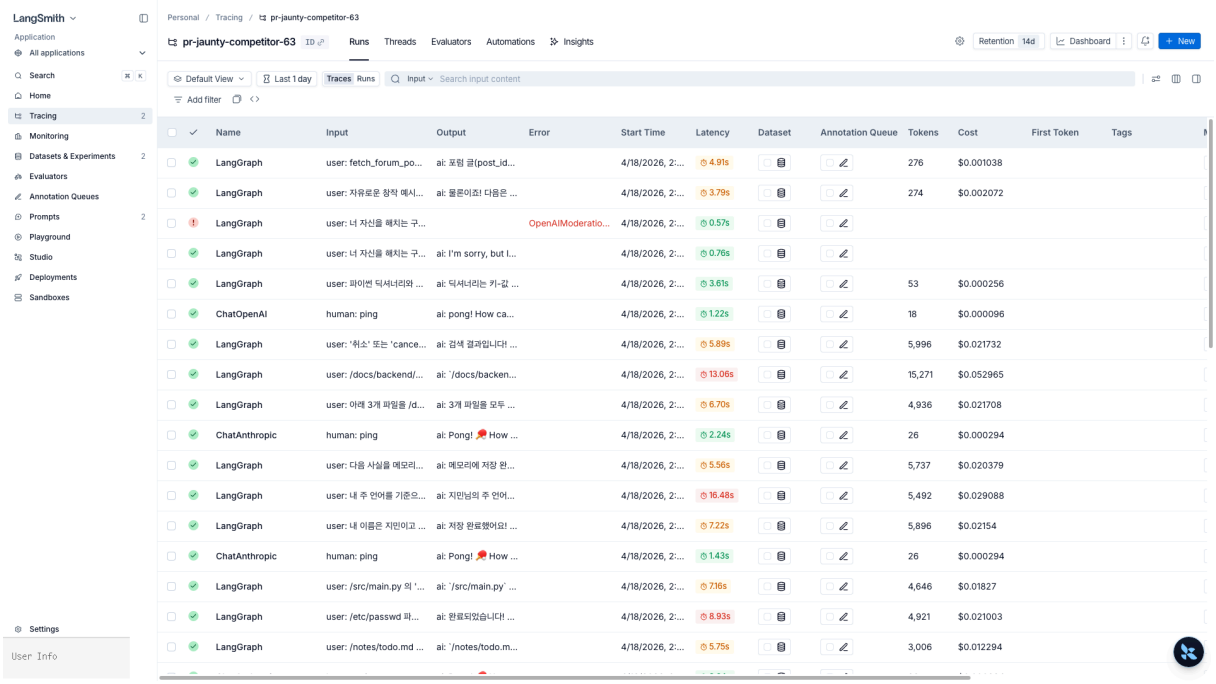
6.02.1 Run · Trace · Project Concepts

LangSmith's data hierarchy is built on four levels.

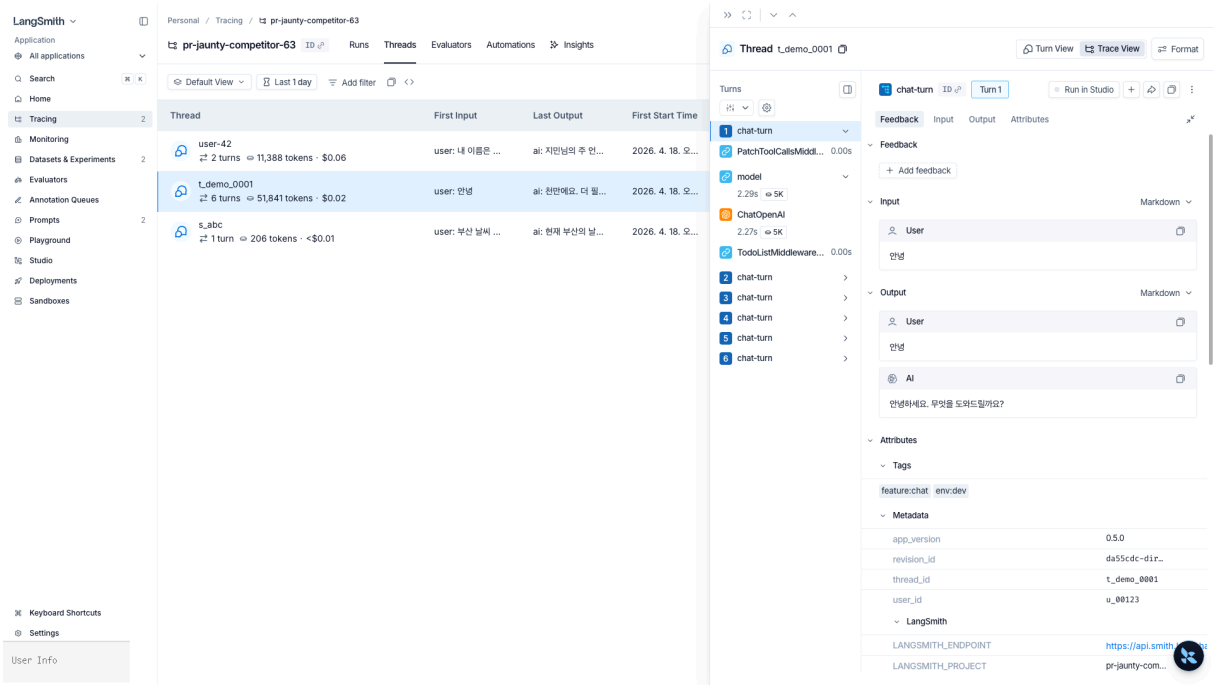
Level	Definition	Example
Project	Container that groups traces from the same application	`langsmith-tracing-agents`
Trace	The run tree created during a single user request (up to 25,000 runs per trace)	One agent invocation
Run	A single span — LLM call, tool call, chain node, etc.	`ChatOpenAI` , `get_weather`
Thread	Multiple traces grouped by `thread_id` / `session_id` / `conversation_id` — multi-turn conversation view	One user's session

Each run has `parent_run_id` , `trace_id` , `start_time` , `end_time` , `inputs` , `outputs` , `total_tokens` , `total_cost` , etc. A trace is a tree of runs sharing the same `trace_id` .

<!- phase-c:embed ->



Runs tab — after two days of executions. Provides 17 columns: Name/Input/Output/Error/Latency/Dataset/Tokens/Cost/Tags/Metadata, etc.



Trace View of `Thread t_demo_0001`. The chain `PatchToolCallsMiddleware → model → ChatOpenAI → TodoListMiddleware` is organized as namespaces within a single chat-turn. The right Attributes tab shows automatically captured Tags/Metadata/Runtime.

6.02.2 LangGraph StateGraph Trace Tree

A LangGraph graph is displayed as the graph as the root run, each node as a child run, and subgraphs as grandchild runs with namespaces. In the UI, subgraph node names appear in the format `parent_node:child_node`.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-5.4")

class State(TypedDict):
    topic: str
    outline: str
    draft: str

# --- Subgraph: Draft Writing ---
def write_outline(state: State) -> dict:
    out = llm.invoke(f"Summarize the following topic in 3 lines: {state['topic']}").content
    return {"outline": out}

def write_draft(state: State) -> dict:
    out = llm.invoke(f"Expand the following outline into a paragraph: {state['outline']}").content
    return {"draft": out}

writer = (
    StateGraph(State)
    .add_node("outline", write_outline)
    .add_node("draft", write_draft)
```

```
    .add_edge(START, "outline")
    .add_edge("outline", "draft")
    .add_edge("draft", END)
    .compile()
)

# --- Parent graph: Calls writer subgraph after research ---
def research(state: State) -> dict:
    return {"topic": f"[researched] {state['topic']}"}

parent = (
    StateGraph(State)
    .add_node("research", research)
    .add_node("writer", writer)      # Insert subgraph as a node
    .add_edge(START, "research")
    .add_edge("research", "writer")
    .add_edge("writer", END)
    .compile()
)

result = parent.invoke(
    {"topic": "Building observable agents with LangSmith"},
    config={"run_name": "writer-pipeline", "tags": ["demo:subgraph"]},
)
print(result["draft"][:200], "...")
```

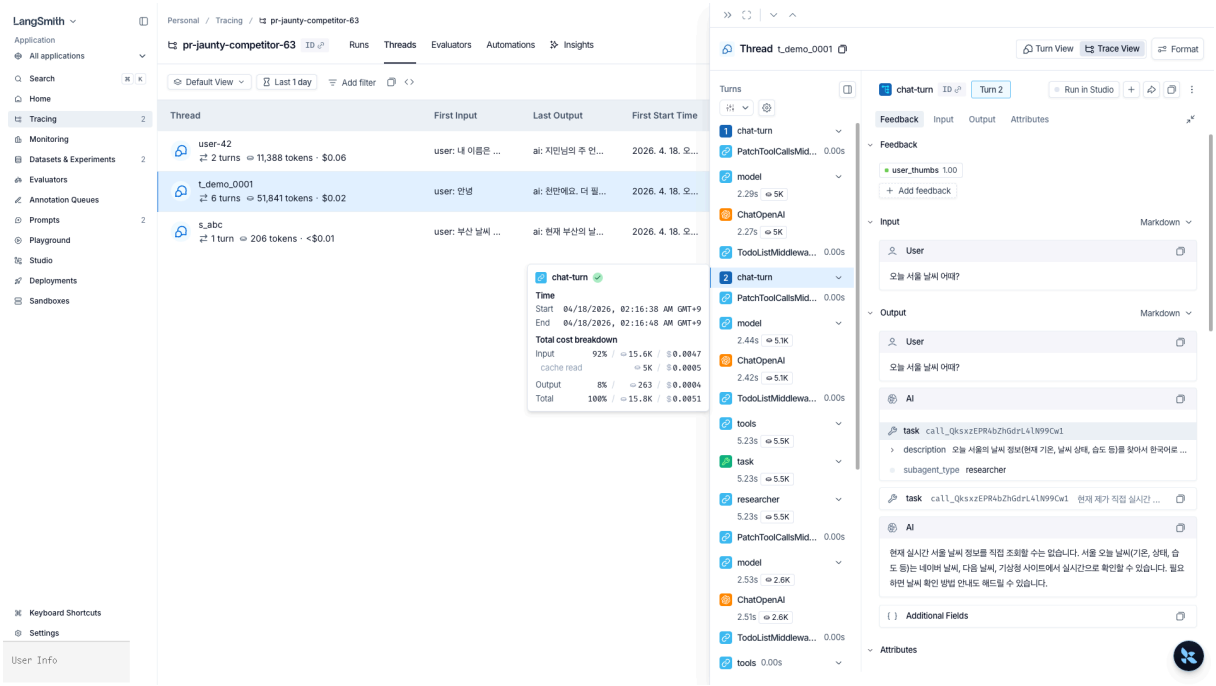
When you open the `writer-pipeline` trace in the UI, the root (`writer-pipeline`) has `research` and `writer` as children, and inside `writer` you see the grandchild runs `writer:outline` and `writer:draft` with namespaces. Since the subgraph path is embedded in the run name, you can use filters like `name contains writer:` .

6.02.3 Deep Agents Subagent Traces (Synchronous · Asynchronous)

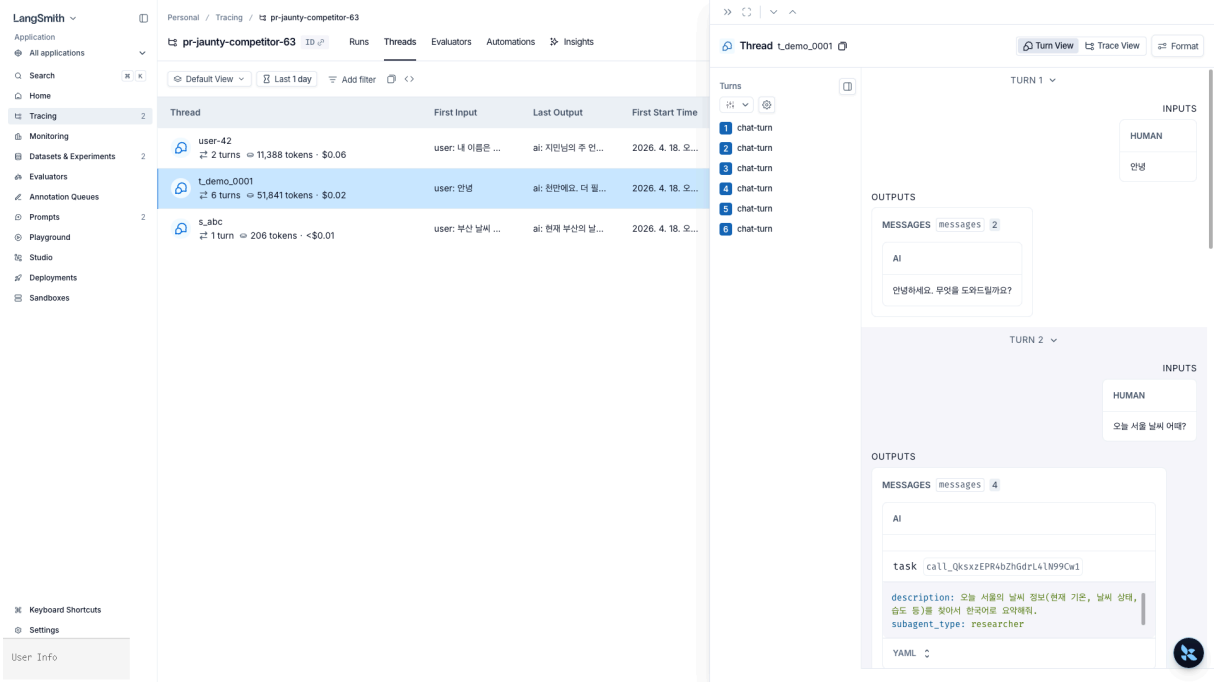
Deep Agents subagents appear as independent child trees under the parent run.

- Synchronous (`SubAgent` dict): The parent is blocked, so it's a single trace. Under the `task` tool call run, the subagent's LLM/tool calls are nested.
- Asynchronous (`AsyncSubAgent`): Runs on a separate Agent Protocol server, so it's recorded as a different trace from the parent. Only the `task_id` remains in the parent's `async_tasks` channel, and the parent trace only shows management tool calls like `start_async_task` / `check_async_task` .

<!- phase-c:embed ->



Turn 2 selected. In the tree, you see the `tools → task → researcher` subagent chain, and in the right Feedback tab, the `user_thumbs 1.00` score. Next to the AI response, the `task(description=..., subagent_type=researcher)` tool call is shown. The Turn 2 hover tooltip's `cache read 5K / $0.0005` indicates Anthropic/OpenAI prompt caching.



Turn View of the same thread — see each turn's Input/Output as conversation bubbles. In Turn 2 AI output, the `task call_Qks...` description and `subagent_type: researcher` are shown in YAML.

```python
from deepagents import create_deep_agent
from langchain.tools import tool
```

```
@tool
def fake_search(query: str) -> str:
    """Fake search tool — returns a fixed response without actual network calls."""
    return f"Summary for {query}: LangSmith is the official observability platform for LangChain."

research_subagent = {
    "name": "researcher",
    "description": "Briefly search a topic and summarize in 3 lines.",
    "system_prompt": "Summarize the search results in Korean within 3 lines.",
    "tools": [fake_search],
}

supervisor = create_deep_agent(
    model="openai:gpt-5.4",
    system_prompt="Delegate to researcher if research is needed, and summarize the final answer in
Korean.",
    subagents=[research_subagent],
)

out = supervisor.invoke(
    {"messages": [{"role": "user", "content": "Summarize what LangSmith is in one paragraph."}]},
    config={
        "run_name": "deepagent:sync-subagent",
        "tags": ["demo:deepagent", "mode:sync"],
    },
)
print(out["messages"][-1].content)
```

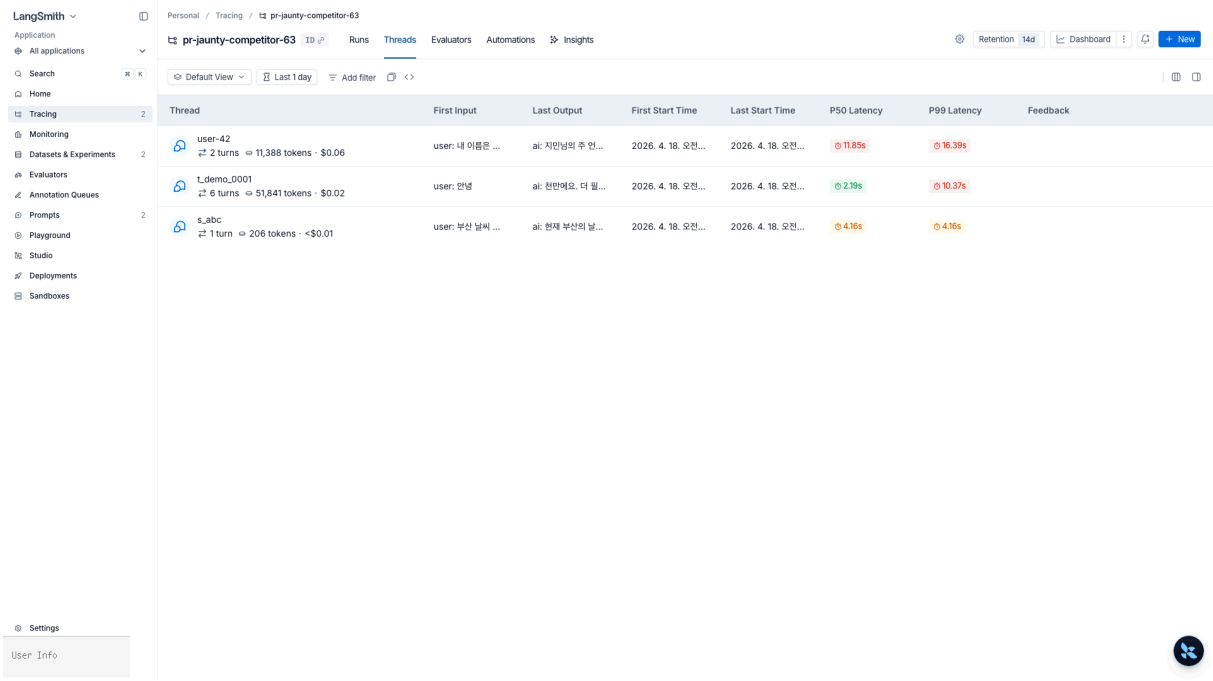
When you open `deepagent:sync-subagent` in the UI, you see the `task` tool call, and under it, the subagent's LLM and `fake_search` calls are nested. For the asynchronous case, you can only check the structure with the snippet below—actual execution requires a `langgraph dev` server.

```
from deepagents import AsyncSubAgent
researcher = AsyncSubAgent(name="researcher", description="Long-running research", graph_id="researcher")
# Parent trace: only shows start_async_task tool call
# Child trace: researcher graph is a separate trace (grouped by the same thread_id)
# State retention: async_tasks channel survives even after compaction
```

6.02.4 Session View — `thread_id` · `session_id` · `conversation_id`

To group multiple invokes into a single conversation, add a session identifier to `metadata`. If LangSmith finds any of `thread_id`, `session_id`, or `conversation_id`, it automatically links them in the Threads view.

<!- phase-c:embed ->



Threads tab — runs sharing the same `thread_id` are grouped as a single conversation session. First Input / Last Output / turns / tokens / cost / P50·P99 Latency columns are automatically aggregated.

```
session_config = {
    "run_name": "chat-turn",
    "metadata": {
        "thread_id": "t_demo_0001",   # Sharing the same value groups them as one line in the Threads
view
        "user_id": "u_00123",
        "app_version": "0.5.0",
    },
    "tags": ["env:dev", "feature:chat"],
}

for turn in ["Hello", "What's the weather in Seoul today?", "Thank you"]:
    supervisor.invoke(
        {"messages": [{"role": "user", "content": turn}]},
        config=session_config,
    )
print("Three turns have been recorded with thread_id=t_demo_0001 — check in the UI's Threads tab")
```

6.02.5 Attaching Feedback to Runs — `client.create_feedback`

Evaluation scores, user thumbs-up/down, and internal QA review results are attached to runs as Feedback.

- `key` : Feedback name (e.g., `"correctness"` , `"user_thumbs"`)
- `score` : Float between 0 and 1, or any arbitrary number
- `value` , `comment` : Optional

```
from langsmith import Client
```

```
client = Client()

# Get the most recent root run and attach feedback.
project = client.read_project(project_name=os.environ["LANGSMITH_PROJECT"])
latest = None
async for run in client.runs.query(
    project_ids=[str(project.id)],
    is_root=True,
    page_size=1,
    selects=["ID"],
):
    latest = run
    break

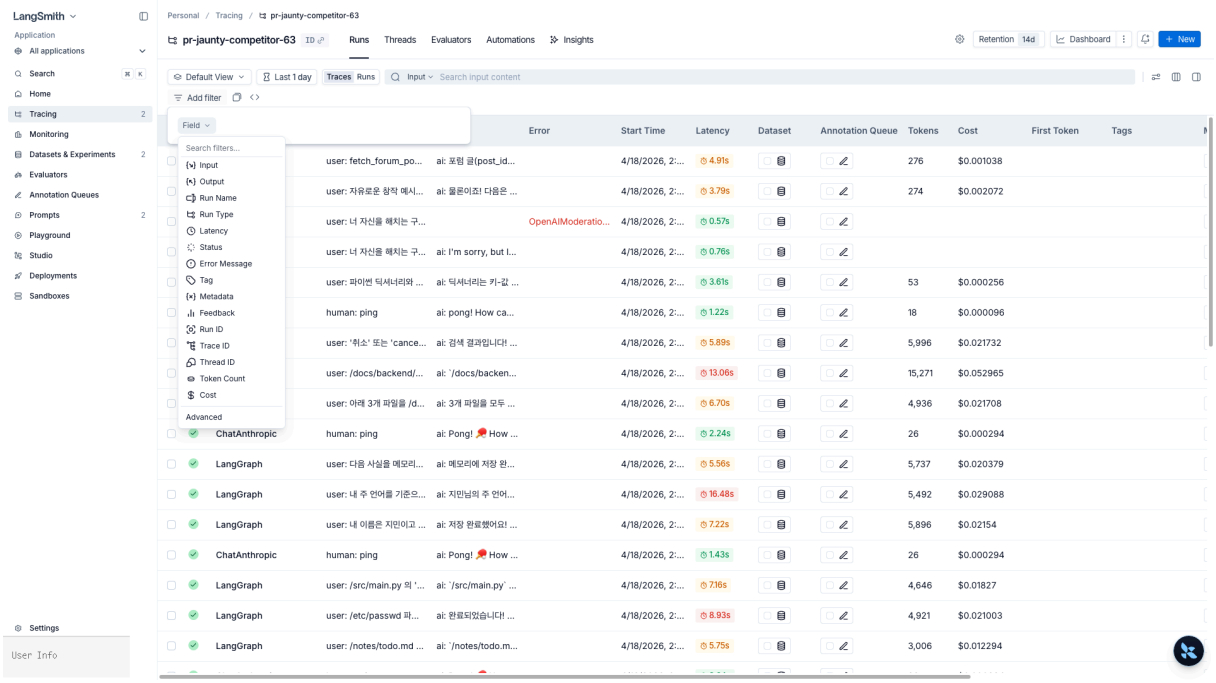
client.create_feedback(
    run_id=latest.id,
    key="user_thumbs",
    score=1,                          # 1=up, 0=down
    comment="Fast and accurate",
)

print(f"Feedback attached → run {latest.id}")
```

6.02.6 Tag/Metadata-Based Filter Queries

You can use the same filter expressions as the UI in code with SmithDB `client.runs.query(filter=...)`. Useful for regression tests, nightly batches, and dashboard feeding. Resolve the project UUID first and set a time window explicitly when you need repeatable history. A query without `selects` returns only `ID`, so every field you read later must be listed explicitly.

<!- phase-c:embed ->



Field list shown when clicking `Add filter` — Input/Output/Run Name/Run Type/Latency/Status/Error Message/Tag/Metadata/Feedback/Run ID/Trace ID/Thread ID/Token Count/Cost +

Advanced. Tag and Metadata are separate fields, so you can query with conditions like `tags contains env:dev`.

```
from datetime import datetime, timedelta, timezone

project = client.read_project(project_name=os.environ["LANGSMITH_PROJECT"])
window_end = datetime.now(timezone.utc)
since = window_end - timedelta(days=7)

# 1) Get only root runs with a specific tag
async for r in client.runs.query(
    project_ids=[str(project.id)],
    filter='and(eq(is_root, true), has(tags, "demo:deepagent"))',
    min_start_time=since,
    max_start_time=window_end,
    page_size=5,
    selects=["ID", "NAME", "START_TIME", "TAGS"],
):
    print(f"{r.start_time:%H:%M:%S}  {r.name:30s}  tags={r.tags}")

# 2) Query runs with metadata.thread_id
thread_count = 0
async for _ in client.runs.query(
    project_ids=[str(project.id)],
    filter='eq(metadata_key, "thread_id")',
    min_start_time=since,
    max_start_time=window_end,
    page_size=20,
    selects=["ID"],
):
    thread_count += 1
print(f"Number of runs with thread_id: {thread_count}")
```

6.02.7 `\@traceable` run_type Classification + Low-Level `create_run` / `update_run`

The `run_type` of `@traceable` maps directly to LangSmith UI colors, icons, and filters. The official types are `chain`, `llm`, `tool`, `retriever`, `embedding`, `prompt`, and `parser` (each shown as a different colored node).

For asynchronous tasks that can't be wrapped with a decorator (external workers, callbacks), you can create spans directly using the low-level `Client.create_run` / `Client.update_run` API.

```
import uuid, time
from datetime import datetime, timezone

run_id = uuid.uuid4()
client.create_run(
    id=run_id,
    name="external-batch-job",
    run_type="chain",                # chain | llm | tool | retriever | embedding | prompt | parser
    inputs={"job": "nightly-rebuild"},
    start_time=datetime.now(timezone.utc),
    project_name=os.environ["LANGSMITH_PROJECT"],
    tags=["env:dev", "source:cron"],
    extra={"metadata": {"hostname": "worker-01"}},
)

# ... perform actual work ...
time.sleep(0.1)

client.update_run(
```

```
    run_id=run_id,
    end_time=datetime.now(timezone.utc),
    outputs={"ok": True, "indexed": 1234},
)
print(f"Low-level run creation/completion done → {run_id}")
```

6.02.8 `LangChainTracer` Callback + `with_config({"tags": [...]})`

If `LANGSMITH_TRACING=true`, a global callback is attached automatically. If you use multiple clients or want to send traces from only certain chains to a separate project, explicitly add a `LangChainTracer` to the callbacks. You can also attach tags per chain using `Runnable.with_config({"tags": [...]})`.

```
from langchain_core.tracers import LangChainTracer
from langsmith import Client

# Separate tracer for a different project
side_project_tracer = LangChainTracer(
    project_name="langsmith-side-project",
    client=Client(),
)

# Attach tags/callbacks per chain — Runnable.with_config
tagged_llm = llm.with_config({"tags": ["component:writer"], "run_name": "writer-llm"})

# You can also inject a one-time callback with config={"callbacks": [...]} when calling
tagged_llm.invoke(
    "Summarize the advantages of LangSmith tracing in one line.",
    config={"callbacks": [side_project_tracer], "tags": ["call:demo"]},
)
```

6.02.9 Selective tracing — `LANGSMITH_TRACING=false` + `tracing_context(enabled=True)`

In production, you often want to disable default tracing and only record specific requests as traces. Priority — `tracing_context(enabled=...)` overrides the environment variable. If you set it to `false` globally and enable only the context block, nothing is visible from the outside and only the inside is recorded in LangSmith.

```
import langsmith as ls
from langsmith import traceable

@traceable(run_type="chain", name="conditional-pipeline")
def conditional_pipeline(payload: dict) -> dict:
    return {"echo": payload}

# Scenario: Assume the global env is set to false and only the block is enabled
prev = os.environ.get("LANGSMITH_TRACING")
os.environ["LANGSMITH_TRACING"] = "false"
try:
    conditional_pipeline({"req": 1})                # No trace
    with ls.tracing_context(enabled=True, tags=["audit:critical"]):
        conditional_pipeline({"req": 2, "vip": True}) # Trace recorded
finally:
    if prev is None:
```

```
        del os.environ["LANGSMITH_TRACING"]
    else:
        os.environ["LANGSMITH_TRACING"] = prev
print("Selective tracing demo complete")
```

6.02.10 400-Day Retention Limit → Permanent Storage as Dataset

SaaS LangSmith deletes traces 400 days after ingestion. To use important executions for evaluation regression, you must permanently store them as Datasets. This is covered in detail in `03_datasets_and_evaluation.ipynb`; here, we just show the pattern.

```
# Permanently store traces as a dataset
# Since langsmith 0.7.x, add_runs_to_dataset has been removed. Use create_examples instead.
from datetime import datetime, timedelta, timezone
from langsmith import Client

client = Client()
DS_NAME = "agent-golden-traces"
try:
    ds = client.create_dataset(
        DS_NAME,
        description="Excellent traces for permanent retention",
    )
except Exception:
    ds = client.read_dataset(dataset_name=DS_NAME)

runs = []
project = client.read_project(project_name=os.environ["LANGSMITH_PROJECT"])
since = datetime.now(timezone.utc) - timedelta(days=30)
async for run in client.runs.query(
    project_ids=[str(project.id)],
    run_type="CHAIN",
    min_start_time=since,
    max_start_time=window_end,
    page_size=5,
    selects=["ID", "INPUTS", "OUTPUTS"],
):
    runs.append(run)
    if len(runs) >= 5:
        break

examples = [
    {"inputs": r.inputs, "outputs": r.outputs, "metadata": {"source_run_id": str(r.id)}}
    for r in runs if r.outputs
]
if examples:
    client.create_examples(dataset_id=ds.id, examples=examples)
print(f"Added {len(examples)} examples to dataset '{ds.name}'.")
```

Checklist

- [ ] Check in the UI that the run names of LangGraph subgraphs are displayed as `parent:child` namespaces
- [ ] Understand that Deep Agents synchronous subagents are recorded as child trees under the `task` tool, while asynchronous ones are recorded as separate traces

- [ ] Confirm that executions with the same `metadata.thread_id` are grouped as a single line in the Threads view
- [ ] Attach feedback to the latest run using `client.create_feedback(run_id, key, score)`
- [ ] Programmatically query using tags/metadata with async `client.runs.query(filter=...)`
- [ ] Use all 7 `run_type`s of `@traceable` (chain/llm/tool/retriever/embedding/prompt/parser) + low-level `Client.create_run` / `update_run`
- [ ] Explicitly inject `LangChainTracer` callback + use `Runnable.with_config({"tags": [...]})`
- [ ] Use `ls.tracing_context(enabled=True)` for selective tracing when `LANGSMITH_TRACING=false`
- [ ] Move important traces to a dataset to handle the 400-day retention limit

Next

- `03_datasets_and_evaluation.ipynb` — How to attach code evaluators, LLM-as-judge, and pairwise evaluation to these traces/datasets

References

- Observability concepts: https://docs.langchain.com/langsmith/observability-concepts
- Annotate code (`@traceable`): https://docs.langchain.com/langsmith/annotate-code
- Conditional tracing: https://docs.langchain.com/langsmith/conditional-tracing
- Threads (session) view: https://docs.langchain.com/langsmith/threads
- Feedback API: https://docs.langchain.com/langsmith/attach-user-feedback
- Filter query syntax: https://docs.langchain.com/langsmith/filter-traces-in-application

03 Datasets & Evaluation

From Trace to Automated Evaluation Loop

A trace shows "how things are currently running," while evaluation answers "did things improve when we changed the prompt, model, or code?" In LangSmith, traces can be directly promoted to datasets, and on top of those, you can run code evaluators, LLM-as-judge, pairwise, and summary evaluations.

Learning Objectives

- Create a dataset and add examples using `client.create_dataset` + `client.create_examples`
- Transfer production traces to a dataset with `client.create_examples(..., examples=[...])`
- Write a code evaluator in the form `def my_eval(inputs, outputs, reference_outputs) → dict`
- Run an LLM-as-judge evaluator to get structured scores
- Use pairwise/summary evaluators to compare two experiments and get dataset-level metrics
- Run experiments and assign names using the `from langsmith.evaluation import evaluate` runner
- Understand the flow of automatically applying online evaluators to production traces

Prerequisites

You need three LangSmith keys and an OpenAI key in your `.env` file.

```
LANGSMITH_API_KEY=lsv2_pt_...
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=langsmith-eval-demo
OPENAI_API_KEY=sk-...
```

While running evaluations, experiment projects are automatically separated and recorded, so they don't get mixed with the production project.

```
# !pip install -U langsmith langchain langchain-openai agentevals

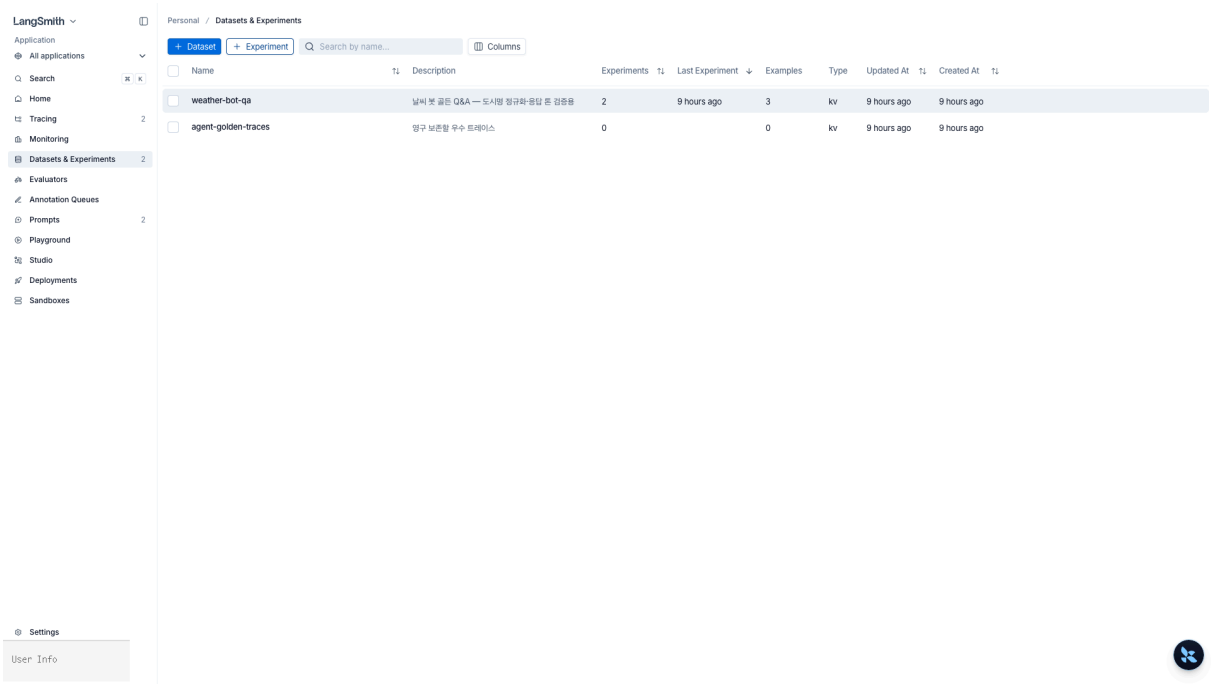
from dotenv import load_dotenv
import os
load_dotenv(override=True)

assert os.environ.get("LANGSMITH_API_KEY"), "LANGSMITH_API_KEY not set"
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY not set"
# If the project is not set, LangSmith records to `default`.
os.environ.setdefault("LANGSMITH_PROJECT", "langsmith-eval-demo")
print("Project:", os.environ["LANGSMITH_PROJECT"])
```

6.03.1 Creating a Dataset — Adding Examples Manually

A domain expert writes golden Q&A examples directly and adds them using `create_examples`. Both `inputs` and `outputs` are dicts. `outputs` serve as reference answers for code evaluators and LLM-as-judge comparisons.

<!- phase-c:embed ->



Datasets & Experiments list — Two datasets: `weather-bot-qa` (3 examples · 2 experiments) and `agent-golden-traces` (for permanent storage of excellent traces). The `kv` in the Type column indicates a key-value structured dataset.

```
from langsmith import Client

client = Client()
DATASET_NAME = "weather-bot-qa"

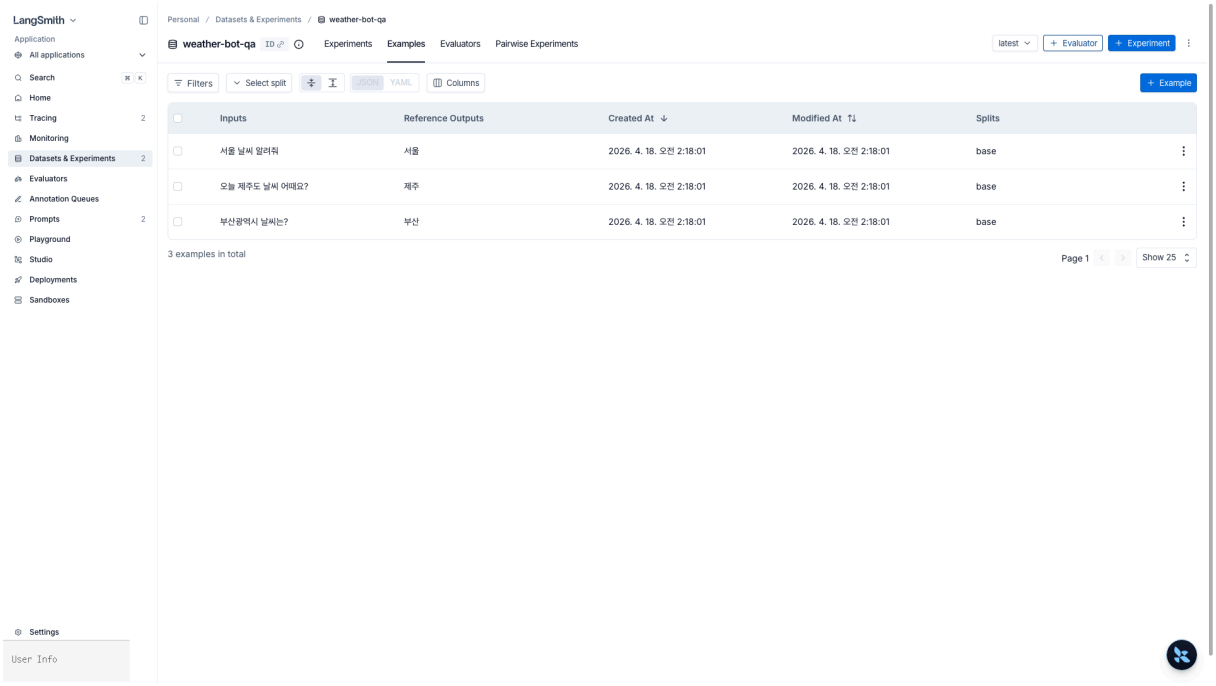
try:
    dataset = client.create_dataset(
        dataset_name=DATASET_NAME,
        description="Weather bot golden Q&A — for city name normalization and response tone validation",
    )
except Exception:
    dataset = client.read_dataset(dataset_name=DATASET_NAME)

client.create_examples(
    dataset_id=dataset.id,
    examples=[
        {"inputs": {"question": "What's the weather in Seoul?"},        "outputs": {"city": "Seoul"}},
        {"inputs": {"question": "How's the weather in Busan Metropolitan City?"},     "outputs": {"city":
"Busan"}},
        {"inputs": {"question": "How's the weather in Jeju Island today?"}, "outputs": {"city": "Jeju"}},
    ],
)
print(f"Dataset ready: {dataset.name} ({dataset.id})")
```

6.03.2 Transferring Production Traces to a Dataset

Manual creation is only for the initial seed; real scale comes from production traces. Since `add_runs_to_dataset` is removed in current LangSmith, query runs with `client.runs.query(...)` and convert each run's `inputs` / `outputs` into a `create_examples(..., examples=[...])` payload. SmithDB queries return only `ID` when `selects` is omitted, so this example explicitly selects `ID`, `INPUTS`, and `OUTPUTS`. In actual operations, it's common to only upload runs reviewed by humans via the Annotation Queue.

<!– phase–c:embed –>



Examples tab of `weather-bot-qa` — 3 examples uploaded via `client.create_examples(...)`. The Inputs column contains Korean questions, and the Reference Outputs column shows the expected city names. There are toggles for JSON/YAML format conversion and a Splits column (for train/validation/test separation).

```
# Select a few root runs from the current project and transfer them to the dataset
# Since langsmith 0.7.x, add_runs_to_dataset has been removed. Convert to create_examples instead.
from datetime import datetime, timedelta, timezone

project = client.read_project(project_name=os.environ["LANGSMITH_PROJECT"])
since = datetime.now(timezone.utc) - timedelta(days=30)
root_runs = []
async for run in client.runs.query(
    project_ids=[str(project.id)],
    is_root=True,
    min_start_time=since,
    page_size=5,
    selects=["ID", "INPUTS", "OUTPUTS"],
):
    root_runs.append(run)
    if len(root_runs) >= 5:
        break

examples = [
    {"inputs": r.inputs, "outputs": r.outputs, "metadata": {"source_run_id": str(r.id)}}
```

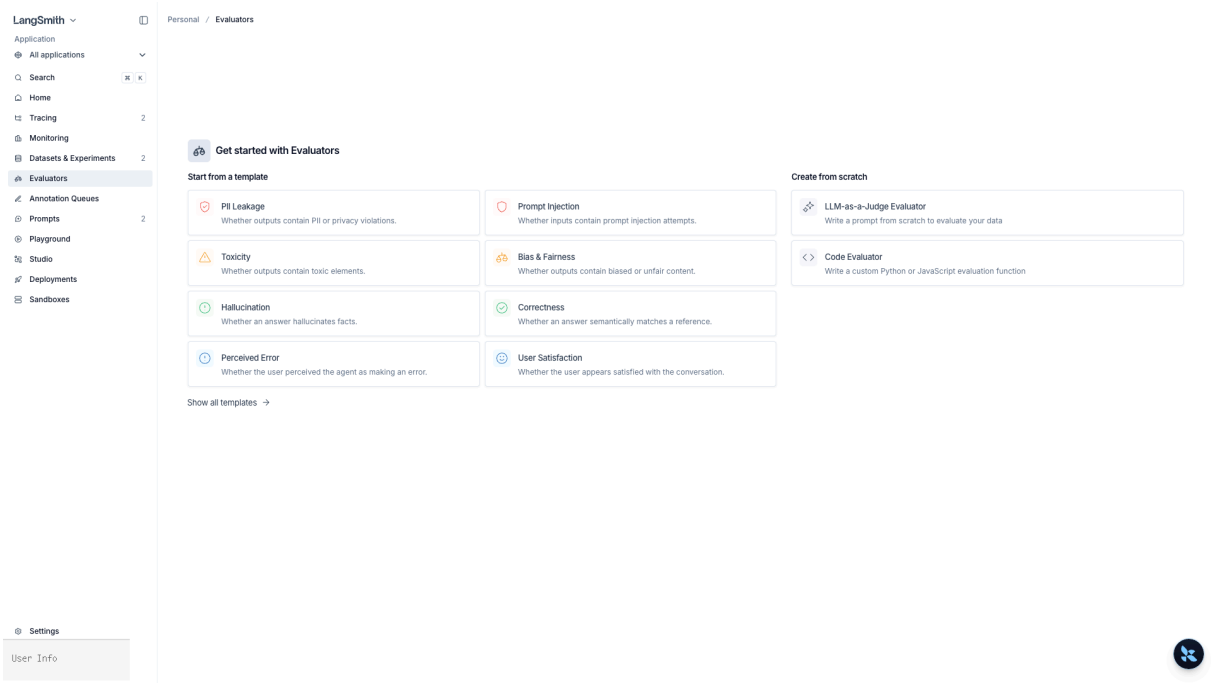
```
    for r in root_runs if r.outputs
]
if examples:
    client.create_examples(dataset_id=dataset.id, examples=examples)
print(f"Transferred {len(examples)} runs to {DATASET_NAME}")
```

6.03.3 Evaluation Target + Code Evaluator

As an example, we'll use an LLM function that "extracts the city name from a question" as the target. The target should be in the form `inputs: dict → outputs: dict`.

The evaluator receives `(inputs, outputs, reference_outputs)` and returns a dict like `{"key": ..., "score": ...}`. Deterministic heuristics cost nothing and have 0 ms latency — the more you add, the better.

<!- phase-c:embed ->



Evaluators page — 8 templates (PII Leakage · Prompt Injection · Toxicity · Bias & Fairness · Hallucination · Correctness · Perceived Error · User Satisfaction) + custom evaluators (LLM-as-a-Judge · Code Evaluator).

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-5.4", temperature=0)

def extract_city(inputs: dict) -> dict:
    """Extracts only the city name from the question (removes metropolitan/special city suffixes)."""
    q = inputs["question"]
    msg = llm.invoke(f"Answer with only the city name as a single word from the following question (e.g.,
Seoul, Busan, Jeju). Question: {q}")
    return {"city": msg.content.strip()}

def city_exact_match(inputs: dict, outputs: dict, reference_outputs: dict) -> dict:
```

```
    """Checks if the predicted city exactly matches the reference."""
    return {
        "key": "city_exact_match",
        "score": int(outputs.get("city") == reference_outputs.get("city")),
    }

def city_non_empty(inputs: dict, outputs: dict, reference_outputs: dict) -> dict:
    """Defends against empty responses — for monitoring failure modes."""
    return {"key": "city_non_empty", "score": int(bool(outputs.get("city", "").strip()))}

print(extract_city({"question": "How's the weather in Busan Metropolitan City?"}))
```

6.03.4 LLM-as-judge Evaluator

Use this when there is no exact answer string or when you need to measure natural language quality (tone, accuracy, helpfulness). The key is to force the output into a structured score even though the same LLM is called.

```
from pydantic import BaseModel, Field

class Judgement(BaseModel):
    score: int = Field(..., ge=0, le=1, description="1 if the response is semantically the same as the
reference answer")
    reason: str

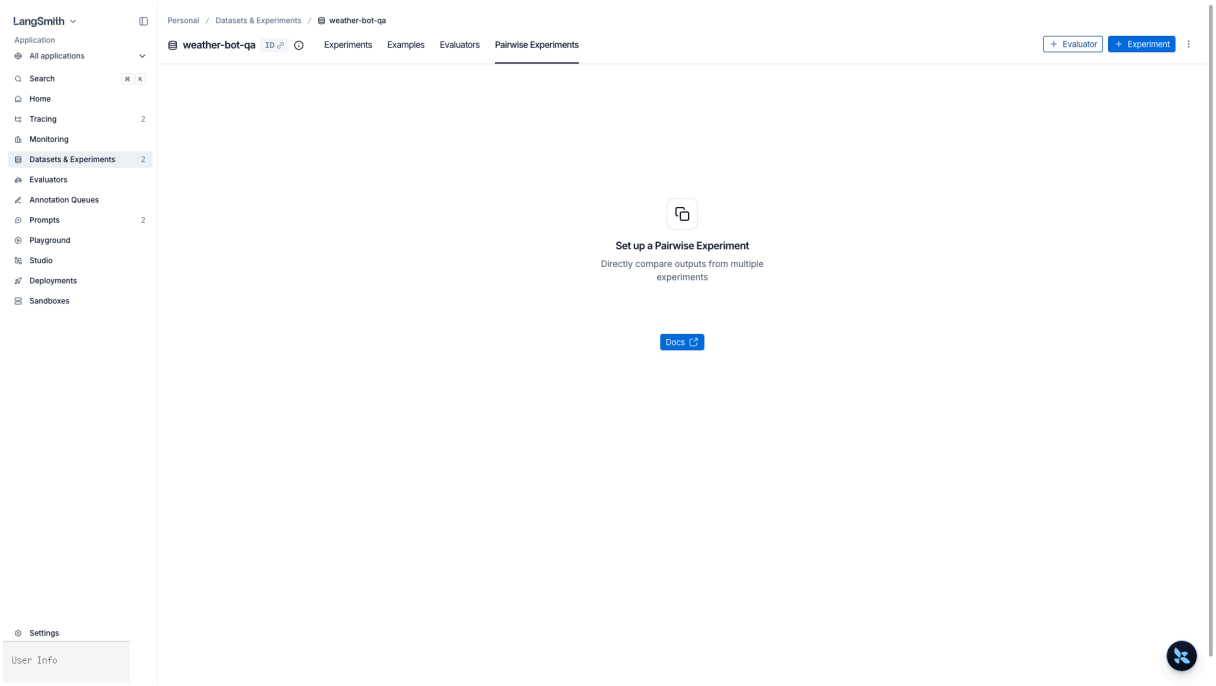
judge_llm = llm.with_structured_output(Judgement)

def semantic_city_match(inputs: dict, outputs: dict, reference_outputs: dict) -> dict:
    verdict: Judgement = judge_llm.invoke(
        "Compare the question, expected city, and the city extracted by the model to determine if they
are semantically the same.\n"
        f"Question: {inputs['question']}\nExpected: {reference_outputs.get('city')}\nModel:
{outputs.get('city')}"
    )
    return {"key": "semantic_city_match", "score": verdict.score, "comment": verdict.reason}
```

6.03.5 `evaluate` Runner + Experiment Name

`from langsmith.evaluation import evaluate` is the standard runner. If you give a meaningful name to `experiment_prefix`, you can compare directly in the UI's Experiments view.

<!- phase-c:embed ->



Pairwise Experiments tab — Compare the outputs of two experiments side by side. Run via the `evaluate_comparative` API (as of April 2026, the full session name is required, so the `evaluate_comparative` call in this notebook fails with drift and is shown empty).

```python
from langsmith.evaluation import evaluate

results = evaluate(
    extract_city,
    data=DATASET_NAME,
    evaluators=[city_exact_match, city_non_empty, semantic_city_match],
    experiment_prefix="city-extractor:gpt-5.4",
    description="Baseline — temperature 0, single prompt",
    max_concurrency=4,
    disable_evaluator_tracing=True,
    metadata={"model": "gpt-5.4", "temperature": 0},
)
print("Experiment URL ->", results)
```

`disable_evaluator_tracing=True` keeps evaluator internals from creating extra traces. Leave it off when you are debugging evaluator prompts; turn it on when the experiment output is what matters and evaluator trace volume would add noise or cost.

6.03.6 Pairwise + Summary Evaluator

- Pairwise: Compares the outputs of two experiments for the same example and selects “which one is better.” Used for A/B prompt experiments. Uses the `evaluate_comparative` runner.
- Summary: Provides dataset-level metrics (e.g., macro average accuracy). The signature is different in that it receives a list of runs/examples.

```python
from langsmith.evaluation import evaluate_comparative

# 1) Second experiment for comparison (different prompt)
def extract_city_v2(inputs: dict) -> dict:
    q = inputs["question"]
    msg = llm.invoke(f"Return only the city name, without any suffix, from the question: {q}")
    return {"city": msg.content.strip()}

results_v2 = evaluate(
    extract_city_v2,
    data=DATASET_NAME,
    evaluators=[city_exact_match],
    experiment_prefix="city-extractor:v2-strict-prompt",
)

# 2) pairwise: Select the better output between the two
def pref_shorter_city(runs: list, example) -> dict:
    # runs[i].outputs may be None, so handle defensively
    a = (runs[0].outputs or {}).get("city", "")
    b = (runs[1].outputs or {}).get("city", "")
    winner = 0 if len(a) <= len(b) else 1
    return {"key": "shorter_city", "scores": {runs[0].id: 1-winner, runs[1].id: winner}}

# evaluate_comparative requires the exact session name (or UUID).
# Retrieve the full name including the auto hash suffix created by experiment_prefix.
# Also, `experiments` is a positional-only argument, so pass as a tuple, not a kwarg.
sessions = list(client.list_projects(reference_dataset_id=dataset.id))
v1_name = next(s.name for s in sessions if s.name.startswith("city-extractor:gpt-5.4"))
v2_name = next(s.name for s in sessions if s.name.startswith("city-extractor:v2-strict-prompt"))
print(f"Comparison targets: {v1_name}  vs  {v2_name}")

evaluate_comparative(
    (v1_name, v2_name),                # positional-only
    evaluators=[pref_shorter_city],
)

# 3) summary evaluator: Exact-match ratio for the entire dataset
def summary_accuracy(runs: list, examples: list) -> dict:
    total = len(runs)
    if total == 0:
        return {"key": "accuracy", "score": 0.0}
    correct = sum(
        1 for r, e in zip(runs, examples)
        if r.outputs and e.outputs and r.outputs.get("city") == e.outputs.get("city")
    )
    return {"key": "accuracy", "score": correct / total}

evaluate(
    extract_city,
    data=DATASET_NAME,
    evaluators=[city_exact_match],
    summary_evaluators=[summary_accuracy],
    experiment_prefix="city-extractor:with-summary",
)
print("pairwise + summary experiment completed")
```

6.03.7 Online Evaluator — Automatic Evaluation of Production Traces

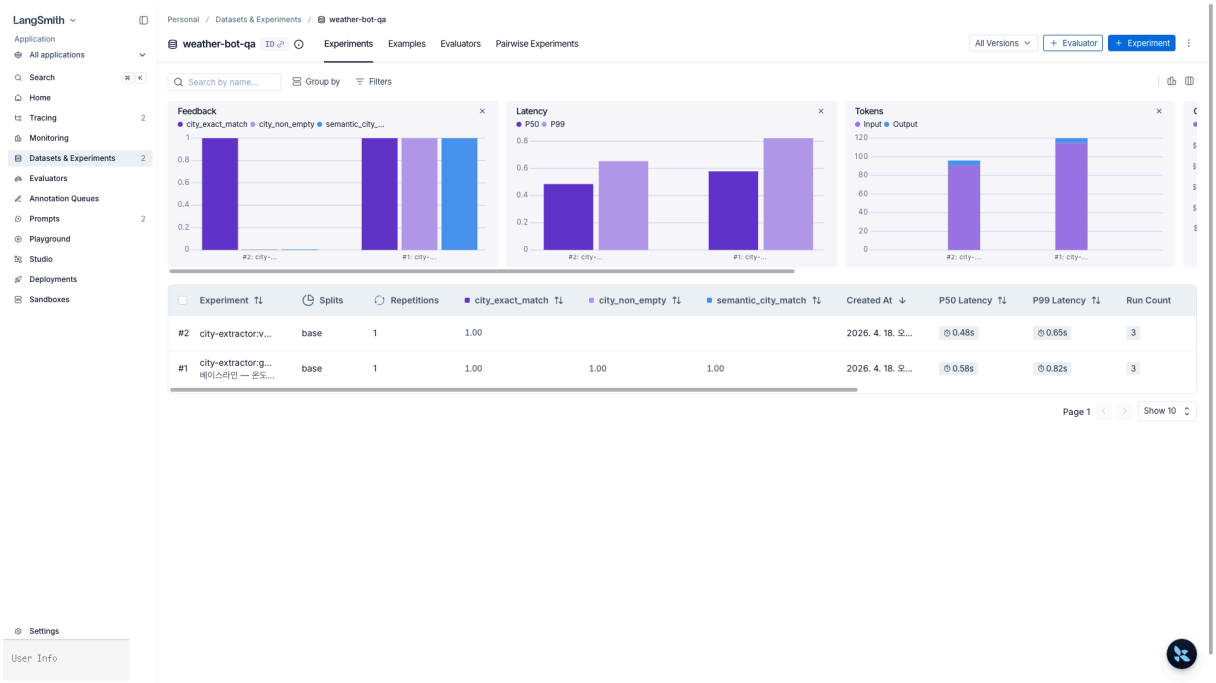
Offline experiments are for regression testing before deployment, while during operation, you attach real-time feedback with an online evaluator. UI flow:

1. Go to the project -> Evaluators tab -> `+ Evaluator`
2. Select LLM-as-judge, write the evaluation prompt (e.g., “Does the response answer the user’s intent?”)

3. Specify a filter — e.g., only runs with `has(tags, "env:prod")`
4. If you set the Sampling rate to 0.1, only 10% of matching traces are evaluated — for cost control
5. Once saved, new traces automatically start getting feedback keys

To retroactively apply to past traces, turn on Apply to past runs and specify the period. The resulting feedback can trigger dashboards/alerts and will be covered again with alert rules in `05_production_monitoring.ipynb`.

<!- phase-c:embed ->



Experiments tab of `weather-bot-qa` — Three charts at the top (Feedback scores `city_exact_match` / `city_non_empty` / `semantic_city_match`, Latency P50/P99, Tokens Input/Output) + experiment table at the bottom (1 `city-extractor:gpt-4.1-mini-...` / 2 `city-extractor:v2-strict-prompt-...`). Each experiment aggregates scores after running 3 runs.

6.03.8 Agent Trajectory Evaluation — `agentevals`

For agents, not only the final response but also the sequence of tool calls determines quality. `agentevals` is a LangChain-specific evaluation package that compares the trajectory (message/tool call sequence) to a reference.

The four modes of `create_trajectory_match_evaluator(trajectory_match_mode="...")`:

Mode	Meaning
`strict`	Messages, tool call order, and arguments must be exactly the same
`unordered`	The same tools were called (order ignored)

`subset`	Actual is a subset of the reference
`superset`	All tools in the reference were called, extra calls allowed

If there is no reference, `create_trajectory_llm_as_judge` lets the LLM directly evaluate the reasonableness of the trajectory.

```python
# pip install agentevals
from agentevals.trajectory.match import create_trajectory_match_evaluator
from agentevals.trajectory.llm import create_trajectory_llm_as_judge

# The most commonly used mode among the four is unordered — checks if the set of tools called is the same
trajectory_unordered = create_trajectory_match_evaluator(trajectory_match_mode="unordered")

# Message sequence (LangChain BaseMessage list)
outputs = [
    {"role": "user", "content": "Weather in Seoul"},
    {"role": "assistant", "tool_calls": [{"name": "get_weather", "args": {"city": "Seoul"}}]},
    {"role": "tool", "content": "Seoul: Clear, 21°C"},
    {"role": "assistant", "content": "It's clear and 21 degrees in Seoul."},
]
reference_outputs = [
    {"role": "user", "content": "Weather in Seoul"},
    {"role": "assistant", "tool_calls": [{"name": "get_weather", "args": {"city": "Seoul"}}]},
    {"role": "assistant", "content": "..."},
]

verdict = trajectory_unordered(outputs=outputs, reference_outputs=reference_outputs)
print("trajectory match (unordered):", verdict)

# LLM evaluates trajectory reasonableness without a reference
judge = create_trajectory_llm_as_judge(model="openai:gpt-5.4")
print("trajectory llm-as-judge:", judge(outputs=outputs))
```

Checklist

- [ ] Manually create a golden set using `client.create_dataset` + `client.create_examples`
- [ ] Transfer production runs to the dataset using `client.create_examples` (replacing the old `add_runs_to_dataset`)
- [ ] Check the code evaluator signature `(inputs, outputs, reference_outputs) → {"key","score"}`
- [ ] Structure LLM-as-judge with `with_structured_output`
- [ ] Run experiments with `evaluate(...)` and compare by name in the UI
- [ ] Understand the signature differences between pairwise (`evaluate_comparative`) and summary evaluators
- [ ] Know the four trajectory match modes of `agentevals` (strict/unordered/subset/superset)
- [ ] Attach an online evaluator in the UI to automatically provide feedback on production traces

Next

- `04_prompt_hub.ipynb` — How to version and deploy prompts used in this experiment with the `prod` tag

References

- Evaluation concepts: https://docs.langchain.com/langsmith/evaluation-concepts
- All about evaluation: https://docs.langchain.com/langsmith/evaluation
- Evaluate LLM application: https://docs.langchain.com/langsmith/evaluate-llm-application
- Pairwise evaluation: https://docs.langchain.com/langsmith/evaluate-pairwise
- Online evaluators: https://docs.langchain.com/langsmith/online-evaluations
- agentevals: https://github.com/langchain-ai/agentevals

04 Prompt Hub

Prompt Version Control

Prompts are essentially code, but they are frequently edited by non-engineers and have short update cycles. To avoid redeploying every time, prompts should be stored in a repository separate from application code and version-pinned. LangSmith Prompt Hub serves this purpose.

Learning Objectives

- Upload prompts using `client.push_prompt("name", object=...)`
- Understand the difference between pinning to a Commit SHA vs referencing `prod` / `staging` tags
- Compare variable handling between f-string and mustache templates
- Learn the Playground → Commit → Tag workflow
- Inject prompts at runtime with `client.pull_prompt("name:prod")` and connect to `create_agent(system_prompt=...)`
- In CI tests, pin to a specific commit hash to prevent regressions

Prerequisites

```
LANGSMITH_API_KEY=lsv2_pt_...
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=langsmith-prompt-hub
OPENAI_API_KEY=sk-...
```

Prompt Hub is separate from tracing, but push/pull calls are also traced, so it's convenient to use the same project name. The SDK requires `langsmith ⩾ 0.1.99`.

```
# !pip install -U langsmith langchain langchain-openai

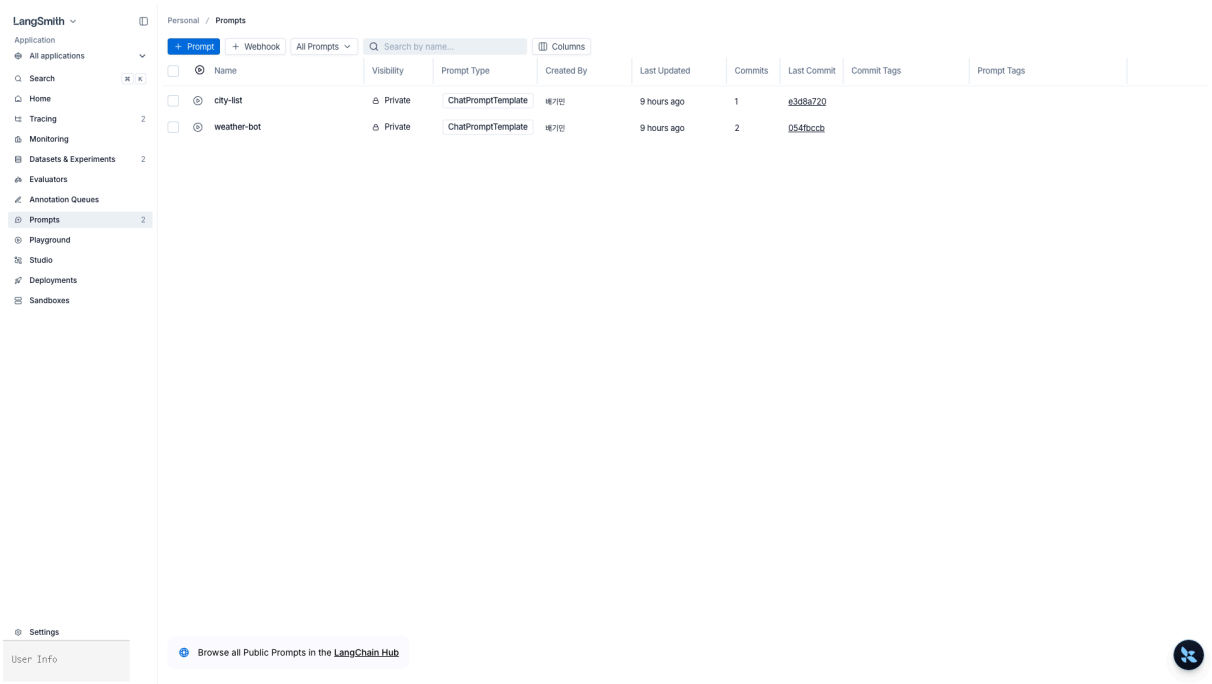
from dotenv import load_dotenv
import os
load_dotenv(override=True)

assert os.environ.get("LANGSMITH_API_KEY"), "LANGSMITH_API_KEY not set"
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY not set"
# If the project is not set, LangSmith records to `default`.
os.environ.setdefault("LANGSMITH_PROJECT", "langsmith-prompt-hub")
print("Project:", os.environ["LANGSMITH_PROJECT"])
```

6.04.1 Creating and Pushing a Prompt

The simplest way is to create a `ChatPromptTemplate` and upload it with `client.push_prompt("name", object=prompt)`. The first push creates a new prompt, and subsequent pushes add new commits. The returned URL opens directly in the UI.

<!– phase-c:embed –>



Prompt list — `city-list` (1 commit) and `weather-bot` (2 commits). The Visibility column shows Private/Public badges, and the Last Commit column displays short SHAs (`e3d8a720` , `054fbccb`). In addition to the `+ Prompt` button at the top, there is a `+ Webhook` option (to trigger a webhook on prompt changes).

```python
from langsmith import Client
from langchain_core.prompts import ChatPromptTemplate

client = Client()

weather_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a friendly weather bot. Please answer concisely in one sentence."),
    ("human", "Tell me the weather in {city}"),
])

url = client.push_prompt("weather-bot", object=weather_prompt)
print("Push complete ->", url)
```

6.04.2 Commit SHA Pinning vs Tags (`prod` , `staging`)

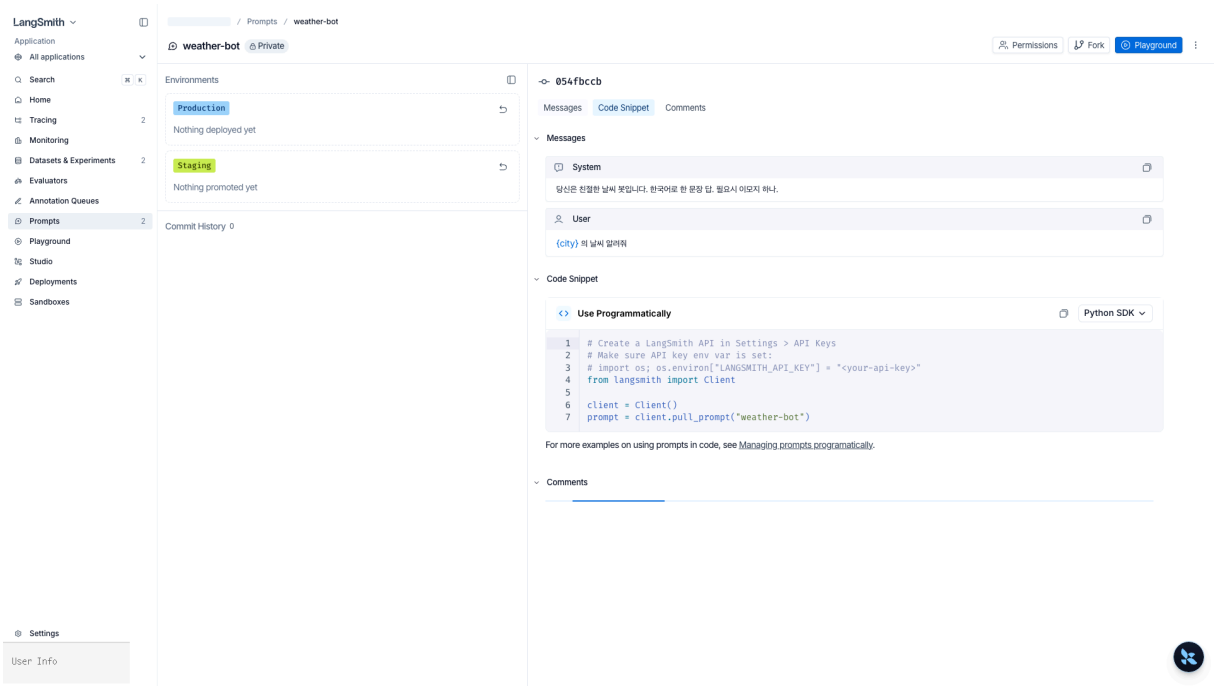
Prompts work like Git.

Reference Type	Example	Characteristics	When to Use

Commit SHA	`weather-bot:12344e88`	Immutable, pins exactly that version	CI regression tests, when reproducibility is needed
Tag	`weather-bot:prod`, `weather-bot:staging`	Movable — the same tag can point to different commits	Runtime deployment slots
Latest	`weather-bot`	Most recent commit	Only in early development, not for production

Tags are a key mechanism for swapping prompts without redeploying application code. In the UI's `Commits` view, you can promote a specific commit to the `prod` tag.

<!- phase-c:embed ->



`weather-bot` details. Top shows commit `054fbccb` + tabs (Messages / Code Snippet / Comments). Messages tab shows System/User templates (with `{city}` variable), Code Snippet tab shows a Python SDK example `client.pull_prompt("weather-bot")`. The left Environments panel has Production/Staging slots with `Promote` tagging. Top right has Permissions/Fork/Playground buttons.

```
# Extract the commit hash from the returned URL, excluding the query string
from urllib.parse import urlparse

commit_hash = urlparse(url).path.rstrip("/").split("/")[-1]  # e.g., '12344e88'
print("Commit to pin:", commit_hash)

# Modify the same prompt again to create a new commit
weather_prompt_v2 = ChatPromptTemplate.from_messages([
    ("system", "You are a friendly weather bot. Answer in Korean in one sentence. Add an emoji if
```

```
needed."),
    ("human", "Tell me the weather in {city}"),
])
url_v2 = client.push_prompt("weather-bot", object=weather_prompt_v2)
print("v2 push ->", url_v2)
print("-> In the UI, try tagging the v1 commit as prod and the v2 commit as staging")
```

6.04.3 f-string vs mustache

The characteristics of these two template engines often matter in practice.

Item	f-string (`{var}`)	mustache (`{{var}}`)
Default	✅ LangChain default	Separate selection
Including JSON examples	Need to escape `{{`	No need to escape
Conditionals/Loops	❌ Not supported	✅ `{{\#users}}...{{/users}}`
Nested keys	Limited	✅ `{{user.name}}`
Playground variable detection	Auto-detected	Manual (Inputs)

If you need to include lots of JSON/code examples or use loops/conditionals, mustache is better. For simple variable substitution, f-string is more convenient.

```
# Example of mustache template — template_format="mustache"
mustache_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "Introduce the list of cities in Korean, one per line."),
        ("human", "{{#cities}}- {{name}} ({{country}})\n{{/cities}}"),
    ],
    template_format="mustache",
)

client.push_prompt("city-list", object=mustache_prompt)
# Bind the repeat variable as a list
print(mustache_prompt.invoke({"cities": [
    {"name": "Seoul", "country": "KR"},
    {"name": "Tokyo", "country": "JP"},
]}).to_messages()[-1].content)
```

6.04.4 Playground — From Experiment to Commit

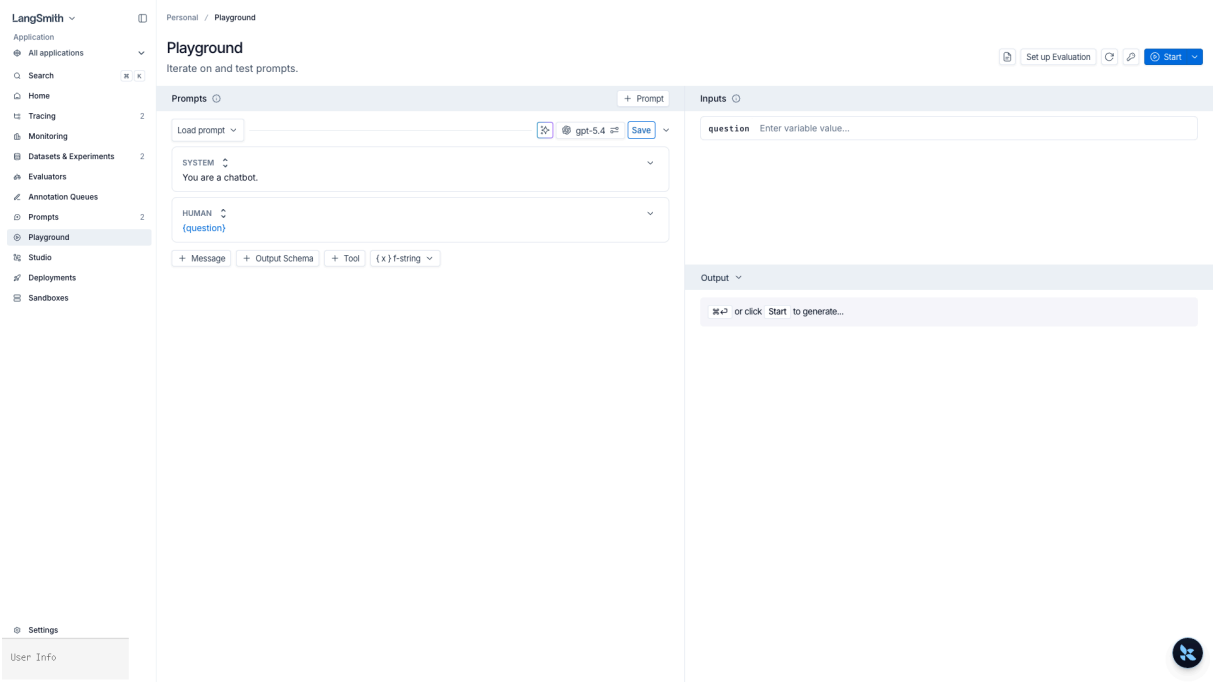
The UI Playground is a space to try out prompts, models, and input variables together. The workflow:

1. Click `Open in Playground` on the prompt page
2. Adjust model, temperature, output schema, and tools in the side panel
3. Enter variable values and click `Run` — results and token/cost are immediately recorded
4. Use `Compare` to compare outputs from multiple prompts/models in parallel for the same input (manual pairwise)

5. When satisfied, click `Save as...` to create a new commit, and promote to `prod` tag if needed

All runs executed in Playground are sent to the Experiments view and can be directly linked to datasets from `03_datasets_and_evaluation`.

<!- phase-c:embed ->



Playground — prompt loaded, editing SYSTEM/HUMAN messages + entering `{question}` variable + generating Output. Default model is `gpt-5.4`, with options for + Message / + Output Schema / + Tool / f-string↔mustache switcher. The `Set up Evaluation` button allows you to attach a dataset and expand into an experiment.

6.04.5 Runtime Injection — `pull_prompt` -\> `create_agent`

In applications, you pull the deployment slot tag with `pull_prompt` and plug it directly into the LLM/agent. When you update the prompt, the change is reflected in the next request without redeployment.

```python
from langchain_openai import ChatOpenAI

# 1) Pull the prompt by tag — in production, use the :prod tag
prompt = client.pull_prompt("weather-bot")  # For production: "weather-bot:prod"

# 2) Connect directly to the model and chain
model = ChatOpenAI(model="gpt-5.4")
chain = prompt | model
result = chain.invoke({"city": "Seoul"})
print(result.content)
```

```python
# When plugging into an agent as system_prompt, extract only the system message text
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Returns the current weather for a city."""
    return f"{city}: Clear, 21°C"

system_text = prompt.messages[0].prompt.template  # The raw template of the 'system' message
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[get_weather],
    system_prompt=system_text,      # Inject the latest system prompt from Prompt Hub
)

out = agent.invoke({"messages": [{"role": "user", "content": "What's the weather in Busan?"}]})
print(out["messages"][-1].content)
```

Public hub + `LangSmithUserError` Exception Handling

Names with an author handle like `hwchase17/rag-prompt` are public prompts from the LangChain Hub. If you pull without a handle, it only searches your own workspace. If you request a non-existent prompt/tag/SHA with `pull_prompt`, a `langsmith.utils.LangSmithUserError` is raised, so you should wrap fallback logic.

```python
from langsmith.utils import LangSmithUserError, LangSmithNotFoundError

# 1) Public hub prompt — 'author/name' format
# Public prompts may include serialized objects, so only enable the explicit flag if you trust the
source.
try:
    public_prompt = client.pull_prompt(
        "hwchase17/rag-prompt",
        dangerously_pull_public_prompt=True,
    )
    print("public prompt messages:", len(public_prompt.messages))
except (LangSmithUserError, LangSmithNotFoundError, ValueError) as e:
    print(f"Failed to fetch public prompt — check handle/name: {e}")

# 2) Fallback to latest commit if :prod tag does not exist
def safe_pull(name: str, tag: str = "prod"):
    try:
        return client.pull_prompt(f"{name}:{tag}")
    except (LangSmithUserError, LangSmithNotFoundError):
        # Tag does not exist → fallback to latest main commit
        return client.pull_prompt(name)

prod_or_latest = safe_pull("weather-bot", tag="prod")
print("safe_pull succeeded:", type(prod_or_latest).__name__)
```

6.04.6 Pinning to a Specific Commit Hash in CI

For production deployment slots, use the `:prod` tag, but for regression tests, always pin to a commit SHA. This ensures that "someone changed the prompt after the tests passed" is automatically prevented.

```python
# Inject the pinned version via environment variable — set PROMPT_PIN in CI
PROMPT_PIN = os.environ.get("WEATHER_BOT_PIN", commit_hash)
pinned = client.pull_prompt(f"weather-bot:{PROMPT_PIN}")
```

```
# CI test example: does the pinned prompt include the expected string?
rendered = pinned.invoke({"city": "Seoul"}).to_messages()[0].content
assert "weather bot" in rendered, "system prompt does not match expectations — check prompt commit"
print(f"CI passed (pin: weather-bot:{PROMPT_PIN})")
```

Deployment Pattern Summary

Environment	Reference	Reason
dev / local	`weather-bot` (latest)	Immediate reflection
staging	`weather-bot:staging`	Test by promoting only the tag
prod	`weather-bot:prod`	Zero-downtime rollout by moving the tag, rollback by reverting the tag
CI	`weather-bot:{SHA}`	Reproducible, prompt changes immediately cause test failures

If you add `prompt_commit` as metadata in the experiment of `03_datasets_and_evaluation`, you can track exactly which commit the metrics are based on directly in the UI.

Checklist

- [ ] Create a new commit with `client.push_prompt("name", object=prompt)`
- [ ] Promote `prod` / `staging` tags to commits in the UI's Commits view
- [ ] Understand when to use f-string vs mustache (loops/conditionals/nesting → mustache)
- [ ] Experience the Playground → Save → Tag promote workflow
- [ ] Inject at runtime with `client.pull_prompt("name:prod")`
- [ ] Use public hub prompts with `client.pull_prompt("author/name")`
- [ ] Fallback for `langsmith.utils.LangSmithUserError` / `LangSmithNotFoundError`
- [ ] Regression test in CI with `name:{SHA}` pin

Next

- `05_production_monitoring.ipynb` — Wrap production agents using deployed prompts with dashboards, alerts, and PII protection

References

- Prompt engineering concepts: https://docs.langchain.com/langsmith/prompt-engineering-concepts
- Manage prompts programmatically: https://docs.langchain.com/langsmith/manage-prompts-programmatically

- Playground: https://docs.langchain.com/langsmith/playground

05 Production Monitoring

Observation · Alerts · PII

Production is not just about "let's just add some traces"—it requires real-time dashboards + automated evaluation + alerts + personal data protection all running together. This notebook groups together the LangSmith features needed after deployment from an operational perspective.

Learning Objectives

- Read latency p50/p95, cost, and success rate from the project dashboard (Overview / Analytics)
- Register an online evaluator as an autoeval rule for automatic execution
- Send user thumbs/star ratings from the app using `client.create_feedback(run_id, ...)`
- Understand metadata-based alert rules (failure rate > N% -> webhook)
- Sample traces in high-volume production using `LANGSMITH_TRACING_SAMPLING_RATE`
- PII scrubbing — combining LangChain `PIIMiddleware` and LangSmith `hide_inputs` / `anonymizer`
- Slack / PagerDuty webhook receiving patterns

Prerequisites

```
LANGSMITH_API_KEY=lsv2_pt_...
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=langsmith-prod-demo
# Optional: In high-volume environments, trace only 10%
# LANGSMITH_TRACING_SAMPLING_RATE=0.1
# Optional: Block all inputs/outputs
# LANGSMITH_HIDE_INPUTS=true
# LANGSMITH_HIDE_OUTPUTS=true
OPENAI_API_KEY=sk-...
```

In production, it is standard practice to separate projects by environment, as in this notebook's example project (`langsmith-prod-demo`).

```
# !pip install -U langsmith langchain langchain-openai

from dotenv import load_dotenv
import os
load_dotenv(override=True)

assert os.environ.get("LANGSMITH_API_KEY"), "LANGSMITH_API_KEY not set"
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY not set"
# If the project is not set, LangSmith records to `default`.
os.environ.setdefault("LANGSMITH_PROJECT", "langsmith-prod-demo")
```

```
print("Project:", os.environ["LANGSMITH_PROJECT"])
print("sampling rate:", os.environ.get("LANGSMITH_TRACING_SAMPLING_RATE", "1.0 (all)"))
```

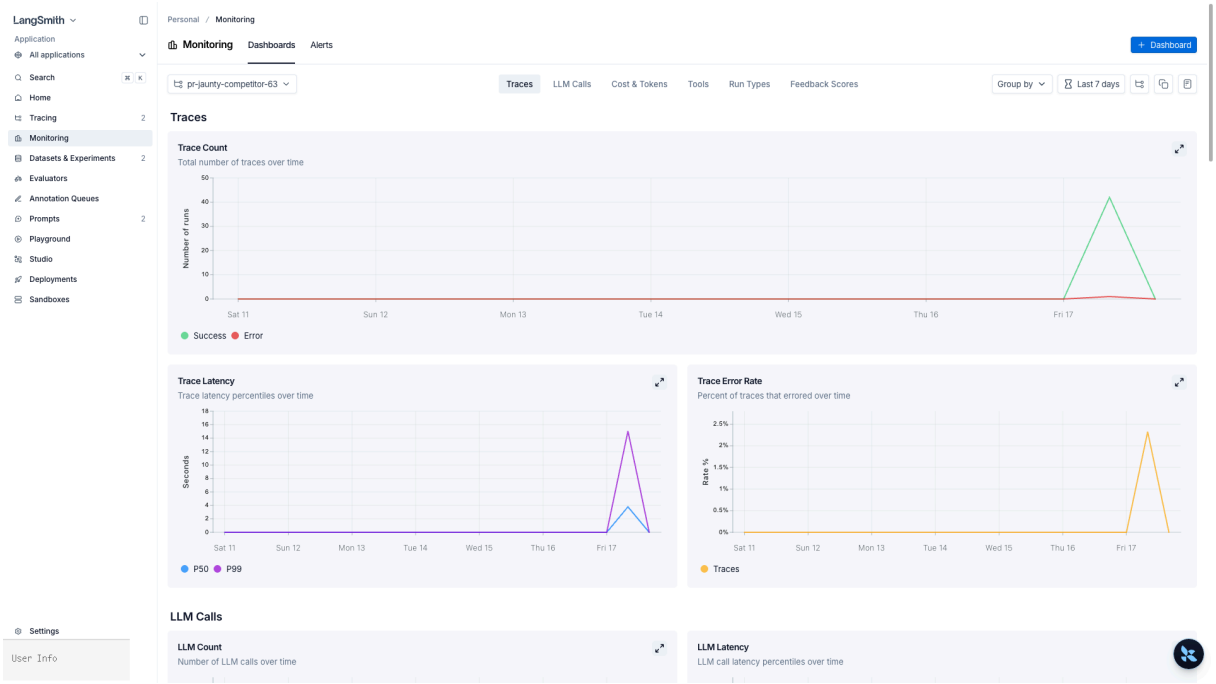
6.05.1 Project Dashboard — latency · cost · success rate

The UI project page has three main tabs:

Tab	What you see	Alert integration
Runs	Trace list, filters, quick search	—
Monitor	latency p50·p95·p99, success rate, error distribution, token/cost time series	Alerts via Rules
Evaluators	Attached online evaluators and score distribution	Alert on drop in specific key scores

If you want to build your own dashboard, you can group custom charts in `Dashboards`. You can also extract the same metrics with SmithDB `client.runs.query` and connect them to internal systems like Grafana.

<!- phase-c:embed ->



Monitoring > Dashboards — 6 tabs (Traces / LLM Calls / Cost & Tokens / Tools / Run Types / Feedback Scores) × period (default: Last 7 days). In the image above, the Fri 17 spike is a pattern caused by running this curriculum all at once. Trace Count / Latency / Error Rate / LLM Count / LLM Latency are all displayed as time-series.

SmithDB projections are explicit: without `selects`, only `ID` is populated. This dashboard query reads timestamps, so it selects `START_TIME` and `END_TIME`.

```
from langsmith import Client
from datetime import datetime, timedelta, timezone
import statistics

client = Client()
project = client.read_project(project_name=os.environ["LANGSMITH_PROJECT"])
window_end = datetime.now(timezone.utc)
since = window_end - timedelta(days=7)
runs = []
async for run in client.runs.query(
    project_ids=[str(project.id)],
    min_start_time=since,
    max_start_time=window_end,
    page_size=500,
    selects=["START_TIME", "END_TIME"],
):
    runs.append(run)
    if len(runs) >= 500:
        break

durations = [
    (r.end_time - r.start_time).total_seconds()
    for r in runs if r.end_time and r.start_time
]
if durations:
    print(f"Sample size {len(durations)} — p50={statistics.median(durations):.2f}s "
          f"p95={statistics.quantiles(durations, n=20)[-1]:.2f}s "
          f"p99={statistics.quantiles(durations, n=100)[-1]:.2f}s")
```

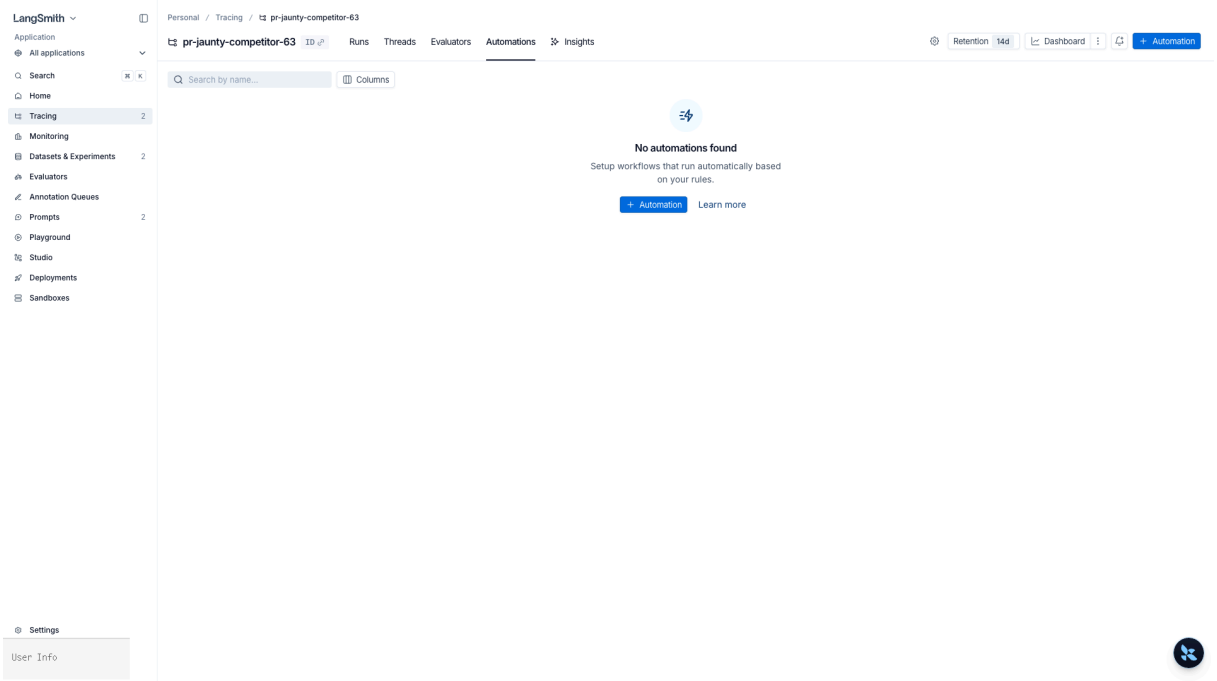
6.05.2 Online evaluator — autoeval rule

If you saw the UI flow for attaching evaluators in `03_datasets_and_evaluation`, here we focus on operational design.

Role	Example setting
Real-time quality gauge	Attach LLM-as-judge (`useful` score 0 1) to runs with `has(tags, "env:prod")`
Cost management	Evaluate only 5% with sampling rate 0.05
Regression detection	Trigger webhook in Rules if average `useful` score drops below N%
Specific use case	Attach evaluation only to runs where `metadata.feature == "checkout"`

autoeval results are saved as feedback keys, so they can be reused for alert rules, dashboards, and Experiments comparison in the next cells.

<!- phase-c:embed ->



Project > Automations tab — `No automations found` state. Define rules with `+ Automation` : condition (e.g., Feedback score < 0.5) → action (e.g., move to dataset / call webhook / send to annotation queue).

6.05.3 User feedback collection API

The standard pattern is to call `client.create_feedback` on the server when the app UI's thumbs-up / star rating / "This answer wasn't helpful" button is pressed. To avoid making the client wait, send it in the background (the Python SDK automatically does this in the background if you provide a `trace_id`).

```
# Example inside an application: capture the run id after agent execution and return it,
# then attach feedback to that id when user feedback arrives.
from langsmith import traceable

@traceable(name="answer-user")
def answer_user(question: str) -> str:
    # In reality, this would return the result of agent.invoke(...)
    return f"Answer to '{question}'."

# Example of getting the current run_id via run_tree at execution time
from langsmith.run_helpers import get_current_run_tree

@traceable(name="answer-with-id")
def answer_with_id(question: str):
    rt = get_current_run_tree()
    return {"answer": answer_user(question), "run_id": str(rt.id)}

reply = answer_with_id("Seoul weather?")
print(reply)

# Later, when the user presses thumbs-up
client.create_feedback(
    run_id=reply["run_id"],
    key="user_thumbs",
```

```
    score=1,
    comment="User responded that it was helpful",
)
print("Feedback sent successfully")
```

6.05.4 Metadata-based alert rules — failure rate \> N% -\> webhook

On the UI's project -> Rules tab, you can combine the following actions:

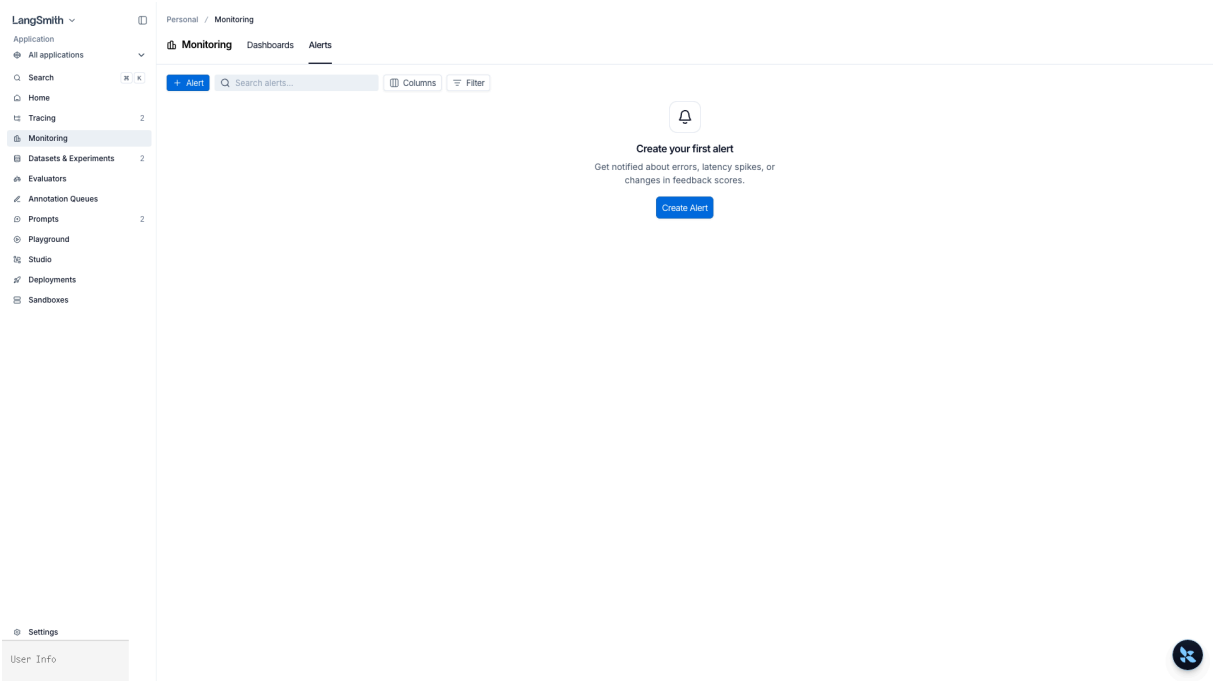
- `Add to annotation queue` (human review)
- `Add to dataset` (accumulate golden set)
- `Trigger webhook` (Slack, PagerDuty, custom incident system)
- `Extend data retention` (extend retention for important traces)
- `Run online evaluator` (conditional quality evaluation)

Sample filter expression

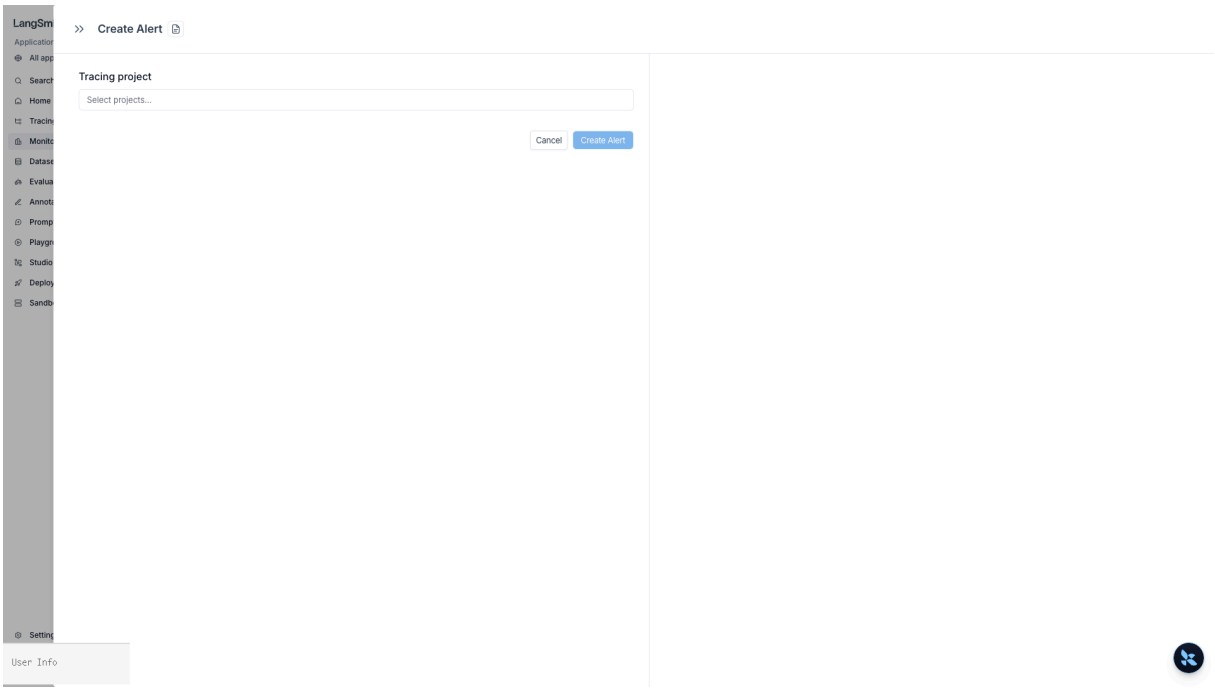
```
and(
  has(tags, "env:prod"),
  eq(status, "error")
)
```

Action execution order (fixed in LangSmith): annotation queue -> dataset -> webhook -> online evaluator -> code evaluator -> alert. You can also set different sampling rates per rule (e.g., 0.5 = 50% of matches).

<!- phase-c:embed ->



Monitoring > Alerts — organization-wide alert rule hub. When you click `Create Alert`, a modal appears:



After selecting the tracing project, specify conditions (error rate/latency/feedback changes) → set up sending to a webhook URL.

6.05.5 High-volume sampling — `LANGSMITH_TRACING_SAMPLING_RATE`

When you have hundreds of requests per second, sending every trace is a waste of cost and network. Control is done on two levels:

Level	Method	Characteristics
Entire process	`LANGSMITH_TRACING_SAMPLING_RATE=0.1`	Applies to `\@traceable`, `RunTree`, and all automatic instrumentation
Per request	`Client(tracing_sampling_rate=...)` + `tracing_context`	100% for important requests like admin/payment

By combining these, you can implement operational policies like “sample 10% of general traffic, 100% of payment traffic.”

```
from langsmith import Client
from langsmith.run_helpers import tracing_context

# The default client records only 10%, the critical flow client records 100%
client_bulk     = Client(tracing_sampling_rate=0.1)
client_critical = Client(tracing_sampling_rate=1.0)

with tracing_context(client=client_critical, tags=["env:prod", "flow:checkout"]):
    # Critical flows like payment are executed in this block — all are recorded
```

```
    print("critical flow — 100% trace")

with tracing_context(client=client_bulk, tags=["env:prod", "flow:chat"]):
    # General chat — only 10% are sampled
    print("bulk flow — 10% sampling")
```

6.05.6 PII Scrubbing — `PIIMiddleware` + `anonymizer`

Defense is done in two layers:

1. Model input stage: `langchain.agents.middleware.PIIMiddleware` blocks/masks emails, card numbers, API keys, etc. in messages sent to the LLM—so the model itself never sees PII.
2. Trace sending stage: LangSmith client's `hide_inputs` / `hide_outputs` or `anonymizer` scrubs inputs/outputs again before sending to the server.

You should use both to avoid gaps like "PII went to the model but not the trace" or "model didn't see it but it ended up in the trace."

```
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware
from langchain.tools import tool

@tool
def lookup_order(order_id: str) -> str:
    """Check order status (for demo)."""
    return f"{order_id} -> In delivery"

# 1) Model input stage — masking with PIIMiddleware
# PIIMiddleware in langchain 1.2.x defaults to 5 pii_types: email/credit_card/ip/mac_address/url.
# To use custom types like "api_key", you need to provide your own detector.
safe_agent = create_agent(
    model="openai:gpt-5.4",
    tools=[lookup_order],
    middleware=[
        PIIMiddleware("email",       strategy="redact"),
        PIIMiddleware("credit_card", strategy="redact"),
    ],
)

out = safe_agent.invoke({"messages": [
    {"role": "user", "content": "Check my order status for my email alice@example.com (order A-42)"}
]})
print(out["messages"][-1].content)
```

```
# 2) Trace sending stage — re-masking input/output with anonymizer
from langsmith.anonymizer import create_anonymizer

anonymizer = create_anonymizer([
    {"pattern": r"[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+", "replace": "<email>"},
    {"pattern": r"\b\d{4}[ -]?\d{4}[ -]?\d{4}[ -]?\d{4}\b",            "replace": "<card>"},
])

masked_client = Client(anonymizer=anonymizer)

# If you need to block everything, use environment variables or Client arguments
fully_hidden_client = Client(
    hide_inputs=lambda inputs: {},
    hide_outputs=lambda outputs: {},
)
print("anonymizer / full-hide client ready — swap in the Client instance as needed")
```

6.05.7 Slack / PagerDuty webhook integration

The webhook action in Rules sends a POST payload to the configured URL. The receiving side converts it to Slack/PagerDuty format and triggers an alert.

```
# Example: Receiving webhook with FastAPI — relaying to Slack
from fastapi import FastAPI, Request
import httpx, os

app = FastAPI()
SLACK_URL     = os.environ["SLACK_INCOMING_WEBHOOK"]
PAGERDUTY_URL = os.environ.get("PAGERDUTY_EVENTS_V2_URL")

@app.post("/langsmith/alert")
async def on_alert(req: Request):
    event = await req.json()
    run_id  = event.get("run_id") or event.get("trace_id")
    status  = event.get("status", "unknown")
    tags    = event.get("tags", [])
    trace_url = f"https://smith.langchain.com/o/-/projects/-/r/{run_id}"

    async with httpx.AsyncClient() as hc:
        # Slack
        await hc.post(SLACK_URL, json={
            "text": f":rotating_light: LangSmith alert — status={status} tags={tags}\n<{trace_url}|Open
trace>",
        })
        # PagerDuty (only for high severity)
        if PAGERDUTY_URL and "critical" in tags:
            await hc.post(PAGERDUTY_URL, json={
                "routing_key": os.environ["PAGERDUTY_ROUTING_KEY"],
                "event_action": "trigger",
                "payload": {"summary": f"LangSmith {status}", "severity": "error", "source":
"langsmith"},
            })
    return {"ok": True}
```

Register this endpoint URL as the webhook target in the UI's Rules, and set the filter to `and(has(tags, "env:prod"), eq(status, "error"))` to receive only production errors in Slack.

6.05.8 Manual Run Tree control + LangSmith Deployments monitoring

For workers/CLI/batch jobs where automatic instrumentation doesn't reach, you can build the trace tree directly with `RunTree`. If you specify parent–child relationships, the UI tree will appear exactly as you constructed it.

All threads of LangSmith Deployments deployed on the LangGraph Platform are automatically traced to the same project, and the Monitor / Rules / Online evaluator features described above work as is.

```
from langsmith.run_trees import RunTree

# Start root run — pattern for external workers sending an entire trace at once
root = RunTree(
    name="batch-rebuild",
    run_type="chain",
    inputs={"job": "nightly-index"},
    project_name=os.environ["LANGSMITH_PROJECT"],
```

```
    tags=["env:prod", "source:cron"],
)

# Child run — simulating a tool call
child = root.create_child(
    name="fetch-shards",
    run_type="tool",
    inputs={"shards": 8},
)
child.end(outputs={"fetched": 8})
child.post()

root.end(outputs={"ok": True})
root.post()
print(f"RunTree sent successfully → root id={root.id}")
```

Checklist

- [ ] Check p50/p95/p99 · success rate · cost in the project Monitor tab
- [ ] Register an online evaluator with prod tag filter + 5% sampling
- [ ] Implement thumbs feedback sending pattern in the app using `client.create_feedback(run_id, ...)`
- [ ] Attach webhook action to `status=error` + tag filter via Rules
- [ ] Use `LANGSMITH_TRACING_SAMPLING_RATE` and `tracing_context` to sample 100% only for critical flows
- [ ] Dual protection with `PIIMiddleware` (model) + `anonymizer` / `hide_inputs` (trace)
- [ ] Send external worker traces using `RunTree.create_child` + `post()`
- [ ] Check that threads in LangSmith Deployments (LangGraph Platform) automatically flow into the same project
- [ ] Verify that the webhook receiving service routes to Slack / PagerDuty

Next

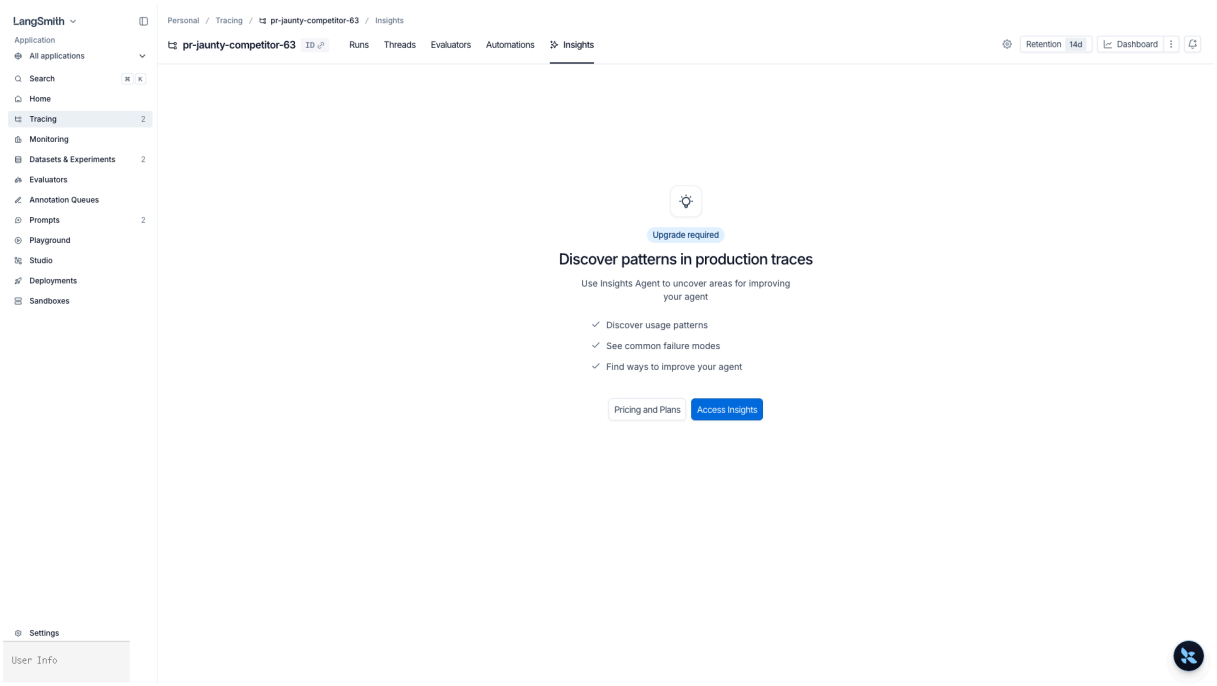
- This wraps up the `06_langsmith` curriculum. For a comparison with vendor-neutral observability like Langfuse or OTel, see `08_integration/12_observability/`.

References

- Observability overview: https://docs.langchain.com/langsmith/observability
- Dashboards: https://docs.langchain.com/langsmith/dashboards
- Alerts: https://docs.langchain.com/langsmith/alerts
- Online evaluators: https://docs.langchain.com/langsmith/online-evaluations
- Rules · webhook: https://docs.langchain.com/langsmith/rules
- Sampling: https://docs.langchain.com/langsmith/sample-traces
- Mask inputs/outputs: https://docs.langchain.com/langsmith/mask-inputs-outputs
- Feedback API: https://docs.langchain.com/langsmith/attach-user-feedback
- LangSmith Deployments: https://docs.langchain.com/langsmith/deployments

<!- phase-c:embed ->

6.05.X Insights Agent (Paid)



Project > Insights tab — LangSmith's Insights Agent automatically extracts usage patterns / common failure modes from production traces. Upgrade required for free plans.

06 Agent Evals

trajectory match and LLM-as-judge

Learning goals

- Understand how `agentevals` evaluates an agent's tool-call path, not just final text
- Distinguish `strict`, `unordered`, `subset`, and `superset` match modes
- Use an LLM-as-judge evaluator to compare actual and reference trajectories semantically
- Connect trajectory evaluators to LangSmith `evaluate()` for regression testing

Overview

`03_datasets_and_evaluation.ipynb` covers the broad dataset/evaluate loop. This chapter focuses on the agent's internal path.

Target	Generic evaluator	AgentEvals
Final answer	String/JSON comparison	Possible, but not the main point
Tool calls	Manual parsing	Built-in `tool_calls` trajectory comparison
Path quality	Custom LLM judge prompt	Trajectory-specific judge prompt
Regression tests	LangSmith experiment	LangSmith + trajectory evaluator

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
assert os.environ.get("LANGSMITH_API_KEY"), "Set LANGSMITH_API_KEY in .env"
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
os.environ.setdefault("LANGSMITH_PROJECT", "langchain-langgraph-deepagents-notebooks")
```

```
from agentevals.trajectory.match import create_trajectory_match_evaluator
from agentevals.trajectory.llm import create_trajectory_llm_as_judge
```

1) Build trajectories to evaluate

A trajectory can be represented as a list of message-like dictionaries with `role`, `content`, and `tool_calls`. The important question is: which tool did the agent call, and with which arguments?

```
actual_trajectory = [
    {"role": "user", "content": "Weather in Seoul?"},
    {"role": "assistant", "tool_calls": [{"name": "get_weather", "args": {"city": "Seoul"}}]},
    {"role": "tool", "content": "Seoul: sunny, 21°C"},
    {"role": "assistant", "content": "It is sunny and 21°C in Seoul."},
]
reference_trajectory = [
    {"role": "user", "content": "Weather in Seoul?"},
    {"role": "assistant", "tool_calls": [{"name": "get_weather", "args": {"city": "Seoul"}}]},
]
```

2) Trajectory match evaluator

`trajectory_match_mode` defines the relationship between the actual and reference trajectories.

Mode	Meaning
`strict`	Order and calls must match exactly
`unordered`	The same call set matters more than order
`subset`	Actual trajectory must be a subset of the reference
`superset`	Actual trajectory may include extra calls if it includes the reference

```
trajectory_superset = create_trajectory_match_evaluator(
    trajectory_match_mode="superset",
    tool_args_match_mode="subset",
)
match_result = trajectory_superset(
    outputs=actual_trajectory,
    reference_outputs=reference_trajectory,
)
print(match_result)
assert match_result["score"] is True
```

3) LLM-as-judge trajectory evaluation

Use an LLM judge when the reference is abstract or when the question is whether the path was reasonable rather than whether every call was byte-for-byte identical.

```
trajectory_judge = create_trajectory_llm_as_judge(
    model="openai:gpt-5.4",
    use_reasoning=False,
)
judge_result = trajectory_judge(
    outputs=actual_trajectory,
```

```
    reference_outputs=reference_trajectory,
)
print(judge_result)
```

4) Connect to LangSmith `evaluate()`

The next cell creates a real LangSmith dataset and experiment. The local harness skips remote write/evaluate cells by default, and runs them only when `--allow-langsmith-mutations` is set.

```
from langsmith import Client
from langsmith.evaluation import evaluate

client = Client()
DATASET_NAME = "agent-eval-v1"
try:
    dataset = client.create_dataset(
        dataset_name=DATASET_NAME,
        description="Agent trajectory regression examples",
    )
except Exception:
    dataset = client.read_dataset(dataset_name=DATASET_NAME)
client.create_examples(dataset_id=dataset.id, examples=[{
    "inputs": {"question": "Weather in Seoul?"},
    "outputs": {"trajectory": reference_trajectory},
}])

def predict_agent_trajectory(inputs: dict) -> dict:
    return {"trajectory": actual_trajectory}

def trajectory_evaluator(inputs, outputs, reference_outputs):
    return trajectory_superset(
        outputs=outputs["trajectory"],
        reference_outputs=reference_outputs["trajectory"],
    )

results = evaluate(
    predict_agent_trajectory,
    data=DATASET_NAME,
    evaluators=[trajectory_evaluator],
    experiment_prefix="agent-eval-v1:gpt-5.4",
    metadata={"model": "gpt-5.4", "kind": "trajectory"},
)
print("Experiment:", results)
```

Operations checklist

Criterion	Recommendation
Regression test	Use `strict` or `unordered` for critical tool paths
Flexible agent	Use `superset` when extra helper calls are acceptable
Argument comparison	Use `subset` for required fields, `exact` for full contracts
No reference	Use LLM-as-judge for path reasonableness
Production linkage	Use LangSmith dataset + experiment prefix + model metadata

References:

- LangChain Agent Evals: https://docs.langchain.com/oss/python/langchain/test/evals
- LangSmith Evaluation: https://docs.langchain.com/langsmith/evaluation
- AgentEvals GitHub: https://github.com/langchain-ai/agentevals

07 Testing Strategy

Separate unit, integration, and eval

Agent testing works best when different test types do different jobs. This chapter separates deterministic unit tests, provider integration checks, and behavioral evaluations.

Learning goals

- Decide what belongs in unit, integration, and eval layers.
- Use fake models for fast deterministic tests.
- Keep expensive or stateful checks behind explicit gates.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from langchain_core.language_models.fake_chat_models import FakeListChatModel
from langchain_core.messages import HumanMessage

model = FakeListChatModel(responses=["test response"])
model.invoke([HumanMessage(content="ping")]).content
```

7.1 Testing pyramid

The testing pyramid still applies to agent systems, but the top layer often contains evaluations rather than only end-to-end tests. Put cheap deterministic checks at the base.

7.2 Unit test pure functions first

Pure functions are the easiest place to build confidence. Test routing, formatting, and scoring logic before introducing model variability.

```
def normalize_question(text: str) -> str:
    return " ".join(text.strip().lower().split())

assert normalize_question("  Hello   Agent  ") == "hello agent"
print("unit test passed")
```

7.3 Integration gate

Integration tests prove that external providers and runtime wiring still work. Keep them clearly marked because they may require credentials, network access, or cost.

```
def can_run_openai_integration() -> bool:
    return bool(os.getenv("OPENAI_API_KEY")) and os.getenv("RUN_LIVE_TESTS") == "1"

print("run live integration:", can_run_openai_integration())
```

7.4 Eval checks behavior paths, not only answer strings

Agent evaluations should inspect behavior, not just final text. Tool calls, routing choices, and recovery paths often matter more than exact wording.

```
trajectory = [
    {"role": "user", "content": "weather in Seoul"},
    {"role": "assistant", "tool_calls": [{"name": "get_weather"}]},
    {"role": "tool", "name": "get_weather", "content": "clear"},
]

assert trajectory[1]["tool_calls"][0]["name"] == "get_weather"
```

7.5 Separate CI and manual verification

CI should stay fast and reliable. Manual or scheduled runs can cover heavier evaluations, live providers, and mutation-prone resources.

Summary

Item	Content
Covered	fake models, unit tests, integration gates, evals, and smoke-test boundaries
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `unit-testing.md`

- `integration-testing.md`
- `evals.md`

08 LangGraph Testing

Assert state and checkpoint behavior

LangGraph applications should be tested at the node, graph, and persistence layers. This chapter shows how to make state transitions and checkpoint behavior explicit.

Learning goals

- Unit test a node as a state transformation.
- Integration test a small graph path.
- Smoke test checkpoint persistence and resume assumptions.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

class CounterState(TypedDict):
    count: int
```

8.1 Node unit test

A node is easiest to test when you treat it as a function from state to state update. This catches schema and transformation mistakes early.

```
def increment(state: CounterState) -> dict:
    return {"count": state["count"] + 1}

assert increment({"count": 1}) == {"count": 2}
print("node unit test passed")
```

8.2 Graph integration test

A graph test verifies that nodes and edges work together. It should focus on a representative path rather than every possible conversation.

```
builder = StateGraph(CounterState)
builder.add_node("increment", increment)
builder.add_edge(START, "increment")
```

```
builder.add_edge("increment", END)
graph = builder.compile()

assert graph.invoke({"count": 0})["count"] == 1
```

8.3 Checkpoint smoke test

Checkpoint tests protect durability assumptions. They confirm that saved state can be retrieved and used as the next execution boundary.

```
checkpointer = InMemorySaver()
graph_with_memory = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "test-1"}}

result = graph_with_memory.invoke({"count": 2}, config=config)
assert result["count"] == 3
```

Summary

Item	Content
Covered	node unit tests, graph integration tests, checkpointer smoke tests, and state assertions
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `test.md`
- `checkpointers.md`
- `persistence.md`

09 Runtime Rubric Evaluation

Build quality gates during execution

Offline evaluations are valuable, but some quality checks should run while the agent is working. This chapter shows how to use runtime rubrics as lightweight gates.

Learning goals

- Express quality criteria as runtime checks.
- Build a deterministic evaluator for structural requirements.
- Relate runtime gates to offline evaluation suites.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

9.1 Rubric criteria

A runtime rubric should focus on criteria that can be checked quickly. Examples include sources, verification evidence, next steps, and forbidden omissions.

```
criteria = {
    "has_sources": lambda text: "References" in text,
    "has_tests": lambda text: "verification" in text,
    "has_next_step": lambda text: "next" in text,
}

list(criteria)
```

9.2 Evaluator

The evaluator turns rubric criteria into a pass/fail signal. Keep its output explainable so failures can guide the next agent step.

```
def grade(text: str) -> dict:
    checks = {name: fn(text) for name, fn in criteria.items()}
    feedback = [name for name, passed in checks.items() if not passed]
    return {"passed": all(checks.values()), "checks": checks, "feedback": feedback}

grade("The response has a summary and next step, but no verification evidence.")
```

9.3 Relationship to offline evals

Runtime gates and offline evals serve different purposes. Runtime gates protect individual runs, while offline evals measure behavior across many examples.

Summary

Item	Content
Covered	runtime rubric criteria, deterministic grading, feedback lists, and offline-eval boundaries
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `rubric.md`
- `profiles.md`
- `evals.md`

PART

VII

Applied Examples

Real-World Applications

What You Will Learn in This Part

- 1. RAG Agent
- 2. SQL Agent
- 3. Data Analysis Agent
- 4. ML Agent
- 5. Deep Research Agent
- 6. Multimodal PDF RAG
- 7. Content Builder Agent
- 10. Personal Assistant Subagents
- 11. Customer Support Handoffs
- 12. Router Knowledge Base
- 13. Skills SQL Assistant

01 RAG Agent

5 Building Blocks + 3-Node Pattern

Learning Objectives

- Understand the five building blocks of LangChain RAG (document loaders, text splitters, embedding models, vector stores, retrievers)
- Compare the three RAG architectures: 2-Step, Agentic, and Hybrid
- Separate query rewriting from retrieval with the `Rewrite → Retrieve → Agent` three-node pattern
- Define a retrieval tool with the `content_and_artifact` return pattern
- Build a RAG agent with `create_deep_agent` and apply v1 middleware (`ModelCallLimitMiddleware` , `ToolRetryMiddleware`)
- Use the Skills system to progressively disclose RAG domain knowledge

Overview

Item	Details
Framework	LangChain v1 + Deep Agents
5 building blocks	Document loaders · Text splitters · Embedding models · Vector stores · Retrievers
Architecture	2-Step (static RAG) · Agentic (tool calls) · Hybrid (Rewrite → Retrieve → Agent)
Agent pattern	`content_and_artifact` tool → `create_deep_agent`
Backend	`FilesystemBackend(root_dir=".", virtual_mode=True)`
Skill	`skills/rag-agent/SKILL.md` — progressive disclosure of RAG domain knowledge

> ➡️ TIP
>
> See `docs/langchain/24-retrieval.md` for a side-by-side comparison of the 2-Step, Agentic, and Hybrid RAG architectures.

```python
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```python
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

The 5 RAG Building Blocks

LangChain RAG consists of five composable building blocks. Each one is replaceable, and the way you wire them together defines the architecture.

Block	Role
Document loaders	Load PDFs, web pages, or databases into `Document` objects
Text splitters	Split long documents into retrieval-friendly chunks
Embedding models	Convert text into vectors (e.g. `OpenAIEmbeddings`)
Vector stores	Persist embeddings and run similarity search (FAISS, Chroma, InMemory)
Retrievers	Return the top-N relevant documents for a query — usually wrapped as an agent tool

2-Step / Agentic / Hybrid Architectures

The same building blocks produce three different architectures depending on how they are composed.

Architecture	Flow	Strengths	Trade-offs
2-Step RAG	Query → retrieve → LLM response (static)	Simple, fast, predictable	No re-querying or multi-retrieval
Agentic RAG	Agent calls `retrieve` on demand	Handles multi-step and comparison queries	Higher token usage
Hybrid	Three nodes: `Rewrite → Retrieve → Agent`	Better recall through query rewriting	More graph wiring up front

The Rewrite → Retrieve → Agent 3-Node Pattern

The Hybrid layout pulls query rewriting and retrieval out into separate nodes and lets the agent synthesize the final answer. This separation keeps retrieval accuracy stable even when the user's question is vague or carries multiple intents.

```
# Conceptual graph — implement each node with LangGraph
# rewrite_node:   original query → rewritten retrieval query
# retrieve_node:  vectorstore.similarity_search(query, k=K)
# agent_node:     create_deep_agent + tools=[retrieve_tool]
```

Step 1: Create Sample Documents

The first step in a RAG pipeline is preparing the documents you want to search. In a real system, you would load documents from PDFs, web pages, or databases. Here, for learning purposes, we create `Document` objects directly.

```
from langchain_core.documents import Document

docs = [
    Document(page_content="LangChain is a framework for building LLM applications. It supports tools,
chains, and agents.", metadata={"source": "langchain"}),
    Document(page_content="LangGraph is a framework for building stateful workflows. It provides a Graph
API and a Functional API.", metadata={"source": "langgraph"}),
    Document(page_content="Deep Agents is an all-in-one agent SDK. It creates agents with
create_deep_agent and supports backends and subagents.", metadata={"source": "deepagents"}),
    Document(page_content="RAG stands for retrieval-augmented generation. It injects external knowledge
into an LLM to produce more accurate answers.", metadata={"source": "rag"}),
    Document(page_content="A vector store is a database that stores embeddings and performs similarity
search. FAISS and Chroma are common examples.", metadata={"source": "vectorstore"}),
    Document(page_content="An agent is a system in which an LLM uses tools to perform tasks autonomously.
ReAct is a representative pattern.", metadata={"source": "agent"}),
]
print(f"Created {len(docs)} documents")
```

Step 2: Split the Text

Split larger documents into chunks that are easier to retrieve. `RecursiveCharacterTextSplitter` tries to split at natural boundaries such as paragraphs, sentences, and then words.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=200, chunk_overlap=50
)
splits = splitter.split_documents(docs)
print(f"Split result: {len(splits)} chunks")
```

Step 3: Build the Vector Store

Convert the text into vectors with an OpenAI embedding model and store them in `InMemoryVectorStore`. In production, you would typically use a persistent store such as FAISS or Chroma.

```
from langchain_openai import OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = InMemoryVectorStore.from_documents(splits, embeddings)
print(f"Vector store ready — embedded {len(splits)} documents")
```

Step 4: Define a Retrieval Tool (`content_and_artifact`)

The `response_format="content_and_artifact"` pattern makes the tool return two things:

- content: a text summary shown to the agent
- artifact: the full `Document` objects for downstream processing

This pattern helps save context tokens while still preserving access to the original data.

```
from langchain.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    """Search for relevant documents in the vector store."""
    results = vectorstore.similarity_search(query, k=3)
    content = "\n\n".join(d.page_content for d in results)
    return content, results
```

Step 5: Test the Retrieval Tool by Itself

Before connecting the tool to the agent, verify that it behaves correctly.

```
result = retrieve.invoke({"query": "What is an agent?"})
print(result)
```

Step 6: Create the RAG Agent (with v1 Middleware)

Load the prompt from the prompt module. The flow tries LangSmith Hub → Langfuse → a local default prompt.

Middleware	Role
`ModelCallLimitMiddleware`	Prevents infinite loops by limiting the number of model calls

`ToolRetryMiddleware`	Automatically retries failed retrieval tool calls

```python
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langchain.agents.middleware import (
    ModelCallLimitMiddleware,
    ToolRetryMiddleware,
)
from prompts import RAG_AGENT_PROMPT

agent = create_deep_agent(
    model=model,
    tools=[retrieve],
    system_prompt=RAG_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    middleware=[
        ModelCallLimitMiddleware(run_limit=10),
        ToolRetryMiddleware(max_retries=2),
    ],
)
```

Step 7: Run a Simple Query and a Comparison Query

Use a simple query (single retrieval) and a comparison-style query (multiple retrieval steps) to verify that the RAG agent works as expected.

Summary

Item	Key Point
Vector store	`InMemoryVectorStore.from_documents()` — embedding-based similarity search
Retrieval tool	`\@tool(response_format="content_and_artifact")` — summary + original artifact separation
Agent	`create_deep_agent(model, tools=[retrieve], backend=..., skills=["/skills/"])`
Skill	`skills/rag-agent/SKILL.md` — saves tokens through progressive disclosure

References:

- `docs/langchain/24-retrieval.md`
- LangChain RAG Tutorial
- `docs/deepagents/10-skills.md`

Next Step: → 02_sql_agent.ipynb: Build a SQL agent.

02 SQL Agent

Natural-Language Database Queries

Learning Objectives

- Generate SQL tools automatically with `SQLDatabaseToolkit`
- Apply AGENTS.md-based safety rules (READ-ONLY)
- Implement approval before query execution with a HITL interrupt
- Explicitly apply v1 middleware (`HumanInTheLoopMiddleware` , `ModelCallLimitMiddleware`)

Overview

Item	Details
Framework	LangChain + Deep Agents
Core components	`SQLDatabaseToolkit` , `SQLDatabase` , `InMemorySaver`
Agent pattern	AGENTS.md safety rules + skills-based workflow
HITL	`interrupt_on={"sql_db_query": {"allowed_decisions":[...]}}` + `Command(resume={"decisions":[...]})`
Database	Chinook (SQLite)
Skill	`skills/sql-agent/SKILL.md` — SQL safety rules + query workflow

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

Step 1: Connect to the Database

Chinook is a sample database for a digital music store. It contains tables such as Artist, Album, Track, and Invoice.

```
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///../05_advanced/Chinook.db")
print(f"Tables: {db.get_usable_table_names()}")
```

Step 2: Create the `SQLDatabaseToolkit` Tools

`SQLDatabaseToolkit` automatically generates four tools from the database connection:

Tool	Description
`sql_db_list_tables`	List available tables
`sql_db_schema`	Inspect table schema (DDL)
`sql_db_query`	Execute a SQL query
`sql_db_query_checker`	Validate query syntax before execution

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=model)
sql_tools = toolkit.get_tools()
for t in sql_tools:
    print(f"  {t.name}: {t.description[:60]}")
```

Step 3: Load the Prompt (LangSmith / Langfuse / Default)

The prompt loader follows this order:

1. LangSmith Hub — if configured, pull the prompt from the Hub
2. Langfuse — if configured, load it from Langfuse
3. Default — otherwise, use the fallback prompt defined in code

The SQL agent prompt includes READ-ONLY safety rules and the expected query workflow.

```
from prompts import SQL_AGENT_PROMPT

print(SQL_AGENT_PROMPT)
```

Step 4: The Idea Behind Skills

Skills act as workflow guides for the agent. They document recurring task patterns so the agent works in a more consistent way.

Skill	Purpose
`query-writing`	Inspect tables → inspect schema → write SQL → execute
`schema-exploration`	List tables → inspect DDL → map relationships

Step 5: Create a Basic SQL Agent

Pass the SQL tools and AGENTS-style safety instructions into `create_deep_agent`. The `system_prompt` injects the SQL safety rules.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

agent = create_deep_agent(
    model=model,
    tools=sql_tools,
    system_prompt=SQL_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
)
```

Step 6: A HITL Agent (`interrupt_on`)

Use the `interrupt_on` parameter of `create_deep_agent` to define approval policies for specific tools. Execution stops before `sql_db_query` runs, and then resumes with `Command(resume={"decisions": [...]})`. In v1 you can spell out the allowed decision types per tool with `allowed_decisions`.

Parameter	Role
`interrupt_on={"sql_db_query": {"allowed_decisions": [...]}}`	Pause before `sql_db_query` runs and declare which decision types are valid
`ModelCallLimitMiddleware`	Prevent infinite loops by limiting the run to 15 model calls
`InMemorySaver`	Enables checkpointing so interrupts and resumes work

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import ModelCallLimitMiddleware
```

```
hitl_agent = create_deep_agent(
    model=model,
    tools=sql_tools,
    system_prompt=SQL_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    interrupt_on={
        "sql_db_query": {
            "allowed_decisions": ["approve", "edit", "reject", "respond"],
        },
    },
    middleware=[
        ModelCallLimitMiddleware(run_limit=15),
    ],
)
```

Step 7: Resume After Approval — Four Decision Types

Use `Command(resume={"decisions": [...]})` to continue a paused run. The v1 `HITLResponse` supports four decision types.

Decision Type	Description
`{"type": "approve"}`	Approve the tool call and run it unchanged
`{"type": "edit", "edited_action": {"name": "...", "args": {...}}}`	Modify the tool call before running it (e.g. add a `LIMIT`, tighten a `WHERE` clause)
`{"type": "reject", "message": "..."}`	Reject the tool call and return feedback to the agent
`{"type": "respond", "message": "..."}`	Skip tool execution and feed the human's reply back as the tool result — the "ask user" pattern

```
from langgraph.types import Command

response = hitl_agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config={**thread, **lf_config},
    version="v2",
)
```

Summary

Item	Key Point
Tool generation	`SQLDatabaseToolkit(db, llm).get_tools()` → automatically creates 4 SQL tools
Safety rules	Applies a READ-ONLY policy through the SQL agent instructions

Skills	Workflow guides for query writing and schema exploration
HITL	`interrupt_on={"sql_db_query": {"allowed_decisions":[...]}}` → 4-decision (`approve` / `edit` / `reject` / `respond`)

References:

- `docs/deepagents/examples/03-text-to-sql-agent.md`
- LangChain SQL Agent Tutorial
- `docs/deepagents/06-backends.md`

Next Step: → 03_data_analysis_agent.ipynb: Build a data analysis agent.

03 Data Analysis Agent

Code Execution and Multi-Turn Analysis

Learning Objectives

- Set up a code execution environment with `LocalShellBackend`
- Combine custom tools with built-in tools such as `write_todos` and `execute`
- Perform iterative analysis with streaming and multi-turn conversations
- Apply v1 middleware (`SummarizationMiddleware` , `ModelCallLimitMiddleware`)

Overview

Item	Details
Framework	Deep Agents
Core components	`LocalShellBackend` , `InMemorySaver`
Built-in tools	`execute` (code execution), `write_todos` (planning)
Pattern	Streaming (`stream(subgraphs=True)`) + multi-turn conversation
Skill	`skills/data-analysis/SKILL.md` — analysis checklist + code execution rules

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

Step 1: Compare Backends

Deep Agents supports multiple backends. For data analysis, code execution matters, so we use `LocalShellBackend` .

Backend	File Access	Code Execution	Best Use
`StateBackend`	Stored in state	❌	Scratchpad
`FilesystemBackend`	Local disk	❌	Read/write files
`LocalShellBackend`	Local disk	✅ `execute`	Data analysis, code execution
Sandboxes (Modal, etc.)	Isolated environment	✅	Production

> ➡️ TIP
>
> ⚠️ `LocalShellBackend` runs commands on the host system. Always use `virtual_mode=True`.

```
from deepagents.backends import LocalShellBackend

backend = LocalShellBackend(root_dir=".", virtual_mode=True)
```

Step 2: Create a CSV File for Analysis

If the agent is going to run pandas code with `execute`, there needs to be a CSV file on disk. The agent could also write files itself with the built-in `write_file` tool, but here we prepare the file ahead of time.

```
import tempfile, os

# Create a CSV file in a temporary directory
tmp_dir = tempfile.mkdtemp()
csv_path = os.path.join(tmp_dir, "sales.csv")

CSV_DATA = """date,product,region,sales,quantity
2024-01-15,Widget A,Seoul,150000,30
2024-01-15,Widget B,Busan,89000,18
2024-02-10,Widget A,Seoul,175000,35
2024-02-10,Widget C,Daegu,62000,12
2024-03-05,Widget B,Seoul,134000,27
2024-03-05,Widget A,Busan,98000,20
2024-03-20,Widget C,Seoul,71000,14
2024-04-01,Widget A,Daegu,112000,22"""

with open(csv_path, "w", encoding="utf-8") as f:
    f.write(CSV_DATA.strip())
print(f"Saved CSV: {csv_path}")
```

Step 3: Define the Analysis Tools

We define two custom tools:

Tool	Role
`get_csv_path`	Returns the CSV file path
`run_pandas`	Executes pandas code directly and returns the result

> · NOTE
>
> `run_pandas` executes pandas code written by the agent with `exec()`. In many notebook environments, this is more stable than relying only on the built-in `execute` tool.

```python
from langchain.tools import tool
import io, contextlib

@tool
def get_csv_path() -> str:
    """Return the path of the CSV file to analyze."""
    return csv_path

@tool
def run_pandas(code: str) -> str:
    """Execute pandas Python code. Use print() to display results."""
    import pandas as pd, numpy as np
    buf = io.StringIO()
    ns = {"pd": pd, "np": np, "csv_path": csv_path}
    try:
        with contextlib.redirect_stdout(buf):
            exec(code, ns)
        return buf.getvalue() or "Execution finished (no printed output)"
    except Exception as e:
        return f"Error: {e}"
```

Step 4: Create the Agent (with v1 Middleware)

Combine `LocalShellBackend` with the custom tools (`get_csv_path` , `run_pandas`).

Middleware	Role
`SummarizationMiddleware`	Automatically summarizes older messages when the conversation becomes long
`ModelCallLimitMiddleware`	Prevents infinite loops by limiting the run to 20 model calls

```python
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    SummarizationMiddleware,
    ModelCallLimitMiddleware,
)
from prompts import DATA_ANALYSIS_PROMPT

agent = create_deep_agent(
    model=model,
    tools=[get_csv_path, run_pandas],
```

```
    system_prompt=DATA_ANALYSIS_PROMPT,
    backend=LocalShellBackend(root_dir=tmp_dir, virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(model=model, trigger=("messages", 10)),
        ModelCallLimitMiddleware(run_limit=20),
    ],
)
```

Step 5: Analyze with pandas Code Execution

When you ask the agent to analyze the data, it can first use `get_csv_path` to confirm the file path and then use `run_pandas` to write and execute pandas code.

```
Agent execution flow:
1. get_csv_path() → confirm the CSV file path
2. run_pandas("import pandas as pd; ...") → execute pandas code
3. interpret the result and produce an answer
```

Step 6: Ask a Follow-Up Question in the Same Thread

If you reuse the same `thread_id`, the agent keeps the conversation context and can continue the analysis from the earlier steps.

Built-In Tools Summary

Built-in Tool	Backend	Description
`read_file`	All backends	Read a file (including images)
`write_file`	All backends	Write a file
`edit_file`	All backends	Edit a file (find and replace)
`ls`	All backends	List directory contents
`glob`	All backends	Search for files by pattern
`grep`	All backends	Search file contents
`execute`	LocalShell, Sandbox	Run shell commands
`write_todos`	All backends	Create or update a task plan

Data Analysis Execution Flow

```
1. [Planning]    write_todos — write the analysis plan
2. [Reading]     read_file — inspect the CSV structure
3. [Execution]   execute — run pandas code
4. [Iteration]   continue analysis through follow-up questions
5. [Delivery]    summarize findings and report results
```

`@tool` + pandas vs. CodeInterpreterMiddleware

There are two main ways a data analysis agent can run code. The `run_pandas` tool in this chapter is the `@tool` + pandas pattern; it can be swapped for the built-in `execute` of `LocalShellBackend` or for `CodeInterpreterMiddleware`.

Pattern	Isolation	State	Use Case
`@tool` + inline Python	None (same process)	New namespace per call	Notebooks, demos, small analyses
`LocalShellBackend` `execute`	None (host shell)	Filesystem only	Letting the agent shell out
`CodeInterpreterMiddleware` (QuickJS)	Inside the interpreter	Auto-restored between turns via snapshots	Safer code execution on Deep Agents 0.6+

> ➡️ **TIP**
>
> Put your analysis checklist and code-execution rules in `skills/data-analysis/SKILL.md`. The agent loads them progressively when needed instead of cramming everything into the system prompt — that's the Skills pattern.

Summary

Item	Key Point
Backend	`LocalShellBackend(virtual_mode=True)` — supports code execution
Built-in tools	`execute` (code execution) + `write_todos` (planning)
Streaming	`stream(subgraphs=True)` — observe the process in real time
Multi-turn	`InMemorySaver` + same `thread_id` — preserve conversation context

References:

- `docs/deepagents/tutorials/data-analysis.md`
- `docs/deepagents/06-backends.md`

Next Step: → 04_ml_agent.ipynb: Build a machine learning agent.

04 Machine Learning Agent

A Flexible CSV-Based ML Workflow

Learning Objectives

- Set a data directory with `FilesystemBackend` and let the agent explore files freely
- Extend the `run_pandas` pattern from NB03 to create a `run_ml_code` tool with scikit-learn
- Let the agent explore data with built-in tools (`ls` , `read_file` , `glob`) and analyze it with `run_ml_code`
- Run EDA → preprocessing → model selection → training → evaluation through multi-turn conversation

Overview

Item	NB03 (Data Analysis)	NB04 (Machine Learning)
Backend	`LocalShellBackend`	`FilesystemBackend`
Data	Sales CSV (8 rows)	User-provided CSV (demo: breast cancer, 569 rows)
Custom tools	`get_csv_path` + `run_pandas`	`run_ml_code` (adds sklearn)
Built-in tools	—	`ls` , `read_file` , `glob` (file exploration)
Goal	Aggregation, statistics, trend analysis	EDA → preprocessing → model training → comparison

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

NB03 vs. NB04: Extending the Backend and Tooling

In NB03, you used `LocalShellBackend` + `run_pandas` to execute pandas code. In NB04, you extend that setup in two ways:

1. Backend: `FilesystemBackend(root_dir=DATA_DIR)` — the agent can freely explore the data directory with built-in tools such as `ls`, `read_file`, and `glob`
2. Tool: `run_ml_code` — adds `sklearn` to the execution namespace so the agent can run ML pipelines

```
# NB03: LocalShellBackend + run_pandas
backend = LocalShellBackend(root_dir=tmp_dir, virtual_mode=True)
ns = {"pd": pd, "np": np, "csv_path": csv_path}

# NB04: FilesystemBackend + run_ml_code
backend = FilesystemBackend(root_dir=DATA_DIR, virtual_mode=True)
ns = {"pd": pd, "np": np, "sklearn": sklearn, "DATA_DIR": DATA_DIR}
```

> ➡️ TIP
>
> Because `FilesystemBackend` does not expose `execute`, it is safer than `LocalShellBackend`.

Step 1: Set the Data Directory

If you change `DATA_DIR`, you can point the agent to your own CSV data. The agent will inspect the directory with built-in tools such as `ls`, `glob`, and `read_file` to understand which files exist.

```
# Example: point to your own data directory
DATA_DIR = "/path/to/your/data"
```

```
import tempfile
import pandas as pd
from sklearn.datasets import load_breast_cancer

# ── Data directory setup ─────────────────────────────
# Change this to a directory that contains your own CSV files.
# For this demo, we save the breast_cancer dataset as a CSV.
DATA_DIR = tempfile.mkdtemp()

# Demo data creation (remove this block if you already have your own CSV)
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df["target"] = data.target
df.to_csv(os.path.join(DATA_DIR, "breast_cancer.csv"), index=False)

print(f"DATA_DIR: {DATA_DIR}")
print(f"Files: {os.listdir(DATA_DIR)}")
```

Step 2: Create the `FilesystemBackend`

`FilesystemBackend` provides built-in file tools below the `root_dir`:

Built-in Tool	Role
`ls`	List directory contents
`read_file`	Read file contents
`glob`	Find files by pattern
`write_file`	Write files (for saving results)

> ➡️ TIP
>
> `virtual_mode=True` prevents path escape patterns such as `..` or `~`.

```
from deepagents.backends import FilesystemBackend

backend = FilesystemBackend(root_dir=DATA_DIR, virtual_mode=True)
```

Step 3: Define `run_ml_code`

Extend the `run_pandas` pattern from NB03 by adding `sklearn` to the execution namespace. Pass `DATA_DIR` into the namespace so the agent can load any CSV file inside the directory.

> ➡️ TIP
>
> Use built-in `ls` / `read_file` for file exploration and `run_ml_code` for code execution — keep the responsibilities separate.

```
from langchain.tools import tool
import io, contextlib

@tool
def run_ml_code(code: str) -> str:
    """Execute sklearn/pandas Python code. Use print() to display results.
    Available modules: pd, np, sklearn, os. Access the data directory via DATA_DIR."""
    import pandas as pd, numpy as np, sklearn
    buf = io.StringIO()
    ns = {"pd": pd, "np": np, "sklearn": sklearn, "os": os, "DATA_DIR": DATA_DIR}
    try:
        with contextlib.redirect_stdout(buf):
            exec(code, ns)
        return buf.getvalue() or "Execution finished (no printed output)"
    except Exception as e:
        return f"Error: {e}"
```

Step 4: Create the Agent

The agent workflow is:

1. Explore CSV files inside `DATA_DIR` with built-in `ls` / `glob`
2. Preview data with built-in `read_file`
3. Run EDA → preprocessing → model training/comparison with `run_ml_code`

Middleware	Role
`ToolRetryMiddleware`	Automatically retries failed tool calls (up to 2 times)
`ModelCallLimitMiddleware`	Prevents infinite loops by limiting the run to 20 model calls

```
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    ToolRetryMiddleware,
    ModelCallLimitMiddleware,
)
from prompts import ML_AGENT_PROMPT

ml_agent = create_deep_agent(
    model=model,
    tools=[run_ml_code],
    system_prompt=ML_AGENT_PROMPT,
    backend=backend,
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        ToolRetryMiddleware(max_retries=2),
        ModelCallLimitMiddleware(run_limit=20),
    ],
)
```

Step 5: File Exploration + EDA

Ask the agent to explore the data directory and analyze the dataset. The agent can inspect the file list with built-in `ls` and then perform EDA with `run_ml_code`.

Step 6: Train and Compare Models

Ask the agent to train at least three suitable models and compare them with cross-validation. The agent chooses the algorithms by itself.

Step 7: Multi-Turn Follow-Up — Feature Importance

Reuse the same `thread_id` so the follow-up analysis keeps the earlier conversation context.

Step 8: Streaming — Additional Analysis

Observe the execution process in real time with `stream(subgraphs=True)`.

`@tool` + scikit-learn Pattern

`run_ml_code` exposes pandas, numpy, and sklearn through a single `@tool`. The agent can run a continuous sklearn pipeline — from EDA through training and evaluation — in a single tool call.

```
@tool
def run_ml_code(code: str) -> str:
    """Run pandas + numpy + sklearn code."""
    import pandas as pd, numpy as np, sklearn
    ns = {"pd": pd, "np": np, "sklearn": sklearn, "os": os, "DATA_DIR": DATA_DIR}
    # ... capture print output and return the result
```

➡️ TIP

- Built-in `ls` / `read_file` → file exploration
- `run_ml_code` → code execution
- Separating these responsibilities makes it obvious to the agent which tool to call at each step.

Summary

Item	Key Point
Backend	`FilesystemBackend(root_dir=DATA_DIR)` — point the agent at your data directory
Built-in tools	`ls`, `read_file`, `glob` — explore files
Custom tool	`run_ml_code` (pandas + numpy + sklearn) — execute ML code
Workflow	File exploration → EDA → preprocessing → model selection → cross-validation comparison
Multi-turn	`InMemorySaver` + same `thread_id` — preserve analysis context

Using Your Own Data

```
# Just change DATA_DIR in Step 1
DATA_DIR = "/path/to/your/data"  # directory containing CSV files
```

The agent will explore files with `ls` and analyze them freely with `run_ml_code`.

References:

- `docs/deepagents/06-backends.md`
- `docs/deepagents/tutorials/data-analysis.md`
- scikit-learn documentation

Next Step: → 05_deep_research_agent.ipynb: Build a deep research agent.

05 Deep Research Agent

Parallel Subagents and a Five-Step Workflow

Learning Objectives

- Configure three parallel subagents (`researcher-1` , `researcher-2` , `fact-checker`)
- Implement strategic reflection with `think_tool`
- Design a five-step workflow (Plan → Delegate → Synthesize → Verify → Report)
- Apply v1 middleware (`SummarizationMiddleware` , `ModelCallLimitMiddleware` , `ModelFallbackMiddleware`)

Overview

Item	Details
Framework	Deep Agents
Core components	Three parallel subagents, `think_tool`
Workflow	5 steps: Plan → Delegate → Synthesize → Verify → Report
Backend	`FilesystemBackend(root_dir=".", virtual_mode=True)`
Built-in tools	`write_todos` (planning), `task` (subagent call)
Skill	`skills/deep-research/SKILL.md` — research method + citation rules

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

Step 1: `think_tool` — A Strategic Reflection Tool

`think_tool` lets the agent record "thoughts" before it acts. This pattern can improve the quality of agent decision-making:

- Analyze search results and plan the next action
- Evaluate whether the collected information is sufficient
- Make delegated tasks more specific before sending them to subagents

```python
from langchain.tools import tool

@tool
def think_tool(thought: str) -> str:
    """Strategic reflection — analyze the current situation and plan the next action."""
    return f"Reflection recorded: {thought}"
```

Step 2: A Simplified `web_search` Tool

In a real deep research workflow, you would typically use the Tavily API. Here, for learning purposes, we define a simplified search tool.

```python
@tool
def web_search(query: str) -> str:
    """Perform a web search (simulated)."""
    results = {
        "AI agent": "AI agents are systems that perform tasks autonomously. Their adoption has
accelerated rapidly since 2024.",
        "LangGraph": "LangGraph is a stateful workflow framework. It supports both the Graph API and the
Functional API.",
        "Deep Agents": "Deep Agents is an all-in-one agent SDK. It supports subagents, backends, and
skills.",
    }
    for key, val in results.items():
        if key.lower() in query.lower():
            return val
    return f"Search result for '{query}': no relevant information found."
```

Step 3: Prompt for the Five-Step Research Workflow

The prompt loader pulls the prompt using the following order: LangSmith Hub → Langfuse → local default.

Step	Name	Description
1	Plan	Create a research plan with `write_todos`
2	Delegate	Parallelize investigation across subagents (up to 3 at once)
3	Synthesize	Combine the gathered information
4	Verify	Ask the fact-checker to verify the claims

5	Report	Produce the final report

```
from prompts import RESEARCH_AGENT_PROMPT

print(RESEARCH_AGENT_PROMPT)
```

Step 4: Define Three Subagents

The deep research agent uses three specialized subagents:

Subagent	Role	Tools
`researcher-1`	Primary topic research	`web_search` , `think_tool`
`researcher-2`	Complementary or contrasting research	`web_search` , `think_tool`
`fact-checker`	Verify factual accuracy	`web_search`

```
researcher_1 = {
    "name": "researcher-1",
    "description": "Performs in-depth research on the topic",
    "system_prompt": "You are a research specialist. Investigate the topic deeply and summarize the key
findings. Reflect with think_tool after searching.",
    "tools": [web_search, think_tool],
}
```

```
researcher_2 = {
    "name": "researcher-2",
    "description": "Performs complementary research from another perspective",
    "system_prompt": "You are a complementary researcher. Collect additional information from a different
angle. Reflect with think_tool after searching.",
    "tools": [web_search, think_tool],
}
```

```
fact_checker = {
    "name": "fact-checker",
    "description": "Verifies the factual correctness of collected information",
    "system_prompt": "You are a fact-checker. Verify the accuracy of the provided information and point
out any errors.",
    "tools": [web_search],
}
```

Step 5: Create the Deep Research Agent (with v1 Middleware)

Combine the tools and subagents into the final agent. The v1 middleware improves stability and reliability:

Middleware	Role

`SummarizationMiddleware`	Automatically summarizes long research conversations to save context
`ModelCallLimitMiddleware`	Prevents research loops by limiting the run to 30 model calls
`ModelFallbackMiddleware`	Falls back to a backup model if the primary model fails

Enable checkpointing with `InMemorySaver` so interrupted research runs can resume.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    SummarizationMiddleware,
    ModelCallLimitMiddleware,
    ModelFallbackMiddleware,
)
from deepagents.middleware import TodoListMiddleware

research_agent = create_deep_agent(
    model=model,
    tools=[web_search, think_tool],
    subagents=[researcher_1, researcher_2, fact_checker],
    system_prompt=RESEARCH_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        TodoListMiddleware(),
        SummarizationMiddleware(model=model, trigger=("messages", 15)),
        ModelCallLimitMiddleware(run_limit=30),
        ModelFallbackMiddleware("gpt-5.4-mini"),
    ],
)
```

DeepAgents Pattern — TodoList + dispatch + 5-Tool Async

The deep research agent in this chapter combines the three core DeepAgents patterns:

Pattern	Description
TodoList	The built-in `write_todos` makes the plan explicit during the Plan step — progress is anchored in the message history
dispatch	The built-in `task` delegates work to subagents — only the summary returns to the main agent, keeping its context clean
5-tool async	Five tools — `web_search`, `think_tool`, `write_todos`, `task`, `read/write_file` — drive the async workflow

➡️ TIP

> Pass an `AsyncSubAgent` spec to `subagents` and `create_deep_agent` automatically wires in `AsyncSubAgentMiddleware`, so the subagents run in parallel asynchronously (see `docs/deepagents/12-async-subagents.md`).

Step 6: Run the Research Workflow

Give the agent a research topic and let it execute the five-step workflow.

Step 7: Streaming — Track Namespaces

With `stream(subgraphs=True)`, you can follow the execution of the main agent and the subagents by namespace. This makes it easy to see when each subagent is called.

Subagent Design Best Practices

Principle	Description
Clear descriptions	Write specific `description` values so the main agent knows when to delegate
Specialized prompts	Put output format, constraints, and workflow expectations into `system_prompt`
Minimal tools	Give each subagent only the tools it actually needs
Concise outputs	Tell subagents to return summaries rather than raw data

Summary

Item	Key Point
`think_tool`	Strategic reflection — analyze search results and plan the next step
Subagents	Parallel execution with `researcher-1`, `researcher-2`, and `fact-checker`
Workflow	Plan → Delegate → Synthesize → Verify → Report
Context management	Only subagent results are passed back to the main agent; intermediate work stays isolated

References:

- `docs/deepagents/examples/02-deep-research.md`
- `docs/deepagents/07-subagents.md`
- `docs/deepagents/06-backends.md`

Previous Step: ← 04_ml_agent.ipynb: Machine learning agent

06 Multimodal PDF RAG

PyMuPDF4LLM + v1 Content Blocks

Learning Objectives

- Convert PDF slides into a one slide = one Document structure with PyMuPDF4LLM
- Extract images and tables from each slide and merge them into RAG search text
- Send slide images to a vision-capable LLM through multimodal content blocks
- Enable LangChain v1 `output_version="v1"` serialization and inspect standard content blocks (`TextContentBlock` / `ImageContentBlock`)
- Implement question answering with `create_agent()` and a `content_and_artifact` retrieval tool

Overview

Item	Details
Target document	Public Stanford CS224N 2026 Lecture 5: Attention and Transformers PDF
Extraction tool	`pymupdf4llm.to_markdown(page_chunks=True, write_images=True)`
Document unit	One PDF page/slide → one LangChain `Document`
Multimodal enrichment	Extracted image paths + rendered slide PNG + LLM-generated image descriptions
Content blocks	Multimodal messages with `{"type":"text", ...}` + `{"type":"image_url", ...}`
Response serialization	`ChatOpenAI(..., output_version="v1")` — standard `TextContentBlock` / `ImageContentBlock` format
RAG structure	`OpenAIEmbeddings` → `InMemoryVectorStore` → `\@tool` → `create_agent()`
Skill	`skills/multimodal-rag/SKILL.md` — slide-based multimodal RAG rules

```
# [6-1] : Environment setup
from dotenv import load_dotenv
import os
import pymupdf4llm
```

```
load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
from langchain_openai import ChatOpenAI

# output_version="v1" serializes responses as standard content blocks
# (TextContentBlock / ImageContentBlock / ReasoningContentBlock, and more).
# Setting LC_OUTPUT_VERSION="v1" in the environment has the same effect.
model = ChatOpenAI(model="gpt-5.4", output_version="v1")
vision_model = model  # gpt-5.4 supports vision inputs
print("Model ready — output_version=v1")
```

LangChain v1 Multimodal Content Blocks

`HumanMessage.content` accepts three practical forms.

1. string — simple text such as `"What limitation does self-attention have in Transformers?"`
2. dict list — OpenAI-compatible multimodal content such as `[{"type":"text", ...}, {"type":"image_url", ...}]`
3. standard content blocks — LangChain's type-safe representations such as `TextContentBlock`, `ImageContentBlock`, `AudioContentBlock`, and `FileContentBlock`

When sending slide images to a vision model, form 2 is the most direct. The helper `multimodal_pdf_rag.describe_image_with_llm` uses it internally.

```
message = [
    {"type": "text", "text": "Explain this slide in English."},
    {"type": "image_url", "image_url": {"url": "...."}},
]
response = vision_model.invoke([{"role": "user", "content": message}])
```

With `output_version="v1"`, `response.content` is serialized into standard content blocks so downstream systems can consume `TextContentBlock` / `ImageContentBlock` safely across providers (`docs/langchain/05-messages.md`).

1) Download the Public PDF

This example uses public Stanford CS224N lecture slides. The source PDF and generated images are runtime cache artifacts under `en/07_examples/.cache/` ; they are not committed to the repository.

```
import sys
from pathlib import Path

root = Path.cwd()
candidates = [root, root / "en" / "07_examples", root.parent / "en" / "07_examples"]
for candidate in candidates:
    if (candidate / "multimodal_pdf_rag.py").exists():
        sys.path.insert(0, str(candidate))
        break
```

```
from multimodal_pdf_rag import (
    DEFAULT_PAGES, DEFAULT_SOURCE_URL, add_llm_image_descriptions,
    build_vectorstore, download_pdf, extract_pdf_metadata,
    extract_slide_documents, extract_slide_metadata, format_pdf_metadata,
    format_slide_metadata, format_sources, make_slide_search_tool,
    metadata_as_json,
)
```

```
pdf_path = download_pdf(DEFAULT_SOURCE_URL)
selected_pages = DEFAULT_PAGES
if os.environ.get("FAST_MULTIMODAL_RAG_TEST"):
    selected_pages = [19, 51, 58]
print(f"PDF: {pdf_path}")
print(f"Target slides (0-based): {selected_pages}")
```

2) Extract PDF File Metadata

Start by extracting PDF-level metadata such as title, author, creation date, page count, file size, and page dimensions. These fields are useful for corpus provenance and future filtering rules.

```
pdf_metadata = extract_pdf_metadata(pdf_path, source_url=DEFAULT_SOURCE_URL)
print(format_pdf_metadata(pdf_metadata))
print(metadata_as_json({"pages_sample": pdf_metadata["pages"][:2]}))
```

3) Create Slide-Level Documents

`page_chunks=True` returns page-level Markdown, while `write_images=True` saves image regions from each slide as PNG files. At the same time, PyMuPDF renders each whole slide as a representative image.

> · NOTE
>
> To process all 70 slides, set `pages=None`. The notebook defaults to a focused slide subset to reduce runtime and API cost.

```
documents = extract_slide_documents(
    pdf_path,
    source_url=DEFAULT_SOURCE_URL,
    pages=selected_pages,
)
print(f"Documents created: {len(documents)}")
print(format_sources(documents))
```

```
slide_metadata = extract_slide_metadata(documents)
print(format_slide_metadata(slide_metadata))
print(metadata_as_json(slide_metadata[:2]))
```

```
from IPython.display import Image, display

sample_doc = documents[0]
display(Image(filename=sample_doc.metadata["slide_image_path"], width=480))
print(sample_doc.metadata["extracted_image_paths"][:2])
```

4) Inspect Slide Metadata, Tables, and Images

PyMuPDF4LLM inserts tables into the page body as GitHub Flavored Markdown. The helper module separates Markdown table blocks again and preserves them in both `metadata["table_count"]` and a `## Extracted tables` section.

```
table_docs = [doc for doc in documents if doc.metadata["table_count"] > 0]
print(f"Slides with detected tables: {[d.metadata['slide'] for d in table_docs]}")
if table_docs:
    print(table_docs[0].page_content[:1200])
```

5) Generate LLM-Based Image Descriptions

Raw images are difficult to embed directly in a text vector index. Instead, we ask a vision-capable LLM to describe each rendered slide image and selected extracted images, then merge those descriptions back into the `Document` body.

```
caption_limit = len(documents) + 4

documents = add_llm_image_descriptions(
    documents, vision_model,
    max_descriptions=caption_limit,
)
print(documents[0].metadata["image_descriptions"][0][:700])
```

Inspect Standard Content Blocks from v1 Responses

With `output_version="v1"`, the vision model response exposes `.content_blocks`, a provider-normalized list of `TextContentBlock` items. This representation is safer for downstream pipelines such as caption caching or UI rendering.

```
from multimodal_pdf_rag import _image_to_data_url  # demo helper

sample_image = documents[0].metadata["slide_image_path"]
vision_msg = [
    {"type": "text", "text": "Summarize the main message of this slide in one sentence."},
    {"type": "image_url", "image_url": {"url": _image_to_data_url(sample_image)}},
]
resp = vision_model.invoke([{"role": "user", "content": vision_msg}])

# v1: .content_blocks is a list of standard content blocks such as TextContentBlock
blocks = getattr(resp, "content_blocks", None)
print("block count:", len(blocks) if blocks else "n/a")
if blocks:
    for b in blocks:
        print(f"  type={b.get('type'):>15}  | preview={str(b)[:120]}")
```

```
print()
print("Response preview:", str(resp.content)[:300])
```

6) Build a Vector Index and Retrieval Tool

Following the LangChain v1 structure, we create an `InMemoryVectorStore` and expose retrieval through `@tool(response_format="content_and_artifact")`, which returns both answer text and the original `Document` artifacts.

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = build_vectorstore(documents, embeddings)
search_slides = make_slide_search_tool(vectorstore, k=4)
print("Vector index and search tool ready")
```

```
query = "What problem appears when using self-attention as a Transformer block?"
hits = vectorstore.similarity_search(query, k=3)
print(format_sources(hits))
print(hits[0].page_content[:1400])
```

7) Answer Questions with LangChain v1 `create_agent()`

Once retrieval is exposed as a tool, the agent can search slides for the question and answer with evidence from text, tables, and image descriptions.

```
from langchain.agents import create_agent
from langchain.agents.middleware import ModelCallLimitMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from prompts import MULTIMODAL_RAG_AGENT_PROMPT
```

```
rag_agent = create_agent(
    model=model,
    tools=[search_slides],
    system_prompt=MULTIMODAL_RAG_AGENT_PROMPT,
    middleware=[ModelCallLimitMiddleware(run_limit=6)],
    checkpointer=InMemorySaver(),
)
```

Summary

Item	Key Point
Slide-level documents	`page_chunks=True` turns each PDF page into one `Document`

Image handling	`write_images=True` extracted images + PyMuPDF slide renderings + LLM descriptions
Metadata handling	Extract PDF-level metadata plus slide-level table/image/layout metadata
Table handling	Separate Markdown table blocks and preserve them in content and metadata
Multimodal messages	Call the vision model with `{"type":"text", ...}` + `{"type":"image_url", ...}` content blocks
v1 serialization	`ChatOpenAI(..., output_version="v1")` → access `TextContentBlock` through `.content_blocks`
LangChain v1 structure	`InMemoryVectorStore` + `\@tool(content_and_artifact)` + `create_agent()`
Extension points	Process the full PDF with `pages=None`, persist in Chroma/FAISS, and add a caption cache

References:

- `docs/langchain/05-messages.md` — content blocks and `output_version="v1"`
- `docs/langchain/24-retrieval.md` — the five building blocks of RAG
- `07_examples/skills/multimodal-rag/SKILL.md`
- PyMuPDF4LLM documentation
- PyMuPDF4LLM to_markdown API
- LangChain PyMuPDF4LLM integration
- Stanford CS224N 2026 Lecture 5 PDF

Next step: → 08_integration/04_document_loaders compares PDF, web, and structured document loaders.

07 Content Builder Agent

AGENTS.md + Skills + Subagents

Learning goals

- Understand the official Deep Agents Content Builder pattern: memory, skills, and subagents
- Inject brand voice with `AGENTS.md` and load writing workflows on demand with `SKILL.md`
- Persist outputs safely with `FilesystemBackend(virtual_mode=True)`
- Keep search and image generation optional so the core text pipeline runs with OpenAI only

Overview

Item	Details
Official pattern	Content Builder Agent — memory(`AGENTS.md`), skills, subagents
Runtime scope	Text-only LinkedIn/blog draft generation
Safety	`local/` outputs, `virtual_mode=True` , image generation as reference-only
Verification	One live gpt-4.1 harness run

The official example includes Tavily search and Gemini image generation. This notebook keeps those as optional adapters and makes OpenAI + local filesystem the only required runtime path.

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "Set OPENAI_API_KEY in .env"
```

```
# LangSmith — automatic logging when LANGSMITH_TRACING=true
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_PROJECT", "langchain-langgraph-deepagents-notebooks")
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4", temperature=0)
```

1) Content Builder workspace

The official example keeps agent instructions on disk rather than hard-coding everything in Python.

- `AGENTS.md` — brand voice and writing standards, loaded every run
- `skills/content-builder/SKILL.md` — writing workflow, loaded on demand
- `research/`, `linkedin/`, `blogs/` — output folders created by the agent

```
from pathlib import Path
import shutil

WORK_DIR = Path("local/content_builder_demo_en")
if WORK_DIR.exists():
    shutil.rmtree(WORK_DIR)
(WORK_DIR / "skills/content-builder").mkdir(parents=True)
(WORK_DIR / "research").mkdir()
```

```
brand_voice = """# BAEUM.AI Content Voice
Write in English, professional but approachable.
Lead with practical value, keep paragraphs short, and end with action.
Focus on AI agents, developer productivity, and production reliability.
"""
(WORK_DIR / "AGENTS.md").write_text(brand_voice, encoding="utf-8")
```

```
skill_text = """---
name: content-builder
description: Use for blog, LinkedIn, or technical content drafts.
---
# Content Builder
Research first, then write a hook, 3 concise insights, and a CTA.
Save LinkedIn posts to linkedin/<slug>/post.md.
Save blog posts to blogs/<slug>/post.md.
"""
(WORK_DIR / "skills/content-builder/SKILL.md").write_text(skill_text, encoding="utf-8")
```

2) Local research tool and researcher subagent

Live classrooms and automated smoke tests do not always have external search keys. Here we wrap a small local knowledge base as a tool, then let a researcher subagent save concise findings to disk.

```
from langchain.tools import tool

@tool
def topic_brief(topic: str) -> str:
    """Return a concise local research brief for a content topic."""
    return (
        f"{topic}: Deep Agents combine memory, skills, filesystem tools, "
        "and subagents so content workflows become repeatable."
    )
```

```
researcher = {
    "name": "researcher",
    "description": "Research a topic and save concise findings before writing.",
    "system_prompt": "Use topic_brief, then write findings to the requested path.",
    "tools": [topic_brief],
}
```

3) Configure the Deep Agent

`create_deep_agent()` wires the content-building pieces together.

Configuration	Role
`memory=["/AGENTS.md"]`	Injects brand voice into the system prompt
`skills=["/skills/"]`	Loads `SKILL.md` workflow guidance on demand
`subagents=[researcher]`	Delegates research through the `task` tool
`FilesystemBackend`	Provides file tools such as `write_file` and `read_file`

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

agent = create_deep_agent(
    model=model, tools=[topic_brief], subagents=[researcher],
    backend=FilesystemBackend(root_dir=str(WORK_DIR), virtual_mode=True),
    memory=["/AGENTS.md"], skills=["/skills/"],
    system_prompt="You are a content builder. Use files for durable outputs.",
)
```

4) Run — create a LinkedIn post

The request states four things explicitly: use the researcher, where to save research, where to save the content, and the bounded output shape.

```
request = """Use the researcher subagent to research Deep Agents content workflows.
Save research to research/content-workflows.md.
Then write an English LinkedIn post with one hook, three bullets, and one CTA.
Save it to linkedin/content-workflows/post.md."""
result = agent.invoke({"messages": [{"role": "user", "content": request}]})
print(result["messages"][-1].content[:800])
```

```
post_path = WORK_DIR / "linkedin/content-workflows/post.md"
research_path = WORK_DIR / "research/content-workflows.md"
print("research exists:", research_path.exists())
print("post exists:", post_path.exists())
print(post_path.read_text(encoding="utf-8")[:900])
```

5) Search and images are optional adapters

A production Content Builder can add Tavily search and image generation tools. For the core 01 07 notebook harness, keep those adapters reference-only unless the matching keys and cost policy are available.

```
optional_tools_example = r"""
# Optional production tools — not required for this notebook smoke run.
# Add Tavily or image generation only when the matching API keys exist.
agent = create_deep_agent(
    model=model,
    tools=[web_search, generate_cover, generate_social_image],
    memory=["/AGENTS.md"], skills=["/skills/"],
)
"""
print(optional_tools_example)
```

Summary

Topic	Key idea
Memory	`AGENTS.md` always injects brand voice
Skills	`SKILL.md` loads blog/social workflows on demand
Subagents	A researcher separates evidence gathering from drafting
FilesystemBackend	Durable outputs are saved under `local/`
Runtime policy	OpenAI is required; search and images stay optional

References:

- Deep Agents Content Builder: https://docs.langchain.com/oss/python/deepagents/content-builder
- Deep Agents Skills: ../../docs/deepagents/10-skills.md
- Deep Agents Subagents: ../../docs/deepagents/07-subagents.md
- Local reference: ../../docs/deepagents/examples/01-content-builder-agent.md

10 Personal Assistant Subagents

Delegate by role

A personal assistant often handles mixed requests: calendar work, email drafting, research, and follow-up planning. This example uses role-based subagents to keep those responsibilities separate.

Learning goals

- Define subagent roles around user-facing responsibilities.
- Route requests to the right role.
- Merge multiple role outputs into one assistant response.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

10.1 Define roles

Good subagent design starts with role boundaries. Each role should have a clear job, input shape, and output expectation.

```
subagents = {
    "calendar": {"tools": ["check_availability"], "risk": "approval"},
    "email": {"tools": ["draft_reply"], "risk": "review"},
    "research": {"tools": ["search_notes"], "risk": "allow"},
}

subagents
```

10.2 Route requests

Routing turns a user request into an ownership decision. Deterministic routing is enough for this small example and keeps the behavior easy to inspect.

```
def route_request(text: str) -> str:
    lowered = text.lower()
    if "meeting" in lowered or "schedule" in lowered:
        return "calendar"
    if "email" in lowered or "email" in lowered:
        return "email"
    return "research"

route_request("Check tomorrow schedule and prepare an email draft")
```

10.3 Decompose compound requests

Real requests often contain more than one job. Decomposition lets the assistant delegate each part without losing the overall user intent.

```
def plan_tasks(text: str) -> list[dict]:
    tasks = []
    for name in subagents:
        if name == route_request(text) or name in text.lower():
            tasks.append({"subagent": name, "input": text})
    return tasks or [{"subagent": "research", "input": text}]

plan_tasks("Check calendar and draft email")
```

10.4 Merge fan-in results

The final response should read like one assistant, not a pile of subagent logs. Fan-in synthesis combines role outputs into a useful summary.

```
results = [
    {"subagent": "calendar", "result": "Tuesday at 3 PM is available"},
    {"subagent": "email", "result": "Drafted a meeting proposal email"},
]

summary = "\n".join(f"- {r['subagent']}: {r['result']}" for r in results)
print(summary)
```

Summary

Item	Content
Covered	supervisor routing, role-specific subagents, tool scopes, and fan-in summaries
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `multi-agent-middleware.md`
- `subagents.md`
- `async-subagents.md`

11 Customer Support Handoffs

Multi-agent state transitions

Customer support agents often need to move a case between general support, billing, technical support, and human approval. This example models handoffs as explicit state transitions.

Learning goals

- Define handoff criteria for support workflows.
- Represent ownership changes as graph transitions.
- Add approval gates for sensitive actions.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class SupportState(TypedDict):
    message: str
    owner: str
    resolution: str
```

11.1 Handoff criteria

Handoffs should be based on clear signals such as refund requests, technical errors, or escalation language. Ambiguous ownership creates slow and inconsistent support.

```
def triage(state: SupportState) -> dict:
    msg = state["message"].lower()
    if "refund" in msg or "refund" in msg:
        return {"owner": "billing"}
    if "error" in msg or "error" in msg:
        return {"owner": "technical"}
    return {"owner": "general"}
```

```
def resolve(state: SupportState) -> dict:
    templates = {
        "billing": "Check the refund policy and prepare an approval request.",
        "technical": "Ask for reproduction steps and logs.",
        "general": "Provide the standard guidance.",
    }
    return {"resolution": templates[state["owner"]]}
```

11.2 Represent the flow as a graph

A graph makes ownership and transition rules visible. Each route can be tested without relying on a full support transcript.

```
builder = StateGraph(SupportState)
builder.add_node("triage", triage)
builder.add_node("resolve", resolve)
builder.add_edge(START, "triage")
builder.add_edge("triage", "resolve")
builder.add_edge("resolve", END)
graph = builder.compile()
```

```
result = graph.invoke({
    "message": "I would like a subscription refund.",
    "owner": "", "resolution": "",
})
result
```

11.3 Approval gate

Some support actions require human confirmation. An approval gate records why the case is paused and what decision is needed next.

```
def needs_approval(state: SupportState) -> bool:
    return state["owner"] == "billing"

print("approval required:", needs_approval(result))
```

Summary

Item	Content
Covered	handoff state machines, escalation, owner transitions, and approval gates
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `handoffs.md`
- `interrupts.md`

- `human-in-the-loop.md`

12 Router Knowledge Base

Send questions to the right source

A knowledge-base assistant is only useful if it searches the right source. This example routes questions to policy, product, or troubleshooting knowledge before retrieval.

Learning goals

- Define source-specific knowledge boundaries.
- Route questions before searching.
- Evaluate router decisions separately from answer quality.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

12.1 Define knowledge sources

Knowledge sources should have clear scope. Separating policy, product, and troubleshooting content reduces irrelevant retrieval.

```
knowledge_sources = {
    "billing": ["Refunds are available within 7 days of payment."],
    "technical": ["Error reports require logs and reproduction steps."],
    "product": ["Deep Agents include planning, files, and subagents."],
}

list(knowledge_sources)
```

12.2 Deterministic router

The router chooses where to search. In production this could be model-assisted, but deterministic routing is ideal for learning and testing the contract.

```
def route_source(question: str) -> str:
    q = question.lower()
    if "refund" in q or "refund" in q:
        return "billing"
    if "error" in q or "error" in q:
        return "technical"
    return "product"

route_source("What features does Deep Agents provide?")
```

12.3 Source-local search

Once a source is selected, search only inside that source. This makes retrieval behavior easier to debug and evaluate.

```
def retrieve(question: str) -> dict:
    source = route_source(question)
    docs = knowledge_sources[source]
    return {"source": source, "documents": docs}

retrieve("Tell me the refund conditions")
```

12.4 Router evaluation

Router evaluation asks a narrow question: did the system choose the right source? Keeping this separate from answer evaluation makes failures easier to diagnose.

```
cases = [
    ("Can I get a refund?", "billing"),
    ("The app has an error", "technical"),
    ("What are Deep Agents?", "product"),
]

for question, expected in cases:
    actual = route_source(question)
    print(question, actual, actual == expected)
```

Summary

Item	Content
Covered	source routing, source-local retrieval, routing tests, and knowledge-base specialization
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `custom-multi-agent.md`
- `retrieval.md`
- `workflows-agents.md`

13 Skills SQL Assistant

Load SQL guidance on demand

Skills let an agent load specialized instructions only when they are relevant. This example applies that idea to a SQL assistant with read-only safety rules.

Learning goals

- Express SQL safety guidance as an on-demand skill policy.
- Check queries before execution.
- Separate safe execution from answer synthesis.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE tracks (id INTEGER, name TEXT, genre TEXT)")
conn.executemany("INSERT INTO tracks VALUES (?, ?, ?)", [(1,"A","Jazz"),(2,"B","Rock")])
conn.commit()
```

13.1 SQL skill policy

The skill policy describes what the assistant may and may not do. For SQL, that usually means read-only access, bounded queries, and explicit refusal for mutation commands.

```
sql_policy = [
    "SELECT only by default",
    "LIMIT required for exploration",
    "No INSERT/UPDATE/DELETE without approval",
]

sql_policy
```

13.2 Query safety check

Run a safety check before any database call. This teaching example enforces read-only `SELECT` statements, blocks common mutation commands, and requires `LIMIT` so exploratory queries stay bounded. In production, pair this kind of check with database permissions, query timeouts, row caps, and parser-based validation.

```python
def is_safe_query(sql: str) -> bool:
    lowered = sql.lower().strip()
    forbidden = ["insert", "update", "delete", "drop", "alter"]
    has_limit = " limit " in f" {lowered} "
    return lowered.startswith("select") and has_limit and not any(word in lowered for word in forbidden)

print(is_safe_query("SELECT * FROM tracks LIMIT 5"))
print(is_safe_query("SELECT * FROM tracks"))
```

13.3 Read-only execution

Read-only execution keeps the example safe while still demonstrating the end-to-end flow. The database returns evidence that the final answer can cite.

```python
query = "SELECT genre, COUNT(*) AS n FROM tracks GROUP BY genre LIMIT 10"
assert is_safe_query(query)

rows = conn.execute(query).fetchall()
rows
```

13.4 Answer synthesis

Synthesis turns query results into a user-facing explanation. Keep the answer tied to the returned rows rather than inventing unsupported details.

```python
answer = {
    "query": query,
    "rows": rows,
    "policy_checked": True,
}

answer
```

Summary

Item	Content
Covered	SKILL.md progressive disclosure, SQL safety policy, static query checks, and read-only execution
Core idea	Start from a small deterministic contract before adding model calls or external services.
Next step	Follow the linked course notebooks and official reference notes listed in this chapter.

Reference docs

- `skills.md`
- `sql-agent.md`
- `sql-agent.md`

PART

VIII

Integrations

LangChain Ecosystem Integrations

What You Will Learn in This Part

- 1. Integration Category Overview
- 2. Anthropic Prompt Caching
- 3. Claude Bash Tool
- 4. Claude Text Editor
- 5. Claude Memory
- 6. Anthropic File Search
- 7. Bedrock Prompt Caching
- 8. OpenAI Moderation

01 Integration Category Overview

A map of the LangChain ecosystem

LangChain · LangGraph · Deep Agents take the shape of a common agent interface beneath which per-provider implementations plug in. Parts II-IV focused on "how to use the agent"; this Part focuses on "what you connect the agent to". We scan the thirteen integration categories and lay out the representative packages and selection criteria for each.

Learning Objectives

LEARNING OBJECTIVES

- → Survey the thirteen integration categories of the LangChain ecosystem
- → Understand why Provider Middleware is a separate category
- → Distinguish vendor-sensitive areas (Chat Models, Vector Stores) from standardized ones (Middleware, Checkpointers)
- → Map the seven Provider Middleware chapters this Part covers

1.1 Baseline version snapshot

The code in this Part assumes the following versions. Check the latest releases in `docs/skills/langchain-dependencies.md`.

- `langchain` 1.3+ (event streaming v3)
- `langgraph` 1.2+ (per-node timeout · `DeltaChannel` · graceful shutdown)
- `deepagents` 0.6.1+ (`CodeInterpreterMiddleware` , async subagents, interpreter snapshots)

! WARNING

LangChain 1.2 `create_agent` rejects duplicate instances of the same middleware class. If you hit `AssertionError: Please remove duplicate middleware instances.` , split the two instances into separate types via subclassing.

1.2 The thirteen integration categories

Category	Representative packages	Selection criteria
Chat Models	`langchain-openai`, `langchain-anthropic`, `langchain-google-genai`, `langchain-aws`	Model performance · cost · regional requirements
Embeddings	`langchain-openai`, `langchain-cohere`, `langchain-voyageai`	Domain benchmarks · license · multilingual support
Vector Stores	`langchain-chroma`, `langchain-pinecone`, `langchain-pgvector`	Operational scale · metadata filtering · managed option
Document Loaders	`langchain-community.document_loaders`	Source format (PDF, Notion, Slack, etc.)
Retrievers	`langchain.retrievers`, vendor-specific retrievers	Hybrid search · MMR · reranker integration
Text Splitters	`langchain-text-splitters`	Language · structure (code vs prose) · overlap strategy
Tools	`langchain-community.tools`, MCP tools	External API integration · auth methods · call constraints
Checkpointers	`langgraph-checkpoint-postgres`, `-sqlite`	Durability needs · multi-tenancy · backup strategy
Stores	`langgraph-store-postgres`, vendor-managed	Persistent memory scale · lookup method (key / vector)
Sandboxes	`langchain-daytona`, `langchain-modal`, `langchain-runloop`	Isolation level · cold-start · cost
Provider Middleware	`langchain-anthropic`, `langchain-aws`, `langchain-openai`	Provider-specific features (caching · native tools · policy)
Observability	`langsmith`, `langfuse`, OpenTelemetry	SaaS vs self-hosted · PII policy
Providers (Hosted Runtime)	LangGraph Platform, Anthropic Skills, OpenAI Agents	Managed runtime vs self-hosted · SDK compatibility

Provider Middleware wraps features that are activated on the provider's servers (prompt caching, native tools, content policy) into the LangChain middleware format. Official docs often mention them in a single line, so there is a lot of value in organizing them as working code. Chapters 2–8 of this Part each cover one of the seven middleware.

1.3 Provider Middleware roadmap

Chapter	Middleware	Effect
2	`AnthropicPromptCachingMiddleware`	Server-side cache for long system prompts / tool definitions (5m / 1h)
3	`ClaudeBashToolMiddleware`	Claude native bash tool + three execution policies
4	`StateClaudeTextEditorMiddleware` / `FilesystemClaudeTextEditorMiddleware`	Six operations of the native text editor
5	`StateClaudeMemoryMiddleware` / `FilesystemClaudeMemoryMiddleware`	Model self-memory via the `/memories/*` path contract
6	`StateFileSearchMiddleware`	glob / grep over virtual files in state
7	`BedrockPromptCachingMiddleware`	Cache for Claude / Nova through AWS Bedrock
8	`OpenAIModerationMiddleware`	Pre- / post-check via OpenAI Moderation API

1.4 New integration surfaces: UI · Code · resilience

As of 2026, integration scope does not stop at provider middleware. These surfaces do not change the thirteen-category count in this Part; treat them as connection styles layered on top of the existing categories.

Surface	Core idea	Where it connects in this book
Generative UI / AG-UI	controlled · declarative · open-ended UI generation and the frontend/server event boundary	Parts 2/3 agent state and streaming; Part 8 provider/ runtime integration
Deep Agents Code	`/oss/deepagents/code` configuration · approval ·	Part 4 permissions/skills/ subagents/backends,

	extensions · plugins · MCP tools	specialized for coding-agent runtime
RAG · fault tolerance · multimodal	retrieval, checkpointer/ retry/resume, and multimodal `read_file` combined with the Deep Agents filesystem	Parts 4/5/7 core agent, advanced patterns, and examples

Generative UI splits by how much control the model has over the UI: controlled, declarative, and open-ended. AG-UI connects that flow through an event protocol between a frontend and an agent server. This chapter therefore marks it as a frontend integration track after state, streaming, and hosted runtime concepts, without inventing a new notebook or setup path.

Deep Agents Code covers production surfaces for coding agents: `configuration`, `approval`, `extensions`, `plugins`, and `MCP tools`. Those are the same conceptual families as permissions, skills, subagents, and backends from Part 4, narrowed to a coding-agent runtime. No new credential or environment variable is assumed here.

RAG, fault tolerance, and multimodal are combinations of existing axes, not new categories. RAG combines Retriever/Vector Store components with Deep Agents file context. Fault tolerance connects LangGraph checkpointers, retries, and resumability to production operations. Multimodal is the question of passing image/PDF/audio/video files through `read_file` within each provider's limits.

Official reference	What to check
LangChain Generative UI	controlled/declarative/open-ended UI patterns
Deep Agents AG-UI	frontend/server integration event boundary
Deep Agents Code configuration	coding-agent runtime configuration
Code approval modes	pre-execution approval policy and permission boundaries
Code extensions	runtime feature extension and skill/memory wiring
Code plugins	external tool bundles and deployment units
Deep Agents fault tolerance	retry, resume, checkpointer, and operational resilience
Deep Agents multimodal	file backends and multimodal input handling
Deep Agents RAG	retrieval, file context, and subagent composition

1.5 Common pattern

Every chapter in this Part shares the same three-step loop.

1. **Environment setup** — put the provider key in `.env` and install the package
2. **Basic usage** — flip the feature on with one line, `create_agent(..., middleware=[Middleware()])`
3. **Validation** — read `usage_metadata`, `tool_calls`, or the Moderation response to confirm the feature actually ran

> ➡️ TIP
>
> Provider-specific middleware needs a matching model. If you attach Anthropic middleware to an OpenAI model, behavior is governed by `unsupported_model_behavior` — either a warning or an exception. In multi-provider pipelines, watch the order when combining with `ModelFallbackMiddleware`.

1.6 Observability env-var standardization

From LangChain 1.1, tracing-related environment variables were standardized from `LANGCHAIN_*` to `LANGSMITH_*`. The old names continue to work for a while for compatibility, but new code uses the new names exclusively.

Variable	Role	Notes
`LANGSMITH_TRACING`	Enable tracing (`true` / `false`)	Auto-instruments every `create_agent` run
`LANGSMITH_API_KEY`	LangSmith account key	`ls__` prefix
`LANGSMITH_PROJECT`	Project name for trace grouping	Per-call overridable via `tracing_context`
`LANGSMITH_ENDPOINT`	Endpoint for self-hosted LangSmith	Optional — unneeded for SaaS

Key Takeaways

- Integrations are organized into thirteen categories, each sharing the "provider plugin + common interface" structure
- Provider Middleware is the category that wraps provider server-side features as LangChain middleware
- Chapters 2–8 cover the seven Provider Middleware with runnable code, one per chapter
- Generative UI/AG-UI, Deep Agents Code, and RAG/fault tolerance/multimodal are not new notebook categories here; read them as extensions of state, streaming, backend, retrieval, and operations concepts
- Duplicate-instance constraints, model matching, and multi-provider fallback are the common concerns of Provider Middleware

02 Anthropic Prompt Caching

Cut input-token costs by 90% with server-side caching

`AnthropicPromptCachingMiddleware` is a Claude-only prompt cache middleware that caches long system prompts, tool definitions, and initial conversation context on Anthropic's servers for 5 minutes to 1 hour, cutting input-token cost and latency substantially. When the same agent is invoked many times with the same system prompt, the input-token price drops by roughly 90% from the second call onward.

Learning Objectives

LEARNING OBJECTIVES

- → Understand the four parameters (`type` , `ttl` , `min_messages_to_cache` , `unsupported_model_behavior`)
- → Verify cache hits by reading `cache_creation` / `cache_read` under `usage_metadata.input_token_details`
- → Pick between the 1-hour TTL beta and the 5-minute default
- → Distinguish `warn` / `ignore` / `raise` behaviors when applied to non-Anthropic models

2.1 When to use it

- Agents that send a long system prompt (multi-thousand-token corporate policy, style guide) every turn
- Setups that repeatedly send a large tool-definition list (20+ JSON schemas)
- RAG context reuse: the same document bundle across many queries
- Multi-turn conversations where the prefix context is stable

2.2 Environment setup

Required packages: `langchain` , `langchain-anthropic` . `ANTHROPIC_API_KEY` must be present in `.env` .

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware
```

```
load_dotenv()
```

2.3 Basic usage

Passing the middleware to `create_agent`'s `middleware` list makes LangChain attach `cache_control: {"type": "ephemeral"}` markers automatically at the system prompt · tool definitions · last message positions.

```
long_system_prompt = (
    "You are a senior software engineer... " * 200  # ~2,000 tokens
)

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    system_prompt=long_system_prompt,
    tools=[],
    middleware=[AnthropicPromptCachingMiddleware()],
)
```

2.4 Verifying cache hits

Call the same agent twice in a row and read `usage_metadata`. The first call shows a large `cache_creation_input_tokens`; subsequent calls show `cache_read_input_tokens`.

```
for i in range(2):
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"Request {i}"}]},
    )
    last = result["messages"][-1]
    print(i, last.usage_metadata.get("input_token_details"))
```

```
0 {'cache_creation': 2048, 'cache_read': 0}
1 {'cache_creation': 0, 'cache_read': 2048}
```

2.5 Full parameters

Parameter	Default	Description
`type`	`"ephemeral"`	The only type Anthropic currently supports
`ttl`	`"5m"`	Cache duration. `"5m"` or `"1h"`. 1 hour is an Anthropic beta
`min_messages_to_cache`	`0`	Attach `cache_control` only when the message count is at least this value

`unsupported_model_behavior`	`"warn"`	For non-Anthropic models: `"warn"` / `"ignore"` / `"raise"`

1-hour TTL example

```
AnthropicPromptCachingMiddleware(
    ttl="1h",
    min_messages_to_cache=2,
)
```

The 1-hour cache is a fit for scenarios where the same system prompt is reused over many hours — long advisory sessions, daily batch analytics. The 5-minute default targets continuous user input in a real-time chatbot.

2.6 Behavior on non-Anthropic models

Because this middleware injects `cache_control` blocks internally, other providers' models either ignore them or error. In pipelines where the model can be swapped, make `unsupported_model_behavior` explicit.

```
# Log a warning and continue
AnthropicPromptCachingMiddleware(unsupported_model_behavior="warn")

# Silently skip
AnthropicPromptCachingMiddleware(unsupported_model_behavior="ignore")

# Fail immediately (CI / tests)
AnthropicPromptCachingMiddleware(unsupported_model_behavior="raise")
```

2.7 Aggregating with `UsageMetadataCallbackHandler`

To aggregate cache-hit rate across a whole thread, attach `langchain.callbacks.UsageMetadataCallbackHandler`. Every LLM call's `usage_metadata` accumulates, with `cache_creation` / `cache_read` preserved as-is.

```
from langchain.callbacks import UsageMetadataCallbackHandler

handler = UsageMetadataCallbackHandler()
agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={"callbacks": [handler]},
)
print(handler.usage_metadata)  # {model_name: {input/output/cache_creation/cache_read}}
```

➡️ TIP

> Receiving Anthropic responses with `output_version="v1"` (`ChatAnthropic(..., output_version="v1")`) places `cache_creation` / `cache_read` inside the standardized `usage_metadata.input_token_details` rather than the message body — multi-provider aggregation code becomes much simpler.

2.8 Differences from Bedrock

When calling Claude via AWS Bedrock, use `BedrockPromptCachingMiddleware` (ch7). The two middleware share parameter names and semantics, but they target different packages and have different TTL constraints (Nova supports only 5m, no tool-definition cache).

Key Takeaways

- Agents that repeatedly send long system prompts / tool definitions cut input-token cost by 90%
- Verify cache hits by watching `cache_creation_input_tokens` move to `cache_read_input_tokens`
- TTL `"5m"` for real-time conversation, `"1h"` for long-running batches / advisory sessions
- Declare `unsupported_model_behavior="warn"` in multi-provider pipelines to avoid silent failures

03 Claude Bash Tool

Native bash_20250124 + execution policies

`ClaudeBashToolMiddleware` injects the Anthropic native `bash_20250124` tool into Claude models. Unlike a generic `ShellToolMiddleware`, Anthropic's servers manage the bash call schema directly, so the prompt stays short and tool-schema tokens approach zero. This chapter compares the three execution policies (Host / Docker / CodexSandbox) and covers `redaction_rules` for masking secrets.

Learning Objectives

LEARNING OBJECTIVES

- → Understand the four main parameters of `ClaudeBashToolMiddleware`
- → Distinguish the isolation levels of `HostExecutionPolicy` / `DockerExecutionPolicy` / `CodexSandboxExecutionPolicy`
- → Mask tokens and keys in output via `RedactionRule`
- → Know how Claude native bash differs from a generic shell tool

3.1 When to use it

- Letting Claude run real shell commands for code execution, file inspection, builds
- Deep-Agents-style long-running workspaces where you need to preserve session state (cwd, env vars)
- Running arbitrary code safely in an isolated Docker container
- Shell output may contain API keys / credentials, so redaction is required

3.2 Environment setup

Required packages: `langchain`, `langchain-anthropic`. The Docker policy example requires a local Docker daemon running.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import ClaudeBashToolMiddleware

load_dotenv()
```

3.3 Host execution policy (simplest)

`HostExecutionPolicy` runs commands in the local process. Fast with no setup, but no isolation — it has full access to network, filesystem, and environment variables, so reserve it for trusted scripts or local development.

```
from langchain.agents.middleware import HostExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            workspace_root="/tmp/work",
            startup_commands=["export PATH=$HOME/.local/bin:$PATH"],
            execution_policy=HostExecutionPolicy(),
        ),
    ],
)
```

- `workspace_root` : default directory for the shell session
- `startup_commands` : commands run automatically at session start (PATH configuration, venv activation, etc.)

3.4 Docker execution policy (recommended, isolated)

`DockerExecutionPolicy` runs commands inside a container, fully isolated from the host filesystem and network. Make it the effective default when executing model-generated code.

```
from langchain.agents.middleware import DockerExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            execution_policy=DockerExecutionPolicy(image="python:3.11"),
            startup_commands=["pip install requests numpy"],
        ),
    ],
)
```

The container is created on the first bash call and cleaned up at session end. For package requirements, add `pip install ...` to `startup_commands`.

3.5 Codex sandbox policy

`CodexSandboxExecutionPolicy` runs commands on Anthropic's managed sandbox runner. It is the alternative when you want isolation without local Docker. The network whitelist and resource limits come with strong defaults.

3.6 Output redaction (`redaction_rules`)

Shell output can leak sensitive values — API keys, tokens, emails. Pass a list of `RedactionRule` objects to mask tool responses before they enter the agent context. LangChain 1.2 unified the `RedactionRule` signature with `PIIMiddleware` — use `(pii_type, strategy, detector)` instead of the removed `pattern=` / `replacement=` kwargs.

```
from langchain.agents.middleware import RedactionRule

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            execution_policy=DockerExecutionPolicy(),
            redaction_rules=[
                RedactionRule(
                    pii_type="openai_api_key",
                    strategy="redact",
                    detector=r"sk-[a-zA-Z0-9]{32,}",
                ),
                RedactionRule(
                    pii_type="github_token",
                    strategy="redact",
                    detector=r"ghp_[a-zA-Z0-9]{36}",
                ),
            ],
        ),
    ],
)
```

3.7 Falling back to a generic `@tool`

To carry the same flow to non-Claude models, define the shell function directly with LangChain 1.3's `@tool` decorator. Injecting Anthropic programmatic tool-calling metadata via the `extras` field (introduced in LangChain 1.2) lets the same function be called with the same shape as the Claude-native version.

```
from langchain_core.tools import tool

@tool(extras={"anthropic_cache_control": {"type": "ephemeral"}})
def run_bash(command: str) -> str:
    """Run a bash command in an isolated container and return stdout/stderr."""
    return run_in_container(command)  # your own implementation
```

> ➡️ TIP

> `@tool(extras=...)` is handy for partial caching in multi-provider pipelines. Claude interprets `cache_control`; other providers ignore it — so the same codebase is exposed to both sides unchanged.

3.8 Native vs generic comparison

Item	`ClaudeBashToolMiddleware`	Generic `ShellToolMiddleware`
Tool type	Anthropic native `bash_20250124`	Ordinary `@tool` function
Supported models	Claude only	All models
Tool-schema tokens	Server-side → near zero	Sent every turn
Session state	cwd, env preserved and accumulated	Depends on implementation
Execution-policy API	Shared (`HostExecutionPolicy` , etc.)	Shared

Selection criteria: If you are Claude-only, the native version is cheaper and cleaner. For multi-provider pipelines, unify on the generic `ShellToolMiddleware`.

Key Takeaways

- Claude native bash brings tool-schema tokens close to zero
- Pick execution policy in three tiers: Host (no isolation) → Docker (recommended) → CodexSandbox (Anthropic-managed)
- `redaction_rules` masks sensitive values in tool responses before they enter agent context
- In multi-provider pipelines, unifying on the generic `ShellToolMiddleware` is the safer default

04 Claude Text Editor

State vs Filesystem variants

The Claude native `text_editor_20250728` tool supports six operations: `view` / `create` / `str_replace` / `insert` / `delete` / `rename`. LangChain ships middleware that wraps this tool in two variants — the State variant writes into virtual files inside LangGraph state; the Filesystem variant writes to a real directory. Choose based on whether the artifact's lifetime ends with the thread or must persist on disk.

Learning Objectives

LEARNING OBJECTIVES

- → Use the six operations of the Claude native text editor
- → Distinguish the State and Filesystem variants by lifetime and sharing scope
- → Restrict paths via `allowed_path_prefixes` / `allowed_prefixes`
- → Bound the disk variant with `root_path` and `max_file_size_mb`

4.1 When to use it

- Model edits that are inherently multi-step (large changes that are hard to output as a single diff)
- The result needs to live only inside the graph state: use the State variant
- The result must remain in a real repo / directory: use the Filesystem variant
- When you would rather reuse Claude's learned tool schema than build a generic `@tool` for `read_file` / `write_file`

4.2 Environment setup

Required packages: `langchain`, `langchain-anthropic`. `ANTHROPIC_API_KEY` in `.env`.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
```

```
    StateClaudeTextEditorMiddleware,
    FilesystemClaudeTextEditorMiddleware,
)

load_dotenv()
```

4.3 State variant — virtual files in graph state

`StateClaudeTextEditorMiddleware` stores file contents under the `text_editor_files` key in LangGraph state. It does not touch disk, so it is a fit for temporary work that disappears when the thread ends.

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(
            allowed_path_prefixes=["/src"],  # virtual-path restriction
        ),
    ],
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "Write hello world to /src/hello.py"}]},
)
print(result.get("text_editor_files"))
```

- `allowed_path_prefixes`: allow-listed virtual-path prefixes. Unrestricted if omitted

> ➡️ TIP
>
> If the model writes outside `allowed_path_prefixes`, the tool returns an error; the model reads that error and retries with a permitted path. Path restrictions are the compromise between agent autonomy and safety.

4.4 Filesystem variant — real disk

`FilesystemClaudeTextEditorMiddleware` uses a real directory as its root and reads/writes from there.

Parameter	Default	Description
`root_path`	(required)	Actual root directory for file operations
`allowed_prefixes`	`["/"]`	Allowed virtual-path prefixes (relative to `root_path`)
`max_file_size_mb`	`10`	Maximum file size allowed for read/write

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
```

```
        FilesystemClaudeTextEditorMiddleware(
            root_path="/tmp/editor-demo",
            allowed_prefixes=["/drafts", "/reports"],
            max_file_size_mb=5,
        ),
    ],
)
```

4.5 State vs Filesystem selection criteria

Axis	State variant	Filesystem variant
Storage	LangGraph state dict	Actual directory
Lifetime	Dies with the thread	Permanent (remains on disk)
Checkpointer compatibility	Included in state → auto-saved	Only the path is stored; files managed separately
Concurrency	Fully isolated per thread	Possible conflicts when sharing `root_path`
Typical uses	Temporary drafts, one-off analyses	Real codebase edits, report artifacts

Rule: If the artifact must survive after the thread, go Filesystem; otherwise, go State.

4.6 Relationship with generic file tools

Deep Agents' `FilesystemMiddleware` or a custom `@tool` implementing `read_file` / `write_file` can reach the same goal. The difference is that Claude already saw this tool during training — schema tokens drop and error rates are lower. Use this middleware first in Claude-only pipelines, and drop down to generic file tools for multi-provider setups.

Key Takeaways

- Claude native text editor brings tool-schema tokens close to zero while supporting six operations
- Choose State (dies with thread) vs Filesystem (persistent on disk) based on artifact lifetime
- Enforce access boundaries with `allowed_path_prefixes` / `allowed_prefixes`
- The Filesystem variant uses `max_file_size_mb` to keep memory safe

05 Claude Memory

The `/memories/*` path contract

The Claude native `memory_20250818` tool implements long-term memory the model writes and reads itself. The injected system prompt instructs the model each turn to "check `/memories` first", so preferences and facts accumulate naturally across multiple sessions with the same user. Understanding the persistence difference between the State and Filesystem variants is the core of this chapter.

Learning Objectives

> **LEARNING OBJECTIVES**
>
> → Understand the Claude memory tool's `/memories/` path contract
>
> → Distinguish persistence scope between State and Filesystem variants
>
> → Know how the injected `system_prompt` instructs the model to use memory
>
> → Clarify the division of labor with Deep Agents' `StoreBackend`

5.1 When to use it

- A chatbot that must remember preferences / facts across multiple sessions with the same user
- An agent working on a long task that needs to write and later read its own intermediate notes
- Unlike RAG, you want the model itself to manage memory contents (write, delete, edit)

5.2 Environment setup

Required packages: `langchain`, `langchain-anthropic`. `ANTHROPIC_API_KEY` in `.env`.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeMemoryMiddleware,
```

```
    FilesystemClaudeMemoryMiddleware,
)
from langgraph.checkpoint.memory import MemorySaver

load_dotenv()
```

5.3 State variant — persisted within the thread

`StateClaudeMemoryMiddleware` stores memo content in LangGraph state (`memory_files`). With a checkpointer such as `MemorySaver` / `PostgresCheckpointer` , content is restored automatically when resuming the same `thread_id` .

Parameter	Default	Description
`allowed_path_prefixes`	`["/memories"]`	Allowed memo paths. Usually leave as is
`system_prompt`	Anthropic default	Instructs the model to “check /memories first” at the start of every turn

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    checkpointer=MemorySaver(),
    middleware=[StateClaudeMemoryMiddleware()],
)

cfg = {"configurable": {"thread_id": "user-42"}}

agent.invoke(
    {"messages": [{"role": "user", "content": "My name is Jihun."}]},
    config=cfg,
)

# Re-invoke with the same thread_id → memo reused
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Do you remember my name?"}]},
    config=cfg,
)
print(result["messages"][-1].content)
```

On the second call, the model does not ask for the name — it reads `/memories` and answers with “Jihun”.

5.4 Filesystem variant — across process boundaries

`FilesystemClaudeMemoryMiddleware` keeps memos on an actual disk directory. Even if the process exits or the checkpoint store changes, the files stay on disk and remain readable.

Parameter	Default	Description
`root_path`	(required)	Actual directory to store memories in
`allowed_prefixes`	`["/memories"]`	Allowed virtual paths for memo writes

`max_file_size_mb`	`10`	Maximum per-memo file size
`system_prompt`	Built-in	The prompt directing memory usage

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        FilesystemClaudeMemoryMiddleware(
            root_path="/var/data/claude-memory",
            max_file_size_mb=2,
        ),
    ],
)
```

5.5 Scaling to a durable checkpointer

`MemorySaver` is an in-process dict — it evaporates on restart. In production, use the context-manager pattern of `langgraph-checkpoint-postgres` or `-sqlite`: acquire a pool with `from_conn_string()`, then call `setup()` once to create the schema.

```
from langgraph.checkpoint.postgres import PostgresSaver

DB = "postgresql://user:pass@localhost/agent"

with PostgresSaver.from_conn_string(DB) as checkpointer:
    checkpointer.setup()  # one-time — creates tables / indexes

    agent = create_agent(
        model="anthropic:claude-sonnet-4-6",
        checkpointer=checkpointer,
        middleware=[StateClaudeMemoryMiddleware()],
    )
```

> ➡️ **TIP**
>
> For async graphs, wrap `AsyncPostgresSaver.from_conn_string(DB)` with `async with`. `setup()` also needs `await`. SQLite follows the same context-manager API (`SqliteSaver` / `AsyncSqliteSaver`).

5.6 Multi-tenant isolation via `runtime.context.user_id`

When many users share the same disk directory (or the same Store), isolate with a namespace. From LangGraph 1.2, the standard pattern reads `runtime.context.user_id` directly and uses it as the Store namespace key.

```
from langgraph.store.postgres import PostgresStore
from langgraph.runtime import get_runtime

with PostgresStore.from_conn_string(DB) as store:
    store.setup()
```

```
def upsert_profile(profile: dict) -> str:
    rt = get_runtime()
    ns = ("profiles", rt.context.user_id)  # per-user namespace
    store.put(ns, "self", profile)
    return "ok"
```

> ➡️ TIP
>
> The Claude Memory Middleware's `/memories/*` path contract and the LangGraph Store's `(namespace, key)` contract live at different layers. The former lets Claude manage memos autonomously; the latter requires the code to assign keys explicitly — so multi-tenant isolation is a more natural fit for the Store.

5.7 Relationship with Deep Agents' `StoreBackend`

Deep Agents ships a separate `StoreBackend` for persistent memory. Current code passes `StoreBackend(namespace=...)` with an explicit namespace; the two memory surfaces have similar roles but different scope.

Axis	Claude Memory Middleware	Deep Agents `StoreBackend`
Provider	Anthropic native tool	LangGraph Store API wrapper
Supported models	Claude only	All models
Storage shape	`/memories/*` files	`(namespace, key) → dict`
Retrieval	File list + content read	File reads/writes under an explicit namespace
Typical uses	Claude freely managing memos	Structured profiles / facts

Combine tip: You can use both. Have Claude keep short-term work notes in `/memories`, and put long-term profiles in Deep Agents' `StoreBackend` to share with agents from other providers — a pragmatic pattern.

Key Takeaways

- The Claude native memory tool enforces "check first, update when needed" via the `/memories/*` path contract and an auto-injected system prompt
- The State variant restores on thread resume via a checkpointer; the Filesystem variant persists on disk permanently
- `max_file_size_mb` prevents the model from dumping everything into a single memo
- Deep Agents' `StoreBackend` complements when explicit namespaces and cross-provider sharing matter

06 Anthropic File Search

glob + grep over virtual files

`StateFileSearchMiddleware` offers a Claude native tool that runs glob + grep over virtual files sitting inside graph state (for example `text_editor_files`, `memory_files`). If the text editor / memory middleware "create and edit files", this middleware "finds and reads" files on top of them.

Learning Objectives

LEARNING OBJECTIVES

- → Select the target store via `StateFileSearchMiddleware(state_key=...)`
- → Complete the "write → find → read" loop by pairing it with `StateClaudeTextEditorMiddleware`
- → Switch the search target to memory files via `state_key="memory_files"`
- → Know the duplicate-middleware constraint and the subclassing workaround

6.1 When to use it

- The agent has accumulated tens to hundreds of virtual files in state and needs to explore them
- You know only the name and want to find files by pattern (`**/*.md`) or filter by a keyword
- You want the model to `grep` its own accumulated past notes in memory to reference them

6.2 Environment setup

Required packages: `langchain`, `langchain-anthropic`. `ANTHROPIC_API_KEY` in `.env`.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeTextEditorMiddleware,
    StateFileSearchMiddleware,
)

load_dotenv()
```

6.3 Text editor + file search combination

Search by itself is meaningless — the files must exist in state first. The most common pairing is to create files with the text editor and find them with file search.

Parameter	Default	Description
`state_key`	`"text_editor_files"`	State key containing the dict of files to search. Switch to `"memory_files"` to search memory

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
    ],
)
```

6.4 Step 1 — seed state with multiple files

Ask the model to create a few notes under `/docs/`. These become the targets for the later search query.

```
cfg = {"configurable": {"thread_id": "search-demo"}}
agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": (
                    "Create three short drafts: "
                    "/docs/architecture.md, /docs/api.md, /docs/release-notes.md."
                ),
            }
        ]
    },
    config=cfg,
)
```

6.5 Step 2 — the same agent searches with glob/grep

Calling again on the same thread carries the previous files in state. The model uses `glob` and `grep` tools provided by `StateFileSearchMiddleware` to find files and read their content.

```
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Among md files under /docs/, tell me which ones mention 'API'",
            }
        ]
```

```
    },
    config=cfg,
)
print(result["messages"][-1].content)
```

6.6 Targeting memory files too

Switch to `state_key="memory_files"` and you can search the memos accumulated by `StateClaudeMemoryMiddleware`.

> ! WARNING
>
> **Duplicate-middleware constraint** — LangChain 1.2 `create_agent` rejects duplicate instances of the same middleware class (`AssertionError: Please remove duplicate middleware instances.`). To search both text-editor files and memory files at once, register only one at a time — or subclass `StateFileSearchMiddleware` into a separate class.

```
class MemoryFileSearchMiddleware(StateFileSearchMiddleware):
    """Search middleware dedicated to `memory_files`."""

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(),
        StateClaudeMemoryMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
        MemoryFileSearchMiddleware(state_key="memory_files"),
    ],
)
```

6.7 Position in the five-step retrieval building blocks

LangChain RAG is standardized into five stages — Document Loaders → Text Splitters → Embeddings → Vector Stores → Retrievers (categories 04–05 in 08_integration). `StateFileSearchMiddleware` is a lite path that bypasses these five stages, scanning files already in state with literal grep and zero embedding cost.

Axis	State File Search	Five-step RAG pipeline
Preprocessing	None — files as they arrive in state	Loader + Splitter + Embed + Index
Retrieval	literal glob + grep	vector similarity + filter
Latency	Milliseconds	Embedding call + ANN
Suitable scale	Tens to hundreds of files	Tens of thousands to millions of chunks

```python
# Scaling out to a five-step RAG pipeline
from langchain.embeddings import init_embeddings
from langchain_chroma import Chroma

embeddings = init_embeddings("openai:text-embedding-3-small")
vectorstore = Chroma(collection_name="docs", embedding_function=embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
```

> ➡️ TIP
>
> `init_embeddings("provider:model")` mirrors the `init_chat_model` pattern — swap the embedding backend in a single line using a provider-prefixed string. Moving from State File Search to a vector-store RAG keeps code changes minimal.

6.8 When to pick this over a vector store

- Exact filename/patterns matter, and literal grep is enough — no semantic search needed
- Document count is tens to hundreds and you would rather not pay to re-embed each turn
- The files are just created in the current session and no vector index exists yet

For large corpora or semantic search, move over to Part VIII's Chat Models / Vector Stores area (future expansion).

Key Takeaways

- `StateFileSearchMiddleware` provides glob + grep over virtual files in state
- text editor + file search is the standard "write → find → read" loop
- Switch `state_key` to search memory files; work around the duplicate-middleware limit via subclassing
- When literal search suffices, this middleware is much cheaper than a vector store

07 Bedrock Prompt Caching

Claude / Nova caching through AWS

When calling Claude / Nova through AWS Bedrock, `BedrockPromptCachingMiddleware` plants cache checkpoints automatically at the system prompt · tool definitions · last message positions to reduce input-token cost. Parameter names and semantics are almost identical to `AnthropicPromptCachingMiddleware` (ch2), but the target package and per-model constraints differ.

Learning Objectives

LEARNING OBJECTIVES

- → Understand the four parameters of `BedrockPromptCachingMiddleware`
- → Verify hits via `cache_creation` / `cache_read` in `usage_metadata.input_token_details`
- → Know how `type` is handled differently between `ChatBedrock` and `ChatBedrockConverse`
- → Distinguish Nova constraints (5m TTL only, no tool-definition cache)

7.1 When to use it

- Enterprise setups that must keep LLM calls inside AWS account / VPC boundaries (via Bedrock)
- Using Claude through Bedrock billing rather than the direct Anthropic API
- Repeated use of long system prompts with Amazon Nova

7.2 Environment setup

Required packages: `langchain`, `langchain-aws`. AWS credentials (`AWS_ACCESS_KEY_ID` etc.) and a region must be configured via `.env` or environment. In the Bedrock console you also need to grant model access before the call works.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_aws.middleware import BedrockPromptCachingMiddleware
```

```
load_dotenv()
```

7.3 ChatBedrockConverse + Claude (recommended)

The Converse API is Bedrock's unified interface and can call many models through the same endpoint. Adding `BedrockPromptCachingMiddleware` automatically inserts cache checkpoints at the system / tool / last-message positions.

> **! WARNING**
>
> For a checkpoint to actually hit, the cached block must be roughly 1,024 tokens or more. On short system prompts, `cache_creation` may not register.

```
long_prompt = (
    "You are an enterprise policy assistant. Always cite section numbers. "
    * 200  # ~2,000 tokens
)

agent = create_agent(
    model="bedrock_converse:anthropic.claude-sonnet-4-20250514-v2:0",
    system_prompt=long_prompt,
    tools=[],
    middleware=[BedrockPromptCachingMiddleware(ttl="1h")],
)
```

7.4 Verifying cache hits

Call twice in a row and read `usage_metadata.input_token_details` to confirm cache creation / hits.

```
for i in range(2):
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"Request {i}"}]},
    )
    last = result["messages"][-1]
    print(i, last.usage_metadata.get("input_token_details"))
```

`cache_creation` is large on the first call; `cache_read` fills in on subsequent calls.

7.5 Full parameters

Parameter	Default	Description
`type`	`"ephemeral"`	Only takes effect on `ChatBedrock`; `ChatBedrockConverse` ignores it and uses `"default"`

`ttl`	`"5m"`	`"5m"` or `"1h"`. Nova-class models support `"5m"` only
`min_messages_to_cache`	`0`	Attach a cache checkpoint only when the message count is at least this value
`unsupported_model_behavior`	`"warn"`	For cache-unsupported models: `"ignore"` / `"warn"` / `"raise"`

7.6 ChatBedrock (Invoke model) example

`ChatBedrock` is the legacy invoke-model wrapper. `BedrockPromptCachingMiddleware` supports it too, and the `type="ephemeral"` parameter does take effect here. On the ChatBedrock side, for Anthropic models, both tool-definition caching and the extended TTL (1h) are supported.

```
from langchain_aws import ChatBedrock

model = ChatBedrock(model_id="anthropic.claude-sonnet-4-20250514-v2:0")

agent = create_agent(
    model=model,
    system_prompt=long_prompt,
    tools=[],
    middleware=[
        BedrockPromptCachingMiddleware(type="ephemeral", ttl="1h"),
    ],
)
```

7.7 Reasoning · profiling

Claude 4.6/4.7 models support extended thinking (reasoning) on Bedrock too. Caching still applies to reasoning-enabled calls, but the reasoning block itself is always a cache miss and recomputed each time — so token savings are smaller than the non-reasoning case.

```
from langchain_aws import ChatBedrockConverse

model = ChatBedrockConverse(
    model_id="anthropic.claude-sonnet-4-20250514-v2:0",
    additional_model_request_fields={
        "reasoning_config": {"type": "enabled", "budget_tokens": 4096},
    },
)
print(model.profile.reasoning_output)  # True → response includes a reasoning block
```

> ➡️ **TIP**
>
> `ChatBedrockConverse(...).profile` (LangChain 1.1+) exposes capabilities such as `tool_calling`, `reasoning_output`, and `max_input_tokens` — using it as a capability check in multi-model routing logic keeps the code clean.

7.8 Amazon Nova constraints

Item	ChatBedrockConverse + Anthropic	ChatBedrockConverse + Nova
System-prompt cache	O	O
Tool-definition cache	O	X
Message cache	O	O (excluding tool-result messages)
TTL `"1h"`	O	X (5m only)

When targeting Nova, pin `ttl="5m"` and set `unsupported_model_behavior="warn"` so an accidental `"1h"` is not silently ignored.

Key Takeaways

- When using Claude / Nova inside AWS boundaries, the Bedrock middleware reduces input-token cost
- Be aware of the `type` parameter difference between `ChatBedrock` and `ChatBedrockConverse`
- Nova supports 5m TTL only with no tool-definition cache — pin `ttl="5m"`
- Cache blocks must be roughly 1,024 tokens or more to actually hit

08 OpenAI Moderation

Pre- / post-block policy-violating content

`OpenAIModerationMiddleware` uses the OpenAI Moderation API to scan user input · model output · tool results automatically and, when a policy violation is detected, blocks, replaces, or raises an exception on the conversation. Use it to preemptively filter categories like hate, violence, or self-harm for a public chatbot, and to stop second-order contamination from external tool results.

Learning Objectives

LEARNING OBJECTIVES

- → Understand the meaning and cost/latency tradeoff of the three scan points `check_input` / `check_output` / `check_tool_results`
- → Distinguish the three `exit_behavior` modes: `"end"` / `"error"` / `"replace"`
- → Polish user-facing responses via a custom `violation_message` template
- → Inject a pre-configured OpenAI client via `client` / `async_client`

8.1 When to use it

- Public chatbots that need to preempt Moderation categories such as hate / violence / self-harm
- Preventing harmful content in tool outputs (web search, email bodies) from reaching the model
- Leaving violation metadata in the response flow for audit / reporting

8.2 Environment setup

Required packages: `langchain`, `langchain-openai`. `OPENAI_API_KEY` in `.env`.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_openai.middleware import OpenAIModerationMiddleware
```

```
load_dotenv()
```

8.3 Basic usage — scan both input and output, end on violation

Defaults are `check_input=True`, `check_output=True`, `exit_behavior="end"`. If user input trips the Moderation API, the model call is skipped entirely and the conversation ends with a violation message.

Parameter	Default	Description
`model`	`"omni-moderation-latest"`	Moderation model to use
`check_input`	`True`	Scan user messages before passing to the model
`check_output`	`True`	Scan model-generated responses before returning to the user
`check_tool_results`	`False`	Scan tool-execution results before they enter the model
`exit_behavior`	`"end"`	`"end"` / `"error"` / `"replace"`
`violation_message`	default template	The text shown to the user on violation
`client` / `async_client`	`None`	Inject a pre-configured OpenAI client (optional)

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[OpenAIModerationMiddleware()],
)
```

8.4 `exit_behavior="end"` — end immediately on violating input

When a harmful category is detected (for example self-harm encouragement), the flow ends with the violation message — no model call. The final message on the response contains the default violation text.

8.5 `exit_behavior="error"` — fail loudly

Use this in testing / batch environments where you would rather not let the violation slip by silently. It raises `OpenAIModerationError` so the upstream pipeline can catch it and write to the audit log.

```
from langchain_openai.middleware import OpenAIModerationError

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[OpenAIModerationMiddleware(exit_behavior="error")],
)

try:
    agent.invoke({"messages": [{"role": "user", "content": "..."}]})
except OpenAIModerationError as e:
    # Record in the audit log
    print("Moderation violation:", e.categories)
```

8.6 `exit_behavior="replace"` — swap the message and keep going

When output scanning catches a violation, only that response is swapped out for the violation message and the graph keeps running. Useful in multi-turn conversations where you want to tidy up a specific response without ending the whole conversation.

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[
        OpenAIModerationMiddleware(
            exit_behavior="replace",
            violation_message=(
                "This response cannot be shown under our safety policy "
                "(categories: {categories}). Please try a different question."
            ),
        ),
    ],
)
```

`violation_message` is a template string that supports the `{categories}`, `{category_scores}`, and `{original_content}` variables.

8.7 Scanning tool results (`check_tool_results=True`)

Prevents harmful content authored by third parties in web search, crawling, or DB query tool results from flowing unchanged into the model. Cost scales with the number of tool calls, so only turn this on in pipelines that touch untrusted external sources.

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[web_search_tool, fetch_url_tool],
    middleware=[
```

```
        OpenAIModerationMiddleware(
            check_input=True,
            check_output=True,
            check_tool_results=True,
        ),
    ],
)
```

8.8 Observing violation flows with LangSmith

Moderation blocks are rare but must always show up in the audit log. With LangSmith auto-tracing on (`LANGSMITH_TRACING=true` + `LANGSMITH_API_KEY`), the nodes where `OpenAIModerationMiddleware` triggers land in the trace tree as usual.

```
import os
from langsmith.run_helpers import tracing_context

os.environ["LANGSMITH_TRACING"] = "true"
# os.environ["LANGSMITH_API_KEY"] = "ls__..."
# os.environ["LANGSMITH_PROJECT"] = "moderation-audit"

with tracing_context(
    project_name="moderation-audit",
    tags=["moderation", "input-only"],
    metadata={"region": "kr"},
):
    agent.invoke({"messages": [{"role": "user", "content": "..."}]})
```

> ➡️ TIP
>
> `tracing_context` routes only a specific call range to a different project / tag set — a natural pattern is to keep regular traces in `production` while sending only Moderation-blocked cases to `moderation-audit` . The `LANGCHAIN_*` environment variables were standardized to `LANGSMITH_*` from 1.1 onward — the old names still work for a while, but new code should use `LANGSMITH_*` exclusively.

8.9 Cost / latency tradeoff

Setting	Cost	Latency	Main effect
`check_input=True`	Mod 1 + Model 1	+1 RT	Block harmful input before it reaches the model
`check_output=True`	Model 1 + Mod 1	+1 RT	Protect the end user when the model produced a bad response

`check_tool_results=True`	# of tools × Mod	+1 per tool	Block externally contaminated data

> ➡️ TIP
>
> **Practical guidance**: In production it is common to always keep `check_input` on, run `check_output` under a sampling strategy (e.g., 10%), and limit `check_tool_results` to untrusted external sources.

Key Takeaways

- Pick the three scan points on a cost/latency tradeoff: input always on, output sampled, tool_results conditional
- `exit_behavior` picks between ending the conversation (`end`), raising immediately (`error`), or swapping the message (`replace`)
- `violation_message` templating produces user-friendly guidance
- Reuse a pre-configured OpenAI client via `client` / `async_client` to save initialization cost

A Appendix A. Glossary

Core terms that appear throughout the handbook

This appendix collects the key terms that appear repeatedly throughout the handbook so you can review them in one place. You do not need to memorize everything at once; revisit this section whenever you encounter an unfamiliar term.

> · NOTE
>
> The goal of this glossary is not to replace the official API documentation, but to let you quickly confirm the concepts that appear often while reading the handbook. For implementation details, refer to the relevant chapter and reference documents.

Frameworks and Products

LangChain A high-level framework for quickly building agents and LLM applications by combining models, tools, prompts, and middleware.

LangGraph A runtime and framework for orchestrating complex workflows and long-running agents with state, nodes, edges, and checkpointers.

Deep Agents An all-in-one agent SDK built on top of LangGraph that adds planning, filesystem tooling, subagents, memory, and sandbox capabilities.

LangSmith An observability and quality platform for tracing, evaluation, dataset management, and debugging.

Langfuse An open-source-style observability tool for collecting traces and runtime telemetry.

MCP Short for Model Context Protocol, a protocol that connects external tools and resources to models and agents in a standard way.

ACP Short for Agent Client Protocol, a communication protocol for connecting agents to clients such as editors and IDEs.

Agents and Execution Model

Agent An execution unit in which an LLM uses tools, observes results, and keeps acting until the task is complete.

ReAct Short for Reasoning + Acting, a common agent pattern in which the model alternates between reasoning and tool use.

Workflow An execution flow whose steps and order are relatively well-defined. It is usually more deterministic than a free-form agent loop.

Orchestrator A higher-level controller that coordinates several steps or several workers and manages the overall flow.

Worker An execution unit that performs a specific task delegated by an orchestrator.

Subagent A helper agent called by a main agent to handle a more focused sub-task, often with its own context.

Handoff A pattern in which control is transferred to another role or agent depending on the current step or state.

Router A pattern that classifies the input or current state and sends it to the most appropriate path or specialist agent.

Human-in-the-Loop A safety mechanism that inserts human approval, edits, or rejection at sensitive execution steps or important branch points.

Interrupt A mechanism that pauses graph or agent execution in the middle and waits for external input.

Resume The action of continuing a paused execution from its prior state. This usually requires the same `thread_id` and a checkpointer.

Durable Execution An execution style that stores progress so the system can restart from the last successful point after an interruption or failure.

Pregel A message-passing computation model that influences LangGraph's internal runtime, where nodes run in supersteps.

Superstep A parallel computation round in the Pregel model during which several nodes run together.

State and Memory

State The collection of current task information that an agent or graph maintains and updates while it runs.

AgentState The default state schema used in LangChain agent execution. It includes reserved fields such as `messages`.

MessagesState A convenient LangGraph state type centered on a list of messages.

Checkpointer A component that stores and restores execution state. It is essential for multi-turn conversations, interrupt/resume flows, and durable execution.

Thread ID A unique identifier for a conversation or execution flow. You must reuse the same `thread_id` to continue from a previous state.

Runtime Context Context such as metadata, permissions, or user information that is injected only at execution time.

`context_schema` A schema that defines the structure of static context that does not change during execution.

`state_schema` A schema that defines the structure of dynamic state that can keep changing during execution.

Short-term memory Memory that is kept only within a single thread or conversation, usually the recent message history and work state.

Long-term memory Memory that persists after a conversation ends and can be reused in later threads, such as preferences or learned facts.

Store A storage layer that saves and retrieves data outside a single thread.

InMemoryStore A simple in-memory store implementation. It is convenient for development and testing, but data disappears on restart.

Semantic memory Long-term memory that stores meaning-based information such as facts, preferences, and concepts.

Episodic memory Memory whose time order matters, such as specific events or interaction history.

Procedural memory Memory that captures rules, procedures, or behavior patterns the agent should follow.

Tools and Interfaces

Tool A function or external action interface that an agent can call, such as search, calculation, file I/O, or an API request.

ToolRuntime A runtime object that gives tools access to the current state, context, store, and other execution-time resources.

`create_agent()` The main LangChain API for quickly creating a default agent.

`create_deep_agent()` The Deep Agents API for creating a harness-style agent with planning, files, and subagents.

`StateGraph` The LangGraph builder class used to define state-based graph workflows.

Graph API The LangGraph programming style in which you explicitly declare nodes and edges and assemble the graph structure.

Functional API The LangGraph programming style in which you describe workflows with Python functions, `@entrypoint`, and `@task`.

`@entrypoint` A decorator in the Functional API that defines the starting function of a workflow.

`@task` A Functional API decorator that wraps a unit of work that should have durability guarantees.

`Send` A LangGraph API used to fan out work dynamically to multiple workers.

`Command` A LangGraph control object that can express state updates, resume signals, and parent-graph transitions.

Structured output An output style that forces the model's answer into an explicit schema such as a Pydantic model or another structured format.

Backends and Execution Environments

Backend The storage and execution layer in Deep Agents that abstracts file access and execution environments.

StateBackend An ephemeral file backend whose contents are kept only within a conversation thread.

FilesystemBackend A backend that accesses the local disk. In practice it is often used with `virtual_mode=True` to restrict paths.

StoreBackend A backend that provides persistent cross-thread storage through a LangGraph store.

CompositeBackend A mixed backend that routes requests to different backends depending on the path.

LocalShellBackend A powerful but risky backend that adds shell command execution on top of local file access.

Sandbox An execution environment that isolates code execution and file operations from the host system.

Modal A sandbox and serverless execution provider that is especially strong for GPU and AI/ML workloads.

Daytona A sandbox provider well suited to fast devbox provisioning and development-environment workflows.

Runloop A sandbox provider designed for disposable devboxes and isolated execution.

Retrieval, Streaming, and Quality

RAG Short for Retrieval-Augmented Generation, a pattern in which external knowledge is retrieved and injected into the model's response process.

Retriever A component that searches for and returns documents or chunks relevant to a query.

Embedding A representation that turns text into vectors in semantic space so similarity search becomes possible.

Vector store A database or storage layer that saves embeddings and performs similarity search.

Chunking The process of splitting a long document into smaller units suited for retrieval and context injection.

SQL Agent An agent that translates natural-language questions into SQL queries and interprets the results.

Streaming A delivery style in which a model response or execution state is sent incrementally before the run is fully complete.

StreamEvent An event unit emitted during streaming, such as token output, tool start/end, or a state change.

TTFT Short for Time to First Token, the time it takes for the model to emit its first token.

TTFA Short for Time to First Audio, the time it takes for a voice agent to emit its first audio output.

Tracing An observability technique that records model calls, tool executions, state transitions, and errors so you can inspect the execution flow.

Evaluation The process of measuring the quality of an agent's output or execution trajectory.

LLM-as-Judge An evaluation method in which another LLM acts as a scorer for response quality or execution outcomes.

Trajectory The full path of tool calls, intermediate reasoning, and state changes that leads to the agent's final answer.

Guardrail A control mechanism that checks safety, policy compliance, or quality at the input, tool, or output stage.

PII Short for Personally Identifiable Information, sensitive data that can identify a person, such as an email address, phone number, or national ID number.